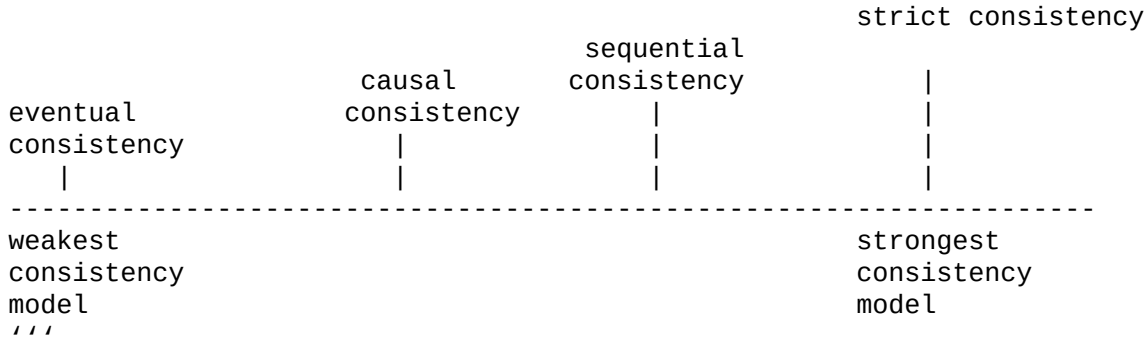


Abstractions/Spectrum of Consistency Models/README.md

'''



Eventual consistency

Eventual consistency is the weakest consistency model. The applications that donât have strict ordering requirements and donât require reads to return the latest write choose this model. Eventual consistency ensures that all the replicas will eventually return the same value to the read request, but the returned value isnât meant to be the latest value. However, the value will finally reach its latest state.

Example

The domain name system is a highly available system that enables name lookups to a hundred million devices across the Internet. It uses an eventual consistency model and doesnât necessarily reflect the latest values.

'''

Note: Cassandra is a highly available NoSQL database that provides eventual consistency.

'''

Causal consistency

This model doesnât ensure ordering for the operations that are not causally related. These operations can be seen in different possible orders.

Causal consistency is weaker overall, but stronger than the eventual consistency model. Itâs used to prevent non-intuitive behaviors.

Example

The causal consistency model is used in a commenting system. For example, for the replies to a comment on a Facebook post, we want to display comments after the comment it replies to. This is because there is a cause-and-effect relationship between a comment and its replies.

Sequential consistency

Sequential consistency is stronger than the causal consistency model. It preserves the ordering specified by each clientâs program. However, sequential consistency doesnât ensure that the writes are visible instantaneously or in the same order as they occurred according to some global clock.

Example

In social networking applications, we usually donât care about the order in which some of our friendsâ posts appear. However, we still anticipate a single friendâs posts to appear in the correct order in which they were created). Similarly, we expect our friendsâ comments in a p

to display in the order that they were submitted. The sequential consistency model captures all of these qualities.

Strict consistency

A strict consistency or linearizability is the strongest consistency model. This model ensures that a read request from any replicas will get the latest write value. Once the client receives the acknowledgment that the write operation has been performed, other clients can read that value.

high consistency=>low availability requests

Example

Updating an account's password requires strict consistency. For example, if we suspect suspicious activity on our bank account, we immediately change our password so that no unauthorized users can access our account. If it were possible to access our account using an old password due to a lack of strict consistency, then changing passwords would be a useless security strategy.

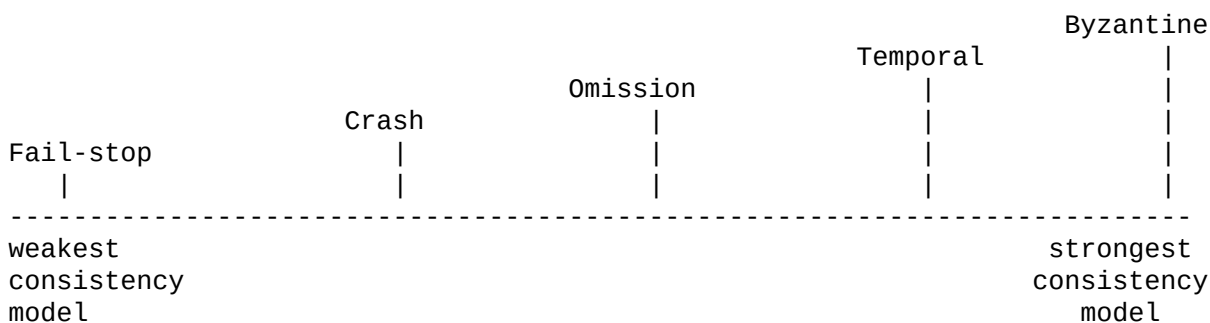
'''

Note: Amazon Aurora provides strong consistency.

'''

Abstractions/The Spectrum of Failure Models/README.md

'''



'''

Fail-stop

In this type of failure, a node in the distributed system halts permanently. However, the other nodes can still detect that node by communicating with it.

From the perspective of someone who builds distributed systems, fail-stop failures are the simplest and the most convenient.

Crash

In this type of failure, a node in the distributed system halts silently, and the other nodes can't detect that the node has stopped working.

Omission

In omission failures, the node fails to send or receive messages. There are two types of omission failures. If the node fails to respond to the incoming request, it's said to be a send omission failure. If the node fails to receive the request and thus can't acknowledge it, it's said to be a receive omission failure.

Temporal

In temporal failures, the node generates correct results, but is too late to be useful. This failure could be due to bad algorithms, a bad design strategy, or a loss of synchronization between the processor clocks.

Byzantine

In Byzantine failures, the node exhibits random behavior like transmitting arbitrary messages at arbitrary times, producing wrong results, or stopping midway. This mostly happens due to an attack by a malicious entity or a software bug. A byzantine failure is the most challenging type of failure to deal with.

Back-of-the-envelope Calculations/Example of Resource Estimation/README.md

Examples of Resource Estimation

Introduction

Now that we've set the foundation for resource estimation, let's make use of our background knowledge to estimate resources like servers, storage, and bandwidth. Below, we consider a scenario and a service, make assumptions, and based on those assumptions, we make estimations. Let's jump right in!

Number of servers required

Let's make the following assumptions about a Twitter-like service.

There are 500 million (M) daily active users (DAU).

A single user makes 20 requests per day on average.

Recall that a single server can handle 8,000 RPS.

'''

Total requests per second $500M \times 20 / 86400 = 115K$ requests per second which is the RPS for our system.

For the number of servers, $(\text{Number of Request/sec}) / \text{RPS of server} = 115k / 8k = 15$ servers to handle all requests.

'''

[Estimation of Number of Servers](./estimating_the_number_of_servers.jpg)

Indeed, the number above doesn't seem right. If we only need 15 commodity servers to serve 500M daily users, then why do big services use millions of servers in a data center? The primary reason for this is that the RPS is not enough to estimate the number of servers required to provide a service. Also, we made some underlying assumptions in the calculations above. One of the assumptions was that a request is handled by one server only. In reality, requests go through to web servers that may interact with application servers that may also request data from storage servers. Each server may take a different amount of time to handle each request. Furthermore, each request may be handled differently depending upon the state of the data center, the application, and the request itself. Remember that we have a variety of servers for providing various services within a data center.

Therefore, we approximate the number of servers by depicting how many clients a server handles on a given day:

In this case, it's equal to $500M / 8000 = 62,500$

'''

Note: This may not be an accurate estimation, but it's a realistic one. Therefore, we use this approach in estimating the number of servers in our design problems. Informally, the equation given above assumes that one server can handle 8,000 users. We use this reference in the rest of the course as well.

'''

Storage requirements

In this section, we attempt to understand how storage estimation is done by using Twitter as a

example. We estimate the amount of storage space required by Twitter for new tweets in a year. Let's make the following assumptions to begin with:

'''

We have a total of 250 M daily active users.

Each user posts three tweets in a day.

Ten percent of the tweets contain images, whereas five percent of the tweets contain a video. Any tweet containing a video will not contain an image and vice versa.

Assume that an image is 200 KB and a video is 3 MB in size on average.

The tweet text and its metadata require a total of 250 Bytes of storage in the database.

'''

Then, the following storage space will be required:

[Estimation of Storage Requirements](./estimating_storage_req.jpg)

'''

Total tweets: $250M \times 3 = 750M \times 10^6$ tweets per day

storage required for tweets in one day: $750 \times 10^6 \times 250 \text{ B} = 187.5 \text{ GB}$

Storage required for 10 percent images for one day: $(750 \times 10^6 \times 10/100) \times 200 \times 10^3 \text{B} = 15 \text{TB}$

Storage required for 5 percent video content for one day: $(750 \times 10^6 \times 10/100) \times 200 \times 10^6 \text{B} = 112.5 \text{TB}$

'''

'''

Total storage required for one day = $0.1875 \text{ TB} + 15 \text{ TB} + 112.5 \text{ TB} = \text{approximately } 128 \text{TB}$

Storage required for one year = $365 \times 128 \text{ TB} = 46.72 \text{ PB}$

'''

[Total Storage of Twittter for 1 year](./total_storage_yearly.jpg)

Bandwidth requirements

In order to estimate the bandwidth requirements for a service, we use the following steps:

Estimate the daily amount of incoming data to the service.

Estimate the daily amount of outgoing data from the service.

Estimate the bandwidth in Gbps (gigabits per second) by dividing the incoming and outgoing data by the number of seconds in a day.

Incoming traffic: Let's continue from our previous example of Twitter, which requires 128 TBs of storage each day. Therefore, the incoming traffic should support the following bandwidth per second:

'''

$128 \times 10^{12} / 86400 \times 8 = \text{approximately } 12 \text{Gbps}$

'''

'''

Note: We multiply by 8 in order to convert Bytes(B) into bits(b) because bandwidth is measured in bits per second.

'''

Outgoing traffic: Assume that a single user views 50 tweets in a day. Considering the same ratio of five percent and 10 percent for videos and images, respectively, for the 50 tweets, 2.5 tweets will contain video content whereas five tweets will contain an image. Considering

that there are 250 M active daily users, we come to the following estimations:

[Estimating Bandwidth Requirements](./estimating_bandwidth_req.jpg)

'''

250M * 50 tweets = 12.5 billion tweets are viewed per day

Tweets viewed per second: $12.5 / 86400 = 145 \times 10^3$ tweets per second

Bandwidth for tweet: $145 \times 10^3 \times 250 \times 8 \text{ bits} = \text{approximately } 0.3 \text{ Gbps}$ (One tweet is equal to 250 bytes).

Bandwidth of 10 percent images in tweets in a second: $145 \times 10^3 \times 10/100 \times 200 \times 10^3 \times 8 \text{ bits} = 23.2 \text{ Gbps}$

Bandwidth for 5 percent video tweets in a second: $145 \times 10^3 \times 5/100 \times 3 \times 10^6 \times 8 \text{ bits} = 174 \text{ Gbps}$

The total outgoing traffic required will be equal to: $0.3 + 23.2 + 174 = \text{approx } 197.5 \text{ Gbps}$.

'''

Twitter will need a total of 12 Gbps of incoming traffic and 197.5 Gbps of outgoing, assuming that the uploaded content is not compressed. Total bandwidth requirements = $12 + 197.5 = 209.5 \text{ Gbps}$

[The total bandwidth required by Twitter](./total_bandwidth.jpg)

Back-of-the-envelope Calculations/Put Back-of-the-envelope Numbers in Perspective/README.md

Put Back-of-the-envelope Numbers in Perspective

Why do we use back-of-the-envelope calculations?

A distributed system has compute nodes connected via a network. There's a wide variety of available compute nodes and they can be connected in many different ways. Back-of-the-envelope calculations help us ignore the nitty-gritty details of the system (at least at the design level) and focus on more important aspects.

Some examples of a back-of-the-envelope calculation could be:

'''

The number of concurrent TCP connections a server can support.

The number of requests per second (RPS) a web, database, or cache server can handle.

The storage requirements of a service.

'''

Choosing an unreasonable number for such calculations can lead to a flawed design. Since we need good estimations in many design problems, we'll discuss all the relevant concepts in detail in this lesson. These concepts include:

'''

The types of data center servers.

The realistic access latencies of different components.

The estimation of RPS that a server can handle.

Examples of bandwidth, servers, and storage estimation.

'''

Types of data center servers

Data centers don't have a single type of server. Enterprise solutions use commodity hardware

save cost and develop scalable solutions. Below, we discuss the types of servers that are commonly used within a data center to handle different workloads.

Web servers

For scalability, the web servers are decoupled from the application servers. Web servers are the first point of contact after load balancers. Data centers have racks full of web servers that usually handle API calls from the clients. Depending on the service that's offered, the memory and storage resources in web servers can be small to medium. However, such servers require good computational resources. For example, Facebook has used a web server with 32 GB of RAM and 500 GB of storage space. But for its high-end computational needs, it partnered with Intel to build a custom 16-core processor.

'''

Note: Many numbers quoted in this lesson are obtained from the data center design that Facebook open-sourced in 2011. Due to the slowing of Moore's law-induced performance circa 2004, the numbers are not stale.

'''

Application servers

Application servers run the core application software and business logic. The difference between web servers and application servers is somewhat fuzzy. Application servers primarily provide dynamic content, whereas web servers mostly serve static content to the client, which is mostly a web browser. They can require extensive computational and storage resources. Storage resources can be volatile and non-volatile. Facebook has used application servers with a RAM of up to 256 GB and two types of storage—traditional rotating disks and flash—with a capacity of up to 6.5 TB.

Storage servers

With the explosive growth of Internet users, the amount of data stored by giant services has multiplied. Additionally, various types of data are now being stored in different storage units. For instance, YouTube uses the following datastores:

'''

Blob storage for its encoded videos.

A temporary processing queue storage that can hold a few hundred hours of video content uploaded daily to YouTube for processing.

Specialized storage called Bigtable for storing a large number of thumbnails of videos.

Relational database management system (RDBMS) for users and videos metadata (comments, likes, user channels, and so on).

'''

Other data stores are still used for analytics—for example, Hadoop's HDFS. Storage servers mainly include structured (for example, SQL) and non-structured (NoSQL) data management systems.

Coming back to the example of Facebook, we know that they've used servers with a storage capacity of up to 120 TB. With the number of servers in use, Facebook is able to house exabyte of storage. One exabyte is 10^{18} Bytes. By convention, we measure storage and network bandwidth in base 10, and not base 2. However, the RAM of these servers is only 32 GB.

'''

Note: The servers described above are not the only types of servers in a data center. Organizations also require servers for services like configuration, monitoring, load balancing, analytics, accounting, caching, and so on.

'''

The numbers open-sourced by Facebook are outdated as of now. In the table below, we depict the capabilities of a server that can be used in the data centers of today:

'''

Typical Server Specifications	
Component	Count
Number of sockets	2
Processor	Intel Xeon X2686
Number of cores	36 cores (72 threads)
RAM	256 GB
Cache (L3)	45 MB
Storage capacity	15 TB

The numbers above are inspired by the Amazon bare-metal server, but there can be more or less powerful machines supporting much higher RAM (up to 8 TB), disk storage (up to 24 disks with up to 20 TB each, circa 2021), and cache memory (up to 120 MB).

Standard numbers to remember

A lot of effort goes into the planning and implementation of a service. But without any basic knowledge of the kind of workloads machines can handle, this planning isn't possible. Latencies play an important role in deciding the amount of workload a machine can handle. The table below depicts some of the important numbers system designers should know in order to perform resource estimation.

Important Latencies	
Component	Time (nanoseconds)
L1 cache reference	0.9
L2 cache reference	2.8
L3 cache reference	12.9
Main memory reference	100
Compress 1KB with Snzip	3,000 (3 microseconds)
Read 1 MB sequentially from memory	9,000 (9 microseconds)
Read 1 MB sequentially from SSD	200,000 (200 microseconds)
Round trip within same datacenter	500,000 (500 microseconds)
Read 1 MB sequentially from SSD with speed ~1GB/sec SSD	1,000,000 (1 milliseconds)
Disk seek	4,000,000 (4 milliseconds)
Read 1 MB sequentially from disk	2,000,000 (2 milliseconds)
Send packet SF->NYC	71,000,000 (71 milliseconds)

Apart from the latencies listed above, there are also throughput numbers measured as queries per second (QPS) that a typical single-server datastore can handle.

Important Rates

QPS handled by MySQL	1000
QPS handled by key-value store	10,000
QPS handled by cache server	100,000â 1 M

The numbers above are approximations and vary greatly depending on a number of reasons like the type of query (point and range), the specification of the machine, the design of the database, the indexing, and so on.

A lookup for a single item is called as Point query. For example, `SELECT * FROM STUDENTS WHERE NAME = "John"`; This is a point query because search is for a single value â Johnâ.

A query that retrieves information/records in between boundaries. For example, `SELECT * FROM STUDENTS WHERE age BETWEEN 15 AND 18;`.

Requests estimation

This section discusses the number of requests a typical server can handle in a second. Within a server, there are limited resources and depending on the type of client requests, different resources can become a bottleneck. Letâs understand two types of requests.

CPU-bound requests: These are the type of requests where the limiting factor is the CPU.

Memory-bound requests: These are the types of requests that are limited by the amount of memory a machine has.

Letâs approximate the RPS for each type of request. But before that, we need to assume the following:

Our server has the specifications of the typical server that we defined in the table above. Operating systems and other auxiliary processes have consumed a total of 16 GB of RAM.

Each worker consumes 300 MBs of RAM storage to complete a request.

For simplicity, we assume that the CPU obtains data from the RAM. Therefore, a caching system ensures that all the required content is available for serving, without there being a need to access the storage layer.

Each CPU-bound request takes 200 milliseconds, whereas a memory-bound request takes 50 milliseconds to complete.

Letâs do the computation for each type of request.

****CPU bound**:** A simple formula used to calculate the RPS for CPU-bound requests is:

$$\text{RPS}_{\text{cpu}} = \text{NUM}_{\text{cpu}} / \text{TASK time}$$

where

RPS_{cpu}: CPU bound RPS

NUM_{cpu}: Number of CPU threads, which are also called hardware threads

TASK time: The time each task takes to complete
'''

Then, $\text{RPS}_{\text{cpu}} = 72/200\text{ms} = 360 \text{ RPS}$

The rationale for the calculation shown above is that we can visualize one second as a box and we calculate how many mini-boxes (tasks) can fit inside the big boxâ that is, the number of tasks that can be completed in one second by a number of CPUs. So, a higher number of CPUs/threads will result in a higher RPS.

****Memory-bound requests****: For memory-bound requests, we use the following formula:
'''

$\text{RPS}_{\text{memory}} = \text{RAM size} / \text{WORKDER memory} \times 1 / \text{TASK time}$

where

RPS memory: Memory bound RPS

RAM size: Total size of RAM

WORKER memory: A worker in memory that manages a request
'''

Then, $\text{RPS}_{\text{memory}} = 240\text{GB} / 300\text{MB} \times 1 / 50 \text{ ms} = 16,000 \text{ RPS}$

Continuing our box analogy from the explanation of CPU-bound processes, here we first calculate the number of boxes there are (how many memory-bound processes a server can host) and then how many mini-boxes (tasks) we can fit in each of the bigger boxes.

A service receives both the CPU-bound and memory-bound requests. Considering the case that half the requests are CPU-bound and the other half memory-bound, we can handle a total of $360/2 + 16,000/2 = 8180$, around 8000 RPS.

The calculations above are only an approximation for developing an understanding of the basic factors involved in estimating RPS. In reality, a lot of other factors come into play. For instance, latency is required to do a disk seek in case the data is not readily available in RAM or if a request is made to the database server, which will also include the database and network latency. Additionally, the type of query also matters. Of course, faults, bugs in code, node failures, power outages, network disruptions, and more are inevitable factors.

On a typical day, various types of requests arrive, and a powerful server that only serves static content from the RAM might handle as many as 500k RPS. On the other end of the spectrum computational-intensive tasks like image processing may only allow a maximum of 50 RPS.

'''

Note: In reality, capacity estimation is a hard problem, and organizations learn how to improve it over the years. A monitoring system keeps an eye on all parts of our infrastructure to give us early warnings about overloading servers.
'''

Blob Store/Design Considerations of a Blob Store/README.md

Design Considerations of a Blob Store

Introduction

Even though we discussed the design of the blob store system and its major components in detail in the previous lesson, a number of interesting questions still require answers. For example, how do we store large blobs? Do we store them in the same disk, in the same machine, or do we divide those blobs into chunks? How many replicas of a blob should be made to ensure reliability and availability? How do we search for and retrieve blobs quickly?. These are just some of the questions that might come up.

This lesson addresses these important design concerns. The table below summarizes the goals of this lesson.

Section	Summary of the Lesson Purpose
Blob metadata and retrieval of blobs.	This is the metadata thatâs maintained to ensure efficient storage
Partitioning	This determines how blobs are partitioned among different data nodes.
Blob indexing	This shows us how to efficiently search for blobs.
Pagination	This teaches us how to conceive a method for the retrieval of a limited number of blobs to ensure improved readability and loading time.
Replication	This teaches us how to replicate blobs and tells us how many copies we should maintain to improve availability.
Garbage collection	This teaches us how to delete blobs without sacrificing performance
Streaming	This teaches us how to stream large files chunk-by-chunk to facilitate interactivity for users.
Caching	This shows us how to improve response time and throughpu

Before we answer the questions listed above, letâs look at how we create layers of abstraction for the user to hide the internal complexity of a blob store. These abstraction layers help us make design-related decisions as well.

There are three layers of abstractions:

1. User account: Users uniquely get identified on this layer through their account_ID. Blobs uploaded by users are maintained in their containers.
2. Container: Each user has a set of containers that are all uniquely identified by a container_ID. These containers contain blobs.
3. Blob: This layer contains information about blobs that are uniquely identified by their blob_ID. This layer maintains information about the metadata of blobs thatâs vital for achieving the availability and reliability of the system.

We can take routing, storage, and sharding decisions on the basis of these layers. The table below summarizes these layers.

Level	Mapping	Uniquely identified by	Layered Information
Sharded by			Information
Userâs blob store	account	account_ID	list of containers_ID values
account_ID	Account ->	list of containers	
Container		container_ID	List of blob_ID values
container_ID	Container ->	list of blobs	
Blob		blob_ID	{list of chunks, chunkInfo: data
node ID's,.. }	blob_ID	Blob ->	list of chunks

Note: We generate unique IDs for user accounts, containers, and blobs using a unique ID generator.

Besides storing the actual blob data, we have to maintain some metadata for managing the blob storage. Let's see what that data is.

Blob metadata

When a user uploads a blob, it's split into small-sized chunks in order to be able to support the storage of large files that can't fit in one contiguous location, in one data node, or in one block of a disk associated with that data node. The chunks for a single blob are then stored on different data nodes that have enough storage space available to store these chunks. There are billions of blobs that are kept in storage. The master node has to store all the information about the blob's chunks and where they are stored, so that it can retrieve the chunks on reads. The master node assigns an ID to each chunk.

The information about a blob consists of chunk IDs and the name of the assigned data node for each chunk. We split the blobs into equal-sized chunks. Chunks are replicated to enable them to deal with data node failure. Hence, we also store the replica IDs for each chunk. We have access to all this information pertaining to each blob.

Let's say we have a blob of 128 MB, and we split it into two chunks of 64 MB each. The metadata for this blob is shown in the following table:

'''

Chunk: A chunk is the minimum unit of data for writing and reading.

'''

'''

Blob Metadata

Chunk	Datanode ID	Replica 1 ID	Replica 2 ID	Replica 3 ID
1	d1b1	r1b1	r2b1	r3b1
2	d1b2	r1b2	r2b2	r3b2

'''

Note: To avoid complexity, the chunk size is fixed for all the blobs in a blob store. The chunk size depends on the performance requirements of a blob store. We desire a larger chunk size to maintain small metadata at the master node because a large chunk size results in a higher disk latency, which leads to slower performance. Interestingly, disks can have almost the same latency for reading and writing a range of data. For example, a disk can have similar latency for writing MBs in the range of 4-8. Additionally, they can have similar latency for writing data in the range of 9-20 MBs. This is due to the use of contiguous sectors on the disks, and caching on the disk and on the server by its OS.

'''

A chunk is the minimum unit of data for writing and reading.

'''

We maintain three replicas for each block. When writing a blob, the master node identifies the data and the replica nodes using its free space management system. Besides handling data node failure, the replica nodes are also used to serve read/write requests, so that the primary node is not overloaded.

In the example above, the blob size is a multiple of the chunk size, so the master node can determine how many Bytes to read for each chunk.

Question

What if the blob size isn't a multiple of our configured chunk size? How does the master node know how many Bytes to read for the last chunk?

Answer

If the blob size isn't a multiple of the chunk size, the last chunk won't be full.

The master node also keeps the size of each blob to determine the number of Bytes to read for

the last chunk.

Partition data

We talked about the different levels of abstraction in a blob storeâ the account layer, the container layer, and the blob layer. There are billions of blobs that are stored and read. There is a large number of data nodes on which we store these blobs. If we look for the data nodes that contain specific blobs out of all of the data nodes, it would be a very slow process. Instead, we can group data nodes and call each group a partition. We maintain a partition map table that contains a list of all the blobs in each partition. If we distribute the blobs on different partitions independent of their container IDs and account IDs, we encounter a problem, as shown in the following illustration:

[Billions of blobs](./blobpath.jpg)

[Range partitioning based on blob IDs](./range_partition.jpg)

Partitioning based on the blob IDs causes certain problems. For example, the blobs under a specific container or account may reside in different partitions that add overhead while reading or listing the blobs linked to a particular account or a particular container.

To overcome the problem described above, we can partition the blobs based on the complete path of the blob. The partition key here is the combination of the account ID, container ID, and blob ID. This helps in co-locating the blobs for a single user on the same partition server, which enhances performance.

'''

Note: The partition mappings are maintained by the master node, and these mappings are stored in the distributed metadata storage.

'''

Blob indexing

Finding specific blobs in a sea of blobs becomes more difficult and time-consuming with an increase in the number of blobs that are uploaded to the storage. The blob index solves the problem of blob management and querying.

To populate the blob index, we define key-value tag attributes on the blobs while uploading the blobs. We use multiple tags, like container name, blob name, upload date and time, and some other categories like the image or video blob, and so on.

As shown in the following illustration, a blob indexing engine reads the new tags, indexes them, and exposes them to a searchable blob index:

[Indexing and searching blobs](./indexing.jpg)

We can categorize blobs as well as sort blobs using indexing. Letâs see how we utilize indexing in pagination.

Pagination for listing

Listing is about returning a list of blobs to the user, depending on the userâs entered prefix. A prefix is a character or string that returns the blobs whose name begins with that particular character or string.

[Pagination](./list_blobs.jpg)

Users may want to list all the blobs associated with a specific account, all the blobs present inside a specific container, or they may want to list some public blobs based on a prefix. The problem is that this list could be very long. We canât return the whole list to the user in one go. So, we have to return the list of the blobs in parts.

Letâs say a user wants a list of blobs associated with their account and there are a total of

2,000 blobs associated with that account. Searching, returning, and loading too many blobs at once affects performance. This is where paging becomes important. We can return the first five results and give users a next button. On each click of the next button, it returns the next five results. This is called pagination.

The application owners set the number of results to return depending on these factors:

- How much time they assume the users should wait for query response.
- How many results they can return in that time. We have shown five results per page, which is a very small number. We use this number just for visualization purposes.

'''

Question

How do we decide which five blobs to return first out of the 2,000 blobs total?

Answer

Here, we utilize indexing to sort and categorize the blobs. We should do this beforehand, while we store the blobs. Otherwise, it becomes challenging at the time of returning the list to the user. There could be millions or billions of blobs and we can't sort them quickly when the request is received.

'''

For pagination, we need a continuation token as a starting point for the part of the list that's returned next. A continuation token is a string token that's included in the response to a query if the total number of queried results exceeds the maximum number of results that we can return at once. As a result, it serves as a pointer, allowing the re-query to pick up where we left off.

Replication

Replication is carried out on two levels to support availability and strong consistency. To keep the data strongly consistent, we synchronously replicate data among the nodes that are used to serve the read requests, right after performing the write operation. To achieve availability, we can replicate data to different regions or data centers after performing the write operation. We don't serve the read request from the other data center or regions until we have replicated data there.

These are the two levels of replication:

- Synchronous replication within a storage cluster.
- Asynchronous replication across data centers and regions.

Synchronous replication within a storage cluster

A storage cluster is made up of N racks of storage nodes, each of which is configured as a fault domain with redundant networking and power.

We ensure that every data written into a storage cluster is kept durable within that storage cluster. The master node maintains enough data replicas across the nodes in distinct fault domains to ensure data durability inside the cluster in the event of a disk, node, or rack failure.

'''

Note: This intra-cluster replication is done on the critical path of the client's write requests.

'''

Success can be returned to the client once a write has been synchronously replicated inside that storage cluster. This allows for quick writes because:

- We replicate data within a storage cluster where all the nodes are nearby, thus reducing the latency.
- We use in-lined data copying where we use redundant network paths to copy data to all the replicas in parallel.

This replication technique helps maintain data consistency and availability inside the storage

cluster.

Asynchronous replication across data centers and region

The blob store's data centers are present in different regions—for example, Asia-Pacific, Europe, eastern US, and more. In each region, we have more than one data center placed at different locations, so that if one data center goes down, we have other data centers in the same region to step in and serve the user requests. There are a minimum of three data centers within each region, each separated by miles to protect against local events like fires, floods, and so on.

The number of copies of a blob is called the replication factor. Most of the time, a replication factor of three is sufficient.

[These are the regions and availability zones. Dark yellow is the primary data and light yellow are the replicas](./replication.jpg)

We keep four copies of a blob. One is the local copy within the data center in the primary region to protect against server rack and drive failures. The second copy of the blob is placed in the other data center within the same region to protect against fire or flooding in the data center. The third copy is placed in the data center of a different region to protect against regional disasters.

Garbage collection while deleting a blob

Since the blob chunks are placed at different data nodes, deleting from many different nodes takes time, and holding a client until that's done is not a viable option. Due to real-time latency optimization, we don't actually remove the blob from the blob store against a delete request. Instead, we just mark a blob as `DELETED` in the metadata to make it inaccessible to the user. The blob is removed later after responding to the user's delete request.

Marking the blob as deleted, but not actually deleting it at the moment, causes internal metadata inconsistencies, meaning that things continue to take up storage space that should be free. These metadata inconsistencies have no impact on the user. For example, for a blob marked as deleted in the metadata, we still have the entries for that blob's chunks in the metadata. The data nodes are still holding on to that blob's chunks. Therefore, we have a service called a garbage collector that cleans up metadata inconsistencies later. The deletion of a blob causes the chunks associated with that blob to be freed. However, there could be an appreciable time delay between the time a blob is deleted by a user and the time of the corresponding increase in free space in the blob store. We can bear this appreciable time delay because, in return, we have a real-time fast response benefit for the user's delete blob request.

The whole deletion process is shown in the following illustration:

[Garbage collection](./gc)

Stream a file

To stream a file, we need to define how many Bytes are allowed to be read at one time. Let's say we read X number of Bytes each time. The first time we read the first X Bytes starting from the 0th Byte (0 to $X-1$) and the next time, we read the next X Bytes (X to $2X-1$).

...

Question

How do we know which Bytes we have read first and which Bytes we have to read next?

Answer

We can use an offset value to keep track of the Byte from which we need to start reading again.

...

Cache the blob store

Caching can take place at multiple levels. Below are some examples:

- The metadata for a blob's chunks is cached on the client side when it's read for the first time. The client can go directly to the data nodes without communicating to the master node to

read the same blob a second time.

- At the front-end servers, we cache the partition map and use it to determine which partition server to forward each request to.
- The frequently accessed chunks are cached at the master node, which helps us stream large objects efficiently. It also reduces disk I/O.

Note: The caching of the blob store is usually done using CDN. The Azure blob store service cache the publicly accessible blob in Azure Content Delivery Network until that blob's TTL (time-to-live) elapses. The origin server defines the TTL, and CDN determines it from the Cache-Control header in the HTTP response from the origin server.

We covered the design factors that should be taken into account while designing a blob store. Now, we'll evaluate what we designed.

Blob Store/Design of a Blob Store/README.md

Design of a Blob Store

High-level design

Let's identify and connect the components of a blob store system. At a high level, the components are clients, front-end servers, and storage disks.

[The high-level design of a blob store](./design.jpg)

The client's requests are received at the front-end servers that process the request. The front-end servers store the client's blob onto the storage disks attached to them.

API design

Let's look into the API design of the blob store. All of the functions below can only be performed by a registered and authenticated user. For the sake of brevity, we don't include functionalities like registration and authentication of users.

****Create container****

The createContainer operation creates a new container under the logged-in account from which this request is being generated.

```
createContainer(containerName)
```

///

Parameter	Description
-----------	-------------

containerName	This is the name of the container. It should be unique within a storage account.
---------------	--

****Upload blobs****

The client's data is stored in the form of Bytes in the blob store. The data can be put into container with the following code:

///

```
putBlob(containerPath, blobName, data)
```

///

///

Parameter	Description
-----------	-------------

containerPath	This is the path of the container in which we upload the blob. It consists of the accountID and containerID.
---------------	--

blobName	This is the name of the blob. It should be unique within a container,
----------	---

otherwise our system will give the blob that was uploaded later a version number.

```
data          This is a file that the user wants to upload to the blob store.
'''
```

```
'''
```

Note: This API is just a logical way to spell out needs. We might use a multistep streaming call for actual implementation if the data size is very large.

```
'''
```

****Download blobs****

Blobs are identified by their unique name or ID.

```
'''
```

```
getBlob(blobPath)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------

blobPath	This is the fully qualified path of the data or file, including its unique ID.
----------	--

```
'''
```

****Delete blob****

The deleteBlob operation marks the specified blob for deletion. The actual blob is deleted during garbage collection.

```
'''
```

```
deleteBlob(blobPath)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------

blobPath	This is the path of the blob that the user wants to delete.
----------	---

```
'''
```

****List blobs****

The listBlobs operation returns a list of blobs under the specified container or path.

```
'''
```

```
listBlobs(containerPath)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------

containerPath	This is the path to the container from which the user wants to get the list of blobs.
---------------	---

```
'''
```

****Delete container****

The deleteContainer operation marks the specified container for deletion. The container and all blobs in it are deleted later during garbage collection.

```
'''
```

```
deleteContainer(containerPath)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------


```
containerPath    This is the path to the container that the user wants to delete.
'''
```

```
**List containers**
```

The listContainers operation returns a list of the containers under the specified user's blob store account.

```
'''
```

```
listContainers(accountID)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------

accountID	This is the ID of the user who wants to list their containers.
-----------	--

```
'''
```

```
'''
```

Note: The APIs used to retrieve blobs provide metadata containing size, version number, access privileges, name, and so on.

```
'''
```

Detailed design

We start this section by identifying the key components that we need to complete our blob store design. Then, we look at how these components connect to fulfill our functional requirements.

Components

Here is a list of components that we use in the blob store design:

- Client: This is a user or program that performs any of the API functions that are specified.
- Rate limiter: A rate limiter limits the number of requests based on the user's subscription or limits the number of requests from the same IP address at the same time. It doesn't allow users to exceed the predefined limit.
- Load balancer: A load balancer distributes incoming network traffic among a group of servers. It's also used to reroute requests to different regions depending on the location of the user, different data centers within the same region, or different servers within the same data center. DNS load balancing can be used to reroute the requests among different regions based on the location of the user.
- Front-end servers: Front-end servers forward the users' requests for adding or deleting data to the appropriate storage servers.
- Data nodes: Data nodes hold the actual blob data. It's also possible that they contain a portion of the blob's data. Blobs are split into small, fixed-size pieces called chunks. A data node can accommodate all of the chunks of a blob or at least some of them.
- Master node: A master node is the core component that manages all data nodes. It stores information about storage paths and the access privileges of blobs. There are two types of access privileges: private and public. A private access privilege means that the blob is only accessible by the account containing that blob. A public access privilege means that anyone can access that blob.

```
'''
```

Note: Each of the data nodes in the cluster send the master node a heartbeat and a chunk report regularly. The presence of a heartbeat indicates that the data node is operational. A chunk report lists all the chunks on a data node. If a data node fails to send a heartbeat, the master node considers that node dead and then processes the user requests on the replica nodes. The master node maintains a log of pending operations that should be replayed on the dead data node when it recovers.

```
'''
```

- Metadata storage: Metadata storage is a distributed database that's used by the master node to store all the metadata. Metadata consists of account metadata, container metadata, and blob metadata.

- Account metadata contains the account information for each user and the containers held by

each account.

- Container metadata consists of the list of the blobs in each container.
- Blob metadata consists of where each blob is stored. The blob metadata is discussed in detail in the next lesson.

- Monitoring service: A monitoring service monitors the data nodes and the master node. It alerts the administrator in case of disk failures that require human intervention. It also gets information about the total available space left on the disks to alert administrators to add more disks.

- Administrator: An administrator is responsible for handling notifications from the monitoring services and conducting routine checkups of the overall service to ensure reliability.

The architecture of how these components interconnect is shown in the diagram below:

[The detailed design of a blob store](./details.jpg)

Workflow

We describe the workflow based on the basic operations we can perform on a blob store. We assume that the user has successfully logged in and a container has already been created. A unique ID is assigned to each user and container. The user performs the following operations in a specific container.

****Write a blob****

1. The client generates the upload blob request. If the client's request successfully passes through the rate limiter, the load balancer forwards the client's request to one of the front-end servers. The front-end server then requests the master node for the data nodes it should contact to store the blob.
2. The master node assigns the blob a unique ID using a unique ID generator system. It then splits the large-size blob into smaller, fixed-size chunks and assigns each chunk a data node where that chunk is eventually stored. The master node determines the amount of storage space that's available on the data nodes using a free-space management system.
3. After determining the mapping of chunks to data nodes, the front-end servers write the chunks to the assigned data nodes.
4. We replicate each chunk for redundancy purposes. All choices regarding chunk replication are made at the master node. Hence, the master node also allocates the storage and data nodes for storing replicas.
5. The master node stores the blob metadata in the metadata storage. We discuss the blob's metadata schema in detail in the next lesson.
6. After writing the blob, a fully qualified path of the blob is returned to the client. The path consists of the user ID, container ID where the user has added the blob, the blob ID, and the access level of the blob.

'''

Question

What does the master node do if a user concurrently writes two blobs with the same name inside the same container?

Answer

The master node serializes such operations and assigns a version number to the blob that's uploaded later.

'''

****Reading a blob****

1. When a read request for a blob reaches the front-end server, it asks the master node for that blob's metadata.
2. The master node first checks whether that blob is private or public, based on the path of the blob and whether we're authorized to transfer that blob or not.
3. After authorizing the blob, the master node looks for the chunks for that blob in the metadata and looks at their mappings to data nodes. The master node returns the chunks and

their mappings (data nodes) to the client.

4. The client then reads the chunk data from the data nodes.

'''

Note: The metadata information for reading the blob is cached at the client machine, so that the next time a client wants to read the same blob, we won't have to burden the master node. Additionally, the client's read operation will be faster the next time.

'''

'''

Question

Suppose the master node moves data from one data node to another because of an impending disk failure. The user will now have stale information if they use the cached metadata to access the data. How do we handle such situations?

Answer

In such cases, the client calls fail. The client then flushes the cache and fetches new metadata information from the master node.

'''

****Deleting a blob**** Upon receiving a delete blob request, the master node marks that blob as deleted in the metadata, and frees up the space later using a garbage collector. We learn more about garbage collectors in the next lesson.

'''

Question

Can the master node be considered a single point of failure? If yes, then how can we cope with this problem?

Answer

Yes, because the master node is the central point of a blob store and is a single point of failure. Therefore, we have to have a backup or shadow server in place of a master node.

The technique that we use for this is called checkpointing, meaning we take snapshots of the data at different time intervals. A snapshot captures the state, data, hardware configuration of the running master node, and messages in transit between master and data nodes. It maintains the operation log in an external storage area or snapshot repository. If the master node fails, an automated system or the administrator uses the snapshot to restart that master node from the state it failed at and replays the operation log.

'''

'''

Operation log: A logical timeline, defining the serialized order of concurrent operations

'''

In the next lesson, we talk about the design considerations of a blob store.

Blob Store/Evaluation of a Blob Store's Design/README.md

Evaluation of a Blob Store's Design

Let's evaluate how our design helps us achieve our requirements.

Availability

The replication part of our design makes the system available. For reading the data, we keep four replicas for each blob. Having replicas, we can distribute the request load. If one node fails, the other replica node can serve the request. Moreover, our replica placement strategy handles a whole data center failure and can even handle the situation of region unavailability due to natural disasters. We ensure that enough replicas are available at any point in time using our monitoring service, which makes a copy of the data in a timely manner if the number of failed replicas exceeds the specified threshold.

For write requests, we replicate the data within the cluster in a fault-tolerant way and

quickly respond to the user, making the system available for write requests.

To keep the master node available, we keep a backup of its state. In the case of a master-node failure, we start a new instance of the master node, initializing it from the saved state.

'''

Natural disasters include floods, earthquakes, wildfires, tornadoes, nuclear reactor meltdowns and so on.

'''

Durability

The replication and monitoring services ensure the durability of the data. The data, once uploaded, is synchronously replicated within a storage cluster. If data loss occurs at one node, we can recover the data from the other nodes. The monitoring service monitors the storage disks. If any disk fails, the monitoring service alerts the administrators to change the disk and sends messages to the master node to copy the content on that disk on to the other available disk or the newly added disk. The master node then updates the mapping accordingly.

[Guarantees provided by the blob store](./blob.jpg)

Scalability

The partitioning and splitting of blobs into small-sized chunks helps us scale for billions of blob requests. Blobs are partitioned into separate ranges and served by different partition servers. The partition mappings specify which partition server will serve which particular blob range requests. Partitioning also provides automatic load balancing between partition servers to fulfill the blobs' traffic needs.

Our system is horizontally scalable for storage. As need for storage arises, we add more data nodes. At some point, though, our master node can become a bottleneck. A single master node can handle 10,000 queries per second (QPS).

'''

Question

How can we further scale when our master server hits its limits and we can't improve its computational abilities by vertical scaling?

Answer

We can make two independent instances of our system. Each instance will have its own master node and a set of data nodes. Deployment of a system similar to ours has been shown to scale up to a few petabytes. Therefore, making additional instances can help us scale further.

For further scaling inside a single instance, we need a new, more complicated design.

Windows Azure Storage: A Highly Available Cloud Storage Service with Strong Consistency.

'''

Throughput

We save chunks of a blob on different data nodes that distribute the requests for a blob to multiple machines. Parallel fetching of chunks from multiple data nodes helps us achieve high throughput.

Additionally, caching at different layers—the client side, front-end servers, and the master node—improves our throughput and reduces latency.

Reliability

We achieve reliability through our monitoring techniques. For example, the heartbeat protocol keeps the master node updated on the status of data nodes. This enables the master node to request data from reliable nodes only. Furthermore, it takes necessary precautions to ensure reliable service. For example, the failure of a node triggers the master node to request an additional replica node.

The monitoring services also alert the administrators to change faulty hardware like failed disks or broken network links or switches. To ensure a reliable service, the master node keeps track of the available space on the disk. If the available space left reaches a certain threshold, an alert is sent to the administrators to add new disks.

Consistency

We synchronously replicate the disk data blocks inside a storage cluster upon a write request, making the data strongly consistent inside the storage cluster. This is done on the user's critical path. We serve the subsequent read requests on this data from the same storage cluster until we haven't replicated this data in another data center and or on other storage cluster.

After responding to the write request and replicating data within the storage cluster, we asynchronously replicate blobs in the data centers placed far away or in other regions to ensure availability.

Conclusion

We saw that a blob store is designed to store large-sized and unstructured data. A blob store helps applications store images, videos, audio, and so on. Nowadays, it's used by many applications like YouTube, Facebook, Instagram, Twitter, and more. We designed a system where users can perform a blob store's basic function. Lastly, we evaluated our design based on our non-functional requirements.

Blob Store/Quiz on the Blob Store's Design/README.md

Quiz on the Blob Store's Design

'''

1. (Select the two most suitable options.) What is a blob store best suited for?.

A)

Storing a huge amount of data

B)

Storing unstructured data

C)

Storing structured data

'''

'''

2. What is the core component of the blob store, according to the design lesson?

A)

The master node

B)

A load balancer

C)

A monitoring mechanism

'''

'''

3. (Select all that apply.) What does the master node contain?

A)

Data bytes

B)

Blob to chunks mappings

C)

Chunks to data node mappings

'''

'''

4

The garbage collection process runs at the _____.

A)

master node

B)

data node

C)

monitoring system

'''

'''

5. Which technique is used to list a large number of blobs?

A)

Partitioning

B)

Replication

C)

Pagination

'''

'''

6. Which of the following is an indicator used in pagination?

A)

Continuation token

B)

Offset

'''

'''

7. (True or False) The blob store supports a hierarchical structure of data.

A)

True

B)

False

'''

'''

8. Garbage collection is used for _____.

A)

real-time latency optimization

B)

capacity optimization

'''

'''

9. How do we achieve availability?

A)

With synchronous replication within a storage cluster

B)

With asynchronous replication along the data center and different regions

C)

Both A and B

'''

Answers

'''

1. (Select the two most suitable options.) What is a blob store best suited for?. (A)(B)

A)

Storing a huge amount of data

B)

Storing unstructured data

C)

Storing structured data

Explanation

Blob store is best suited for storing massive amounts of unstructured data.

'''

'''

2. What is the core component of the blob store, according to the design lesson? (A)

A)

The master node

B)

A load balancer

C)

A monitoring mechanism

'''

'''

3. (Select all that apply.) What does the master node contain? (B)

A)

Data bytes

Explanation

The data bytes are stored on the data nodes. Metadata node just contain the mappings not the original data bytes.

B)

Blob to chunks mappings

C)

Chunks to data node mappings

'''

'''

The garbage collection process runs at the _____. (B)

A)

master node

B)

data node

C)

monitoring system

'''

'''

5. Which technique is used to list a large number of blobs? (C)

A)

Partitioning

B)

Replication

C)

Pagination

'''

'''

6. Which of the following is an indicator used in pagination? (A)

A)

Continuation token

B)

Offset

'''

'''

7. (True or False) The blob store supports a hierarchical structure of data. (B)

A)

True

B)

False

Explanation

It is a flat storage.

'''

'''

8. Garbage collection is used for _____. (A)

A)

real-time latency optimization

B)

capacity optimization

'''

'''

9. How do we achieve availability? (C)

A)

With synchronous replication within a storage cluster

B)

With asynchronous replication along the data center and different regions

C)

Both A and B

'''

Blob Store/Requirements of a Blob Store's Design/README.md

Requirements of a Blob Store's Design

Requirements

Let's understand the functional and non-functional requirements below:

Functional requirements

Here are the functional requirements of the design of a blob store:

- Create a container: The users should be able to create containers in order to group blobs. For example, if an application wants to store user-specific data, it should be able to store blobs for different user accounts in different containers. Additionally, a user may want to group video blobs and separate them from a group of image blobs. A single blob store user can create many containers, and each container can have many blobs, as shown in the following illustration. For the sake of simplicity, we assume that we can't create a container inside container.

'''

A container is like a folder in a file system used to group blobs. Don't mix up this container with a Docker container.

'''

[Multiple containers associated with a single storage account, and multiple blobs inside a single container](./blob.jpg)

- Put data: The blob store should allow users to upload blobs to the created containers.
- Get data: The system should generate a URL for the uploaded blob, so that the user can access that blob later through this URL.
- Delete data: The users should be able to delete a blob. If the user wants to keep the data for a specified period of time (retention time), our system should support this functionality.
- List blobs: The user should be able to get a list of blobs inside a specific container.
- Delete a container: The users should be able to delete a container and all the blobs inside it.
- List containers: The system should allow the users to list all the containers under a specific account.

[Functional requirements of a blob store](./f.jpg)

Non-functional requirements

Here are the non-functional requirements of a blob store system:

- Availability: Our system should be highly available.
- Durability: The data, once uploaded, shouldn't be lost unless users explicitly delete that data.
- Scalability: The system should be capable of handling billions of blobs.
- Throughput: For transferring gigabytes of data, we should ensure a high data throughput.
- Reliability: Since failures are a norm in distributed systems, our design should detect and recover from failures promptly.
- Consistency: The system should be strongly consistent. Different users should see the same view of a blob.

[The non-functional requirements of a blob store](./nf.jpg)

Resource estimation

Let's estimate the total number of servers, storage, and bandwidth required by a blob storage system. Because blobs can have all sorts of data, mentioning all of those types of data in our estimation may not be practical. Therefore, we'll use YouTube as an example, which stores videos and thumbnails on the blob store. Furthermore, we'll make the following assumptions to complete our estimations.

Assumptions:

- The number of daily active users who upload or watch videos is five million.
- The number of requests per second that a single blob store server can handle is 500. This number can be significantly higher, depending upon the blob store. For example, Microsoft Azure can handle a maximum of 20,000 IOPS.
- The average size of a video is 50 MB.
- The average size of a thumbnail is 20 KB.
- The number of videos uploaded per day is 250,000.
- The number of read requests by a single user per day is 20.

Number of servers estimation

From our assumptions, we use the number of daily active users (DAUs) and queries a blob store server can handle per second. The number of servers that we require is calculated using the formula given below:

Number of active users/Queries handled per server = 10 K servers

Storage estimation

Considering the assumptions written above, we use the formula given below to compute the total storage required by YouTube in one day:

$$\text{Total_storage/day} = \text{No. of videos per day} \times (\text{Storage per video} + \text{Storage per thumbnail})$$

Putting the numbers from above into the formula gives us 12.51 TB /day, which is the approximate storage required by YouTube per day for keeping a single copy of the uploaded videos in a single resolution.

'''

Total Storage Required to Store Videos and Thumbnails Uploaded Per Day on

YouTube

No. of videos per day	Storage per video (MB)	Storage per thumbnail (KB)
Total storage per day (TB)		
250000	50	20
12.51		

'''

Bandwidth estimation

Let's estimate the bandwidth required for uploading data to and retrieving data from the blob store.

Incoming traffic: To estimate the bandwidth required for incoming traffic, we consider the total data uploaded per day, which indirectly means the total storage needed per day that we calculated above. The amount of data transferred to the servers per second can be computed using the following formula:

$$\text{Total bandwidth} = \text{Total storage per day} / (24 \times 60 \times 60)$$

'''

Bandwidth Required for Uploading Videos on YouTube

Total storage per day (TB)	Seconds in a day	Bandwidth (Gb/s)
12.51	86400	1.16

'''

Outgoing traffic: Since the blob store is a read-intensive store, most of the bandwidth is required for outgoing traffic. Considering the aforementioned assumptions, we calculate the bandwidth required for outgoing traffic using the following formula:

Total bandwidth = (No. of active users per day x No. of requests per day per user x Total data size)/ seconds in a day

'''

No. of active users per day	No. of requests per user	Data size (MB)
Bandwidth required (Gb/s)		
5000000	50	20
462.96		

[Summarizing the bandwidth requirements of a blob store system for YouTube videos only](./bandwidth.jpg)

Building blocks we will use

We use the following building blocks in the design of our blob store system:

[Building blocks for the design of a task scheduler](./bb.jpg)

- Rate Limiter: A rate limiter is required to control the usersâ interaction with the system
- Load balancer: A load balancer is needed to distribute the request load onto different servers.
- Database: A database is used to store metadata information for the blobs.
- Monitoring: Monitoring is needed to inspect storage devices and the space available on them in order to add storage on time if needed.

In this lesson, we discussed the requirements and estimations of the blob store system. Weâ design the blob store system in the next lesson, all while following the delineated requirements.

Blob Store/System Design A Blob Store/README.md

System Design: A Blob Store

What is a blob store?

Blob store is a storage solution for unstructured data. We can store photos, audio, videos, binary executable codes, or other multimedia items in a blob store. Every type of data is stored as a blob. It follows a flat data organization pattern where there are no hierarchies, that is, directories, sub-directories, and so on.

Mostly, itâ s used by applications with a particular business requirement called write once, read many (WORM), which states that data can only be written once and that no one can change it. Just like in Microsoft Azure, the blobs are created once and read many times. Additionally, these blobs canâ t be deleted until a specified interval, and they also canâ t be modified to protect critical data.

'''

A blob (binary large object) consists of a collection of binary data stored as a single unit.

'''

'''

Note: It isnâ t necessary for all applications to have this WORM requirement. However, we are assuming that the blobs that are written canâ t be modified. Instead of modifying, we can upload a new version of a blob if needed.

'''

Why do we use a blob store?

Blob store is an important component of many data-intensive applications, such as YouTube, Netflix, Facebook, and so on. The table below displays the blob storage used by some of the most well-known applications. These applications generate a huge amount of unstructured data every day. They require a storage solution that is easily scalable, reliable, and highly available, so that they can store large media files. Since the amount of data continuously increases, these applications need to store an unlimited number of blobs. According to some estimates, YouTube requires more than a petabyte of additional storage per day. In a system like YouTube, a video is stored in multiple resolutions. Additionally, the video in all resolutions is replicated many times across different data centers and regions for availability.

purposes. That's why the total storage required per video is not equal to the size of the uploaded video.

'''

System Blob Store

Netflix S3

YouTube Google Cloud Storage

Facebook Tectonic

'''

How do we design a blob store system?

We have divided the design of the blob store into five lessons and a quiz.

1. Requirements: In this lesson, we identify the functional and non-functional requirements of a blob store. We also estimate the resources required by our blob store system.

2. Design: This lesson presents us with a high-level design, the API design, and a detailed design of the blob store, while explaining the details of all the components and the workflow.

3. Design considerations: In this lesson, we discuss some important aspects of design. For example, we learn about the database schema, partitioning strategy, blob indexing, pagination, and replication.

4. Evaluation: In this lesson, we evaluate our blob store based on our requirements.

5. Quiz: In this lesson, we assess understanding of the blob store design.

Let's start with the requirements of a blob store system.

Building Blocks/README.md

Introduction to Building Blocks for Modern System Design

The bottom-up approach for modern system design

System design problems usually have some similarities, though specific details are often unique. We have extracted these similarities across design problems as the basic building blocks we'll be covering. One example of a building block is a load-balancing component, which we'll probably use in every design problem in one way or the other.

The purpose of separating the building blocks is to thoroughly discuss their design just once. This means that later we can use them anywhere without having to go over them in detail again. We can think about building blocks as bricks to construct more effective, capable systems.

Many of the building blocks we discuss are also available for actual use in the public clouds, such as Amazon Web Services (AWS), Azure, and Google Cloud Platform (GCP). We can use such constructs to build a system to further cement our understanding. (We won't construct the system in this course, but we've left it as an exercise for interested learners.)

We'll discuss the following building blocks in detail:

1. Domain Name System

This building block focuses on how to design hierarchical and distributed naming systems for computers connected to the Internet via different Internet protocols.

[Domain Name System](./dns.jpg)

2. Load Balancers

Here, we'll understand the design of a load balancer, which is used to fairly distribute incoming clients' requests among a pool of available servers. It also reduces load and can bypass failed servers.

[Load Balancers](./load_balancer.jpg)

3. Databases

This building block enables us to store, retrieve, modify, and delete data in connection with different data-processing procedures. Here, weâ ll discuss database types, replication, partitioning, and analysis of distributed databases.
[Databases](./db.jpg)

4. Key-Value Store

It is a non-relational database that stores data in the form of a key-value pair. Here, weâ ll explain the design of a key-value store along with important concepts such as achieving scalability, durability, and configurability.
[Key-Value Store](./kv.jpg)

5. Content Delivery Network

In this chapter, weâ ll design a content delivery network (CDN) thatâ s used to keep viral content such as videos, images, audio, and webpages. It efficiently delivers content to end users while reducing latency and burden on the data centers.
[Content Delivery Network](./cdn.jpg)

6. Sequencer

In this building block, weâ ll focus on the design of a unique IDs generator with a major focus on maintaining causality. It also explains three different methods for generating unique IDs.
[Sequencer](./sequencer.jpg)

7. Service Monitoring

Monitoring systems are critical in distributed systems because they help analyze the system and alert the stakeholders if a problem occurs. Monitoring is often useful to get early warning systems so that system administrators can act ahead of an impending problem becoming a huge issue. Here, weâ ll build two monitoring systems, one for the server-side and the other for client-side errors.
[Service Monitoring](./service_monitor.jpg)

8. Distributed Caching

In this building block, weâ ll design a distributed caching system where multiple cache servers coordinate to store frequently accessed data.
[Distributed Caching](./distributed_caching.jpg)

9. Distributed Messaging Queue

In this building block, weâ ll focus on the design of a queue consisting of multiple servers, which is used between interacting entities called producers and consumers. It helps decouple producers and consumers, results in independent scalability, and enhances reliability.
[Distributed Messaging Queue](./distributed_message_queue.jpg)

10. Publish-Subscribe System

In this building block, weâ ll focus on the design of an asynchronous service-to-service communication method called a pub-sub system. It is popular in serverless, microservices architectures and data processing systems.
[Publish-Subscribe System](./pub_sub_sys.jpg)

11. Rate Limiter

Here, weâ ll design a system that throttles incoming requests for a service based on the predefined limit. It is generally used as a defensive layer for services to avoid their excessive usage-whether intended or unintended.
[Rate Limiter](./rate_limiter.jpg)

12. Blob Store

This building block focuses on a storage solution for unstructured dataâ for example, multimedia files and binary executables.
[Blob Store](./blob_store.jpg)

13. Distributed Search

A search system takes a query from a user and returns relevant content in a few seconds or

less. This building block focuses on the three integral components: crawl, index, and search.
[Distributed Search](./distributed_search.jpg)

14. Distributed Logging

Logging is an I/O intensive operation that is time-consuming and slow. Here, weâ ll design a system that allows services in a distributed system to log their events efficiently. The system will be made scalable and reliable.

[Distributed Logging](./distributed_logging.jpg)

15. Distributed Task Scheduling

Weâ ll design a distributed task scheduler system that mediates between tasks and resources. intelligently allocates resources to tasks to meet task-level and system-level goals. Itâ s often used to offload background processing to be completed asynchronously.

[Distributed Task Scheduling](./distributed_task_scheduling.jpg)

16. Sharded Counters

This building block demonstrates an efficient distributed counting system to deal with millions of concurrent read/write requests, such as likes on a celebrityâ s tweet.

[Sharded Counters](./shared_counter.jpg)

Conventions

For elaboration, weâ ll use a â Requirementsâ section whenever we design a building block (design problem). The â Requirementsâ section will highlight the deliverables we expect from the developed design. â Requirementsâ will have two sub-categories:

'''

Functional requirements: These represent the features a user of the designed system will be able to use. For example, the system will allow a user to search for content using the search bar.

Non-functional requirements (NFRs): The non-functional requirements are criteria based on which the user of a system will consider the system usable. NFR may include requirements like high availability, low latency, scalability, and so on.

'''

Letâ s start with our building blocks.

Concluding Remarks/README.md

Conclusions

Before we conclude, weâ d like to congratulate you for completing such a challenging course!

In this course, we learned system design activity from the lens of basic building blocksâ the fundamental subsystems that combine to make a bigger system. Doing so enabled us to focus on the specific business use cases instead of repeating the recurring concepts.

We carefully selected our design problems from many different domains, and weâ ve made an explicit effort to incorporate some unique aspects or issues of system design activity in each of them.

'''

Unique Aspects of Each Design Problem	
Design Problem	Unique Aspect of Design
YouTube needs	Building custom data stores like Vitess and BigTable to meet scalability needs

Quora	Vertical sharding of MySQL database to meet the scalability requirements
Google Maps high performance	The use of segmentation of a map to meet scalability needs and achieve high performance
Yelp	Usage of Quadtrees for speedy access to spatial data
Uber driver and rider on maps	Improved payment service to ensure fraud detection, and matching the driver and rider on maps
Twitter thousands of instances	The use of client-side load balancers for multiple services that had thousands of instances in order to reduce latency
Newsfeed	A recommendation system to ensure ranking and feed suggestions
Instagram scalable and performant system	A perfect example of how different building blocks combine to build a scalable and performant system
TinyURL	Encoding IDs in the base-58 system for generating unique short URLs
Web crawler	Detection, identification, and resolution of Web crawler traps
WhatsApp	Message management for offline users
Typeahead	The usage of an efficient trie data structure to provide suggestions
Google Docs	Concurrency management for simultaneous writes, using techniques like operational transformation (OT) and Conflict-free Replicated Data Type (CRDT)

'''

Concluding the Building Blocks Discussion/The RESHADED Approach for System Design/README.md

The RESHADED Approach for System Design

Introduction

System design problems are not straightforward. We don't have a universal formula that we can use for all design problems. However, we can use a high-level common strategy to set the tone for a good solution to any design problem. We call this the RESHADED approach. Generally, all our design problems are solved by keeping this strategy in mind. RESHADED is a guideline that we'll use to resolve different design problems. Although there's no such thing as a one-size-fits-all solution, using this approach will have its advantages, as we will see.

[Breakdown of each letter in the RESHADED approach]

Advantages of RESHADED

Before we go through each word in RESHADED, let's cover the benefits of our overall approach. Some of the key advantages of this approach are below:

1. The RESHADED approach helps us remember some key steps for the resolution of every design problem. This means that, at any point in time, there will always be a next step laid out ahead of us.
2. The solution we come up with will have all the basic ingredients required to solve any design problem. Not only that, but the solution offered through the RESHADED approach will be systematic and thoughtful.

Exploring RESHADED

Below, we describe what steps we take under each word in RESHADED:

****Requirements****: During this step, we gather all the requirements of the design problem and

define its scope. Requirements include understanding what the service is, how it works, and what its main features are. Our goal in this step is to gather the functional and non-functional requirements of a service we are about to design.

****Estimation****: As the name suggests, this step estimates the resources required to provide the service to a defined number of users. By resources, we mean the hardware or infrastructural resources. Some sample estimation questions are the following:

1. How many servers will we require to provide smooth services to 500 million daily active users (DAU)?
2. How much storage do we need if we have to store 125 million tweets per day, and 20% of tweets contain media?

Estimations are important because they help us understand the scale of the system we'll design. We'll make key decisions based on the estimate. For example, we'll decide what type of data to use for storing our data, which data structure will give optimal performance, and so on.

****Storage schema (optional)****: This step involves articulating our data model—that is, we define which tables we need and what type of fields are part of each table. However, this is an optional step, and we may not exercise this effort in every design problem.

****High-level design****: This step involves identifying the main components and building blocks we'll use to design our desired system. We do this by getting inspiration from our functional and non-functional requirements.

This is considered the first step toward the complete design of our system and therefore requires further iteration and improvement. Primarily, this section will focus on fulfilling the functional requirements.

****API design****: The goal in this phase is to build interfaces for our service. Using these interfaces, users can call various services within our system. These interfaces are in the form of API calls and are generally a translation of our functional requirements.

****Detailed design****: The detailed design starts by recognizing the limitations of the high-level design. We'll capitalize on these limitations to evolve our design. During this step, we'll finalize our design by mentioning all the components and building blocks that we'll use.

We also define the workflow of our design and its usage of different technologies. The detailed design aims to fulfill the functional as well as non-functional requirements of the problem.

****Evaluation****: This step will measure the effectiveness of our solution. In other words, we justify how our design fulfills the functional and non-functional requirements.

We discuss different trade-offs we made in our solution and also identify room for improvement.

****Distinctive component/feature****: At the start of this lesson, we discussed how no one-size-fits-all solution exists. This step is to identify a unique aspect for each design problem and discuss it. For example, the Uber design problem has payment service and fraud detection as its unique feature. In contrast, Google Docs has concurrency control, which is required when different users want to edit the same section of a document simultaneously.

Next, weâ ll apply our guideline (RESHADED) to many design problems.

Concluding the Building Blocks Discussion/Wrapping Up the Building Blocks Discussion/README.md

More Kinds of Constructors

We have covered a good number of building blocks that will enable us to solve bigger design problems. In the coming chapters, weâ ll solve a variety of problems that will use various building blocks.

Since weâ ve already covered the building blocks, we can focus on solving bigger design problems. From this point onwards, weâ ll assume that youâ re well acquainted with all the discussed building blocks when we use them in our coming design problems.

Whatâ s next?

Weâ ll learn and explore thirteen design problems in the next chapters. Weâ ll study the following system designs:

1. YouTube
2. Quora
3. Google Maps
4. Yelp
5. Uber
6. Twitter
7. Newsfeed
8. Instagram
9. TinyURL
10. Web Crawler
11. WhatsApp
12. Typeahead
13. Google Docs

Each of these designs is an independent chapter, but we recommend that you go through these chapters in the assigned order. Some problems have background material that is helpful in other design problems. For example, working through the Google Maps problem first can be useful for readers interested in learning about the Uber design problem.

At the end of the course, weâ ll conclude our course with the â Spectacular Failuresâ chap where weâ ll discuss how a minor bug or mistake led to a significant outage or failure of some of the most successful systems designed.

Content Delivery Network (CDN)/Design of a CDN/README.md

Design of a CDN

CDN design

Weâ ll explain our CDN design in two phases. In the first phase, weâ ll cover the components that comprise a CDN. By the end of this phase, weâ ll understand why we need a specific component. In the second phase, weâ ll explore the workflow by explaining how each component interacts with others to develop a fully functional CDN. Letâ s dive in.

CDN components

The following components comprise a CDN:

- Clients: End users use various clients, like browsers, smartphones, and other devices, to request content from the CDN.
- Routing system: The routing system directs clients to the nearest CDN facility. To do that effectively, this component receives input from various systems to understand where content is placed, how many requests are made for particular content, the load a particular set of servers is handling, and the URI (Uniform Resource Identifier) namespace of various contents. In the

next lesson, weâ ll discuss different routing mechanisms to forward users to the nearest CDN facility.

- Scrubber servers: Scrubber servers are used to separate the good traffic from malicious traffic and protect against well-known attacks, like DDoS. Scrubber servers are generally used only when an attack is detected. In that case, the traffic is scrubbed or cleaned and then routed to the target destination.
- Proxy servers: The proxy or edge proxy servers serve the content from RAM to the users. Proxy servers store hot data in RAM, though they can store cold data in SSD or hard drive as well. These servers also provide accounting information and receive content from the distribution system.
- Distribution system: The distribution system is responsible for distributing content to all the edge proxy servers to different CDN facilities. This system uses the Internet and intelligent broadcast-like approaches to distribute content across the active edge proxy servers.
- Origin servers: The CDN infrastructure facilitates users with data received from the origin servers. The origin servers serve any unavailable data at the CDN to clients. Origin servers will use appropriate stores to keep content and other mapping metadata. Though, we wonâ t discuss the internal architecture of origin infrastructure here.
- Management system: The management systems are important in CDNs from a business and managerial aspect where resource usage and statistics are constantly observed. This component measures important metrics, like latency, downtime, packet loss, server load, and so on. For third-party CDNs, accounting information can also be used for billing purposes.

[CDN components]

Workflow

The workflow for the abstract design is given below:

1. The origin servers provide the URI namespace delegation of all objects cached in the CDN to the request routing system.
2. The origin server publishes the content to the distribution system responsible for data distribution across the active edge proxy servers.
3. The distribution system distributes the content among the proxy servers and provides feedback to the request routing system. This feedback is helpful in optimizing the selection of the nearest proxy server for a requesting client. This feedback contains information about which content is cached on which proxy server to route traffic to relevant proxy servers.
4. The client requests the routing system for a suitable proxy server from the request routing system.
5. The request routing system returns the IP address of an appropriate proxy server.
6. The client request routes through the scrubber servers for security reasons.
7. The scrubber server forwards good traffic to the edge proxy server.
8. The edge proxy server serves the client request and periodically forwards accounting information to the management system. The management system updates the origin servers and sends feedback to the routing system about the statistics and detail of the content. However, the request is routed to the origin servers if the content isnâ t available in the proxy servers. Itâ s also possible to have a hierarchy of proxy servers if the content isnâ t found in the edge proxy servers. For such cases, the request gets forwarded to the parent proxy servers.

API Design

This section will discuss the API design of the functionalities offered by CDN. This will help us understand how the CDN will receive requests from the clients, receive content from the origin servers, and communicate to other components in the network. Letâ s develop APIs for each of the following functionalities:

- Retrieve content
- Deliver content
- Request content
- Search content
- Update content
- Delete content

Content can be anything, like a file, video, audio, or other web object. Here, weâ€™ll use the word â€˜contentâ€™ to refer to all of the above. For clarity, we wonâ€™t discuss the privacy-related parametersâ€”like if the content is public or private, who should be able to access this content if it should be encrypted, and so onâ€”in the following APIs.

Retrieve (proxy server to origin server)

If the proxy servers request content, the GET method retrieves the content through the /retrieveContent API below:

'''

retrieveContent(proxyserver_id, content_type, content_version, description)

'''

Letâ€™s see the details of the parameters:

Details of Parameters

'''

Parameter	Description
-----------	-------------

proxyserver_id	This is a unique ID of the requesting proxy server.
----------------	---

content_type	This data structure will contain information about the requested content. Specifically, it will contain the category (audio, video, document, script, and so on), the type of clients itâ€™s requested for, and the requested quality (if any).
--------------	---

content_version	This represents the version number of the content. For the /retrieveContent API, the content_version will contain the current version of the content residing in the proxy server. The content_version will be NULL if no previous version is available at the proxy server.
-----------------	--

description	This specifies the content detailâ€”for example, the videoâ€™s extension, resolution detail, and so on if the content_type is video.
-------------	--

'''

The above API gives a response in a JSON file, which contains the text, content types, links to the images or videos in the content, and so on.

'''Javascript

"Object_links": [

{

 "name": "videos"

 "link": "https://app_server.com/api/assets/videos/"

},

{

 "name": "illustrations"

 "link": "https://app_server.com/api/assets/illustrations/"

},

]

The JSON file from where various objects will be downloaded at the proxy

servers.

'''

Deliver (origin server to proxy servers)

The origin servers use this API to deliver the specified content, the updated version, to the proxy servers through the distribution system. We call this the /deliverContent API:

'''

deliverContent(origin_id, server_list, content_type, content_version, description)

'''

Details of Parameters

'''

Parameter	Description
origin_id	This recognizes each origin server uniquely.
server_list	This identifies the list of servers the content will be pushed to by the distribution system.
content_version	This represents the updated version of the content at the origin server. The proxy server receiving the content will discard the previous version.

'''

The rest of the parameters have been explained above already.

Request (clients to proxy servers)

The users use this API to request the content from the proxy servers. We call this the /requestContent API:

```
'''
requestContent(user_id, content_type, description)
'''
```

Details of Parameter

'''

Parameter	Description
user_id	This is the unique ID of the user who requested the content.

'''

The specified proxy server returns the particular content to the requested users in response to the above API.

'''Javascript

```
"Object_links": [
    {
        "name": "components"
        "link": https://cdn.app_server.com/api/components/
    },
    {
        "name": "css"
        "link": https://cdn.app_server.com/api/css/
    },
    {
        "name": "illustrations"
        "link": https://cdn.app_server.com/api/assets/illustrations/
    },
    {
        "name": "videos"
        "link": https://cdn.app_server.com/api/assets/videos/
    },
    {
        "name": "icons"
        "link": https://cdn.app_server.com/api/icons/
    },
    {
        "name": "fonts"
        "link": https://cdn.app_server.com/api/fonts/
    },
]
```

'''

The JSON file from where various objects will be downloaded at the user end.

Search (proxy server to peer proxy servers)

Although the content is first searched locally at the proxy server, the proxy servers can also probe requested content in the peer proxy servers in the same PoP through the /searchContent API. This could flood the query to all proxy servers in a PoP. Alternatively, we can use a data store in the PoP to query the content, though proxy servers will need to maintain what content is available on which proxy server.

The /searchContent API is shown below:

```
'''
searchContent(proxyserver_id, content_type, description)
'''
```

Update (proxy server to peer proxy servers)

The proxy servers use the /updateContent API to update the specified content in the peer proxy servers in the PoP. It does so when specified isolated scripts run on the CDN to provide image resizing, video resolution conversion, security, and many more services. This type of scripting is known as serverless scripting.

The /updateContent API is shown below:

```
'''
updateContent(proxyserver_id, content_type, description)
'''
```

Details of Parameter

```
'''
```

Parameter	Description
-----------	-------------

porxyserver_id	This recognizes the proxy server uniquely in the PoP to update the content.
----------------	---

```
'''
```

The rest of the parameters have been explained above already.

```
'''
```

Note: The Delete API isnâ t discussed here. In our caching chapter, we discussed different eviction mechanisms in detail. Those mechanisms are also applicable for a CDN content eviction. Nevertheless, situations can arise where the Delete APIs may be required. Weâ ll discuss a few content consistency mechanisms, like how much time content stays in the cache, in the next lesson.

```
'''
```

In the upcoming lessons, weâ ll dive deep into the characteristics of CDNs.

Content Delivery Network (CDN)/Evaluation of CDN's Design/README.md

Evaluation of CDN's Design

Evaluation

Here, we see how our design fulfills the requirements we discussed in the previous lessons. Our main requirements are high performance, availability, scalability, reliability, and security. Letâ s discuss them all one by one.

Performance

CDN achieves high performance by minimizing latency. Some of the key design decisions that minimize latency are as follows:

- Proxy servers usually serve content from the RAM.
- CDN proxy servers are placed near the users to provide faster access to content.
- A CDN can also be the provider of proxy servers located in the ISP or Internet exchange points (IXPs) to handle high traffic.
- The request routing system ensures that users are directed to the nearest proxy servers. Weâ ll have a detailed discussion on the request routing system in the next lesson.
- The proxy servers have long-tail content stored in nonvolatile storage systems like SSD or HDD. Serving from these resources results in a more negligible latency than weâ d see from serving content from origin servers.

```
'''
```

Long-tail

If we monitor the access frequency of our content over some time, we can see a trend where some content is read by many clients (for example, 20% of content asked by 80% of clients), and there is a long list of the content that very few clients ask for. Such an access distribution is called a long-tailed distribution.

'''

[Long-tail](./longtail.jpg)

- As was discussed previously, proxy servers can be implemented in layers where if one layer doesn't have the content, the request can be entertained by the next layer of proxy servers. For example, the edge proxy servers can request the parent proxy servers. Placing proxy servers at specific ISPs could be the best option when most traffic comes from those ISP regions.

Availability

A CDN can deal with massive traffic due to its distributed nature. A CDN ensures availability through its cached content that serves as a backup whenever the origin servers fail. Moreover, if one or more proxy servers in the CDN stop working, other operational proxy servers step in and continue to drive the web traffic. In addition, edge proxy servers can be made available through redundancy by replicating data to as many proxy servers as needed to avoid a single point of failure and to meet the request load. Finally, we can use a load balancer to distribute the users' requests to nearby active proxy servers.

Scalability

The design of CDN facilitates scalability in the following ways:

- It brings content closer to the user and removes the requirement of high bandwidth, thereby ensuring scalability.
- Horizontal scalability is possible by adding the number of reading replicas in the form of edge proxy servers.
- The limitations with horizontal scalability and storage capacity of an individual proxy server can be dealt with using the layered architecture of the proxy servers we described above.

Reliability and security

A CDN ensures no single failure point by carefully implementing maintenance cycles and integrating additional hardware and software when required. Apart from failures, the CDN handles massive traffic loads by equally distributing the load to the edge proxy servers. We can use scrubber servers to prevent DDoS attacks and securely host content. Moreover, we can use the heartbeat protocol to monitor the health of servers and omit faulty servers. Real-time applications also build their own specified CDNs to prevent content leakage problems and securely serve content to their end users.

Conclusion

Since its inception in the 1990s, the CDN has played a vital role in providing high availability and low-latency content delivery. Nowadays, CDNs are considered a key player in improving the overall performance of giant services.

Content Delivery Network (CDN)/In-depth Investigation of CDN Part 1/README.md

In-depth Investigation of CDN: Part 1

In this lesson, we'll go into the details of certain concepts, such as CDN models and multi-tier/layered CDN architecture, that we mentioned in the previous lessons. We'll also introduce some new concepts, including dynamic content caching optimization and various techniques to discover the nearby proxy servers in CDNs.

Content caching strategies in CDN

Identifying content to cache is important in delivering up-to-date and popular web content. To ensure timely updates, two classifications of CDNs are used to get the content from the origin servers.

Push CDN

Content gets sent automatically to the CDN proxy servers from the origin server in the push CD

model. The content delivery to the CDN proxy servers is the content provider's responsibility. Push CDN is appropriate for static content delivery, where the origin server decides which content to deliver to users using the CDN. The content is pushed to proxy servers in various locations according to the content's popularity. If the content is rapidly changing, the push model might struggle to keep up and will do redundant content pushes.

[Push content to PoPs](./push.jpg)

Pull CDN

A CDN pulls the unavailable data from origin servers when requested by a user. The proxy servers keep the files for a specified amount of time and then remove them from the cache if they're no longer requested to balance capacity and cost.

When users request web content in the pull CDN model, the CDN itself is responsible for pulling the requested content from the origin server and serving it to the users. Therefore, this type of CDN is more suited for serving dynamic content.

[Content pull from origin server to the CDN PoPs](./pull.jpg)

As stated, the push CDN is mostly used for serving static content. Since static content is served to a wide range of users for longer than dynamic content, the push CDN scheme maintains more replicas than the pull CDN, thus improving availability. On the other hand, the pull CDN is favored for frequently changing content and a high traffic load. Low storage consumption is one of the main benefits of the pull CDN.

'''

Note: Most content providers use both pull and push CDN caching approaches to get the benefits of both.

'''

Dynamic content caching optimization

Since dynamic content often changes, it's a good idea to cache it optimally. This section deals with the optimization of frequently changing content.

Certain dynamic content creation requires the execution of scripts that can be executed at proxy servers instead of running on the origin server. Dynamic data can be generated using various parameters, which can be beneficial if executed at the proxy servers. For example, we can generate dynamic content based on user location, time of day at a location, third-party APIs specific to a location (for instance, weather API), and so on. So, it's optimal to run scripts at proxy servers instead of the origin servers.

To reduce the communication between the origin server and proxy servers and storage requirements at proxy servers, it's useful to employ compression techniques as well. For example, Cloudflare uses Railgun to compress dynamic content.

Another popular approach for dynamic data compression is Edge Side Includes (ESI) markup language. Usually, a small portion of the web pages changes in a certain time. It means fetching a full web page on each small change contains a lot of redundant data. To resolve this performance penalty, ESI specifies where content was changed so that the rest of the web page content can be cached. It assembles dynamic content at the CDN edge server or client browser. ESI isn't standardized yet by the World Wide Web Consortium (W3C), but many CDN providers use it.

'''

Note: Dynamic Adaptive Streaming over HTTP (DASH) uses a manifest file with URIs of the video with different resolutions so that the client can fetch whatever is appropriate as per prevailing network and end node conditions. Netflix uses a proprietary DASH version with a Byte-range in the URL for further content request and delivery optimization.

'''

Multi-tier CDN architecture

The content provider sends the content to a large number of clients through a CDN. The task of

distributing data to all the CDN proxy servers simultaneously is challenging and burdens the origin server significantly. CDNs follow a tree-like structure to ease the data distribution process for the origin server. The edge proxy servers have some peer servers that belong to the same hierarchy. This set of servers receives data from the parent nodes in the tree, which eventually receive data from the origin servers. The data is copied from the origin server to the proxy servers by following different paths in the tree.

The tree structure for data distribution allows us to scale our system for increasing users by adding more server nodes to the tree. It also reduces the burden on the origin server for data distribution. A CDN typically has one or two tiers of proxy servers (caches). The following illustration shows the two tiers of proxy servers:

[Data distribution among CDN proxy servers](./distribution.jpg)

Whenever a new proxy server enters the tree of a CDN, it requests the control core, which maintains information on all the proxy servers in the CDN and provides initial content with the configuration data.

'''

The control core is a set of servers that manage the proxy servers within the PoP.

'''

Research shows that many contents have long-tail distribution. This means that, at some point, only a handful of content is very popular, and then we have a long tail of less popular content. Here, a multi-layer cache might be used to handle long-tail content.

[Many kinds of data exhibit the long-tailed phenomenon](./statistics.jpg)

'''

Question

What happens if a child or parent proxy server fails or if the origin server fails?

Answer

Each PoP contains a collection of CDN proxy servers. When any child proxy server stops working due to any failures, DNS can route clients to a different child-level proxy server. Each child proxy server knows many upper-layer parent servers, and if one fails, it can go to the other one. The origin server is a set of servers with a hot backup(s), and content is in replicated store. If any of the origin servers fail, other servers take the load.

'''

Now that we've seen a way to distribute content from the origin server to all the proxy servers of the CDN, we should also educate ourselves on how users can use these proxy servers to get the data more efficiently. We'll discuss how the nearest proxy server is chosen when clients make requests and how the CDN is located in the upcoming sections of this lesson.

Find the nearest proxy server to fetch the data

It's vital for the user to fetch data from the nearest proxy server because the CDN aims to reduce user-perceived latency by bringing the data close to the user. However, the question remains of how users worldwide request data from the nearest proxy server. The goal of this section is to answer that question.

Important factors that affect the proximity of the proxy server

There are two important factors that are relevant to finding the nearest proxy server to the user:

- Network distance between the user and the proxy server is crucial. This is a function of the following two things:

- The first is the length of the network path.
- The second is the capacity (bandwidth) limits along the network path.

The shortest network path with the highest capacity (bandwidth) is the nearest proxy server to the user in question. This path helps the user download content more quickly.

- Requests load refers to the load a proxy server handles at any point in time. If a set of proxy servers are overloaded, the request routing system should forward the request to a location with a lesser load. This action balances out the proxy server load and, consequently, reduces the response latency.

Let's look at the techniques that can be used to route users to the nearest proxy server.

DNS redirection

In a typical DNS resolution, we use a DNS system to get an IP against a human-readable name. However, the DNS can also return another URI (instead of an IP) to the client. Such a mechanism is called DNS redirect.

Content providers can use DNS redirect to send a client to a specific CDN. As an example, if the client tries to resolve a name that has the word "video" in it, the authoritative DNS server provides another URL (for example, `cdn.xyz.com`). The client does another DNS resolution and the CDN's authoritative DNS provides an IP address of an appropriate CDN proxy server to fetch the required content.

Depending on the location of the user, the response of the DNS can be different. Let's see the slides below to understand how DNS redirection works:

[DNS redistribution](./redistribution)

'''

Note: The nearest proxy server doesn't necessarily mean the one that's geographically the closest. It could be, but it's not only the geography that matters. Other factors like network distance, bandwidth, and traffic load already on that route also matter.

'''

There are two steps in the DNS redirection approach:

1. In the first step, it maps the clients to the appropriate network location.
2. In the second step, it distributes the load over the proxy servers in that location to balance the load among the proxy servers (see DNS and Load Balancers building blocks for more details on this).

DNS redirection takes both of these important factors—network distance and requests load—into consideration, and that reduces the latency towards a proxy server.

To shift a client from one machine in a cluster to another, the DNS replies at the second step are given with short TTLs so that the client repeats the resolution after a short while. DNS keeps delivering the content by routing requests to other active servers in case of hardware failures and network congestion. It does so by load balancing the traffic, using intelligent failover, and maintaining servers across many data centers, which achieves good reliability and performance.

Since the load at proxy servers changes over time, the content provider needs to make appropriate changes in the DNS to make the DNS redirection effective. Many CDN providers like Akamai use DNS redirection in their routing system.

Anycast

Anycast is a routing methodology in which all the edge servers located in multiple locations share the same single IP address. It employs the Border Gateway Protocol (BGP) to route client requests based on the Internet's natural network flow. A CDN provider can use the anycast mechanism so that clients are directed to the nearest proxy servers for content.

'''

BGP is a network-level protocol used by Internet edge routers to share routing and reachability information so that every node on the network, even if independent, is aware of the status of their closest network neighbors.

'''

Client multiplexing

Client multiplexing involves sending a client a list of candidate servers. The client then chooses one server from the list to send the request to. This approach is inefficient because the client lacks the overall information to choose the most suitable server for their request.

This may result in sending requests to an already-loaded server and experiencing higher access latency.

HTTP redirection

HTTP redirection is the simplest of all approaches. With this scheme, the client requests content from the origin server. The origin server responds with an HTTP protocol to redirect the user via a URL of the content.

Below is an example of an HTML snippet provided by Facebook. As is highlighted in line 8, the user is redirected to the CDN to download the logo of Facebook:

```
'''Html
<!â â The code below is taken from Facebook. -->
<div class="fb_content clearfix " id="content" role="main">
  <div>
    <div class="_8esj _95k9 _8esf _8opv _8f3m _8ilg _8icx _8op_ _95ka">
      <div class="_8esk">
        <div class="_8esl">
          <div class="_8ice">
            <!â â DNS redirection -->
            
          </div>
          <h2 class="_8eso">Facebook helps you connect and share with the people in your life.</h2>
        </div>
      </div>
    </div>
  </div>
</div>
'''
                                An example of HTML containing a redirection URL
'''
```

In the next lesson, we discuss the different details of content consistency and proxy server deployment in CDN.

Content Delivery Network (CDN)/In-depth Investigation of CDN Part 2/README.md

In-depth Investigation of CDN: Part 2

In this lesson, we learn how content consistency can be achieved using different consistency mechanisms. We also learn about where we should deploy the proxy servers and the difference between CDN as a service and specialized CDN.

Content consistency in CDN

Data in the proxy servers should be consistent with data in the origin servers. Thereâ s always a risk of users accessing stale data if the proxy servers donâ t remain consistent with the origin servers. Different consistency mechanisms can be used to ensure consistency of data, depending on the push or pull model.

Periodic polling

Using the pull model, proxy servers request the origin server periodically for updated data and change the content in the cache accordingly. When content changes infrequently, the polling approach consumes unnecessary bandwidth. Periodic polling uses time-to-refresh (TTR) to adjust the time period for requesting updated data from the origin servers.

Time-to-live (TTL)

Because of the TTR, the proxy servers may uselessly request the origin servers for updated data. A better approach that could be employed to reduce the frequency of refresh messages is the time-to-live (TTL) approach. In this approach, each object has a TTL attribute assigned to it by the origin server. The TTL defines the expiration time of the content. The proxy servers serve the same data version to the users until that content expires. Upon expiration, the proxy server checks for an update with the origin server. If the data is changed, it gets the updated data from the origin server and then responds to the userâ s requests with the updated data. Otherwise, it keeps the same data with an updated expiration time from the origin servers.

Leases

The origin server grants a lease to the data sent to a proxy server using this technique. The lease denotes the time interval for which the origin server agrees to notify the proxy server if there's any change in the data. The proxy server must send a message requesting a lease renewal after the expiration of the lease. The lease method helps to reduce the number of messages exchanged between the proxy and origin server. Additionally, the lease duration can be optimized dynamically according to the observed load on the proxy servers. This technique is referred to as an adaptive lease.

In the following section, we discuss where to place the CDN proxy server to transmit data effectively.

Deployment

We have to be clear with the answers to the following questions before we install the CDN facility:

- What are the best locations to install proxy servers to maximally utilize CDN technology?
- How many CDN proxy servers should we install?

Placement of CDN proxy servers

The CDN proxy servers must be placed at network locations with good connectivity. See the options below:

- On-premises represents a smaller data center that could be placed near major IXPs.
- Off-premises represents placing CDN proxy servers in ISP's networks.

Today, it might be feasible to keep a large portion of a movie's data in a CDN infrastructure that's housed inside an ISP. Still, for services like YouTube, data is so large and ever-expanding that it's challenging to decide what we should put near a user. Google uses split to reduce user-perceived delays by keeping persistent connections with huge TCP windows from the IXP-level infrastructure to their primary data centers. The client's TCP requests terminate at the IXP-level infrastructure and are then forwarded on already established, low latency TCP connections.

Doing this substantially reduces client-perceived latency, which is due to the avoidance of the initial three-way handshake of TCP connection and slow-start stages to a host far away (had the client wanted to go to the primary data centers of Google). A round-trip delay to IXP is often very low. Therefore, three-way handshakes and slow starts at that level are negligible. Predictive push is a significant research field to decide what to push near the customers.

We can use measurements to facilitate the decision of proxy server placement. One such tool is ProxyTeller to decide where to place the proxy server and how many proxy servers are required to achieve high performance. ProxyTeller uses hit ratio, network bandwidth, and client-response time (latency) as performance parameters to decide the placement of proxy servers. Other greedy, random, and hotspot algorithms are also used for proxy server placements.

A proxy placement tool for content delivery under performance constraints. Source: Triantafyllou P. and Aekaterinidis, I., 2003, December. Proceedings of the Fourth International Conference on Web Information Systems Engineering, 2003. WISE 2003. (pp. 62-71). IEEE.

Note: Akamai and Netflix popularized the idea of keeping their CDN proxy servers inside the client's ISPs. For many clients of Akamai, content is just one network hop away. On the other hand, Google also has its private CDN infrastructure but relies more on its servers near IXPs. One reason for this could be the sheer amount of data and the change patterns.

Question

What benefits could an ISP get by placing the CDN proxy servers inside their network?

Answer

- A CDN node cuts down the amount of external bandwidth that ISPs need and pay for. The SLA defined by ISPs with the proxy server providers may benefit the ISPs economically.
- A CDN node improves responsiveness for the ISP's customers, which makes the ISP's customers happier. This gives them a competitive advantage over ISPs that don't have a CDN node.
- Bringing content closer to the users reduces data on the Internet core.

CDN as a service

Most companies don't build their own CDN. Instead, they use the services of a CDN provider, such as Akamai, Cloudflare, Fastly, and so on, to deliver their content. Similarly, players like AWS make it possible for anyone to use a global CDN facility.

The companies sign a contract with the CDN service provider and deliver their content to the CDN, thereby allowing the CDN to distribute the content to the end users. A public CDN raises the following concerns for content providers:

- The content provider can't do anything if the public CDN is down.
- If a public CDN doesn't have any proxy servers located in the region or country where some website traffic comes from, then those specific customers are out of luck. In such cases, the content providers have to buy CDN services from other CDN providers or deploy and use their own private CDN.
- It's possible that some domains or IP addresses of CDN providers are blocked or restricted in some countries because they might be delivering content that's banned in those countries.

Note: Some companies make their own CDN instead of using the services of CDN providers. For example, Netflix has its own purpose-built CDN called Open Connect.

Specialized CDN

We've discussed that many companies use CDN as a service, but there are cases where companies build their own CDN. A number of reasons factor into this decision. One is the cost of a commercial CDN service. A specialized CDN consists of points of presence (PoPs) that only serve content for their own company. These PoPs can be caching servers, reverse proxies, or application delivery controllers. Although a specialized CDN has high costs at its first setup, the costs eventually decrease with time. In essence, it's a buy versus build decision.

The specialized CDN's PoPs consist of many proxy servers to serve petabytes of content. A private CDN can be used in coexistence with a public CDN. In case the capacity of a private CDN isn't enough or there's a failure that leads to capacity reduction, the public CDN is used as backup. Netflix's Open Connect Appliance (OCA) is an example of a CDN that's specialized in video delivery.

Netflix's OCA servers don't store user data. Instead, they fulfill the following tasks:

- They report their status, health, learned routes, and details of cached content to the Open Connect control plane that resides in AWS (Amazon Web Services).
- They serve the cached content that's requested by the user.

[Netflix's Open Connect Appliances](./netflix.jpg)

All the deployed OCAs situated in IXP or embedded in the ISP network are monitored by the Open Connect operation team.

Why Netflix built its CDN

As Netflix became more popular, it decided to build and manage its own CDN for the following

reasons:

- The CDN service providers were scuffling to expand their infrastructure due to the rapid growth in customer demand for video streaming on Netflix.
- With the increasing volume of streaming videos, the expense of using CDN services increased.
- Video streaming is the main business and a primary revenue source for Netflix. So, protecting the data of all the videos on the platform is critical. Netflix's OCA manages potential data leakage risks in a better way.
- To provide optimal streaming media delivery to customers, Netflix needed to maximize its control over the user's video player, the network between the user, and the Netflix servers.
- Netflix's OCA can use custom HTTP modules and TCP connection algorithms to detect network problems quickly and troubleshoot any issues in their CDN network.
- Netflix wanted to keep popular content for a long time. This wasn't entirely possible while operating with a public CDN due to the high costs that would be incurred to keep and maintain it.

'''

Note: Netflix is able to achieve a hit ratio close to 95% using OCA.

'''

We'll evaluate our proposed design in the next lesson.

Content Delivery Network (CDN)/Introduction to a CDN/README.md

Introduction to a CDN

Proposed solution

The solution to all the problems discussed in the previous lesson is the content delivery network (CDN). A CDN is a group of geographically distributed proxy servers. A proxy server is an intermediate server between a client and the origin server. The proxy servers are placed on the network edge. As the network edge is close to the end users, the placement of proxy servers helps quickly deliver the content to the end users by reducing latency and saving bandwidth. A CDN has added intelligence on top of being a simple proxy server.

'''

The network edge is the zone where a device or local network interfaces with the Internet.

'''

We can bring data close to the user by placing a small data center near the user and storing copies of the data there. CDN mainly stores two types of data: static and dynamic. A CDN primarily targets propagation delay by bringing the data closer to its users. CDN providers make the extra effort to have sufficient bandwidth available through the path and bring data closer to the users (possibly within their ISP). They also try to reduce transmission and queuing delays because the ISP presumably has more bandwidth available within the autonomous system.

'''

static data: This type of data doesn't change frequently and remains on the servers for a long period.

dynamic data: This type of data changes most frequently—for example, newsletters, ads, live videos, and so on.

'''

Let's look at the different ways CDN solves the discussed problems:

- High latency: CDN brings the content closer to end users. Therefore, it reduces the physical distance and latency.

- Data-intensive applications: Since the path to the data includes only the ISP and the nearby CDN components, there's no issue in serving a large number of users through a few CDN components in a specific area. As shown below, the origin data center will have to provide the

data to local CDN components only once, whereas local CDN components can provide data to different users individually. No user will have to download their own copy of data from the origin servers.

Note: Various streaming protocols are used to deliver dynamic content by the CDN providers. For example, CDNs use the Real-time Messaging Protocol (RTMP), HTTP Live Streaming (HLS), Real-time Streaming Protocol (RTSP), and many more to deliver dynamic content.

- Scarcity of data center resources: A CDN is used to serve popular content. Due to this reason, most of the traffic is handled at the CDN instead of the origin servers. So, different local or distributed CDN components share the load on origin servers.

Note: A few well-known CDN providers are Akamai, StackPath, Cloudflare, Rackspace, Amazon CloudFront, and Google Cloud CDN.

Question

Does a CDN cache all content from the origin server?

Answer

Not likely. A CDN caches a considerable portion of the content depending on its capabilities, and it mostly caches the static content.

static content: The content that isn't frequently changed is known as static content.

It also depends on the size of the content. For example, it might be possible for Netflix to store more than 90% of its movies in the CDN, while this might not be feasible for a service like YouTube due to the humongous volume of content.

Requirements

Let's look at the functional and non-functional requirements that we expect from a CDN.

Functional requirements

- Retrieve: Depending upon the type of CDN models--push or pull, a CDN should be able to retrieve content from the origin servers. We'll cover CDN models in the coming lesson.
- Request: Content delivery from the proxy server is made upon the user's request. CDN proxy servers should be able to respond to each user's request in this regard.
- Deliver: In the case of the push model, the origin servers should be able to send the content to the CDN proxy servers.
- Search: The CDN should be able to execute a search against a user query for cached or otherwise stored content within the CDN infrastructure.
- Update: In most cases, content comes from the origin server, but if we run script in CDN, the CDN should be able to update the content within peer CDN proxy servers in a PoP.
- Delete: Depending upon the type of content (static or dynamic), it should be possible to delete cached entries from the CDN servers after a certain period.

[Functional requirements of a CDN](./req.jpg)

A Point of Presence (PoP) is a physical place that allows two or more networks or devices to communicate with each other. Typically, each CDN PoP has a large number of cache servers.

Non-functional requirements

- Performance: Minimizing latency is one of the core missions of a CDN. The proposed design should have minimum possible latency.
- Availability: CDNs are expected to be available at all times because of their effectiveness. Availability includes protection against attacks like DDoS.
- Scalability: An increasing number of users will request content from CDNs. Our proposed CDN design should be able to scale horizontally as the requirements increase.

- Reliability and security: Our CDN design should ensure no single point of failure. Apart from failures, the designed CDN must reliably handle massive traffic loads. Furthermore, CDNs should provide protection to hosted content from various attacks.

'''

In DDoS attacks, malicious agents overwhelm the origin or application server by sending a massive number of requests.

'''

[Non-functional requirements of CDN](./nfreq.jpg)

Building blocks we will use

The design of a CDN utilizes the following building blocks:

[The building blocks used in CDN design](./bb.jpg)

- DNS is the service that maps human-friendly CDN domain names to machine-readable IP addresses. This IP address will take the users to the specified proxy server.

- Load balancers distribute millions of requests among the operational proxy servers.

In the next lesson, we'll discuss the design of the CDN.

Content Delivery Network (CDN)/Quiz on CDN's Design/README.md

Quiz on CDN's Design

'''

1. Which CDN approach will be the most suitable to use when most of the web content is static?

A)

Pull CDN

B)

Push CDN

'''

'''

2. (Select all that apply.) What are the limitations of using a public CDN?

A)

An additional point of failure

B)

Risk of a data breach

C)

Loss of control

D)

An additional DNS lookup

E)

Increased latency

'''

'''

3. (Select all that apply.) What are the main reasons for building a specialized CDN?

A)

It optimizes the content delivery.

B)

It helps us avoid the increasing cost of CDN providers.

C)

It prevents ISP intervention.

D)

It reduces the hit ratio.

'''

'''

4. Suppose youâ re asked to scale the Quora system to reduce the burden of data distribution the origin server. Which technique will you use to do this?

A)

A single-tier CDN

B)

A multi-tier CDN

C)

A public CDN

D)

A multi CDN

'''

'''

5. (Select all that apply.) Why do most websites use a CDN?

A)

A CDN ensures improved scalability.

B)

A CDN offers protection against DDoS attacks.

C)

A CDN creates a backup for all the evicted content.

D)

A CDN reduces the load on the origin server.

E)

A CDN guarantees better performance.

'''

'''

6. Which metric pair is the most suitable for finding the nearest proxy servers?

A)

Traffic load, caching capacity

B)

Request load, network distance

C)

Location, bandwidth

D)

Caching capacity, network distance

'''

Answers

'''

1. Which CDN approach will be the most suitable to use when most of the web content is static?

(B)

A)

Pull CDN

Explanation

Use Pull CDN where web content changes frequently.

Correct Answer

B)

Push CDN

Explanation

Use Push CDN where web content changes infrequently and caches in the edge proxy servers for a long period.

'''

'''

2. (Select all that apply.) What are the limitations of using a public CDN? (A)(B)(C)(D)

A)

An additional point of failure

Explanation

The content hosted on CDN will not be available.

B)

Risk of a data breach

Explanation

There are always risks when third parties manage the data.

C)

Loss of control

Explanation

When content publishes to the proxy servers, the CDN providers control it.

D)

An additional DNS lookup

Explanation

CDN network layer increases DNS lookup.

E)

Increased latency

Explanation

CDN is used to reduce the latency between end-users and the content. Therefore, this option cannot be a limitation.

'''

'''

3. (Select all that apply.) What are the main reasons for building a specialized CDN? (A)(B)

A)

It optimizes the content delivery.

Explanation

Specified CDN is dedicated to a particular content provider which optimizes the delivery of the requested content.

B)

It helps us avoid the increasing cost of CDN providers.

Explanation

The cost of CDN providers increases when we increase the cache size or add more components.

C)

It prevents ISP intervention.

Explanation

Each request comes through ISP using encrypted connections. So, ISPs can't intervene.

D)

It reduces the hit ratio.

Explanation

Specialized CDN will increase the hit ratio instead of reducing it because it is dedicated to particular content.

'''

'''

4. Suppose you're asked to scale the Quora system to reduce the burden of data distribution on the origin server. Which technique will you use to do this? (B)

A)

A single-tier CDN

Explanation

Single tier does not reduce the burden because an origin server has to distribute the content to all CDN servers.

Correct Answer

B)

A multi-tier CDN

Explanation

Using this approach, data distribution is divided into the CDN layers like an origin to parent CDN servers and parent to the following layer CDN servers.

C)

A public CDN

Explanation

The origin server will need to provide data to CDN, be it public or private.

D)

A multi CDN

Explanation

A multi-CDN combines multiple CDNs from different CDN providers into a single network. Often such a strategy is complex and expensive.

'''

'''

5. (Select all that apply.) Why do most websites use a CDN? (A)(B)(D)(E)

A)

A CDN ensures improved scalability.

Explanation

Horizontal scaling is not a problem for the CDN.

B)

A CDN offers protection against DDoS attacks.

Explanation

Scrubber servers are used to prevent DDoS attacks in the CDN network.

C)

A CDN creates a backup for all the evicted content.

Explanation

Content is deleted from all storage of the specified CDN servers for consistency in the eviction process.

D)

A CDN reduces the load on the origin server.

Explanation

CDN reduces the burden from the origin servers like data distribution, handling massive traffic, etc.

E)

A CDN guarantees better performance.

Explanation

CDN is placed near the end-user, so it increases the performance by reducing latency and speeding up the request/response process.

'''

'''

6. Which metric pair is the most suitable for finding the nearest proxy servers? (B)

A)

Traffic load, caching capacity

Explanation

Caching capacity is not a relevant factor, but Traffic load is.

Correct Answer

B)

Request load, network distance

Explanation

This is the best suitable pair to find the nearest proxy server.

C)

Location, bandwidth

Explanation

Location does not describe any details, but the distance between two locations does.

D)

Caching capacity, network distance

Explanation

Caching capacity is not a relevant factor, but network distance is.

'''

Content Delivery Network (CDN)/System Design The Content Delivery Network (CDN)/README.md

System Design: The Content Delivery Network (CDN)

Problem statement

Let's start with a question: If millions of users worldwide use our data-intensive applications, and our service is deployed in a single data center to serve the users' requests, what possible problems can arise?

The following problems can arise:

- High latency: The user-perceived latency will be high due to the physical distance from the serving data center. User-perceived latency has many components, such as transmission delays (a function of available bandwidth), propagation delays (a function of distance), queuing delays (a function of network congestion), and nodal processing delays. Therefore, data transmission over a large distance results in higher latency. Real-time applications require a latency below 200 milliseconds (ms) in general. For the Voice over Internet Protocol (VoIP), latency should not be more than 150 ms, whereas video streaming applications cannot tolerate a latency above a few seconds.

'''

Note: According to one of the readings taken on December 21, 2021, the average latency from US East (N. Virginia) to US West (N. California) was 62.9 ms. Across continents—for example, from the US East (N. Virginia) to Africa (Cape Town)—was 225.63 ms. This is two-way latency, known as round-trip latency.

'''

[Origin data center entertaining users' requests across the globe](./latency.jpg)

- Data-intensive applications: Data-intensive applications require transferring large traffic. Over a longer distance, this could be a problem due to the network path stretching through different kinds of ISPs. Because of some smaller Path message transmission unit (MTU) links, the throughput of applications on the network might be reduced. Similarly, different portions of the network path might have different congestion characteristics. The problem multiplies as the number of users grows because the origin servers will have to provide the data individually to each user. That is, the primary data center will need to send out a lot of redundant data when multiple clients ask for it. However, applications that use streaming services are both data-intensive and dynamic in nature.

'''

A Path MTU refers to the smallest data unit that can traverse from source to destination without the need for splitting.

'''

'''

Note: According to a survey, 78% of the United States consumers use streaming services, which is an increase of 25% in five years.

'''

- Scarcity of data center resources: Important data center resources like computational capacity and bandwidth become a limitation when the number of users of a service increases significantly. Services engaging millions of users simultaneously need scaling. Even if scaling is achieved in a single data center, it can still suffer from becoming a single point of failure when the data center goes offline due to natural calamity or connectivity issues with the Internet.

[User growth over the years for Facebook and YouTube applications](./users.jpg)

'''

Note: According to one study, YouTube, Netflix, and Amazon Prime collectively generated 80% of

Internet traffic in 2020. Circa 2016, the CDN provider Akamai served 15% to 30% of web traffic (about 30 terabits per second). For 90% of Internet users, Akamai was just one hop away. Therefore, we have strong reasons to optimize the delivery and consumption of this data without making the Internet core a bottleneck.

'''

How will we design a CDN?

We've divided the design of CDN into six lessons:

- Introduction to a CDN: We'll provide a thorough introduction to CDNs and identify the functional and non-functional requirements.
 - Design of a CDN: We'll explain how to design the CDN. We'll also briefly describe the AP design.
 - In-depth Investigation of CDN: Part 1: This lesson explains caching strategies and CDN architecture. Also, we'll discuss various approaches to finding the nearest proxy server.
 - In-depth Investigation of CDN: Part 2: We'll discuss how to make content consistent in a C and the deployment of proxy servers. We'll also cover the custom and specialized CDN in detail.
 - Evaluation of CDN: This lesson will provide an evaluation of our proposed design.
 - Quiz on CDN System Design: We'll reinforce major concepts of CDN design with a quiz.
- Let's think about the solution to the discussed issues in the next lesson.

Databases/Data Partitioning/README.md

Data Partitioning

Why do we partition data?

Data is an asset for any organization. Increasing data and concurrent read/write traffic to the data put scalability pressure on traditional databases. As a result, the latency and throughput are affected. Traditional databases are attractive due to their properties such as range queries, secondary indices, and transactions with the ACID properties.

At some point, a single node-based database isn't enough to tackle the load. We might need to distribute the data over many nodes but still export all the nice properties of relational databases. In practice, it has proved challenging to provide single-node database-like properties over a distributed database.

One solution is to move data to a NoSQL-like system. However, the historical codebase and its close cohesion with traditional databases make it an expensive problem to tackle.

Organizations might scale traditional databases by using a third-party solution. But often, integrating a third-party solution has its complexities. More importantly, there are abundant opportunities to optimize for the specific problem at hand and get much better performance than a general-purpose solution.

Data partitioning (or sharding) enables us to use multiple nodes where each node manages some part of the whole data. To handle increasing query rates and data amounts, we strive for balanced partitions and balanced read/write load.

We'll discuss different ways to partition data, related challenges, and their solutions in this lesson.

'''

A range query is a common database operation that retrieves all records where some value is between an upper and lower boundary.

A secondary index is a way to efficiently access records in a database by means of some piece of information other than the primary key.

A transaction is a single logical unit of work that accesses and possibly modifies the content of a database.

'''

Sharding

To divide load among multiple nodes, we need to partition the data by a phenomenon known as partitioning or sharding. In this approach, we split a large dataset into smaller chunks of data stored at different nodes on our network.

The partitioning must be balanced so that each partition receives about the same amount of data. If partitioning is unbalanced, the majority of queries will fall into a few partitions. Partitions that are heavily loaded will create a system bottleneck. The efficacy of partitioning will be harmed because a significant portion of data retrieval queries will be sent to the nodes that carry the highly congested partitions. Such partitions are known as hotspots. Generally, we use the following two ways to shard the data:

'''

Vertical sharding

Horizontal sharding

'''

Vertical sharding

We can put different tables in various database instances, which might be running on a different physical server. We might break a table into multiple tables so that some columns are in one table while the rest are in the other. We should be careful if there are joins between multiple tables. We may like to keep such tables together on one shard.

Often, vertical sharding is used to increase the speed of data retrieval from a table consisting of columns with very wide text or a binary large object (blob). In this case, the column with large text or a blob is split into a different table.

As shown in the figure a couple paragraphs below, the Employee table is divided into two tables: a reduced Employee table and an EmployeePicture table. The EmployeePicture table has just two columns, EmployeeID and Picture, separated from the original table. Moreover, the primary key EmployeeID of the Employee table is added in both partitioned tables. This makes the data read and write easier, and the reconstruction of the table is performed efficiently.

Vertical sharding has its intricacies and is more amenable to manual partitioning, where stakeholders carefully decide how to partition data. In comparison, horizontal sharding is suitable to automate even under dynamic conditions.

[Vertical partitioning](./vertical.jpg)

Horizontal sharding

At times, some tables in the databases become too big and affect read/write latency. Horizontal sharding or partitioning is used to divide a table into multiple tables by splitting data row-wise, as shown in the figure in the next section. Each partition of the original table distributed over database servers is called a shard. Usually, there are two strategies available:

'''

Key-range based sharding

Hash based sharding

'''

Key-range based sharding

In the key-range based sharding, each partition is assigned a continuous range of keys.

In the following figure, horizontal partitioning on the Invoice table is performed using the key-range based sharding with Customer_Id as the partition key. The two different colored tables represent the partitions.

[Horizontal partitioning](./horizontal.jpg)

Sometimes, a database consists of multiple tables bound by foreign key relationships. In such case, the horizontal partition is performed using the same partition key on all tables in a relation. Tables (or subtables) that belong to the same partition key are distributed to one database shard. The following figure shows that several tables with the same partition key are

placed in a single database shard:

[Horizontal partitioning on a set of tables](./horizontal_tables.jpg)

The basic design techniques used in multi-table sharding are as follows:

- There's a partition key in the Customer mapping table. This table resides on each shard and stores the partition keys used in the shard. Applications create a mapping logic between the partition keys and database shards by reading this table from all shards to make the mapping efficient. Sometimes, applications use advanced algorithms to determine the location of a partition key belonging to a specific shard.
- The partition key column, Customer_Id, is replicated in all other tables as a data isolation point. It has a trade-off between an impact on increased storage and locating the desired shards efficiently. Apart from this, it's helpful for data and workload distribution to different database shards. The data routing logic uses the partition key at the application tier to map queries specified for a database shard.
- Primary keys are unique across all database shards to avoid key collision during data migration among shards and the merging of data in the online analytical processing (OLAP) environment.
- The column Creation_date serves as the data consistency point, with an assumption that the clocks of all nodes are synchronized. This column is used as a criterion for merging data from all database shards into the global view when essential.

'''

Online analytical processing, or OLAP, is an approach to answer multi-dimensional analytical (MDA) queries swiftly in computing. MDA queries are frequently issued against a fact database with predicates on (categorical or ordinal) dimensions and aggregations on one or more measures. Credits: Wikipedia

'''

Advantages

'''

Using this method, the range-query-based scheme is easy to implement.

Range queries can be performed using the partitioning keys, and those can be kept in partitioned in sorted order.

'''

Disadvantages

'''

Range queries can't be performed using keys other than the partitioning key.

If keys aren't selected properly, some nodes may have to store more data due to an uneven distribution of the traffic.

'''

Hash-based sharding

Hash-based sharding uses a hash-like function on an attribute, and it produces different values based on which attribute the partitioning is performed. The main concept is to use a hash function on the key to get a hash value and then mod by the number of partitions. Once we've found an appropriate hash function for keys, we may give each partition a range of hashes (rather than a range of keys). Any key whose hash occurs inside that range will be kept in that partition.

In the illustration below, we use a hash function of Value mod = n. The n is the number of nodes, which is four. We allocate keys to nodes by checking the mod for each key. Keys with a mod value of 2 are allocated to node 2. Keys with a mod value of 1 are allocated to node 1. Keys with a mod value of 3 are allocated to node 3. Because there's no key with a mod value

0, node 0 is left vacant.

[Hash-based sharding](./hash.jpg)

Advantages

Keys are uniformly distributed across the nodes.

Disadvantages

We can't perform range queries with this technique. Keys will be spread over all partitions.
'''

Note: How many shards per database?

Empirically, we can determine how much each node can serve with acceptable performance. It can help us find out the maximum amount of data that we would like to keep on any one node. For example, if we find out that we can put a maximum of 50 GB of data on one node, we have the following:

Database size = 10 TB

Size of a single shard = 50 GB

Number of shards the database should be distributed in $10 \text{ TB} / 50 \text{ GB} = 200$ shards
'''

Consistent hashing

Consistent hashing assigns each server or item in a distributed hash table a place on an abstract circle, called a ring, irrespective of the number of servers in the table. This permits servers and objects to scale without compromising the system's overall performance.

Advantages of consistent hashing

It's easy to scale horizontally.

It increases the throughput and improves the latency of the application.

Disadvantages of consistent hashing

Randomly assigning nodes in the ring may cause non-uniform distribution.

Rebalance the partitions

Query load can be imbalanced across the nodes due to many reasons, including the following:

'''

The distribution of the data isn't equal.

There's too much load on a single partition.

There's an increase in the query traffic, and we need to add more nodes to keep up.

'''

We can apply the following strategies to rebalance partitions.

Avoid hash mod n

Usually, we avoid the hash of a key for partitioning (we used such a scheme to explain the concept of hashing in simple terms earlier). The problem with the addition or removal of nodes in the case of hash mod n

hashmodn is that every node's partition number changes and a lot of data moves. For example, assume we have $\text{hash}(\text{key}) = 1235$. If we have five nodes at the start, the key will start on node 0 ($1235 \bmod 5 = 0$). Now, if a new node is added, the key would have to be moved to node 6 ($1235 \bmod 6 = 5$), and so on. This moving of keys from one node to another makes rebalancing costly.

Fixed number of partitions

In this approach, the number of partitions to be created is fixed at the time when we set our database up. We create a higher number of partitions than the nodes and assign these partitions to nodes. So, when a new node is added to the system, it can take a few partitions from the existing nodes until the partitions are equally divided.

There's a downside to this approach. The size of each partition grows with the total amount of data in the cluster since all the partitions contain a small part of the total data. If a partition is very small, it will result in too much overhead because we may have to make a

large number of small-sized partitions, each costing us some overhead. If the partition is very large, rebalancing the nodes and recovering from node failures will be expensive. It's very important to choose the right number of partitions. A fixed number of partitions is used in Elasticsearch, Riak, and many more.

Dynamic partitioning

In this approach, when the size of a partition reaches the threshold, it's split equally into two partitions. One of the two split partitions is assigned to one node and the other one to another node. In this way, the load is divided equally. The number of partitions adapts to the overall data amount, which is an advantage of dynamic partitioning.

However, there's a downside to this approach. It's difficult to apply dynamic rebalancing while serving the reads and writes. This approach is used in HBase and MongoDB.

Partition proportionally to nodes

In this approach, the number of partitions is proportionate to the number of nodes, which means every node has fixed partitions. In earlier approaches, the number of partitions was dependent on the size of the dataset. That isn't the case here. While the number of nodes remains constant, the size of each partition rises according to the dataset size. However, as the number of nodes increases, the partitions shrink. When a new node enters the network, it splits a certain number of current partitions at random, then takes one half of the split and leaves the other half alone. This can result in an unfair split. This approach is used by Cassandra and Ketama.

'''

Question

Who performs the rebalancing? Is it automatic or manual?

Answer

There are two ways to perform rebalancing: automatic and manual. In automatic rebalancing, there's no administrator. The system determines when to perform the partitions and when to move data from one node to another.

In manual rebalancing, the administrator determines when and how to perform the partitioning. Organizations perform rebalancing according to their needs. Some use automatic rebalancing, and some use manual.

'''

Partitioning and secondary indexes

We've discussed key-value data model partitioning schemes in which the records are retrieved with primary keys. But what if we have to access the records through secondary indexes? Secondary indexes are the records that aren't identified by primary keys but are just a way of searching for some value. For example, the above illustration of horizontal partitioning contains the customer table, searching for all customers with the same creation year.

We can partition with secondary indexes in the following ways.

Partition secondary indexes by document

Each partition is fully independent in this indexing approach. Each partition has its secondary indexes covering just the documents in that partition. It's unconcerned with the data held in other partitions. If we want to write anything to our database, we need to handle that partition only containing the document ID we're writing. It's also known as the local index. In the illustration below, there are three partitions, each having its own identity and data. If we want to get all the customer IDs with the name John, we have to request from all partitions.

However, this type of querying on secondary indexes can be expensive. As a result of being restricted by the latency of a poor-performing partition, read query latencies may increase.

[Partitioning secondary indexes by document](./partition1.jpg)

Partition secondary indexes by the term

Instead of creating a secondary index for each partition (a local index), we can make a global index for secondary terms that encompasses data from all partitions.

In the illustration below, we create indexes on names (the term on which we're partitioning).

and store all the indexes for names on separated nodes. To get the cust_id of all the customers named John, we must determine where our term index is located. The index 0 contains all the customers with names starting with 'A' to 'M'. The index 1 includes all the customers names beginning with 'N' to 'Z'. Because John lies in index 0, we fetch a list of customers with the name John from index 0.

Partitioning secondary indexes by the term is more read-efficient than partitioning secondary indexes by the document. This is because it only accesses the partition that contains the term. However, a single write in this approach affects multiple partitions, making the method write-intensive and complex.

[Partitioning secondary indexes by term](./partition2.jpg)

Request routing

We've learned how to partition our data. However, one question arises here: How does a client know which node to connect to while making a request? The allocation of partitions to nodes varies after rebalancing. If we want to read a specific key, how do we know which IP address we need to connect to read?

This problem is also known as service discovery. Following are a few approaches to this problem:

'''

Allow the clients to request any node in the network. If that node doesn't contain the requested data, it forwards that request to the node that does contain the related data.

The second approach contains a routing tier. All the requests are first forwarded to the routing tier, and it determines which node to connect to fulfill the request.

The clients already have the information related to partitioning and which partition is connected to which node. So, they can directly contact the node that contains the data they need.

'''

In all of these approaches, the main challenge is to determine how these components know about updates in the partitioning of the nodes.

ZooKeeper

To track changes in the cluster, many distributed data systems need a separate management server like ZooKeeper. Zookeeper keeps track of all the mappings in the network, and each node connects to ZooKeeper for the information. Whenever there's a change in the partitioning, or a node is added or removed, ZooKeeper gets updated and notifies the routing tier about the change. HBase, Kafka and SolrCloud use ZooKeeper.

Conclusion

For all current distributed systems, partitioning has become the standard protocol. Because systems contain increasing amounts of data, partitioning the data makes sense since it speeds up both writes and reads. It increases the system's availability, scalability, and performance.

Databases/Data Replication/README.md

Data Replication

Data is an asset for an organization because it drives the whole business. Data provides critical business insights into what's important and what needs to be changed. Organizations also need to securely save and serve their clients' data on demand. Timely access to the required data under varying conditions (increasing reads and writes, disks and node failures, network and power outages, and so on) is required to successfully run an online business.

'''

We need the following characteristics from our data store:

Availability under faults (failure of some disk, nodes, and network and power outages).

Scalability (with increasing reads, writes, and other operations).

Performance (low latency and high throughput for the clients).

'''

Itâs challenging, or even impossible, to achieve the above characteristics on a single node.

Replication

Replication refers to keeping multiple copies of the data at various nodes (preferably geographically distributed) to achieve availability, scalability, and performance. In this lesson, we assume that a single node is enough to hold our entire data. We wonât use this assumption while discussing the partitioning of data in multiple nodes. Often, the concepts of replication and partitioning go together.

However, with many benefits, like availability, replication comes with its complexities. Replication is relatively simple if the replicated data doesnât require frequent changes. The main problem in replication arises when we have to maintain changes in the replicated data over time.

Additional complexities that could arise due to replication are as follows:

'''

How do we keep multiple copies of data consistent with each other?

How do we deal with failed replica nodes?

Should we replicate synchronously or asynchronously?

|

+How do we deal with replication lag in case of asynchronous replication?

How do we handle concurrent writes?

What consistency model needs to be exposed to the end programmers?

'''

Weâll explore the answer to these questions in this lesson.

[Replication in action](./replicas.jpg)

Before we explain the different types of replication, letâs understand the synchronous and asynchronous approaches of replication.

Synchronous versus asynchronous replication

There are two ways to disseminate changes to the replica nodes:

'''

Synchronous replication

Asynchronous replication

'''

In synchronous replication, the primary node waits for acknowledgments from secondary nodes about updating the data. After receiving acknowledgment from all secondary nodes, the primary node reports success to the client. Whereas in asynchronous replication, the primary node doesnât wait for the acknowledgment from the secondary nodes and reports success to the client after updating itself.

The advantage of synchronous replication is that all the secondary nodes are completely up to date with the primary node. However, thereâs a disadvantage to this approach. If one of the secondary nodes doesnât acknowledge due to failure or fault in the network, the primary node would be unable to acknowledge the client until it receives the successful acknowledgment from the crashed node. This causes high latency in the response from the primary node to the client.

On the other hand, the advantage of asynchronous replication is that the primary node can continue its work even if all the secondary nodes are down. However, if the primary node fails, the writes that werenât copied to the secondary nodes will be lost.

The above paragraph explains a trade-off between data consistency and availability when different components of the system can fail.

[Synchronous versus asynchronous replication](./sync_async.jpg)

Data replication models

Now, let's discuss various mechanisms of data replication. In this section, we'll discuss following models along with their strengths and weaknesses:

Single leader or primary-secondary replication

Multi-leader replication

Peer-to-peer or leaderless replication

Single leader/primary-secondary replication

In primary-secondary replication, data is replicated across multiple nodes. One node is designated as the primary. It's responsible for processing any writes to data stored on the cluster. It also sends all the writes to the secondary nodes and keeps them in sync.

Primary-secondary replication is appropriate when our workload is read-heavy. To better scale with increasing readers, we can add more followers and distribute the read load across the available followers. However, replicating data to many followers can make a primary bottleneck. Additionally, primary-secondary replication is inappropriate if our workload is write-heavy.

Another advantage of primary-secondary replication is that it's read resilient. Secondary nodes can still handle read requests in case of primary node failure. Therefore, it's a helpful approach for read-intensive applications.

Replication via this approach comes with inconsistency if we use asynchronous replication. Clients reading from different replicas may see inconsistent data in the case of failure of the primary node that couldn't propagate updated data to the secondary nodes. So, if the primary node fails, any missed updates not passed on to the secondary nodes can be lost.

[Primary-secondary data replication model where data is replicated from primary to secondary](./1primary.jpg)

Question

What happens when the primary node fails?

Answer

In case of failure of the primary node, a secondary node can be appointed as a primary node, which speeds up the process of recovering the initial primary node. There are two approaches to select the new primary node: manual and automatic.

In a manual approach, an operator decides which node should be the primary node and notifies all secondary nodes.

In an automatic approach, when secondary nodes find out that the primary node has failed, they appoint the new primary node by conducting an election known as a leader election. A leader is typically assigned based on a factor, such as choosing the processor with the highest identifier.

Primary-secondary replication methods

There are many different replication methods in primary-secondary replication:

Statement-based replication

Write-ahead log (WAL) shipping

Logical (row-based) log replication

Let's discuss each of them in detail.

Statement-based replication

In the statement-based replication approach, the primary node saves all statements that it executes, like insert, delete, update, and so on, and sends them to the secondary nodes to perform. This type of replication was used in MySQL before version 5.1.

This type of approach seems good, but it has its disadvantages. For example, any

nondeterministic function (such as NOW()) might result in distinct writes on the follower and leader. Furthermore, if a write statement is dependent on a prior write, and both of them reach the follower in the wrong order, the outcome on the follower node will be uncertain.

Write-ahead log (WAL) shipping

In the write-ahead log (WAL) shipping approach, the primary node saves the query before executing it in a log file known as a write-ahead log file. It then uses these logs to copy the data onto the secondary nodes. This is used in PostgreSQL and Oracle. The problem with WAL is that it only defines data at a very low level. It's tightly coupled with the inner structure of the database engine, which makes upgrading software on the leader and followers complicated.

Logical (row-based) log replication

In the logical (row-based) log replication approach, all secondary nodes replicate the actual data changes. For example, if a row is inserted or deleted in a table, the secondary nodes will replicate that change in that specific table. The binary log records change to database tables on the primary node at the record level. To create a replica of the primary node, the secondary node reads this data and changes its records accordingly. Row-based replication doesn't have the same difficulties as WAL because it doesn't require information about data layout inside the database engine.

Multi-leader replication

As discussed above, single leader replication using asynchronous replication has a drawback. There's only one primary node, and all the writes have to go through it, which limits the performance. In case of failure of the primary node, the secondary nodes may not have the updated database.

Multi-leader replication is an alternative to single leader replication. There are multiple primary nodes that process the writes and send them to all other primary and secondary nodes to replicate. This type of replication is used in databases along with external tools like the Tungsten Replicator for MySQL.

This kind of replication is quite useful in applications in which we can continue work even if we're offline—for example, a calendar application in which we can set our meetings even if we don't have access to the internet. Once we're online, it replicates its changes from our local database (our mobile phone or laptop acts as a primary node) to other nodes.

Conflict

Multi-leader replication gives better performance and scalability than single leader replication, but it also has a significant disadvantage. Since all the primary nodes concurrently deal with the write requests, they may modify the same data, which can create a conflict between them. For example, suppose the same data is edited by two clients simultaneously. In that case, their writes will be successful in their associated primary nodes, but when they reach the other primary nodes asynchronously, it creates a conflict.

Handle conflicts

Conflicts can result in different data at different nodes. These should be handled efficiently without losing any data. Let's discuss some of the approaches to handle conflicts:

Conflict avoidance

A simple strategy to deal with conflicts is to prevent them from happening in the first place. Conflicts can be avoided if the application can verify that all writes for a given record go via the same leader.

However, the conflict may still occur if a user moves to a different location and is now near a different data center. If that happens, we need to reroute the traffic. In such scenarios, the conflict avoidance approach fails and results in concurrent writes.

Last-write-wins

Using their local clock, all nodes assign a timestamp to each update. When a conflict occurs, the update with the latest timestamp is selected.

This approach can also create difficulty because the clock synchronization across nodes is challenging in distributed systems. There's clock skew that can result in data loss.

Custom logic

In this approach, we can write our own logic to handle conflicts according to the needs of our application. This custom logic can be executed on both reads and writes. When the system

detects a conflict, it calls our custom conflict handler.

Multi-leader replication topologies

There are many topologies through which multi-leader replication is implemented, such as circular topology, star topology, and all-to-all topology. The most common is the all-to-all topology. In star and circular topology, there's again a similar drawback that if one of the nodes fails, it can affect the whole system. That's why all-to-all is the most used topology.

Peer-to-peer/leaderless replication

In primary-secondary replication, the primary node is a bottleneck and a single point of failure. Moreover, it helps to achieve read scalability but fails in providing write scalability. The peer-to-peer replication model resolves these problems by not having a single primary node. All the nodes have equal weightage and can accept reads and writes requests. Amazon popularized such a scheme in their DynamoDB data store.

[Peer-to-peer data replication model where all nodes apply reads and writes to all the data](./p2p.jpg)

Like primary-secondary replication, this replication can also yield inconsistency. This is because when several nodes accept write requests, it may lead to concurrent writes. A helpful approach used for solving write-write inconsistency is called quorums.

Quorums

Let's suppose we have three nodes. If at least two out of three nodes are guaranteed to return successful updates, it means only one node has failed. This means that if we read from two nodes, at least one of them will have the updated version, and our system can continue working.

If we have n nodes, then every write must be updated in at least w nodes to be considered a success, and we must read from r nodes. We'll get an updated value from reading as long as $w + r > n$ because at least one of the nodes must have an updated write from which we can read. Quorum reads and writes adhere to these r and w values. These n , w and r are configurable in Dynamo-style databases.

[Reader getting an updated value from replica 2](./dynamo.jpg)

Databases/Introduction to Databases/README.md

Introduction to Databases

Problem statement

Let's start with a simple question. Can we make a software application without using databases? Let's suppose we have an application like WhatsApp. People use our application to communicate with their friends. Now, where and how we can store information (a list of people's names and their respective messages) permanently and retrieve it?

We can use a simple file to store all the records on separate lines and retrieve them from the same file. But using a file for storage has some limitations.

Limitations of file storage

'''

We can't offer concurrent management to separate users accessing the storage files from different locations.

We can't grant different access rights to different users.

How will the system scale and be available when adding thousands of entries?

How will we search content for different users in a short time?

'''

[The limitations of file storage](./file.jpg)

Solution

The above limitations can be addressed using databases.

A database is an organized collection of data that can be managed and accessed easily. Databases are created to make it easier to store, retrieve, modify, and delete data in connection with different data-processing procedures.

Some of the applications where we use database management are the banking systems, online shopping stores, and so on. Different organizations have different sizes of databases according to their needs.

'''

Note: According to <https://www.comparebusinessproducts.com/>, the World Data Center for Climate (WDCC) is the largest database in the world. It contains around 220 terabytes of web data and petabytes of additional data.

'''

There are two basic types of databases:

'''

SQL (relational databases)

NoSQL (non-relational databases)

'''

They differ in terms of their intended use case, the type of information they hold, and the storage method they employ.

[SQL versus. NoSQL](./db_type.jpg)

Relational databases, like phone books that record contact numbers and addresses, are organized and have predetermined schemas. Non-relational databases, like file directories that store anything from a person's constant information to shopping preferences, are unstructured, scattered, and feature a dynamic schema. We'll discuss their differences and their types in detail in the next lesson.

Advantages

A proper database is essential for every business or organization. This is because the database stores all essential information about the organization, such as personnel records, transactions, salary information, and so on. Following are some of the reasons why the database is important:

'''

Managing large data: A large amount of data can be easily handled with a database, which wouldn't be possible using other tools.

Retrieving accurate data (data consistency): Due to different constraints in databases, we can retrieve accurate data whenever we want.

Easy updation: It is quite easy to update data in databases using data manipulation language (DML).

Security: Databases ensure the security of the data. A database only allows authorized users to access data.

Data integrity: Databases ensure data integrity by using different constraints for data.

Availability: Databases can be replicated (using data replication) on different servers, which can be concurrently updated. These replicas ensure availability.

Scalability: Databases are divided (using data partitioning) to manage the load on a single node. This increases scalability.

'''

How will we explain databases?

We have divided the database chapter into four lessons:

'''

Types of Databases: We'll discuss the different types of databases, their advantages, and the

disadvantages.

Data Replication: Weâ ll discuss what data replication is and its different models with their pros and cons.

Data Partitioning: Weâ ll discuss what data partitioning is and its different models with the pros and cons.

Cost-benefit analysis: Weâ ll discuss which database sharding approach is best for different kinds of databases.

'''

Letâ s get started by understanding different types of databases and their preferred use cases.

Databases/Trade-offs in Databases/README.md

Trade-offs in Databases

Which is the best database sharding approach?

Both horizontal and vertical sharding involve adding resources to our computing infrastructure. Our business stakeholders must decide which is suitable for our organization. We must scale our resources accordingly for our organization and business to grow, to prevent downtime, and to reduce latency. We can scale these resources through a combination of adjustments to CPU, physical memory requirements, hard disk adjustments, and network bandwidth.

The following sections explain the pros and cons of no sharding versus sharding.

Advantages and disadvantages of a centralized database

Advantages

- Data maintenance, such as updating and taking backups of a centralized database, is easy.
- Centralized databases provide stronger consistency and ACID transactions than distributed databases.
- Centralized databases provide a much simpler programming model for the end programmers as compared to distributed databases.
- Itâ s more efficient for businesses to have a small amount of data to store that can reside on a single node.

Disadvantages

- A centralized database can slow down, causing high latency for end users, when the number of queries per second accessing the centralized database is approaching single-node limits.
- A centralized database has a single point of failure. Because of this, its probability of not being accessible is much higher.

Advantages and disadvantages of a distributed database

Advantages

- Itâ s fast and easy to access data in a distributed database because data is retrieved from the nearest database shard or the one frequently used.
- Data with different levels of distribution transparency can be stored in separate places.
- Intensive transactions consisting of queries can be divided into multiple optimized subqueries, which can be processed in a parallel fashion.

Disadvantages

- Sometimes, data is required from multiple sites, which takes more time than expected.
- Relations are partitioned vertically or horizontally among different nodes. Therefore, operations such as joins need to reconstruct complete relations by carefully fetching data.

These operations can become much more expensive and complex.

- It's difficult to maintain consistency of data across sites in the distributed database, and it requires extra measures.

- Updates and backups in distributed databases take time to synchronize data.

Query optimization and processing speed in a distributed database

A transaction in the distributed database depends on the type of query, number of sites (shards) involved, communication speed, and other factors, such as underlying hardware and the type of database used. However, as an example, let's assume a query accessing three tables, Store, Product, and Sales, residing on different sites.

The number of attributes in each table is given in the following figure:

[Database schema consisting of three tables: Store, Product, and Sales](./3tables.jpg)

Let's assume the distribution of both tables on different sites is the following:

- The Store table has 10,000 tuples stored at site A.
- The Product table has 100,000 tuples stored at site B.
- The Sales table has one million tuples stored at site A.

Now, assume that we need to process the following query:

```
---
```

```
Select Store_key from (Store JOIN Sales JOIN Product)
```

```
where Region= 'East' AND Brand='Wolf';
```

```
---
```

The above query performs the join operations on the Store, Sales, and Product tables and retrieves the Store_key values from the table generated in the result of join operations.

Next, assume every stored tuple is 200 bits long. That's equal to 25 Bytes. Furthermore, estimated cardinalities of certain intermediate results are as follows:

- The number of the Wolf brand is 10.
- The number of East region stores is 100,000.

Communication assumptions are the following:

Data rate = 50M bits per second

Access delay = 0.1 second

Parameters assumption

Before processing the query using different approaches, let's define some parameters:

a=Total access delay

b= Data rate

v= Total data volume

Now, let's compute the total communication time, T, according to the following formula:

$T = a + v/b$

Let's try the following possible approaches to execute the query.

Possible approaches

- Move the Product table to site A and process the query at A.

$T = 100,000 \times 200 / 50,000,000 = 0.5$ seconds

Here, 0.1 is the access delay of the table on site A, and 100,000 is the number of tuples in the Product table. The size of each tuple in bits is 200, and 50,000,000 is the data rate. The 200 and 50,000,000 figures are the same for all of the following calculations.

- Move Store and Sales to site B and process the query at B:

$T = 0.2 + ((10,000 + 1,000,000)^{200} / 50,000,000) = 4.24$ seconds

Here, 0.2 is the access delay of the Store and Product tables. The numbers 10,000 and 1,000,000 are the number of tuples in the Store and Product tables, respectively.

- Restrict Brand at site B to Wolf (called projection) and move the result to site A:

$T = 0.1 + (10^{200} / 50,000,000) =$ approximately 0.1 seconds

Here, 0.1 is the access delay of the Product table. The number of the Wolf brand is 10, hence the number of tuples.

When we compare the three approaches, the third approach provides us the least latency (0.1 seconds). This example shows that careful query optimization is also critical in the distributed database.

Conclusion

Data distribution (vertical and horizontal sharding) across multiple nodes aims to improve the following features, considering that the queries are optimized:

- Reliability (fault-tolerance)
- Performance
- Balanced storage capacity and dollar costs

Both centralized and distributed databases have their pros and cons. We should choose them according to the needs of our application.

Databases/Types of Databases/README.md

Types of Databases

As we discussed earlier, databases are divided into two types: relational and non-relational. Let's discuss these types in detail.

Relational databases

Relational databases adhere to particular schemas before storing the data. The data stored in relational databases has prior structure. Mostly, this model organizes data into one or more relations (also called tables), with a unique key for each tuple (instance). Each entity of the data consists of instances and attributes, where instances are stored in rows, and the attributes of each instance are stored in columns. Since each tuple has a unique key, a tuple in one table can be linked to a tuple in other tables by storing the primary keys in other tables, generally known as foreign keys.

A Structure Query Language (SQL) is used for manipulating the database. This includes insertion, deletion, and retrieval of data.

There are various reasons for the popularity and dominance of relational databases, which include simplicity, robustness, flexibility, performance, scalability, and compatibility in managing generic data.

Relational databases provide the atomicity, consistency, isolation, and durability (ACID) properties to maintain the integrity of the database. ACID is a powerful abstraction that simplifies complex interactions with the data and hides many anomalies (like dirty reads, dirty writes, read skew, lost updates, write skew, and phantom reads) behind a simple transaction abort.

But ACID is like a big hammer by design so that it's generic enough for all the problems. If some specific application only needs to deal with a few anomalies, there's a window of opportunity to use a custom solution for higher performance, though there is added complexity.

Let's discuss ACID in detail:

- Atomicity: A transaction is considered an atomic unit. Therefore, either all the statements

within a transaction will successfully execute, or none of them will execute. If a statement fails within a transaction, it should be aborted and rolled back.

- Consistency: At any given time, the database should be in a consistent state, and it should remain in a consistent state after every transaction. For example, if multiple users want to view a record from the database, it should return a similar result each time.

- Isolation: In the case of multiple transactions running concurrently, they shouldn't be affected by each other. The final state of the database should be the same as the transactions were executed sequentially.

- Durability: The system should guarantee that completed transactions will survive permanently in the database even in system failure events.

Various database management systems (DBMS) are used to define relational database schema along with other operations, such as to store, retrieve, and run SQL queries on data. Some of the popular DBMS are as follows:

```
'''
MySQL
Oracle Database
Microsoft SQL Server
IBM DB2
Postgres
SQLite
'''
```

Why relational databases?

Relational databases are the default choices of software professionals for structured data storage. There are a number of advantages to these databases. One of the greatest powers of the relational database is its abstractions of ACID transactions and related programming semantics. This makes it very convenient for the end-programmer to use a relational database. Let's review some important features of relational databases:

Flexibility

In the context of SQL, data definition language (DDL) provides us the flexibility to modify the database, including tables, columns, renaming the tables, and other changes. DDL even allows us to modify schema while other queries are happening and the database server is running.

```
'''
DDL is a computer language used to create and modify the structure of database objects in a database.
'''
```

Reduced redundancy

One of the biggest advantages of the relational database is that it eliminates data redundancy. The information related to a specific entity appears in one table while the relevant data to that specific entity appears in the other tables linked through foreign keys. This process is called normalization and has the additional benefit of removing an inconsistent dependency.

```
'''
Inconsistent dependencies arise when a user searches for some data in the irrelevant table or the data is missing in the relevant table. For example, to find the customer's address it should be searched in the customer table instead of the product table.
'''
```

Concurrency

Concurrency is an important factor while designing an enterprise database. In such a case, the data is read and written by many users at the same time. We need to coordinate such interactions to avoid inconsistency in data—for example, the double booking of hotel rooms. Concurrency in a relational database is handled through transactional access to the data. As explained earlier, a transaction is considered an atomic operation, so it also works in error handling to either roll back or commit a transaction on successful execution.

Integration

The process of aggregating data from multiple sources is a common practice in enterprise applications. A common way to perform this aggregation is to integrate a shared database where multiple applications store their data. This way, all the applications can easily access each other's data while the concurrency control measures handle the access of multiple applications.

Backup and disaster recovery

Relational databases guarantee the state of data is consistent at any time. The export and import operations make backup and restoration easier. Most cloud-based relational databases perform continuous mirroring to avoid loss of data and make the restoration process easier and quicker.

Drawback

Impedance mismatch

Impedance mismatch is the difference between the relational model and the in-memory data structures. The relational model organizes data into a tabular structure with relations and tuples. SQL operation on this structured data yields relations aligned with relational algebra. However, it has some limitations. In particular, the values in a table take simple values that can't be a structure or a list. The case is different for in-memory, where a complex data structure can be stored. To make the complex structures compatible with the relations, we would need a translation of the data in light of relational algebra. So, the impedance mismatch requires translation between two representations, as denoted in the following figure:

[A single aggregated value in the view is composed of several rows and tables in the relational database](./mismatch.jpg)

Why non-relational (NoSQL) databases?

A NoSQL database is designed for a variety of data models to access and manage data. There are various types of NoSQL databases, which we'll explain in the next section. These databases are used in applications that require a large volume of semi-structured and unstructured data, low latency, and flexible data models. This can be achieved by relaxing some of the data consistency restrictions of other databases. Following are some characteristics of the NoSQL database:

'''

Simple design: Unlike relational databases, NoSQL doesn't require dealing with the impedance mismatch—for example, storing all the employees' data in one document instead of multiple tables that require join operations. This strategy makes it simple and easier to write less code, debug, and maintain.

Horizontal scaling: Primarily, NoSQL is preferred due to its ability to run databases on a large cluster. This solves the problem when the number of concurrent users increases. NoSQL makes it easier to scale out since the data related to a specific employee is stored in one document instead of multiple tables over nodes. NoSQL databases often spread data across multiple nodes and balance data and queries across nodes automatically. In case of a node failure, it can be transparently replaced without any application disruption.

Availability: To enhance the availability of data, node replacement can be performed without application downtime. Most of the non-relational databases' variants support data replication to ensure high availability and disaster recovery.

Support for unstructured and semi-structured data: Many NoSQL databases work with data that doesn't have schema at the time of database configuration or data writes. For example, document databases are structureless; they allow documents (JSON, XML, BSON, and so on) to have different fields. For example, one JSON document can have fewer fields than the other.

Cost: Licenses for many RDBMSs are pretty expensive, while many NoSQL databases are open source and freely available. Similarly, some RDBMSs rely on costly proprietary hardware and storage

systems, while NoSQL databases usually use clusters of cheap commodity servers.

'''

Unstructured data: The schema of the data is not known at the time of writing the data. Schema is usually defined at the time of reading such data.

'''

NoSQL databases are divided into various categories based on the nature of the operations and features, including document store, columnar database, key-value store, and graph database. We'll discuss each of them along with their use cases from the system design perspective in following sections.

Types of NoSQL databases

Various types of NoSQL databases are described below:

[The types of NoSQL databases](./nonrelational.jpg)

Key-value database

Key-value databases use key-value methods like hash tables to store data in key-value pairs. We can see this depicted in the figure a couple of paragraphs below. Here, the key serves as a unique or primary key, and the values can be anything ranging from simple scalar values to complex objects. These databases allow easy partitioning and horizontal scaling of the data. Some popular key-value databases include Amazon DynamoDB, Redis, and Memcached DB.

Use case: Key-value databases are efficient for session-oriented applications. Session oriented-applications, such as web applications, store users' data in the main memory or in database during a session. This data may include user profile information, recommendations, targeted promotions, discounts, and more. A unique ID (a key) is assigned to each user's session for easy access and storage. Therefore, a better choice to store such data is the key-value database.

The following figure shows an example of a key-value database. The Product ID and Type of the item are collectively considered as the primary key. This is considered as a key for this key-value database. Moreover, the schema for storing the item attributes is defined based on the nature of the item and the number of attributes it possesses.

[Data stored in the form of key-value pair in DynamoDB, where the key is the combination of two attributes (Product ID and Type)](./dynamo.jpg)

Document database

A document database is designed to store and retrieve documents in formats like XML, JSON, BSON, and so on. These documents are composed of a hierarchical tree data structure that can include maps, collections, and scalar values. Documents in this type of database may have varying structures and data. MongoDB and Google Cloud Firestore are examples of document databases.

Use case: Document databases are suitable for unstructured catalog data, like JSON files or other complex structured hierarchical data. For example, in e-commerce applications, a product has thousands of attributes, which is unfeasible to store in a relational database due to its impact on the reading performance. Here comes the role of a document database, which can efficiently store each attribute in a single file for easy management and faster reading speed. Moreover, it's also a good option for content management applications, such as blogs and video platforms. An entity required for the application is stored as a single document in such applications.

The following example shows data stored in a JSON document. This data is about a person. Various attributes are stored in the file, including id, name, email, and so on.

'''

```
{ "id": 1001,
```

```

"name": "Brown",
"title": "Mr.",
"email": "brown@anyEmail.com",
"cell": "123-465-9999",
"likes": [
    "designing",
    "cycling",
    "skiing"],
"businesses": [
    { "name": "ABC co.",
      "partner": "Vike",
      "status": "Bankrupt",
      "date_founded": {
        "$date": "2021-12-10" } }]]}
    A JSON file containing data of a businessman
, , ,

```

Graph database

Graph databases use the graph data structure to store data, where nodes represent entities, and edges show relationships between entities. The organization of nodes based on relationships leads to interesting patterns between the nodes. This database allows us to store the data once and then interpret it differently based on relationships. Popular graph databases include Neo4J, OrientDB, and InfiniteGraph. Graph data is kept in store files for persistent storage. Each of the files contains data for a specific part of the graph, such as nodes, links, properties, and so on.

In the following figure, some data is stored using a graph data structure in nodes connected to each other via edges representing relationships between nodes. Each node has some properties, like Name, ID, and Age. The node having ID: 2 has the Name of James and Age of 29 years.

[A graph consists of nodes and links. This graph captures entities and their relationships with each other](./graph.jpg)

Use case: Graph databases can be used in social applications and provide interesting facts and figures among different kinds of users and their activities. The focus of graph databases is to store data and pave the way to drive analyses and decisions based on relationships between entities. The nature of graph databases makes them suitable for various applications, such as data regulation and privacy, machine learning research, financial services-based applications, and many more.

Columnar database

Columnar databases store data in columns instead of rows. They enable access to all entries in the database column quickly and efficiently. Popular columnar databases include Cassandra, HBase, Hypertable, and Amazon SimpleDB.

Use case: Columnar databases are efficient for a large number of aggregation and data analytic queries. It drastically reduces the disk I/O requirements and the amount of data required to load from the disk. For example, in applications related to financial institutions, there's a need to sum the financial transaction over a period of time. Columnar databases make this operation quicker by just reading the column for the amount of money, ignoring other attributes of customers.

The following figure shows an example of a columnar database, where data is stored in a column-oriented format. This is unlike relational databases, which store data in a row-oriented fashion:

[Column-oriented and row-oriented database](./col_row.jpg)

Drawbacks of NoSQL databases

Lack of standardization

NoSQL doesn't follow any specific standard, like how relational databases follow relational algebra. Porting applications from one type of NoSQL database to another might be a challenge.

Consistency

NoSQL databases provide different products based on the specific trade-offs between consistency and availability when failures can happen. We won't have strong data integrity, like primary and referential integrities in a relational database. Data might not be strongly consistent but slowly converging using a weak model like eventual consistency.

Choose the right database

Various factors affect the choice of database to be used in an application. A comparison between the relational and non-relational databases is shown in the following table to help us choose:

[Relational and Non-relational Databases](relational_nonrelational.jpg)

'''

Note: When NoSQL databases first came into being, they were drastically different to program and use as compared to traditional databases. Though, due to extensive research in academia and industry over the last many years, the programmer-facing differences between NoSQL and traditional stores are blurring. We might be using the same SQL constructs to talk to a NoSQL store and get a similar level of performance and consistency as a traditional store. Google's Cloud Spanner is one such database that's geo-replicated with automatic horizontal sharding ability and high-speed global snapshots of data.

'''

'''

Spanner: Google's Globally-Distributed Database, Google, Inc.

'''

Design Collaborative Document Editing Service Google Docs/Concurrency in Collaborative Editing/README.md

Concurrency in Collaborative Editing

Introduction

We've discussed the design for a collaborative document editing service, but we haven't addressed how we'll deal with concurrent changes in the document by different users. However, before discussing concurrency issues, we need to understand the collaborative text editor.

What is a document editor?

A document is a composition of characters in a specific order. Each character has a value and positional index. The character can be a letter, a number, an enter (μ), or a space. An index represents the character's position within the ordered list of characters.

The job of the text or document editor is to perform operations like insert(), delete(), edit(), and more on the characters within the document. A depiction of a document and how the editor will perform these operations is below.

[How a document editor performs different operations](./editor.jpg)

Concurrency issues

Collaboration on the same document by different users can lead to concurrency issues. Conflicts may arise whenever multiple users edit the same portion of a document. Since users have a local copy of the document, the final status of the document may be different at the server from what the users see at their end. After the server pushes the updated version, users discover the unexpected outcome.

Example 1

Let's consider a scenario where two users want to add some characters at the same positional index. Below, we've depicted how two users modifying the same sentence can lead to conflicting results:

[How two users modifying the same text can lead to concurrency issues](./concurrency.jpg)

As shown above, two users are trying to modify the same sentence, "Educative by developers." Both users are performing insert() at index 10. The two possibilities are as follows:

- The phrase "for developers" overwrites "platform." This leads to an outcome of "for developers."
 - The phrase "platform" overlaps "for developers." This leads to an outcome of "platformopers."
- This example shows that operations applied in a different order don't hold the commutative property.

Example 2

Let's look at another simple example where two users are trying to delete the same character from a word. Let's use the word "EEDUCATIVE." Because the word has an extra "E," both would want to remove the extra character. Below, we see how an unexpected result can occur:

[How deleting the same character can lead to unexpected changes](./deletion.jpg)

This second example shows that different users applying the same operation won't be idempotent. So, conflict resolution is necessary where multiple collaborators are editing the same portion of the document at the same time.

'''

Idempotent: An operation with the same input produces the same results no matter how many times an operation repeats.

'''

From the examples above, we understand that a solution to concurrency issues in collaborative editing should respect two rules:

- Commutativity: The order of applied operations shouldn't affect the end result.
 - Idempotency: Similar operations that have been repeated should apply only once.
- Below, we identify two well-known techniques for conflict resolution.

Techniques for conflict resolution

Let's discuss two leading technologies that are used for conflict resolution in collaborative editing.

Operational transformation

Operational transformation (OT) is a technique that's widely used for conflict resolution in collaborative editing. OT emerged in 1989 and has improved throughout the years. It's a lock-free and non-blocking approach for conflict resolution. If operations between collaborators conflict, OT resolves conflicts and pushes the correct converged state to end users. As a result, OT provides consistency for users.

OT performs operations using the positional index method to resolve conflicts, such as the one we discussed above. OT resolves the problems above by holding commutativity and idempotency.

[Ensuring commutativity 1](./ot1.jpg)

[Ensuring commutativity 2](./ot2.jpg)

Collaborative editors based on OT are consistent if they have the following two properties:

- Causality preservation: If operation a happened before operation b, then operation a is executed before operation b.
- Convergence: All document replicas at different clients will eventually be identical.

The properties above are a part of the CC consistency model, which is a model for consistency

maintenance in collaborative editing.

'''

The research community has proposed various consistency models over the years. Some of them are specific to collaborative editing, while others are specific to OT algorithms. The key consistency models are the following:

- CC model: As we defined above, this includes causality preservation and convergence.
- CCI model: This includes causality preservation (Same as CC model.), convergence (Same as CC model.), and intention preservation (This property ensures that operations are executed as intended by the users.).

Other models include the CSM (causality, single-operation effects, and multi-operation effects) model and the CA (causality and admissibility) model.

Various consistency models are suggested, and usually, the newer ones are supersets of the earlier ones. Because there are so many proposed algorithms, discussing them is beyond the scope of our lesson. Interested readers can find more details in <https://dl.acm.org/doi/pdf/10.1145/289444.289469>.

'''

OT has two disadvantages:

- Each operation to characters may require changes to the positional index. This means that operations are order dependent on each other.
- It's challenging to develop and implement.

Operational transformation is a set of complex algorithms, and its correct implementation has proved challenging for real-world applications. For example, the Google Wave team took two years to implement an OT algorithm.

Conflict-free Replicated Data Type (CRDT)

The Conflict-free Replicated Data Type (CRDT) was developed in an effort to improve OT. A CRDT has a complex data structure but a simplified algorithm.

A CRDT satisfies both commutativity and idempotency by assigning two key properties to each character:

- It assigns a globally unique identity to each character.
- It globally orders each character.

Each operation now has an updated data structure:

[Simplified data structure of a CRDT](./crdt.jpg)

The SiteID uniquely identifies a user's site requesting an operation with a Value and a PositionalIndex. The value of PositionalIndex can be in fractions for two main reasons.

- The PositionalIndex of other characters won't change because of certain operations.
- The order dependency of operations between different users will be avoided.

The example below depicts that a user from site ID 123e4567-e89b-12d3 is inserting a character with a value of A at a PositionalIndex of 1.5. Although a new character is added, the positional indexes of existing characters are preserved using fractional indices. Therefore, the order dependency between operations is avoided. As shown below, an insert() between O and T didn't affect the position of T.

[Preventing order dependency between operations](./prevention.jpg)

CRDTs ensure strong consistency between users. Even if some users are offline, the local replicas at end users will converge when they come back online.

Although well-known online editing platforms like Google Docs, Etherpad, and Firepad use OT, CRDTs have made concurrency and consistency in collaborative document editing easy. In fact, with CRDTs, it's possible to implement a serverless peer-to-peer collaborative document editing service.

'''

Note: OT and CRDTs are good solutions for conflict resolution in collaborative editing, but the use of WebSockets makes it possible to highlight a collaborator's cursor. Other users will anticipate the position of a collaborator's next operation and naturally avoid conflict.

'''

'''

Question 1

Why can't we use locks to synchronize between users?

Answer

Locks require us to segment documents into small sections where users could lock a portion and edit it. This will help developers come up with an easy solution and avoid complexities like CRDTs and CRDTs. However, this also leads to poor user experience. For example, two users may want to add characters to the same section of the document, but their operations may not necessarily conflict.

A lock is a good choice for services like Google Sheets. This is because the document is divided into equal sizes of small cells, and only one user can add or edit the contents of a specific cell.

'''

'''

Question 2

What happens if there are two users collaborating on a document and they have different Internet speeds? Which technology, OT or CRDT, is better suited for conflict resolution in such a case?

Answer

With a varied Internet connection speed, the order of operations between users can lead to problems. However, operations are order dependent in OT, whereas operations in CRDTs are order independent. This is why CRDTs are a suitable solution to such a problem.

'''

Design Collaborative Document Editing Service Google Docs/Design of Google Docs/README.md

Design of Google Docs

Design

We'll complete our design in two steps. In the first step, we'll explain different components and building blocks and the reason for their choice in our design. The second step will describe how we fulfill various functional requirements by depicting a workflow.

Components

We've utilized the following set of components to complete our design:

- API gateway: Different client requests will get intercepted through the API gateway. Depending on the request, it's possible to forward a single request to multiple components, reject a request, or reply instantly using an already cached response, all through the API gateway. Edit requests, comments on a document, notifications, authentication, and data storage requests will all go through the API gateway.
- Application servers: The application servers will run business logic and tasks that generally require computational power. For instance, some documents may be converted from one file type to another (for example, from a PDF to a Word document) or support features like import and export. It's also central to the attribute collection for the recommendation engine.
- Data stores: Various data stores will be used to fulfill our requirements. We'll employ a relational database for saving users' information and document-related information for imposing

privilege restrictions. We can use NoSQL for storing user comments for quicker access. To save the edit history of documents, we can use a time series database. We'll use blob storage to store videos and images within a document. Finally, we can use distributed cache like Redis or a CDN to provide end users good performance. We use Redis specifically to store different data structures, including user sessions, features for the typeahead service, and frequently accessed documents. The CDN stores frequently accessed documents and heavy objects, like images and videos.

- Processing queue: Since document editing requires frequently sending small-sized data (usually characters) to the server, it's a good idea to queue this data for periodic batch processing. We'll add characters, images, videos, and comments to the processing queue. Using an HTTP call for sending every minor character is inefficient. Therefore, we'll use WebSockets to reduce overhead and observe live changes to the document by different users.

- Other components: Other components include the session servers that maintain the user's session information. We'll manage document access privileges through the session servers. Essentially, there will also be configuration, monitoring, pub-sub, and logging services that will handle tasks like monitoring and electing leaders in case of server failures, queueing tasks like user notifications, and logging debugging information.

The illustration below provides a depiction of how different components and building blocks coordinate to provide the service.

[A detailed design of the collaborative document editing service](./design.jpg)

'''

Question 1

Why should we use WebSockets instead of HTTP methods? Why are WebSockets optimal for this kind of communication?

Answer

WebSockets offer us the following characteristics:

- They have a long-lasting connection between clients and servers.
- They enable full-duplex communication. That is, we can simultaneously communicate from client to server and vice versa.
- There's no overhead of HTTP request or response headers.

The lightweight nature of WebSockets reduces the latency and allows the server to push changes to clients as soon as they are available.

'''

'''

Question 2

The design above depicts a microservices architecture instead of a monolithic one. Why is that suitable here?

Answer

Microservices are preferred for the following reasons:

- Development is simpler and faster.
- Each service within the architecture is isolated. That is, failure of one service doesn't produce a cascading effect.
- Different components may have different programming language requirements for various reasons. Microservices give the freedom of using different programming languages for different components.
- Because of microservices's modular nature, it's easy to scale and update services individually.

'''

'''

Question 3

What queuing algorithm is best suited for the operations queue in the design above?

Answer

First in, first out (FIFO) with strict ordering is best suited for the operations queue so that operations are performed in the order requested by the users.

'''

Workflow

In the following steps, we'll explain how different requests will get entertained after they reach the API gateway:

- Collaborative editing and conflict resolution: Each request gets forwarded to the operations queue. This is where conflicts get resolved between different collaborators of the same document. If there are no conflicts, the data is batched and stored in the time series database via session servers. Data like videos and images get compressed for storage optimization, while characters are processed right away.

- History: It's possible to recover different versions of the document with the help of a time series database. Different versions can be compared using DIFF operations that compare the versions and identify the differences to recover older versions of the same document.

- Asynchronous operations: Notifications, emails, view counts, and comments are asynchronous operations that can be queued through a pub-sub component like Kafka. The API gateway generates these requests and forwards them to the pub-sub module. Users sharing documents can generate notifications through this process.

- Suggestions: Suggestions are in the form of the typeahead service that offers autocomplete suggestions for typically used words and phrases. The typeahead service can also extract attributes and keywords from within the document and provide suggestions to the user. Since the number of words can be high, we'll use a NoSQL database for this purpose. In addition, most frequently used words and phrases will be stored in a caching system like Redis.

- Import and export documents: The application servers perform a number of important tasks, including importing and exporting documents. Application servers also convert documents from one format to another. For example, a .doc or .docx document can be converted into .pdf or vice versa. Application servers are also responsible for feature extraction for the typeahead service.

'''

Note: Our use of WebSockets speeds up the overall performance and enables us to facilitate chatting between users who are collaborating on the same document. If we combine WebSockets with a Redis-like cache, it's possible to develop an effective chatting feature.

'''

'''

Question 1

We're implementing view counters through an asynchronous method, which means that the number of views of a document may be outdated. Could we use sharded counters or Redis counters to get effective results?

Answer

Both solutions (sharded counters or Redis counters) will work, though they seem excessive for counting views on documents.

To provide near real-time view counting, we can use a streaming pub-sub system, such as Kafka, where a topic can be based on a document identifier.

'''

'''

Question 2

The detailed design above doesn't depict where the view counter data is saved. What would be suitable storage for the view counter, and what design changes will have to be made?

Answer

For scalability purposes, it's suitable to store the view counter data in NoSQL because the read/write latency of NoSQL is generally lower.

To complete the design, weâ ll have to connect the view counter with the NoSQL database.
'''

Design Collaborative Document Editing Service Google Docs/Evaluation of Google Docs'
Design/README.md

Evaluation of Google Docsâ Design

Weâ ve now explained the design and how it fulfills the functional requirements for a collaborative document editing service. This lesson will focus on how our design meets the non-functional requirements. In particular, weâ ll focus on consistency, latency, scalability, and availability.

Consistency

Weâ ve looked at how weâ ll achieve strong consistency for conflict resolution in a document through two technologies: operational transformation (OT) and Conflict-free Resolution Data Types (CRDTs). In addition, a time series database enables us to preserve the order of events. Once OT or CRDT has resolved any conflicts, the final result is saved in the database. This helps us achieve consistency in terms of individual operations.

Weâ re also interested in keeping the document state consistent across different servers in a data center. To replicate an updated state of a document within the same data center at the same time, we can use peer-to-peer protocols like Gossip protocol. Not only will this strategy improve consistency, it will also improve availability.

'''

Also referred to as epidemic protocol which is used for disseminating data across all nodes in a network to achieve consistency.

'''

'''

Question

Why should we use strong consistency instead of eventual consistency for conflict resolution in a collaborative document editing service?

Answer

From Amazonâ s Dynamo system, we learn that if we use eventual consistency for conflict resolution, we might have multiple versions of a document that are eventually reconciled, either automatically or manually. In the case of automatic reconciliation, the document might update abruptly, which defeats the purpose of collaboration. The second case, manual resolution, is tedious labor that we want to avoid.

Therefore, we use strong consistency for conflict resolution, and the logically centralized server provides the final order of events to all clients. We use a replicated operations queue so that even if our ordering service dies, it can easily restart on a new server and resume where it left off. Clients might encounter short service unavailability while the failed component is being respawned.
'''

Latency

Latency may feel like a challenge, specifically when two users are distant from each other or the server. However, users maintain a replica of documents at their end while data is being propagated through WebSockets to end-servers. Therefore, user-perceived latency will be low. Apart from this, users mostly tend to write textual data in a document thatâ s small. Therefore, dissemination of data among different servers within the same facility and among different data centers or zones will be carried out with low latency. Also, files like videos and images can be stored in CDNs for quick serving because this content doesnâ t change frequently.

Practically speaking, there will be a limited number of readers and writers of an online

document. For readers specifically, latency won't be an issue because the document will be loaded only once. So, most readers can be served from the same data center. For writers, an optimal zone should be selected as the centralized location between collaborators of the same document. However, for popular documents, asynchronous replication will be an effective method to achieve good performance and low latency for a large number of users. In general, achieving strong consistency becomes a challenge when replication is asynchronous.

Availability

Our design ensures availability by using replicas and monitoring the primary and replica servers using monitoring services. Key components like the operations queue and data stores internally manage their replication.

Since we use WebSockets, our WebSocket servers can connect users to the session maintenance servers that will determine if a user is actively viewing or collaborating on a document. Therefore, keeping multiple WebSocket servers will increase the availability of the design. Lastly, we employ caching services and CDNs to improve availability in case of failures.

However, at the moment, we haven't devised a disaster recovery management scheme.

'''

Fulfilling Non-functional Requirements

Requirements	Techniques
Consistency same data center	- Gossip protocol to replicate operations of a document within the
operations	- Concurrency techniques like OT and CRDTs - Usage of time series database for maintaining the order of
Latency	- Replication between data centers - Employing WebSockets - Asynchronous replication of data - Choosing optimal location for document creation and serving - Using CDNs for serving videos and images - Using Redis to store different data structures including CRDTs - Appropriate NoSQL databases for the required functionality
Availability disconnect	- Replication of components to avoid SPOFs - Using multiple WebSocket servers for users that may occasionally
to different zones, and global server load balancing	- Component isolation improves availability - Implementing disaster recovery protocols like backup, replication
Scalability	- Using monitoring and configuration services - Different data stores for different purposes enable scalability - Horizontal sharding of RDBMS - CDNs capable of handling a large number of requests for big files

'''

Scalability

Since we've used microservice architecture, we can easily scale each component individually in case the number of requests on the operations queue exceeds its capacity. We can use multiple operations queues. In that case, each operations queue will be responsible for a single document. We can forward operations requested by different users that are associated with a single document to a specific queue. The number of spawned queues will be equal to the number of active documents. As a result, we're able to achieve horizontal scalability.

Conclusion

In this chapter, we designed an online collaborative document editing service. In our design, we provided features like collaborative editing, keeping version history for reverting to older

versions, giving users suggestions on frequently used terms and phrases, and view counts of a document. We also assessed the possibility of adding a chatting feature between users collaborating on the same document. A unique aspect of the design was the conflict resolution between concurrent editing operations by different users. We solved the concurrency issues through OT and CRDT.

Design Collaborative Document Editing Service Google Docs/Requirements of Google Docs' Design/README.md

Requirements of Google Docsâ Design

Requirements

Letâ s look at the functional and non-functional requirements for designing a collaborative editing service.

Functional requirements

The activities a user will be able to perform using our collaborative document editing service are listed below:

- Document collaboration: Multiple users should be able to edit a document simultaneously. Also, a large number of users should be able to view a document.
 - Conflict resolution: The system should push the edits done by one user to all the other collaborators. The system should also resolve conflicts between users if theyâ re editing the same portion of the document.
 - Suggestions: The user should get suggestions about completing frequently used words, phrases and keywords in a document, as well as suggestions about fixing grammatical mistakes.
 - View count: Editors of the document should be able to see the view count of the document.
 - History: The user should be able to see the history of collaboration on the document.
- A real-world document editor also has to have functions like document creation, deletion, and managing user access. We focus on the core functionalities listed above, but we also discuss the possibility of other functionalities in the lessons ahead.

[Functional and non-functional requirements of collaborative editing service](./req.jpg)

Non-functional Requirements

- Latency: Different users can be connected to collaborate on the same document. Maintaining low latency is challenging for users connected from different regions.
- Consistency: The system should be able to resolve conflicts between users editing the document concurrently, thereby enabling a consistent view of the document. At the same time, users in different regions should see the updated state of the document. Maintaining consistency is important for users connected to both the same and different zones.
- Availability: The service should be available at all times and show robustness against failures.
- Scalability: A large number of users should be able to use the service at the same time. They can either view the same document or create new documents.

Resource estimation

Letâ s make some resource estimations based on the following assumptions:

- We assume that there are 80 million daily active users (DAU).
 - The maximum number of users able to edit a document concurrently is 20.
 - The size of a textual document is 100 KB.
 - Thirty percent of all the documents contain images, whereas only 2% of documents contain videos.
 - The collective storage required by images in a document is 800 KB, whereas each video is 3 MB.
 - A user creates one document in one day.
- Based on these assumptions, weâ ll make the following estimations.

Storage estimation

Considering that each user is able to create one document a day, there are a total of 80

million documents created each day. Below, we estimate the storage required for one day:
'''

Note: We can adjust the values in the table below to see how the storage requirement estimations change.
'''

'''

Estimation for Storage Requirements

Number of documents created by each user	1 per day
Number of active users	80 Million
Number of documents in a day	80 Million
Storage required for textual content per day	8 TB
Storage required for images per day	19.2 TB
Storage required for video content per day	4.8 TB
Total storage required per day	32TB

'''

- Total number of documents in a day: $80M \times 1\text{document} = 80M\text{documents in a day}$
- Storage for each textual document: $80M \times 100\text{KBs} = 8\text{TBs}$
- Storage required for images for one day: $(80M \times 30)/100 \times 800\text{KBs} = 19.2\text{ TBs}$ (Thirty percent of documents contain images.)
- Storage required for video content for one day: $(80M \times 2)/100 \times 3\text{MBs} = 4.8\text{ TBs}$ (Two percent of documents contain videos.)

[Storage required by online collaborative document editing service per day](./storage.jpg)

Total storage required for one day is as follows: $8 + 19.2 + 4.8 = 32\text{ TBs per day}$
'''

Note: Although our functional requirements state that we should keep a history of documents, we didn't include storage requirements for historical data for the sake of brevity.
'''

Bandwidth estimation

Incoming traffic: Assuming that 32 TB of data are uploaded per day to the network of a collaborative editing service, the network requirement for incoming traffic will be the following:

$32\text{ TB}/86400 \times 8 = 3\text{Gbps}$ approximately

Outgoing traffic: To estimate outgoing traffic bandwidth, we'll assume the number of documents viewed by a user each day. Let's consider that a typical user views five documents per day. Then, the following calculations apply:
'''

Note: We can adjust the values in the table below to see how the calculations change.
'''

'''

Number of documents viewed by users	5 per day
Number of active users	80 Million
Number of documents viewed in a day	400 Million
Number of documents viewed in a second	4630 per second
Bandwidth required for textual content per second	3.704 Gigabits per second(Gbps)
Bandwidth for image-based content per second	8.89 Gbps
Bandwidth for video content per second	2.22 Gbps
Total outgoing bandwidth required	14.81 Gbps

'''

- Total documents viewed per day by all users: $80M \times 5 = 400M\text{ documents viewed per day}$
- Documents viewed per second: $400M/86400 = 4.6 \times 10^3\text{ documents viewed per second}$
- Calculating the bandwidth required for textual content: $4.6 \times 10^3 \times 100\text{KB} \times 8 = 3.7\text{Gbps}$
- Calculating the bandwidth required for image-based content: $(4.6 \times 10^3 \times 30)/100$

$800KB \times 8 = 8.8Gbps$
 - Calculating the bandwidth required for video content: $(4.6 \times 10^3 \times 2) / 100 \times 3MB \times 8 = 2.2Gbps$
 - Total outgoing bandwidth: $3.7 + 8.8 + 2.2 = 14.7 Gbps$

[Summarizing the bandwidth requirement](./bandwidth.jpg)

'''

Note: The total bandwidth required is equal to the sum of incoming and outgoing traffic.
 $= 3 + 14.7 \approx 18Gbps$ approximately.

'''

Number of servers estimation

Let's assume that one user is able to generate 100 requests per day. Keeping in mind the number of daily active users, the number of requests per second (RPS) will be the following:

$100 \times 80M = 8000M / 86400 = 92.6$ thousands/sec

'''

	Number of RPS
Requests by a user	100 per day
Number of DAU	80 Million
RPS	92.6 Thousands per second

'''

We discussed in the Back of the Envelope lesson that RPS isn't sufficient information to calculate the number of servers. We'll use the following approximation for server count.

To estimate the number of servers required to fulfill the requests of 80 million users, we simply divide the number of users by the number of requests a server can handle. In the Back of the Envelope lesson, we discussed that our reference server can handle 8,000 requests per second. So, we see the following:

Number of daily active users/Queries handled per second = $80M / 8000 = 10,000$

Informally, the equation above assumes that one server can handle 8,000 users.

.

[Number of servers required](./servers.jpg)

Building blocks we will use

We'll use the following building blocks in designing the collaborative document editing service.

[Building blocks required to be integrated in the design]

- Load balancers will be the first point of contact for users.
- Databases will be needed to store several things including textual content, history of documents, user data, etc. For this purpose, we may need different types of databases.
- Pub-sub systems can complete tasks that can't be performed right away. We'll complete a number of tasks asynchronously in our design. Therefore, we'll use a pub-sub system.
- Caching will help us improve the performance of our design.
- Blob storage will store large files, such as images and videos.
- A Queueing system will queue editing operations requested by different users. Because many editing requests can't be performed simultaneously, we have to temporarily put them in a queue.
- A CDN can store frequently accessed media in a document. We can also put read-only documents that are frequently requested in a CDN.

Design Collaborative Document Editing Service Google Docs/System Design Google Docs/README.md

System Design: Google Docs

Problem statement

Imagine two students are preparing a report on a project theyâ€™ve just completed. Since the students live apart, the first student asks the second student to start writing the report, and the first student will improve the received copy. Although quite motivated, the students soon understand that such a collaboration is disorganized. The following illustration shows how tedious the process can get.

[Problems arising when thereâ€™s no collaborative document editing service](./demo.jpg)

The scenario above is one example that leads to time wastage and frustration when users collaborate on a document by exchanging files with each other.

Google Docs

To combat the problem above, we can use an online collaborative document editing service like Google Docs. Some advantages of using an online document editing service instead of a desktop application are as follows:

- Users can review and comment on a document while itâ€™s being edited.
- There are no special hardware specifications required to get the latest features. A machine that can run a browser will suffice.
- It's possible to work from any location.
- Unlike local desktop editors, users can view long-term document history and restore an older version if need be.
- The service is free of cost.

Other than Google docs, some popular online editing services include Etherpad, Microsoft Office 365, Slite, and many others.

Designing Google Docs

A collaborative document editing service can be designed in two ways:

- It could be designed as a centralized facility using client-server architecture to provide document editing service to all users.
- It could be designed using peer-to-peer technology to collaborate on a single document.

Most commercial solutions focus on client-service architecture to have finer control. Therefore, weâ€™ll focus on designing a service using the client service architecture. Letâ€™s see how we will progress in this chapter.

'''

Note: According to a survey, 64% of people use Google Docs for document editing at least once a week.

See: <https://www.statista.com/forecasts/1011649/frequency-of-using-google-docs-in-the-us>

'''

How will we design Google Docs?

We've divided the design problem into four stages:

1. Requirements for Google Docsâ€™ Design: This lesson will focus on establishing the requirements for designing a collaborative document editing service. Weâ€™ll also quantify the infrastructure requirements in this stage.
2. Google Docsâ€™ Design: The goal of this lesson is to come up with a design that fulfills the requirements of the service. This lesson will explain why a component is used and how it integrates with other components to fulfill functional requirements.
3. Concurrency in Collaborative Editing: Online document editing services have to resolve conflicts between users editing the same portion of a document. This lesson covers the type of problems that can arise and the techniques used to resolve such conflicts.
4. Evaluating Google Docsâ€™ Design: The main objective of this lesson is to evaluate our design for non-functional requirements. Mainly, we see if our design is performant, consistent,

available, and scalable.

Design Google Maps/Challenges of Google Maps' Design/README.md

Challenges of Google Maps' Design

Meeting the challenges

We listed two challenges in the Introduction lesson: scalability and ETA computation. Let's see how we meet these challenges.

Scalability

Scalability is about the ability to efficiently process a huge road network graph. We have a graph with billions of vertices and edges, and the key challenges are inefficient loading, updating, and performing computations. For example, we have to traverse the whole graph to find the shortest path. This results in increased query time for the user. So, what could be the solution to this problem?

The idea is to break down a large graph into smaller subgraphs, or partitions. The subgraphs can be processed and queried in parallel. As a result, the graph construction and query processing time will be greatly decreased. So, we divide the globe into small parts called segments. Each segment corresponds to a subgraph.

Segment

A segment is a small area on which we can work easily. Finding paths within these segments works because the segment's road network graph is small and can be loaded easily in the main memory, updated, and traversed. A city, for example, can be divided into hundreds of segments, each measuring 5[^] 5 miles.

'''

Traversing is visiting the graph vertices to reach a specific vertex.

'''

'''

Note: A segment is not necessarily a square. It can also be a polygon. However, we are assuming square segments for ease of explanation.

'''

Each segment has four coordinates that help determine which segment the user is in. Each coordinate consists of two values, the latitude and longitude.

[Partitioning the globe into small segments, and each segment has four coordinates with latitude and longitude values](./segment.jpg)

Let's talk about finding paths between two locations within a segment. We have a graph representing the road network in that segment. Each intersection/junction acts as a vertex and each road acts as an edge. The graph is weighted, and there could be multiple weights on each edge—such as distance, time, and traffic—to find the optimal path. For a given source and destination, there can be multiple paths. We can use any of the graph algorithms on that segment's graph to find the shortest paths. The most common shortest path algorithm is the Dijkstra's algorithm.

'''

The traditional Dijkstra algorithm performs a guided search to find the shortest path. However, it comes at the cost of an increased query time. We have to set a limit on the query time because we wouldn't want the user to wait too long to get their results. So, we'll go with finding the optimal path within the given time cost.

The A* algorithm is an extension of the Dijkstra algorithm that uses heuristics to guide its search. Instead of finding the shortest path, A* finds the optimal path that is calculated by minimizing the following function.

$f(n)=g(n)+h(n)$

Here, n is the next node on the path, $g(n)$ is the cost of the path from the start node to n , and $h(n)$ is a heuristic function that estimates the cost of the cheapest path from n to the goal.

'''

'''

Note: There are some algorithms that are specifically designed for road networks that focus on achieving shorter query time. These algorithms are A*, Arc flags, contraction hierarchies, transit node routing, reach-based routing, and hub labeling.

<https://arxiv.org/pdf/1603.01607.pdf>

<http://www.diag.uniroma1.it/challenge9/papers/kohler.pdf>

https://en.wikipedia.org/wiki/Contraction_hierarchies

https://www.researchgate.net/publication/6366597_Fast_Routing_in_Road_Networks_with_Transit_Nodes

<https://archive.siam.org/meetings/alenex04/abstracts/rgutman1.pdf>

<https://www.microsoft.com/en-us/research/wp-content/uploads/2010/12/HL-TR.pdf>

'''

After running the shortest path algorithm on the segment's graph, we store the algorithm's output in a distributed storage to avoid recalculation and cache the most requested routes. The algorithm's output is the shortest distance in meters or miles between every two vertices in the graph, the time it takes to travel via the shortest path, and the list of vertices along every shortest path. All of the above processing (running the shortest path algorithm on the segment graph) is done offline (not on a user's critical path).

[Finding a path between vertices within a segment](./path.jpg)

The illustration above shows how we can find the shortest distance in terms of miles between two points. For example, the minimum distance (m) between points A and D is 11 miles by taking the path A->C->D. It has an ETA of 15 minutes.

We've found the shortest path between two vertices. What if we have to find the path between two points that lie on the edges? What we do is find the vertices of the edge on which the points lie, calculate the distance of the point from the identified vertices, and choose the vertices that make the shorter total distance between the source and the destination. The distance from the source (and destination) to the nearest vertices is approximated using latitude/longitude values.

[Choosing a path between points that lie on the edge](./selection.jpg)

We're able to find the shortest paths within the segment. Let's see what happens when the source and the destination belong to two different segments, and how we connect the segments.

Connect two segments

Each segment has a unique name and boundary coordinates. We can easily identify which location (latitude, longitude) lies in which segment. Given the source and the destination, we can find the segments in which they lie. For each segment, there are some boundary edges, which we call exit points. In the illustration below, we have four exit points, S4E1, S4E2, S4E3, and S4E4.

'''

Note: Besides the vertices we have inside the segment, we also consider the exit points of a segment as vertices and calculate the shortest path for these exit points. So for each segment we calculate the shortest path between exit points in addition to the shortest path from the exit points to vertices inside the segment. Each vertex's shortest path information from the segment's exit points is cached.

'''

[Connecting the segments by considering exit points of a segment as vertices](./connection.jpg)

An exit point connects neighboring segments and is shared between them. In the illustration above, each exit point is connecting two segments and each is shared between two segments. For example, the exit point S4E1 is also an exit point for S1. Having all the exit points for each segment, we can connect the segments and find the shortest distance between two points in different segments. While connecting the segments, we don't care about the inside segment graph. We just need exit points and the cached information about the exit points. We can visualize it as a graph made up of exiting vertices, as shown in the following illustration.

[A graph made up of exit points, where the lines connecting the exit points are not actually straight. There could be many vertices in between the two exit points](./exit.jpg)

Since we can't run the shortest path algorithm for all the segments throughout the globe, it's critical to figure out how many segments we need to consider for our algorithm while traveling inter-segment. The aerial distance between the two places is used to limit the number of segments. With the source and destination points, we can find the aerial distance between them using the haversine formula.

'''

Aerial distance is the distance between two places measured in a straight line through the air rather than using roads on the ground.

'''

Suppose the aerial distance between the source and the destination is 10 kilometers. In that case, we can include segments that are at a distance of 10 kilometers from the source and destination in each direction. This is a significant improvement over the large graph.

[How](./shortest_path)

Let's summarize how we met the challenge of scalability. We divided our problem so that instead of working on a large road network as a whole, we worked on parts (segments) of it. The queries for a specific part of the road network are processed on that part only, and for the queries that require processing more than one part of the network, we connect those parts, as we have shown above.

Once the number of segments is limited, we can constrain our graph so that the vertices of the graph are the exit points of each segment, and the calculated paths between the exit points are the graph's edges. All we have to do now is run the shortest path algorithm on this graph to find the route. The following illustration shows how we find the path between two points that lie in different segments.

ETA computation

For computing the ETA with reasonable accuracy, we collect the live location data ((userID, timestamp, (latitude, longitude))) from the navigation service through a pub-sub system. With location data streams, we can calculate and predict traffic patterns on different roads. Some of the things that we can calculate are:

- Traffic (high/medium/low) on different routes or roads.
 - The average speed of a vehicle on different roads.
 - The time intervals during which a similar traffic pattern repeats itself on a route or road.
- For example, highway X will have high traffic between 8 to 10 AM.

The information above helps us provide a more accurate ETA. For example, if we know that the traffic will be high at a specific time on a particular road, the ETA should also be greater than usual.

In this lesson, we looked at how we meet the scalability challenge through segments and ETA computation using live data. In the next lesson, we'll discuss the design in more detail.

Design Google Maps/Design of Google Maps/README.md

Design of Google Maps

High-level design

Let's start with the high-level design of a map system. We split the discussion into two sections:

1. The components we'll need in our design.
2. The workflow that interconnects these components.

Components

We'll use the following components in our design:

- Location finder: The location finder is a service used to find the user's current location and show it on the map since we can't possibly personally remember the latitude and longitude for every place in the world.

- Route finder: For the people who are new to a place, it's difficult to travel because they don't know the correct routes. The route finder is a service used to find the paths between locations, or points.

- Navigator: Suggesting a route through the route finder is not enough. A user may deviate from the optimal path. In that case, a navigator service is used. This service keeps track of users' journeys and sends updated directions and notifications to the users as soon as they deviate from the suggested route.

- GPS/Wi-Fi/Cellular technology: These are the technologies that we used to find the user's ground position.

- Distributed search: For converting place names to latitude/longitude values, we need a conversion system behind the source and destination fields. A distributed search maintains an index consisting of places names and their mapping to latitude and longitude. User's entered keywords are searched using this service to find a location on the map.

- Area search service: The area search service coordinates between the distributed search and graph processing service to obtain the shortest path against a user query. The area search service will request the distributed search to obtain the locations of the source and destination on the map. Then, it will use the graph processing service to find the optimal path from the source to the destination.

- Graph processing service: There can be multiple paths from one place to another. The graph processing service runs the shortest path algorithm on a shorter graph based on the area spanning the source and destination points and helps us determine which path to follow.

- Database: As discussed in the previous lesson, we have the road data from various sources stored in the form of a graph. We'll map this data to a database to develop the road network graph. We're using a graph database like DataStax Graph to store the graph for our design.

- Pub-sub system: Users might deviate from the first suggested path. In that case, they'll need information on a new path to their destination. Pub-sub is a system that listens to various events from a service and triggers another service accordingly. For example, when a user deviates from the suggested path, it pings the area search service to find a new route from the user's current location to their destination point. It also collects the streams of location data for different users from the navigator. This data can be processed later to find traffic patterns on different roads at different times. We'll use Kafka as a pub-sub system in our design.

- Third-party road data: How can we build a map system if we don't have the road networks data? We need to collect the road data from third-party resources and preprocess the collected data to bring it into a single format that can be utilized to build the graph.

- Graph building: We'll use a service that builds the graph from the given data, either

collected from the third-party resources or from the users.

- User: This refers to a person or a program that uses the services of the map system.
- Load balancer: This is a system that is used to distribute user requests among different servers and services.

The illustration below shows how the above components are connected.

[A high-level design of a map system](./design.jpg)

Workflow

We explain the workflow by assuming that a user has to travel between two points but doesn't know their current location or how to get to their destination. So, let's see how the maps system helps.

'''

For this exercise, we assume that the data about road networks and maps has already been collected from the third parties and the graph is built and stored in the Graph DB.

'''

- In a maps system, the user has to enter their starting point and their destination to create a path between the two. For the starting source point, the user uses the current location service.

- The location finder determines the current location by maintaining a persistent connection with the user. The user will provide an updated location using GPS, Wi-Fi, and cellular technology. This will be the user's source point.

- For the destination point, the user types an address in text format. It's a good idea to make use of our Typeahead service here to provide useful suggestions and avoid spelling mistakes.

- After entering the source and the destination points, the user requests the optimal path.

- The user's path request is forwarded to the route-finder service.

- The route finder forwards the requests to an area search service with the source and the destination points.

- The area search service uses the distributed search to find the latitude/longitude for the source and the destination. It then calculates the area on the map spanning the two (source's and destination's) latitude/longitude points.

- After finding the area, the area search service asks the graph processing service to process part of the graph, depending on the area to find the optimal path.

- The graph processing service fetches the edges and nodes within that specified area from the database, finds the shortest path, and returns it to the route-finder service that visualizes the optimal path with the distance and time necessary to complete the route. It also displays the steps the user should follow for navigation.

- Now that the user can visualize the shortest path on the map, they also want to get directions towards the destination. The direction request is handled by the navigator.

- The navigator tracks that the user is following the correct path, which it has from the route-finder service. It updates the user's location on the map while the user is moving, and shows where to turn with the distance. If a user deviates from the path, it generates an event that is fed to Kafka.

- Upon receiving the event from the navigator, Kafka updates the subscribed topic of the area search service, which in turn recalculates the optimal path and suggests it to the user. The navigator also provides a stream of live location data to the graph, building it through the pub-sub system. Later, this data can be used to improve route suggestions provided to the

users.

API design

Let's look at different APIs for the maps service.

****Show the user's current location on the map****

The currLocation function displays the user's location on the map.

'''

currLocation(location)

'''

'''

Parameter	Description
-----------	-------------

location	This depicts whether the user location is on or off. This call will establish a persistent connection between the client and the server, where the client will periodically update the server about its current location. If the location is false, the service requests the user to turn on their location.
----------	--

'''

****Find the optimal route****

The findRoute function helps find the optimal route between two points.

'''

findRoute(source, destination, transport_type)

'''

'''

Parameter	Description
-----------	-------------

source	This is the place (in text format) where the users want to start their journey.
--------	---

destination	This is the place (in text format) where the users want to end their journey.
-------------	---

transport_type (optional)	This can be a bicycle, car, airplane, and so on. The default transport type is set to a car if the user doesn't provide this parameter.
---------------------------	---

'''

****Get directions****

The directions function helps us get alerts in the form of texts or sounds that indicate where to turn next and after we've traveled how far.

'''

directions(curr_location, steps)

'''

'''

Parameter	Description
-----------	-------------

current_location	This is the latitude/longitude value of the user's current location on the map.
------------------	---

steps	These are the steps the user should follow in order to reach their destination.
-------	---

'''

We described the high-level design by explaining the services we will need. We also discussed

API design. Next, weâ ll discuss how we met the scalability challenge through segments.

Design Google Maps/Detailed Design of Google Maps/README.md

Detailed Design of Google Maps

In this lesson, weâ ll discuss our detailed design by answering the following questions:

1. How do user requests benefit from segments?
2. How do we improve the user experience by increasing the accuracy of ETAs?

Segment setup and request handling

This section will describe how the segment data is stored in the database and how user requests are handled using the already stored data.

Starting with the storage schema, we discuss how the segments are added and hosted on the servers and also how the user requests are processed.

Storage schema

We store the following information for each segment:

****Key-value store:****

The segmentâ s ID.

The serverID on which the segment is hosted.

In reality, each segment is a polygon, so we store boundary coordinates (latitude/longitude), possibly as a list.

A list of segment IDs of the neighbors segments.

****Graph database****

The road network inside the segment in the form of a graph.

[Storage schema](./storage.jpg)

Relational DB

We store the information to determine whether, at a particular hour of the day, the roads are congested. This later helps us decide whether or not to update the graph (weights) based on the live data.

- edgeID identifies the edge.

- hourRange tells us which hour of the day it is when there are typical road conditions (non-rush hour) on the road.

- rush is a Boolean value that depicts whether there is congestion or not on a specific road at a specific time.

'''

Note:segID, serverID, and edgeID are unique IDs generated by a uniqueID generator (see the Sequencer lessons for details).

'''

Design

The following illustration consists of two workflows. One adds segments to the map and hosts them on the servers, while the other shows how the user request to find a path between two points is processed.

[Workflow 1) Adding segments (in yellow), and workflow 2) Processing user requests (in blue)](./design.jpg)

****Add segment****

1. Each segment has its latitude/longitude boundary coordinates and the graph of its road network.
2. The segment adder processes the request to add the segment along with the segment information. The segment adder assigns a unique ID to each segment using a unique ID generator system.
3. After assigning the ID to the segment, the segment adder forwards the segment information to the server allocator.
4. The server allocator assigns a server to the segment, hosts that segment graph on that server, and returns the serverID to the segment adder.
5. After the segment is assigned to the server, the segment adder stores the segment to server mapping in the key-value store. It helps in finding the appropriate servers to process user requests. It also stores each segment's boundary latitude/longitude coordinates in a separate key-value object.

****Handle the user's request****

1. The user provides the source and the destination so that our service can find the path between them.
2. The latitude and longitude of the source and the destination are determined through a distributed search.
3. The latitude/longitude for the source and the destination are passed to the graph processing service that finds the segments in which the source and the destination latitude/longitude lie from the key-value store.
4. After finding the segment IDs, the graph processing service finds the servers that are hosting these segments from the key-value store.
5. The graph processing service connects to the relevant servers to find the shortest path between the source and the destination. If the source and the destination belong to the same segment, the graph processing service returns the shortest path by running the query only on a single segment. Otherwise, it will connect the segments from different servers, as we have seen in the previous lesson.

Improve estimations using live data

This section describes how we can improve the ETA estimation accuracy using live data. If we have a sequence of location data for different devices, we can find movement patterns and perform analytics to predict various factors that may influence a user's trip. Google Maps uses a combination of GPS, Wi-Fi, and cell towers to track users' locations. To collect this data, our maps system servers need to have a persistent connection with all the devices that have their location turned on. Below, we discuss the tools, techniques, and components involved in the process of improving estimations using live data.

- WebSocket is a communication protocol that allows users and servers to have a two-way, interactive communication session. This helps in the real-time transfer of data between user and server.

- The load balancer balances the connection load between different servers since there is a limit on the number of WebSocket connections per server. It connects some devices to server 1, some to server 2, and so on.

[Performing analytics on the live location data to keep the map up-to-date and also improve ETA estimations](./analysis.jpg)

- A pub-sub system collects the location data streams (device, time, location) from all servers. The location data from pub-sub is read by a data analytics engine like Apache Spark. The data analytics engine uses data science techniques such as machine learning, clustering, and so on to measure and predict traffic on the roads, identify gatherings, hotspots, events, find out new roads, and so on. These analytics help our system improve ETAs.

'''

Note: The amount of traffic, road conditions, and hotspots directly affect average travel speed, which ultimately affects users' ETAs.

'''

- The data analytics engine publishes the analytics data to a new pub-sub topic.

- The map update service listens to the updates from the pub-sub topic for the analytics. It updates the segment graphs if there is a new road identified or if there is a change in the weight (average speed (traffic, road condition)) on the edges of the graph. Depending on the location, we know which segment the update belongs to. We find the routing server on which the segment is placed from the key-value store and update the graph on that server.

'''

Question

Many traffic conditions are transitory (like stopping at a signal), so updating the graph very often wouldn't scale well because it requires excessive processing. What could be the solution to this problem?

Answer

To detect transitory and normal conditions, we keep two copies of all relevant data (weights): one for normal conditions and the other for transitory conditions. That way, we won't have to do excessive processing.

'''

- The graph preprocessing service recalculates the new paths on the updated segment graph. We've seen how the paths are updated continuously in the background based on the live data. We've learned how the segments work, how a user finds the location between two points, and how the ETA's accuracy is improved by utilizing the live location data.

Design Google Maps/Evaluation of Google Maps' Design/README.md

Evaluation of Google Maps' Design

Let's see how the system we designed will handle millions of queries per second, ensuring a fast response time.

Availability

With a large road network graph hosted on a single server, we ran into these issues:

- We couldn't process user queries, since it was impossible to load such a large graph into memory, making the system unavailable to the users.
- It wasn't possible to make a persistent two-way connection (for navigation) between the server and millions of users per second.
- It was also a single point of failure.

We solved the above problems by dividing the world into small segments. Each small segment consists of a graph that can be easily loaded into a server's memory. With segments, we completed these objectives:

- We hosted each segment on a separate server, mitigating the issue of loading a large, global graph.
- The load balancer divides the request load across different segment servers depending on the user's area of search. It mitigates the issue of putting the burden on a single server, which was affecting the system's availability.
- We didn't discuss replication, but we can replicate each segment, which will help deal with segment server as a single point of failure and distribute the request load for a segment to replicas.

'''

Note: Google Maps uses lazy loading of data, putting less burden on the system, which improves availability.

'''

'''

Lazy loading is postponing the loading or initialization of resources or objects until they are needed. It reduces initial load time by reducing the amount of content to load, saves bandwidth by delivering content to users when required, and preserves server and client resources by

rendering only some of the content. Thus, it increases performance and saves system resources.
'''

Lazy loading reduces initial load time by reducing the amount of content to load, saves bandwidth by delivering content to users when needed, and preserves server and client resources by rendering only some of the content.

[Non-functional requirements fulfilled by our maps system]

Scalability

We scaled our system for large road networks. Scalability can be seen in two ways:

- The ability of the system to handle an increasing amount of user requests.
- The ability of the system to work with more data (segments).

We divided the world into small segments. Each segment is hosted on a different server in the distributed system. The user requests for different routes are served from the different segment servers. In this way, we can serve millions of user requests.
'''

Note: It wouldn't have been possible to serve millions of user requests if we had a single large graph spanning the whole road network. There would have been memory issues loading and processing a huge graph.
'''

We can also add more segments easily because we don't have to change the complete graph. We further improve scalability by non-uniformly selecting the size of a segment - selecting small segment sizes in densely connected areas and bigger segments for the outskirts.

Smaller response times

We're running the user requests on small subgraphs. Processing a small subgraph of hundreds of vertices is far faster than a graph of millions of vertices. We can cache the processed small subgraph in the main memory and quickly respond to user requests. This is how our system responds to the user in less time.

There's another aspect that helps our system to respond quickly, and that is keeping the segment information in the key-value store. The key-value store helps different services to get the required information quickly.

- The graph processing service checks for the relevant segments in which the source and the destination latitude/longitude lie by querying the key-value store for the segmentID values.
- For load-balancing user requests among different segment servers, the key-value store is queried for the serverID against the segment on which the graph processing should run for a specific request.

Accuracy

Besides the road data we had initially, we also captured the live location data of users, on which we performed analytics using data science techniques. Our system improves the accuracy of the results (path, ETA) using these analytics. Based on the analytics of the traffic patterns, the maps are updated, and the routes and ETA estimations are improved.
'''

Meeting Non-functional Requirements	
Requirements	Techniques
Availability	Process user queries on small graphs (segments). Load balance requests across different segment servers. Replicate segment servers.
Scalability	Partition the large graphs into small graphs to ease segment addition. Host the segments on different servers to enable serving more queries per second.
	Less response time Cache the processed graphs. Use a key-value store to quickly get the required information.
Accuracy	Collect live data. Perform data analytics.

Conclusion

Google Maps is one of the most widely used applications in the world, where users find the shortest route between two locations. A map system models the road network with a graph data structure. To find the route, the shortest path algorithm runs over the graph. Weâve seen scalability issues with a large graph of the road network. We solved the problem by splitting the world into small segments. Each segment consists of a small graph that can be loaded into the memory to find the paths quickly. Weâve also seen that the estimated time of arrival can be improved by analyzing the live location data.

Design Google Maps/Requirements of Google Maps' Design/README.md

Requirements of Google Maps' Design

Requirements

Before we start requirements, letâs clarify that we will design a system like Google Maps by picking a few key features because actual Google Maps is feature-rich and complex.

Letâs list the functional and non-functional requirements of the system under design.

Functional requirements

The functional requirements of our system are as follows.

- Identify the current location: Users should be able to approximate their current location (latitude and longitude in decimal values) on the world map.
- Recommend the fastest route: Given the source and destination (place names in text), the system should recommend the optimal route by distance and time, depending on the type of transportation.
- Give directions: Once the user has chosen the route, the system should list directions in text format, where each item in the list guides the user to turn or continue in a specific direction to reach the destination.

[Google Maps functional requirements](./fq.jpg)

Non-functional requirements

The non-functional requirements of our system are as follows.

- Availability: The system should be highly available.
- Scalability: It should be scalable because both individuals and other enterprise applications like Uber and Lyft use Google Maps to find appropriate routes.
- Less response time: It shouldnât take more than two or three seconds to calculate the ETA for the route, given the source and the destination points.
- Accuracy: The ETA we predict should not deviate too much from the actual travel time.

[Non-functional requirements of Google maps](./nfq.jpg)

Note: Weâre not getting into the details of how we get the data on roads and layout. Government agencies provide maps, and in some places, Google itself drives mapping vehicles to find roads and their intersection points. Road networks are modeled with a graph data structure, where intersection points are the vertices, and the roads between intersections are the weighted edges.

Challenges

Some of the challenges that we need to focus on while designing a system like Google Maps are below:

- Scalability: Serving millions of queries for different routes in a second, given a graph with billions of nodes and edges spanning over 194 countries, requires robust scalability measures. A simple approach, given the latitude and longitude of the source and destination, would be to apply an algorithm like Dijkstra to find the shortest path between the source and the

destination. However, this approach wouldn't scale well for billions of users sending millions of queries per second. This is because running any path-finding algorithm on a graph with billions of nodes running a million times per second is inefficient in terms of time and cost, ultimately leading to a bad user experience. Therefore, our solution needs to find alternative techniques to scale well.

[Spanning](./spanning.jpg)

- ETA computation: In an ideal situation with empty roads, it's straightforward to compute ETA using the distance and the speed of the vehicle we want to ride on. However, we cannot ignore factors like the amount of traffic on the roads and road conditions, which affect the ETA directly. For example, a road under construction, collisions, and rush hours all might slow down traffic. Quantifying the factors above to design our system is not trivial. Therefore, we'll categorize the factors above in terms of traffic load to complete our design.

[Factors affecting the ETA computation](./factors.jpg)

Resource estimation

Let's estimate the total number of servers, storage, and bandwidth required by the system.

Number of servers estimation

To estimate the number of servers, we need to know how many daily active users are using Google Maps and how many requests per second a single Google Maps server can handle. We assume the following numbers:

- Daily active users who use Google Maps: 32 million (about 1 billion monthly users).
- Number of requests a single server can handle per second: 8,000.

The number of servers required has been calculated using the below formula:

Number of active users/requests handled per server = 4K servers

[Number of servers required for Google Maps](./servers.jpg)

Storage estimation

Google Maps is essentially a system with a one-time storage requirement. The road data from many countries has already been added, which is over 20 petabytes as of 2022. Since there are minimal changes on the road networks, the daily storage requirement is going to be negligible for Google Maps. Also, short-term changes in the road network is a small amount of data as compared to the full network data. Therefore, our storage needs don't change rapidly.

Bandwidth estimation

As a standard practice, we have to estimate the bandwidth required for the incoming and outgoing traffic of our system. Most of the bandwidth requirements for Google Maps are due to requests sent by users. Therefore, we've devised the following formula to calculate bandwidth

Total bandwidth = Total requests_second ^ Total query_size

The Total_{requests_second} represents the number of requests per second, whereas the Total_{query_size} represents the size of each request.

Incoming Traffic

To estimate the incoming query traffic bandwidth, we assume the following numbers:

- Maximum number of requests by a single user per day: 50.
- Request size (source and destination): 200 Bytes.

Using the assumptions above, we can estimate the total number of requests per second on Google Maps using the following formula:

Total requests_second = Daily active users ^ Requests per_user / (24 x 60 x 60) = 18,518

requests per second.

We can calculate the incoming query traffic bandwidth required by Google Maps by inserting the request per second calculated above and the size of each request in the aforementioned bandwidth formula.

'''

Bandwidth Required for Incoming Query Traffic		
No. of requests per second	Request size (Bytes)	Bandwidth (Mb/s)
18518	200	29.63

'''

****Outgoing traffic****

Outgoing application traffic will include the response that the server generates for the user when they make a navigation request. The response consists of visuals and textual content, and typically includes the route shown on the map, estimated time, distance, and more detail about each step in the route. We assume the following numbers to estimate the outgoing traffic bandwidth:

- Total requests per second (calculated above): 18,518.
- Response size: 2 MB + 5 KB = 2005 KB.
 - Size of visual data on average: 2 MB.
 - Size of textual data on average: 5 KB.

We can calculate the bandwidth required for outgoing traffic using the same formula.

'''

Bandwidth Required for the Outgoing Application Traffic		
No. of requests per second	Response size (KB)	Bandwidth (Gb/s)
18518	2005	297.03

'''

'''

Total bandwidth = Total requests_second ^ Total query_size = 18518 ^ 2005KB ^ 8 bits = 297.03Gbps

'''

[Summarizing the bandwidth requirements for Google Maps](./bandwidth.jpg)

Building blocks we will use

Now that we've completed our estimates of resources required, let's identify the building blocks that will be an integral part of our design for the Google Maps system. Below, we have the key building blocks:

[Building blocks used in the high-level and detailed design](./bb.jpg)

- Load balancers are necessary to distribute user requests among different servers and services.
- Databases are required to store data in the form of a graph along with metadata information.
- Distributed search is needed to search different places on the map.
- A pub-sub system is required for generating and responding to important events during navigation and notifying the corresponding services.
- A key-value store is also used to store some metadata information.

Besides the building blocks mentioned above, other components will also be required for designing our maps system. These components will be discussed in the design lessons. We are now ready to explore the system and API design of Google Maps.

Design Google Maps/System Design Google Maps/README.md

System Design: Google Maps

What is Google Maps?

Let's introduce the problem by assuming that we want to travel from one place to another. Here are the possible things that we might want to know:

- What are the best possible paths that take us to our destination, depending on the vehicle type we're using?
- How long in miles is each path?
- How much time does each path take to get us to our destination?

A maps application (like Google Maps or Apple Maps) enables users to answer all of the above questions easily. The following illustration shows the paths calculated by Google maps from Los Angeles, USA to New York City, USA.

[Three paths suggested by Google maps for traveling from Los Angeles to New York](./map.jpg)

When do we use a maps service?

Individuals and organizations rely on location data to navigate around the world. Maps help in these cases:

- Individuals can find the locations of and directions to new places quickly, instead of wasting their time and the costs of travel, such as gas.
- Individuals use maps to see their estimated time of arrival (ETA) and the shortest path based on current traffic data.
- Many modern applications rely heavily on maps, such as ride-hailing services, autonomous vehicles, and hiking maps. For example:
 - Waymo's self-driving car system uses Google Maps to navigate efficiently, quickly, and safely.
 - Uber uses Google Maps as part of its application to assist drivers and provide customers with a visual representation of their journey.
 - Routing and logistics-based companies reduce the time it takes to make deliveries. By using map's unique real-time and historical traffic data, it minimizes the overall cost of delivery by reducing gas usage and time spent stuck in traffic.

In 2022, more than five million businesses are using Google Maps. It provides an API for enterprises to use a map system in their application.

How will we design Google Maps?

We divide the design of Google Maps into five lessons:

1. Requirements: In this lesson, we'll list the functional and non-functional requirements of Google Maps system. We will also identify the challenges involved in designing such a system. Lastly, we'll estimate the resources like servers and bandwidth needed to serve queries by millions of users.
2. Design: This lesson consists of the high-level and API design of a system like Google maps. We'll describe the services and the workflow of the system.
3. Meeting the challenges: We will discuss how we overcome the challenges that we highlighted in the requirements lesson.
4. Detailed design: Based on the solution to the challenges, we will improve our earlier design and also elaborate on different aspects of it. We will describe the detailed design, including storage schema.
5. Evaluation: This lesson explains how our designed Google Maps system fulfills all the requirements.

Let's start by understanding the requirements for designing a system like Google Maps.

Design Instagram/Design of Instagram/README.md

Design of Instagram

High-level design

Our system should allow us to upload, view, and search images and videos at a high level. To upload images and videos, we need to store them, and upon fetching, we need to retrieve the data from the storage. Moreover, the users should also be allowed to follow each other.

[The high-level design of Instagram](./hld.jpg)

API design

This section describes APIs invoked by the users to perform different tasks (upload, like, and view photos/videos) on Instagram. Weâll implement REST APIs for these tasks. Letâs develop for each of the following features:

- Post photos and videos
- Follow and unfollow users
- Like or dislike posts
- Search photos and videos
- Generate a news feed

All of the following calls will have a `userID`, that uniquely specifies the user performing the action. Weâll only discuss new parameters in the calls.

Post photos or videos

The POST method is used to post photos/videos to the server from the user through the `/postMedia` API. The `/postMedia` API is as follows:

'''

`postMedia(userID, media_type, list_of_hashtags, caption)`

'''

'''

Parameter	Description
-----------	-------------

<code>media_type</code>	It indicates the type of media (photo or video) in a post.
-------------------------	--

<code>list_of_hashtags</code>	It represents all hashtags (maximum limit 30 hashtags) in a post.
-------------------------------	---

<code>caption</code>	It is a text (maximum limit is 2,200 characters) in a userâs post.
----------------------	--

'''

Follow and unfollow users

The `/followUser` API is used when a user follows other users on Instagram. The `/followUser` API is as follows:

'''

`followUser(userID, target_userID)`

'''

'''

Parameter	Description
-----------	-------------

<code>target_userID</code>	It indicates the user to be followed.
----------------------------	---------------------------------------

'''

The `/unfollowUser` API uses the same parameters when a user unfollows someone on Instagram.

Like or dislike posts

The `/likePost` API is used when users like someoneâs post on Instagram.

'''

`likePost(userID, target_userID, post_id)`

'''

'''

Parameter	Description
-----------	-------------

<code>target_userID</code>	It specifies the user whose post is liked.
----------------------------	--

<code>post_id</code>	It specifies the postâs unique ID.
----------------------	--------------------------------------

'''

The `/dislikePost` API uses the same parameters when a user dislikes someoneâs post on Instagram.

Search photos or videos

The GET method is used when the user searches any photos or videos using a keyword or hashtag.

The /searchPhotos API is as follows:

'''

searchPhotos(userID, keyword)

'''

'''

Parameter	Description
-----------	-------------

keyword	It indicates the string (username, hashtag, and places) typed by the user in the search bar.
---------	--

'''

'''

Note: Instagram shows the posts with the highest reach (those with more likes and views) upon searching for a particular key. For example, if a user does a location-based search using "London, United Kingdom," Instagram will show the posts in order with maximum to minimum reach. Instead of showing all the posts, the data will be loaded upon scrolling.

'''

Generate news feed

The GET method is used when users view their news feed through the /viewNewsfeed API. The /viewNewsfeed API is as follows:

'''

viewNewsfeed(userID, generate_timeline)

'''

'''

Parameter	Description
-----------	-------------

generate_timeline	It indicates the time when a user requests news feed generation. Instagram shows the posts that are not seen by the user between the last news feed request and the current news feed request.
-------------------	--

'''

Storage schema

Let's define our data model now:

Relational or non-relational database

It is essential to choose the right kind of database for our Instagram system, but which is the right choice - SQL or NoSQL? Our data is inherently relational, and we need an order for the data (posts should appear in chronological order) and no data loss even in case of failures (data durability). Moreover, in our case, we would benefit from relational queries like fetching the followers or images based on a user ID. Hence, SQL-based databases fulfill these requirements.

So, we'll opt for a relational database and store our relevant data in that database.

Define tables

On a basic level, we need the following tables:

- Users: This stores all user-related data such as ID, name, email, bio, location, date of account creation, time of the last login, and so on.

- Followers: This stores the relations of users. In Instagram, we have a unidirectional relationship, for example, if user A accepts a follow request from user B, user B can view user A's posts.

It's post, but vice versa is not valid.

- Photos: This stores all photo-related information such as ID, location, caption, time of creation, and so on. We also need to keep the user ID to determine which photo belongs to which user. The user ID is a foreign key from the users table.

- Videos: This stores all video-related information such as ID, location, caption, time of creation, and so on. We also need to keep the user ID to determine which video belongs to which user. The user ID is a foreign key from the users table.

'''

Question

Where should we store the photos and videos?

Answer

We'll store the photos and videos in a blob store (like S3) and save the path of the photo or video in the table as it is efficient to save larger data in a distributed storage.

'''

The following illustration visualizes the data model:

[The Data model of Instagram](./model.jpg)

Data estimation

Let's figure out how much data each table will store. The column per row size (in bytes) in the following calculator shows the data per row of each table. It also calculates the storage needed for the specified counts. For example, the storage required for 500 million users is 111000 MB, 2000 MB for 250 followers of a single user, 23640 MB for 60 million photos, and 13790 MB for 35 million videos.

You can change the values in the calculator to observe the change in storage needed. This gives us an estimate of how fast data will increase in our tables.

'''

Table Name	Per row size (in bytes)	Count in Millions	Storage Needed (in MBs)
Users	222	500	111000
Followers	8	250	2000
Photos	394	60	23640
Videos	394	35	13790

'''

'''

Note: Most modern services use both SQL and NoSQL stores. Instagram officially uses a combination of SQL (PostgreSQL) and No-SQL (Cassandra) databases. The loosely structured data like timeline generation is usually stored in No-SQL, while relational data is saved in SQL-based storage.

'''

In the next lesson, we'll identify more components to tweak our design.

Design Instagram/Detailed Design of Instagram/README.md

Detailed Design of Instagram

Add more components

Let's add a few more components to our design:

- Load balancer: To balance the load of the requests from the end-users.

- Application servers: To host our service to the end-users.
- Relational database: To store our data.
- Blob storage: To store the photos and videos uploaded by the users.

[Adding components to design](./components.jpg)

Upload, view, and search a photo

The client requests to upload the photo, load balancer passes the request to any of the application servers, which adds an entry to the database. An update that the photo is stored successfully is sent to the user. If an error is encountered, the user is communicated about it as well.

The photo viewing process is also similar to the above-mentioned flow. The client requests to view a photo, and an appropriate photo that matches the request is fetched from the database and shown to the user. The client can also provide a keyword to search for a specific image.

The read requests are more than write requests and it takes time to upload the content in the system. It is efficient if we separate the write (uploads) and read services. The multiple services operated by many servers handle the relevant requests. The read service performs the tasks of fetching the required content for the user, while the write service helps upload content to the system.

We also need to cache the data to handle millions of reads. It improves the user experience by making the fetching process fast. We'll also opt for lazy loading, which minimizes the client waiting time. It allows us to load the content when the user scrolls and therefore save the bandwidth and focus on loading the content the user is currently viewing. It improves the latency to view or search a particular photo or video on Instagram.

'''

Lazy loading contributes to efficiency in the program's operation. It is ideal in use cases where network content is accessed and initialization times are to be kept at a minimum, such as in the case of web pages. For example, deferring the loading of images on a web page until they are needed can make the initial display of the web page faster. Source: Wikipedia

'''

The updated design is as follows:

[Various operations on photos](./photos.jpg)

Generate a timeline

Now our task is to generate a user-specific timeline. Let's explore various approaches and their advantages and disadvantages to opt for the appropriate strategy.

The pull approach

When a user opens their Instagram, we send a request for timeline generation. First, we fetch the list of people the user follows, get the photos they recently posted, store them in queues and display them to the user. But this approach is slow to respond as we generate a timeline every time the user opens Instagram.

We can substantially reduce user-perceived latency by generating the timeline offline. For example, we define a service that fetches the relevant data for the user before, and as the person opens Instagram, it displays the timeline. This decreases the latency rate to show the timeline. Let's take a look at the slides below to understand the problem and its solution.

[Pull](./pull)

'''

Question

What are the shortcomings of the pull approach?

Answer

Instagram is a read-heavy system. Many people don't post any photos and instead just view others' posts. So, the calls we make to fetch the recent posts from every follower will usually return nothing. Hence, we should keep in mind that it is not a write-heavy system and should develop a possible solution to cater to it.

'''

The push approach

In a push approach, every user is responsible for pushing the content they posted to the people's timelines who are following them. In the previous approach, we pulled the post from each follower, but in the current approach, we push the post to each follower.

Now we only need to fetch the data that is pushed towards that particular user to generate the timeline. The push approach has stopped a lot of requests that return empty results when followed users have no post in a specified time.

[Push-based approach](./push.jpg)

'''

Question

What are the shortcomings of the push approach?

Answer

Consider an account that belongs to a celebrity, like Cristiano Ronaldo, who has over 400 million followers. So if he posts a photo or a video, we will push the links of the photo/video to 400 million+ users, which is inefficient.

'''

Hybrid approach

Let's split our users into two categories:

- Push-based users: The users who have a followers count of hundreds or thousands.
- Pull-based users: The users who are celebrities and have followers count of a hundred thousand or millions.

The timeline service pulls the data from pull-based followers and adds it to the user's timeline. The push-based users push their posts to the timeline service of their followers so the timeline service can add to the user's timeline.

We have used the method which generates the timeline, but where do we store the timeline? We store a user's timeline against a userID in a key-value store. Upon request, we fetch the data from the key-value store and show it to the user. The key is userID, while the value is timeline content (links to photos and videos). Because the storage size of the value is often limited to a few MegaBytes, we can store the timeline data in a blob and put the link to the blob in the value of the key as we approach the size limit.

We can add a new feature called story to our Instagram. In the story feature, the users can add a photo that stays available for others to view for 24 hours only. We can do this by maintaining an option in the table where we can store a story's duration. We can set it to 24 hours, and the task scheduler deletes the entries whose time exceeds the 24 hours limit.

Finalized design

We'll also use CDN (content delivery network) in our design. We can keep images and videos of celebrities in CDN which make it easier for the followers to fetch them. The load balancer first routes the read request to the nearest CDN, if the requested content is not available there, then it forwards the request to the particular read application server (see the load balancing chapter for the details). The CDN helps our system to be available to millions of

concurrent users and minimizes latency.

The final design is given below:

[The final design of Instagram](./final.jpg)

'''

Question

How can we count millions of interactions (like or view) on a celebrity post?

Answer

We can use sharded counters to count the number of multiple interactions for a particular user. Each counter has a number of shards distributed across various edge servers to reduce the load on the application server and latency. Users nearest to the edge server get the updated count frequently on a specific post compared to those in distant regions.

'''

Ensure non-functional requirements

We evaluate the Instagram design with respect to its non-functional requirements:

- Scalability: We can add more servers to application service layers to make the scalability better and handle numerous requests from the clients. We can also increase the number of databases to store the growing usersâ data.

- Latency: The use of cache and CDNs have reduced the content fetching time.

- Availability: We have made the system available to the users by using the storage and databases that are replicated across the globe.

- Durability: We have persistent storage that maintains the backup of the data so any uploaded content (photos and videos) never gets lost.

- Consistency: We have used storage like blob stores and databases to keep our data consistent globally.

- Reliability: Our databases handle replication and redundancy, so our system stays reliable and data is not lost. The load balancing layer routes requests around failed servers.

Conclusion

This design problem highlights that we can provide major services by connecting our building blocks appropriately. The scalable and fault-tolerant building blocks enable us to concentrate on use-case-specific issues (such as the efficient formation of timelines).

Design Instagram/Quiz on Instagramâs Design/README.md

Quiz on Instagramâs Design

'''

1 If we opt for a NoSQL database to store Instagram data, which of the non-functional requirements would be easier to fulfill?

A)

Availability

B)

Scalability

C)

Reliability

D)

Latency
'''

'''

2 Which of the following options is correct to generate photos and videos for the user's newsfeed?

A)

Photos or videos posted between the last and the current news feed generation request

B)

Photos or videos posted between the first and the current news feed generation request

C)

Photos or videos of the last 24 hours

D)

None of the above

'''

'''

3 Which approach does our design use to generate the user's timeline?

A)

Pull-based

B)

Push-based

C)

Hybrid

D)

None of the above

'''

Answers

'''

1

If we opt for a NoSQL database to store Instagram data, which of the non-functional requirements would be easier to fulfill?

A)

Availability

Correct Answer

B)

Scalability

Explanation

The No-SQL database is easier to scale as compared to traditional relational databases.

C)

Reliability

D)

Latency

'''

2 (A)

3 (C)

Design Instagram/Requirements of Instagram s Design/README.md

Requirements of Instagram s Design

Requirements

We'll concentrate on some important features of Instagram to make this design simple. Let's list down the requirements for our system:

Functional requirements

- Post photos and videos: The users can post photos and videos on Instagram.
- Follow and unfollow users: The users can follow and unfollow other users on Instagram.
- Like or dislike posts: The users can like or dislike posts of the accounts they follow.
- Search photos and videos: The users can search photos and videos based on captions and location.
- Generate news feed: The users can view the news feed consisting of the photos and videos (in chronological order) from all the users they follow. Users can also view suggested and promoted photos in their news feed.

Non-functional requirements

- Scalability: The system should be scalable to handle millions of users in terms of computational resources and storage.
- Latency: The latency to generate a news feed should be low.
- Availability: The system should be highly available.
- Durability Any uploaded content (photos and videos) should never get lost.
- Consistency: We can compromise a little on consistency. It is acceptable if the content (photos or videos) takes time to show in followers's feeds located in a distant region.
- Reliability: The system must be able to tolerate hardware and software failures.

Resource estimation

Our system is read-heavy because service users spend substantially more time browsing the feed of others than creating and posting new content. Our focus will be to design a system that can fetch the photos and videos on time. There is no restriction on the number of photos or videos that users can upload, meaning efficient storage management should be a primary consideration in the design of this system. Instagram supports around a billion users globally who share 95 million photos and videos on Instagram per day. We'll calculate the resources and design our system based on the requirements.

Let's assume the following:

- We have 1 billion users, with 500 million as daily active users.
- Assume 60 million photos and 35 million videos are shared on Instagram per day.
- We can consider 3 MB as the maximum size of each photo and 150 MB as the maximum size of each video uploaded on Instagram.
- On average, each user sends 20 requests (of any type) per day to our service.

Storage estimation

We need to estimate the storage capacity, bandwidth, and the number of servers to support such an enormous number of users and content.

The storage per day will be:

60 million photos/day * 3 MB = 180 TeraBytes / day

35 million videos/day * 150 MB = 5250 TB / day

Total content size = 180 + 5250 = 5430 TB

The total space required for a year:

5430 TB/day * 365 (days a year) = 1981950 TB = 1981.95 PetaBytes

[Total storage required by the Instagram storage system for a year](./storage.jpg)

Besides photos and videos, we have ignored comments, status sharing data, and so on. Moreover, we also have to store users' information and post metadata, for example, userID, photo, and on. So, to be precise, we need more than 5430 TB/day, but to keep our design simple, let's stick to the 5430 TB/day.

Bandwidth estimation

According to the estimate of our storage capacity, our service will get 5430 TB of data each day, which will give us:

5430 TB/(24 * 60 * 60) = 5430 TB/86400 sec ≈ 62.84 GB/s ≈ 502.8 Gbps

Since each incoming photo and video needs to reach the users' followers, let's say the ratio of readers to writers is 100:1. As a result, we need 100 times more bandwidth than incoming bandwidth. Let's assume the following:

Incoming bandwidth ≈ 502.8 Gbps

Required outgoing bandwidth ≈ 100 * 502.8 Gbps ≈ 50.28 Tbps

[Total bandwidth required](./bandwidth.jpg)

Number of servers estimation

We need to handle concurrent requests coming from 500 million daily active users. Let's assume that a typical Instagram server handles 100 requests per second:

Requests by each user per day = 20 Queries handled by a server per second = 100

Queries handled by a server per day = 100 * 60 * 60 * 24 = 8640000

Number of active users' Requests by each user per day / Queries handled per server a day = 1157 servers approx

This calculation shows that we need 1157 servers to handle queries in our Instagram system.

[Number of servers required for the Instagram system](./servers.jpg)

Try it yourself

Let's analyze how the number of photos per day affects the storage and bandwidth requirements. For this purpose, try to change values in the following table to compute the estimates. We have set 20 requests by each user per day in the following calculation:

```
'''
    Requirements  Calculations
Number of photos per day (in millions)          60
Size of each photo (in MB)                      3
Number of videos per day (in millions)          35
Size of each video (in MB)                     150
Number of daily active users (in millions)      500
Number of requests a server can handle per day  8640000
Storage estimation per day (in TB)              5430
Incoming bandwidth (Gb/s)                      502.8
Number of servers needed                       1157
'''
```

Building blocks we will use

In the next lesson, weâ ll focus on the high-level design of Instagram. The design will utilize many building blocks that have been discussed in the initial chapters also. Weâ ll use the following building blocks in our design:

[Building blocks used in Instagram design](./bb.jpg)

- A load balancer at various layers will ensure smooth requests distribution among available servers.
- A database is used to store the user and accounts metadata and relationship among them.
- Blob storage is needed to store the various types of content such as photos, videos, and so on.
- A task scheduler schedules the events on the database such as removing the entries whose time to live exceeds the limit.
- A cache stores the most frequent content related requests.
- CDN is used to effectively deliver content to end-users which reduces delay and burden on end-servers.

In the next lesson, weâ ll discuss the high-level design of the Instagram system.

Design Instagram/System Design Instagram/README.md

System Design: Instagram

What is Instagram?

Instagram is a free social networking application that allows users to post photos and short videos. Users can add a caption for each post and utilize hashtags or location-based geotags to index them and make them searchable within the application. A userâ s Instagram posts display their followersâ feeds and can be seen by the general public if they are tagged with hashtag or geotags. Alternatively, users can choose to make their profile private, which limits access to those who have chosen to follow them.

The expansion in the number of users requires more resources (servers, databases, and so on) over time. Knowing the usersâ growth rate helps us predict the resources to scale our system accordingly. The following illustration shows the Instagram user base in different countries as of January 2022 (source: Statista).

[Countries with the most Instagram users](./countries.jpg)

How will we design Instagram?

We have divided the design of Instagram into four lessons:

1. Requirements: This lesson will put forth the functional and non-functional requirements of Instagram. It will also estimate the resources required to achieve these requirements.
2. Design: This lesson will explain the workflow and usage of each component, API design and database schema.
3. Detailed design: In this lesson, weâ ll explore the components of our Instagram design in detail and discuss various approaches to generate timelines. Moreover, weâ ll also evaluate our proposed design.

4. Quiz: This lesson will test our understanding of the Instagram design. Let's start by understanding the requirements for designing our Instagram system and provide resource estimations in the next lesson.

Design Newsfeed System/Design of a Newsfeed System/README.md

Design of a Newsfeed System

Let's discuss the high-level and detailed design of a newsfeed system based on the requirements discussed in the previous lesson.

High-level design of a newsfeed system

Primarily, the newsfeed system is responsible for the following two tasks:

1. Feed generation: The newsfeed is generated by aggregating friends' and followers' posts (feed items) based on some ranking mechanism.
2. Feed publishing: When a feed is published, the relevant data is written into the cache and database. This data could be textual or any media content. A post containing the data from friends and followers is populated to a user's newsfeed.

Let's move to the high-level design of our newsfeed system. It consists of the above two essential parts, shown in the following figure:

[High-level design of the Newsfeed system](./hld.jpg)

Let's discuss the main components shown in the high-level design:

1. User(s): Users can make a post with some content or request their newsfeed.
2. Load balancer: It redirects traffic to one of the web servers.
3. Web servers: The web servers encapsulate the back-end services and work as an intermediate layer between users and various services. Apart from enforcing authentication and rate-limiting, web servers are responsible to redirect traffic to other back-end services.
4. Notification service: It informs the newsfeed generation service whenever a new post is available from one's friends or followers, and sends a push notification.
5. Newsfeed generation service: This service generates newsfeeds from the posts of followers/friends of a user and keeps them in the newsfeed cache.
6. Newsfeed publishing service: This service is responsible for publishing newsfeeds to a user's timeline from the newsfeed cache. It also appends a thumbnail of the media content from the blob storage and its link to the newsfeed intended for a user.
7. Post-service: Whenever a user requests to create a post, the post-service is called, and the created post is stored on the post database and corresponding cache. The media content in the post is stored in the blob storage.

API design

APIs are the primary ways for clients to communicate with servers. Usually, newsfeed APIs are HTTP-based that allow clients to perform actions, including posting a status, retrieving newsfeeds, adding friends, and so on. We aim to generate and get a user's newsfeed; therefore, the following APIs are essential:

Generate user's newsfeed

The following API is used to generate a user's newsfeed:

```
'''
```

```
generateNewsfeed(user_id)
```

```
'''
```

This API takes usersâ€™ IDs, and determines their friends and followers. This API generates newsfeeds that consist of several posts. Since internal system components use this API, therefore, it can be called offline to pre-generate newsfeeds for users. The pre-generated newsfeeds are stored on persistent storage and associated cache.

The following parameter is used in this API call:

'''

Parameter	Description
-----------	-------------

user_id	A unique identification of the user for whom the newsfeed is generated.
---------	---

'''

Get userâ€™s newsfeed

The following API is used to get a userâ€™s newsfeed:

'''

getNewsfeed(user_id, count)

'''

The getNewsfeed(.) API call returns a JSON object consisting of a list of posts.

The following parameters are used for this API:

'''

Parameter	Description
-----------	-------------

user_id	A unique identification of the user for whom the system will fetch the newsfeed.
---------	--

count	The number of feed items (posts) that will be retrieved per request.
-------	--

'''

Storage schema

The database relations for the newsfeed system are as follows:

- User: This relation contains data about a user. A user can also be a follower or friend of other users.
- Entity: This relation stores data related to any entity, such as pages, groups, and so on.
- Feed_item: The data about posts created by users is stored in this relation.
- Media: The information about the media content is stored in this relation.

[The database schema for the newsfeed system](./schema.jpg)

We use a graph database to store relationships between users, friends, and followers. For this purpose, we follow the property graph model. We can think of a graph database consisting of two relational tables:

1. For vertices that represent users
2. For edges that denotes relationships among them

Therefore, we follow a relational schema for the graph store, as shown in the following figure. The schema uses the PostgreSQL JSON datatype to store the properties of each vertex (user) or edge (relationship).

'''

In the property graph model, connections (edges) carry a name and some properties that represent the relationship between two entities.

'''

An alternative representation of a User can be shown in the graph database below. Where the

Users_ID remains the same and attributes are stored in a JSON file format.

[Schema of the graph database to hold relationships between users](./user_relationship.jpg)

The following figure demonstrates how graph can be represented using the relational schema:

[A graph between two users consisting of two vertices and an edge](./relationship.jpg)

Detailed design

Let's explore the design of the newsfeed system in detail.

As discussed earlier, there are two parts of the newsfeed system; newsfeed publishing and newsfeed generation. Therefore, we'll discuss both parts, starting with the newsfeed generation service.

The newsfeed generation service

Newsfeed is generated by aggregated posts (or feed items) from the user's friends, followers and other entities (pages and groups).

In our proposed design, the newsfeed generation service is responsible for generating the newsfeed. When a request from a user (say Alice) to retrieve a newsfeed is received at web servers, the web server either:

- Calls the newsfeed generation service to generate feeds because some users don't often visit the platform, so their feeds are generated on their request.
- It fetches the pre-generated newsfeed for active users who visit the platform frequently.

The following steps are performed in sequence to generate a newsfeed for Alice:

1. The newsfeed generation service retrieves IDs of all users and entities that Alice follows from the graph database.
2. When the IDs are retrieved from the graph database, the next step is to get their friends' (followers and entities) information from the user cache, which is regularly updated whenever the users database gets updated/modified.
3. In this step, the service retrieves the latest, most popular, and relevant posts for those IDs from the post cache. These are the posts that we might be able to display on Alice's newsfeed.
4. The ranking service ranks posts based on their relevance to Alice. This represents Alice's current newsfeed.
5. The newsfeed is stored in the newsfeed cache from which top N posts are published to Alice's timeline. (The publishing process is discussed in detail in the following section).
6. In the end, whenever Alice reaches the end of her timeline, the next top N posts are fetched to her screen from the newsfeed cache.

The process is illustrated in the following figure:

[Working of the newsfeed generation service](./newsfeed_generation.jpg)

'''

Question

The creation and storage of newsfeeds for each user in the cache requires an enormous amount of memory (step 5 in the above section). Is there any way to reduce this memory consumption?

Answer

A memory-efficient way would be to store just the mapping between users and their corresponding posts in a table in the cache, that is., < Post_ID, User_ID>. During the feed publishing phase the system will retrieve posts from the post database and generate the newsfeed for a user who follows another user with User_ID.

'''

The newsfeed publishing service

At this stage, the newsfeeds are generated for users from their respective friends, followers, and entities and are stored in the form of <Post_ID, User_ID> in the news feed cache.

Now the question is how the newsfeeds generated for Alice will be published to her timeline?

The newsfeed publishing service fetches a list of post IDs from the newsfeed cache. The data fetched from the newsfeed cache is a tuple of post and user IDs, that is., <Post_ID, User_ID>. Therefore, the complete data about posts and users are retrieved from the users and posts cache to create an entirely constructed newsfeed.

In the last step, the fully constructed newsfeed is sent to the client (Alice) using one of the fan-out approaches. The popular newsfeed and media content are also stored in CDN for fast retrieval.

'''

Suppose if our service receives billions of requests at once and the system starts generating live newsfeeds. This situation would put an enormous number of users on hold and could crash servers. To avoid this, we can assign dedicated servers that will continuously rank and generate newsfeeds and store them in the newsfeed database and memory. Therefore, upon a request for new posts from a user, our system will provide the pre-generated feeds.

'''

[The newsfeed publishing service in action](./newfeed_publishing.jpg)

'''

Question

How is the newsfeed of the friends and followers updated when a user creates a new post?

Answer

When a user, say Bob, creates a new post, it is stored in the post database and cache. In the next step, the newsfeed generation service generates a newsfeed for Bob's friends and followers, and the newsfeed cache is updated. The updated newsfeed is delivered to the corresponding users on the next page/screen refresh event.

'''

The newsfeed ranking service

Often we see the relevant and important posts on the top of our newsfeed whenever we log in to our social media accounts. This ranking involves multiple advanced ranking and recommendation algorithms.

In our design, the newsfeed ranking service consists of these algorithms working on various features, such as, a user's past history, likes, dislikes, comments, clicks, and many more. These algorithms also perform the following functions:

- Select a candidates' posts to show in a newsfeed.
- Eliminate posts including misinformation or clickbait from the candidate posts.
- Create a list of friends a user frequently interacts with.
- Choose topics on which a user spent more time.

The ranking system considers all the above points to predict relevant and important posts for user.

[The ranked newsfeeds are stored in the cache servers](./ranked_newsfeed.jpg)

Posts ranking and newsfeed construction

The post database contains posts published by different users. Assume that there are 10 posts in the database published by 5 different users. We aim to rank only 4 posts out of 10 for a user (say Bob) who follows those five different users. We perform the following to rank each post and create a newsfeed for Bob:

1. Various features such as likes, comments, shares, category, duration, etc and so on, are

- extracted from each post.
2. Based on Bob's previous history, stored in the user database, the relevance is calculated for each post via different ranking and machine learning algorithms.
3. A relevance score is assigned, say from 1 to 5, where 1 shows the least relevant post and 5 means highly relevant post.
4. The top 4 posts are selected out of 10 based on the assigned scores.
5. The top 4 posts are combined and presented on Bob's timeline in decreasing order of the score assigned.

The following figure shows the top 4 posts published on Bob's timeline:

[A newsfeed consisting of top 4 posts based on the relevance scores](./newsfeed_example.jpg)

Newsfeed ranking with various machine learning and ranking algorithms is a computationally intensive task. In our design, the ranking service ranks posts and constructs newsfeeds. This service consists of big-data processing systems that might utilize specialized hardware like graphics processing units (GPUs) and tensor processing units (TPUs).

'''

Note: According to Facebook:

â For each person on Facebook, we need to evaluate thousands of features to determine what the person might find most relevant to predict what each of those people wants to see in their feed.â

This implies that we need enormous computational power and sophisticated learning algorithms to incorporate all the features to get good quality feed in a reasonably short time.

'''

Putting everything together

The following figure combines all the services related to the detailed design of the newsfeed system:

[The detailed design of newsfeed system](./detailed_design.jpg)

In this lesson, we discussed the design of the newsfeed system, its database schema, and the newsfeed ranking system. In the next lesson, we'll evaluate our system's requirements.

Design Newsfeed System/Evaluation of a Newsfeed System's Design/README.md

Evaluation of a Newsfeed System's Design

Fulfill requirements

Our non-functional requirements for the proposed newsfeed system design are scalability, fault tolerance, availability, and low latency. Let's discuss how the proposed system fulfills the requirements:

1. Scalability: The proposed system is scalable to handle an ever-increasing number of users. The required resources, including load balancers, web servers, and other relevant servers, are added/removed on demand.
2. Fault tolerance: The replication of data consisting of users' metadata, posts, and newsfeeds makes the system fault-tolerant. Moreover, the redundant resources are always there to handle the failure of a server or its component.
3. Availability: The system is highly available by providing redundant servers and replicating data on them. When a user gets disconnected due to some fault in the server, the session is recreated via a load balancer with a different server. Moreover, the data (users metadata, posts and newsfeeds) is stored on different and redundant database clusters, which provides high availability and durability.
4. Low latency: We can minimize the system's latency at various levels by:
 - Geographically distributed servers and the cache associated with them. This way, we bring the service close to users.

- Using CDNs for frequently accessed newsfeeds and media content.

Quiz on the newsfeed system's design

Test your understanding of the concepts related to the design of the newsfeed system with a quiz.

'''

1 Which component is responsible for storing relationships between users, their friends, and followers?

A)

The users database

B)

The graph database

C)

The posts database

D)

None of the above

'''

'''

2 Where are fully constructed and ranked newsfeeds stored?

A)

The published feeds cache

B)

The newsfeed cache

C)

The posts cache

D)

None of the above

'''

'''

3 When do the web servers call the posts service?

A)

When new posts are created

B)

When a request for a newsfeed is received

C)

When users are to be notified due to some failure events

D)

All of the above

'''

'''

4 Which option below is stored in blob storage?

A)

Comments on posts

B)
Count of likes and dislikes

C)
Media content

D)
All of the above
'''

Answers

1. (B)

2. (B)

3. (A)

4. (C)

Summary

In this chapter, we learned to design a newsfeed system at scale. Our design ranked enormous user data to show carefully curated content to the user for better user experience and engagement. Our newsfeed design is general enough that it can be used in many places such as Twitter feeds, Facebook posts, YouTube and Instagram recommendations, News applications, and so on.

Design Newsfeed System/Requirements of a Newsfeed System's Design/README.md

Requirements of a Newsfeed System's Design

Requirements

To limit the scope of the problem, we'll focus on the following functional and non-functional requirements:

Functional requirements

- Newsfeed generation: The system will generate newsfeeds based on pages, groups, and followers that a user follows. A user may have many friends and followers. Therefore, the system should be capable of generating feeds from all friends and followers. The challenge here is that there is potentially a huge amount of content. Our system needs to decide which content to pick for the user and rank it further to decide which to show first.

- Newsfeed contents: The newsfeed may contain text, images, and videos.

- Newsfeed display: The system should affix new incoming posts to the newsfeed for all active users based on some ranking mechanism. Once ranked, we show content to a user with higher-ranked first.

Non-functional requirements

- Scalability: Our proposed system should be highly scalable to support the ever-increasing number of users on any platform, such as Twitter, Facebook, and Instagram.

- Fault tolerance: As the system should be handling a large amount of data; therefore, partition tolerance (system availability in the events of network failure between the system's components) is necessary.

- Availability: The service must be highly available to keep the users engaged with the platform. The system can compromise strong consistency for availability and fault tolerance, according to the PACELC theorem.

- Low latency: The system should provide newsfeeds in real-time. Hence, the maximum latency should not be greater than 2 seconds.

'''

The PACELC theorem is an extension of the CAP theorem that states, in the event of network Partition, one should choose between Availability or Consistency; else, choose between Latency and Consistency.

Resource estimation

Let's assume the platform for which the newsfeed system is designed has 1 billion users per day, out of which, on average, 500 million are daily active users. Also, each user has 300 friends and follows 250 pages on average. Based on the assumed statistics, let's look at the traffic, storage, and servers estimation.

Traffic estimation

Let's assume that each daily active user opens the application (or social media page) 10 times a day. The total number of requests per day would be:

$500M \times 10 = 5 \text{ billions request per day} \approx 58K \text{ requests per second.}$

[Traffic estimation for the newsfeed system](./traffic.jpg)

Storage estimation

Let's assume that the feed will be generated offline and rendered upon a request. Also, we'll precompute the top 200 posts for each user. Let's calculate storage estimates for users' metadata, posts containing text, and media content.

1. Users' metadata storage estimation: Suppose the storage required for one user's metadata is 50 KB. For 1 billion users, we would need $1B \times 50KB = 50 \text{ TB}$
 $1B \times 50KB = 50TB.$

We can tweak the estimated numbers and calculate the storage for our desired numbers in the following calculator:

```
'''
    Storage Estimation for the Users' Metadata
Number of users (in billion)           1
Required storage for one users' metadata (in KBs)    50
Total storage required for all users (in TBs)        50
'''
```

2. Textual posts storage estimation: All posts could contain some text, we assume it's 500 characters on average. The storage estimation for the top 200 posts for 500 million users would be:

$200 \times 500M \times 50KB = 5PB$

3. Media content storage estimate: Along with text, a post can also contain media content. Therefore, we assume that 1/5th posts have videos and 4/5th include images. The assumed average image size is 200KB and the video size is 2MB.

Storage estimate for 200 posts of one user: $(200 \times 2MB \times 1/5) + (200 \times 200KB \times 4/5) = 80MB + 32MB = 112MB$

Total storage required for 500 million users' posts: $112MB \times 500M = 56PB$

So we'll need at least 56PB of blob storage to store the media content.

[Storage required for 500 million active users per day (each with approx. 200 posts) by newsfeed system](./storage.jpg)

```
'''
    Storage Estimation of Posts Containing Text and Media Content
Number of active users (in million)           500
Maximum allowed text storage per post (in KBs)    50
Number of precomputed posts per user (top N)      200
'''
```

Storage required for textual posts (in PBs)	5
Total required media content storage for active users (in PBs)	56

'''

Number of servers estimation

Considering the above traffic and storage estimation, let's estimate the required number of servers for smooth operations. Recall that a single typical server can serve 8000 requests per second (RPS). Since our system will have approximately 500 million daily active users (DAU). Therefore, according to estimation in Back-of-the-Envelope Calculations chapter, the number of servers we would require is:

$DAU/ServerRPS = 500M/8000 = 62500$ servers.

[Number of servers required for the newsfeed system](./servers.jpg)

'''

Servers Estimation

Number of active users (in million)	500
RPS of a server	8000
Number of servers required	62500

'''

Building blocks we will use

The design of newsfeed system utilizes the following building blocks:

[The building blocks to design a newsfeed system](./bb.jpg)

- Database(s) is required to store the posts from different entities and the generated personalized newsfeed. It is also used to store users' metadata and their relationships with other entities, such as friends and followers.
- Cache is an important building block to keep the frequently accessed data, whether posts and newsfeeds or users' metadata.
- Blob storage is essential to store media content, for example, images and videos.
- CDN effectively delivers content to end-users reducing delay and burden on back-end servers.
- Load balancers are necessary to distribute millions of incoming clients' requests for newsfeed among the pool of available servers.

In the next lesson, we'll focus on the high-level and detailed design of the newsfeed system

Design Newsfeed System/SystemDesign Newsfeed System/README.md

System Design: Newsfeed System

What is a newsfeed?

A newsfeed of any social media platform (Twitter, Facebook, Instagram) is a list of stories generated by entities that a user follows. It contains text, images, videos, and other activities such as likes, comments, shares, advertisements, and many more. This list is continuously updated and presented to the relevant users on the user's home page. Similarly, newsfeed system also displays the newsfeed to users from friends, followers, groups, and other pages, including a user's own posts.

'''

An entity could be a page, group, friends, and followers of a user.

'''

A newsfeed is essential for social media platform users because it keeps them informed about the latest industry developments, current affairs, and relevant information. It also provides them additional reasons to return and connect with a platform on a regular basis. Billions of users use such platforms. The challenging task is to provide a personalized newsfeed in real-time while keeping the system scalable and highly available.

This chapter will discuss the high-level and detailed design of a newsfeed system for a social platform like Facebook, Twitter, Instagram, etc.

[Newsfeeds on a mobile application]

How will we design the newsfeed system?

We have divided the design of the newsfeed system into the following three lessons:

1. Requirements: In this lesson, weâ ll identify the functional and non-functional requirements. Weâ ll also estimate the resource requirements to provide a personalized newsfeed to billions of users each day.

2. Design: Weâ ll discuss the high-level and detailed design of the newsfeed system in this lesson. Weâ ll also describe the API design and database schema for our proposed design. Moreover, this lesson will also help us to rank newsfeed to provide a better experience to users.

3. Evaluation: In this lesson, weâ ll evaluate the design of the newsfeed system based on the non-functional requirements. Weâ ll also take a quiz to assess our understanding of the design of the newsfeed system.

Letâ s list down the requirements for designing our version of the newsfeed system.

Design Quora/Evaluation of Quoraâ s Design/README.md

Evaluation of Quoraâ s Design

Fulfilling requirements

We have used various techniques to fulfill our functional requirements. However, we need to determine if we have fulfilled the non-functional requirements. Weâ ll highlight some of the mechanisms we have utilized to address the non-functional requirements:

- Scalability: Our system is highly scalable for several reasons. The updated design uses powerful and homogeneous service hosts. Quora uses powerful machines because service hosts use an in-memory cache, some level of queueing, maintain manager, worker, and routing library. The horizontal scaling of these service hosts is convenient because they are homogeneous.

'''

Quora now uses Amazon EC2 instances with 32 cores, a shared 20 MB cache, and high network throughput.

'''

On the database end, our design shards the MySQL databases vertically, which avoids issues in scalability because of overloaded MySQL servers. To reduce complex join queries, tables anticipating join operations are placed in the same shard or partition.

'''

Note: As mentioned earlier, vertical sharding may not be enough because each shard can grow large horizontally. For large MySQL tables, writing becomes a bottleneck. Therefore, our design may have to adhere to horizontal sharding, a well-known practice in database scaling.

'''

- Consistency: Due to the variety of functionalities offered by Quora, different consistency schemes may be selected for different types of data. For example, certain critical data like questions and answers should be stored synchronously. In this case, performance can take a hit because users donâ t expect instantaneous responses to their questions. It means that a user gets a reply in five minutes, one hour, one day, or no response at all, depending on the userâ s question and the availability of would-be respondents.

Other data like view counts may not necessarily be stored synchronously because it is not a goal of the Quora service to ensure that all users see the same number of views as soon as the question is posted. For such cases, eventual consistency is favored for improved performance.

'''

Note: In general, our design is equipped with strong techniques to reduce the user-perceived latency as a whole.

'''

- Availability: Some of the main ideas to improve availability include isolation between different components, keeping redundant instances, using CDN, using configuration services like ZooKeeper, and load balancers to hide failures from users.

'''

Decoupling services help to maintain, upgrade, and recover during failures of individual service or hosts. If services are collocated on the same node or host, a failed node results in the loss of multiple services at once.

'''

Our design, however, lacks any disaster recovery management, which weâ ll explore in the next section.

- Performance: This design has a strong performance because we have employed the right technology for the right feature. For example, we have used several datastores for different reasons. On top of that, we used different distributed caches depending upon the use case and access frequency. Also, we employed Kafka to queue similar tasks and assign them to cron jobs that otherwise take a long time if executed via API calls.

'''

Note: Quora claims that using its custom queuing solution, it can handle roughly 15,000 tasks per second.

'''

'''

Facts

Quora has developed a custom in-memory caching system called Pycache that can store tens of GB of cache in a local machine.

The Pycache API contains simple but effective functions like `pycache.get(key)`, `pycache.set(key, value)`, `pycache.delete(key)`, `pycache.iteritems()`, and `pycache.update()`.

Both the set and delete functions have variants to set and delete multiple entries at once.

Quora has developed and open-sourced a tool called Asynq that simplifies batching of cache requests to Memcached. So, developers can write code fasterâ and the latency of a round trip to the caching service is reduced

'''

'''

Meeting Non-functional Requirements

Requirements	Techniques
--------------	------------

Scalability	Based on AWS, which supports automatic scaling.
-------------	---

	Uses the same servers to reduce complexity in horizontal
--	--

scalability	sharding of MySQL database.
-------------	-----------------------------

	Employs various data stores for different purposes.
--	---

	Asynq can enable developers to code quickly by batching cache requests, separate compute, and feature extraction modules. Therefore, a generic feature extraction facility allows scalability of different recommendation systems.
--	--

Consistency	Uses MySQL and synchronous replication within a data center for critical data.
-------------	--

	Offers eventual consistency for non-critical data like the view counter.
--	--

Availability	Use of different data stores prevents failure of multiple services at once by using database sharding and replicas.
--------------	---

failures. Uses CDN as a backup to serve static/dynamic data in case of

ZooKeeper enables service hosts to get updates about MySQL shards.

Load balancers hide server failures from end users.

AWS supports an availability above 99 percent.

Thrift isolates services and therefore failures.

Performance

MyRocks has a much lower P99 latency.

as C++ for the routing library. Uses the right programming language to deliver tasks quickly, such

once. Uses 'Multiget()' to retrieve multiple entries from Memcached at

Eliminates network round trip time with Asynq.

Kafka improves the performance of service hosts.

Sharding improves QPS of MySQL.

accessed data. Custom, in-memory caching system reduces the latency of frequently

Disaster recovery

Our proposed and detailed design does not cater to the situation of natural disasters. While we have met other non-functional requirements, durability, fault-tolerance, and availability are incomplete without a disaster recovery management plan. This section will explore some mechanisms that provide resilience against disasters.

The first and foremost approach of handling a disaster is frequent backups. The frequency of backups depends on the size of the data. Daily backups are suitable for our design because we can backup individual data stores and shards without any hassle. Of course, backups will be stored at remote destinations because natural disasters can wipe out the entire facility at a location.

Natural disasters: Disasters include large-scale floods, earthquakes, fires, and outages on multiple backbone Internet connections taking off a whole data center or multiple data centers

Note: Taking regular backup at remote locations is not enough. Quick restoration of backed-up data in a timely manner completes a disaster recovery plan.

The following are important questions for designing a disaster recovery plan:

- What data and systems are considered critical to recover from disasters?
- How fast is the restoration from the backup facility?
- Can all systems be recovered through backups?
- How can we deal with potential loss of data that we couldn't replicate before the disaster hit?

The illustration below shows a simple architecture of how a disaster recovery scheme works:

[Disaster recovery management](./recovery.jpg)

The approach is fairly straightforward. The data, application servers, and configurations are backed up in the Amazon S3 storage service in the same zone. Zonal replication between S3 storage facilitates transfer to another zone. Later, the application and database servers can be restored from the S3 storage in another zone.

The restoration strategy is simple and effective but it has a couple of drawbacks:

- We might lose some data that is not backed up since we take backups on a daily basis. However, this issue can be mitigated if we do synchronous replication across regions.
- Restoration can take a long time (a few hours), and most databases do not serve queries while data is being recovered.

'''

Note: In general, Amazon provides service with high reliability and availability. For instance, S3 service reports 99.999999999% durability and 99.9% availability over a year.

'''

Conclusion

Throughout this design, we learned how Quora is able to scale its services as the number of users increases. One interesting aspect of the design includes the vertical sharding of the MySQL database. Apart from that, the Quora design discusses a variety of techniques to meet the functional and non-functional requirements. However, our scope did not include the usage of techniques like natural language processing (NLP) to remove spelling mistakes in user's questions or typeahead services during the search.

'''

Question

How can using different data stores for different types of data be beneficial in disaster recovery?

Answer

When we use different data stores, it means that we can recover various forms of data simultaneously instead of one large blob of data. Since we recover data stores with variable recovery times, the final recovery time equals the maximum recovery time among all data stores.

However, using many databases also means that administrators need to know the details of checkpointing or restoring them.

'''

Design Quora/Final Design of Quora/README.md

Final Design of Quora

Limitations of the proposed design

The proposed design serves all the functional requirements. However, it has a number of serious drawbacks that emerge as we scale. This means that we are unable to fulfill the non-functional requirements. Let's explore the main shortcomings below:

- Limitations of web and application servers: To entertain the user's request, payloads are transferred between web and application servers, which increases latency because of network I/O between these two types of servers. Even if we achieve parallel computation by separating the web from application servers (that is, the manager and worker processes), the added latency due to an additional network link erodes a user's experience. Apart from data transfer, control communication between the router library with manager and worker processes also imposes additional performance penalties.

- In-memory queue failure: The internal architecture of application servers logs tasks and forwards them to the in-memory queues, which serve them to the workers. These in-memory queues of different priorities can be subject to failures. For instance, if a queue gets lost, all the tasks in that queue are lost as well, and manual engineering is required to recover those tasks. This greatly reduces the performance of the system. On the other hand, replicating these

queues requires increasing RAM size. Also, with the number of features (functional requirements) that our system offers, many tasks can get assembled, which results in insufficient memory. At the same time, it is not desirable to choke application servers with not-so-urgent tasks. For example, application servers should not be burdened with tasks like storing view counts for answers, adding statistics to the database for later analysis, and so on.

- Increasing QPS on MySQL: Because we have a higher number of features offered by our system, few MySQL tables receive a lot of user queries. This results in a higher number of QPS on certain MySQL servers, which can result in higher latency. Furthermore, there is no scheme defined for disaster recovery management in our design.

- Latency of HBase: Even though HBase allows high real-time throughput, its P99 latency is not among the best. A number of Quora features require the ML engine that has a latency of its own. Due to the addition of the higher latency of HBase, the overall performance of the system degrades over time.

'''

P99 stands for 99th percentile. It means that 99 percent of the queries are entertained below specific number. For example, P99 of 20 ms means that 99 percent of the queries will be replied to within 20 ms by a server.

'''

The issues highlighted above require changes to the earlier proposed design. Therefore, weâll make the following adjustments and update our design:

[Detailed design of Quora](./detailed_design.jpg)

Detailed design of Quora

Letâs understand the improvements in our design:

Service hosts

We combine the web and application servers within a single powerful machine that can handle all the processes at once. This technique eliminates the network I/O and the latency introduced due to the network hops required between the manager, worker, and routing library processes. The illustration below provides an abstract view of the updated web server architecture:

[The updated design, where web and application servers are combined in the service host](./servers.jpg)

Vertical sharding of MySQL

Tables in the MySQL server are converted to separate shards that we refer to as partitions. A partition has a single primary server and multiple replica servers.

The goal is to improve performance and reduce the load due to an increasing number of queries on a single database table. To achieve that, we do vertical sharding in two ways:

1. We split tables of a single database into multiple partitions. The concept is depicted in Partitions 2 and 3, which embed Tables 4 and 3, respectively.
 2. We combine multiple tables into a single partition, where join operations are anticipated. The concept is depicted in Partition 1, which embeds Tables 1 and 2.
- Therefore, we are able to co-locate related data and reduce traffic on hot data. The illustration below depicts vertical sharding at Quora.

[The architecture of vertical sharding at Quora](./partitions.jpg)

After we complete the partitioning, we require two types of mappings or metadata to complete our scaling process:

1. Which partitions contain which tables and columns?
 2. Which hosts are primary and replicas of a particular partition?
- Both of these mappings are maintained by a service like ZooKeeper.

The sharded design above ensures scalability because we are able to locate related data in a single partition, and therefore it eliminates the need for querying data from multiple shards. Also, the number of read-replicas can be increased for hot shards, or further sharding may be performed. For edge cases where joining may be needed, we can perform it at the application level.

'''

Note: Vertical sharding is of particular interest in Quora's design because horizontal sharding is more common in the database community. The main idea behind vertical sharding is to achieve scalability by carefully dividing or re-locating tables and eliminating join operations across different shards. Nevertheless, a vertically sharded partition or table can grow horizontally to an extent where horizontal sharding will be necessary to retain acceptable performance.

'''

MyRocks

The new design embeds MyRocks as the key-value store instead of HBase. We use the MyRocks version of RocksDB for two main reasons:

1. MyRocks has a lower p99 latency instead of HBase. Quora claims to have reduced P99 latency from 80 ms to 4 ms using MyRocks.
2. There are operational tools that can transfer data between MyRocks and MySQL.

'''

Note: Quora serves the ML compute engine by extracting features from questions and answers stored in MySQL. In this case, the operational tools come in handy to transfer data between MyRocks and MySQL.

'''

Kafka

Our updated design reduces the request load on service hosts by separating not-so-urgent tasks from the regular API calls. For this purpose, we use Kafka, which can disseminate jobs among various queues for tasks such as the view counter (see Sharded Counters), notification system, analytics, and highlight topics to the user. Each of these jobs is executed through cron jobs.

'''

cron jobs: It is also called a cron, and is a command-line utility that is used to schedule tasks on UNIX-like operating systems.

'''

Technology usage

Services that scale quickly have little time to develop new features and handle an increasing number of requests from users. Such services employ cloud infrastructure to handle spikes in traffic. Also, the choice of programming language is important. Just like we mentioned that YouTube chose Python for faster programming, we can apply the same logic to Quora. In fact, Quora uses the Python Paste web framework.

It is desirable to use a faster programming language like C++ to develop the feature extraction service. For online recommendation services through a ML engine, feature extraction service should be quick, to enable the ML engine to accomplish accurate recommendations. Not only that but reducing the latency burden on the ML engine allows it to provide a larger set of services. We can employ the Thrift service to support interoperability between programming languages within different components.

'''

Feature extraction service: Quora refers to their feature extraction service as Alchemy, a scalable and high-performance tool.

'''
Features like comments, upvotes, and downvotes require frequent page updates from the client side. Polling is a technique where the client (browser) frequently requests the server for new updates. The server may or may not have any updates but still responds to the client. Therefore, the server may get uselessly overburdened. To resolve this issue, Quora uses a technique called long polling, where if a client requests for an update, the server may not respond for as long as 60 seconds if there are no updates. However, if there is an update, the server will reply immediately and allow the client to make new requests.

[Polling vs. long polling]

Lastly, Memcached can employ multiget() to obtain multiple keys from the cache shards to reduce the retrieval latency of multiple keys.

'''
Note: Quora has employed AWS to set up a good number of its infrastructure elements, including S3 (see the Blob Storage chapter) and Redshift storage.
'''

'''
Question 1

What would be considered a good approach for communication between different manager and worker processes within the service hosts?

Answer

Two approaches are feasible for communication:

1. UNIX sockets
2. TCP connections

Sockets (Unix or TCP) allow data streaming between sender and receiver with appropriate flow control (and congestion control in the case of TCP). That means the sender and receiver can send variable-size data in a decoupled fashion.

Other interprocess communication techniques like shared memory may not be feasible because they require estimating the size of the required memory segment, which makes the participants more coupled. Also, it will not work across physical servers.

So, we prefer sockets due to their high decoupling, flow control, and ability to work for both single servers or over the network.
'''

'''
Question 2

In our detailed design, we have employed Kafka to handle our view counter of answers. Why do you think we made this decision?

Answer

Depending on the topic label, many users can view the response to a question at once. This can choke the servers. Instantaneous update of the view counter is also not an important product feature. Therefore, Kafka is suitable for handling these tasks.

Quora handles these tasks in two minutes or less.

Nonetheless, we can use sharded counters as an effective solution to the view counter problem.
'''

'''
Question 3

What is the main advantage of using long polling instead of polling?

Answer

Long polling transfers the control to the server side instead of the client, which has no information about the updates in content. If the server is in control, it can reply as soon as there is fresh content. As a result, it reduces the request load on itself. However, long polling is a resource-intensive solution because it keeps the connection persistent or alive for a longer period of time.

WebSockets are another low-latency solution with low overhead. However, WebSockets might be an overkill for the features offered by Quora.

'''

Design Quora/Initial Design of Quora/README.md

Initial Design of Quora

Initial design

The initial design of Quora will be composed of the following building blocks and components:

- Web and application servers: A typical Quora page is generated by various services. The web and application servers maintain various processes to generate a webpage. The web servers have manager processes and the application servers have worker processes for handling various requests. The manager processes distribute work among the worker processes using a router library. The router library is enqueued with tasks by the manager processes and dequeued by worker processes. Each application server maintains several in-memory queues to handle different user requests. The following illustration provides an abstract view of web and application servers:

[Web and application servers at Quora](./servers.jpg)

- Data stores: Different types of data require storage in different data stores. We can use critical data like questions, answers, comments, and upvotes/downvotes in a relational database like MySQL because it offers a higher degree of consistency. NoSQL databases like HBase can be used to store the number of views of a page, scores used to rank answers, and the extracted features from data to be used for recommendations later on. Because recomputing features is an expensive operation, HBase can be a good option to store and retrieve data at high bandwidth. We require high read/write throughput because big data processing systems use high parallelism to efficiently get the required statistics. Also, blob storage is required to store videos and images posted in questions and answers.

'''

Quora was founded in 2009, whereas HBase was developed in 2008 by Apache. Because it is open-source and modeled after Google's BigTable, it is suitable for storing a large amount of small-sized data. Furthermore, it has high read/write throughput. Therefore, it was a natural choice for Quora to use in its inception.

'''

- Distributed cache: For performance improvement, two distributed cache systems are used: Memcached and Redis. Memcached is primarily used to store frequently accessed critical data that is otherwise stored in MySQL. On the other hand, Redis is mainly used to store an online view counter of answers because it allows in-store increments. Therefore, two cache systems are employed according to their use case. Apart from these two, CDNs serve frequently accessed videos and images.

- Compute servers: A set of compute servers are required to facilitate features like recommendations and ranking based on a set of attributes. These features can be computed in online or offline mode. The compute servers use machine learning (ML) technology to provide effective recommendations. Naturally, these compute servers have a substantially high amount of RAM and processing power.

'''

online mode: Recommendations are computed as soon as they are requested.
offline mode: Recommendations are computed beforehand and served when requested.
,,,

Of course, other basic building blocks like load balancers, monitoring services, and rate limiters will also be part of the design. A high-level design is provided below:

[The high-level design of Quora](./hld.jpg)

Workflow

The design of Quora is complex because we have a large number of functional and non-functional requirements. Therefore, we'll explain the workflow on the basis of each feature:

- Posting question, answers, comments: The web servers receive user requests through the load balancer and direct them to the application servers. Meanwhile, the web servers generate part of the web page and let the worker process in the application servers do the rest of the page generation. The questions and answers data is stored in a MySQL database, whereas any videos and images are stored in the blob storage. A similar approach is used to post comments and upvote or downvote answers. Task prioritization is performed by employing different queues for different tasks. We perform prioritization because certain tasks require immediate attentionâ for example, fetching data from the database for a user requestâ while others are so urgentâ for example, sending a weekly email digest. The worker processes will perform tasks by fetching from these queues.

Question 1

When would a service like Quora require a notifications feature?

Answer

Quora will need a notification service in the following scenarios:

A user posts a question on a topic subscribed by a would-be respondent.

A user posts an answer to a question asked by another user.

A post a user is interested in or wrote received new comments or upvotes/downvotes, and so on.

Question 2

How is it possible for Quora to send notifications to users through the above-described mechanism?

Hide Answer

Since tasks are added to different priority queues, it is possible to maintain a queue for notifications. While higher priority tasks are enqueued to high-priority queues, a notification task will be added to a medium or lower priority queue. The illustration below visualizes the described concept:

[Notifications]

- Answer ranking system: Answers to questions can be sorted based on date. Although it is convenient to develop a ranking system on the basis of date (using time stamps), users prefer to see the most appropriate answer at the top. Therefore, Quora uses ML to rank answers. Different features are extracted over time and stored in the HBase for each type of question. These features are forwarded to the ML engine to rank the most useful answer at the top. We cannot use the number of upvotes as the only metric for ranking answers because a good number of answers can be jokesâ and such answers also get a lot of upvotes. It is good to implement ranking system offline because good answers get upvotes and views over time. Also, the offline mode poses a lesser burden on the infrastructure. Implementing the ranking system offline and the need for special ML hardware makes it suitable to use some public cloud elastic services.

- Recommendation system: The recommendation system is responsible for several features. For example, we might need to develop a user feed, find related questions and ads, recommend questions to potential respondents, and even highlight duplicate content and content in violation of the service's terms of use. Unlike the answer ranking system, the recommendation system must provide both online and offline services. This system receives requests from the application server and forwards selected features to the ML engine.

- Search feature: Over time, as questions and answers are fed to the Quora system, it is possible to build an index in the HBase. User search queries are matched against the index, and related content is suggested to the user. Frequently accessed indexes can be served from cache for low latency. The index can be constructed from questions, answers, topics labels, and usernames. Tokenization of the search index returns the same results for reordered words also (see Scaling Search and Indexing in Distributed Search chapter for more details).

'''

Fun facts

- Quora is mostly built on Amazon Web Services (AWS).
- Initially, Quora used Amazon's EC2 instances as their application servers used eight cores with an 8 MB cache.
- Quora used the search server called Sphinx. Later, because of its slow performance, Quora custom-built its search engine using Thrift and Python Unicode libraries only.
- The full-text search feature was launched by Quora in 2013. Before that, it was possible to get results for individual words only.

'''

API design

We'll design the API calls for Quora in this section. We'll define APIs for the following features only:

- Post a question
- Post an answer
- Upvote or downvote a question or answer
- Comment on an answer
- Search

'''

Note: We don't consider APIs for a recommendation system or ranking because they are not planned as an explicit request by the user. Instead, the web server coordinates with other components to ensure the service.

'''

Post a question

The POST method of HTTP is used to call the /postQuestion API:

'''

postQuestion(user_id, question, description, topic_label, video, image)

'''

Let's understand each parameter of the API call:

'''

Parameter	Description
-----------	-------------

user_id	This is the unique identification of the user that posts the question.
---------	--

question	This is the text of the question posed by the user.
----------	---

description	This is the description of a question. This is an optional field.
-------------	---

topic_label	This represents a list of domains to which the user's question is related.
-------------	--

video	This is a video file embedded in a user question.
-------	---

image This is an image that is a part of a user question.
'''

The video and image parameters can be NULL if no image or video is embedded within the question. Otherwise, it is uploaded as part of the question.

Post an answer

For posting an answer, the POST method is a suitable choice for /postAnswer API:

'''
postAnswer(user_id, question_id, answer_text, video, image)
'''

'''

Parameter	Description
-----------	-------------

question_id	This refers to the question the answer is posted against.
-------------	---

answer_text	This is the textual answer posted by the responder.
-------------	---

'''

The rest of the parameters are self-explanatory.

Upvote an answer

The /upvote API is below:

'''
upvote(user_id, question_id, answer_id)
'''

'''

Parameter	Description
-----------	-------------

user_id	This represents the user upvoting the answer.
---------	---

answer_id	This represents the identity of the answer that is upvoted for a particular question, which is identified by the question_id.
-----------	---

'''

'''

Note: The downvote API is the same as the upvote API because both are similar functionalities.

'''

Comment on an answer

The /comment API has the following structure:

'''
comment(user_id, answer_id, comment_text)
'''

'''

'''

Parameter	Description
-----------	-------------

user_id	It represents the user commenting on the answer.
---------	--

comment_text	It represents the text a user posts against an answer identified by the answer_id.
--------------	--

'''

Search

The /search API has the following details:

'''

search(user_id, search_text)

'''

```
'''
Parameter      Description

user_id        This is the user_id performing the search query. It is optional in this case
because a non-registered user can also search for questions.

search_text    This is the search query entered by a user.
'''
```

```
'''
```

We use a sequencer to generate the different IDs mentioned in the API calls.

```
'''
```

```
'''
```

Question

Why is there a custom routing layer between the web and application servers instead of a load-balancing layer?

Hide Answer

The primary reason is performance. A generic load-balancing layer will have little application understandability and higher latency.

As stated, the manager processes generate the skeleton of the web page and leave the majority of the work to worker processes. In that sense, one manager process can generate multiple worker tasks. The custom router library is a queue between manager and worker, where any worker process can consume a task generated by the manager process.

This design improves the overall performance and supports scalability.

router library: Quora refers to it as a Fastrouter - it is a custom C++ built queue server

```
## Design Quora/Requirements of Quora's Design/README.md
```

```
# Requirements of Quora's Design
```

```
## Requirements
```

Let's understand the functional and non-functional requirements below:

```
### Functional requirements
```

A user should be able to perform the following functionalities:

- Questions and answers: Users can ask questions and give answers. Questions and answers can include images and videos.
- Upvote/downvote and comment: It is possible for users to upvote, downvote, and comment on answers.
- Search: Users should have a search feature to find questions already asked on the platform by other users.
- Recommendation system: A user can view their feed, which includes topics they're interested in. The feed can also include questions that need answers or answers that interest the reader. The system should facilitate user discovery with a recommender system.
- Ranking answers: We enhance user experience by ranking answers according to their usefulness. The most helpful answer will be ranked highest and listed at the top.

[Functional and non-functional requirements of Quora](./quora.jpg)

```
### Non-functional requirements
```

- Scalability: The system should scale well as the number of features and users grow with time. It means that the performance and usability should not be impacted by an increasing number of users.
- Consistency: The design should ensure that different users' views of the same content should

be consistent. In particular, critical content like questions and answers should be the same for any collection of viewers. However, it is not necessary that all users of Quora see a newly posted question, answer, or comment right away.

- Availability: The system should have high availability. This applies to cases where servers receive a large number of concurrent requests.
- Performance: The system should provide a smooth experience to the user without a noticeable delay.

Resource estimation

In this section, we'll make an estimate about the resource requirements for Quora service. We'll make assumptions to get a practical and tractable estimate. We'll estimate the number of servers, the storage, and the bandwidth required to facilitate a large number of users.

Assumptions: It is important to base our estimation on some underlying assumptions. We, therefore, assume the following:

- There are a total of 1 billion users, out of which 300 million are daily active users.
- Assume 15% of questions have an image, and 5% of questions have a video embedded in them. A question cannot have both at the same time.
- We'll assume an image is estimated to be 250 KBs, and a video is considered 5 MBs.

Number of servers estimation

Let's estimate our requests per second (RPS) for our design. If there are an average of 300 million daily active users and each user can generate 20 requests per day, then the total number of requests in a day will be:

$$300 \times 10^6 \times 20 = 6 \times 10^9$$

Therefore, the RPS = $6 \times 10^9 / 86400 = 69500$ approximately requests per second.

'''

Estimating RPS

Daily active users	300	million
Requests per day per user	20	
Requests Per Second (RPS)	69444	

'''

We already established in the back-of-the-envelope calculations chapter that we'll use the following formula to estimate a pragmatic number of servers:

$$\text{Number of daily active users} / \text{RPS of a server} = 300 \times 10^6 / 8000 = 37500$$

[The estimated number of servers required for Quora](./servers.jpg)

'''

Therefore, the total number of servers required to facilitate 300 million users generating an average of 69,500 requests per second will be 37,500.

'''

Storage estimation

Let's keep in mind our assumption that 15% of questions have images and 5% have videos. So, we'll make the following assumptions to estimate the storage requirements for our design:

- Each of the 300 million active users posts 1 question in a day, and each question has 2 responses on average, 10 upvotes, and 5 comments in total.
- The collective storage required for the textual content of one question equals 1 KB.

'''

Storage Requirements Estimation Calculator

Questions per user	1	per day
Total questions per day	300	millions
Size of textual content per question	1	KB
Image size	250	KB
Video size	5	MB
Questions containing images	15	percent
Questions containing videos	5	percent
Storage for textual content	0.3	TB
Storage for image content	11.25	TB
Storage for video content	75	TB

'''

'''

The following are the default calculations:

- Total questions: $300M \times 1 = 300 \times 10^6$ questions per day.
- Storage required for textual content of all questions in one day: $300 \times 10^6 \times 1 \text{ KB} = 300\text{GB}$
- Storage required for images for one day: $(300 \times 10^6 \times 15) / 100 \times 250 \times 10^3 \text{ B} = 11.25\text{TB}$
- Storage required for video content for one day: $(300 \times 10^6 \times 15) / 100 \times 5 \times 10^6 \text{ B} = 75\text{TB}$

'''

Total storage required for one day = $0.3\text{TB} + 11.25\text{TB} + 75\text{TB} \approx 86.55\text{TB}$ per day

[Summarizing storage requirements of Quora](./storage.jpg)

The daily storage requirements of Quora seem very high. But for service with 300 million DAU, yearly requirement of $86.55\text{TB} \times 365 = 31.6\text{PB}$
 $86.55\text{TB} \times 365 = 31.6\text{PB}$ is feasible. The practical requirement will be much higher because we have disregarded the storage required for a number of things. For example, non-active (out of 1B) users' data will require storage.

Bandwidth estimation

The bandwidth estimate requires the calculation of incoming and outgoing data through the network.

- Incoming traffic: The incoming traffic bandwidth required per day will be equal to $86.55\text{TB} / 86400 \times 8 = 8\text{Gbps}$
- Outgoing traffic: We have assumed that 300 million active users views 20 questions per day, so the total bandwidth requirements can be found in the below calculator:

'''

Bandwidth Requirements Estimation Calculator

Total storage required per day	86.55	TB
Incoming traffic bandwidth	8	Gbps
Questions viewed per user	20	per day
Total questions viewed	69444	per second

Bandwidth for text of all questions	0.56	Gbps
Bandwidth for 15% of image content	20.83	Gbps
Bandwidth for 5% of video content	138.89	Gbps
Outgoing traffic bandwidth	160.3	Gbps

'''

'''

300M^20 questions=6 billion questions are viewed per day.

- Questions viewed per second: $6 \times 10^9 / 86400 \approx 69.4 \times 10^3$ questions are viewed per second.
- Bandwidth for the textual content of all questions and their answers: $69.4 \times 10^3 \times 1 \times 10^3 \times 8 \text{ bits} = 0.56 \text{ Gbps}$
- Bandwidth of the 15% of content which contain images per second: $69.4 \times 10^3 \times 15/100 \times 250 \times 10^3 \times 8 \text{ bits} = 20.82 \text{ Gbps}$
- Bandwidth for the 5% of content that contains video per second: $69.4 \times 10^3 \times 5/100 \times 5 \times 10^6 \times 8 \text{ bits} = 138.9 \text{ Gbps}$
- Total outgoing traffic bandwidth: $0.56 \text{ Gbps} + 20.82 \text{ Gbps} + 138.9 \text{ Gbps} = 160.3 \text{ Gbps}$

We use rounding at each step in this explanation. The answers in the calculator above are slightly different due to rounding.

'''

[Summarizing the bandwidth requirements of Quora](./bandwidth.jpg)

Total bandwidth requirement of Quora is equal to:

'''

Incoming + outgoing traffic bandwidth = 8Gbps + 160.3Gbps = 168.3Gbps

'''

Building blocks we will use

We'll use the following building blocks for the initial design of Quora:

[Building blocks required for our design]

- Load balancers will be used to divide the traffic load among the service hosts.
- Databases are essential for storing all sorts of data, such as user questions and answers, comments, and likes and dislikes. Also, user data will be stored in the databases. We may use different types of databases to store different data.
- A distributed caching system will be used to store frequently accessed data. We can also use caching to store our view counters for different questions.
- The blob store will keep images and video files.

Design Quora/System Design Quora/README.md

System Design: Quora

Introduction

With so much information available online, finding answers to questions can be daunting. That's why information sharing online has become so widespread.. Search engines help us dig for information across the web. For example, Google's search engine is intelligent enough to answer questions by extracting information from web pages. While search engines have their advantages sometimes finding the information we want isn't a straightforward process. Let's look at the illustration below to understand how seeking information through search engines is different in comparison to people.

[Comparing information seeking through search engines vs. humans](./infoseeking.jpg)

The illustration above depicts that information-seeking through other people can be more instructive, even if it comes at the cost of additional time. Seeking information through search engines can lead to dead ends because of content unavailability on a topic. Instead, we can ask questions of others.

What is Quora?

Quora is a social question-and-answer service that allows users to ask questions to other users. Quora was created because of the issue that asking questions from search engines result in fast answers but shallow information. Instead, we can ask the general public, which feels more conversational and can result in deeper understanding, even if it's slower. Quora enables anyone to ask questions, and anyone can reply. Furthermore, there are domain experts that have in-depth knowledge of a specific topic who occasionally share their expertise by answering questions.

The following plot shows global user search trends for the term "Quora" using Google:

[Popularity of Quora according to Google by year](./popularity.jpg)

More than 300 million monthly active users post thousands of questions daily related to more than 400,000 topics on Quora.

In this chapter, we'll design Quora and evaluate how it fulfills the functional and non-functional requirements.

How will we design Quora?

We'll design Quora by dividing the design problem into the following four lessons.

1. Requirements: This lesson will focus on the functional and non-functional requirements for designing Quora. We'll also estimate the resources required to design the system.
2. Initial design: We'll create an initial design that fulfills all the functional requirements for Quora and also formulate the API design in this lesson. Primarily, we'll discuss the system's building blocks, other components involved in completing the design, their integration, and workflow.
3. Final design: In this lesson, we'll start by identifying the limitations of the initial design. Then, we'll update our final design to fulfill all the functional and non-functional requirements while addressing these limitations. We'll also focus on some interesting aspects of our design, like vertical sharding of the database.
4. Evaluation: We'll assess our design specifically for non-functional requirements and discuss some of its trade-offs. We'll also discuss some ideas on how we can improve the availability of our design in this lesson.

'''

Note: The information provided in this chapter is inspired by the engineering blog of Quora.

'''

Design Twitter/Client-side Load Balancer for Twitter/README.md

Client-side Load Balancer for Twitter

Introduction

In the previous lesson, we conceived the design of Twitter using a dedicated load balancer. Although this method works, and we've employed it in other designs, it may not be the optimal choice for Twitter. This is because Twitter offers a variety of services on a large scale, using numerous instances and dedicated load-balancers are not a suitable choice for such systems. To understand the concept better, let's understand the history of Twitter's design.

Twitter's design history

The initial design of Twitter included a monolithic (Ruby on Rails) application with a MySQL database. As Twitter scaled, the number of services increased and the MySQL database was sharded. A monolithic application design like this is a disaster because of the following reasons:

- A large number of developers work on the same codebase, which makes it difficult to update individual services.
- One service's upgrade process may lead to the breaking of another.
- Hardware costs grow because a single machine performs numerous services.
- Recovery from failures is both time-consuming and complex.

With the way Twitter has evolved, the only way out was many microservices where each service can be served through hundreds or thousands of instances.

Client-side load balancing

In a client-side load balancing technique, there's no dedicated intermediate load-balancing infrastructure between any two services with a large number of instances. A node requesting a set of nodes for a service has a built-in load balancer. We refer to the requesting node or service as a client. The client can use a variety of techniques to select a suitable instance to request the service. The illustration below depicts the concept of client-side load balancing. Every new arriving service or instance will register itself with a service registry so that other services are aware of its existence

[How client-side load balancing works](./clients.jpg)

In the diagram above, Service A has to choose a relatively less-burdened instance of Service B. Therefore, it will make use of the load balancer component inside it to choose the most suitable instance of Service B. Using the same client-side load balancing method, Service B will talk to other services. It's clear that Service A is the client when it is calling Service B, whereas Service B is the client when it is talking to Service C and D. As a result, there will be no central entity doing load balancing. Instead, every node will do load balancing of its own.

Advantages: Using client-side load-balancing has the following benefits.

- Less hardware infrastructure/layers are required to do load balancing.
- Network latency will be reduced because of no intermediate hop.
- Client-side load balancers eliminate bandwidth bottlenecks. On the other hand, in a dedicated load balancing layer, all requests go through a single machine that can choke if traffic multiplies.
- Fewer points of failure in the overall system.
- There is no end users queue waiting for the resource (server) for the particular services because many load balancers are routing the traffic. Eventually, it increases the quality of experience (QoE).

Many real-world applications like Twitter, Yelp, Netflix, and others use client-side load balancing. We'll discuss the client-side load balancing techniques used by Twitter in the next section.

Client-side load balancing in Twitter

Twitter uses a client-side load balancer referred to as deterministic aperture that is part of a larger RPC framework called Finagle. Finagle is a protocol-agnostic, open-source, and asynchronous RPC library.

'''

Aperture specifies the number of sessions to be made with the calling service.

'''

'''

Protocol-agnostic systems are protocol-independent such that it enables the communication between a variety of protocols.

'''

'''

Note: Consider the following question. What does the client-side load balancer of Twitter balance on? That is, what parameters are used to fairly balance the load across different servers?

'''

Twitter primarily uses two distributions to measure the effectiveness of a load balancer:

1. The distribution of requests (OSI layer 7)
2. The distribution of sessions (OSI layer 5)

Although requests are the key metric, sessions are an important attribute for achieving a fair distribution. Therefore, we'll develop techniques for fair distribution of requests as well as sessions. Let's start with the simple technique of Power of Two Random Choices (P2C) for request distribution first.

Request distribution using P2C

The P2C technique for request distribution gives uniform request distribution as long as sessions are uniformly distributed. In P2C, the system randomly picks two unique instances (servers) against each request, and selects the one with the least amount of load. Let's look at the illustration below to understand P2C.

[Power of two random choices](./power.jpg)

P2C is based on the simple idea that comparison between two randomly selected nodes provides load distribution that is exponentially better than random selection

'''

See â The power of two choices in randomized load balancingâ by M. Mitzenmacher.

'''

Now that we've established how to distribute requests fairly, let's explore different techniques of session distribution.

Session distribution

Solution 1: We'll start our discussion with the Mesh topology. Using this approach, each client (load balancer) starts a session with all of the given instances of a service. The illustration below represents the concept of mesh topology.

Obviously, this method is fair because the sessions are evenly distributed. However, the even distribution of sessions comes at a very high cost, especially when scaled to thousands of servers. Apart from that, some unhealthy or stale sessions may lead to failures.

Solution 2: To solve the problem of scalability with solution 1, Twitter devised another solution called random aperture. This technique randomly chooses a subset of servers for session establishment.

[Random aperture](./random_aperture.jpg)

Of course, random selection will reduce the number of established sessions as we can see in the diagram above. However, the question is, how many servers will be chosen randomly? This is not an easy question to answer and the answer varies, depending on the request concurrency of a client.

To answer the question above, Twitter installed a feedback controller inside the random aperture that decides the subset size based on the client's load. As a result, we're able to reduce the session load and ensure scalability. However, this solution isn't fair. We can see the imbalance in the illustration above such that Server 4 has no session whereas Server 0 has three sessions.

This imbalance creates problems such as idle servers, high load on a few servers, and so on. Therefore, we need a solution that is both fair and scalable.

Solution 3: We learned from solution 2 that we can scale by subsetting the number of session establishments. However, we failed to do the session distribution fairly. To resolve the problem, Twitter came up with a deterministic aperture solution. The key benefits of this

approach are:

- Little coordination is required between clients and servers to converge on a solution.
- Minimal disruption occurs if there are any changes in the number of clients or server instances for whatever reason.

For this solution, we represent clients and servers in a ring with equally distanced intervals where each object on the ring represents the node or server labeled with a number. The illustration below shows what Twitter refers to as the discrete ring coordinates (that is, a specific point on the ring).

[Ring coordinates](./ring.jpg)

Now, we compose client and server rings to derive a relationship between them. Weâve also defined an aperture size of 3. Each client will create a session with three servers in the ring. Clients select a server by circulating clockwise in the ring. This approach is similar to consistent hashing, except that here we guarantee equal distribution of servers across the ring. Letâs see the illustration below where clients have sessions with servers chosen from given list of servers, respectively.

[Merge client and server ring coordinates](./merge.jpg)

In the diagram above, client 0 establishes sessions with servers 4, 5, and 0. Client 1 establishes sessions with servers 0, 1, and 2, whereas Client 2 establishes sessions with servers 2, 3, and 4. When the aperture rotates or moves in the given subset of arrays of the same size, each clientâs service has new servers available to create sessions. The number of sessions established per server is shown in the figure below.

[Number of sessions established per server](./sessions.jpg)

We can see that thereâs no idle server or server with a high load in the histogram graph above. Thus, we achieved a fair distribution for the clients belonging to the same service. Itâs clear that weâve half resolved the problem by improving the session distribution among servers as compared to the random aperture (solution 02). However, the problem of unfair distribution can escalate as the number of clients grows. In that case, some servers will get crowded when more than one client (service) talks to the back-end servers. Letâs see the illustration below to understand the problem.

[Two different clients creating session with backend servers](./2clients.jpg)

The illustration above depicts that the problem compounded as we increased the number of clients (services). The solution is continuous ring coordinates where we derive relationships between clients and servers using overlapping ring slices, rather than specific points on the rings. The important thing here is that the overlapping of rings can be partial since servers on the ring may not have enough instances to divide among the clients equally. Letâs look at the illustration below.

[Overlapping among clients' services and servers](./overlapping.jpg)

The illustration above shows that clients 0 and 1 (symmetrically) share the same server (server 1). It means that there is fractional overlapping concerning the ring coordinates. However, this overlapping can lead to asymmetric sharing as well. Symmetric or asymmetric, the load balancing can be done through P2C. For instance, in the diagram above, servers 1 and 3 will receive half the load that server 2 gets. This means that a server can receive more or less load, depending on its overlapped slice size with a client.

Now, we know how to map clients to the subset of the servers. Weâll use P2C with continuous ring coordinates. The client will pick points (coordinates) in its range instead of picking the instance of the server. The selection process of clients to create sessions with backend servers are as follows:

1. Pick two coordinates (offset, offset + width) within its range and map these coordinates to the discrete instances (servers).

2. Select the one instance with the most negligible load from two chosen instances (P2C) by its degree of intersection.

'''

Note: Continuous aperture is scalable and fair at the cost of an additional session over boundary node due to partial overlapping. It is also able to achieve little coordination and minimal disruption.

'''

Let's look at the below table to get an overview of the discussed approaches.

'''

Summary of Solutions for Session Distribution			
Approach effectiveness	Scalability	Fair distribution	Cost
Mesh topology â€œ	â€œ	â€œ	
Random aperture â€œ	â€œ	â€œ	
Deterministic aperture (discrete ring) â€œ	â€œ	weak	
Deterministic aperture (continuous ring) â€œ	â€œ	â€œ	

Conclusion

This design problem highlights that tweaking the performance of common building blocks and utilizing the right (rich) combination of them is necessary for real-world services. Our needs from one service to the next will dictate which design point we choose in our solution space. In this chapter, we saw examples of how Twitter uses a combination of data stores and custom algorithms for load-balancing.

Design Twitter/Detailed Design of Twitter/README.md

Detailed Design of Twitter

Storage system

Storage is one of the core components in every real-time system. Although we have a detailed chapter on storage systems, here, we'll focus on the storage system used by Twitter specifically. Twitter uses various storage models for different services to take full advantage of each model. We'll discuss each storage model and see how Twitter shifted from various databases, platforms, and tools to other ones and how Twitter benefits from all of these.

The content in this lesson is primarily influenced by Twitter's technical blogs, though the analysis is ours.

'''

A storage model represents a data store's essential physical aspects.

'''

- Google Cloud: In Twitter, HDFS (Hadoop Distributed File System) consists of tens of thousands of servers to host over 300PB data. The data stores in HDFS are mostly compressed by the LZ4 (data compression algorithm) because LZ4 works efficiently in Hadoop. This data includes logs

(client events, Tweet events, and timeline events), MySQL and Manhattan (discussed later) backups, ad targeting and analytics, user engagement predictions, social graph analysis, and so on. In 2018, Twitter decided to shift data from Hadoop clusters to the Google Cloud to better analyze and manage the data. This shift is named a partly cloudy strategy. Initially, they migrated Ad-hoc clusters (occasional analysis) and cold storage clusters (less accessed and less frequently used data), while the real-time and production Hadoop clusters remained. The big data is stored in the BigQuery (Google cloud service), a fully managed and highly scalable serverless data warehouse. Twitter uses the Presto (distributed SQL query engine) to access data from Google Cloud (BigQuery, Ad-hoc clusters, Google cloud storage, and so on).

- **Manhattan:** On Twitter, users were growing rapidly, and it needed a scalable solution to increase the throughput. Around 2010, Twitter used Cassandra (a distributed wide-column store) to replace MySQL but could not fully replace it due to some shortcomings in the Cassandra store. In April 2014, Twitter launched its own general-purpose real-time distributed key-value store, called Manhattan, and deprecated Cassandra. Manhattan stores the backend for Tweets, Twitter accounts, direct messages, and so on. Twitter runs several clusters depending on the use cases, such as smaller clusters for non-common or read-only and bigger for heavy read/write traffic (millions of QPS). Initially, Manhattan had also provided the time-series (view, like, and so on.) counters service that the MetricsDB now provides. Manhattan uses RocksDB as a storage engine responsible for storing and retrieving data within a particular node.

[Twitter's data move to the Manhattan database](./manhattandb.jpg)

- **Blobstore:** Around 2012, Twitter built the Blobstore storage system to store photos attached to Tweets. Now, it also stores videos, binary files, and other objects. After a specified period, the server checkpoints the in-memory data to the Blobstore as durable storage. We have a detailed chapter on the Blob Store, which can help you understand what it is and how it works.

- **SQL-based databases:** Twitter uses MySQL and PostgreSQL, where it needs strong consistency, ads exchange, and managing ads campaigns. Twitter also uses Vertica to query commonly aggregated datasets and Tableau dashboards. Around 2012, Twitter also built the Gizzard framework on top of MySQL for sharding, which is done by partitioning and replication. We have a detailed discussion on relational stores in our Databases chapter.

- **Kafka and Cloud dataflow:** Twitter evaluates around 400 billion real-time events and generates petabytes of data every day. For this, it processes events using Kafka on-premise and uses Google Dataflow jobs to handle deduping and real-time aggregation on Google Cloud. After aggregation, the results are stored for ad-hoc analysis to BigQuery (data warehouse) and the serving system to the Bigtable (NoSQL database). Twitter converts Kafka topics into Cloud Pub-sub topics using an event processor, which helps avoid data loss and provides more scalability. See the Pub-sub chapter for a deep dive into this.

[Real-time processing of billions of requests](./realtime.jpg)

- **FlockDB:** A relationship refers to a user's followers, who the user follows, whose notifications the user has to receive, and so on. Twitter stores this relationship in the form of a graph. Twitter used FlockDB, a graph database tuned for huge adjacency lists, rapid reads and writes, and so on, along with graph-traversal operations. We have a chapter on Databases and Newsfeed that discuss graph storage in detail.

- **Apache Lucene:** Twitter constructed a search service that indexes about a trillion records and responds to requests within 100 milliseconds. Around 2019, Twitter's search engine had an indexing latency (time to index the new tweets) of roughly 15 seconds. Twitter uses Apache Lucene for real-time search, which uses an inverted index. Twitter stores a real-time index (recent Tweets during the past week) in RAM for low latency and quick updates. The full index is a hundred times larger than the real-time index. However, Twitter performs batch processing for the full indexes. See the Distributed Search chapter to deep dive into how indexing works.

'''

The inverted index is a data structure like a hashmap that directs the system from a word to a document or database table.

'''

[Processing the search request](./processing.jpg)

The solution based on the “one size fits all” approach is infrequently effective. Real-time applications always focus on providing the right tool for the job, which needs to understand all possible use cases. Lastly, everything has upsides and downsides and should be applied with a sense of reality.

Cache

As we know, caches help to reduce the latency and increase the throughput. Caching is mainly utilized for storage (heavy read traffic), computation (real-time stream processing and machine learning), and transient data (rate limiters). Twitter has been used as multi-tenant (multiple instances of an application have the shared environment) Twitter Memcached (Twemcache) and Redis (Nighthawk) clusters for caching. Due to some issues such as unexpected performance, debugging difficulties, and other operational hassles in the existing cache system (Twemcache and Nighthawk), Twitter has started to use the Pelikan cache. This cache gives high-throughput and low latency. Pelikan uses many types of back-end servers such as the peliken_twemcache replacement of Twitter’s Twemcache server, the peliken_slimcache replacement of Twitter’s Memcached/Redis server, and so on. To dive deep, we have a detailed chapter on an In-memory Cache. Let’s have a look at the below illustration representing the relationship of application servers with distributed Pelikan cache.

[Distributed Pelikan cache](./cache.jpg)

'''

Note: Pelikan also introduced another back-end server named Segcache, which is extremely scalable and memory-efficient for small objects. Typically, the median size of the small objects is between 200 to 300 bytes in a large-scale application’s cache. Most solutions (Memcache and Redis) have a high size (56 bytes) of metadata with each object. This signifies that the metadata takes up more than one-third of the memory. Pelikan reduced metadata size per object to 38 bytes. Segcache also received NSDI Community Award and is used as an experimental server as of 2021.

'''

Observability

Real-time applications use thousands of servers that provide multiple services. Monitoring resources and their communication inside or outside the system is complex. We can use various tools to monitor services’ health, such as providing alerts and support for multiple issues. Our alert system notifies broken or degraded services triggered by the set metrics. We can also use the dynamic configuration library that deploys and updates the configuration for multiple services without restarting the service. This library leverages the Zookeeper (discussed later) for the configuration as a source of truth. Twitter used the LonLens service that delivers visualization and analytics of service logs. Later, it was replaced by Splunk Enterprise, a central logging system.

Tracing billions of requests is challenging in large-scale real-time applications. Twitter uses Zipkin, a distributed tracing system, to trace each request (spent time and request count) for multiple services. Zipkin selects a portion of all the requests and attaches a lightweight trace identifier. This sampling also reduces the tracing overhead. Zipkin receives data through the Scribe (real-time log data aggregation) server and stores it in the key-value stores with a few indexes.

[Observability](./observability.jpg)

Most real-time applications use Zookeeper to store critical data. It can also provide multiple services such as distributed locking and leader election in the distribution system. Twitter

uses Zookeeper to store service registry, Manhattan clustersâ topology information, metadata and so on. Twitter also uses it for the leader election on the various systems.

Real-world complex problems

Twitter has millions of accounts, and some accounts (public figures) have millions of followers. When these accounts post Tweets, millions of followers of the respective account engage with their Tweets in a short time. The problem becomes big when the system handles the billions of interactions (such as views and likes) on these Tweets. This problem is also known as the heavy hitter problem. For this, we need millions of counters to count various operations on Tweets.

Moreover, a single counter for each specific operation on the particular Tweet is not enough. Itâs challenging to handle millions of increments or decrements requests against a particular Tweet in a single counter. Therefore, we need multiple distributed counters to manage burst write requests (increments or decrements) against various interactions on celebritiesâ Tweets. Each counter has several shards working on different computational units. These distributed counters are known as sharded counters. These counters also help in another real-time problem named the Top-k problem. Letâs discuss an example of Twitterâs Top-k problems: trends and timeline.

Trends: Twitter shows Top-k trends (hashtags or keywords) locally and globally. Here, âlocalâ refers to when a topic or hashtag is used within the exact location where the requested user is active. Alternatively, âgloballyâ refers to when the particular hashtag is used worldwide. There is a possibility that users from some regions are not using a specific hashtag in their Tweets but get this hashtag in their trends timeline. Hashtags with the maximum frequency (counts) become trends both locally and globally. Furthermore, Twitter shows various promoted trends (known as âpaid trendsâ) in specified regions under trends. The below slides represent hashtags in the sliding window selected as Top-k trends over time.

[Trends](./topK)

Timeline: Twitter shows two types of timelines: home and user timelines. Here, weâll discuss the home timeline that displays a stream of Tweets posted by the followed accounts. The decision to show Top-k Tweets in the timeline includes followed accounts Tweets and Tweets that are liked or Retweeted by the followed accounts. Thereâs also another category of promoted Tweets displayed in the home timeline.

Sharded counters solve the discussed problems efficiently. We can also place shards of the specified counter near the user to reduce latency and increase overall performance like CDN. Another benefit we can get is a frequent response to the users when they interact (like or view) with a Tweet. The nearest servers managing various shards of the respective counters are continuously updating the like or view counts with short refresh intervals. We should note, however, that the near real-time counts will update on the Tweets with a long refresh interval. The reason is the application server waits for multiple counts submitted by the various servers placed in different regions. We have a detailed chapter on Sharded Counters, explaining how it works in real-time applications.

The complete design overview

This section will discuss what happens in the back-end system when the end users generate multiple requests. The following are the steps:

- First, end users get the address of the nearest load balancer from the local DNS.
- Load balancer routes end usersâ requests to the appropriate servers according to the requested services. Here, weâll discuss the Tweet, timeline, and search services.
- Tweet service: When end users perform any operation, such as posting a Tweet or liking other Tweets, the load balancers forward these requests to the server handling the Tweet service. Consider an example where users post Tweets on Twitter using the /postTweet API. The

server (Tweet service) receives the requests and performs multiple operations. It identifies the attachments (image, video) in the Tweet and stores them in the Blobstore. Text in the Tweets, user information, and all metadata are stored in the different databases (Manhattan, MySQL, Postgresql, Vertica). Meanwhile, real-time processing, such as pulling Tweets, user interactions data, and many other metrics from the real-time streams and client logs, is achieved in the Apache Kafka.

Later, the data is moved to the cloud pub-sub through an event processor. Next, data is transferred for deduping and aggregation to the BigQuery through Cloud Dataflow. Finally, data is stored in the Google Cloud Bigtable, which is fully managed, easily scalable, and sorted keys.

- Timeline service: Assume the user sends a home timeline request using the /viewHome_timeline API. In this case, the request is forwarded to the nearest CDN containing static data. If the requested data is not found, it's sent to the server providing timeline services. This service fetches data from different databases or stores and returns the Top-k Tweets. This service collects various interactions counts of Tweets from different sharded counters to decide the Top-k Tweets. In a similar way, we will obtain the Top-k trends attached in the response to the timeline request.

- Search service: When users type any keyword(s) in the search bar on Twitter, the search request is forwarded to the respective server using the /searchTweet API. It first looks into the RAM in Apache Lucene to get real-time Tweets (Tweets that have been published recently). Then, this server looks up in the index server and finds all Tweets that contain the requested keyword(s). Next, it considers multiple factors, such as time, or location, to rank the discovered Tweets. In the end, it returns the top Tweets.

- We can use the Zipkin tracing system that performs sampling on requests. Moreover, we can use Zookeeper to maintain different data, including configuration information, distributed synchronization, naming registry, and so on.

[Design](./twitter_design.jpg)

Design Twitter/High-level Design of Twitter/README.md

High-level Design of Twitter

User-system interaction

Let's begin with the high-level design of our Twitter system. We'll initially highlight and discuss the building blocks, as well as other components, in the context of the Twitter problem briefly. Later on, we'll dive deep into a few components in this chapter.

[Twitter components](./components.jpg)

- Users post Tweets delivered to the server through the load balancer. Then, the system stores it in persistent storage.

- DNS provides the specified IP address to the end user to start communication with the requested service.

- CDN is situated near the users to provide requested data with low latency. When users search for a specified term or tag, the system first searches in the CDN proxy servers containing the most frequently requested content.

- Load balancer chooses the operational application server based on traffic load on the available servers and the user requests.

- Storage system represents the various types of storage (SQL-based and NoSQL-based) in the above illustration. We'll discuss significant storage systems later in this chapter.

- Application servers provide various services and have business logic to orchestrate between

different components to meet our functional requirements.

We have detailed chapters on DNS, CDN, specified storage systems (Databases, Key-value store, Blob store), and Load balancers in our building blocks section. We'll focus on further details specific to the Twitter service in the coming lessons. Let's first understand the service API.

API design

This section will focus on designing various APIs regarding the functionalities we are providing. We learn how users request various services through APIs. We'll only concentrate the significant parameters of the APIs that are relevant to our design. Although the front-end server can call another API or add more parameters in the API received from the end users to fulfill the given request, we consider all relevant arguments specified for the particular request in a single API. Let's develop APIs for each of the following features:

- Post Tweet
- Like or dislike Tweet
- Reply to Tweet
- Search Tweet
- View user or home timeline
- Follow or unfollow the account
- Retweet a Tweet

Post Tweet

The POST method is used to send the Tweet to the server from the user through the /postTweet API.

```
'''  
postTweet(user_id, access_type, tweet_type, content, tweet_length, media_field,  
list_of_followers, post_time, tweet_location, list_of_used_hashtags, list_of_tagged_people)  
'''
```

Let's discuss a few of the parameters:

```
'''
```

Parameter	Description
user_id	It indicates the unique ID of the user who posted the Tweet.
access_type	It tells us whether the Tweet is protected (that is, only visible to followers) or public.
tweet_type	It indicates whether the Tweet is text-based, video-clip based, image(s)-based, or consisting of different types.
content	It specifies the Tweet's actual content (text).
tweet_length	It represents the text length in the Tweet. In the case of video, it tells us the duration and size of a video.
media_field	It specifies the type of media (image, video, GIF, and so on) delivered in each Tweet.
list_of_followers	It provides the current followers of the account posting the Tweet. This parameter is filled by the front-end server after it fetches this information using another API.

The rest of the parameters are self-explanatory.

```
'''
```

```
'''
```

Note: Twitter uses the Snowflake service to generate unique IDs for Tweets. We have a detailed

chapter (Sequencer) that explains this service.
'''

'''

Question 1

At most, how many hashtags can a Tweet have?

Answer

The text limit is 280 characters in a Tweet. So, users can use the hashtag(s) such that text length (including plain text, any links, and hashtags) does not exceed the limit of 280 characters.
'''

'''

Question 2

What is the need for the list_of_tagged_people in the /postTweet API?

Answer

The system has to notify the people tagged in the Tweet.
'''

Like or dislike Tweet

The /likeTweet API is used when users like public Tweets.
'''

likeTweet(user_id, tweet_id, tweeted_user_id, user_location)
'''

'''

Parameter	Description
-----------	-------------

user_id	It indicates the unique ID of the user who liked the Tweet.
---------	---

tweet_id	It indicates the Tweet's unique ID.
----------	-------------------------------------

tweeted_user_id	This is the unique ID of the user who posted the Tweet.
-----------------	---

user_location	It denotes the location of the user who liked the Tweet.
---------------	--

'''

The parameters above are also used in the /dislikeTweet API when users dislike others's Tweet.

Reply to Tweet

The /replyTweet API is used when users reply to public Tweets.
'''

replyTweet(user_id, tweet_id, tweeted_user_id, reply_type, reply_length, ,list_of_followers)
'''

'''

Parameter	Description
-----------	-------------

list_of_followers	It specifies the list of user followers who replied to someone's Tweet.
-------------------	---

'''

The reply_type and reply_length parameters are the same as tweet_type and tweet_length respectively.

Search Tweet

When the user searches any keyword in the home timeline, the GET method is used. The following is the /searchTweet API:
'''

searchTweet(user_id, search_term, max_result, exclude, media_field, expansions, sort_order, next_token, user_location)

```
'''
```

Some new parameters introduced in this case are:

```
'''
```

Parameter	Description
-----------	-------------

search_term	It is a string containing the search keyword or phrase.
-------------	---

max_result	It is the number of Tweets returned per response page. By default, the Tweets per response is 10.
------------	---

exclude	It specifies what to exclude from the returned Tweets, that is, replies and retweets. The maximum limit on returned Tweets is 3200, but when we exclude replies, the maximum limit is reduced to 800 Tweets.
---------	--

media_field	It specifies the media (image, video, GIF) delivered in each returned Tweet.
-------------	--

expansions	It enables us to request additional data objects in the returned Tweets, such as any mentioned user, referenced Tweet, attached media, attached places objects, and so on.
------------	--

sort_order	It specifies the order in which Tweets are returned. By default, it will return the most recent Tweets first.
------------	---

next_token	It is used to get the next page of results. For instance, if max_result is set to 100 Tweets, and the result set contains 200 Tweets, then the value of next_token is directly pulled from the response to request the next page containing the following 100 Tweets. The last result (page) will not have a next_token.
------------	--

```
'''
```

Response

Let's look at a sample response in JSON format. The id is the user's unique ID who posted Tweet and the text is the Tweet's content. The result_count is the count of the returned Tweets which we set in the max_result in the request. Here, we're displaying the default fields only.

```
'''json
```

```
{
  "data": [
    {
      "id": "7300333948034441183",
      "text": "This is a most matched Tweet"
    },
    {
      "id": "6498431343154916376",
      "text": "This is next to most matched Tweet"
    },
    :
    :
    :
    {
      "id": "6427456107642019844",
      "text": "This is the last matched Tweet in the current page of the result."
    }
  ],
  "meta": {
    "newest_id": "7300333948034441183",
    "oldest_id": "6427456107642019844",
    "result_count": 10
  }
},
'''
```

'''

Note: Twitter performs various types of searches. The following are two of them:

One search type returns the result of the last seven days, which all registered users usually use.

The other type returns all matching results on all Tweets ever posted (remind that service does not delete a posted Tweet). Indeed, matches can contain the first Tweet on Twitter. This search is usually used for academic research.

'''

View home_timeline

The GET method is suitable when users view their home timelines through the /viewHome_timeline API.

'''

viewHome_timeline(user_id, tweets_count, max_result, exclude, next_token, list_of_followers, user_location)

'''

In the /viewUser_timeline API, we'll exclude the list_of_followers and user_location to get user timeline.

'''

Question

Which parameter in the viewHome_timeline method is the most relevant when deciding which promoted ads (Tweets) to be returned in response?

Answer

The decision to send specified promoted ads is made on the user_location parameter. For example, the user belongs to the NewYork city, so most probably it gets promoted ads associated or originated in that region.

'''

Follow the account

The /followAccount API is used when users follow someone's account on Twitter.

'''

followAccount(account_id, followed_account_id,)

'''

'''

Parameter	Description
account_id	It specifies the unique ID of a user who follows that account on Twitter.

followed_account_id	It indicates the unique ID of the account that the user follows.
---------------------	--

'''

The /unfollowAccount API will use the same parameters when a user unfollows someone's account on Twitter.

Retweet a Tweet

When a registered user Retweets (re-posts) someone's Tweet on Twitter, the following /retweet API is called:

'''

retweet(user_id, tweet_id, retweet_user_id, list_of_followers)

'''

The same parameters will be required in the /undoRetweet API when users undo a Retweet of someone's Tweet.

Design Twitter/Quiz on Twitter's Design/README.md

Quiz on Twitter's Design

'''

1 Suppose you work as an engineer at a well-known company providing social services. Your task is to improve real-time event processing to get frequent results. Which tool below will help you the most in this scenario?

A)

BigQuery

B)

Apache Kafka

C)

Vertica

D)

FlockDB

'''

'''

2 Assume that an account of a public figure with millions of followers posts a Tweet on Twitter. Your system will need to handle millions of interactions with this Tweet by the account's millions of followers. Which solution below is the best way to handle this traffic assuming that the system knows the audience details of the Tweet?

A)

Dedicate a few servers to the public figure in the central data center(s).

B)

Increase or dedicate some servers to particular locations where a large portion of the audience belongs.

C)

Increase the number of servers in all Twitter's data centers situated in different regions to handle massive traffic.

D)

Show the Tweet to followers region-wise to avoid handling massive traffic.

'''

'''

3 When the user sends a home timeline request, the system responds with a specified number of Top-k tweets to the user. On what basis does the system identify the Top-k tweets for a particular user?

A)

The user liked the Tweets (posted by the unfollowed account) and Trends.

B)

The user's followers and location.

C)

The user's followed accounts' Tweets and Retweeted Tweets by the followed accounts.

D)

Location and unfollowed accounts

'''

'''

4 Sharded counters are an efficient solution to count multiple interactions (like or view) on

thousands of celebritiesâ Tweets individually. The problem is that when the user interacts with any Tweet, it needs an immediate response (for instance, an increase or decrease in the like count). Which solution below will help in this regard?

A)

Maintain counters in the server cache.

B)

Place specified shards of the particular counters near the end users based on the celebrityâs history (Last few Tweetsâ audiences).

C)

Increase the number of shards of multiple counters that belong to the Tweet in the central data center.

D)

Create a real-time counter in the client cache with the specified detail of the Tweet.

'''

'''

5 (Select all that apply.) We discussed how the client-side load balancers work and what kind of benefit it provides. Which of the options below were identified as possible drawbacks of client-side load balancers?

A)

They act as a single point of failure.

B)

Theyâre not suitable for small-scale applications because there are no dedicated servers for the services.

C)

They increase network hop.

D)

They make it more difficult to update all clients when servers add/drop or algorithm modify.

'''

Answers

'''

1

Suppose you work as an engineer at a well-known company providing social services. Your task is to improve real-time event processing to get frequent results. Which tool below will help you the most in this scenario?

A)

BigQuery

Explanation

BigQuery is a serverless, fully managed data warehouse that allows for scalable analysis of vast amounts of data.

Correct Answer

B)

Apache Kafka

Explanation

Apache Kafka is a framework for distributed real-time event processing and storage.

C)
Vertica

Explanation

Vertica is a columnar data storage that can manage massive amounts of data and provides lightning-fast query performance in usually time-consuming circumstances.

D)
FlockDB

Explanation

FlockDB is a distributed, fault-tolerant graph database for handling large network graphs.
'''

'''

2

Assume that an account of a public figure with millions of followers posts a Tweet on Twitter. Your system will need to handle millions of interactions with this Tweet by the account's millions of followers. Which solution below is the best way to handle this traffic, assuming that the system knows the audience details of the Tweet?

A)
Dedicate a few servers to the public figure in the central data center(s).

Explanation

Increase burden on central data center and also increase latency.

Correct Answer

B)
Increase or dedicate some servers to particular locations where a large portion of the audience belongs.

C)
Increase the number of servers in all Twitter's data centers situated in different regions to handle massive traffic.

Explanation

If most of the traffic is coming from a few regions, increasing servers in other datacenters is useless and resource wastage.

D)
Show the Tweet to followers region-wise to avoid handling massive traffic.

Explanation

All users having connections with tweets must get the tweet. So, it is not fulfilling the service.
'''

'''

3

When the user sends a home timeline request, the system responds with a specified number of Top-k tweets to the user. On what basis does the system identify the Top-k tweets for a particular user?

A)
The user liked the Tweets (posted by the unfollowed account) and Trends.

Explanation

Trends metric is irrelevant here.

B)

The user's followers and location.

Explanation

Users will not get the tweets posted by their followers.

Correct Answer

C)

The user's followed accounts' Tweets and Retweeted Tweets by the followed accounts.

D)

Location and unfollowed accounts

Explanation

Users will not get unfollowed accounts' tweets as long as tweets are not promoted.

'''

'''

4

Sharded counters are an efficient solution to count multiple interactions (like or view) on thousands of celebrities' Tweets individually. The problem is that when the user interacts with any Tweet, it needs an immediate response (for instance, an increase or decrease in the like count). Which solution below will help in this regard?

A)

Maintain counters in the server cache.

Explanation

It is difficult to manage millions of counters in the cache because they are limited in size (memory).

Correct Answer

B)

Place specified shards of the particular counters near the end users based on the celebrity's history (Last few Tweets' audiences).

Explanation

In this case, the nearest server will immediately respond to the end-users who liked or viewed the tweet.

C)

Increase the number of shards of multiple counters that belong to the Tweet in the central data center.

Explanation

It will increase latency because central data centers are distant from the end-users and also increase the central servers' burden.

D)

Create a real-time counter in the client cache with the specified detail of the Tweet.

Explanation

This solution has multiple problems, such as limited client cache. For instance, if the end-user performs many operations (like, view, reply) on many tweets, creating and managing many counters in the cache is difficult. Second, what if the client liked a post and the relevant counter is incremented in the client cache. But somehow client cache gets empty (due to any failure) before moving that details to the application server. So, this solution is not

reliable.
'''

'''

5

(Select all that apply.) We discussed how the client-side load balancers work and what kind of benefit it provides. Which of the options below were identified as possible drawbacks of client-side load balancers?

A)

They act as a single point of failure.

Selected Option

B)

They are not suitable for small-scale applications because there are no dedicated servers for the services.

Explanation

Each service is managed within the backend server. Therefore, no such type of communication (server selection service to service) is required among the server.

C)

They increase network hop.

Selected Option

D)

They make it more difficult to update all clients when servers add/drop or algorithm modify.

'''

Design Twitter/Requirements of Twitter's Design/README.md

Requirements of Twitter's Design

Requirements

Let's understand the functional and non-functional requirements below:

Functional requirements

The following are the functional requirements of Twitter:

- Post Tweets: Registered users can post one or more Tweets on Twitter.
- Delete Tweets: Registered users can delete one or more of their Tweets on Twitter.
- Like or dislike Tweets: Registered users can like and dislike public and their own Tweets on Twitter.
- Reply to Tweets: Registered users can reply to the public Tweets on Twitter.
- Search Tweets: Registered users can search Tweets by typing keywords, hashtags, or usernames in the search bar on Twitter.
- View user or home timeline: Registered users can view the user's timeline, which contains their own Tweets. They can also view the home's timeline, which contains followers' Tweets on Twitter.
- Follow or unfollow the account: Registered users can follow or unfollow other users on Twitter.
- Retweet a Tweet: Registered users can Retweet public Tweets of other users on Twitter.

Non-functional requirements

- Availability: Many people and organizations use Twitter to communicate time-sensitive information (service outage messages). Therefore, our service must be highly available and have a good uptime percentage.
- Latency: The latency to show the most recent top Tweets in the home timeline could be a little high. However, near real-time services, such as Tweet distribution to followers, must have low latency.
- Scalability: The workload on Twitter is read-heavy, where we have many readers and relatively few writers, which eventually requires the scalability of computational resources. Some estimates suggest a 1:1000 write-to-read ratio for Twitter. Although the Tweet is limited to 280 characters, and the video clip is limited to 140s by default, we need high storage capacity to store and deliver Tweets posted by public figures to their millions of followers.
- Reliability: All Tweets remain on Twitter. This means that Twitter never deletes its content. So there should be a promising strategy to prevent the loss or damage of the uploaded content.
- Consistency: There's a possibility that a user on the east coast of the US does not get an immediate status (like, reply, and so on.) update on the Tweet, which is liked or Retweeted by a user on the west coast of the US. However, the user on the west coast of the US needs an immediate status update on his like or reply. An effective technique is needed to offer rapid feedback to the user (who liked someone's post), then to other specified users in the same region, and finally to all worldwide users linked to the Tweet.

[Non-functional requirements of Twitter](./nfr.jpg)

Furthermore, we must identify which essential resources must be estimated in the Twitter design. We used Twitter as an example in our Back-of-the-Envelope chapter, so we won't repeat that exercise here.

Building blocks we will use

Twitter's design utilizes the following building blocks that we discussed in the initial chapters.

[Building blocks used in Twitter design](./bb.jpg)

- DNS is the service that maps human-friendly Twitter domain names to machine-readable IP addresses.
 - Load balancers distribute the read/write requests among the respective services.
 - Sequencers generate the unique IDs for the Tweets.
 - Databases store the metadata of Tweets and users.
 - Blob stores store the images and video clips attached with the Tweets.
 - Key-value stores are used for multiple purposes such as indexing, identifying the specified counter to update its value, identifying the Tweets of a particular user and many more.
 - Pub-sub is used for real-time processing such as elimination of redundant data, organizing data, and much more.
 - Sharded counters help to handle the count of multiple features such as viewing, liking, Retweeting, and so on, of the accounts with millions of followers.
 - A cache is used to store the most requested and recent data in RAM to give users a quick response.
 - CDN helps end users to access the data with low latency.
 - Monitoring analyses all outgoing and incoming traffic, identifies the redundancy in the storage system, figures out the failed node, and so on.
- In the next lesson, we'll discuss the design of the Twitter system.

Design Twitter/System Design Twitter/README.md

System Design: Twitter

Twitter

Twitter is a free microblogging social network where registered users post messages called â Tweetsâ . Users can also like, reply to, and retweet public tweets. Twitter has about 397 million users as of 2021 that is proliferating with time. One of the main reasons for its popularity is the vast sharing of the breaking news on the platform. Another important reason is that Twitter allows us to engage with and learn from people belonging to different communities and cultures.

[A user performs various operation on Twitter](./operations.jpg)

The illustration below shows the country-wise userbase of Twitter as of January 2022 (source: Statista), where the US is taking the lead. Such statistics are important when we design our infrastructure because it allows us to allocate more capacity to the users of a specific region, and traffic served from near specific users.

[Country wise userbase on Twitter](./stats.jpg)

How will we design Twitter?

Weâ ll divide Twitterâ s design into four sections:

1. Requirements: This lesson describes the functional and non-functional requirements of Twitter. Weâ ll also estimate multiple aspects of Twitter, such as storage, bandwidth, and computational resources.
2. Design: In this lesson, weâ ll discuss the high-level design of Twitter in this lesson. We also briefly explain the API design and identify the significant components of the Twitter architecture. Moreover, we will discuss how to manage the Top-k problem, such as Tweets liked or viewed by millions of users on Twitter.
3. Client-side load balancers: This lesson discusses how Twitter performs load balancing for its microservices system to manage billions of requests between various servicesâ instances. Furthermore, we also see why Twitter uses a customized load-balancing technique instead of other commonly used approaches.
4. Quiz: Finally, weâ ll reinforce major concepts of Twitter design with a quiz. Letâ s begin with defining Twitterâ s requirements.

Design Typeahead Suggestion/Data Structure for Storing Prefixes/README.md

Data Structure for Storing Prefixes

The trie data structure

Before we move on to the discussion of the detailed design of the typeahead suggestion system, we must choose an efficient data structure to store the prefixes. Prefixes are the groups of characters a user types. The issue weâ re attempting to tackle is that we have many strings that we need to store in a way that allows users to search for them using any prefix. Our service suggests the next words that match the provided prefix. Letâ s suppose our database contains phrases UNITED, UNIQUE, UNIVERSAL, and UNIVERSITY. Our system should suggest â UNIVERSALâ as â UNIVERSITYâ when the user types â UNIV.â

There should be a method that can efficiently store our data and help us conduct fast searches because we have to handle a lot of requests with minimal latency. We canâ t rely on a database for this because providing suggestions from the database takes longer as compared to reading suggestions from the RAM. Therefore, we need to store our index in memory in an efficient data structure. However, for durability and availability, this data is stored in the database.

'''

Recall from the Back-of-the-Envelope Calculation chapter that reading 1MB of data sequentially from RAM takes 9 microseconds while from persistent storage like SSD takes 200 microseconds.

'''

The trie (pronounced â tryâ) is one of the data structures thatâ s best suited to our needs.

trie is a tree-like data structure for storing phrases, with each tree node storing a character in the phrase in order. If we needed to store UNITED, UNIQUE, UNIVERSAL, and UNIVERSITY in the trie, it would look like this:

[The trie for UNITED, UNIQUE, UNIVERSAL, and UNIVERSITY](./trie1.jpg)

If the user types â UNIV,â our service can traverse the trie to go to the node V to find all terms that start with this prefixâ for example, UNIVERSAL, UNIVERSITY, and so on.

The trie can combine nodes as one where only a single branch exists, which reduces the depth of the tree. This also reduces the traversal time, which in turn increases the efficiency. As an example, a space- and time-efficient model of the above trie is the following:

[A reduced Trie for UNITED, UNIQUE, UNIVERSAL, and UNIVERSITY](./reduced.jpg)

Track the top searches

Since our system keeps track of the top searches and returns the top suggestion, we store the number of times each term is searched in the trie node. Letâ s say that a user searches for UNITED 15 times, UNIQUE 20 times, UNIVERSAL 21 times, and UNIVERSITY 25 times. In order to provide the top suggestions to the user, these counts are stored in each node where these terms terminate. The resultant trie looks like this:

[A trie showing the search frequency for UNITED, UNIQUE, UNIVERSAL, and UNIVERSITY](./freq.jpg)

If a user types â UNI,â the system starts traversing the tree under the root node for UNI. By comparing all the terms originating from the root node, the system provides suggestions of all the possible words. Since the frequency of the word UNIVERSITY is high, it appears at the top. Similarly, the frequency of the word UNITED is relatively low, so it appears last. If the user picks UNIQUE from the list of suggestions, the number against UNIQUE increases to 21.

Question

We reduced the time to traverse the trie by combining nodes with single branches and reducing the number of levels. Is there any other way to minimize the trie traversal time?

Answer

One way to reduce the trie traversal time is to pre-compute and save the top ten (or any number of our choosing) suggestions for every prefix in the node. This means that instead of traversing the trie each time a user types in â UNIVERSâ into the search box, the system will have precomputed, sorted, and stored the solution to the prefix UNIVERSâ that is, UNIVERSITY, UNIVERSAL, and so onâ inside the node that carries the prefix UNIVERS. However, this approach requires extra space to save precomputed results.

'''

Pre-compute: Instead of traversing the tree each time, the system traverses the tree only once and stores the suggestions in each node which we call pre-computed suggestions.

'''

[Trie](./trie.jpg)

Trie partitioning

We aim to design a system like Google that we can use to handle billions of queries every second. One server isnâ t sufficient to handle such an enormous amount of requests. In addition to this, storing all the prefixes in a single trie isnâ t a viable option for the systemâ s availability, scalability, and durability. A good solution is to split the trie into multiple tries for a better user experience.

Letâ s assume that the trie is split into two parts, and each part has a replica for durability.

purposes. All the prefixes starting from 'A' to 'M' are stored on Server/01, and the rest are stored on Server/02. Similarly, all the prefixes starting from 'N' to 'Z' are stored on Server/03, and the replica is stored on Server/04. It should be noted that this simple technique doesn't always balance the load equally because some prefixes have many more words while others have fewer. We use this simple technique to understand partitioning.

We can split the trie into as many parts as we wish to distribute the load on to different servers and achieve the desired performance.

'''

Prefixes	Partitioned Trie	
	Primary	Secondary
A to M	Server/01	Server/02
N to Z	Server/03	Server/04

'''

'''

Question

Where will the mapping between the prefixes and their primary and secondary storage be stored? Who will manage and direct the requests to these servers?

Answer

In a distributed system where multiple clusters consisting of several servers can be used for specific service, we use a cluster manager like ZooKeeper to store the mapping between clusters.

'''

Process a query after partitioning

When a user types a query, it hits the load balancer and is forwarded to one of the application servers. The application server searches the appropriate trie depending on the prefix typed by the user. For example, if a user types something starting from 'U', it either accesses Server/03 or Server/04 since both have the tries stored on them that have prefixes starting with 'U'.

Update the trie

Billions of searches every day give us hundreds of thousands of queries per second. Therefore, the process of updating a trie for every query is highly resource intensive and time-consuming and could hamper our read requests. This issue can be resolved by updating the trie offline after a specific interval. To update the trie offline, we log the queries and their frequency in a hash table and aggregate the data at regular intervals. After a specific amount of time, the trie is updated with the aggregated information. After the update of the trie, all the previous entries are deleted from the hash table.

'''

Prefixes and Their Frequencies Updated Periodically		
Prefix	Time Interval (One Hour)	Frequency
UNIVERSITY	1st hour	25
UNIVERSITY	2nd hour	60
UNIVERSITY	3rd hour	100

'''

We can put up a MapReduce (MR) job to process all of the logging data regularly, let's say every 15 minutes. These MR services calculate the frequency of all the searched phrases in the previous 15 minutes and dump the results into a hash table in a database like Cassandra. After that, we may further update the trie with the new data. We can update the current copy of the

trie with all of the new words and their frequencies. We should perform this offline because our priority is to provide suggestions to users instead of keeping them waiting.

'''

MapReduce is a big data processing engine.

'''

Primarily, we can update the trie using the following two approaches.

- We can replicate the trie on each server to update it offline. After that, we can start using it for suggestions and throw away the old ones.
- Another way is to have one primary copy and several secondary copies of the trie. While the main copy is used to answer the queries, we may update the secondary copy. We may also make the secondary our main copy once the upgrade is complete. We can then upgrade our previous primary which will then be able to serve the traffic as well.

'''

Question

If the prefix frequencies keep increasing over time, the corresponding integers storing them can overflow. How can we manage this issue?

Answer

We can normalize frequencies by mapping them in a range, let's say between zero and 1,000. Alternatively, we can stop any further additions after a certain threshold is reached, assuming that any prefix reaching that threshold is at the top of the rankings.

'''

Design Typeahead Suggestion/Detailed Design of the Typeahead Suggestion System/README.md

Detailed Design of the Typeahead Suggestion System

Detailed design

Let's go over the flow and interaction of the components shown in the illustration below. Our design is divided into two main parts:

- A suggestion service
- An assembler

[The detailed design of the typeahead suggestion system](./design.jpg)

Suggestion service

At the same time that a user types a query in the search box, the `getSuggestions(prefix)` API calls hit the suggestions services. The top ten popular queries are returned from the distributed cache, Redis.

Assembler

In the previous lesson, we discussed how tries are built, partitioned, and stored in the database. However, the creation and updation of a trie shouldn't come in the critical path of user's query. We shouldn't update the trie in real time for the following reasons:

- There could be millions of users entering queries every second. During such phases with large amounts of incoming traffic, updating the trie in real time on every query can slow down our suggestion service.
 - We have to provide top suggestions that might not frequently change after the creation or updation of the trie. So, it's less important to update the trie frequently.
- In light of the reasons given above, we have a separate service called an assembler that's responsible for creating and updating tries after a certain configurable amount of time. The assembler consists of the following different services:

- Collection service: Whenever a user types, this service collects the log that consists of phrases, time, and other metadata and dumps it in a database that's processed later. Since t

size of this data is huge, the Hadoop Distributed File System (HDFS) is considered a suitable storage system for storing this raw data.

'''

Phrases	Raw Data Collected by the Collection Service Date and Time (DD-MM-YYYY HH:MM:SS)
UNIVERSAL	23-03-2022 10:16:18
UNIVERSITY	23-03-2022 10:20:11
UNIQUE	23-03-2022 10:21:10
UNIQUE	23-03-2022 10:22:24
UNIVERSITY	23-03-2022 10:25:09

'''

An example of the raw data from the collection service is shown in the following table. We record the time so that the system knows when to update the frequency of a certain phrase.

- Aggregator: The raw data collected by the collection service is usually not in a consolidated shape. We need to consolidate the raw data to process it further and to create or update the tries. An aggregator retrieves the data from the HDFS and distributes it to different workers. Generally, the MapReducer is responsible for aggregating the frequency of the prefixes over a given interval of time, and the frequency is updated periodically in the associated Cassandra database. Cassandra is suitable for this purpose because it can store large amounts of data in a tabular format.

The following table shows the processed and consolidated data within a particular period. This table is updated regularly by the aggregator and is stored in a hash table in a database like Cassandra. For simplicity, we assume that our data is case insensitive.

'''

Phrases	Useful Information Extracted from the Raw Data Frequency	Time Interval
UNIVERSAL	1400	1st 15 minutes
UNIVERSITY	1340	1st 15 minutes
UNIQUE	1200	1st 15 minutes

'''

- Trie builder: This service is responsible for creating or updating tries. It stores these new and updated tries on their respective shards in the trie database via ZooKeeper. Tries are stored in persistent storage in a file so that we can rebuild our trie easily if necessary. NoSQL document databases such as MongoDB are suitable for storing these tries. This storage of a trie is needed when a machine restarts.

The trie is updated from the aggregated data in the Cassandra database. The existing snapshot of a trie is updated with all the new terms and their corresponding frequencies. Otherwise, a new trie is created using the data in the Cassandra database.

Once a trie is created or updated, the system makes it available for the suggestion service.

'''

Question

Should we collect data and build a trie per user, or should it be shared among all users?

Answer

Since we aim to design a system on a scale thatâs similar to Google Search, billions of users will be using our service. Since the number of users would be huge, maintaining a tree for each user would become complex and time-consuming. Thereâs also a possibility of duplicated trees as several users have typed some common searches, resulting in resource wastage.

Therefore, our design assumes a common trie thatâs shared among users, where the ranking would be based on single phrases in the trie and the frequency of the terms.

'''

Design Typeahead Suggestion/Evaluation of the Typeahead Suggestion Systemâs Design/README

Evaluation of the Typeahead Suggestion Systemâs Design

Fulfill requirements

The non-functional requirements of the proposed typeahead suggestion system are low latency, fault tolerance, and scalability.

- Low latency: There are various levels at which we can minimize the systemâs latency. We can minimize the latency with the following options:

- Reduce the depth of the tree, which reduces the overall traversal time.
- Update the trie offline, which means that the time taken by the update operation isnât on the clientâs critical path.
- Use geographically distributed application and database servers. This way, the service is provided near the user, which also reduces any communication delays and aids in reducing latency.
- Use Redis and Cassandra cache clusters on top of NoSQL database clusters.
- Appropriately partition tries, which leads to a proper distribution of the load and results in better performance.
- Fault tolerance: Since the replication and partitioning of the trees are provided, the system operates with high resilience. If one server fails, others are on standby to deliver the services.

- Scalability: Since our proposed system is flexible, more servers can be added or removed as the load increases. For example, if the number of queries increases, the number of partitions or shards of the trees is increased accordingly.

'''

Approaches to Fulfill Non-functional Requirements

Non-functional Requirements

Approaches

Low latency
traversal faster

- Reducing the depth of the tries makes the
- Updating the tries offline and not in real time)
- Partitioning of the tries
- Caching servers

Fault tolerance

- Replicating the tries and the NoSQL databases

Scalability
the incoming traffic

- Adding or removing application servers based on
- Increasing the trie partitions

'''

Client-side optimisation

To improve the userâs experience, we can implement the following client-side optimizations:

- The client should only attempt to contact the server if the user hasnât pressed any keys f

some timeâ for example, any delay greater than 160 ms, which is the average delay between two keystrokes. This way, we can also avoid unnecessary bandwidth consumption. This suggestion might not be useful when a user is typing rapidly.

- The client can initially wait till the user types a few characters.
- Clients can save a local copy of the recent history of suggestions. The rate of reuse of recent history in the suggestions list is relatively high.
- One of the most crucial elements is establishing a connection with the server as soon as possible. The client can establish a connection with the server as soon as the user visits the search page. As a result, the client doesnâ t waste time establishing the connection when the user inputs the first character. Usually, the connection is established with the server via a WebSocket protocol.
- For efficiency, the server can push a portion of its cache to CDNs and other edge caches at Internet exchange points (IXPs) or even inside a clientâ s Internet service provider (ISP).

Personalization

Users receive typeahead suggestions based on their previous searches, location, language, and other factors. We can save each userâ s personal history on the server separately and cache it on the client. Before transmitting the final set to the user, the server might include these customized phrases. Personalized searches should always take precedence over other types of searches.

Summary

In this design problem, we learned how pushing resource-intensive processing to the offline infrastructure and using appropriate data structures enables us to serve our customers with low latency. Many optimizations lend themselves to specific use cases. We saw multiple optimizations on our trie data structures for condensed data storage and quick serving.

Design Typeahead Suggestion/High-level Design of the Typeahead Suggestion System/README.md

High-level Design of the Typeahead Suggestion System

High-level design

According to our requirements, the system shouldnâ t just suggest queries in real time with minimum latency but should also store the new search queries in the database. This way, the user gets suggestions based on popular and recent searches.

Our proposed system should do the following:

- Provide suggestions based on the search history of the user.
- Store all the new and trending queries in the database to include them in the list of suggestions.

When a user starts typing a query, every typed character hits one of the application servers. Letâ s assume that we have a suggestions service that obtains the top ten suggestions from the cache, Redis, and returns them as a response to the client. In addition to this, suppose we have another service known as an assembler. An assembler collects the user searches, applies some analytics to rank the searches, and stores them in a NoSQL database thatâ s distributed across several nodes.

[The high-level design of the typeahead suggestion system](./hld.jpg)

Furthermore, we also need load balancers to distribute the incoming requests evenly. We also add application servers as entry points for clients so that they can forward requests to the appropriate microservices. These web servers encapsulate the internal system architecture and provide other services, such as authentication, monitoring, request shaping, management, and more.

API design

Since the system provides suggestions to the user and adds trending queries to the databases, we need the following APIs.

```
### Get suggestion
'''
```

```
getSuggestions(prefix)
'''
```

This API call retrieves suggestions from the servers. This call is made via the suggestion service and returns the response to the client.

The following table explains the parameter that's passed to the API call:

Parameter	Description
prefix	This refers to whatever the user has typed in the search bar.

```
### Add trending queries to the database
'''
```

```
addToDatabase(query)
'''
```

This API call adds a trending query to the database via an assembler if the query has already been searched and has crossed a certain threshold.

Parameter	Description
query	This represents a frequently searched query that crosses the predefined limit.

Question

Instead of updating the whole page, we just need to update the suggested query in the search box in real time. What technique can we use for this purpose?

Answer

We should update the query asynchronously using AJAX. AJAX is a set of web development techniques. It allows web pages to exchange a small amount of data, as in the typeahead suggestion system, with the server without interfering with the display and behaviors of the existing web page.

In the next lesson, we'll learn about an efficient data structure that's used to store the suggestions or prefixes.

```
## Design Typeahead Suggestion/Quiz on the Typeahead Suggestion System's Design/README.md
```

```
# Quiz on the Typeahead Suggestion System's Design
'''
```

```
1
Which component is responsible for updating the trie offline in the design of the typeahead suggestion system?
```

- A)
The assembler
- B)
The suggestion service
- C)
ZooKeeper
- D)
None of the above

'''

'''

2

What is the purpose of MapRreduce in the design of the typeahead suggestion system?

A)

To disseminate data toward a database for storage

B)

To aggregate the prefix count

C)

To increase the number of required operations for optimization

D)

None of the above

'''

'''

3

Why do we aggregate the counts and update them offline in their respective shards?

A)

To enhance the write operations

B)

To provide accurate results to the user

C)

To minimize the latency of the read operations

D)

None of the above

'''

'''

4

(Select all that apply.) Why is the trie partitioned and stored on several servers?

A)

To reduce the load on a single server

B)

To store more data

C)

To increase the performance of the system

D)

None of the above

'''

'''

5

(Select all that apply.) The replication of the trie partition is required to enhance _____

A)

fault tolerance

B)
the durability of the data

C)
the security of the data
'''

Answers

1. (A)
2. (B)
3. (C)
4. (A) (C)
5. (A) (B)

Design Typeahead Suggestion/Requirements of the Typeahead Suggestion System's Design/README.md

Requirements of the Typeahead Suggestion System's Design

Requirements

In this lesson, we look into the requirements and estimated resources that are necessary for the design of the typeahead suggestion system. Our proposed design should meet the following requirements.

Functional requirements

The system should suggest top N (let's say top ten) frequent and relevant terms to the user based on the text a user types in the search box.

Non-functional requirements

- Low latency: The system should show all the suggested queries in real time after a user types. The latency shouldn't exceed 200 ms. A study suggests that the average time between keystrokes is 160 milliseconds. So, our time-budget of suggestions should be greater than 160 ms to give a real-time response. This is because if a user is typing fast, they already know what to search and might not need suggestions. At the same time, our system response should be greater than 160 ms. However, it should not be too high because in that case, a suggestion might be stale and less useful.

- Fault tolerance: The system should be reliable enough to provide suggestions despite the failure of one or more of its components.

- Scalability: The system should support the ever-increasing number of users over time.

Resource estimation

As was stated earlier, the typeahead feature is used to enhance the user experience while typing a query. We need to design a system that works on a scale that's similar to Google Search. Google receives more than 3.5 billion searches every day. Designing such an enormous system is a challenging task that requires different resources. Let's estimate the storage and bandwidth requirements for the proposed system.

Storage estimation

Assuming that out of the 3.5 billion queries per day, two billion queries are unique and need to be stored. Let's also assume that each query consists of 15 characters on average, and each character takes 2 Bytes of storage. According to this formulation, we would require the following:

$2 \text{ billion} \times 15^2 = 60\text{GB}$ to store all the queries made in a day.

Storage required per year:

$60\text{GB/day} \times 365 = 21.9\text{TB/year}$

[Storage requirements for the typeahead suggestion system](./storage.jpg)

Bandwidth estimation

3.5 billion queries will reach our system every day. Assume that each query a user types is 15 characters long on average.

Keeping this in mind, the total number of reading requests of characters per day would be as follows:

$15 \times 3.5 \text{ billion} = 52.5 \text{ billion characters per day.}$

Total read requests per second: $52.5 \text{B} / 86400 = 0.607 \text{M characters/sec.}$ 86,400 is the number of seconds per day.

Since each character takes 2 Bytes, the bandwidth our system would need is as follows:

$0.607 \text{M} \times 2 = 1.214 \text{Mb/sec}$

1.214Mb/sec is the incoming bandwidth requirement for queries that have a maximum length of 15 characters. Our system would suggest the top ten queries that are roughly of the same length as the query length after each character a user types. Therefore, the outgoing bandwidth requirement would become the following: $15 \times 10 \times 1.214 \text{Mb/sec} = 18.21 \text{Gb/sec.}$

[The total bandwidth required by the typeahead suggestion system](./bandwidth.jpg)

Number of servers estimation

Our system will receive 607,000 requests per second concurrently. Therefore, we need to have many servers installed to avoid burdening a single server. Let's assume that a single server can handle 8,000 queries per second. So, we require around 76 servers to handle 607,000 queries.

'''

The system will suggest queries after each character a user types. Therefore, against 0.607 million characters per second, our system receives 0.607 million or 607,000 requests per second.

'''

$607 \text{K} / 8000 = 76 \text{ servers}$

Here, we only refer to the number of application servers. For simplicity, we ignored the number of cache and database servers. The actual number is higher than 76 because we would also need redundant servers to achieve availability.

[The number of servers required for the typeahead suggestion system](./servers.jpg)

In the table below, adjust the values to see how the resource estimations change.

'''

Resource Estimation for the Typeahead Suggestion System

Total Queries per Day	3.5	billion
Unique Queries per Day	2	billion
Minimum Characters in a Query	15	Characters
Server's QPS	8000	Queries per second
Storage	60	GB/day
Incoming Bandwidth	9.7	Mb/sec
Outgoing Bandwidth	1.455	Gb/sec
Number of Servers	76	Severs

'''

Building blocks we will use

The design of the typeahead suggestion system consists of the following building blocks that have been discussed in the initial chapters of the course:

[Building blocks required in the design of the typeahead suggestion system](./bb.jpg)

- Databases are required to keep the data related to the queriesâ prefixes.
- Load balancers are required to disseminate incoming queries among a number of active servers.
- Caches are used to keep the top N suggestions for fast retrieval.

In the next lesson, weâll focus on the high-level design and APIs of the typeahead suggestion system.

Design Typeahead Suggestion/System Design The Typeahead Suggestion System/README.md

System Design: The Typeahead Suggestion System

Introduction

Typeahead suggestion, also referred to as the autocomplete feature, enables users to search for a known and frequently searched query. This feature comes into play when a user types a query in the search box. The typeahead system provides a list of suggestions to complete a query based on the userâs search history, the current context of the search, and trending content across different users and regions. Frequently searched queries always appear at the top of the suggestion list. The typeahead system doesnât make the search faster. However, it helps the user form a sentence more quickly. Itâs an essential part of all search engines that enhance the user experience.

[The typeahead suggestion system in action]

How will we design a typeahead suggestion system?

Weâve divided the chapter on the design of the typeahead suggestion system into six lessons:

1. Requirements: In this lesson, we focus on the functional and non-functional requirements for designing a typeahead suggestion system. We also discuss resource estimations for the smooth operation of the system.
 2. High-level design: In this lesson, we discuss the high-level design of our version of the typeahead suggestion system. We also discuss some essential APIs used in the design.
 3. Data Structure for Storing Prefixes: In this lesson, we cover an efficient tree data structure called trie thatâs used to store search prefixes. We also discuss how it can be further optimized to reduce the tree traversal time.
 4. Detailed design: In this lesson, we explain the two main components, the suggestions service and the assembler, which make up the detailed design of the typeahead suggestion system.
 5. Evaluation: This lesson evaluates the proposed design of the typeahead suggestion system based on the non-functional requirements of the system. It also presents some client-side optimization and personalization that could significantly affect the systemâs design.
 6. Quiz: In this lesson, we assess our understanding of the design via a quiz.
- Letâs start by identifying the requirements for designing a typeahead suggestion system.

Design Uber/Detailed Design of Uber/README.md

Detailed Design of Uber

Letâs look at the detailed design of our Uber system and learn how the various components work together to offer a functioning service:

[The detailed design of Uber](./dd.jpg)

Components

Letâs discuss the components of our Uber system design in detail.

Location manager

The riders and drivers are connected to the location manager service. This service shows the nearby drivers to the riders when they open the application. This service also receives location updates from the drivers every four seconds. The location of drivers is then communicated to the quadtree map service to determine which segment the driver belongs to on a map. The location manager saves the last location of all drivers in a database and saves the route followed by the drivers on a trip.

Quadtree map service

The quadtree map service updates the location of the drivers. The main problem is how we deal with finding nearby drivers efficiently.

We'll modify the solution discussed in the Yelp chapter according to our requirements. We use quadrees on Yelp to find the location. Quadrees help to divide the map into segments. If the number of drivers exceeds a certain limit, for example, 500, then we split that segment into four more child nodes and divide the drivers into them.

Each leaf node in quadrees contains segments that can't be divided further. We can use the same quadrees for finding the drivers. The most significant difference we have now is that our quadtree wasn't designed with regular upgrades in consideration. So, we have the following issues with our dynamic segment solution.

We must update our data structures to point out that all active drivers update their location every four seconds. It takes a longer amount of time to modify the quadtree whenever a driver's position changes. To identify the driver's new location, we must first find a proper grid depending on the driver's previous position. If the new location doesn't match the current grid, we should remove the driver from the current grid and shift it to the correct grid. We have to repartition the new grid if it exceeds the driver limit, which is the number of drivers for each region that we set initially. Furthermore, our platform must tell both the driver and the rider, of the car's current location while the ride is in progress.

To overcome the above problem, we can use a hash table to store the latest position of the drivers and update our quadtree occasionally, say after 10-15 seconds. We can update the driver's location in the quadtree around every 15 seconds instead of four seconds, and we use a hash table that updates every four seconds and reflects the driver's latest location. By doing this, we use fewer resources and time.

'''

A quadtree is a tree data structure in which each internal node has exactly four children. Quadrees are the two-dimensional analog of octrees and are most often used to partition a two-dimensional space by recursively subdividing it into four quadrants or regions. The data associated with a leaf cell varies by application, but the leaf cell represents a unit of interesting spatial information. Source: Wikipedia

'''

[Updating driver's location](./driver's_location.jpg)

Request vehicle

The rider contacts the request vehicle service to request a ride. The rider adds the drop-off location here. The request vehicle service then communicates with the find driver service to book a vehicle and get the details of the vehicle using the location manager service.

Find driver

The find driver service finds the driver who can complete the trip. It sends the information of the selected driver and the trip information back to the request vehicle service to communicate the details to the rider. The find driver service also contacts the trip manager to manage the trip information.

Trip manager

The trip manager service manages all the trip-related tasks. It creates a trip in the database and stores all the information of the trip in the database.

ETA service

The ETA service deals with the estimated time of arrival. It shows riders the pickup ETA when their trip is scheduled. This service considers factors such as route and traffic. The two basic components of predicting an ETA given an origin and destination on a road network are the following:

- Calculate the shortest route from origin to destination.
- Compute the time required to travel the route.

The whole road network is represented as a graph. Intersections are represented by nodes, while edges represent road segments. The graph also depicts one-way streets, turn limitations, and speed limits.

[Graph representation](./presentation.jpg)

To identify the shortest path between source and destination, we can utilize routing algorithms such as Dijkstra's algorithm. However, Dijkstra, or any other algorithm that operates on top of an unprocessed graph, is quite slow for such a system. Therefore, this method is impractical at the scale at which these ride-hailing platforms operate.

To resolve these issues, we can split the whole graph into partitions. We preprocess the optimum path inside partitions using contraction hierarchies and deal with just the partition boundaries. This strategy can considerably reduce the time complexity since it partitions the graph into layers of tiny cells that are largely independent of one another. The preprocessing stage is executed in parallel in the partitions when necessary to increase speed. In the illustration below, all the partitions process the best route in parallel. For example, if each partition takes one second to find the path, we can have the complete path in one second since all partitions work in parallel.

'''

Contraction hierarchies is a speed-up method optimized to exploit the properties of graphs representing road networks. The speed-up is achieved by creating shortcuts in a preprocessing phase, which are then used during a shortest-path query to skip over unimportant vertices.

Source: Wikipedia

'''

[Partitioning the graph](./partitions.jpg)

Once we determine the best route, we calculate the expected time to travel the road segment while accounting for traffic. The traffic data will be the edge weights between the nodes.

DeepETA

We use a machine learning component named DeepETA to deliver an immediate improvement to metrics in production. It establishes a model foundation that can be reused for multiple consumer use cases.

We also use a routing engine that uses real-time traffic information and map data to predict a ETA to traverse the best path between the source and the destination. We use a post-processing ML model that considers spatial and temporal parameters, such as the source, destination, time of the request, and knowledge about real-time traffic, to forecast the ETA residual.

[DeepETA](./deepeta.jpg)

Database

Let's select the database according to our requirements:

The database must be able to scale horizontally. There are new customers and drivers on a regular basis, so we should be able to add more storage without any problems.

The database should handle a large number of reads and writes because the location of drivers is updated every four seconds.

Our system should never be down.

Weâve already discussed the various database types and their specifications in the Database chapter. So, according to our understanding and requirements (high availability, high scalability, and fault tolerance), we can use Cassandra to store the driverâs last known location and the trip information after the trip has been completed, and there will be no updates to it.

We can use a MySQL database to store trip information while itâs in progress. We use MySQL for in-progress trips for frequent updates since the trip information is relational, and it needs to be consistent across tables. We use Cassandra because the data we store is enormous and it increases continuously.

'''

Cassandra is a columnar database, which is a type of NoSQL database designed to handle large amounts of data across many commodity servers, providing high availability with no single point of failure. Source: Wikipedia

'''

'''

Note: Recently, Uber migrated their data storage to Google Cloud Spanner. It provides global transactions, strongly consistent reads, and automatic multisite replication and failover features.

'''

'''

Question 1

Why havenât we used only the MySQL database?

Answer

As the data becomes massive with the passage of time, it reduces our systemâs performance. Also, it wonât be easy to scale the MySQL database.

'''

'''

Question 2

Why havenât we used only the Cassandra database?

Answer

The data we store for each trip is highly relational, itâs spread over multiple tables, and must be identical among all tables. MySQL helps to store this relational data and keep it consistent across tables.

'''

Storage schema

On a basic level in the Uber application, we need the following tables:

- Riders: We store the riderâs related information, such as ID, name, email, photo, phone number, and so on.
- Drivers: We store the driverâs related information, such as ID, name, email, photo, phone number, vehicle name, vehicle type, and so on.
- Driver_location: We store the driverâs last known location.
- Trips: We store the tripâs related information, such as trip ID, rider ID, driver ID, status, ETA, location of the vehicle, and so on.

The following illustration visualizes the data model:

[The storage schema](./schema.jpg)

Fault tolerance

For availability, we need to have replicas of our database. We use the primary-secondary replication model. We have one primary database and a few secondary databases. We synchronously replicate data from primary to secondary databases. Whenever our primary database is down, we can use a secondary database as a primary one.

'''

Question

How will we handle a driver's slow and disconnecting network?

Answer

We can use the driver's phone as local storage and save the state of a trip every few seconds there. All requests and the last recorded state are stored on a local disk, which ensures that they're preserved even if the application is restarted. Suppose the driver exits the application while there are a few requests waiting to be synchronized with the server. The requests and last known state are loaded from the local disk upon relaunch. When the driver relaunches the application, they remain in the same situation as before. The requests are queued in order to keep the server up to date.

'''

Load balancers

We use load balancers between clients (drivers and riders) and the application servers to uniformly distribute the load among the servers. The requests are routed to the specified server that provides the requested service.

Cache

A million drivers need to send the updated location every four seconds. A million requests to the quadtree service affects how well it works. For this, we first store the updated location in the hash table stored in Redis. Eventually, these values are copied into the persistent storage every 10-15 seconds.

We've discussed the detailed design of Uber and how different components work together to fulfill the requirements. In the next lesson, we'll learn how the payment service works to move the money from riders to drivers and detect frauds.

Design Uber/Evaluation of Uber's Design/README.md

Evaluation of Uber's Design

Fulfill non-functional requirements

Let's evaluate how our system fulfills the non-functional requirements.

Availability

Our system is highly available. We used WebSocket servers. If a user gets disconnected, the session is recreated via a load balancer with a different server. We've used multiple replicas of our databases with a primary-secondary replication model. We have the Cassandra database, which provides highly available services and no single point of failure. We used a CDN, cache, and load balancers, which increase the availability of our system.

Scalability

Our system is highly scalable. We used many independent services so that we can scale these services horizontally, independent of each other as per our needs. We used quadrees for searching by dividing the map into smaller segments, which shortens our search space. We used CDN, which increases the capacity to handle more users. We also used a NoSQL database, Cassandra, which is horizontally scalable. Additionally, we used load balancers, which improve speed by distributing read workload among different servers.

Reliability

Our system is highly reliable. The trip can continue even if the rider's or driver's connection is broken. This is achieved by using their phones as local storage. The use of multiple WebSocket servers ensures smooth, nearly real-time operations. If any of the servers fail, the

user is able to reconnect with another server. We also used redundant copies of the servers and databases to ensure that thereâs no single point of failure. Our services are decoupled and isolated, which eventually increases the reliability. Load balancers help move the requests away from any failed servers to healthy ones.

Consistency

We used storage like MySQL to keep our data consistent globally. Moreover, our system does synchronous replication to achieve strong consistency. Because of a limited number of data writers and viewers for a trip (rider, driver, some internal services), the usage of traditional databases doesnât become a bottleneck. Also, data sharding is easier in this scenario.

Fraud detection

Our system is able to detect any fraudulent activity related to payment. We used the RADAR system to detect any suspicious activity. RADAR recognizes the beginning of a fraud attempt and creates a rule to prevent it.

'''

Meeting Non-functional Requirements	Techniques
-------------------------------------	------------

Availability	- Using server replicas - Using database replicas with Cassandra database - Load balancers hide server failures from end users
Scalability	- Horizontal sharding of the database - The Cassandra NoSQL database
Reliability	- No single point of failure - Redundant components
Consistency	- Strong consistency using synchronous replications
Fraud detection payments	- Using RADAR to recognize and prevent any fraud related to

'''

Conclusion

This chapter taught us how to design a ride-hailing service like Uber. We discussed its functional and non-functional requirements. We learned how to efficiently locate drivers on the map using quadrees. We also discussed how to efficiently calculate the estimated time of arrival using routing algorithms and machine learning. Additionally, we learned that guarding our service against fraudulent activities is important for the businessâs success.

Design Uber/High-level Design of Uber/README.md

High-level Design of Uber

Workflow of our application

Before diving deep into the design, letâs understand how our application works. The following steps show the workflow of our application:

1. All the nearby drivers except those already serving rides can be seen when the rider starts our application.
2. The rider enters the drop-off location and requests a ride.
3. The application receives the request and finds a suitable driver.
4. Until a matching driver is found, the status will be âWaiting for the driver to respondâ.

5. The drivers report their location every four seconds. The application finds the trip information and returns it to the driver.

6. The driver accepts or rejects the request:

- The driver accepts the request, and status information is modified on both the rider's and the driver's applications. The rider finds that they have successfully matched and obtains the driver's information.

- The driver refuses the ride request. The rider restarts from step 2 and rematches to another driver.

[Rides](./uber)

High-level design of Uber

At a high level, our system should be able to take requests for a ride from the rider and return the matched driver information and trip information to the rider. It also regularly takes the driver's location. Additionally, it returns the trip and rider information to the driver when the driver is matched to a rider.

[High-level design](./hld.jpg)

API design

Let's discuss the design of APIs according to the functionalities we provide. We'll design to translate our feature set into technical specifications.

We won't repeat the description of repeating parameters in the following APIs.

Update driver location

'''

updateDriverLocation(driverID, oldlat, oldlong, newlat, newlong)

'''

'''

Parameter	Description
-----------	-------------

driverID	The ID of the driver
----------	----------------------

oldlat	The previous latitude of the driver
--------	-------------------------------------

oldlong	The previous longitude of the driver
---------	--------------------------------------

newlat	The new latitude of the driver
--------	--------------------------------

newlong	The new longitude of the driver
---------	---------------------------------

'''

The updateDriverLocation API is used to send the driver's coordinates to the driver location servers. This is where the location of the driver is updated and communicated to the riders.

Find nearby drivers

'''

findNearbyDrivers(riderID, lat, long)

'''

'''

Parameter	Description
-----------	-------------

riderID	The ID of the rider
---------	---------------------

lat The latitude of the rider

long The longitude of the rider
'''

The findNearbyDrivers API is used to send the location of the rider for whom we want to find the nearby drivers.

Request a ride
'''

requestRide(riderID, lat, long, dropOfflat, dropOfflong, typeOfVehicle)
'''

'''

Parameter	Description
-----------	-------------

lat	The current latitude of the rider
-----	-----------------------------------

long	The current longitude of the rider
------	------------------------------------

dropOfflat	The latitude of the rider's drop-off location
------------	---

dropOfflong	The longitude of the rider's drop-off location
-------------	--

typeOfVehicle	The type of vehicle required by the rider for example, business, economy, so on.
---------------	--

'''

The requestRide API is used to send the location of the rider and the type of vehicle the rider needs.

Show driver ETA
'''

showETA(driverID, eta)
'''

'''

Parameter	Description
-----------	-------------

eta	The estimated time of arrival of the driver
-----	---

'''

The showEta API is used to show the estimated time of arrival to the rider.

Confirm pickup
'''

confirmPickup(driverID, riderID, timestamp)
'''

'''

Parameter	Description
-----------	-------------

timestamp	The time at which the driver picked up the rider
-----------	--

'''

The confirmPickup API is used to determine when the driver has picked up the rider.

Show trip updates
'''

showTripUpdates(tripID, riderID, driverID, driverlat, driverlong, time_elapsed, time_remaining)
'''

'''

Parameter	Description
tripID	The ID of the trip
driverlat	The latitude of the driver
driverlong	The longitude of the driver
time_elapsed	The total time of the trip
time_remaining	The time remaining (extract the current time from the ETA) to reach the destination

The showTripUpdates API is used to show the updates of the trip, including the position of the driver and the time remaining to reach the destination.

End the trip

endTrip(tripID, riderID, driverID ,time_elapsed, lat, long)

The endTrip API is used to end the trip.

Design Uber/Payment Service and Fraud Detection in Uber Design/README.md

Payment Service and Fraud Detection in Uber Design

Our goal in this lesson is to highlight the need to secure the system against malicious activities. Weâ ll discuss fraudulent activities in the context of payment. Weâ ll first bri go through how the payment system works, and then weâ ll cover how to guard our system against malicious activity.

The payment service deals with the payments side of the Uber application. It includes collecting payment from the rider to give it to the driver. As the trip completes, it collects the trip information, calculates the cost of the journey, and initiates the payment process.

Major functionality

The Uber payments platform includes the following functionality:

- New payment options: It adds new payment options for users.
- Authorization: It authorizes the payments for transactions.
- Refund: It refunds a payment that was authorized before.
- Charging: It moves money from a user account to Uber.

'''

In a payment service, authorization is a process to verify the payment methodâ for example, whether a certain credit card is valid or not.

'''

What to prevent

We need to prevent the following scenarios for the success of the payment system:

- Lack of payment
- Duplicate payments
- Incorrect payment
- Incorrect currency conversion
- Dangling authorization

The Uber payment platform is based on the double-entry bookkeeping method.

'''

Double-entry bookkeeping, also known as, double-entry accounting, is a method of bookkeeping that relies on a two-sided accounting entry to maintain financial information. Every entry to an account requires a corresponding and opposite entry to a different account. Source: Wikipedia

'''

Design

Let's look at the design of the payment platform in Uber and learn how different components work together to complete the transactions:

[The payment platform in Uber](./payment_platform.jpg)

The key components used in the payment service are as follows:

- API: The Uber API is used to access the payment service.
- Order store: This stores all the orders. Orders collect payments and contain information regarding money flow between different riders and drivers.
- Account store: This stores all the accounts of riders and drivers.
- Risk engine: The risk engine analyzes different risks involved in collecting payment from a particular rider. It checks the history of the rider—for example, the rating of the rider, if the rider has sufficient funds in their rider account, any pending dues, if the rider cancels the rides frequently, and so on.
- Payment profile service: This provides information on the payment mechanisms, such as credit and debit cards.
- User profile service: This provides information regarding users' payments.
- Payment authorization service: This offers payment authentication services.
- PSP gateways: This connects with payment service providers

Workflow

The following steps show the workflow of the payment service:

1. When a rider requests a ride, the Uber application uses the Uber API to request the risk engine to check the risks involved.
 3. The risk engine obtains user information from the user profile service and evaluates the risk involved.
 3. If the risks are high, the rider's request isn't entertained.
 4. If the risks are low, the risk engine creates an authorization token and sends it to the payment profile service (for record-keeping), which fetches that token and sends it to the payment authorization service.
 5. The payment authorization service sends that request to the PSP gateway, which contacts the service provider for authorization.
 6. The PSP gateway sends the authorization token back to the payment authorization service, which sends the token back to the Uber application that the trip request is approved.
 7. After the trip is completed, the Uber application uses the API to send the payment request to the PSP gateway with authorization data.
 8. The PSP gateway contacts the service provider and sends the status back to the Uber application through the API.
- The Uber payment service is constructed with a set of microservices grouped together as a stream-processing architecture.

Apache Kafka

Kafka is an open-source stream-processing software platform. It's the primary technology used in payment services. Let's see how Kafka helps to process an order:

'''

Stream processing is a computer programming paradigm that simplifies parallel software and hardware by restricting the parallel computation that can be performed. Given a sequence of data (a stream), a series of operations (kernel functions) is applied to each element in the stream. Source: Wikipedia

'''

[Payment service with Kafka](./payment_service.jpg)

The order creator gets a business event—for example, the trip is finished. The order creator creates the money movement information and the metadata. This order creator publishes that information to Kafka. Kafka processes that order and sends it to the order processor. The order

processor then takes that information from Kafka, processes it, and sends it as intent to Kafka. From that, the order processor again processes and contacts the PSP. The order processor then takes the answer from PSP and transmits it to Kafka as a result. The result is then saved by the order writer.

The key capabilities of Kafka that the payment service uses are the following:

- It works like message queues to publish and subscribe to message queues.
- It stores the records in a way that is fault tolerant.
- It processes the payment records asynchronously.

Fraud detection

Fraud detection is a critical operational component of our design. Payment fraud losses are calculated as a percentage of gross amounts. Despite fraud activities accounting for a tiny portion of gross bookings, these losses considerably influence earnings. Moreover, if malicious behavior isn't promptly detected and handled, it may be further leveraged, which results in significant losses for the business. Earlier, we discussed the risk engine that detects fraud before the trip starts. Here, we'll focus on the fraud detection that happens during trips and at the end of the trips.

Fraud detection is challenging because many instances of fraud are like detecting zero-day security bugs. Therefore, our system needs intelligence to detect anomalies, and it also needs accountability for automated decisions in the form of human-led audits.

The activities that are considered fraudulent by Uber are as follows:

'''

The following list of eight fraudulent activities is taken from:

<https://www.uber.com/gb/en/drive/driver-app/fraud-activities/>

'''

- A driver deliberately increases the time or distance of a trip in a dishonest manner for example, if they take a much longer route than necessary to the rider's destination.
- A driver manipulates the GPS data or uses fake location GPS apps.
- A driver confirms trips with no intention of completing them, forcing riders to cancel.
- A driver creates false driver or rider accounts for fraudulent purposes.
- A driver provides incorrect or inaccurate information about themselves when opening their account.
- A driver drives a vehicle that hasn't been approved by Uber.
- A driver claims fraudulent fees or charges, such as unwarranted cleaning fees.
- A driver willfully confirms or completes fraudulent trips.

RADAR

Uber introduced RADAR, a human-assisted AI fraud detection and mitigation solution to handle the discussed scenarios of fraud. The RADAR system detects fraud by analyzing the activity time series of the payment system. It then generates a rule for it and stops it from further processing. This proactive approach can help to detect unseen fraud in real time. This detection model also uses human knowledge for continuous improvements. Here, we'll briefly discuss how RADAR works:

[The RADAR fraud detection solution](./radar.jpg)

RADAR recognizes the beginning of a fraud attempt and creates a rule to prevent it. The fraud analyst is involved in the next step. They review the rule and approve or reject it if required. They then send feedback (approved or not approved) to the protection system. The feedback is also sent to the fraud detection system by the fraud analysts to improve detection.

Furthermore, the system examines the data in the following time dimensions:

- It examines the trip time when a trip has been completed. Multiple factors are involved in real-time trip monitoring. For instance, the system can check whether or not the driver and rider locations are the same. The system can also monitor the speed and analyze the real traffic to check if the driver is intentionally driving slow or if there's traffic congestion.
- Payment settlement time refers to obtaining payment processing data. It might take days or weeks to settle.

We've just discussed the basic details of how guarding against malicious activity at scale is necessary for the success of a business. In the proposed model, the human experts are involved, potentially decreasing the scalability of the system. Research is being carried out on how to ensure that the decisions made by the AI system are fair and ethical.

```
## Design Uber/Quiz on Uber's Design/README.md
```

```
# Quiz on Uber's Design
'''
```

```
1 (Select all that apply.) Which factors can help optimize the ETA at the start of the trip to
lower the cost and duration of the trip?
```

- A)
Real-time traffic detail
- B)
Distance of a driver before the trip starts
- C)
Professional driver
- D)
Speed of vehicle

```
'''
```

```
2 (Select all that apply.) Why is it essential to store the information of routes in a ride-
hailing service design?
```

- A)
It helps to optimize the ETA.
- B)
It minimizes the ride cost fluctuation.
- C)
It helps to determine the bonus for the driver.
- D)
It helps to engage more passengers.

```
'''
```

```
'''
```

```
3 (Select all that apply.) What information can we store in the CDN?
```

- A)
Driversâ current locations
- B)
Driver and passenger profile details
- C)
Last trips of the passengers
- D)
The traffic details
'''

Answers

1. (A) (D)

'''

2

(Select all that apply.) Why is it essential to store the information of routes in a ride-hailing service design?

Selected

A)
It helps to optimize the ETA.

Explanation

We have actual values with each route stored in the database. Next time someone chooses this route, we could give a more accurate ETA.

Selected

B)
It minimizes the ride cost fluctuation.

Explanation

We know the route detail by the ride details. So, the ETA estimation will be close to the actual time taken. Thatâ s why the initial cost estimate (decided earlier the ride) fluctuates less.

C)
It helps to determine the bonus for the driver.

Explanation

Saving the route detail does not affect the bonus. In contrast, the distance covered by the driver matters in bonus determination.

D)
It helps to engage more passengers.
'''

'''

3

(Select all that apply.) What information can we store in the CDN?

A)
Driversâ current locations

Explanation

It will frequently change, so it is not suitable.

Selected

B)

Driver and passenger profile details

Explanation

Names, driver photos, vehicle detail, etc., can store in the CDN.

Selected

C)

Last trips of the passengers

Explanation

The trips detail of the passengers can be stored in the CDN because once a trip is over, these details don't change.

D)

The traffic details

Explanation

Traffic is not consistent. It changes every second. CDN usually stores the static data that is not changed frequently.

'''

Design Uber/Requirements of Uber's Design/README.md

Requirements of Uber's Design

Requirements

Let's start with the requirements for designing a system like Uber.

Functional requirements

The functional requirements of our system are as follows:

- Update driver location: The driver is a moving entity, so the driver's location should be automatically updated at regular intervals.

- Find nearby drivers: The system should find and show the nearby available drivers to the rider.

- Request a ride: A rider should be able to request a ride, after which the nearest driver should be notified about the rider's requests.

- Manage payments: At the start of the trip, the system must initiate the payment process and manage the payments.

- Show driver estimated time of arrival (ETA): The rider should be able to see the estimated time of arrival of the driver.

- Confirm pickup: Drivers should be able to confirm that they have picked up the rider.

- Show trip updates: Once a driver and a rider accept a ride, they should be able to constantly see trip updates like ETA and current location until the trip finishes.

- End the trip: The driver marks the journey complete upon reaching the destination, and they then become available for the next ride.

'''

Question

What if two drivers are at the same distance from the rider? How will we select the driver to whom we'll send the request?

Answer

This decision will be made on multiple factors, such as the distance, type of vehicle, rank of the driver, and so on. Still, if two drivers are identical, we can randomly select one and send a request to that driver. If one driver doesn't accept the ride within a few seconds, we retract the ride offer from this driver and present it to a new one.

'''

Non-functional requirements

The non-functional requirements of our system are as follows:

- Availability: The system should be highly available. The downtime of even a fraction of a second can result in a trip failure, in the driver being unable to locate the rider, or in the rider being unable to contact the driver.
- Scalability: The system should be scalable to handle an ever-increasing number of drivers and riders with time.
- Reliability: The system should provide fast and error-free services. Ride requests and location updates should happen smoothly.
- Consistency: The system must be strongly consistent. The drivers and riders in an area should have a consistent view of the system.
- Fraud detection: The system should have the ability to detect any fraudulent activity related to payment.

Resource estimation

Now, let's estimate the resources for our design. Let's assume it has around 500 million riders and about five million drivers. We'll assume the following numbers for our estimates:

- We have 20 million daily active riders and three million daily active drivers.
- We have 20 million daily trips.
- All active drivers send a notification of their current location every four seconds.

Storage estimation

Let's estimate the storage required for our system:

Rider's metadata

Let's assume we need around 1,000 Bytes to store each rider's information, including ID, name, email, and so on, when the rider registers in our application. To store the 500 million riders, we require 500 GB of storage:

$$500 \times 10^6 \times 1000 = 500 \text{ GB}$$

Additionally, if we have around 500,000 new riders registered daily, we'll need a further 500 MB to store them.

Driver's metadata

Let's assume we need around 1,000 Bytes to store each driver's information, including ID, name, email, vehicle type, and so on, when the driver registers in our application. To store the five million drivers, we require 5 GB of storage:

$$5 \times 10^6 \times 1000 = 5 \text{ GB}$$

Additionally, if we have around 100,000 new drivers registered daily, we'll need around 100 MB to store them.

Driver location metadata

Let's assume we need around 36 Bytes to store the driver's location updates. If we have five million drivers, we need around 180 MB of storage just for the drivers' locations.

Trip metadata

Let's assume we need around 100 Bytes to store single trip information, including trip ID, rider ID, driver ID, and so on. If we have 20 million daily rides, we need around 2 GB of storage for the trip data.

Let's calculate the total storage required for Uber in a single day:

'''

Storage Capacity Estimation

Number of drivers (millions)	5
Storage required to store a driver's location (Bytes)	36
Total storage required to store drivers' locations (MB per day)	180
Number of trips (millions)	20
Storage required to store a trip (Bytes)	100
Total storage required to store trips (GB per day)	2
Storage required for new riders daily (MB per day)	500
Storage required for new drivers daily (MB per day)	100
Total storage (GB per day)	2.78

'''

[Total storage required by Uber in an year](./storage.jpg)

'''

Note: We can adjust the values in the table to see how the estimations change.

'''

Bandwidth estimation

We'll only consider drivers' location updates and trip data for bandwidth calculation since other factors don't require significant bandwidth. We have 20 million daily rides, which means we have approximately 232 trips per second.

$20000000 / 86400 \approx 232$ trips per second

Now, each trip takes around 100 Bytes. So, it takes around 23 KB per second of bandwidth.

$232 \times 100 = 23 \text{ KB} \times 8 = 185 \text{ kbps}$

As already stated, the driver's location is updated every four seconds. If we receive the driver's ID (3 Bytes) and location (16 Bytes), our application will take the following bandwidth:

$3M \text{ active drivers} \times (3+16)B = 57MB \times 8 = 456/4 = 114 \text{ Mbps}$

These bandwidth requirements are modest because we didn't include bandwidth needs for maps and other components that are present in the real Uber service.

'''

Bandwidth Requirements

Trips per second	232
Bandwidth for each trip (Bytes)	100
Total bandwidth for trips (kilo bits per second)	185.6
Active drivers (millions)	3
Bandwidth for each driver (Bytes)	19
Bandwidth for drivers (Megabits per second)	114
Total bandwidth (Megabits per second)	114.19

'''

[Total bandwidth required by Uber](./bandwidth.jpg)

'''

Note: We can adjust the values in the table to see how the requirements change.

'''

Weâ ve ignored the bandwidth from the Uber service to users because it was very small. More bandwidth will be required for sending map data from the Uber service to users, which weâ ve also discussed in the Google Maps chapter.

Number of servers estimation

We need to handle concurrent requests coming from 20 million daily active users. Weâ ll use the following formula to estimate a pragmatic number of servers. We established this formula in the Back-of-the-envelope Calculations chapter:

Number of daily active users/RPS of a server= $20 \times 10^6 / 8000 = 2500$

[Number of servers required for the Uber service](./servers.jpg)

'''

Estimating the Number of Servers

Daily active users (millions)	20
RPS of a server	8000
Number of servers required	2500

'''

'''

Note: We can adjust the values in the table to see how the estimations change.

'''

Building blocks we will use

The design of Uber utilizes the following building blocks:

[Building blocks in high-level design of Uber](./bb.jpg)

- Databases store the metadata of riders, drivers, and trips.
- A cache stores the most requested data for quick responses.
- CDNs are used to effectively deliver content to end users, reducing delay and the burden on end-servers.
- A load balancer distributes the read/write requests among the respective services.
- Ridersâ and driversâ devices should have sufficient bandwidth and GPS equipment for smooth navigation with maps.

'''

Note: The information provided in this chapter is inspired by the engineering blog of Uber.

'''

Design Uber/System Design Uber/README.md

System Design: Uber

What is Uber?

Uber is an application that provides ride-hailing services to its users. Anyone who needs a ride can register and book a vehicle to travel from source to destination. Anyone who has a vehicle can register as a driver and take riders to their destination. Drivers and riders can communicate through the Uber app on their smartphones.

[Uber](./uber)

The illustration below shows the number of active users of Uber from the start of 2017 to 2020 (source: Statista):

[Monthly number of Uber's active users worldwide from 2017 to 2020(by quarter)](stat.jpg)

How will we design Uber?

There are many unanswered questions regarding Uber. How does it work? How do drivers connect with riders? These are only two of many. This chapter will design a system like Uber and find the answer to such questions.

We've divided the design of Uber into six sections:

1. Requirements: This lesson will describe the functional and non-functional requirements of a system like Uber. We'll also estimate the requirements of multiple aspects of Uber, such as storage, bandwidth, and the computation resources.
 2. High-level Design: We'll discuss the high-level design of Uber in this lesson. In addition, we'll also briefly explain the API design of the Uber service.
 3. Detailed Design: We'll explore the detailed design of Uber in this lesson. Moreover, we will also discuss the working of different components used in designing Uber.
 4. Payment Service and Fraud Detection: We'll learn how the payment system works in Uber design. Moreover, we'll also discuss how we can catch different frauds related to payments in Uber-like systems.
 5. Evaluation: This lesson will explain how Uber can fulfill all the non-functional requirements through the proposed design.
 6. Quiz: We'll reinforce major concepts of Uber design via a quiz.
- Let's go over the requirements for designing a system like Uber in the next lesson.

Design WhatsApp/Detailed Design of WhatsApp/README.md

Detailed Design of WhatsApp

Detailed design

The high-level design discussed in the previous lesson doesn't answer the following questions:

- How is a communication channel created between clients and servers?
- How can the high-level design be scaled to support billions of users?
- How is the user's data stored?
- How is the receiver identified to whom the message is delivered?

To answer all these questions, let's dive deep into the high-level design and explore each component in detail. Let's start with how users make connections with the chat servers.

Connection with a WebSocket server

In WhatsApp, each active device is connected with a WebSocket server via WebSocket protocol. A WebSocket server keeps the connection open with all the active (online) users. Since one server isn't enough to handle billions of devices, there should be enough servers to handle billions of users. The responsibility of each of these servers is to provide a port to every online user. The mapping between servers, ports, and users is stored in the WebSocket manager that resides on top of a cluster of the data store. In this case, that's Redis.

[Two users are connected via WebSocket handlers]

'''

Question

Why is WebSocket preferred over HTTP(S) protocol for client-server communication?

Answer

HTTP(S) doesn't keep the connection open for the servers to send frequent data to a client. With HTTP(S) protocol, a client constantly requests updates from the server, commonly called polling, which is resource intensive and causes latency. WebSocket maintains a persistent connection between the client and a server. This protocol transfers data to the client

immediately whenever it becomes available. It provides a bidirectional connection used as a common solution to send asynchronous updates from a server to a client.

Send or receive messages

The WebSocket manager is responsible for maintaining a mapping between an active user and a port assigned to the user. Whenever a user is connected to another WebSocket server, this information will be updated in the data store.

A WebSocket server also communicates with another service called message service. Message service is a repository of messages on top of the Mnesia database cluster. The message service exposes APIs to receive messages by various filters, such as user ID, message ID, and so on.

Mnesia database cluster

As mentioned earlier, WhatsApp stores messages for a short period. Therefore, we need a database capable of storing ever-increasing data and should support frequent delete operations. For this purpose, WhatsApp uses Erlang-based distributed DBMS called Mnesia. This DBMS provides the following benefits.

- Real-time key-value lookup
- High fault tolerance
- Dynamic reconfiguration
- Complex objects

[WebSocket communicates with message service on top of Mnesia database cluster]

Now, let's assume that user A wants to send a message to user B. As shown in the above figure, both users are connected to different WebSocket servers. The system performs the following steps to send messages from user A to user B:

1. User A communicates with the corresponding WebSocket server to which it is connected.
2. The WebSocket server associated with user A identifies the WebSocket to which user B is connected via the WebSocket manager. If user B is online, the WebSocket manager responds back to user A's WebSocket server that user B is connected with its WebSocket server.
3. Simultaneously, the WebSocket server sends the message to the message service and is stored in the Mnesia database where it gets processed in the first-in-first-out order. As soon as these messages are delivered to the receiver, they are deleted from the database.
4. Now, user A's WebSocket server has the information that user B is connected with its own WebSocket server. The communication between user A and user B gets started via their WebSocket servers.
5. If user B is offline, messages are kept in the Mnesia database. Whenever they become online, all the messages intended for user B are delivered via push notification. Otherwise, these messages are deleted permanently after 30 days.

Both users (sender and receiver) communicate with the WebSocket manager to find each other's WebSocket server. Consider a case where there can be a continuous conversation between both users. This way, many calls are made to the WebSocket manager. To minimize the latency and reduce the number of these calls to the WebSocket manager, each WebSocket server caches the following information:

- If both users are connected to the same server, the call to the WebSocket manager is avoided.
- It caches information of recent conversations about which user is connected to which

WebSocket server.

'''

Question

The data in the cache will become outdated if a user gets disconnected and connects with another server. Keeping this in mind, how long should a WebSocket server cache information?

Answer

The information will be updated in the WebSocket manager when a user disconnects due to some faults in the connection and reconnects with a different WebSocket server. The WebSocket manager, in turn, invalidates the data in the cache used by the WebSocket servers, and the updated data is sent to the corresponding cache. So, the information in the cache will remain there until it receives an invalidate signal from the WebSocket manager.

'''

Send or receive media files

So far, we've discussed the communication of text messages. But what happens when a user sends media files? Usually, the WebSocket servers are lightweight and don't support heavy logic such as handling the sending and receiving of media files. We have another service called the asset service, which is responsible for sending and receiving media files.

Moreover, the sending of media files consists of the following steps:

1. The media file is compressed and encrypted on the device side.
2. The compressed and encrypted file is sent to the asset service to store the file on blob storage. The asset service assigns an ID that's communicated with the sender. The asset service also maintains a hash for each file to avoid duplication of content on the blob storage. For example, if a user wants to upload an image that's already there in the blob storage, the image won't be uploaded. Instead, the same ID is forwarded to the receiver.
3. The asset service sends the ID of media files to the receiver via the message service. The receiver downloads the media file from the blob storage using the ID.
4. The content is loaded onto a CDN if the asset service receives a large number of requests for some particular content.

The following figure demonstrates the components involved in sharing media files over WhatsApp messenger:

[Sending media files via the asset service]

Support for group messages

WebSocket servers don't keep track of groups because they only track active users. However, some users could be online and others could be offline in a group. For group messages, the following three main components are responsible for delivering messages to each user in a group:

- Group message handler
- Group message service
- Kafka

Let's assume that user A wants to send a message to a group with some unique ID for example Group/A. The following steps explain the flow of a message sent to a group:

1. Since user A is connected to a WebSocket server, it sends a message to the message service intended for Group/A.
2. The message service sends the message to Kafka with other specific information about the group. The message is saved there for further processing. In Kafka terminology, a group can be

a topic, and the senders and receivers can be producers and consumers, respectively.

3. Now, here comes the responsibility of the group service. The group service keeps all information about users in each group in the system. It has all the information about each group, including user IDs, group ID, status, group icon, number of users, and so on. This service resides on top of the MySQL database cluster, with multiple secondary replicas distributed geographically. A Redis cache server also exists to cache data from the MySQL servers. Both geographically distributed replicas and Redis cache aid in reducing latency.

4. The group message handler communicates with the group service to retrieve data of Group/A users.

5. In the last step, the group message handler follows the same process as a WebSocket server and delivers the message to each user.

[Components responsible for sending group messages]

'''

Optional: How encryption and decryption work in WhatsApp

End-to-end encryption

Similar to the group message service, thereâs another service known as user service that keeps track of each user. It contains user-related data, such as name, profile picture, status, and so on. This service resides on top of the MySQL database and utilizes cache for frequently used data.

At registration time, a WhatsApp client generates a number of public-private key pairs that include keys pairs for identity, a signed prekey, and a batch of one-time prekeys. These keys are used for different purposes. The public keys are stored in the database associated with the user service along with the userâs identifier. Whenever a sender wants to establish a connection with the receiver, it also requests the server for the keys associated with a receiverâs ID. Private keys are only stored on the clientâs end devices.

One-to-one communication

To communicate securely, WhatsApp clients, on both sender and receiver sides, establish a pairwise encrypted session with each other. The session is created using the following steps:

The sender requests the public identity key, signed prekey, and a single public one-time prekey of the receiverâs device. The one-time prekey is used only once.

The WhatsApp server returns the requested public keys of the receiver.

The sender generates an ephemeral Curve25519 key pair.

The sender creates a master secret key using its own identity key, the ephemeral key pair, and the receiverâs public keys.

The sender uses the HMAC-based key-derivation function (HKDF) to create two other keys known as a root key and chain key from the master secret key. The sender starts sending messages to the receiver after establishing the pairwise encrypted session. The receiver creates the same session based on the information present in the header of the first receiving message.

After establishing a session, an 80-byte Message Key is generated based on the root and chain Keys. The Message Key is used for encryption and authentication of messages.

Group communication

The communication in a WhatsApp group is handled by the same process discussed in the previous section on one-to-one communication. A sender key, used by the signaling messaging protocol, is generated and shared with each memberâs device of the group using the pairwise encrypted

session. The messages sent to a group are encrypted using the sender key.

The communication in a group is directed by Kafka, where each user in a group subscribes to the relevant topic (a group with an associated messaging queue). The messages are then delivered in a fanout messaging pattern where messages are delivered from a single sender to multiple receivers.

Simultaneous maintenance of WhatsApp sessions on multiple devices

Every device on WhatsApp is identified by a device identity key. If a user uses multiple devices (a primary and several other companion devices), each device identity key is stored against the user's account on the WhatsApp server. When the WhatsApp server receives a message from a user, it transmits messages multiple times to each device linked with a user's account.

Similarly, each device used by a user has its own set of encryption keys. If a set of keys of one companion device is compromised, the attacker won't be able to see messages communicated with the other device.

We've just touched the basics here because security is a deep topic. See the WhatsApp security whitepaper for further details.

Reference:

<https://www.whatsapp.com/security/WhatsApp-Security-Whitepaper.pdf>

'''

Put everything together

We discussed the features of our WhatsApp system design. It includes user connection with a server, sending messages and media files, group messages, and end-to-end encryption, individually. The final design of our WhatsApp messenger is as follows:

[The detailed WhatsApp design. Interaction of each component with other components is shown with arrows]

In the next lesson, we'll evaluate our design and look into the non-functional requirements.

Design WhatsApp/Evaluation of WhatsApp's Design/README.md

Evaluation of WhatsApp's Design

Fulfill the requirements

Our non-functional requirements for the proposed WhatsApp design are low latency, consistency, availability, and security. Let's discuss how we have achieved these requirements in our system:

- Low latency: We can minimize the latency of the system at various levels:

- We can do this through geographically distributed WebSocket servers and the cache associated with them.

- We can use Redis cache clusters on top of MySQL database clusters.

- We can use CDNs for frequently sharing documents and media content.

- Consistency: The system also provides high consistency in messages with the help of a FIFO messaging queue with strict ordering. However, the ordering of messages would require the Sequencer to provide ID with appropriate causality inference mechanisms to each message. For offline users, the Mnesia database stores messages in a queue. The messages are sent later in sequence after the user goes online.

- Availability: The system can be made highly available if we have enough WebSocket servers and replicate data across multiple servers. When a user gets disconnected due to some fault in the WebSocket server, the session is re-created via a load balancer with a different server. Moreover, the messages are stored on the Mnesia cluster following the primary-secondary replication model, which provides high availability and durability.
- Security: The system also provides an end-to-end encryption mechanism that secures the chat between users.

'''

Question

In the event of a network partition, what should the system choose to compromise between consistency and availability?

Answer

According to the CAP theorem, the system would provide either consistency or availability in the event of a network partition. In our system of a WhatsApp messenger, the correct ordering of messages is essential. Otherwise, the context of the information communicated between users might change significantly. Therefore, availability in our system can take a hit if the network partition occurs.

'''

'''

Approaches to Achieve the Non-functional Requirements	
Non-functional Requirements	Approaches
Minimizing latency and servers	- Geographically distributed cache management system - CDN
Consistency other mechanisms	- Provide unique IDs to messages using Sequencer or - Use FIFO messaging queue with strict ordering
Availability establish connections between users users and groups on different servers	- Provide multiple WebSocket servers and managers to - Replication of messages and data associated with - Follow disaster recovery protocols
Security	Via end-to-end encryption

'''

Trade-offs

Weâ ve seen that our proposed WhatsApp system fulfills the functional and non-functional requirements. However, two major trade-offs exist in the proposed WhatsApp design:

- Thereâ s a trade-off between consistency and availability.
- Thereâ s a trade-off between latency and security.

The trade-off between consistency and availability

According to the CAP theorem, in the event of a network failure or partition, the system can provide either consistency or availability. So, in the case of our WhatsApp design, we have to choose either consistency or availability. In WhatsApp, the order of messages sent or received by a user is essential. Therefore, we should prioritize consistency rather than availability.

The trade-off between latency and security

Low latency is an essential factor in system design that provides a real-time experience to

users. However, on the other side, sharing information or data over WhatsApp might be insecure without encryption. The absence of a proper security mechanism makes the data vulnerable to unauthorized access. So, we can accept a trade-off prioritizing the secure transmission of messages over low latency.

We might wonder where the trade-off is. Often, communication involves multimedia. Encrypting them in near real-time on the sender device and decrypting on the receiver side can be taxing for the devices, causing latency. The process is illustrated in the following figure:

[End-to-end encryption causes a delay in processing, which has an impact on latency]

Summary

In this chapter, we designed a WhatsApp messenger. First, we identified the functional and non-functional requirements along with the resources estimation crucial for the design. Second, we focused on the high-level and detailed design of the WhatsApp system, where we described various components responsible for different services. Finally, we evaluated the non-functional requirements and highlighted some trade-offs in the design.

This design problem highlighted that we can optimize general-purpose computational resources for specific use cases. WhatsApp optimized its software stack to handle a substantially large number of connections on the commodity servers.

Design WhatsApp/High-level Design of WhatsApp/README.md

High-level Design of WhatsApp

High-level design

At an abstract level, the high-level design consists of a chat server responsible for communication between the sender and the receiver. When a user wants to send a message to another user, both connect to the chat server. Both users send their messages to the chat server. The chat server then sends the message to the other intended user and also stores the message in the database.

[The high-level design of WhatsApp messenger](./design.jpg)

The following steps describe the communication between both clients:

1. User A and user B create a communication channel with the chat server.
2. User A sends a message to the chat server.
3. Upon receiving the message, the chat server acknowledges back to user A.
4. The chat server sends the message to user B and stores the message in the database if the receiver's status is offline.
5. User B sends an acknowledgment to the chat server.
6. The chat server notifies user A that the message has been successfully delivered.
7. When user B reads the message, the application notifies the chat server.
8. The chat server notifies user A that user B has read the message.

The process is shown in the following illustrations:

[How](./process)

API design

WhatsApp provides a vast amount of features to its users via different APIs. Some features are mentioned below:

- Send message
- Get message or receive message
- Upload a media file or document
- Download document or media file
- Send a location
- Send a contact

- Create a status
However, weâ ll discuss essential APIs related to the first four features.

Send message

The sendMessage API is as follows:

'''

```
sendMessage(sender_ID, reciever_ID, type, text=None, media_object=None, document=None)
```

'''

This API is used to send a text message from a sender to a receiver by making a POST API call to the /messages API endpoint. Generally, the senderâ s and receiverâ s IDs are their phone numbers. The parameters used in this API call are described in the following table:

'''

Parameter	Description
sender_ID	This is a unique identifier of the user who sends the message.
reciever_ID	This is a unique identifier of the user who receives the message.
type	The default message type is text. This represents whether the sender sends a media file or a document.
text	This feild contains the text that has to be sent as a message.
media_object	This parameter is defined based on the type parameter. It represents the media file to be sent.
document	This represents the document file to be sent.

'''

Get message

The getMessage API is as follows:

'''

```
getMessage(user_Id)
```

'''

Using this API call, users can fetch all unread messages when they come online after being offline for some time.

'''

Parameter	Description
user_Id	This is a unique identifier representing the user who has to fetch all unread messages.

'''

Upload media or document file

The uploadFile API is as follows:

'''

```
uploadFile(file_type, file)
```

'''

We can upload media files via the uploadFile API by making a POST request to the /v1/media API endpoint. A successful response returns an ID thatâ s forwarded to the receiver. The maximum file size for media that can be uploaded is 16 MB, while the limit is 100 MB for a document.

'''

Parameter	Description
file_type	This represents the type of file uploaded via the API call.
file	This contains the file being uploaded via the API call.

'''

Download a document or media file

The downloadFile API is as follows:

'''

```
downloadFile(user_id, file_id)
```

'''

The parameters of this API call are explained in the following table:

'''

Parameter	Description
-----------	-------------

user_id	This is a unique identifier of the user who will download a file.
---------	---

file_id	This is a unique identifier of a file. It's generated while uploading a file through the uploadFile() API call. The downloadFile() API call downloads the media file through this identifier. The client can find the file_id by providing the file name to the server. That API call is not shown here.
---------	--

'''

In the next lesson, we'll focus on the detailed design of the WhatsApp system.

Design WhatsApp/Quiz on WhatsApp's Design/README.md

Quiz on WhatsApp's Design

'''

1

The WebSocket manager is responsible for which function?

A)

Routing data toward a user

B)

Maintaining the mapping between users and their WebSocket handlers

C)

Reconnecting users to blob storage

D)

None of the above

'''

'''

2

What happens to the message if it's sent to an offline user?

A)

The message is discarded, and an error response is sent back to the sender.

B)

The message is forwarded to another user.

C)

The message is stored in a database for a temporary period unless the user becomes online.

D)

None of the above

'''

'''

3

Why can't WhatsApp use the Mnesia database to store media files instead of using blob storage?

A)

Mnesia provides limited capacity.

B)

Blob store is optimized to store and retrieve large files.

C)

Mnesia is more costly than blob storage.

D)

None of the above

'''

'''

4

(Fill in the blank.) If a media file is shared and accessed multiple times, itâs stored in _____ from blob storage.

A)

Kafka

B)

the Mnesia database

C)

a CDN

D)

a MySQL database

'''

'''

5

(Fill in the blank.) WebSocket servers use cache to _____.

A)

store messages temporarily

B)

store messages permanently

C)

store mapping of users and their corresponding WebSocket handlers

D)

store metadata of each user

'''

Answers

1. (B)

2. (C)

3. (B)

4. (C)

5. (C)

Design WhatsApp/Requirements of WhatsAppâs Design/README.md

Requirements of WhatsAppâs Design

Requirements

Our design of the WhatsApp messenger should meet the following requirements.

Functional requirements

- Conversation: The system should support one-on-one and group conversations between users.
- Acknowledgment: The system should support message delivery acknowledgment, such as sent, delivered, and read.
- Sharing: The system should support sharing of media files, such as images, videos, and audio.
- Chat storage: The system must support the persistent storage of chat messages when a user is offline until the successful delivery of messages.
- Push notifications: The system should be able to notify offline users of new messages once their status becomes online.

Non-functional requirements

- Low latency: Users should be able to receive messages with low latency.
- Consistency: Messages should be delivered in the order they were sent. Moreover, users must see the same chat history on all of their devices.
- Availability: The system should be highly available. However, the availability can be compromised in the interest of consistency.
- Security: The system must be secure via end-to-end encryption. The end-to-end encryption ensures that only the two communicating parties can see the content of messages. Nobody in between, not even WhatsApp, should have access.

[The non-functional requirements of the WhatsApp system](./nfr.jpg)

Resource estimation

WhatsApp is the most used messaging application across the globe. According to WhatsApp, it supports more than two billion users around the world who share more than 100 billion messages each day. We need to estimate the storage capacity, bandwidth, and number of servers to support such an enormous number of users and messages.

Storage estimation

As there are more than 100 billion messages shared per day over WhatsApp, let's estimate the storage capacity based on this figure. Assume that each message takes 100 Bytes on average. Moreover, the WhatsApp servers keep the messages only for 30 days. So, if the user doesn't get connected to the server within these days, the messages will be permanently deleted from the server.

$100 \text{ billion/day} \times 100 \text{ Bytes} = 10 \text{ TB/day}$

For 30 days, the storage capacity would become the following:

$30 \times 10 \text{ TB/day} = 300 \text{ TB/month}$

Besides chat messages, we also have media files, which take more than 100 Bytes per message. Moreover, we also have to store users' information and messages' metadata—for example, timestamp, ID, and so on. Along the way, we also need encryption and decryption for secure communication. Therefore, we would also need to store encryption keys and relevant metadata. So, to be precise, we need more than 300 TB per month, but for the sake of simplicity, let's stick to the number 300 TB per month.

[The total storage required by WhatsApp in a month](./storage.jpg)

Bandwidth estimation

According to the storage capacity estimation, our service will get 10TB of data each day, giving us a bandwidth of 926 Mb/s.

10 TB/86400secâ 926 Mb/s

'''

Note: To keep our design simple, weâ ve ignored the media content (images, videos, documents, and so on). So, the number 926 might seem low.

'''

We also require an equal amount of outgoing bandwidth as the same message from the sender would need to be delivered to the receiver.

[The total bandwidth required by WhatsApp](./bandwidth.jpg)

'''

High-level Estimates

Type	Estimates
Total messages per day	100 billion
Storage required per day	10 TB
Storage for 30 days	300 TB
Incoming data per second	926 Mb/s
Outgoing data per second	926 Mb/s

'''

Number of servers estimation

WhatsApp handles around 10 million connections on a single server, which seems quite high for server. However, itâ s possible by extensive performance engineering. Weâ ll need to know al in-depth details of a system, such as a serverâ s kernel, networking library, infrastructure configuration, and so on.

'''

Note: We can often optimize a general-purpose server for special tasks by careful performance engineering of the full software stack.

'''

Letâ s move to the estimation of the number of servers:

No. of servers=Total connections per day/No. of connections per server=2 billion/10 million=200 servers

So, according to the above estimates, we require 200 chat servers.

[Number of chat servers required for WhatsApp](./servers.jpg)

Try it out

Letâ s analyze how the number of messages per day affects the storage and bandwidth requirements. For this purpose, we can change values in the following table to compute the estimates:

'''

Number of users per day (in billions)	2
Number of messages per day (in billions)	100
Size of each message (in bytes)	100

Number of connections a server can handle (in millions) 10
Storage estimation per day (in TB) 10
Incoming and Outgoing bandwidth (Mb/s) 926.4
Number of chat servers required 200
'''

Building blocks we will use

The design of WhatsApp utilizes the following building blocks that have also been discussed in the initial chapters:

[The building blocks required to design WhatsApp](./bb.jpg)

- Databases are required to store usersâ and groupsâ metadata.
- Blob storage is used to store multimedia content shared in messages.
- A CDN is used to effectively deliver multimedia content thatâs frequently shared.
- A load balancer distributes incoming requests among the pool of available servers.
- Caches are required to keep frequently accessed data used by various services.
- A messaging queue is used to temporarily keep messages in a queue on a database when a user is offline.

In the next lesson, weâll focus on the high-level design of the WhatsApp messenger.

Design WhatsApp/System Design WhatsApp/README.md

System Design: WhatsApp

WhatsApp

In todayâs technological world, WhatsApp is an important messaging application that connects billions of people around the globe. Many users start their day by reading or sending WhatsApp messages to their friends and family. According to July 2021 estimates, WhatsApp has two billion active users worldwide. Further, an average WhatsApp user spends approximately 19.4 hours per month on the application.

In December 2020, the WhatsApp CEO tweeted that WhatsApp users share more than 100 billion messages per day, an increase of approximately 54% since 2018. The increase in messages sent globally per day is depicted in the following chart:

Design problem

As system designers, we should be aware of the growth rate of users. There are many interesting questions about WhatsApp:

- How is this application designed?
- How does it work?
- What are the different types of components involved in it?
- How does WhatsApp enable billions of users to communicate with each other?
- How does WhatsApp keep all that data secure?

In this chapter, weâll focus on the high-level and detailed design of the WhatsApp application to answer the above questions.

How will we design WhatsApp?

Weâve divided the design of WhatsApp messenger into the following five lessons:

1. Requirements: In this lesson, weâll identify functional and non-functional requirements. Weâll also discuss resource estimations required for better and smooth operations of the proposed design of WhatsApp.
2. High-level Design: Weâll focus on the high-level design of our WhatsApp version. Weâll discuss essential APIs for our WhatsApp design.
3. Detailed Design: In this lesson, weâll describe the design of our WhatsApp messenger in detail. Initially, weâll explain the design of each microservice, including connection with servers, send and receive messages and media content, and group messages. In the end, the design of each microservice is combined into the detailed design of WhatsApp.
4. Evaluation: This lesson will explain how our version of WhatsApp fulfills non-functional

requirements. Weâ ll also evaluate some trade-offs of our design.

5. Quiz: Here, weâ ll assess what weâ ve learned in this chapter through a quiz.

Letâ s start by discussing the requirements of our version of WhatsApp.

Design Youtube/Design of YouTube/README.md

Design of YouTube

High-level design

The high-level design shows how weâ ll interconnect the various components we identified in the previous lesson. We have started developing a solution to support the functional and non-functional requirements with this design.

[The high-level design of YouTube](./sys.jpg)

The workflow for the abstract design is provided below:

1. The user uploads a video to the server.
2. The server stores the metadata and the accompanying user data to the database and, at the same time, hands over the video to the encoder for encoding (see 2.1 and 2.2 in the illustration above).
3. The encoder, along with the transcoder, compresses the video and transforms it into multiple resolutions (like 2160p, 1440p, 1080p, and so on). The videos are stored on blob storage (similar to GFS or S3).
4. Some popular videos may be forwarded to the CDN, which acts as a cache.
5. The CDN, because of its vicinity to the user, lets the user stream the video with low latency. However, CDN is not the only infrastructure for serving videos to the end user, which we will see in the detailed design.

'''

Question

Why donâ t we upload the video directly to the encoder instead of to the server? Doesnâ t the current strategy introduce an additional delay?

Answer

There are several reasons why itâ s a good idea to introduce a server in between the encoder and the client:

- The client could be malicious and could abuse the encoder.
- If the uploaded video is a duplicate, the server could filter it out.
- Encoders will be available on a private IP address within YouTubeâ s network and not available for public access.

'''

API design

Letâ s understand the design of APIs in terms of the functionalities weâ re providing. Weâ design APIs to translate our feature set into technical specifications. In this case, REST API can be used for simplicity and speed purposes. Our API design section will help us understand how the client will request services from the back-end application of YouTube. Letâ s develop APIs for each of the following features:

- Upload videos
- Stream videos
- Search videos
- View thumbnails
- Like or dislike videos
- Comment on videos

[API design overview](./api.jpg)

Upload video

The POST method can upload a video to the /uploadVideo API:

'''

```
uploadVideo(user_id, video_file, category_id, title, description, tags, default_language,
privacy_settings)
```

'''

Let's take a look at the description of the following parameters here.

'''

Parameter	Description
user_id	This is the user that is uploading the video.
video_file	This is the video file that the user wants to upload.
category_id	This refers to the category a video belongs to. Typical categories can be "Entertainment," "Engineering," "Science," and so on.
title	This is the title of the video.
description	This is the description of the video.
tags	This refers to the specific topics the content of the video covers. The tags can improve search results.
default_language	This is the default language a page will show to the user when the video is streamed.
privacy_settings	This refers to the privacy of the video. Generally, videos can be a public asset or private to the uploader.

'''

'''

Parameter	Description
user_id	This is the user that is uploading the video.
video_file	This is the video file that the user wants to upload.
category_id	This refers to the category a video belongs to. Typical categories can be "Entertainment," "Engineering," "Science," and so on.
title	This is the title of the video.
description	This is the description of the video.
tags	This refers to the specific topics the content of the video covers. The tags can improve search results.
default_language	This is the default language a page will show to the user when the video is streamed.
privacy_settings	This refers to the privacy of the video. Generally, videos can be a public asset or private to the uploader.

'''

'''

The video file is broken down into smaller packets and uploaded to the server in order. In case of failure, YouTube can store data for a limited time and resume the upload if the user retries. To understand the concept in detail, read more about asynchronous APIs.

'''

'''

Asynchronous API: APIs where the server cannot fulfill the request immediately, either because the data is too large or isn't readily available. In that case, the client gets an immediate response that the server has accepted the request while the server responds with the data later on. Such a mechanism saves the CPU and network bandwidth, and does not keep the client waiting

'''

Stream video

The GET method is best suited for the /streamVideo API:

streamVideo(user_id, video_id, screen_resolution, user_bitrate, device_chipset)

Some new things introduced in this case are the following parameters:

'''

Parameter	Description
screen_resolution	The server can best optimize the video if the user's screen resolution is known.
user_bitrate	The transmission capacity of the user is required to understand which quality of video chunks should be transferred to the client or user.
device_chipset	Many YouTube users watch content on handheld devices, which makes it important to know the handling capabilities of these devices to better serve the users.

'''

'''

The server will store different qualities of the same video in its storage and serve users based on their transmission rate.

'''

Search videos

The /searchVideo API uses the GET method:

'''

searchVideo(user_id, search_string, length, quality, upload_date)

'''

'''

Parameter	Description
search_string	This is the string used for searching videos by their title.
length (optional)	This is used to filter videos based on their length in terms of time.
quality (optional)	This is used to filter videos based on the resolution, like 2048p, 1440p, 1080p, and so on.
upload_date (optional)	This is used to filter videos based on their upload date to YouTube.

'''

View thumbnails

We can use the GET method to access the /viewThumbnails API:

'''

viewThumbnails(user_id, video_id)

'''

'''

Parameter	Description
video_id	This specifies the unique ID of the video associated with the thumbnails.

This API will return the thumbnails of a video in a sequence.

Like and dislike a video

The like and dislike API uses the GET method. As shown below, it's fairly simple.

```
'''  
likeDislike(user_id, video_id, like)  
'''
```

We can use the same API for the like and dislike functionality. Depending on what is passed as a parameter to the like field, we can update the database accordinglyâ that is, 0 for like and 1 for dislike.

Comment video

Much like the like and dislike API, we only have to provide the comment string to the API. This API will also use the GET method.

```
'''  
commentVideo(user_id, video_id, comment_text)  
'''
```

'''

Parameter	Description
-----------	-------------

comment_text	This refers to the text that is typed by the user on the particular video.
--------------	--

'''

Storage schema

Each of the above features in the API design requires support from the databaseâ we'll need to store the details above in our storage schema to provide services to the API gateway.

[Storage schema](./db.jpg)

'''

Note: Much of the underlying details regarding database tables that can be mapped to services provided by YouTube have been omitted for simplicity. For example, one video can have different qualities and that is not mentioned in the "Video" table.

'''

Detailed design

Now, let's get back to our high-level design and see if we can further explore parts of the design. In particular, the following areas require more discussion:

- Component integration: We'll cover some interconnections between the servers and storage components to better understand how the system will work.
- Thumbnails: It's important for users to see some parts of the video through thumbnails. Therefore, we'll add thumbnail generation and storage to the detailed design.
- Database structure: Our estimation showed that we require massive storage space. We also require storing varying types of data, such as videos, video metadata, and thumbnails, each of which demands specialized data storage for performance reasons. Understanding the database details will enable us to design a system with the least possible lag.

Let's take a look at the diagram below. We'll explain our design in two steps, where the first looks at what the newly added components are, and the second considers how they coordinate to build the YouTube system.

Detailed design components

Since we highlighted the requirements of smooth streaming, server-level details, and thumbnail features, the following design will meet our expectations. Let's explain the purpose of each added component here:

- Load balancers: To divide a large number of user requests among the web servers, we require load balancers.
- Web servers: Web servers take in user requests and respond to them. These can be considered the interface to our API servers that entertain user requests.

- Application server: The application and business logic resides in application servers. They prepare the data needed by the web servers to handle the end usersâ€™ queries.
- User and metadata storage: Since we have a large number of users and videos, the storage required to hold the metadata of videos and the content related to users must be stored in different storage clusters. This is because a large amount of not-so-related data should be decoupled for scalability purposes.
- Bigtable: For each video, weâ€™ll require multiple thumbnails. Bigtable is a good choice for storing thumbnails because of its high throughput and scalability for storing key-value data. Bigtable is optimal for storing a large number of data items each below 10 MB. Therefore, it is the ideal choice for YouTubeâ€™s thumbnails.
- Upload storage: The upload storage is temporary storage that can store user-uploaded videos.
- Encoders: Each uploaded video requires compression and transcoding into various formats. Thumbnail generation service is also obtained from the encoders.
- CDN and colocation sites: CDNs and colocation sites store popular and moderately popular content that is closer to the user for easy access. Colocation centers are used where itâ€™s not possible to invest in a data center facility due to business reasons.

'''

Bigtable: Chang, Fay, et al. â€” Bigtable: A distributed storage system for structured data.â€” Transactions on Computer Systems (TOCS) 26.2 (2008): 1-26.

'''

[Detailed design of YouTubeâ€™s components](./c.jpg)

Design flow and technology usage

Now that we understand the purpose of every component, letâ€™s discuss the flow and technology used in different components in the following steps:

1. The user can upload a video by connecting to the web servers. The web server can run Apache or Lighttpd. Lighttpd is preferable because it can serve static pages and videos due to its fast speed.
2. Requests from the web servers are passed onto application servers that can contact various data stores to read or write user, videos, or videosâ€™ metadata. There are separate web and application servers because we want to decouple clientsâ€™ services from the application and business logic. Different programming languages can be used on this layer to perform different tasks efficiently. For example, the C programming language can be used for encryption. Moreover, this gives us an additional layer of caching, where the most requested objects are stored on the application server while the most frequently requested pages will be stored on the web servers.
3. Multiple storage units are used. Letâ€™s go through each of these:
 - I. Upload storage is used to store user-uploaded videos before they are temporarily encoded.
 - II. User account data is stored in a separate database, whereas videos metadata is stored separately. The idea is to separate the more frequently and less frequently accessed storage clusters from each other for optimal access time. We can use MySQL if there are a limited number of concurrent reads and writes. However, as the number of usersâ€™ and therefore the number of concurrent reads and writesâ€™ grows, we can move towards NoSQL types of data management systems.
 - III. Since Bigtable is based on Google File System (GFS), it is designed to store a large number of small files with low retrieval latency. It is a reasonable choice for storing thumbnails.
4. The encoders generate thumbnails and also store additional metadata related to videos in the metadata database. It will also provide popular and moderately popular content to CDNs and colocation servers, respectively.
5. The user can finally stream videos from any available site.

'''

A colocation server is placed in a colocation/carrier site that is equipped with storage and networking resources. These resources can be rented to customers based on their needs.

YouTube search

Since YouTube is one of the most visited websites, a large number of users will be using the search feature. Even though we have covered a building block on distributed search, we'll provide a basic overview of how search inside the YouTube system will work.

Each new video uploaded to YouTube will be processed for data extraction. We can use a JSON file to store extracted data, which includes the following:

- Title of the video.
- Channel name.
- Description of the video.
- The content of the video, possibly extracted from the transcripts.
- Video length.
- Categories.

Each of the JSON files can be referred to as a document. Next, keywords will be extracted from the documents and stored in a key-value store. The key in the key-value store will hold all the keywords searched by the users, while the value in the key-value store will contain the occurrence of each key, its frequency, and the location of the occurrence in the different documents. When a user searches for a keyword, the videos with the most relevant keywords will be returned.

[An abstraction of how YouTube search works](./search.jpg)

The approach above is simplistic, and the relevance of keywords is not the only factor affecting search in YouTube. In reality, a number of other factors will matter. The processing engine will improve the search results by filtering and ranking videos. It will make use of other factors like view count, the watch time of videos, and the context, along with the history of the user, to improve search results.

Design Youtube/Evaluation of YouTube's Design/README.md

Evaluation of YouTube's Design

Fulfilling requirements

Our proposed design needs to fulfill the requirements we mentioned in the previous lessons. Our main requirements are smooth streaming (low latency), availability, and reliability. Let's discuss them one by one.

1. Low latency/Smooth streaming can be achieved through these strategies:

- Geographically distributed cache servers at the ISP level to keep the most viewed content.
- Choosing appropriate storage systems for different types of data. For example, we'll can use Bigtable for thumbnails, blob storage for videos, and so on.
- Using caching at various layers via a distributed cache management system.
- Utilizing content delivery networks (CDNs) that make heavy use of caching and mostly serve videos out of memory. A CDN deploys its services in close vicinity to the end users for low-latency services.

2. Scalability: We've taken various steps to ensure scalability in our design as depicted in the table below. The horizontal scalability of web and application servers will not be a problem as the users grow. However, MySQL storage cannot scale beyond a certain point. As we'll see in the coming sections, that may require some restructuring.

3. Availability: The system can be made available through redundancy by replicating data to as many servers as possible to avoid a single point of failure. Replicating data across data centers will ensure high availability, even if an entire data center fails because of power or network issues. Furthermore, local load balancers can exclude any dead servers, and global load balancers can steer traffic to a different region if the need arises.

4. Reliability: YouTube's system can be made reliable by using data partitioning and fault-

tolerance techniques. Through data partitioning, the non-availability of one type of data will not affect others. We can use redundant hardware and software components for fault tolerance. Furthermore, we can use the heartbeat protocol to monitor the health of servers and omit servers that are faulty and erroneous. We can use a variant of consistent hashing to add or remove servers seamlessly and reduce the burden on specific servers in case of non-uniform load.

'''

The heartbeat protocol is a way of identifying failures in distributed systems. Using this protocol, every node in a cluster periodically reports its health to a monitoring service.

'''

'''

Variant consistent hashing: Mirrokni, Vahab, Mikkel Thorup, and Morteza Zadimoghaddam. *Consistent hashing with bounded loads*. Proceedings of the Twenty-Ninth Annual ACM-SIAM Symposium on Discrete Algorithms. Society for Industrial and Applied Mathematics, 2018.

'''

'''

Question

Isn't the load balancer a single point of failure (SPOF)?

Answer

Just like with servers, we can use multiple load balancers. Users can be randomly forwarded to different load balancers from the Domain Name System (DNS).

'''

[How YouTube achieves scalability, availability, and good performance](./req.jpg)

Trade-offs

How YouTube achieves scalability, availability, and good performance

Consistency

Our solution prefers high availability and low latency. However, strong consistency can take a hit because of high availability (see the CAP theorem). Nonetheless, for a system like YouTube we can afford to let go of strong consistency. This is because we don't need to show a consistent feed to all the users. For example, different users subscribed to the same channel may not see a newly uploaded video at the same time. It's important to mention that we'll maintain strong consistency of user data. This is another reason why we've decoupled user data from video metadata.

Distributed cache

We prefer a distributed cache over a centralized cache in our YouTube design. This is because the factors of scalability, availability, and fault-tolerance, which are needed to run YouTube require a cache that is not a single point of failure. This is why we use a distributed cache. Since YouTube mostly serves static content (thumbnails and videos), Memcached is a good choice because it is open source and uses the popular Least Recently Used (LRU) algorithm. Since YouTube video access patterns are long-tailed, LRU-like algorithms are suitable for such data sets.

Bigtable versus MySQL

Another interesting aspect of our design is the use of different storage technologies for different data sets. Why did we choose MySQL and Bigtable?

The primary reason for the choice is performance and flexibility. The number of users in YouTube may not scale as much as the number of videos and thumbnails do. Moreover, we require storing the user and metadata in structured form for convenient searching. Therefore, MySQL is a suitable choice for such cases.

However, the number of videos uploaded and the thumbnails for each video would be very large in number. Scalability needs would force us to use a custom or NoSQL type of design for that storage. One could use alternatives to GFS and Bigtable, such as HDFS and Cassandra.

Public versus private CDN

Our design relies on CDNs for low latency serving of the content. However, CDNs can be private or public. YouTube can choose between any one of the two options.

This choice is more of a cost issue than a design issue. However, for areas where there is little traffic, YouTube can use the public CDN because of the following reasons:

1. Setting up a private CDN will require a lot of CAPEX.
2. For rather little viral traffic in certain regions, there will not be enough time to set up a new CDN.
3. There may not be enough users to sustain the business.

'''

Private CDN: A CDN infrastructure that is for internal use only.

Public CDN: A CDN infrastructure that is available to the public on a pay-as-you-use basis.

'''

However, YouTube can consider building its own CDN if the number of users is too high, since public CDNs can prove to be expensive if the traffic is high. Private CDNs can also be optimized for internal usage to better serve customers.

Duplicate videos

The current YouTube design doesn't handle duplicate videos that have been uploaded by a user or spammers. Duplicated videos take extra space, which leads to a trade-off. As a result, we either waste storage space or face an additional complexity to the upload process for handling duplicate videos.

Let's perform some calculations to resolve this problem. Assume that 50 out of 500 hours of videos uploaded to YouTube are duplicates. Considering that one minute of video requires 6 MB of storage space, the duplicated content will take up the following storage space:

'''

$(50 \times 60) \text{ minutes} \times 6 \text{ MB/min} = 18 \text{ GB}$

'''

If we avoid video duplication, we can save up to 9.5 petabytes of storage space in a year. The calculations are as follows:

'''

$18 \text{ GB/min} \times (60 \times 24 \times 365) \text{ total minutes in an year} = 9.5 \text{ Peta Bytes}$

'''

Storage space being wasted, and other computational costs are not the only issues with duplicate videos. An important aspect of duplicate videos is the copyright issue. No content creator would want their content plagiarized. Therefore, it's plausible to add the complexity of handling duplicate videos to the design of YouTube.

Duplication can be solved with simple techniques like locality-sensitive hashing. However, there can be complex techniques like Block Matching Algorithms (BMAs) and phase correlation to find duplications. Implementing this solution can be quite complex in a huge database of videos. We may have to use technologies like artificial intelligence (AI).

Future scaling

So far, we've focused on the design and analysis of the proposed design for YouTube. In reality, the design of YouTube is quite complex and requires advanced systems. In this section, we'll focus on the pragmatic structure of data stores and the web server.

We'll begin our discussion with some limitations in terms of scaling YouTube. In particular, we'll consider what design changes we'll have to make if the traffic load to our service goes up by, say, a few folds.

We already know that we'll have to scale our existing infrastructure, which includes the below elements:

- Web servers
- Application servers

- Datastores
- Placing load balancers among each of the layers above
- Implementing distributed caches

Any infrastructure mentioned above requires some modifications and adaptation to the application-level logic. For example, if we continue to increase our data in MySQL servers, it can become a choke point. To effectively use a sharded database, we might have to make changes to our database client to achieve a good level of performance and maintain the ACID (atomicity, consistency, isolation, durability) properties. However, even if we continue to change to the database client as we scale, its complexity may reach a point where it is no longer manageable. Also note that we haven't incorporated a disaster recovery mechanism into our design yet.

To resolve the problems above, YouTube has developed a solution called Vitess.

[Vitess system for scalability](./vitess.jpg)

The key idea in Vitess is to put an abstraction on top of all the database layers, giving the database client the illusion that it is talking to a single database server. The single database in this case is the Vitess system. Therefore, all the database-client complexity is migrated to and handled by Vitess. This maintains the ACID properties because the internal database in use is MySQL. However, we can enable scaling through partitioning. Consequently, we'll get a MySQL structured database that gives the performance of a NoSQL storage system. At the same time, we won't have to live with a rich database client (application logic). The following illustration highlights how Vitess is able to achieve both scalability and structure.

[Vitess on the scalability versus structure spectrum](./spectrum.jpg)

One could imagine using techniques like data denormalization instead of the Vitess system. However, data denormalization won't work because it comes at the cost of reduced writing performance. Even if our work is read-intensive, as the system scales, writing performance will degrade to an unbearable limit.

Web server

A web server is an extremely important component and, with scale, a custom web server can be a viable solution. This is because most commercial or open-source solutions are general purpose and are developed with a wide range of users in mind. Therefore, a custom solution for such a successful service is desirable.

Let's try an interesting exercise to see which server YouTube currently uses. Click on the terminal below and execute the following command:

```
'''
```

```
lynx -head -dump http://www.youtube.com | grep ^Server
```

```
'''
```

Design Youtube/Quiz on YouTube's Design/README.md

Quiz on YouTube's Design

1^a Which pair of requirements among the following cannot be compromised if we're asked to do so on Netflix?

A)

Low latency and high availability

B)

Low latency and HD video

C)

High availability and HD video

D)

2^a Consider the following design to upload content to the Netflix service:

[design](./design.jpg)

A.[A](./2A.jpg)

B.[B](./2B.jpg)

C.[C](./2C.jpg)

D.[D](./2D.jpg)

3 Which encoding technique is the most effective in terms of streaming to end users?

A)

Use a single encoding technique for all types of content to achieve simplicity in design.

B)

Encode the entire video based on the content inside itâ for example, encode an action movie with dynamic colors differently than a 2D cartoon.

C)

Use different encoding techniques for different shots or segments within a video. For example, highly dynamic scenes should be encoded differently than lesser dynamic scenes within a single video.

D)

Encoding doesnât matter at all.

4 Adaptive bitrate streaming is a common technique used to provide a good streaming experience to the user. It works by detecting a userâs bandwidth and the CPU capacity of the userâs device in real time, and adjusts the quality of the media stream accordingly.

Considering the above explanation, which diagram explains adaptive bitrate the best?

A.[A](./4A.jpg)

B.[B](./4B.jpg)

C.[C](./4C.jpg)

D.[D](./4D.jpg)

5. What is the most appropriate sequence from development to end user delivery to stream content?

A)

Encoding, bitrate conversion, subtitles addition, and deployment

B)

Deployment, bitrate conversion, subtitles, and encoding

C)

Encoding, bitrate conversion, uploading to CDN, and deployment

D)
Uploading to CDN, encoders, and bitrate conversion

6. Assume that 100,000 users try to watch a football stream at 1080p in a densely populated city. If the CDN point of presence (PoP) in that area can provide a bandwidth of 300 Gbps for end users, which design best serves the clients without any buffering issues?

A.[A](./6A.jpg)

B.[B](./6B.jpg)

C.[C](./6C.jpg)

D.[D](./6D.jpg)

Answers

'''

1. Which pair of requirements among the following cannot be compromised if weâre asked to design Netflix?

Correct Answer

A)
Low latency and high availability

Explanation
Streaming services highly require these two design goals to provide a good QoE to end-user.

B)
Low latency and HD video

Explanation
It is extremely important for the service to be available even if the video is not high quality.

C)
High availability and HD video

Explanation
This could lead to buffering and lag issues. Therefore, low latency is required.

D)
Consistency and HD video

Explanation
Consistency isnât the most challenging or desired metric when it comes to streaming services
'''

'''

2. Consider the following design to upload content to the Netflix service:

Note that there are two unnamed components. Select the correct option to complete the design.

A)

Explanation

Metadata storage cannot be the first point of contact for the end-users.

B)

Explanation

It doesn't make sense for the uploading servers and Mapreduce to store data in colocation instead of a datastore.

Correct Answer

C)

Explanation

Users' requests are distributed among uploading servers using load balancers. Also, we can use a datastore to save (meta)data from uploading servers and Mapreduce.

D)

Explanation

Web servers require load balancers before them in practical designs. Moreover, uploading servers would extract metadata from raw videos and store it in a database that isn't a key-value store.

'''

'''

3. Which encoding technique is the most effective in terms of streaming to end users?

A)

Use a single encoding technique for all types of content to achieve simplicity in design.

Explanation

Depending on the type of content within a video, it is better to use different encoding schemes to get optimal results.

B)

Encode the entire video based on the content inside it—for example, encode an action movie with dynamic colors differently than a 2D cartoon.

Explanation

Encoding entire video helps but it is not the most optimal solution.

Correct Answer

C)

Use different encoding techniques for different shots or segments within a video. For example, highly dynamic scenes should be encoded differently than lesser dynamic scenes within a single video.

Explanation

Using a per-shot encoding technique can save storage space, transmission bandwidth, as well as make streaming convenient in terms of user bandwidth.

D)

Encoding doesn't matter at all.

Explanation

This isn't true. Encoding helps in the overall optimization.
'''

'''

4. Adaptive bitrate streaming is a common technique used to provide a good streaming experience to the user. It works by detecting a user's bandwidth and the CPU capacity of the user's device in real time, and adjusts the quality of the media stream accordingly.

Considering the above explanation, which diagram explains adaptive bitrate the best?

A)

Explanation

Webservers don't generate multiple video qualities. It is the job of encoders.

Your Answer

B)

Explanation

CDNs don't generate multiple video qualities. It is the job of encoders.

C)

Explanation

Application servers run the business or logic of the application instead of serving videos to the clients.

Correct Answer

D)

Explanation

The encoder converts videos into multiple formats and the streaming server sends it to the client.
'''

'''

5. What is the most appropriate sequence from development to end user delivery to stream content?

Correct Answer

A)

Encoding, bitrate conversion, subtitles addition, and deployment

B)

Deployment, bitrate conversion, subtitles, and encoding

Explanation

It is non-sensical to first deploy a video and then encode it.

C)

Encoding, bitrate conversion, uploading to CDN, and deployment

Explanation

Uploading to CDN is the same thing as deployment. Also, subtitles addition is missed out.

D)

Uploading to CDN, encoders, and bitrate conversion

Explanation

First, videos must be encoded and then deployed to CDNs.

'''

'''

6. Assume that 100,000 users try to watch a football stream at 1080p in a densely populated city. If the CDN point of presence (PoP) in that area can provide a bandwidth of 300 Gbps for end users, which design best serves the clients without any buffering issues?

A)

Explanation

Not the best choice because it will require an extraordinary bandwidth for the CDN to serve out 100,000 users each with 1080p quality of streaming service.

B)

Explanation

This strategy will put huge burden on the backbone Internet. Bottlenecks will create issues.

Correct Answer

C)

Explanation

Cache servers maintained at the ISP will provide coverage to all the users reducing bandwidth requirements for the CDN as well as the origin servers.

D)

Explanation

Streaming servers are an alternative to colocation sites. These will not be enough for 100,000 users within the same geographical location.

'''

Design Youtube/Requirements of YouTube's Design/README.md

Requirements of YouTube's Design

Requirements

Let's start with the requirements for designing a system like YouTube.

Functional requirements

We require that our system is able to perform the following functions:

1. Stream videos
2. Upload videos
3. Search videos according to titles
4. Like and dislike videos
5. Add comments to videos
6. View thumbnails

[Representation of the functional and non-functional requirements](./youtube.jpg)

Non-functional requirements

It's important that our system also meets the following requirements:

- High availability: The system should be highly available. High availability requires a good percentage of uptime. Generally, an uptime of 99% and above is considered good.

- Scalability: As the number of users grows, these issues should not become bottlenecks: storage for uploading content, the bandwidth required for simultaneous viewing, and the number of concurrent user requests should not overwhelm our application/web server.
- Good performance: A smooth streaming experience leads to better performance overall.
- Reliability: Content uploaded to the system should not be lost or damaged.

We don't require strong consistency for YouTube's design. Consider an example where a creator uploads a video. Not all users subscribed to the creator's channel should immediately get the notification for uploaded content.

To summarize, the functional requirements are the features and functionalities that the user will get, whereas the non-functional requirements are the expectations in terms of performance from the system.

Based on the requirements, we'll estimate the required resources and design of our system.

Resource estimation

Estimation requires the identification of important resources that we'll need in the system.

Hundreds of minutes of video content get uploaded to YouTube every minute. Also, a large number of users will be streaming content at the same time, which means that the following resources will be required:

- Storage resources will be needed to store uploaded and processed content.
- A large number of requests can be handled by doing concurrent processing. This means web/application servers should be in place to serve these users.
- Both upload and download bandwidth will be required to serve millions of users.

To convert the above resources into actual numbers, we assume the following:

- Total number of YouTube users: 1.5 billion.
- Active daily users (who watch or upload videos): 500 million.
- Average length of a video: 5 minutes.
- Size of an average (5 minute-long) video before processing/encoding (compression, format changes, and so on): 600 MB.
- Size of an average video after encoding (using different algorithms for different resolutions like MPEG-4 and VP9): 30 MB.

Storage estimation

To find the storage needs of YouTube, we have to estimate the total number of videos and the length of each video uploaded to YouTube per minute. Let's consider that 500 hours worth of content is uploaded to YouTube in one minute. Since each video of 30 MB is 5 minutes long, we require $30/5 = 6$ MB to store 1 minute of video.

Let's put this in a formula by assuming the following:

Total_{storage} : Total storage requirement.

Total_{upload/min} : Total content uploaded (in minutes) per minute.

- Example: 500 hours worth of video is uploaded in one minute.

Storage_{min} : Storage required for each minute of content

Then, the following formula is used to compute the storage:

Total_{storage} = Total_{upload/min} x Storage_{min}

Below is a calculator to help us estimate our required resources. We'll look first at the storage required to persist 500 hours of content uploaded per minute, where each minute of video costs 6 MBs to store:

'''

Storage Required for Storing Content per Minute on YouTube

No. of video hours per minute	Minutes per hour	MB per minute
500	60	6
180		

'''

'''

Try changing the values of Hours and MB per minute to see their impact on storage space requirements.

'''

The numbers mentioned above correspond to the compressed version of videos. However, we need to transcode videos into various formats for reasons that we will see in the coming lessons. Therefore, we'll require more storage space than the one estimated above.

[Total storage required by YouTube in a year](./storage.jpg)

'''

Question

Assuming YouTube stores videos in five different qualities and the average size of a one-minute video is 6 MB, what would the estimated storage requirements per minute be?

Answer

Since we need 6 MBs per minute and we need to store files in five different formats, the overall requirement is $5 \times 6\text{MB} = 30\text{ MBs}$.

Next, we'll calculate the below value:

Number of hours uploaded to Youtube/min x Number of minutes per hour x storage requirement per minute = 500 hours/ minute x 60 minutes/hour x 30 MB/minute = 900 GB/minute

'''

Bandwidth estimation

A lot of data transfer will be performed for streaming and uploading videos to YouTube. This is why we need to calculate our bandwidth estimation too. Assume the upload:view ratio is 1:300 that is, for each uploaded video, we have 300 video views per second. We'll also have to keep in mind that when a video is uploaded, it is not in compressed format, while viewed videos can be of different qualities. Let's estimate the bandwidth required for uploading the videos.

We assume:

Total_{bandwidth}: Total bandwidth required.

Total_{content}: Total content (in minutes) uploaded per minute.

Size_{minute}: Transmission required (in MBs) for each minute of content.

Then, the following formula is used to do the computation below:

Total bandwidth = Total content_transferred ^ Size minute

'''

The Bandwidth Required for Uploading Videos to YouTube

No. of video hours per minute	Minutes per hour	MB per minute
500	60	50
180		

'''

'''

We calculate bandwidth in bits per second (bps), as shown above. Therefore, the detailed calculation of the above number (200 Gbps) is given below.

500 hours/minute ^ 60 minutes/hour ^ 50 MB/minute x 8 bits/60 seconds =200 Gbps
'''

'''

Question

If 200 Gbps of bandwidth is required for satisfying uploading needs, how much bandwidth would be required to stream videos? Assume each minute of video requires 10 MB of bandwidth on average.

Hint: The upload:view ratio is provided.

Hide Answer

For every video uploaded, 300 videos are watched. Therefore, the equation becomes this:

total number of viewing hours per minutes ^ size in MB of each minute ^ view ratio=500
hours/minute x 60 minutes/hour x 10 MB/minute x 300 = 90 TB/minute x 8 bits= 720 Tb/minute = 1
Tbps
'''

[Total bandwidth required by YouTube](./bandwidth.jpg)

Number of servers estimation

We need to handle concurrent requests coming from 500 million daily active users. Let's assume that a typical YouTube server handles 8,000 requests per second.

Number of Active Users/Queries handled per server=62,500 servers

[Number of servers required for YouTube](./servers.jpg)

'''

Note: In a real-world scenario, YouTube's design requires storage for thumbnails, users' video metadata, users' channel information, and so on. Since the storage requirement for the data sets will not be significant compared to video files, we ignore it for simplicity's sake.
'''

Building blocks we will use

Now that we have completed the resource estimations, let's identify the building blocks that will be an integral part of our design for the YouTube system. The key building blocks are given below:

[Building blocks in a high-level design](./bb.jpg)

- Databases are required to store the metadata of videos, thumbnails, comments, and user-related information.
- Blob storage is important for storing all videos on the platform.
- A CDN is used to effectively deliver content to end users, reducing delay and burden on end-servers.
- Load balancers are a necessity to distribute millions of incoming clients requests among the pool of available servers.

Other than our building blocks, we anticipate the use of the following components in our high-level design:

- Servers are a basic requirement to run application logic and entertain user requests.
- Encoders and transcoders compress videos and transform them into different formats and qualities to support varying numbers of devices according to their screen resolution and

bandwidth.

[Components in YouTube's high-level design](./c.jpg)

Design Youtube/System Design Youtube/README.md

System Design: YouTube

What is YouTube?

YouTube is a popular video streaming service where users upload, stream, search, comment, share, and like or dislike videos. Large businesses as well as individuals maintain channels where they host their videos. YouTube allows free hosting of video content to be shared with users globally. YouTube is considered a primary source of entertainment, especially among young people, and as of 2022, it is listed as the second-most viewed website after Google by Wikipedia.

[A depiction of a content creator uploading a video to YouTube that is streamed by many viewers](./youtube.jpg)

YouTube's popularity

Some of the reasons why YouTube has gained popularity over the years include the following reasons:

- Simplicity: YouTube has a simple yet powerful interface.
- Rich content: Its simple interface and free hosting has attracted a large number of content creators.
- Continuous improvement: The development team of YouTube has worked relentlessly to meet scalability requirements over time. Additionally, Google acquired YouTube in 2007, which has added to its credibility.
- Source of income: YouTube coined a partnership program where it allowed content creators to earn money with their viral content.

Apart from the above reasons, YouTube also partnered up with well-established market giants like CNN and NBC to set a stronger foothold.

YouTube's growth

Let's look at some interesting statistics about YouTube and its popularity.

Since its inception in February 2005, the number of YouTube users has multiplied. As of now, there are more than 2.5 billion monthly active users of YouTube. Let's take a look at the chart below to see how YouTube users have increased in the last decade.

[Increase in monthly active users of YouTube per year](./growth.jpg)

YouTube is the second most-streamed website after Netflix. A total of 694,000 hours of video content is streamed on YouTube per minute!

'''

Source: <https://www.statista.com/statistics/195140/new-user-generated-content-uploaded-by-users-per-minute/>

'''

Now that we realize how successful YouTube is, let's understand how we can design it.

How will we design YouTube?

We've divided the design of YouTube into five lessons:

1. Requirements: This is where we identify the functional and non-functional requirements. We also estimate the resources required to serve millions of users each day. This lesson answers questions like how much storage space YouTube will need to store 500 hours of video content uploaded to YouTube per day.

2. Design: In this lesson, we explain how we'll design the YouTube service. We also briefly

explain the API design and database schemas. Lastly, we will also briefly go over how YouTube's search works.

3. Evaluation: This lesson explains how YouTube is able to fulfill all the requirements through the proposed design. It also looks at how scaling in the future can affect the system and what solutions are required to deal with scaling problems.

4. Reality is more complicated: During this lesson, we'll explore different techniques that YouTube employs to deliver content effectively to the client and avoid network congestions.

5. Quiz: We reinforce major concepts we learned designing YouTube by considering how we could design Netflix's system.

Our discussion on the usage of various building blocks in the design will be limited since we've already explored them in detail in the building blocks chapter.

Design Youtube/The Reality Is More Complicated/README.md

The Reality Is More Complicated

Introduction

Now that we've understood the design well, let's see how YouTube can optimize the usage of storage and network demands while maintaining good quality of experience (QoE) for the end user.

'''

QoE is the measure of a customer's satisfaction with the experience of a particular service.
'''

When talking about providing effective service to end users, the following three steps are important:

- Encode: The raw videos uploaded to YouTube have significant storage requirements. It's possible to use various encoding schemes to reduce the size of these raw video files. Apart from compression, the choice of encoding scheme will also depend on the types of end devices used to stream the video content. Since multiple devices could be used to stream the same video, we may have to encode the same video using different encoding schemes resulting in one raw video file being converted into multiple files each encoded differently. This strategy will result in a good user-perceived experience because of two reasons: users will save bandwidth because the video file will be encoded and compressed to some limit, and the encoded video file will be appropriate for the client for a smooth playback experience.

- Deploy: For low latency, content must be intelligently deployed so that it is closer to a large number of end users. Not only will this reduce latency, but it will also put less burden on the networks as well as YouTube's core servers.

- Deliver: Delivering to the client requires knowledge about the client or device used for playing the video. This knowledge will help in adapting to the client and network conditions. Therefore, we'll enable ourselves to serve content efficiently.

Let's understand each phase in detail now.

Encode

Until now, we've considered encoding one video with different encoding schemes. However, if we encode videos on a per-shot basis, we'll divide the video into smaller time frames and encode them individually. We can divide videos into shorter time frames and refer to them as segments. Each segment will be encoded using multiple encoding schemes to generate different files called chunks. The choice of encoding scheme for a segment will be based on the detail within the segment to get optimized quality with lesser storage requirements. Eventually, each shot will be encoded into multiple chunk sizes depending on the segment's content and encoding scheme used. As we divide the raw video into segments, we'll see its advantages during the deployment and delivery phase.

Let's understand how the per-segment encoding will work. For any video with dynamic colors and high depth, we'll encode it differently from a video with fewer colors. This means that a non-so-dynamic segment will be encoded such that it's compressed more to save additional storage space. Eventually, we'll have to transfer smaller file sizes and save bandwidth during the deployment and streaming phases.

[Higher quality requiring higher bitrate from left to right](./bitrate.jpg)

Using the strategy above, we'll have to encode individual shots of a video in various formats. However, the alternative to this would be storing an entire video (using no segmenting) after encoding it in various formats. If we encode on a per-shot basis, we would be able to optimally reduce the size of the entire video by doing the encoding on a granular level. We can also encode audio in various formats to optimally allow streaming for various clients like TVs, mobile phones, and desktop machines. Specifically, for services like Netflix, audio encoding is more useful because audios are offered in various languages.

[A raw video file being encoded into different formats including its audio and subtitles](./encoding.jpg)

Deploy

As discussed in our design and evaluation sections, we have to bring the content closer to the user. This has three main advantages:

1. Users will be able to stream videos quickly.
2. There will be a reduced burden on the origin servers.
3. Internet service providers (ISPs) will have spare bandwidth.

So, instead of streaming from our data centers directly, we can deploy chunks of popular videos in CDNs and point of presence (PoPs) of ISPs. In places where there is no collaboration possible with the ISPs, our content can be placed in internet exchange point (IXPs). We can place content in IXPs that will not only be closer to users, but can also be helpful in filling the cache of ISP PoPs.

'''

PoP is a point where two or more large networks combine to enable communication between their clients

'''

[Streaming from data center to users through IXP and ISPs](./streaming.jpg)

We should keep in mind that the caching at the ISP or IXP is performed only for the popular content or moderately popular content because of limited storage capacity. Since our per-shot encoding scheme saves storage space, we'll be able to serve out more content using the cache infrastructure closer to end users.

Additionally, we can have two types of storage at the origin servers:

1. Flash servers: These servers hold popular and moderately popular content. They are optimized for low-latency delivery.
2. Storage servers: This type holds a large portion of videos that are not popular. These servers are optimized to hold large storage.

'''

Origin servers: The servers responsible for listening and responding to clients' requests

'''

'''

Note: When we transfer streaming content, it can result in the congestion of networks. That is why we have to transfer the content to ISPs in off-peak hours.

'''

Recommendations

Recommendations

YouTube recommends videos to users based on their profile, taking into account factors such as their interests, view and search history, subscribed channels, related topics to already viewed content, and activities on content such as comments and likes.

An approximation of the recommendation engine of YouTube is provided below. YouTube filters videos in two phases:

1. Candidate generation: During this phase, millions of YouTube videos are filtered down to hundreds based on the user's history and current context.
2. Ranking: The ranking phase rates videos based on their features and according to the user's interests and history. Hundreds of videos are filtered and ranked down to a few dozen videos during this phase.

YouTube employs machine learning technology in both phases to provide recommendations to users

'''

Covington, Paul, Jay Adams, and Emre Sargin. "Deep neural networks for youtube recommendations." Proceedings of the 10th ACM conference on recommender systems. 2016.

'''

[YouTube's recommendation engine using two neural networks to filter videos from millions to dozens](./ml.jpg)

'''

Question 1

Previously, we said that popular content is sent to ISPs, IXPs, and CDNs. We've now discussed YouTube's feature that recommends content. What is the difference between popular and recommended content on YouTube?

Answer

Recommendations are specific to users' profiles and interests, whereas popular content is recognized on a regional or global basis. It is possible to present popular content to the general audience.

'''

'''

Question 2

Can you provide some formulaic representation of how the YouTube algorithm for popular content would work?

Answer

To approximate a formula, we can assume a threshold value which is the weighted sum of all the considered factors like this:

$$Th_{reg} = Comments_{wt} \cdot Comments_{num} + Likes_{wt} \cdot Likes_{num} + Links_{wt} \cdot Links_{num} \hat{=} fl$$

Where

Th_{reg} is the number that will be used to decide whether a video is viral in a specific region.

$Comments_{wt}$ is the weight of the comments.

$Comments_{num}$ is the number of comments.

$Likes_{wt}$ is the weight of likes.

$Likes_{num}$ is the number of likes.

$Links_{wt}$ is the weight of the links.

$Links_{num}$ is the number of links.

The cumulative sum of each weight is equal to 1.

We may also assume that different threshold levels can be considered for different regions. Finally, we can consider a video to be globally popular if the sum of thresholds of different regions passes a global threshold value. YouTube may also decide to put some videos directly in CDN depending on the channel used to upload the videos.

'''

'''

Question 3

In reference to Question 2, how often do you think that the approximation calculation for whether content is popular will be made? On each click, like, or comment?

Answer

Calculation per click, like, or comment requires special infrastructure to perform calculation correctly and in real-time. This should be limited to:

The most popular channels.

Alternatively, a particular metric that triggers the computation every time it crosses a certain value. A good trigger could be an increasing number of requests made for a specific video in a shorter period of time.

'''

Deliver

Let's see how the end user gets the content on their device. Since we have the chunks of video already deployed near users, we redirect users to the nearest available chunks. As shown below whenever a user requests content that YouTube has recognized as popular, YouTube redirects the user to the nearest CDN.

However, in the case of non-popular content, the user is served from colocation sites or YouTube's data center where the content is stored initially. We have already learned how YouTube can reduce latency times by having distributed caches at different design layers.

[Deliver](./deliver)

Adaptive streaming

While the content is being served, the bandwidth of the user is also being monitored at all times. Since the video is divided into chunks of different qualities, each of the same time frame, the chunks are provided to clients based on changing network conditions.

As shown below, when the bandwidth is high, a higher quality chunk is sent to the client and vice versa.

[Chunk size in each time frame provided to the client changes according to the bandwidth](./adaptive_streaming.jpg)

The adaptive bitrate algorithm depends on the following four parameters:

End-to-end available bandwidth (from a CDN/servers to a specific client).

The device capabilities of the user.

Encoding techniques used.

The buffer space at the client [source: The Networking Channel: Netflix adaptive streaming and more by Renata and TY].

[Parameters affecting adaptive bitrate algorithm](./adaptive_streaming2.jpg)

Potential follow-up questions

There can be many different aspects of the system design of YouTube as there is more depth in the subject area. Therefore, many questions can arise. Some questions and directions toward their answers are as follows:

1. Question: It is possible to quantify availability by adding numbers like 99.99% availability or 99.999% availability. In that case, what design changes will be required?

This is a hard question. In reality, such numbers are part of a service's SLA, and they are generated from models or long-term empirical studies. While it is good to know how the above numbers are obtained and how organizations use monitoring to keep availability high, it might be a better strategy to discuss the fault tolerance of the system—what will happen if there are software faults, server failures, full data center failures, and so on. If the system is resilient against faults, that implies the system will have good availability.

2. Question: We assumed some reasonable numbers to come up with broad resource requirements. For example, we said the average length of a video is five minutes. In this case, are we designing for average behavior? What will happen to users who don't follow the average profile?

A possible answer could be that the above number will likely change over time. Our system design should be horizontally scalable so that with increasing users, the system keeps functioning adequately. Practically, systems might not scale when some aspect of the system increases by an order of magnitude. When some aspects of a system increase by an order of magnitude (for example, 10x), we usually need a new, different design. Cost points of designing at 10x and 100x scales are very different.

3. Question: Why didn't we discuss and estimate resources for video comments and likes?

Concurrent users' comments on videos are roughly at the same complexity as designing a messaging system. We'll discuss that problem in System Design: WhatsApp.

4. Question: How to manage unexpected spikes in system load?

A possible answer is that because our design is horizontally scalable, we can shed some load onto the public cloud due to its elasticity features. However, public clouds are not infinitely scalable. They often need a priori business contracts that might put a limit on maximum, concurrently allowed resource use at different data centers.

5. Question: How will we deploy a global network to connect data centers and CDN sites?

In practice, YouTube uses Google's network, which is built for that purpose and peers with many ISPs of the world. It is a fascinating field of study that we have left outside this course for further review.

6. Question: Why isn't there more detail on audio/video encoding?

There are many audio/video encoding choices, many publicly known and some proprietary. Due to excessive redundancy in multimedia content, encoding is often able to reduce a huge raw format content to a much smaller size (for example, from 600 MB to 30 MB). We have left the details of such encoding algorithms to you if you're interested in further exploration.

7. Question: Can't we use specialized hardware (or accelerators like GPUs) to speed up some aspects of the YouTube computation?

When we estimated the number of servers, we assumed that any server could fulfill any required functionality. In reality, with the slowing of Moore's law, we have special-purpose hardware available (for example, hardware encoders/decoders, machine-learning accelerators like Tensor Processing Units, and many more). All such platforms need their own courses to do justice to the content. So, we avoided that discussion in this design problem.

'''

Moore's law was presented by Gordon Moore in 1965, who stated that the number of transistors on an integrated circuit (IC) doubles every year.

'''

8. Question: Should compression be performed at the client-side or the server-side during the content uploading stage?

We might use some lossless but fast compression (for example, Google Snappy) on the client end to reduce data that needs to be uploaded. This might mean that weâll need a rich client, or would have to fall back to plain data if the compressor was unavailable. Both of those options add complexity to the system.

9. Question: Are there any other benefits to making file chunks other than in adaptive bitrates?

We discussed video file chunks in the context of adaptive bit rates only. Such chunks also help to parallelize any preprocessing, which is important to meet real-time requirements, especially for live streams. Parallel processing is again a full-fledged topic in itself, and weâve left it to you for further exploration.

Design a Proximity Service Yelp/Design Considerations of Yelp/README.md

Design Considerations of Yelp

Introduction

We discussed the design, building blocks, components, and the entire workflow of our system in the previous lesson, which brought up a number of interesting questions that now need answering. For example, what approach do we use to find the places, or how do we rank places based on their popularity? This lesson addresses important design concerns like the ones we just mentioned. The table given below summarizes the goals of this lesson.

'''

Summary of the Lesson

Section/Sub-section

Purpose

Searching
optimize the search, so that all nearby sites in a given location and radius can be identified

Storing Indexes
We index the places to improve query performance and estimate the storage required for all of the indexes.

Searching Using Segments
We search all the desired places by combining the segments.

Dynamic Segments
We solve the static segment limitations using dynamic segments.

Searching Using a QuadTree
We explore the searching functionality using a QuadTree.

Space Estimations for a QuadTree
We estimate the storage required for a QuadTree.

Data Partitioning
We look into the options we can use to partition data.

Ensuring Availability
system.
We look into how weâll ensure the availability of the

Inserting a New Place
We look into how weâll insert the place in a QuadTree

Ranking Popular Places
We rank the places of the system.

Evaluation
requirements.
We evaluate how our system fulfills the non-functional

'''

Searching

From Google Maps, we were able to connect segments and meet the scalability challenge to process a large graph efficiently. The graph of the world had numerous nodes and vertices, and traversing them was time-consuming. Therefore, we divided the whole world into smaller segments/subgraphs to process and query them simultaneously. The segmentation helps us improve the scalability of the system.

Each segment will be of the size 5^5 miles and will contain a list of places that exist within it. We only search a few segments to locate destinations that are close by. We can use a given location and defined radius to find all the nearby segments and identify sites that are close.

[Partitioning the globe into small segments, where each segment has four coordinates (lat, long) and all segments hold the coordinates of different places](./segment.jpg)

'''

Question 1

How will we find nearby places if we create a table for storing all places?

Answer

We can make a table of places that have Place_ID as a unique ID and store each place in it. The longitude and latitude columns help us specify the exact location. Indexing both of these columns can help us fetch data efficiently. We can specify each location as a pair of latitude (M) and longitude (N). We can also search for a place within a given radius of R by finding all the places between the latitude M-R and M+R and the longitude N-R and N+R. We can apply Dijkstra's algorithm to find the distance between two points.

'''

'''

Question 2

How efficient will our searching be when based on the table?

Answer

We can have multiple lists of places within (M-R, M+R) and (N-R, N+R). It will be a challenge to handle concurrent requests. When queries that are used to fetch the places will be coming at a high rate for different segments, the response time and performance will be affected.

'''

We can store all the places in a table and uniquely identify a segment by having a segment_ID. We can index each segment in the database. Now, we have limited the number of segments we need to search, so the query will be optimized and return results quickly.

Improve data fetching

We use a key-value store for quick access to places in the segments. The key is the segment_ID while the value contains the list of places in that segment. Let's estimate how much space we need to store the indexes.

'''

We can usually store a few MBs in the value of the key-value store. If we assume that each segment has 500 places then it will take up $500 \times 1296 \text{ Bytes} = 0.648 \text{ MB}$, and we can easily store it as a value.

'''

The total area of the earth is around 200 million square miles, and the land surface area is about 60 million square miles. If we consider our radius to be ten miles, then we will have 60 million segments. An 8-Byte identifier will identify each segment.

Let's calculate how much memory we need.

'''

Total Area of the Earth (Million Square Miles)	60
Search Radius (Miles)	10

Number of Segments (Millions)	6
Segment_ID (Bytes)	8
Place_ID (Bytes)	8
Number of Places (Millions)	500
Total Space (TB)	4.048

'''

Search using segments

A user may select a radius for searching places that aren't present in a single segment. So, need to combine multiple segments by connecting the segments to find locations within the specified radius—say, five miles.

First, we constrain the number of segments. This reduces the graph size and makes the searching process optimizable. Then, we identify all the relevant locations, compute the distance from the searching point, and show it to the user.

[Searching for locations using segments](./searching.jpg)

'''

Question

Can you identify a problem with the current approach?

Answer

Our locations are not evenly distributed across segments, and this approach may still take a long time to run on segments with a lot of places. For example, in a city like New York, we can have lots of places even within a small radius because it is a densely-populated area. And for less populated areas, the same radius might not have enough places, and we might need to expand our radius to find more.

'''

Dynamic segments

We solve the problem of uneven distribution of places in a segment by dynamically sizing the segments. We do this by focusing on the number of places. We split a segment into four more segments if the number of places reaches a certain limit. We assume 500 places as our limit. While using this approach, we need to decide on the following questions:

How will we map the segments?

How will we connect to other segments?

We use a QuadTree to manage our segments. Each node contains the information of a segment. If the number of places exceeds 500, then we split that segment into four more child nodes and divide the places between them. In this way, the leaf nodes will be those segments that can't be broken down any further. Each leaf node will have a list of places in it too.

'''

A quadtree is a tree data structure in which each internal node has exactly four children. Quadrees are the two-dimensional analog of octrees and are most often used to partition a two-dimensional space by recursively subdividing it into four quadrants or regions. The data associated with a leaf cell varies by application, but the leaf cell represents a unit of interesting spatial information. Source: Wikipedia

'''

Search using a QuadTree

We start searching from the root node and continue to visit the nodes to find our desired segment. We check every node to see if it has more child nodes. If a node has no more children then we stop our search because that node is the required one. We also connect each child node with its neighboring nodes with a doubly-linked list. All the child nodes of all the parents nodes are connected through the doubly-linked list. This list allows us to find the neighboring segments when we can move forward and backward as per our requirement. After identifying the segments, we have the required PlaceID values of the places and we can search our database to find more details on them.

****Question****

Is there an alternative approach to find the neighboring segments?

****Answer****

We can use the pointers of parent nodes to find the neighboring segments. In each node, we can keep a pointer that points towards the parent node. Every parent node has pointers to its children nodes, so we can use those to find the adjacent leaf nodes. We can extend our search by going up through the parent pointers.

In the following illustration, node F can find its neighboring node, G, by first going to the parent node, B, and then to G.

[Tree](./traverse.jpg)

The following slides show how the process of searching for a place works. If a node has the places we need, we stop there. Otherwise, we explore more nodes until we reach our search radius. After finding the node, we query the database for information related to the places and return the desired ones.

The following slides show how the process of searching for a place works. If a node has the places we need, we stop there. Otherwise, we explore more nodes until we reach our search radius. After finding the node, we query the database for information related to the places and return the desired ones.

[steps](./quadtree)

Storage space estimation for QuadTrees

Let's calculate the storage we need for keeping QuadTrees:

- We store the PlaceID, Latitude, and Longitude for each place in the node. Each of these values is of the size 8 Bytes. For 500 million places, we need the following amount of storage for all places:

- $(8+8+8) \times 10^6 \times 500 = 12\text{GB}$
- We'll have 500 places in a single node. So, for 500 million places, we'll need $500 \times 10^6 / 500 = 10^6$ leaf nodes.

The QuadTree with 1 million leaf nodes has approximately 1/3rd of the leaf nodes of the internal nodes (all the nodes up the leaf level), and each internal node has four pointers to its child nodes. If we'll assume 8 Bytes for each pointer, then we'll need $1\text{M} \times 1/3 \times 4 \times 8 = 10.67\text{ MB}$ of space for the internal nodes.

'''

Source: https://tildesites.bowdoin.edu/~ltoma/teaching/cs3225-GIS/fall16/Lectures/gis_qdt1.pdf

'''

We can get the total space needed to store all the internal nodes by adding 12GB and 10.67MB, that is, 12.01GB approximately.

We can easily store a QuadTree on a server.

Let's try the following calculator to calculate the storage needed for a QuadTree:

'''

PlaceID, Latitude, and Longitude (Bytes)	24
Total Number of Places (Millions)	500
Total Space Needed to Store All Places (GB)	12
Limit of Places for Each Segment	500
Number of Segments (Millions)	1

Number of Pointers to Hold Children Pointers	4
Size of Each Pointer (Bytes)	8
Total Space to Store All Internal Nodes (MB)	10.67
Total Space to Store QuadTrees (GB)	12.01

'''

Data partitioning

Keeping 20% growth per year in mind, the number of places will increase. We can partition data on the following basis:

1. Regions: We can split our places into regions on the basis of zip codes. This way, all the places that belong to a specific region are stored on a single node. We store information on the region along with the place, so that we can query on the basis of regions too. We can use the user's region to find the places in that specific region.

This data partitioning comes with a few challenges. For example, if a region becomes popular during tourist season, it can affect the performance of our system. We might have numerous queries on the server that might result in slow responsiveness to user queries.

2. PlaceID: We can partition data on the basis of PlaceID instead of the region to avoid the query overload in popular seasons or rush hours. We can use a key-value store to store the places. In this case, the key is the PlaceID and the value contains the server in which that place is stored. This will make the process of fetching places more efficient.

So, we'll opt for partitioning on the basis of places. Moreover, we'll also use caches for popular places. The cache will have information about that particular place.

Ensure availability

Consider a scenario where multiple people in the same radius place a search request. If we have a single QuadTree, it'll affect the availability of the users. So, we can't rely on a single QuadTree. To cater to this problem, we replicate our QuadTrees on multiple servers to ensure availability. This allows us to distribute the read traffic and decrease the response time. QuadTrees are built on a server, so we can use the server ID as a key to identify the server on which the QuadTree is present. The value is the list of places that the QuadTree holds. The key-value store eases the rebuilding of the QuadTree in case we lose it.

'''

Question

How will the leader-follower approach help us in replication?

Answer

We'll have a single leader and two followers. The leader will have the QuadTree and it'll handle all the write requests. It'll update the followers about any change made to the QuadTrees synchronously, so there'll be a chance of delay. The followers will handle all the read requests. In case a follower dies, we'll choose another follower and replicate the data to it. If the leader is down, we'll choose any of the followers to step in as the leader. This way, we'll be able to replicate our QuadTrees onto three servers.

'''

Insert a new place

We insert a new place into the database as well as in our QuadTree. We find the segment of the new place if the Quadtree is distributed on different servers, and then we add it to that segment. We split the segment if required and update the QuadTree accordingly.

Rank popular places

We need a service, a rating calculator, which calculates the overall rating of a service. We can store the rating of a place in the database and also in the QuadTree, along with the ID, latitude, and longitude of the place. The QuadTree returns the top 50 or 100 popular places within the given radius. The aggregator service determines the actual top places and returns them to the user.

'''

Note: We don't expect the rating to be updated within hours, as such frequent changes in QuadTrees or databases can be expensive. We update them once a day, when the load is minimal.

Finalized design

We added a few new components to our design. We introduced caches to store popular places. This allows us to fetch the places much faster. Moreover, we also added a rating calculator that sorts the places based on their ratings. This will enhance user experience, since the places with a good rating will be displayed first.

The updated design of our system is shown below:

[Updated Yelp design](./updated_design.jpg)

Evaluation

Let's see how our system design fulfills our requirements.

- Availability: We partitioned the data into smaller segments instead of having to deal with a huge dataset consisting of all the places on the world map. This made our system highly available. We also replicated the QuadTrees data using key-value stores to ensure availability.
- Scalability: We split the whole world into smaller dynamic segments. This allows us to search for a place within a specific radius and shorten our search area. We then used QuadTrees in which each child node holds a single segment. Upon adding or removing a place, we can restructure our QuadTrees. So, we were able to make our system scalable.
- Performance: We reduced the latency by using caches. We cached all the famous and popular places, so request time was minimized.
- Consistency: The users have a consistent view of the data regarding places, reviews, and photos because we used reliable and fault-tolerant databases like key-value stores and relational databases.

Summary

The proximity-based servers allow the user to search for a specific place or places nearby. The map data of the world is huge and dividing it into segments and finding the specific segment was a challenge in itself. So, we used QuadTrees to optimize our search and provided the user with a list of places with minimum latency.

Design a Proximity Service Yelp/Design of Yelp/README.md

Design of Yelp

We identified the requirements and calculated the estimations for our Yelp system in the previous lesson. In this lesson, we discuss the API design, go through the storage schema, and then dive into the details of the system's building blocks and additional components.

API design

Let's discuss the API design for Yelp.

Search

We need to implement the search function. The API call for searching based on categories like "cafes" will be:

```
search(category, user_location, radius)
```

'''

Parameter	Description
category	This is the type of search the user makes for example, a search for restaurants, cinemas, cafes, and so on.
user_location	This contains the location of the user who's searching with Yelp.
radius	This is the specified radius where the user is trying to find the required

category.

'''

This process returns a JSON object that contains a list of all the possible items in the specified category that also fall within the specified radius. Each entry has a place name, address, category, rating, and thumbnail.

The API call for searching based on the name of a place like âBurger Hutâ will be:

'''

search(name_of_place, user_location, radius)

'''

'''

Parameter	Description
-----------	-------------

name_of_place	This contains the name of the place that the user wants to search for.
---------------	--

'''

This process returns a JSON object that contains information of the specified place.

Add a place

The API call for adding a place is below:

'''

add_place(name_of_place, description_of_place, category, latitude, longitude, photo)

'''

'''

Parameter	Description
-----------	-------------

name_of_place	This contains the name of the place, for example, "Burger Hut".
---------------	---

description_of_place	This contains a description of the place. For example, "Burger Hut sells the yummiest burgers".
----------------------	---

category	This specifies the category of the placeâ for example, "cafe".
----------	--

latitude	This tells us the latitude of the place.
----------	--

longitude	This tells us the longitude of the place.
-----------	---

photo photos.	This contains photos of the place. There can be a single or multiple
------------------	--

'''

This process returns a response saying that a place has been added, or an appropriate error if it fails to add a place.

Add a review

Add a review

The API call for adding a place is below:

'''

add_review(place_ID, user_ID, review_description, rating)

'''

'''

Parameter	Description
-----------	-------------

place_ID	This contains the ID of the place whose review is added.
----------	--

user_ID	This contains the ID of the user who adds the review.
---------	---

review_description	This contains the review of the placeâ for example, "the food and
--------------------	---

ambiance were superb".

rating This contains the rating of the placeâ for example, 4 out of 5.
'''

This process returns a response that a review has been added, or an appropriate error if it fails to add a review.

Storage schema

Letâs define the storage schema for our system. A few of the tables we might need are â Placeâ
â Photos,â â Reviews,â and â Users.â

Letâs define the columns of the â Placeâ table:

- Place_ID: We use the sequencer to generate an 8 Bytes (64 bits) unique ID for a place.
'''

Note: We generate IDs using the unique ID generator.
'''

- Name_of_Place: This is a string that contains the name of the place. We use 256 Bytes for it.

- Description_of_Place: This holds a description of the place. We use 1,000 Bytes for it.

- Category: This specifies the type of place like restaurants, cinemas, bookshops, and so on (8 Bytes).

- Latitude: This stores the latitude of the location (8 Bytes).

- Longitude: This stores the longitude of the location (8 Bytes).

- Photos: This contains the foreign key (8 Bytes) of another table â Photos,â which contains the photos related to a particular place.

- Rating: This stores the rating of the place. It shows how many stars a place gets out of five. The rating is calculated based on the reviews it gets from the users.

The columns mentioned above are the most important ones in the table. We can add more columns like â menu,â â address,â â opening and closing hours,â and so on. Therefore, keeping only the essential columns, the size of one row of our table will be:

'''
Size = 8 + 256 + 1000 + 8 + 8 + 8 + 8 = 1296 bytes
'''

Now letâs define the â Photosâ table:

- Photo_ID: We use the sequencer to generate a unique ID for a photo (8 Bytes or 64 bits).

- Place_ID: We use the foreign key (8 Bytes) from the â Placeâ table to identify which photo belongs to which place.

- Photo_path: We store the photos in blob storage and save the photoâs path (256 Bytes) in the column.
'''

Size = 8 + 8 + 8 + 256 = 280 bytes
'''

We need another table called â Reviewsâ to store the reviews, ratings, and photos of a place.

- Review_ID: We use the sequencer to generate a unique ID of 8 Bytes (64 bits) for a review.

- Place_ID: The foreign key (8 Bytes) from the â Placeâ table to determine which place the rating belongs to.

- User_ID: The foreign key (8 Bytes) from the â Usersâ table to identify which review belongs to.

which user.

- Review_description: This holds a description of the review. We use 512 Bytes for it.

- Rating: This stores how many stars a place gets out of five (1 Byte).

'''

Size = 8 + 8 + 8 + 512 + 1 = 537 bytes

'''

We use the `Users` table to store user information.

- User_ID: We use the sequencer to generate a unique ID for a user (8 Bytes).

- User_name: This is a string that contains the user's name. We use 256 Bytes for it.

'''

Size = 8 + 256 = 264 bytes

'''

'''

Note: The INT in the following schema contains an 8-Byte ID that we generate using the unique ID generator.

'''

[Storage schema](./schema.jpg)

Design

Now we'll discuss the individual building blocks and components used in the design of Yelp and how they work together to complete various functional requirements.

Components

These are the components of our system:

- Segments producer: This component is responsible for communicating with the third-party world map data services (for example, Google Maps). It takes up that data and divides the world into smaller regions called segments. The segment producer helps us narrow down the number of places to be searched.

- QuadTree servers: These are a set of servers that have trees that contain the places in the segments. A QuadTree server finds a list of places based on the given radius and the user's provided location and returns that list to the user. This component mainly aids the search functionality.

- Aggregators: The QuadTrees accumulate all the places and send them to the aggregators. Then, the aggregators aggregate the results and return the search result to the user.

- Read servers: We use a set of read servers that we use to handle all the read requests. Since we have more read requests, it's efficient to separate these requests from the write requests. Each read server directs the search requests to the QuadTree's servers and returns the result to the user.

- Write server: We use a set of write servers to handle all the write requests. Each write server handles the write requests of the user and updates the storage accordingly. Examples for write requests include adding a place, writing a comment, rating a place, and so on.

- Storage: We'll use two types of storage to fulfill our diverse needs.

- SQL database: Our system will have different tables like `Users`, `Place`, `Reviews`, `Photos`, and others as described below. The data in these tables is inherently relational structured. We need to perform queries like places a user visited, reviews they added, or view all the reviews of a specific place. It's easy to perform such queries in a SQL-based database. We also want all users to have a consistent view of the data, and SQL-based databases are better suited for such use cases. We'll use reliable and scalable databases, as is discussed in the Database building block.

- Key-value stores: Weâ ll need to fetch the places in a segment efficiently. For that, we store the list of places against a segment ID in a key-value store to minimize searching time. We also save the QuadTree information in the key-value store, by storing the QuadTree data against a unique ID.

- Load balancer: A load balancer distributes usersâ incoming requests to all the servers uniformly.

[Yelp design](./design.jpg)

Workflow

The user puts in a search request. We find all the relevant places in the given radius, while considering the userâ s location (latitude, longitude).

We explain the detailed workflow of our system in terms of the required functionalities below:

Searching a place: The load balancers route read requests to the read servers upon receiving them. The read servers direct them to the QuadTree servers to find all the places that fall within the given radius. The QuadTree servers then send the results to the aggregators to refine them and send them to the user.

Adding a place or feedback: The load balancers route the write requests to the write servers upon receiving them. Depending on the provided content, meaning the place information or review, the write servers add an entry in the relational database and put all the related images in the blob storage.

Making segments: The segmentâ s producer splits the world map taken from the third-party map service into smaller segments. The places inside each segment are stored in a key-value store. Even though this is a one-time job, this process is repeated periodically for newer segments and places. Since the probability of new places being added is low, we update our segments every month.

Weâ ve discussed the design of Yelp, its API design, and the relevant storage schema. In the next lesson, weâ ll talk about the design considerations.

Design a Proximity Service Yelp/Quiz on Yelp's Design/README.md

Quiz on Yelp's Design

'''

1^a (Select all that apply.) Which requirement will not be fulfilled if we donâ t constrain the number of segments to reduce the graph size?

A)
Availability

B)
Consistency

C)
Scalability

D)
Reliability

'''

'''

2 (True or False) The type of partitioning (region- or PlaceID-based) affects the structure of QuadTrees.

A)
True

B)
False
'''

'''

3 In which scenario is there a greater chance of delay in response time if we use the leader-follower approach to help us in replication?

A)
When the leader updates the followers asynchronously

B)
When the leader updates the followers synchronously

C)
When the leader saves the updates of a day and sends the changes to followers once a day
'''

'''

4 (Select all that apply.) What will happen if the leader and followers both die simultaneously?

A)
We would have to rebuild the QuadTrees.

B)
The performance will be affected.

C)
The availability will be affected.
'''

Answers

'''

1 (Select all that apply.) Which requirement will not be fulfilled if we don't constrain the number of segments to reduce the graph size?

Selected Option

A)
Availability

Explanation

The query will return the results slower affecting the user response time.

B)
Consistency

Selected Option

C)
Scalability

Explanation

It won't be easy to modify a humongous graph.

D)

Reliability

'''

'''

2 (True or False) The type of partitioning (region- or PlaceID-based) affects the structure of QuadTrees.

Correct Answer

A)

True

Explanation

Yes, it does as we have no assurance that each segment will have the same number of places in it. Though this won't be an issue as we will be finding our neighboring segments within the defined radius.

B)

False

'''

'''

3 In which scenario is there a greater chance of delay in response time if we use the leader-follower approach to help us in replication?

A)

When the leader updates the followers asynchronously

Explanation

The leader will not wait until all followers have updated the data, it'll update the data in its storage and then send the update to followers.

Correct Answer

B)

When the leader updates the followers synchronously

Explanation

The leader waits until all followers have updated the data.

C)

When the leader saves the updates of a day and sends the changes to followers once a day

Explanation

The leader will not ask the followers to update the data at once so there won't be a delay.

'''

'''

4 (Select all that apply.) What will happen if the leader and followers both die simultaneously?

Selected Option

A)

We would have to rebuild the QuadTrees.

Explanation

We would have to rebuild QuadTrees from the key-value stores where we kept them.

Selected Option

B)

The performance will be affected.

Explanation

We would not be able to serve any user query which will affect the performance.

Selected Option

C)

The availability will be affected.

Explanation

Both leader and followers will be down so they will be unable to process the requests.

'''

Design a Proximity Service Yelp/Requirements of Yelp's Design/README.md

Requirements of Yelp's Design

Requirements

Let's identify the requirements of our system.

Functional requirements

The functional requirements of our systems are below:

- User accounts: Users will have accounts where they're able to perform different functionalities like log in, log out, add, delete, and update places' information.

'''

Note: There can be two types of users: business owners who can add their places on the platform and other users who can search, view, and give a rating to a place.

'''

- Search: The users should be able to search for nearby places or places of interest based on their GPS location (longitude, latitude) and/or the name of a place.

- Feedback: The users should be able to add a review about a place. The review can consist of images, text, and a rating.

[Functional requirements](./fr.jpg)

Non-functional requirements

The non-functional requirements of our systems are:

- High availability: The system should be highly available to the users.

- Scalability: The system should be able to scale up and down, depending on the number of requests. The number of requests can vary depending on the time and number of days. For example, there are usually more searches made at lunchtime than at midnight. Similarly, during tourist season, our system will receive more requests as compared to in other months of the year.

- Consistency: The system should be consistent for the users. All the users should have a consistent view of the data regarding places, reviews, and images.

- Performance: Upon searching, the system should respond with suggestions with minimal latency.

Resource estimation

Let's assume that we have:

- A total of 178 million unique users.
- 60 million daily active users.
- 500 million places.

Number of servers estimation

We need to handle concurrent requests coming from 60 million daily active users. As we did in our discussion in the back-of-the-envelope lessons, we assume an RPS of 8,000.

Number of daily active users/RPS of a server= 60 million / 8000=7500 servers

'''

Estimating the Number of Servers

Number of Daily Active Users (in Millions)	60
RPS of a Server	8000
Number of Servers Required	7500

'''

[The number of servers required for Yelp](./servers.jpg)

Storage estimation

Let's calculate the storage we need for our data. Let's make the following assumptions:

We have a total of 500 million places.

For each place, we need 1,296 Bytes of storage.

- We have one photo attached to each place, so we have 500 million photos.
- For each photo, we need 280 Bytes of storage. Here, we consider the row size of the photo entity in the table, which contains a link to the actual photo in the blob store.
- At least 1 million reviews of different places are added daily.
- For each review, we need 537 Bytes of storage.
- We have a total of 178 million users.
- For each user, we need 264 Bytes of storage.

'''

Note: The Bytes used for each place, photo, review, and user are based on the database schema that we'll discuss in the next lesson.

'''

The following calculator computes the total storage we need:

'''

Estimating Storage Requirements

Type of information	Size	Required by an Entity (in Bytes)	Count (in
Millions)	Total Size (in GBs)		
Place		1296	500
648			
Photo		280	500
140			
Review		537	1
0.54			
User		264	178
46.99			
Total Storage Required			
835.53			

'''

[The total amount of storage required by Yelp](./storage.jpg)

Bandwidth estimation

To estimate the bandwidth requirements for Yelp, we categorize the bandwidth calculation of incoming and outgoing traffic.

For incoming traffic, let's assume the following:

- On average, five places are added every day.
- For each place, we take up 1,296 Bytes.
- A photo of size 3 MB is also attached with each place. This is the size of the photo that we save in the blob store.
- One million reviews of different places are added every day.
- Each review, takes up 537 Bytes.

We divide the total size of information per day by 86,400 to convert it into per second bandwidth.

'''

Estimating Incoming Bandwidth Requirements

Average Number of Places Added Daily	5
Storage Needed for Each Place (Bytes)	1296
Size of Photo (in MBs)	3
Total Size of Place Information (Bytes)	15006480
Average Number of Reviews Added Daily (in Millions)	1
Storage Needed for Each Review (Bytes)	537
Total Size of Reviews (Bytes)	537000000
Total Incoming Bandwidth (KBps)	6.39
Total Incoming Bandwidth (Kbps)	51.12

'''

For outgoing traffic, let's assume the following:

- A single search returns 20 places on average.
- Each place has a single photo attached to it that has an average size of 3 MB.
- Every returned entry contains the place and photo information.

Considering that there are 60 million active daily users, we come to the following estimations

'''

Estimating Outgoing Bandwidth Requirements

Average Number of Places Returned on Each Search Request	20
Size of Place (in Bytes)	1296
Size of Photo (in MB)	3
Total Size of Place Information (Bytes)	60025920
Outgoing Bandwidth Required for a Single Request (Kbps)	0.69
Outgoing Bandwidth Required for a Single Request (KBps)	5.52
Daily Active Users (in Millions)	60
Total Outgoing Bandwidth Required (Kbps)	331200000
Total Outgoing Bandwidth Required (Gbps)	331.2

'''

We need a total of approximately 51 Kbps of incoming traffic and approximately 331 Gbps of outgoing, assuming that the uploaded content is not compressed.

Total bandwidth requirements = 51 Kbps+331 Gbps ≈ 331 Gbps.

[The total bandwidth required by Yelp](./bandwidth.jpg)

Building blocks we will use

The design process of Yelp utilizes many building blocks that have already been discussed in the initial chapters of the course. We'll consider the following concepts while designing Yelp

[Building blocks in the high-level design of Yelp](./bb.jpg)

- Caching: We'll use the cache to store information about popular places.

- Load balancer: Weâ ll use the load balancer to manage the large amount of requests.
- Blob storage: Weâ ll store images in the blob storage.
- Database: Weâ ll store information about places and users in the database.

Weâ ll also rely on Google Maps to understand the feature of searching for places within a particular radius.

Design a Proximity Service Yelp/System Design Yelp/README.md

System Design: Yelp

What is Yelp?

Yelp is a one-stop platform for consumers to discover, connect, and transact with local businesses. With it, users can join a waitlist, make a reservation, schedule an appointment, or purchase goods easily. Yelp also provides information, photos, and reviews about local businesses.

The user provides the name of a place or its GPS location, and the system finds places that are nearby. The user can also upload their opinions on this platform in the form of text, pictures or ratings for a place they visited. Other location-based services include Foursquare and Google Nearby.

Services based on proximity servers are helpful in finding nearby attractions such as restaurants, theaters, or recreational sites. Designing such a system is challenging because we have to efficiently find all the possible places in a given radius with minimum latency. This means that we have to narrow down all the locations in the world, which could be in the billions, and only pinpoint the relevant ones.

[Yelp](./yelp)

How will we design Yelp?

Here is the breakdown of Yelp's design:

1^a Requirements: In this lesson, we define the requirements and estimate the required servers, storage, and bandwidth of our system.

2^a Design: In this lesson, we define the API design, the database schema, the components of our system, and the workflow of Yelp.

3^a Design considerations: In this lesson, we dive deep into the design of the Yelp system.

4^a Quiz: In this lesson, we take a quiz to test our knowledge of Yelp design.

Let's start our design by defining its requirements.

Design a URL Shortening Service TinyURL/Design and Deployment of TinyURL/README.md

Design and Deployment of TinyURL

System APIs

To expose the functionality of our service, we can use REST APIs for the following features:

- Shortening a URL
- Redirecting a short URL
- Deleting a short URL

[System API design overview](./api.jpg)

Shortening a URL

We can create new short URLs with the following definition:

'''

shortURL(api_dev_key, original_url, custom_alias=None, expiry_date=None)

'''

The API call above has the following parameters:

'''

Parameter	Description
api_dev_key	A registered user account's unique identifier. This is useful in tracking a user's activity and allows the system to control the associated services accordingly.
original_url	The original long URL that is needed to be shortened.
custom_alias	The optional key that the user defines as a customer short URL.
expiry_date	The optional expiration date for the shortened URL.

'''

A successful insertion returns the user the shortened URL. Otherwise, the system returns an appropriate error code to the user.

Redirecting a short URL

To redirect a short URL, the REST API's definition will be:

'''

redirectURL(api_dev_key, url_key)

'''

With the following parameters:

'''

Parameter	Description
api_dev_key	The registered user account's unique identifier.
url_key	The shortened URL against which we need to fetch the long URL from the database.

'''

A successful redirection lands the user to the original URL associated with the url_key.

Deleting a short URL

Similarly, to delete a short URL, the REST API's definition will be:

'''

deleteURL(api_dev_key, url_key)

'''

and the associated parameters will be:

'''

Parameter	Description
api_dev_key	The registered user account's unique identifier.
url_key	The shortened URL against which we need to fetch the long URL from the database.

'''

A successful deletion returns a system message, URL Removed, conveying the successful URL removal from the system.

Design

Let's discuss the main design components required for our URL shortening service. Our design depends on each part's functionality and progressively combines them to achieve different workflows mentioned in the functional requirements.

Components

We'll explain the inner mechanism of different components within our system, as well as their usage as a part of the whole system below. We'll also highlight the design choices made for each component to achieve the overall functionality.

Database: For services like URL shortening, there isn't a lot of data to store. However, the storage has to be horizontally scalable. The type of data we need to store includes:

- User details.

- Mappings of the URLs, that is, the long URLs that are mapped onto short URLs. Our service doesn't require user registration for the generation of a short URL, so we can skip adding certain data to our database. Additionally, the stored records will have no relationships among themselves other than linking the URL-creating user's details, so we don't need structured storage for record-keeping. Considering the reasons above and the fact that our system will be read-heavy, NoSQL is a suitable choice for storing data. In particular, MongoDB is a good choice for the following reasons:

1. It uses leader-follower protocol, making it possible to use replicas for heavy reading.
2. MongoDB ensures atomicity in concurrent write operations and avoids collisions by returning duplicate-key errors for record-duplication issues.

'''

Question

Why are NoSQL databases like Cassandra or Riak not good choices instead of MongoDB?

Answer

Since our service is more read-intensive and less write-intensive, MongoDB suits our use case the best for the following reasons:

NoSQL databases like Cassandra, Riak, and DynamoDB need read-repair during the reading stage and hence provide slower reads to write performance.

They are leader-less NoSQL databases that provide weaker atomicity upon concurrent writes. Being a single leader database, MongoDB provides a higher read throughput as we can either read from the leader replica or follower replicas. The write operations have to pass through the leader replica. It ensures our system's availability for reading-intensive tasks even in cases where the leader dies.

Since Cassandra inherently ensures availability more than MongoDB, choosing MongoDB over Cassandra might make our system look less available. However, the time taken by the leader election algorithm is negligible compared to the time elapsed between short URL generation and its first usage, so it doesn't hamper our system's availability.

'''

Short URL generator: Our short URL generator will comprise a building block and an additional component:

- A sequencer to generate unique IDs
- A Base-58 encoder to enhance the readability of the short URL

We built a sequencer in our building blocks section to generate 64-bit unique numeric IDs. However, our proposed design requires 64-bit alphanumeric short URLs in base-58. To convert the numeric (base-10) IDs to alphanumeric (base-58), we'll need a base-10 for the base-58 encoder. We'll explore the rationale behind these decisions alongside the internal working of the base-58 encoder in the next lesson.

Take a look at the diagram below to understand how the overall short URL generation unit will work.

[Internal working of a short URL generator](./internal.jpg)

Other building blocks: Beside the elements mentioned above, we'll also incorporate other building blocks like load balancers, cache, and rate limiters.

- Load balancing: We can employ Global Server Load Balancing (GSLB) apart from local load balancing to improve availability. Since we have plenty of time between a short URL being generated and subsequently accessed, we can safely assume that our DB is geographically consistent and that distributing requests globally won't cause any issues.
- Cache: For our specific read-intensive design problem, Memcached is the best choice for a

cache solution. We require a simple, horizontally scalable cache system with minimal data structure requirements. Moreover, we'll have a data-center-specific caching layer to handle native requests. Having a global caching layer will result in higher latency.

- Rate limiter: Limiting each user's quota is preferable for adding a security layer to our system. We can achieve this by uniquely identifying users through their unique `api_dev_key` and applying one of the discussed rate-limiting algorithms (see Rate Limiter from Building Blocks). Keeping in view the simplicity of our system and the requirements, the fixed window counter algorithm would serve the purpose, as we can assign a set number of shortening and redirection operations per `api_dev_key` for a specific timeframe.

'''

Question 1

How will we maintain a unique mapping if redirection requests can go to different data centers that are geographically apart? Does our design assume that our DB is consistent geographically?

Answer

We initially assumed that our data center was globally consistent. Let's look at the problem differently and consider the opposite case: we need to filter the redirection requests based on data centers.

Solution: A simple way of achieving this functionality is to introduce a unique character in the short URL. This special character will act as an indicator for the exact data center.

Example: Let's assume that the short URL that needs redirection is `service.com/x/short123/`, where `x` indicates the data center containing this record.

In this solution, if the short URL goes to the wrong data center, it can be redirected to the correct one. However, if a specific data center is not reachable for a specific short URL (and that URL is not yet cached), the redirection will fail.

'''

'''

Question 2

How will the data-center-specific caching handle an unseen redirection request?

Answer

Since we've assumed the data-center-specific caching solution for our system, a case needs highlighting: handling unseen redirection requests by our system.

Scenario: The scenario entails receiving an unknown redirection request at a data center. Since the local cache wouldn't have that entry, it would fetch that record from the globally consistent database and place this entry into the local cache for future use.

'''

'''

Question 3

What is the probability of collision when we ask the short URL generator for a new short URL?

Answer

We ask the sequencer for a unique ID and by the definition of our sequencer's design, there will never be duplication in IDs. We then encode those IDs, which also ensures no duplication. Hence, the regular short URL generation process ensures no duplication in records.

Now let's consider the case of custom short URLs. Since the user is providing the short URL, there can be a duplication. We can easily calculate the probability of this collision by taking into account the size of the database containing short URL records.

Let's assume there are n already generated short URLs in the database. The probability that user-provided custom short URL will be similar to an already existing one can be given by $1/n$.

'''

Design diagram

A simple design diagram of the URL shortening system is given below.

[A design diagram of the URL shortening service](./design_diagram.jpg)

Workflow

Let's analyze the system in-depth and how the individual pieces fit together to provide the overall functionality.

Keeping in view the functional requirements, the workflow of the abstract design above would be as follows.

1. Shortening: Each new request for short link computation gets forwarded to the short URL generator (SUG) by the application server. Upon successful generation of the short link, the system sends one copy back to the user and stores the record in the database for future use.

'''

Question 1

How does our system avoid duplicate short URL generation?

Answer

- Computing a short URL for an already existing long URL is redundant, and the system sends the long URL to the database server to check its existence in the system. The system will check the respective entry in the cache first and then query the database.

- If the short URL for the corresponding long URL is already present, the database returns the saved short URL to the application server which reroutes the requested short URL to the user.

- If the requested short URL is unavailable in the system, the application server requests the SUG to compute the short URL for the requested long URL. Once computed, the SUG sends back a copy of the requested short URL to the application server and another copy to the database server.

'''

'''

Question 2

How do we ensure that two concurrent requests for a short URL do not overwrite?

Answer

Concurrency in handling URL shortening requests is essential, and we achieve it as follows:

MongoDB ensures consistency by locking and concurrency control protocols, preventing the users from modifying the same data simultaneously.

In MongoDB, all the write requests go through the single leader and hence exclude the possibility of race conditions due to the serialization of requests via a single leader.

'''

2. Redirection: Application servers, upon receiving the redirection requests, check the storage units (caching system and database) for the required record. If found, the application server redirects the user to the associated long URL.

'''

Question

How does our system ensure that our data store will not be a bottleneck?

Hide Answer

We can ensure that our data store doesn't become a bottleneck, using the following two approaches:

1. We use a range-based sequencer in our design, which ensures basic level mapping between the

servers and the short URLs. We can redirect the request to the respective database for a quick search.

2. As discussed above, we can also have unique IDs for various data stores and integrate them into short URLs. We can subsequently redirect requests to the respective datastore for efficient request handling.

Both of these approaches ensure smooth traffic handling and mitigate the risk of the datastore becoming a bottleneck.

'''

3. Deletion: A logged-in user can delete a record by requesting the application server which forwards the user details and the associated URL's information to the database server for deletion. A system-initiated deletion can also be triggered upon an expiry time, as we'll see ahead.

4. Custom short links: This task begins with checking the eligibility of the requested short URL. The maximum length allowed is 11 alphanumeric digits. We can find the details on the allowed format and the specific digits in the next lesson. Once verified, the system checks its availability in the database. If the requested URL is available, the user receives a successful short URL generation message, or an error message in the opposite case.

The illustration below depicts how URL shortening, redirection, and deletion work.

[Operations](./op)

Question

Upon successful allocation of a custom short URL, how does the system modify its records?

Answer

Since the custom short URL is the base-58 encoding of an available base-10 unique ID, marking that unique ID as unavailable for future use is necessary for the system's integrity.

On the backend, the system accesses the server with the base-10 equivalent unique ID of that specific base-58 short URL. It marks the ID as unavailable in the range, eliminating any chance of reallocating the same ID to any other request.

[Conversion](./bases.jpg)

This technique also helps availability. The node generating short URLs will no longer need to maintain a list of used and unused unique IDs in memory. Instead, a database is maintained for each of the lists. For good performance, this database can be NoSQL.

The above part explains the post-processing of a custom short URL association. Some further details include the following:

1. Once we generate IDs, we put them in the unused list. As soon as we use an ID from the unused list, we put it in the used list. This eliminates the possibility of duplicate association.
2. As encoding guarantees unique mapping between base-10 and base-58, no two long URLs will have the same short URL.

Design a URL Shortening Service TinyURL/Encoder for TinyURL/README.md

Encoder for TinyURL

Introduction

We've discussed the overall design of a short URL generator (SUG) in detail, but two aspects need more clarification:

1. How does encoding improve the readability of the short URL?
2. How are the sequencer and the base-58 encoder in the short URL generation related?

Why to use encoding

Our sequencer generates a 64-bit ID in base-10, which can be converted to a base-64 short URL. Base-64 is the most common encoding for alphanumeric strings' generation. However, there are some inherent issues with sticking to the base-64 for this design problem: the generated short URL might have readability issues because of look-alike characters. Characters like 0 (capital o) and 0 (zero), I (capital I), and l (lower case L) can be confused while characters like + and / should be avoided because of other system-dependent encodings.

So, we slash out the six characters and use base-58 instead of base-64 (includes A-Z, a-z, 0-9, + and /) for enhanced readability purposes. Let's look at our base-58 definition.

'''

Base-58

Value	Character	Value	Character	Value	Character	Value	Character
0	1	15	G	30	X	45	n
1	2	16	H	31	Y	46	o
2	3	17	J	32	Z	47	p
3	4	18	K	33	a	48	q
4	5	19	L	34	b	49	r
5	6	20	M	35	c	50	s
6	7	21	N	36	d	51	t
7	8	22	P	37	e	52	u
8	9	23	Q	38	f	53	v
9	A	24	R	39	g	54	w
10	B	25	S	40	h	55	x
11	C	26	T	41	i	56	y
12	D	27	U	42	j	57	z
13	E	28	V	43	k		
14	F	29	W	44	m		

'''

'''

The highlighted cells contain the succeeding characters of the omitted ones: 0, 0, I, and l.

'''

Converting base-10 to base-58

Since we're converting base-10 numeric IDs to base-58 alphanumeric IDs, explaining the conversion process will be helpful in grasping the underlying mechanism as well as the overall scope of the SUG. To achieve the above functionality, we use the modulus function.

Process: We keep dividing the base-10 number by 58, making note of the remainder at each step. We stop where there is no remainder left. Then we assign the character indexes to the remainders, starting from assigning the recent-most remainder to the left-most place and the oldest remainder to the right-most place.

Example: Let's assume that the selected unique ID is 2468135791013. The following steps show u

the remainder calculations:

Base-10 = 2468135791013

2468135791013 % 58=17

42554065362 % 58=6

733690782 % 58=4

12649841 % 58=41

218100 % 58=20

3760 % 58=48

64 % 58=6

1 % 58=1

Now, we need to write the remainders in order of the most recent to the oldest order.

Base-58 = [1] [6] [48] [20] [41] [4] [6] [17]

Using the table above, we can write the associated characters with the remainders written above.

Base-58 = 27qMi57J

Note: Both the base-10 numeric IDs and base-64 alphanumeric IDs have 64-bits, as per our sequencer design.

Let's see the example above with the help of the following illustration.

[How a base-10 number is converted into a base-58 alphanumeric short URL]

Converting base-58 to base-10

The decoding process holds equal importance as the encoding process, as we used a decoder in case of custom short URLs generation, as explained in the design lesson.

Process: The process of converting a base-58 number into a base-10 number is also straightforward. We just need to multiply each character index (value column from the table above) by the number of 58s that position holds, and add all the individual multiplication results.

Example: Let's reverse engineer the example above to see how decoding works.

Base-58: 27qMi57J

$2_{58} = 1 \times 58^7 = 2207984167552$

$7_{58} = 6 \times 58^6 = 228412155264$

$q_{58} = 48 \times 58^5 = 31505124864$

$M_{58} = 20 \times 58^4 = 226329920$

$i_{58} = 41 \times 58^3 = 7999592$

$5_{58} = 4 \times 58^2 = 13456$

$$7_{\{58\}} = 6 \times 58^{\{1\}} = 348$$

$$\hat{a} \hat{a} J_{\{58\}} = 17 \times 58^{\{0\}} = 17$$

$$\text{Base-10} = 17 + 348 + 13456 + 7999592 + 226329920 + 31505124864 + 228412155264 + 2207984167552$$

$$\text{Base-10} = 2468135791013.$$

This is the same unique ID of base-10 from the previous example.

Let's see the example above with the help of the following illustration.

[How a base-58 number is converted into a base-10 numeric ID]

The scope of the short URL generator

The short URL generator is the backbone of our URL shortening service. The output of this short URL generator depends on the design-imposed limitations, as given below:

- The generated short URL should contain alphanumeric characters.
- None of the characters should look alike.
- The minimum default length of the generated short URL should be six characters.

These limitations define the scope of our short URL generator. We can define the scope, as shown below:

- Starting range: Our sequencer can generate a 64-bit binary number that ranges from 1 to $(2^{\{64\}}-1)$. To meet the requirement for the minimum length of a short URL, we can select the sequencer IDs to start from at least 10 digits, i.e., 1 Billion.

- Ending point: The maximum number of digits in sequencer IDs that map into the short URL generator's output depends on the maximum utilization of 64 bits, that is, the largest base-10 number in 64-bits. We can estimate the total number of digits in any base by calculating these two points:

1. The numbers of bits to represent one digit in a base-n. This is given by $\log_2\{n\}$
2. Number of digits = Total bits available/Number of bits to represent one digit

Let's see the calculations above for both the base-10 and base-58 mathematically:

- Base-10:
 - The number of bits needed to represent one decimal digit = $\log_2\{10\} = 3.13$
 - The total number of decimal digits in 64-bit numeric ID = $64 / 3.13 = 20$

Base-58:

The number of bits needed to represent one decimal digit = $\log_2\{58\} = 5.85$

The total number of base-58 digits in a 64-bit numeric ID = $64 / 5.85 = 11$

'''

Maximum digits: The calculations above show that the maximum digits in the sequencer generated ID will be 20 and consequently, the maximum number of characters in the encoded short URL will be 11.â â

'''

'''

Question 1

Since we're using the 10 digits and beyond sequencer IDs, is there a way we can use the sequencer IDs shorter than 10 digits?

Answer

We can use the range below the ten digits sequencer IDs for custom short links for users with premium memberships. It will ensure two benefits:

Utilization of the blocked range of IDs

Less than six characters short URLs

Example: Let's assume that the user requests abc as a custom short URL, and it's available in our system, as there is no instance in the data store matching with this short URL. We need to perform the following two operations:

1. Assign this short URL to the requested long URL and store this record in the datastore.
2. Mark the associated unique ID unusable. To find the associated unique ID, we need to decode abc into base-10. Using the above decode method, we come up with the base-10 unique ID value as 113019. The unique ID is less than 1 Billion, as the custom short URL is less than six characters, conforming to the above-stated two benefits.

Our system doesn't ensure a guaranteed custom short link generation, as some other premium member might have claimed the requested custom short URL.

'''

'''

Question 2

What should the short URL be for the sequencer's largest number?

Hide Answer

Since the sequencer's largest number is 18,446,744,073,709,551,615. That is, $2^{64}-1$, the base-58 equivalent of it will be jpXCZedGfVQ, conforming to our calculations above.

'''

The sequencer's lifetime

The number of years that our sequencer can provide us with unique IDs depends on two factors:

- Total numbers available in the sequencer = $2^{64} - 10^9$ (starting from 1 Billion as discussed above)

- Number of requests per year = 200 Million per month $\times 12 = 2.4$ Billion (as assumed in Requirements)

So, taking the above two factors into consideration, we can calculate the expected life of our sequencer.

The lifetime of the sequencer = total numbers available/yearly requests = $\{2^{64}-10^9\} / \{2.4 \text{ Billion}\} = 7,686,143,363.63$ years

'''

Life expectancy for sequencer

Number of requests per month 200 Million

Number of requests per year 2.4 Billion

Lifetime of sequencer 7,686,143,363.63 years

'''

Therefore, our service can run for a long time before the range depletes.

Design a URL Shortening Service TinyURL/Evaluation of TinyURL's Design/README.md

Evaluation of TinyURL's Design

Reviewing the requirements

The last stage of a system design is to evaluate it as per the non-functional requirements mentioned initially. Let's look at each metric one by one.

Availability

We need high availability for users generating new short URLs and redirecting them based on the existing short URLs.

Most of our building blocks, like databases, caches, and application servers have built-in replication that ensures availability and fault tolerance. The short URL generation system will not impact the availability either, as it depends on an easily replicable database of available and used unique IDs.

To handle disasters, we can perform frequent backups of the storage and application servers, preferably twice a day, as we can't risk losing URLs data. We can use the Amazon S3 storage service for backups, as it facilitates cross-zonal replicating and restoration as well. In the worst-case scenario, we might lose 3.3 Million (with 6.6 Million daily requests assumed) newly generated short URLs that are not backed up on that specific day.

Our design uses global server load balancing (GSLB) to handle our system traffic. It ensures intelligent request distribution among different global servers, especially in the case of on-site failures.

We also apply a limit on the requests from clients to secure the intrinsic points of failures. To protect the system against DoS attacks, we use rate limiters between the client and web servers to limit each user's resource allocation. This will ensure a good and smooth traffic influx and mitigate the exploitation of system resources.

[Guarantees provided by our URL shortening service](./e.jpg)

Scalability

Our design is scalable because our data can easily be distributed among horizontally sharded databases. We can employ a consistent hashing scheme to balance the load between the application and database layers.

'''

We can also use the range-based sharding while scaling our database server. However, if we partition our database based on a predefined range, it might lead to imbalanced partitions due to the variable distribution of URLs in each range. An example of this could be the distribution based on the first letter of the URL.

'''

Our choice of the database for mapping URLs, MongoDB, also facilitates horizontal scaling. Some interesting reasons for selecting a NoSQL database are:

1. When a user accesses our system without logging in, our system doesn't save the UserID. Since we're flexible with storing data values, and it aligns more with the schematic flexibility provided by the NoSQL databases, using one for our design is preferable.
2. Scaling a traditional relational database horizontally is a daunting process and poses challenges to meeting our scalability requirements. We want to scale and automatically distribute our system's data across multiple servers. For this requirement, a NoSQL database would best serve our purpose.

Moreover, the large number of unique IDs available in the sequencer's design also ensures the scalability of our system.

Readability

The use of a base-58 encoder, instead of the base-64 encoder, enhances the readability of our system. We divide the readability into two sections:

- Distinguishable characters like 0 (zero), O (capital o), I (capital i), and l (lower case L) are eliminated, excluding the possibility of mistaking a character for another look-alike character.
- Non-alphanumeric characters like + (plus) and / (slash) are also eliminated to only have alphanumeric characters in short URLs. Second, it also helps avoid other system-dependent encodings and makes the URLs readily available for all modern file systems and URLs schemes. Such characters may lead to undesired behavior and output during parsing. This non-functional requirement enhances the user interactivity of our system and makes short URL usage less error-prone for the users.

'''

Fulfilling Non-functional Requirements

Requirements	Techniques
Availability	- The Amazon S3 service backs up the storage and cache servers daily. We can restore them upon fault occurrence. - Global server load balancing to handle the system's traffic. - Rate limiters to limit each user's resource allocation.
Scalability	- Horizontal sharding of the database. - Distribution of the data based on consistent hashing. - MongoDB - as the NoSQL database.
Readability	- Introduction of the base-58 encoder to generate short URLs. - Removal of non-alphanumeric characters. - Removal of look-alike characters.
Latency	- Unnoticeable delay in the overall operation. - MongoDB, because of its low latency and high throughput in reading tasks. - Distributed cache to minimize the service delays. - Unpredictability

'''

Randomly selecting and associating an ID to each request, from the pool of unused and readily available unique IDs.

Latency

Our system ensures low latency with its following features:

- Even the most time-consuming step across the short URL generation process, encoding, takes a few milliseconds. The overall time to generate a short URL is relatively low, ensuring there are no significant delays in this process.
- Our system is redirection-heavy. Writing on the database is minimal compared to reading, and its performance depends on how well it copes with all the redirection requests, compared to the shortening requests. We deliberately chose MongoDB because of its low latency and high throughput in reading-intensive tasks.
- Moreover, the probability of the user using the freshly generated short URL in the next few seconds is relatively low. During this time, synchronous replication to other locations is feasible and therefore adds to the overall low latency of the system for the user.
- The deployment of a distributed cache in our design also ensures that the system redirects the user with the minimum delay possible.

As a result of such design modifications, the system enjoys low latency and high throughput, providing good performance.

Unpredictability

One of the requirements is to make our system's short URLs unpredictable, enhancing the security of our system.

As the sequencer generates unique IDs in a sequence and distributes ranges among servers. Each server has a pre-assigned range of unique IDs, assigning them serially to the requests will make it easy to predict the following short URL. To counter it, we can randomly select a unique ID from the available ones and associate it to the long URL, encompassing the unpredictability of our system.

Conclusion

The URL shortening system is an effective service with multiple advantages. Our design of the URL shortening service is simple, yet it fulfills all the requirements of a performant design. The key features offered by our design are:

1. A dynamic short URL range
2. Improved readability

A possible addition could be the introduction of (local) salt to further increase the unpredictability (security) of the design.

Design a URL Shortening Service TinyURL/Quiz on TinyURL's Design/README.md

Quiz on TinyURL's Design
'''

1 In the URL shortener system's design, the main reason to change the base of the sequencer 58 was:

- A)
To make the generated short URLs more secure.
- B)
To enhance the readability of the generated short URLs.
- C)
To improve the system's availability.
- D)
To ensure the system's low latency.

'''

'''

2 In the system design phase, which reason below was the reason why we decided to use MongoDB?

- A)
It has a higher write throughput as compared to reading.
- B)
It ensures atomicity upon concurrent writes.
- C)
It is the only NoSQL database for more specific problems like TinyURL.
- D)
It's easier to scale MongoDB as compared to other leader-less NoSQL databases.

'''

'''

3 (Select all that apply.) To meet the functional requirement of creating a custom alias for the short link, which of the following restriction(s) do we need to impose on the custom_alias variable for the API call?

You can select multiple correct answers.

- A)
The length of the custom_alias shouldn't be greater than the allowed character limit of the short links.
- B)
The user must declare the value of the custom_alias variable.
- C)

Every time a user creates a short link, a single user must enter the same custom_alias.

D)

The custom_alias should not contain the look-alike removed characters.

'''

'''

4 Which strategy did we adopt to counter the predictability of the sequencer-generated unique IDs?

A)

We changed the base of the sequencer to 58.

B)

We randomly selected a unique ID from the available pool for each short URL generation.

C)

We added a load balancer before the sequencer.

D)

We eliminated the alpha-numeric characters.

'''

Answers

'''

1

In the URL shortener system's design, the main reason to change the base of the sequencer to 58 was:

A)

To make the generated short URLs more secure.

Explanation

Security is independent of the base of the sequencer.

Your Answer

B)

To enhance the readability of the generated short URLs.

Explanation

We change the base to 58 to remove the look-alike characters.

C)

To improve the system's availability.

Explanation

Availability has no direct correspondence with the base of the sequencer.

D)

To ensure the system's low latency.

Explanation

Low-latency has no direct correspondence with the base of the sequencer.

'''

'''

2

In the system design phase, which reason below was the reason why we decided to use MongoDB?

A)
It has a higher write throughput as compared to reading.

Explanation
The opposite is true for MongoDB.

Selected Option
B)
It ensures atomicity upon concurrent writes.

C)
It is the only NoSQL database for more specific problems like TinyURL.

Explanation
It is a very vague statement and, more importantly, false. Every database has its pros and cons.

D)
It's easier to scale MongoDB as compared to other leader-less NoSQL databases.

Explanation
All NoSQL databases are easily scalable, and such a comparison is not readily available.
'''

'''

3
(Select all that apply.) To meet the functional requirement of creating a custom alias for the short link, which of the following restriction(s) do we need to impose on the custom_alias variable for the API call?

You can select multiple correct answers.

Selected Option
A)
The length of the custom_alias shouldn't be greater than the allowed character limit of the short links.

Explanation
In our case, we have limited to 11 alpha-numeric characters from base-58.

B)
The user must declare the value of the custom_alias variable.

Explanation
It's not a requirement of the system we designed, as the default value is None.

C)
Every time a user creates a short link, a single user must enter the same custom_alias.

Explanation
It's a possibility but not a requirement; we can update the long URL against the custom URLs but it's not mandatory for a user.

Selected Option
D)
The custom_alias should not contain the look-alike removed characters.

Explanation
It's an essential requirement to just include the allowed base-58 characters for custom_alias
'''

4. (B)

Design a URL Shortening Service TinyURL/Requirements of TinyURL's Design/README.md

Requirements of TinyURL's Design

Requirements for URL Shortening Design

Let's look at the functional and non-functional requirements for the service we'll be designing:

Functional requirements

- Short URL generation: Our service should be able to generate a unique shorter alias of the given URL.
- Redirection: Given a short link, our system should be able to redirect the user to the original URL.
- Custom short links: Users should be able to generate custom short links for their URLs using our system.
- Deletion: Users should be able to delete a short link generated by our system, given the rights.
- Update: Users should be able to update the long URL associated with the short link, given the proper rights.
- Expiry time: There must be a default expiration time for the short links, but users should be able to set the expiration time based on their requirements.

'''

Question

As a design choice, we don't reuse the expired short URLs. Since we don't reuse them, why need to delete them from our system?

Answer

So far, we've kept the default expiration time to five years. If we relax that limitation and start saving the records forever, our datastore's search index will grow without bound, and querying time from it can add noticeable latency.

'''

Non-functional requirements

- Availability: Our system should be highly available, because even a fraction of the second downtime would result in URL redirection failures. Since our system's domain is in URLs, we don't have the leverage of downtime, and our design must have fault-tolerance conditions instilled in it.
- Scalability: Our system should be horizontally scalable with increasing demand.
- Readability: The short links generated by our system should be easily readable, distinguishable, and typeable.
- Latency: The system should perform at low latency to provide the user with a smooth experience.
- Unpredictability: From a security standpoint, the short links generated by our system should be highly unpredictable. This ensures that the next-in-line short URL is not serially produced, eliminating the possibility of someone guessing all the short URLs that our system has ever produced or will produce.

'''

Question

Why is producing unpredictable short URLs mandatory for our system?

Hide Answer

The following two problems highlight the necessity of producing non-serial, unpredictable short URLs:

1. Attackers can have access to the system-level information of the short URLs's total count, giving them a defined range to plan out their attacks. This type of internal information

shouldnâ t be available outside the system.

2. Our users might have used our service for generating short URLs for secret URLs. With the information above, the attackers can try to list all the short URLs, access them and gain insights about the associated long URLs, risking the secrecy of the private URLs. It will compromise the privacy of a userâ s data, making our system less secure.

Hence, randomly assigning unique IDs deprives the attackers of such system insights, which are needed for enumerating and compromising the userâ s private data.
'''

[Functional and non-functional requirements](./req.jpg)

Resource estimation

Itâ s better to have realistic estimations at the start. For instance, we might need to change them in the future based on the design modifications. Letâ s make some assumptions to complete our estimation.

Assumptions

- We assume that the shortening:redirection request ratio is 1:100.
- There are 200 million new URL shortening requests per month.
- A URL shortening entry requires 500 Bytes of database storage.
- Each entry will have a maximum of five years of expiry time, unless explicitly deleted.
- There are 100 million Daily Active Users (DAU).

Storage estimation

Since entries are saved for a time period of 5 years and there are a total of 200 million entries per month, the total entries will be approximately 12 billion.

$200 \text{ Million/month} \times 12 \text{ months/year} \times 5 \text{ years} = 12 \text{ Billion URL shortening requests}$

Since each entry is 500 Bytes, the total storage estimate would be 6 TB: $12 \text{ Billion} \times 500 \text{ Bytes} = 6 \text{ TB}$

'''

URL Shortening Service Storage Estimation Calculator		
URL shortening per month	200	Million
Expiration time	5	Years
URL object size	500	Bytes
Total number of requests	12	Billion
Total storage	6	TB

'''

[Total storage required by the URL shortening service in 5 years](./storage.jpg)

Query rate estimation

Based on the estimations above, we can expect 20 billion redirection requests per month.

$200 \text{ Million} \times 100 = 20 \text{ Billion}$

We can extend our calculations for Queries Per Second (QPS) for our system from this baseline. The number of seconds in one month, given the average number of days per month is 30.42:

$30.42 \text{ days} \times 24 \text{ hours} \times 60 \text{ minutes} \times 60 \text{ seconds} = 2628288 \text{ seconds}$

Considering the calculation above, new URL shortening requests per second will be:

$200 \text{ Million} / 2628288 \text{ seconds} = 76 \text{ URLs/s}$

With a 1:100 shortening to redirecting ratio, the URL redirection rate per second will be:

$100 \times 76 \text{ URLs/s} = 7.6 \text{ K URLs/s}$

Bandwidth estimation

Shortening requests: The expected arrival rate will be 76 new URLs per second. The total incoming data would be 304 Kbps per second:

$76 \times 500 \text{ Bytes} \times 8 \text{ bits} = 304 \text{ Kbps}$

Redirection requests: Since the expected rate would be 20K URLs redirections per second, the total outgoing data would be 30.4 Mbps

30.4Mbps:

$7.6 \text{ K} \times 500 \text{ Bytes} \times 8 \text{ bits} = 30.4 \text{ Mbps}$

[The total bandwidth required by the URL shortening service](./bandwidth.jpg)

Memory estimation

We need memory estimates in case we want to cache some of the frequently accessed URL redirection requests. Let's assume a split of 80-20 in the incoming requests. 20 percent of redirection requests generate 80 percent of the traffic.

Since the redirection requests per second are 7.6 K, the total would be 0.66 billion for one day.

$7.6 \text{ K} \times 3600 \text{ seconds} \times 24 \text{ hours} = 0.66 \text{ billion}$

Since we would only consider caching 20 percent of these per-day redirection requests, the total memory requirements estimate would be 66 GB.

$0.2 \times 0.66 \text{ Billion} \times 500 \text{ Bytes} = 66 \text{ GB}$

'''

URL Shortening Service Estimates Calculator

URL shortening per month	200	Million
URL redirection per month	20	Billion
Query rate for URL shortening	76	URLs / s
Query rate for URL redirection	7600	URLs / s
Single entry storage size	500	Bytes
Incoming data	304	Kbps
Outgoing data	30.4	Mbps
Cache memory	66	GB

'''

Number of servers estimation

We adopt the same approximation discussed in the back-of-the-envelope calculations to calculate the number of servers needed: the number of daily active users and the daily user handling limit of a server are the two main factors in depicting the total number of servers required. According to the approximation, we need to divide the Daily Active Users (DAU) by 8000 to calculate the approximated number of servers.

Number of servers = $DAU / 8000 = 100 \text{ M} / 8000 = 12500$ servers

[The number of servers required for the URL shortening service](./servers.jpg)

Summarizing estimation

Based on the assumption above, the following table summarizes our estimations:

'''

Type of operation	Time estimates
-------------------	----------------

New URLs	76/s
URL redirections	7.6 K/s
Incoming data	304 Kbps
Outgoing data	30.4 Mbps
Storage for 5 years	6 TB
Memory for cache	66 GB
Servers	12500

Building blocks we will use

With the estimations done, we can identify the key building blocks in our design. Such a list is given below:

[Building blocks in high level design](./bb.jpg)

- Database(s) will be needed to store the mapping of long URLs and the corresponding short URLs.
- Sequencer will provide unique IDs that will serve as a starting point for each short URL generation.
- Load balancers at various layers will ensure smooth requests distribution among available servers.
- Caches will be utilized to store the most frequent short URLs related requests.
- Rate limiters will be used to avoid system exploitation.

Besides these building blocks, we'll also need the following additional components to achieve the desired service:

- Servers to handle and navigate the service requests along with running the application logic.
- A Base-58 encoder to transform the sequencer's numeric output to a more readable and usable alphanumeric form.

[Components in high level design](./hld.jpg)

Design a URL Shortening Service TinyURL/System Design TinyURL/README.md

System Design: TinyURL

Introduction

URL shortening is a service that produces short aliases for long URLs, commonly referred to as short links. Upon clicking, these short links direct to the original URLs. The following illustration depicts how the process works:

'''

Uniform Resource Locator (URL), is an address that uniquely identifies a resource on the web.

'''

[How URL shortening service works](./url.jpg)

Advantages

The key advantages of a URL shortening service are:

- Shortened URLs are convenient to use: they optimize link usage across different forms of devices due to their enhanced accessibility and non-breakability.
- They are visually professional, engaging and facilitate a higher degree of sharing possibilities.
- They are less error-prone while typing.
- They require smaller storage space at the user's end.

Disadvantages

The URL shortening service has some associated drawbacks as well. Some of them are as follows:

- We lose the originality of our brand by using a short URL generated by a third-party service. Many different brands using the same service get short URLs containing the same domain.
- Since weâre using a third-party service for URL shortening, the possibility of it getting shut down and wiping all our shortened URLs will always be there.
- Our business brand image depends on the trustworthiness of a URL shortening service. The wrong business move might negatively impact our business. The competition for acquiring the in-demand custom short URLs is immense, so it might be possible that the best custom URLs are already taken by the time we start generating short URLs.

How will we design a URL shortening service?

Weâve divided the URL shortening service design into the following five lessons:

1. Requirements: This lesson discusses the functional and non-functional requirements of the URL shortening service, along with estimating the resources required to achieve these requirements. Moreover, it also lists down the fundamental building blocks needed to build such a service.
 2. Design and Deployment: This explains the working and usage of each component, the linkage among them, and the overall working mechanism of them as a unit.
 3. Encoder: This particular lesson unfolds the inner mechanism of the encoder used in the design, stating the reason we use it along with the mathematical explanation.
 4. Evaluation: Lastly, we test our design by considering different dimensions of our design requirements and include the possibility of improving it.
 5. Quiz: This lesson will test our understanding of the TinyURL design.
- Letâs start by defining the requirements for a TinyURL-like URL shortening service.

Distributed Cache/Background of Distributed Cache/README.md

Background of Distributed Cache

The main goal of this chapter is to design a distributed cache. To achieve this goal, we should have substantial background knowledge, mainly on different reading and writing techniques. This lesson will help us build that background knowledge. Letâs look at the structure of this lesson in the table below

'''

Structure of This Lesson

Section	Motivation
Writing policies	Data is written to cache and databases. The order in which data writing happens has performance implications. Weâll discuss various writing policies to help decide which writing policy would be suitable for the distributed cache we want to design.
Eviction policies	Since the cache is built on limited storage (RAM), we ideally want to keep the most frequently accessed data in the cache. Therefore, weâll discuss different eviction policies to replace less frequently accessed data with most frequently accessed data.
Cache invalidation	Certain cached data may get outdated. Weâll discuss different invalidation methods to remove stale or outdated entries from the cache in this section.
Storage mechanism	A distributed storage has many servers. Weâll discuss important design considerations, such as which cache entry should be stored in which server and what data structure to use for storage.
Cache client	A cache server stores cache entries, but a cache client calls the cache server to request data. Weâll discuss the details of a cache client library in this section.

'''

Writing policies

Often, cache stores a copy (or part) of data, which is persistently stored in a data store. When we store data to the data store, some important questions arise:

- Where do we store the data first? Database or cache?
- What will be the implication of each strategy for consistency models?

The short answer is, it depends on the application requirements. Let's look at the details of different writing policies to understand the concept better:

- Write-through cache: The write-through mechanism writes on the cache as well as on the database. Writing on both storages can happen concurrently or one after the other. This increases the write latency but ensures strong consistency between the database and the cache.
- Write-back cache: In the write-back cache mechanism, the data is first written to the cache and asynchronously written to the database. Although the cache has updated data, inconsistency is inevitable in scenarios where a client reads stale data from the database. However, systems using this strategy will have small writing latency.
- Write-around cache: This strategy involves writing data to the database only. Later, when a read is triggered for the data, it's written to cache after a cache miss. The database will have updated data, but such a strategy isn't favorable for reading recently updated data.

Eviction policies

One of the main reasons caches perform fast is that they're small. Small caches mean limited storage capacity. Therefore, we need an eviction mechanism to remove less frequently accessed data from the cache.

Several well-known strategies are used to evict data from the cache. The most well-known strategies include the following:

Least recently used (LRU)

Most recently used (MRU)

Least frequently used (LFU)

Most frequently used (MFU)

Other strategies like first in, first out (FIFO) also exist. The choice of each of these algorithms depends on the system the cache is being developed for.

Data can be classified into three temperature regions depending on the access frequency:

- Hot: This is highly accessed data.
- Warm: This is less frequently accessed data.
- Cold: This is rarely accessed data.

Cold data frequently gets evicted from the cache and gets replaced with hot or warm data. The following image shows the spectrum of data temperatures:

[Data temperature](./temperature.jpg)

Cache invalidation

Apart from the eviction of less frequently accessed data, some data residing in the cache may become stale or outdated over time. Such cache entries are invalid and must be marked for deletion.

The situation demands a question: How do we identify stale entries?

Resolution of the problem requires storing metadata corresponding to each cache entry. Specifically, maintaining a time-to-live (TTL) value to deal with outdated cache items.

We can use two different approaches to deal with outdated items using TTL:

- Active expiration: This method actively checks the TTL of cache entries through a daemon process or thread.
- Passive expiration: This method checks the TTL of a cache entry at the time of access.

'''

TTL is the time after which a cache entry is considered expired.
'''

Each expired item is removed from the cache upon discovery.

Storage mechanism

Storing data in the cache isn't as trivial as it seems because the distributed cache has multiple cache servers. When we use multiple cache servers, the following design questions need to be answered:

- Which data should we store in which cache servers?
- What data structure should we use to store the data?

The above two questions are important design issues because they'll decide the performance of our distributed cache, which is the most important requirement for us. We'll use the following techniques to answer the questions above.

Hash function

It's possible to use hashing in two different scenarios:

- Identify the cache server in a distributed cache to store and retrieve data.
- Locate cache entries inside each cache server.

For the first scenario, we can use different hashing algorithms. However, consistent hashing and its flavors usually perform well in distributed systems because simple hashing won't be ideal in case of crashes or scaling.

In the second scenario, we can use typical hash functions to locate a cache entry to read or write inside a cache server. However, a hash function alone can only locate a cache entry. It doesn't say anything about managing data within the cache server. That is, it doesn't say anything about how to implement a strategy to evict less frequently accessed data from the cache server. It also doesn't say anything about what data structures are used to store the data within the cache servers. This is exactly the second design question of the storage mechanism. Let's take a look at the data structure next.

Linked list

We'll use a doubly linked list. The main reason is its widespread usage and simplicity. Furthermore, adding and removing data from the doubly linked list in our case will be a constant time operation. This is because we either evict a specific entry from the tail of the linked list or relocate an entry to the head of the doubly linked list. Therefore, no iterations are required.

'''

Note: Bloom filters are an interesting choice for quickly finding if a cache entry doesn't exist in the cache servers. We can use bloom filters to determine that a cache entry is definitely not present in the cache server, but the possibility of its presence is probabilistic. Bloom filters are quite useful in large caching or database systems.

'''

Sharding in cache clusters

To avoid SPOF and high load on a single cache instance, we introduce sharding. Sharding involves splitting up cache data among multiple cache servers. It can be performed in the following two ways.

Dedicated cache servers

In the dedicated cache servers method, we separate the application and web servers from the cache servers.

The advantages of using dedicated cache servers are the following:

- There's flexibility in terms of hardware choices for each functionality.
- It's possible to scale web/application servers and cache servers separately.

Apart from the advantages above, working as a standalone caching service enables other microservices to benefit from them—for example, Cache as a Service. In that case, the caching system will have to be aware of different applications so that their data doesn't collide.

[A depiction of service hosts coordinating with dedicated cache servers](./cache_servers1.jpg)

Co-located cache

The co-located cache embeds cache and service functionality within the same host.

The main advantage of this strategy is the reduction in CAPEX and OPEX of extra hardware. Furthermore, with the scaling of one service, automatic scaling of the other service is obtained. However, the failure of one machine will result in the loss of both services simultaneously.

'''

CAPEX: Capital expenditures are major investments over assets that are held for a long time.

OPEX: Operational Expenditures are day-to-day expenses that keep the company running.

'''

[Hosting cache and application logic in the same machine](./cache_servers2.jpg)

Cache client

We discussed that the hash functions should be used for the selection of cache servers. But what entity performs these hash calculations?

A cache client is a piece of code residing in hosting servers that do (hash) computations to store and retrieve data in the cache servers. Also, cache clients may coordinate with other system components like monitoring and configuration services. All cache clients are programmed in the same way so that the same PUT, and GET operations from different clients return the same results. Some of the characteristics of cache clients are the following:

- Each cache client will know about all the cache servers.
- All clients can use well-known transport protocols like TCP or UDP to talk to the cache servers.

'''

PUT: Adding data to a cache server.

GET: Retrieving data from a cache server.

'''

'''

Question

What will be the behavior of cache clients to an access request if one of the cache servers is dead?

Answer

Since the data within the cache servers will no longer be available, cache clients will mark that access request as a cache miss.

'''

Conclusion

In this lesson, we learned what distributed caches are and highlighted their significance in distributed systems. We also discussed different storage and eviction mechanisms for caches. Caches are vital for any distributed system and are located at different points within the design of a system. It's important to understand how distributed caches can be designed as part of a large system.

Distributed Cache/Detailed Design of a Distributed Cache/README.md

Detailed Design of a Distributed Cache

This lesson will identify some shortcomings of the high-level design of a distributed cache and improve the design to cover the gaps. Let's get started.

Find and remove limitations

Before we get to the detailed design, we need to understand and overcome some challenges:

- Thereâ s no way for the cache client to realize the addition or failure of a cache server.
- The solution will suffer from the problem of single point of failure (SPOF) because we have single cache server for each set of cache data. Not only that, if some of the data on any of the cache servers is frequently accessed (generally referred to as a hotkey problem), then our performance will also be slow.
- Our solution also didnâ t highlight the internals of cache servers. That is, what kind of structures will it use to store and what eviction policy will it use?

Maintain cache servers list

Letâ s start by resolving the first problem. Weâ ll take incremental steps toward the best possible solution. Letâ s look at the following slides to get an idea of each of the solutions described below:

[[Solution 1] Maintaining a configuration file in each server that the cache client can use](./solution1.jpg)

[[Solution 2] Maintaining a configuration file on a centralized location](./solution2.jpg)

[[Solution 3] Using a configuration service to monitor cache servers and keep the cache client updated](./solution3.jpg)

- Solution 1: Itâ s possible to have a configuration file in each of the service hosts where cache clients reside. The configuration file will contain the updated health and metadata required for the cache clients to utilize the cache servers efficiently. Each copy of the configuration service can be updated through a push service by any DevOps tool. The main problem with this strategy is that the configuration file will have to be manually updated and deployed through some DevOps tools.

- Solution 2: We can store the configuration file in a centralized location that the cache clients can use to get updated information about cache servers. This solves the deployment issue, but we still need to manually update the configuration file and monitor the health of each server.

- Solution 3: An automatic way of handling the issue is to use a configuration service that continuously monitors the health of the cache servers. In addition to that, the cache clients will get notified when a new cache server is added to the cluster. When we use this strategy, no human intervention or monitoring will be required in case of failures or the addition of new nodes. Finally, the cache clients obtain the list of cache servers from the configuration service.

'''

The configuration service has the highest operational cost. At the same time, itâ s a complex solution. However, itâ s the most robust among all the solutions we presented.

'''

Improve availability

The second problem relates to cache unavailability if the cache servers fail. A simple solution is the addition of replica nodes. We can start by adding one primary and two backup nodes in a cache shard. With replicas, thereâ s always a possibility of inconsistency. If our replicas are in close proximity, writing over replicas is performed synchronously to avoid inconsistencies between shard replicas. Itâ s crucial to divide cache data among shards so that neither the problem of unavailability arises nor any hardware is wasted.

This solution has two main advantages:

- Thereâ s improved availability in case of failures.
- Hot shards can have multiple nodes (primary-secondary) for reads.

Not only will such a solution improve availability, but it will also add to the performance.

Internals of cache server

Each cache client should use three mechanisms to store and evict entries from the cache servers:

- Hash map: The cache server uses a hash map to store or locate different entries inside the RAM of cache servers. The illustration below shows that the map contains pointers to each cache value.
- Doubly linked list: If we have to evict data from the cache, we require a linked list so that we can order entries according to their frequency of access. The illustration below depicts how entries are connected using a doubly linked list.
- Eviction policy: The eviction policy depends on the application requirements. Here, we assume the least recently used (LRU) eviction policy.

A depiction of a sharded cluster along with a node's data structure is provided below:

[A shard primary and replica, each with the same internal mechanisms](./lru.jpg)

'''

It's evident from the explanation above that we don't provide a delete API. This is because eviction (through eviction algorithm) and deletion (of expired entries through TTL) is done locally at cache servers. Nevertheless, situations can arise where the delete API may be required. For example, when we delete a recently added entry from the database, it should result in the removal of items from the cache for the sake of consistency.

'''

Detailed design

We're now ready to formalize the detailed design after resolving each of the three previously highlighted problems. Look at the detailed design below:

[Detailed design of a distributed caching system](./caching.jpg)

Let's summarize the proposed detailed design in a few points:

- The client's requests reach the service hosts through the load balancers where the cache clients reside.
- Each cache client uses consistent hashing to identify the cache server. Next, the cache client forwards the request to the cache server maintaining a specific shard.
- Each cache server has primary and replica servers. Internally, every server uses the same mechanisms to store and evict cache entries.
- Configuration service ensures that all the clients see an updated and consistent view of the cache servers.
- Monitoring services can be additionally used to log and report different metrics of the caching service.

'''

Note: An important aspect of the design is that cache entries are stored and retrieved from RAM. We discussed the suitability of RAM for designing a caching system in the previous lesson.

'''

'''

Question

While consistent hashing is a good choice, it may result in unequal distribution of data, and certain servers may get overloaded. How do we resolve this problem?

Answer

A number of consistent hashing algorithms' flavors have been suggested over time. We can use one such flavor that distributes load uniformly and even makes multiple copies of the same data on different cache servers. Each cache server can have virtual servers inside it, and the number of virtual servers in a machine depends on the machine's capability. This results in finer control on the amount of load on a cache server. At the same time, it improves availability.

One example is in DeCandia, Giuseppe, et al. "Dynamo: Amazon's highly available key-value store." ACM SIGOPS operating systems review 41.6 (2007): 205-220.

```
'''  
## Distributed Cache/Evaluation of a Distributed Cache's Design/README.md
```

```
# Evaluation of a Distributed Cache's Design
```

```
## High performance
```

```
Here are some design choices we made that will contribute to overall good performance:
```

- We used consistent hashing. Finding a key under this algorithm requires a time complexity of $O(\log(N))$, where N represents the number of cache shards.
- Inside a cache server, keys are located using hash tables that require constant time on average.
- The LRU eviction approach uses a constant time to access and update cache entries in a doubly linked list.
- The communication between cache clients and servers is done through TCP and UDP protocols, which is also very fast.
- Since we added more replicas, these can reduce the performance penalties that we have to face if there's a high request load on a single machine.
- An important feature of the design is adding, retrieving, and serving data from the RAM. Therefore, the latency to perform these operations is quite low.

```
'''  
Note: A critical parameter for high performance is the selection of the eviction algorithm  
because the number of cache hits and misses depends on it. The higher the cache hit rate, the  
better the performance.  
'''
```

```
To get an idea of how important the eviction algorithm is, let's assume the following:
```

- Cache hit service time (99.9th percentile): 5 ms
- Cache miss service time (99.9th percentile): 30 ms (this includes time to get the data from the database and set the cache)

```
Let's assume we have a 10% cache miss rate using the most frequently used (MFU) algorithm,  
whereas we have a 5% cache miss rate using the LRU algorithm. Then, we use the following  
formula:
```

$$EAT = Ratio_{hit} \times Time_{hit} + Ratio_{miss} \times Time_{miss}$$

```
Here, this means the following:
```

```
EAT: Effective access time.
```

```
Ratio_{hit}: The percentage of times a cache hit will occur.
```

```
Ratio_{miss}: The percentage of times a cache miss will occur.
```

```
Time_{hit}: Time required to serve a cache hit.
```

```
Time_{miss}: Time required to serve a cache miss.
```

```
For MFU, we see the following:
```

```
'''  
EAT = 0.90 x 5 milliseconds + 0.10 x 30 milliseconds = 0.0045 + 0.003 = 0.0075 = 7.5  
milliseconds  
'''
```

```
For LRU, we see the following:
```

```
'''  
EAT = 0.95 x 5 milliseconds + 0.05 x 30 milliseconds = 0.00475 + 0.0015 = 0.00625 = 6.25  
milliseconds.  
'''
```

'''

The numbers above highlight the importance of the eviction algorithm to increase the cache hit rate. Each application should conduct an empirical study to determine the eviction algorithm that gives better results for a specific workload.

Scalability

We can create shards based on requirements and changing server loads. While we add new cache servers to the cluster, we also have to do a limited number of rehash computations, thanks to consistent hashing.

Adding replicas reduces the load on hot shards. Another way to handle the hotkeys problem is to do further sharding within the range of those keys. Although the scenario where a single key will become hot is rare, it's possible for the cache client to devise solutions to avoid the single hotkey contention issue. For example, cache clients can intelligently avoid such a situation that a single key becomes a bottleneck, or we can use dynamic replication for specific keys, and so on. Nonetheless, the solutions are complex and beyond the scope of this lesson.

[Guarantees provided by a distributed cache]

High availability

We have improved the availability through redundant cache servers. Redundancy adds a layer of reliability and fault tolerance to our design. We also used the leader-follower algorithm to conveniently manage a cluster shard. However, we haven't achieved high availability because we have two shard replicas, and at the moment, we assume that the replicas are within a data center.

It's possible to achieve higher availability by splitting the leader and follower servers among different data centers. But such high availability comes at a price of consistency. We assumed synchronous writes within the same data center. But synchronous writing for strong consistency in different data centers has a serious performance implication that isn't welcomed in caching systems. We usually use asynchronous replication across data centers.

For replication within the data center, we can get strong consistency with good performance. We can compromise strong consistency across data center replication to achieve better availability (see CAP and PACELC theorems).

Consistency

It's possible to write data to cache servers in a synchronous or asynchronous mode. In the context of caching, the asynchronous mode is favored for improved performance. Consequently, our caching system suffers from inconsistencies. Alternatively, strong consistency comes from synchronous writing, but this increases the overall latency, and the performance takes a hit.

Inconsistency can also arise from faulty configuration files and services. Imagine a scenario where a cache server is down during a write operation, and a read operation is performed on it just after its recovery. We can avoid such scenarios for any joining or rejoining server by not allowing it to serve requests until it's reasonably sure that it's up to date.

Affordability

Our proposed design has a low cost because it's feasible and practical to create such a system using commodity hardware.

'''

Question 1

What happens if the leader node fails in the leader-follower protocol?

Answer

Two possible solutions can be used in that case:

Use the leader election algorithm to elect a new leader among the set of available followers. Use a separate distributed configuration management service to monitor and select leaders.

'''

'''

Question 2

Is it a good idea to introduce synchronous replication to cache servers?

Answer

Synchronous replication on the cache clusters will introduce a performance penalty because it takes time to replicate the same data to all the cache servers. Therefore, there's a trade-off between performance and consistency in the usage of asynchronous versus synchronous replication, respectively.

The answer depends on the use case. In case of cache servers, synchronous replication is a good choice if the replicated servers are within close proximity of the primary server.

'''

'''

Question 3

Assume that a large number of concurrent requests are forwarded to a single shard server. If the server uses the LRU algorithm, some of the requests will result in a cache miss and write new entry to the cache, while some will result in a cache hit and end up reading from the server. These concurrent requests may compete with each other. Therefore, some locking mechanisms may be required. However, locking an entire data structure for all the reading and writing operations will heavily reduce the performance. How do you think such a problem will be dealt with?

Answer

Generally, for concurrent access to shared data, some locking mechanisms like Semaphore, Monitors, Mutex locks, and others are good choices. But as the number of users (readers in case of cache hit or writers in case of a cache miss) grows, locking the entire data structure isn't a suitable solution. In that case, we can consider the following options:

- Limited locking: In this strategy, only specific sections of the entire data structure will be locked. While some threads or processes can read from the data structure simultaneously, some threads may temporarily block access to specific sections of the data structure.
- Offline eviction: Offline eviction may be a possibility where instead of making actual changes to the data structure, only the required changes will be recorded while performing different operations until it's necessary to commit the changes. This solution is desirable and easy if the cache hit rate is high because a change to the data structure will likely be required when a cache miss occurs.
- Lock-free implementation: Multiple solutions have been proposed which suggest that simultaneous reading and writing over doubly linked list is feasible to support large number of concurrent reading and writing.

locking mechanism: A mechanism by which access to shared data is limited to a single thread only. Thus achieving synchronization.

'''

Summary

We studied the basics of the cache and designed a distributed cache that has a good level of availability, high performance, high scalability, and low cost. Our mechanism maintains high availability using replicas, though if all replicas are in one data center, such a scheme won't tackle full data center failures. Now that we've learned about the basics of design, let's explore popular open-source frameworks like Memcached and Redis.

Distributed Cache/High-level Design of a Distributed Cache/README.md

High-level Design of a Distributed Cache

In this lesson, we'll learn to design a distributed cache. We'll also discuss the trade-offs and design choices that can occur while we progress in our journey towards developing a solution.

Requirements

Let us start by understanding the requirements of our solution.

Functional

The following are the functional requirements:

- Insert data: The user of a distributed cache system must be able to insert an entry to the cache.
- Retrieve data: The user should be able to retrieve data corresponding to a specific key.

[Functional and non-functional requirements of a distributed cache](./req.jpg)

Non-functional requirements

We'll consider the following non-functional requirements:

- High performance: The primary reason for the cache is to enable fast retrieval of data. Therefore, both the insert and retrieve operations must be fast.
- Scalability: The cache system should scale horizontally with no bottlenecks on an increasing number of requests.
- High availability: The unavailability of the cache will put an extra burden on the database servers, which can also go down at peak load intervals. We also require our system to survive occasional failures of components and network, as well as power outages.
- Consistency: Data stored on the cache servers should be consistent. For example, different cache clients retrieving the same data from different cache servers (primary or secondary) should be up to date.
- Affordability: Ideally, the caching system should be designed from commodity hardware instead of an expensive supporting component within the design of a system.

API design

The API design for this problem is sufficiently easy since there are only two basic operations

Insertion

The API call to perform insertion should look like this:

```
'''
```

```
insert(key, value)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------

key	This is a unique identifier.
-----	------------------------------

value	This is the data stored against a unique key.
-------	---

```
'''
```

This function returns an acknowledgment or an error depicting the problem at the server end.

Retrieval

The API call to retrieve data from the cache should look like this:

```
'''
```

```
retrieve(key)
```

```
'''
```

```
'''
```

Parameter	Description
-----------	-------------

key	This returns the data stored against the key.
-----	---

```
'''
```

This call returns an object to the caller.

```
'''
```

Question

The API design of the distributed cache looks exactly like the key-value store. What are the possible differences between a key-value store and a distributed cache?

Answer

Some of the key differences are the following:

- Key-value stores need to durably store data (persistence). A cache is used in addition to persistent storage to increase reading performance.
- A cache serves data from the RAM. A key-value store writes data to non-volatile storage.
- Key-value stores are robust and should survive failures. However, caches can crash and be populated from scratch after recovery.

'''

Design considerations

Before designing the distributed cache system, it's important to consider some design choices. Each of these choices will be purely based on our application requirements. However, we can highlight some key differences here:

Storage hardware

If our data is large, we may require sharding and therefore use shard servers for cache partitions. Should these shard servers be specialized or commodity hardware? Specialized hardware will have good performance and storage capacity, but it will cost more. We can build large cache from commodity servers. In general, the number of shard servers will depend on the cache's size and access frequency.

Furthermore, we can consider storing our data on the secondary storage of these servers for persistence while we still serve data from RAM. Secondary storage may be used in cases where a reboot happens, and cache rebuilding takes a long time. Persistence, however, may not be a requirement in a cache system if there's a dedicated persistence layer, such as a database.

'''

Facebook built 28 TeraBytes of cache using commodity machines in 2013.

'''

Data structures

A vital part of the design has to be the speed of accessing data. Hash tables are data structures that take a constant time on average to store and retrieve data. Furthermore, we need another data structure to enforce an eviction algorithm on the cached data. In particular, linked lists are a good option (as discussed in the previous lesson).

Also, we need to understand what kind of data structures a cache can store. Even though we discussed in the API design section that we'll use strings for simplicity, it's possible to store different data structures or formats, like hash maps, arrays, sets, and so on, within the cache. In the next lesson, we'll see a practical example of such a cache.

Cache client

It's the client process or library that places the insert and retrieve calls. The location of the client process is a design issue. For example, it's possible to place the client process within a serving host if the cache is for internal use only. Otherwise, in the case where the caching system is provided as a service for external use, a dedicated cache client can send users' requests to the cache servers.

Writing policy

The writing strategy over the cache and database has consistency implications. In general, there's no optimal choice, but depending on our application, the preference of writing policy is significantly important.

Eviction policy

By design, the cache provides low-latency reads and writes. To achieve this, data is often served from RAM memory. Usually, we can't put all the data in the cache due to the limited size of the cache as compared to the full dataset. So, we need to carefully decide what stays in the cache and how to make room for new entries.

With the addition of new data, some of the existing data may have to be evicted from the cache. However, choosing a victim entry depends on the eviction policy. Numerous eviction policies exist, but the choice again depends on the application using it. For instance, least recently used (LRU) can be a good choice for social media services where recently uploaded content will likely get the most views.

Apart from the details in the sections above, optimizing the time-to-live (TTL) value can play an essential role in reducing the number of cache misses.

High-level design

The following figure depicts our high-level design:

[High-level design of distributed cache](./design.jpg)

The main components in this high-level design are the following:

- Cache client: This library resides in the service application servers. It holds all the information regarding cache servers. The cache client will choose one of the cache servers using a hash and search algorithm for each incoming insert and retrieve request. All the cache clients should have a consistent view of all the cache servers. Also, the resolution technique to move data to and from the cache servers should be the same. Otherwise, different clients will request different servers for the same data.

- Cache servers: These servers maintain the cache of the data. Each cache server is accessible by all the cache clients. Each server is connected to the database to store or retrieve data. Cache clients use TCP or UDP protocol to perform data transfer to or from the cache servers. However, if any cache server is down, requests to those servers are resolved as a missed cache by the cache clients.

Distributed Cache/Memcached versus Redis/README.md

Memcached versus Redis

Introduction

This lesson will discuss some of the widely adopted real-world implementations of a distributed cache. Our focus will be on two well-known open-source frameworks: Memcached and Redis. Theyâre highly scalable, highly performant, and robust caching tools. Both of these techniques follow the client-server model and achieve a latency of sub-millisecond. Letâs discuss each one of them and then compare their usefulness.

Memcached

Memcached was introduced in 2003. Itâs a key-value store distributed cache designed to store objects very fast. Memcached stores data in the form of a key-value pair. Both the key and the value are strings. This means that any data that has been stored will have to be serialized. So, Memcached doesnât support and canât manipulate different data structures.

'''

Serialization is the process of translating data into a format that can be transmitted or stored elsewhere. Later, reconstruction or deserialization of the data should be possible.

'''

Memcached has a client and server component, each of which is necessary to run the system. The system is designed in a way that half the logic is encompassed in the server, whereas the other half is in the client. However, each server follows the shared-nothing architecture. In this architecture, servers are unaware of each other, and thereâs no synchronization, data sharing, and communication between the servers.

Due to the disconnected design, Memcached is able to achieve almost a deterministic query speed ($O(1)$) serving millions of keys per second using a high-end system. Therefore, Memcached offers a high throughput and low latency.

[Design of a typical Memcached cluster](./memcached.jpg)

As evident from the design of a typical Memcached cluster, Memcached scales well horizontally. The client process is usually maintained with the service host that also interacts with the authoritative storage (back-end database).

Facebook and Memcached

The data access pattern in Facebook requires frequent reads and updates because views are presented to the users on the fly instead of being generated ahead of time. Because Memcached is simple, it was an easy choice for the solution because Memcached started developing in 2003 whereas Facebook was developed in 2004. In fact, in some cases, Facebook and Memcached teams worked together to find solutions.

'''

Redis was developed in 2009. So, using Redis wasn't a possibility at Facebook by then.

'''

Some of the simple commands of Memcached include the following:

'''

get <key_1> <key_2> <key_3> ...

set <key> <value> ...

delete <key>[<time>] ...

'''

At Facebook, Memcached sits between the MySQL database and the web layer that uses roughly 28 TeraBytes of RAM spread across more than 800 servers (as of 2013, Facebook and Memcached - TechTalk). By an approximation of least recently used (LRU) eviction policy, Facebook is able to achieve a cache hit rate of 95%.

The following illustration shows the high-level design of caching architecture at Facebook. As we can see, out of a total of 50 million requests made by the web layer, only 2.5 million requests reach the persistence layer.

[Facebook using a layer of Memcached sitting between persistence and web layer](./fb.jpg)

Redis

Redis is a data structure store that can be used as a cache, database, and message broker. It offers rich features at the cost of additional complexity. It has the following features:

- Data structure store: Redis understands the different data structures it stores. We don't have to retrieve data structures from it, manipulate them, and then store them back. We can make in-house changes that save both time and effort.
- Database: It can persist all the in-memory blobs on the secondary storage.
- Message broker: Asynchronous communication is a vital requirement in distributed systems. Redis can translate millions of messages per second from one component to another in a system. Redis provides a built-in replication mechanism, automatic failover, and different levels of persistence. Apart from that, Redis understands Memcached protocols, and therefore, solutions using Memcached can translate to Redis. A particularly good aspect of Redis is that it separates data access from cluster management. It decouples data and controls the plane. This results in increased reliability and performance. Finally, Redis doesn't provide strong consistency due to the use of asynchronous replication.

[Redis structure supporting automatic failover using redundant secondary replicas](./redis_replicas.jpg)

Redis cluster

Redis has built-in cluster support that provides high availability. This is called Redis Sentinel. A cluster has one or more Redis databases that are queried using multithreaded proxies. Redis clusters perform automatic sharding where each shard has primary and secondary nodes. However, the number of shards in a database or node is configurable to meet the expectations and requirements of an application.

Each Redis cluster is maintained by a cluster manager whose job is to detect failures and

perform automatic failovers. The management layer consists of monitoring and configuration software components.

[Architecture of Redis clusters](./redis_clusters.jpg)

Pipelining in Redis

Since Redis uses a client-server model, each request blocks the client until the server receives the result. A Redis client looking to send subsequent requests will have to wait for the server to respond to the first request. So, the overall latency will be higher.

Redis uses pipelining to speed up the process. Pipelining is the process of combining multiple requests from the client side without waiting for a response from the server. As a result, it reduces the number of RTT spans for multiple requests.

'''

The round-trip-time (RTT) is the latency for a request to travel from the client to the server and back.

'''

[Redis client-server communication without pipelining versus Redis client-server communication with pipelining](./redis_communication.jpg)

The process of pipelining reduces the latency through RTT and the time to do socket level I/O. Also, mode switching through system calls in the operating system is an expensive operation that's reduced significantly via pipelining. Pipelining the commands from the client side has no impact on how the server processes these requests.

For example, two requests pipelined by the client reach the server, and the server can't entertain the second. The server provides a result for the first and returns an error for the second. The client is independent in batching similar commands together to achieve maximum throughput.

'''

Note: Pipelining improves the latency to a minimum of five folds if both the client and server are on the same machine. The request is sent on a loopback address (127.0.0.1). The true power of pipelining is highlighted in systems where requests are sent to distant machines.

'''

Memcached versus Redis

Even though Memcached and Redis both belong to the NoSQL family, there are subtle aspects that set them apart:

- Simplicity: Memcached is simple, but it leaves most of the effort for managing clusters left to the developers of the cluster. This, however, means finer control using Memcached. Redis, on the other hand, automates most of the scalability and data division tasks.
- Persistence: Redis provides persistence by properties like append only file (AOF) and Redis database (RDB) snapshot. There's no persistence support in Memcached. But this limitation can be catered to by using third-party tools. For example, libmemcached-tools
- Data types: Memcached stores objects, whereas Redis supports strings, sorted sets, hash maps, bitmaps, and hyper logs. However, the maximum key or value size is configurable.
- Memory usage: Both tools allow us to set a maximum memory size for caching. Memcached uses the slab allocation method for reducing fragmentation. However, when we update the existing entries's size or store many small objects, there may be a wastage of memory. Nonetheless, there are configuration workarounds to resolve such issues.
- Multithreading: Redis runs as a single process using one core, whereas Memcached can efficiently use multicore systems with multithreading technology. We could argue that Redis was designed to be a single-threaded process that reduces the complexity of multithreaded systems. Nonetheless, multiple Redis processes can be executed for concurrency. At the same time, Redis has improved over the years by tweaking its performance. Therefore, Redis can store small data items efficiently. Memcached can be the right choice for file sizes above 100 K.

- Replication: As stated before, Redis automates the replication process via few commands, whereas replication in Memcached is again subject to the usage of third-party tools. Architecturally, Memcached can scale well horizontally due to its simplicity. Redis provides scalability through clustering that's considerably complex.

The table below summarizes some of the main differences and common features between Memcached and Redis:

'''

Features Offered by Memcached and Redis			
Feature	Memcached	Redis	
Low latency	Yes	Yes	
Persistence	Possible via third-party tools	Multiple options	
Multilanguage support	Yes	Yes	
Data sharding	Possible via third-party tools	Built-in solution	
Ease of use	Yes	Yes	
Multithreading support	Yes	No	
Support for data structure structures	Objects	Multiple data	
Support for transaction	No	Yes	
Eviction policy	LRU	Multiple algorithms	
Lua scripting support	No	Yes	
Geospatial support	No	Yes	

'''

To summarize, Memcached is preferred for smaller, simpler read-heavy systems, whereas Redis is useful for systems that are complex and are both read- and write-heavy.

'''

Question 1

Based on the implementation details, which of the two frameworks (Memcached or Redis) has a striking similarity with the distributed cache that we designed in the previous lesson?

Answer

The answer is Memcached. The reasons are as follows:

- Client software chooses which cache server to use with a hashing algorithm.
- Server software stores the values against each key using an internal hash table.
- Least recently used (LRU) is used as the eviction policy.
- There's no communication between different cache servers.

'''

'''

Question 2

Why do third-party tools exist for persisting Memcached data?

Answer

It's because a lot of data is read and written to cache servers, and they may occasionally crash for whatever reason. After restarting, building a cache from scratch can take up to hours in specific scenarios, and that ultimately reduces system performance. Therefore, cache data

may be persisted to disk to be loaded on a restart.
'''

'''

Question 3

What is the advantage of storing different data structures as compared to strings only?

Answer

The main advantage is that Redis can modify data in place without wasting network bandwidth by downloading and uploading. It saves network bandwidth, but it also saves time and effort by avoiding the serialization and deserialization of data.

'''

Conclusion

It's impossible to imagine high-speed large-scale solutions without the use of a caching system. In this chapter, we covered the need for a caching system and its fundamental details, and we orchestrated a basic distributed cache system. We also familiarized ourselves with the design and features of two of the most well-known caching frameworks.

Distributed Cache/System Design The Distributed Cache/README.md

System Design: The Distributed Cache

Problem statement

A typical system consists of the following components:

- It has a client that requests the service.
- It has one or more service hosts that entertain client requests.
- It has a database used by the service for data storage.

Under normal circumstances, this abstraction performs fine. However, as the number of users increases, the database queries also increase. And as a result, the service providers are overburdened, resulting in slow performance.

In such cases, a cache is added to the system to deal with performance deterioration. A cache is a temporary data storage that can serve data faster by keeping data entries in memory. Caches store only the most frequently accessed data. When a request reaches the serving host, it retrieves data from the cache (cache hit) and serves the user. However, if the data is unavailable in the cache (cache miss), the data will be queried from the database. Also, the cache is populated with the new value to avoid cache misses for the next time.

'''

Cache hit: When the requested data is found in the cache, the server responds with the data immediately.

Cache miss: When the requested data isn't found in the cache, it's called a cache miss.

'''

[Service before using caching](./no_cache.jpg)

[Service using caching to improve performance.](./no_cache.jpg)

A cache is a nonpersistent storage area used to keep repeatedly read and written data, which provides the end user with lower latency. Therefore, a cache must serve data from a storage component that is fast, has enough storage, and is affordable in terms of dollar cost as we scale the caching service. The following illustration highlights the suitability of RAM as the raw building block for caching:

[An approximation that depicts how RAM is the optimal choice for serving cached data](./interest.jpg)

We understand the need for a cache and suitable storage hardware, but what is distributed cache? Let's discuss this next.

What is a distributed cache?

A distributed cache is a caching system where multiple cache servers coordinate to store frequently accessed data. Distributed caches are needed in environments where a single cache server isn't enough to store all the data. At the same time, it's scalable and guarantees higher degree of availability.

Caches are generally small, frequently accessed, short-term storage with fast read time. Caches use the locality of reference principle.

Locality of reference is the notion that a program will access a particular set of data over a short period of time. Locality can be spatial or temporal.

Generally, distributed caches are beneficial in the following ways:

- They minimize user-perceived latency by precalculating results and storing frequently accessed data.
- They pre-generate expensive queries from the database.
- They store user session data temporarily.
- They serve data from temporary storage even if the data store is down temporarily.
- Finally, they reduce network costs by serving data from local resources.

Why distributed cache?

When the size of data required in the cache increases, storing the entire data in one system is impractical. This is because of the following three reasons:

- It can be a potential single point of failure (SPOF).
- A system is designed in layers, and each layer should have its caching mechanism to ensure the decoupling of sensitive data from different layers.
- Caching at different locations helps reduce the serving latency at that layer.

In the table below, we describe how caching at different layers is performed through the use of various technologies. It's important to note that key-value store components are used in various layers.

'''

Caching at Different Layers of a System

System Layer Usage	Technology in Use
Web Accelerate retrieval of static web content, and manage sessions	HTTP cache headers, web accelerators, key-value store, CDNs, and so on
Application Accelerate application-level computations and data retrieval	Local cache and key-value data store
Database Reduce data retrieval latency and I/O load from database	Database cache, buffers, and key-value data store

'''

'''

Apart from the three system layers above, caching is also performed at DNS and client-side technologies like browsers or end-devices.

'''

How will we design distributed cache?

We'll divide the task of designing and reinforcing learning major concepts of distributed cache into five lessons:

1. Background of Distributed Cache: It's imperative to build the background knowledge necessary

to make critical decisions when designing distributed caches. This lesson will revisit some basic but important concepts.

2. High-level Design of a Distributed Cache: Weâ ll build a high-level design of a distributed cache in this lesson.
3. Detailed Design of a Distributed Cache: Weâ ll identify some limitations of our high-level design and work toward a scalable, affordable, and performant solution.
4. Evaluation of a Distributed Cache Design: This lesson will evaluate our design for various non-functional requirements, such as scalability, consistency, availability, and so on.
5. Memcached versus Redis: Weâ ll discuss well-known industrial solutions, namely Memcached and Redis. Weâ ll also go through their details and compare their features to help us understand their potential use cases and how they relate to our design.

Letâ s begin by exploring the background of the distributed cache in the next lesson.

Distributed Logging/Design of a Distributed Logging Service/README.md

Design of a Distributed Logging Service

Weâ ll design the distributed logging system now. Our logging system should log all activities or messages (weâ ll not incorporate sampling ability into our design).

Requirements

Letâ s list the requirements for designing a distributed logging system:

Functional requirements

The functional requirements of our system are as follows:

- Writing logs: The services of the distributed system must be able to write into the logging system.

- Searchable logs: It should be effortless for a system to find logs. Similarly, the applicationâ s flow from end-to-end should also be effortless.

- Storing logging: The logs should reside in distributed storage for easy access.

- Centralized logging visualizer: The system should provide a unified view of globally separated services.

Non-functional requirements

The non-functional requirements of our system are as follows:

- Low latency: Logging is an I/O-intensive operation that is often much slower than CPU operations. We need to design the system so that logging is not on an applicationâ s critical path.

- Scalability: We want our logging system to be scalable. It should be able to handle the increasing amounts of logs over time and a growing number of concurrent users.

- Availability: The logging system should be highly available to log the data.

Building blocks we will use

The design of a distributed logging system will utilize the following building blocks:

Pub-sub system: Weâ ll use a pub-sub- system to handle the huge size of logs.

Distributed search: Weâ ll use distributed search to query the logs efficiently.

[Building blocks we will use](./bb.jpg)

API design

The API design for this problem is given below:

****Write a message****

The API call to perform writing should look like this:

'''

write(unique_ID, message_to_be_logged)

'''

```
'''
```

Parameter	Description
unique_ID	It is a numeric ID containing application-id, service-id, and a time stamp.
message_to_be_logged	It is the log message stored against a unique key.

```
'''
```

****Search log****

The API call to search data should look like this:

```
'''
searching(keyword)
'''
```

This call returns a list of logs that contain the keyword.

```
'''
```

Parameter	Description
keyword	It is used for finding the logs containing the keyword.

```
'''
```

Initial design

In a distributed system, clients across the globe generate events by requesting services from different serving nodes. The nodes generate logs while handling each of the requests. These logs are accumulated on the respective nodes.

In addition to the building blocks, let's list the major components of our system:

- Log accumulator: An agent that collects logs from each node and dumps them into storage. So, if we want to know about a particular event, we don't need to visit each node, and we can fetch them from our storage.
- Storage: The logs need to be stored somewhere after accumulation. We'll choose blob storage to save our logs.
- Log indexer: The growing number of log files affects the searching ability. The log indexer will use the distributed search to search efficiently.
- Visualizer: The visualizer is used to provide a unified view of all the logs.

The design for this method looks like this:

[Initial design](./initial_design.jpg)

Logging at various levels

Let's explore how the logging system works at various levels.

In a server

In this section, we'll learn how various services belonging to different apps will log in to the server.

Let's consider a situation where we have multiple different applications on a server, such as App 1, App 2, and so on. Each application has various microservices running as well. For example, an e-commerce application can have services like authenticating users, fetching carts, and more running at the same time. Every service produces logs. We use an ID with application-id, service-id, and its time stamp to uniquely identify various services of multiple applications. Time stamps can help us to determine the causality of events.

Each service will push its data to the log accumulator service. It is responsible for these actions:

- Receiving the logs.
- Storing the logs locally.
- Pushing the logs to a pub-sub system.

[In a server](./in_a_server)

3 of 3 We should be mindful that data can be lost in the process of logging huge amounts of messages. There is a trade-off between user-perceived latency and the guarantee that log data persists. For lower latency, log services often keep data in RAM and persist them asynchronously. Additionally, we can minimize data loss by adding redundant log accumulators to handle growing concurrent users.

'''

Question

How does logging change when we host our service on a multi-tenant cloud (like AWS) versus when an organization has exclusive control of the infrastructure (like Facebook), specifically in terms of logs?

Answer

Security might be one aspect that differs between multi-tenant and single-tenant settings. When we encrypt all logs and secure a logging service end-to-end, it does not come free, and has performance penalties. Additionally, strict separation of logs is required for a multi-tenant setting, while we can improve the storage and processing utilization for a single-tenant setting.

Let's take the example of Meta's Facebook. They have millions of machines that generate logs, and the size of the logs can be several petabytes per hour. So, each machine pushes its logs to a pub-sub system named Scribe. Scribe retains data for a few days and various other systems process the information residing in the Scribe. They store the logs in distributed storage also. Managing the logs can be application-specific.

On the other hand, for multi-tenancy, we need a separate instance of pub-sub per tenant (or per application) for strict separation of logs.

'''

'''

Note: For applications like banking and financial apps, the logs must be very secure so hackers cannot steal the data. The common practice is to encrypt the data and log. In this way, no one can decrypt the encrypted information using the data from logs.

'''

At datacenter level

All servers in a data center push the logs to a pub-sub system. Since we use a horizontally-scalable pub-sub system, it is possible to manage huge amounts of logs. We may use multiple instances of the pub-sub per data center. It makes our system scalable, and we can avoid bottlenecks. Then, the pub-sub system pushes the data to the blob storage.

[Log accumulator sending data to the pub-sub system](./datacenter.jpg)

The data does not reside in pub-sub forever and gets deleted after a few days before being stored in archival storage. However, we can utilize the data while it is available in the pub-sub system. The following services will work on the pub-sub data:

- Filterer: It identifies the application and stores the logs in the blob storage reserved for that application since we do not want to mix logs of two different applications.
- Error aggregator: It is critical to identify an error as quickly as possible. We use a service that picks up the error messages from the pub-sub system and informs the respective client. It saves us the trouble of searching the logs.

- Alert aggregator: Alerts are also crucial. So, it is important to be aware of them early. This service identifies the alerts and notifies the appropriate stakeholders if a fatal error is encountered, or sends a message to a monitoring tool.

The updated design is given below:

[Adding a filterer, error aggregator, and alert aggregator](./updated.jpg)

'''

Question

Do we store the logs for a lifetime?

Answer

Logs also have an expiration date. We can delete regular logs after a few days or months. Compliance logs are usually stored for up to three to five years. It depends on the requirements of the application.

'''

In our design, we have identified another component called the expiration checker. It is responsible for these tasks:

- Verifying the logs that have to be deleted. Verifying the logs to store in cold storage. Moreover, our components log indexer and visualizer work on the blob storage to provide a good searching experience to the end user. We can see the final design of the

'''

Cold storages are low-cost storages where archived data is kept. It is done to save money on storage charges and to migrate data that isn't used frequently.

'''

logging service below:

[Logging service design]

'''

Question

We learned earlier that a simple user-level API call to a large service might involve hundreds of internal microservices and thousands of nodes. How can we stitch together logs end-to-end for one request with causality intact?

Answer

Most complex services use a front-end server to handle an end user's request. On reception of request, the front-end server can get a unique identifier using a sequencer. This unique identifier will be appended to all the fanned-out services. Each log message generated anywhere in the system also emits the unique identifier.

Later, we can filter the log (or preprocess it) based on the unique identifiers. At this step, we are able to collect all the logs across microservices against a unique request. In the Sequencer building block, we discussed that we can get unique identifiers that maintain happens-before causality. Such an identifier has the property that if ID 1 is less than ID 2, then ID 1 represents a time that occurred before ID 2. Now, each log item can use a time-stamp, and we can sort log entries for a specific request in ascending order.

Correctly ordering the log in a chronological (or causal) order simplifies log analyses.

'''

'''

Note: Windows Azure Storage System (WAS) uses an extensive logging infrastructure in its development. It stores the logs in local disks, and given a large number of logs, they do not push the logs to the distributed storage. Instead, they use a grep-like utility that works as distributed search. This way, they have a unified view of globally distributed logs data.

'''

There can be various ways to design a distributed logging service, but it solely depends on the requirements of our application.

Conclusion

- We learned how logging is crucial in understanding the flow of events in a distributed system. It helps to reduce the mean time to repair (MTTR) by steering us toward the root cause of issues.
- Logging is an I/O-intensive operation that is time-consuming and slow. It is essential to handle it carefully and not affect the critical path of other servicesâ execution.
- Logging is essential for monitoring because the data fetched from logs helps monitor the health of an application. (Alert and error aggregators serve this purpose.)

Distributed Logging/Introduction to Distributed Logging/README.md

Introduction to Distributed Logging

Logging in a distributed system

In todayâs world, an increasing number of designs are moving to microservice architecture instead of monolithic architecture. In microservice architecture, logs of each microservice are accumulated in the respective machine. If we want to know about a certain event that was processed by several microservices, it is difficult to go into every node, figure out the flow, and view error messages. But, it becomes handy if we can trace the log for any particular flow from end to end.

Moreover, it is also not necessary that a microservice is deployed on only one node. It can be deployed on thousands of nodes. Consider the following example, where hundreds of microservices are interdependent, and failure of one service can result in failures of other services. And if we do not have logs, we might not determine the root cause of failure. This emphasizes the need for logging.

[Logging](./logging)

Restrain the log size

The number of logs increases over time. At a time, perhaps hundreds of concurrent messages need to be logged. But the question is, are they all important enough to be logged? To solve this, logs have to be structured. We need to decide what to log into the system on the application or logging level.

Use sampling

Weâll determine which messages we should log into the system in this approach. Consider a situation where we have lots of messages from the same set of events. For example, there are people commenting on a post, where Person X commented on Person Yâs post, then Person Z commented on Person Yâs post, and so on. Instead of logging all the information, we can use a sampler service that only logs a smaller set of messages from a larger chunk. This way, we can decide on the most important messages to be logged.

'''

Note: For large systems like Facebook, where billions of events happen per second, it is not viable to log them all. An appropriate sampling threshold and strategy are necessary to selectively pick a representative data set.

'''

'''

Question

What is a scenario where the sampling approach will not work?

Answer

Letâs consider an application that processes a financial ATM transaction. It runs various services like fraud detection, expiration time checking, card validation, and many more. If we start to miss out logging of any service, we cannot identify an end-to-end flow that affects

the debugging in case an error occurs. Using sampling, in this case, is not ideal and results in the loss of useful data.

We can also categorize the types of messages and apply a filter that identifies the important messages and only logs them to the system.

'''

Use categorization

Let's look into the logging support provided by various programming languages. For example, there's log4j and logging in Python. The following severity levels are commonly used in logging:

- DEBUG
- INFO
- WARNING
- ERROR
- FATAL/CRITICAL

Usually, the production logs are set to print messages with the severity of WARNING and above. But for more detailed flow, the severity levels can be set to DEBUG and INFO levels too.

Structure the logs

Applications have the liberty to choose the structure of their log data. For example, an application is free to write to log as binary or text data, but it is often helpful to enforce some structure on the logs. The first benefit of structured logs is better interoperability between log writers and readers. Second, the structure can make the job of a log processing system easier.

'''

Note: The structuring of logs is a dense topic in itself. We refer the interested learners to the PhD thesis by Ryan Braud titled "Query-based debugging of distributed systems."

'''

Points to consider while logging

We should be careful while logging. The logging information should only contain the relevant information and not breach security concerns. For secure data, we should log encrypted data. We should consider the following few points while logging:

- Avoid logging personally identifiable information (PII), such as names, addresses, emails, and so on.
- Avoid logging sensitive information like credit card numbers, passwords, and so on.
- Avoid excessive information. Logging all information is unnecessary. It only takes up more space and affects performance. Logging, being an I/O-heavy operation, has its performance penalties.
- The logging mechanism should be secure and not vulnerable because logs contain the application's flow, and an insecure logging mechanism is vulnerable to hackers.

Vulnerability in logging infrastructure

A zero-day vulnerability in Log4j, a famous logging framework for Java, has been identified recently. Log4j has contained the hidden vulnerability, Log4Shell (CVE-2021-44228), since 2013. Apache gave the highest available score, a CVSS severity rating of 10, to Log4Shell. The exploit is simple to execute and affects hundreds of millions of devices. Security experts are convinced that this vulnerability can allow devastating cyberattacks internationally because it can enable attackers to run malicious code and take control of the machine.

Distributed Logging/System Design Distributed Logging/README.md

System Design: Distributed Logging

Logging

A log file records details of events occurring in a software application. The details may consist of microservices, transactions, service actions, or anything helpful to debug the flow of an event in the system. Logging is crucial to monitor the application's flow.

Need for logging

Logging is essential in understanding the flow of an event in a distributed system. It seems like a tedious task, but upon facing a failure or a security breach, logging helps pinpoint

when and how the system failed or was compromised. It can also aid in finding out the root cause of the failure or breach. It decreases the meantime to repair a system.

Why don't we simply print out our statements to understand the application flow? It's possible but not ideal. Simple print statements have no way of tracking the severity of the message. The output of print functions usually goes to the terminal, while our need could be to persist such data on a local or remote store. Moreover, we can have millions of print statements, so it's better to structure and store them properly.

'''

Mean time to repair (MTTR) is a basic measure of the maintainability of repairable items. It represents the average time required to repair a failed component or device. (Source: Wikipedia)

'''

[Issues with using print statements as an alternative to logging](./printf.jpg)

Concurrent activity by a service running on many nodes might need causality information to stitch together a correct flow of events properly. We must be careful while dealing with causality in a distributed system. We use a logging service to appropriately manage the diagnostic and exploratory data of our distributed software.

Logging allows us to understand our code, locate unforeseen errors, fix the identified errors, and visualize the application's performance. This way, we are aware of how production works, and we know how processes are running in the system.

Log analysis helps us with the following scenarios:

- To troubleshoot applications, nodes, or network issues.
- To adhere to internal security policies, external regulations, and compliance.
- To recognize and respond to data breaches and other security problems.
- To comprehend users' actions for input to a recommender system.

How will we design a distributed logging system?

1. Introduction: We'll discuss how logging works at a distributed level. We'll also show how we can restrict the huge size of a log file, and structure them. This lesson will guide us about the requirements we should consider while logging information about a system.

2. Design: In this lesson, we'll define the requirements, API design, and detailed design of our distributed logging system.

Distributed Messaging Queue/Considerations of a Distributed Messaging Queue's Design/README.md

Considerations of a Distributed Messaging Queue's Design

Before embarking on our journey to design a distributed messaging queue, let's discuss some major factors that could significantly affect the design. These include the order of messages, the effect of the ordering on performance, and the management of concurrent access to the queue. We discuss each of these factors in detail below.

Ordering of messages

A messaging queue is used to receive messages from producers. These messages are consumed by the consumers at their own pace. Some operations are critical in that they require strict ordering of the execution of the tasks, driven by the messages in the queue. For example, while chatting over a messenger application with a friend, the messages should be delivered in order; otherwise, such communication can be confusing, to say the least. Similarly, emails received by a user from different users may not require strict ordering. Therefore, in some cases, the strict order of incoming messages in the queue is essential, while many use cases can tolerate some reordering.

Let's discuss the following two categories of messages ordering in a queue:

- Best-effort ordering
- Strict ordering

```
'''
```

In a queue, the order of messages is implicitly associated with the incoming messages. Once the messages are put in a queue, the same order is followed in the consumption and processing of these messages.

For concurrent producers putting messages in the same queue, the order is not well defined until producers provide order informationâ for example, timestamps or sequence numbers. Without any ordering information, the queue puts messages in the queue in whatever order they arrive at the service.

For concurrent consumers fetching messages from the same queue, ordering can again become a complicated issue. While the queue can hand over messages one after the other in the same order as they were entered in the queue, two consumers almost concurrently processing two messages might need an application-specific ordering mechanism. The queue might help by tagging a messageâ s ordering information, sequence number or timestamp, while handing out a message from the queue.

```
'''
```

Best-effort ordering

With the best-effort ordering approach, the system puts the messages in a specified queue in the same order that theyâ re received.

For example, as shown in the following figure, the producer sends four messages, A, B, C, and D, in the same order as illustrated. Due to network congestion or some other issue, message B is received after message D. Hence, the order of messages is A, C, D, and B at the receiving end. Therefore, in this approach, the messages will be put in the queue in the same order they were received instead of the order in which they were produced on the client side.

[Best-effort ordering: Messages are placed in a queue in the same order that theyâ re received and not in the order they were sent](./ordering.jpg)

Strict ordering

The strict ordering technique preserves the ordering of messages more rigorously. Through this approach, messages are placed in a queue in the order that theyâ re produced.

Before putting messages in a queue in the correct sequence, itâ s crucial to have a mechanism to identify the order in which the messages were produced on the client side. Often, a unique identifier or time-stamp is used to mark a message when itâ s produced.

```
'''
```

Question

Whoâ ll be responsible for providing the sequence numbers?

Answer

The system provides essential libraries or APIs to the client to give sequence numbers to the messages produced at the client side.

```
'''
```

One of the following three approaches can be used for ordering incoming messages:

1. Monotonically increasing numbers: One way to order incoming messages is to assign monotonically increasing numbers to messages on the server side. When the first message arrives, the system assigns it a number, such as 1. It then assigns the number 2 to the second message, and so on.

However, there are potential drawbacks to this approach. First, when a burst of requests is received, it acts as a bottleneck that affects the systemâ s performance because the system has to assign an ID in a specified sequence to a message while the other messages wait for their turn.

Second, it still doesn't tackle the problem that arises when a message is received before the one that's produced earlier at the client side. Because of this, it doesn't guarantee that it will generate the correct order for the messages produced at the client side.

2. Causality-based sorting at the server side: Keeping in view the drawbacks of using monotonically increasing numbers, another approach that can be used for time-stamping and ordering of incoming messages is causality-based sorting. In this approach, messages are sorted based on the time stamp that was produced at the client side and are put in a queue accordingly. The major drawback of this approach is that for multiple client sessions, the service can't determine the order in terms of wall-clock time.

3. Using time stamps based on synchronized clocks: To tackle the potential issues that arise with both of the approaches described above, we can use another appropriate method to assign time stamps to messages that's based on synchronized clocks. In this approach, the time stamp (ID) provided to each message through a synchronized clock is unique and in the correct sequence of production of messages. We can tag a unique process identifier with the time stamp to make the overall message identifier unique and tackle the situation when two concurrent sessions ask for a time stamp at the exact same time. Moreover, with this approach, the server can easily identify delayed messages based on the time stamp and wait for the delayed messages.

As we discussed in the section on the sequencer building block, we can get sequence numbers that fulfill double duty as sequence numbers and globally synchronized wall clock time stamps. Using this approach, our service can globally order messages across client sessions as well.

To conclude, the most appropriate mechanism to provide a unique ID or time stamp to incoming messages, from among the three approaches described above, involves the use of synchronized clocks.

Sorting

Once messages are received at the server side, we need to sort them based on their time stamps. Therefore, we use an appropriate online sorting algorithm for this purpose.

'''

Online sorting means that we need to keep some data sorted as new data keeps coming in.

'''

'''

Question

Suppose that a message sent earlier arrives late due to a network delay. What would be the proper approach to handle such a situation?

Answer

The simple solution in such cases is to reorder the queue. Two scenarios can arise from this. First, reordering puts the messages in the correct order. Second, we've already handed out newer messages to the consumers.

If an old message comes when we've already handed out a newer message, we put it in a special queue and the client handles that situation. If it were a best-effort queue, we could just put such a message at the head of the queue.

'''

Effect on performance

Primarily, a queue is designed for first-in, first-out (FIFO) operations;. First-in, first-out operations suggest that the first message that enters a queue is always handed out first. However, it isn't easy to maintain this strict order in distributed systems. Since message A was produced before message B, it's still uncertain that message A will be consumed before message B. Using monotonically increasing message identifiers or causality-bearing identifiers provide high throughput while putting messages in a queue. Though the need for the online sorting to provide a strict order takes some time before messages are ready for extraction. To minimize latency caused by the online sorting, we use a time-window approach.

Similarly, for strict ordering at the receiving end, we need to serialize all the requests to give out messages one by one. If that's not required, we have better throughput and lower latency at the receiving end.

Due to the reasons mentioned above, many distributed messaging queue solutions either don't guarantee a strict order or have limitations around throughput. As we saw previously, the queues have to perform many additional validations and coordination operations to maintain the order.

'''

time-window: We have to sort messages received within a specific time frame and then put them in the relevant queue (the oldest will be the first to be put in the queue).

'''

Managing concurrency

Concurrent queue access needs proper management. Concurrency can take place at the following stages:

- When multiple messages arrive at the same time.
- When multiple consumers request concurrently for a message.

The first solution is to use the locking mechanism. When a process or thread requests a message, it should acquire a lock for placing or consuming messages from the queue. However, as was discussed earlier, this approach has several drawbacks. It's neither scalable nor performant.

Another solution is to serialize the requests using the system's buffer at both ends of the queue so that incoming messages are placed in an order and consumer processes also receive messages in their arrival sequence. This is a more viable solution because it can help us avoid the occurrence of race conditions.

Applications might use multiple queues with dedicated producers and consumers to keep the ordering cost per queue under check, although this comes at the cost of more complicated application logic.

[Avoiding race conditions: Producers and consumers are serialized at both ends of the queue](./serialization.jpg)

In this lesson, we discussed some key considerations and challenges in the design process of a messaging queue and answered the following questions:

- Why is the order of messages important, and how do we enforce that order?
 - How does ordering affect performance? How do we handle concurrency while accessing a queue?
- Now, we are ready to start designing a distributed messaging queue.

Distributed Messaging Queue/Design of a Distributed Messaging Queue Part 1/README.md

Design of a Distributed Messaging Queue: Part 1

So far, we've discussed the requirements and design considerations of a distributed messaging queue. Now, let's begin with learning about the high-level design of a distributed messaging queue.

Distributed messaging queue

Unlike a single-server messaging queue, a distributed messaging queue resides on multiple servers. A distributed messaging queue has its own challenges. However, it resolves the drawbacks of a single-server messaging queue if designed properly.

The following sections focus on the scalability, availability, and durability issues of designing a distributed messaging queue by introducing us to a more fault-tolerant architecture.

of a messaging queue.

High-level design

Before diving deep into the design, let's assume the following points to make the discussion more simple and easy to understand. In the upcoming material, we discuss how the following assumptions enable us to eliminate the problems in a single-server solution to the messaging queue.

1. Queue data is replicated using either a primary-secondary or quorum-like system inside a cluster (read through the Data Replication lesson for more details). Our service can use data partitioning if the queue gets too long to fit on a server. We can use a consistent hashing-like scheme for this purpose, or we may use a key-value store where the key might be the sequence numbers of the messages. In that case, each shard is appropriately replicated (refer to the Partition lesson for more details on this).
2. We also assume that our system can auto-expand and auto-shrink the resources as per the need to optimally utilize resources.

The following figure demonstrates a high-level design of a distributed messaging queue that's composed of several components.

[High-level architecture of distributed messaging queue](./design.jpg)

The essential components of our design are described in detail below.

Load balancer

The load balancer layer receives requests from producers and consumers, which are forwarded to one of the front-end servers. This layer consists of numerous load balancers. Therefore, requests are accepted with minimal latency and offer high availability.

Front-end service

The front-end service comprises stateless machines distributed across data centers. The frontend provides the following services:

- Request validation: This ensures the validity of a request and checks if it contains all the necessary information.
- Authentication and authorization: This service checks if the requester is a valid user and if these services are authorized for use to a requester.
- Caching: In the front-end cache, metadata information is stored related to the frequently used queues. Along with this, user-related data is also cached here to reduce request time to authentication and authorization services.
- Request dispatching: The frontend is also responsible for calling two other services, the backend and the metadata store. Differentiating calls to both of these services is one of the responsibilities of the frontend.
- Request deduplication: The frontend also tracks information related to all the requests, therefore, it also prevents identical requests from being put in a queue. Deciding what to do about duplicates might be as easy as searching a hash key in a store. If something is found in the store, this implies a duplicate and the message can be rejected.

'''

A question might arise here: how to identify that two (or more) requests are duplicates. There are several ways to identify duplicate messages; however, the most reliable one is to compute the hash (preferably SHA256) of the message content (not the message attributes). This way, if incoming messages have the same content, the service will detect duplication based on their hash.

'''

- Usage data collection: This refers to the collection of real-time data that can be used for audit purposes.

Metadata service

This component is responsible for storing, retrieving, and updating the metadata of queues in the metadata store and cache. Whenever a queue is created or deleted, the metadata store and cache are updated accordingly. The metadata service acts as a middleware between the front-end servers and the data layer. Since the metadata of the queues is kept in the cache, the cache is checked first by the front-end servers for any relevant information related to the receipt of the request. If a cache miss occurs, the information is retrieved from the metadata store and the cache is updated accordingly.

'''

store: A cluster of persistent data storage responsible for storing metadata related to all the queues available in the system.

cache: A cluster of metadata, frequently accessed whenever clients's requests arrive.

'''

There are two different approaches to organizing the metadata cache clusters:

1. If the metadata that needs to be stored is small and can reside on a single machine, then it's replicated on each cluster server. Subsequently, the request can be served from any random server. In this approach, a load balancer can also be introduced between the front-end servers and metadata services.

[A load balancer is introduced between the front-end servers and metadata services](./lb.jpg)

2^a If the metadata that needs to be stored is too large, then one of the following modes can be followed:

- The first strategy is to use the sharding approach to divide data into different shards. Sharding can be performed based on some partition key or hashing techniques, as was discussed in the lesson on database partitioning. Each shard is stored on a different host in the cluster. Moreover, each shard is also replicated on different hosts to enhance availability. In this cluster-organization approach, the front-end server has a mapping table between shards and the hosts. Therefore, the front-end server is responsible for redirecting requests to the host where the data is stored.

[Mapping table resides on the front-end servers](./mapping_table1.jpg)

- The second approach is similar to the first one. However, the mapping table in this approach is stored on each host instead of just on the front-end servers. Because of this, any random host can receive a request and forward it to the host where the data resides. This technique is suitable for read-intensive applications.

[Mapping table resides on each metadata server](./mapping_table2.jpg)

In our discussion on distributed messaging queues, we focused on the high-level design of this type of queue. Furthermore, we explored each component in the high-level design, including the following:

- Front-end servers and the services required of them for smooth operations.
- Load balancers.
- Metadata services.
- Metadata clusters and their organization.

A vital part of the design is the organization of servers at the backend for queue creation, deletion, and other such operations. The next lesson focuses on the organization of back-end servers and the management of queues, along with other important operations related to message delivery and retrieval.

Design of a Distributed Messaging Queue: Part 2

In the previous lesson, we discussed the responsibilities of front-end servers and metadata services. In this lesson, we'll focus on the main part of the design where the queues and messages are stored: the back-end service.

Back-end service

This is the core part of the architecture where major activities take place. When the frontend receives a message, it refers to the metadata service to determine the host where the message needs to be sent. The message is then forwarded to the host and is replicated on the relevant hosts to overcome a possible availability issue. The message replication in a cluster on different hosts can be performed using one of the following two models:

1. Primary-secondary model

2. A cluster of independent hosts

Before delving into the details of these models, let's discuss the two types of cluster managers responsible for queue management: internal and external cluster managers. The differences between these two cluster managers are shown in the following table.

'''

Internal versus External Cluster Managers	Internal Cluster Manager	External Cluster Manager
		It manages the assignment of queues within a cluster.
It manages the assignment of queues across clusters. each and every node within a cluster.	It knows about each cluster. However, it doesn't have information on every host that's present inside a cluster.	It knows about the health of each independent cluster.
It listens to the heartbeat from each node. the health of each independent cluster.	It monitors	
It manages host failure, instance addition, and removals from the cluster. utilizes clusters.	It manages and	
It partitions a queue into several parts and each part gets a primary server. queue across several clusters, so that messages for the same queue are equally distributed between several clusters.	It may split	

'''

Primary-secondary model

In the primary-secondary model, each node is considered a primary host for a collection of queues. The responsibility of a primary host is to receive requests for a particular queue and be fully responsible for data replication. The request is received by the frontend, which in turn communicates with the metadata service to determine the primary host for the request.

For example, suppose we have two queues with the identities 101 and 102 residing on four different hosts A, B, C, and D. In this example, instance B is the primary host of queue 101 and the secondary hosts where the queue 101 is replicated are A and C. As the frontend receives message requests, it identifies the primary server from the internal cluster manager through the metadata service. The message is retrieved from the primary instance, which is also responsible for deleting the original message upon usage and all of its replicas.

As shown in the following illustration, the internal cluster manager is a component that's responsible for mapping between the primary host, secondary hosts, and queues. Moreover, it also helps in the primary host selection. Therefore, it needs to be reliable, scalable, and performant.

[Primary-secondary model of distributed queue: A request is received for a queue with ID 101,

which is served accordingly](./primary-secondary.jpg)

A cluster of independent hosts

In the approach involving a cluster of independent hosts, we have several clusters of multiple independent hosts that are distributed across data centers. As the frontend receives a message, it determines the corresponding cluster via the metadata service from the external cluster manager. The message is then forwarded to a random host in the cluster, which replicates the message in other hosts where the queue is stored.

Question

How does a random host within a cluster replicate dataâ that is, messagesâ in the queues on hosts within the same cluster?

Answer

Each host consists of mapping between the queues and the hosts within a cluster, making the replication easier.

Assume that we have a cluster, say Y, having hosts A, B, and C. This cluster has two queues with IDs 101 and 103 stored on different hosts, as shown in the following table. This table is stored on each host within the cluster Y. When a random host receives a message, say host C, for a queue having ID 103, host C replicates this message on the other hosts where the queue 103 is stored, i.e., Node A and Node B.

[Mapping](./mapping.jpg)

The same process is applied to receive message requests from the consumer. Similar to the first approach, the randomly selected host is responsible for message delivery and cleanup upon a successful processing of the message.

Furthermore, another component called an external cluster manager is introduced, which is accountable for maintaining the mapping between queues and clusters, as shown in the following figure. The external cluster manager is also responsible for queue management and cluster assignment to a particular queue.

The following figure illustrates the cluster of independent hosts. There are two clusters, A and B, which consist of several nodes. The external cluster manager has the mapping table between queues and their corresponding cluster. Whenever a frontend receives a request for a queue, it determines the corresponding cluster for the queue and forwards the request to the cluster where the queue resides. The nodes within that cluster are responsible for storing and sending messages accordingly.

[A cluster of independent hosts that consist of distributed queues](./cluster.jpg)
,,,

Question

What kind of anomalies can arise while replicating messages on other hosts?

Answer

There are two ways to replicate messages in a queue residing on multiple hosts.

1. Synchronous replication
2. Asynchronous replication

In synchronous replication, the primary host is responsible for replicating the message in all the relevant queues on other hosts. After acknowledgment from secondary hosts, the primary host then notifies the client regarding the reception of messages. In this approach, messages remain consistent in all queues replicas; however, it costs extra delay in communication and causes partial to no availability while an election is in progress to promote a secondary as a primary.

In asynchronous replication, once the primary host receives the messages, it acknowledges the client, and in the next step, it starts replicating the message in other hosts. This approach comes with other problems such as replication lag and consistency issues.

Based on the needs of an application, we can pick one or the other.
'''

We have completed the design of a distributed messaging queue and discussed the two models for organizing the back-end server. We also described the management process of queues and how messages are processed at the backend. Furthermore, we discussed how back-end servers are managed through different cluster managers.

In the next lesson, we discuss how our system fulfills the functional and non-functional requirements that were described earlier in the chapter.

Distributed Messaging Queue/Evaluation of a Distributed Messaging Queue's Design/README.m

Evaluation of a Distributed Messaging Queue's Design

We completed the process of designing a distributed messaging queue. Now, let's analyze whether the design met the functional and non-functional requirements of a distributed messaging queue.

Functional requirements

- Queue creation and deletion: When a request for a queue is received at the frontend, the queue is created with all the necessary details provided by the client after undergoing some essential checks. The corresponding cluster manager assigns servers to the newly created queue and updates the information in the metadata stores and caches through a metadata service.

Similarly, the queue is deleted when the client doesn't need it anymore. The responsible cluster manager deallocates the space occupied by the queue and, consequently, deletes the data from all the metadata stores and caches.
'''

Question

How do we handle messages that can't be processed here meaning consumed after maximum processing attempts by the consumer?

Answer

A special type of queue, called a dead-letter queue, can be provided to handle messages that aren't consumed after the maximum number of processing attempts have been made by the consumer. This type of queue is also used for keeping messages that can't be processed successfully due to the following factors:

- The messages intended for a queue that doesn't exist anymore.
- The queue length limit is exceeded, although this would rarely occur with our current design.
- The message expires due to per-message time to live (TTL).

A dead-letter queue is also important for determining the cause of failure and for identifying faults in the system.
'''

- Send and receive messages: Producers can deliver messages to specific queues once they are created. At the backend, receiving messages are sorted based on time stamps to preserve their order and are placed in the queue. Similarly, a consumer can retrieve messages from a specific queue.

When a message is received from a producer for a specific queue, the frontend identifies the primary host or cluster, depending on the replication model, where the queue resides. The request is then forwarded to the corresponding entity and put in the queue.

- Message deletion: Primarily, two options are used to delete a message from a queue.

- The first option is to not delete a message after it's consumed. However, in this case, consumer is responsible for keeping track of what's consumed. For this, we need to maintain order of messages in the queue and keep track of a message within a queue. A job can then delete the message when the expiration conditions are met. Apache Kafka mostly uses this idea where multiple processes can consume a message.

- The second approach also doesn't delete a message after it's consumed. However, it's invisible for some time via an attribute—for example, `visibility_timeout`. This way, the other consumers are unable to get messages that have already been consumed. The message is then deleted by the consumer via an API call.

In both cases, the message being retrieved by the consumer is only deleted by the consumer. The reason behind this is to provide high durability if a consumer can't process a message due to some failure. In such a case, in the absence of a delete call, the consumer can retrieve the message again when it comes back.

Moreover, this approach also provides at-least-once delivery semantic. For example, when a worker fails to process the message, another worker can retrieve the message after it becomes visible in the queue.

'''

at-least-once delivery semantic: In this type of delivery semantic, a message can be delivered more than once. Most distributed systems implement this type of delivery semantic because it is easier to implement. An underlying assumption is that the application will take care of possible redundant deliveries and processing (usually, duplicates do not hurt an application because processing is idempotent).

'''

'''

Question

What happens when the visibility timeout of a specific message expires and the consumer is still busy processing the message?

Answer

The message becomes visible, and another worker can receive the message, thereby duplicating the processing. To avoid such a situation, we ensure that the application sets a safe threshold for visibility timeout.

'''

Non-functional requirements

- **Durability:** To achieve durability, the queue's metadata is replicated on different nodes. Similarly, when a message is received, it's replicated in the queues that reside on different nodes. Therefore, if a node fails, other nodes can be used to deliver or retrieve messages.

- **Scalability:** Our design components, such as front-end servers, metadata servers, caches, back-end clusters, and more are horizontally scalable. We can add to or remove their capacity to match our needs. The scalability can be divided into two dimensions:

- **Increase in the number of messages:** When the number of messages touches a specific limit—say, 80%—the specified queue is expanded. Similarly, the queue is shrunk when the number of messages drops below a certain threshold.

- **Increase in the number of queues:** With an increasing number of queues, the demand for more servers also increases, in which case the cluster manager is responsible for adding extra servers. We commission nodes so that there is performance isolation between different queues. An increased load on one queue shouldn't impact other queues.

- **Availability:** Our data components, metadata and actual messages, are properly replicated inside or outside the data center, and the load balancer routes traffic around failed nodes.

Together, these mechanisms make sure that our system remains available for service under faults.

- Performance: For better performance we use caches, data replication, and partitioning, which reduces the data reads and writes time. Moreover, the best effort ordering strategy for ordering messages is there to use to increase the throughput and lower the latency when it's necessary. In the case of strict ordering, we also suggest time-window based sorting to potentially reduce the latency.

Conclusion

We discussed many subtleties in designing a FIFO queue in a distributed setting. We saw that there is a trade-off between strict message production, message extraction orders, and achievable throughput and latency. Relaxed ordering gives us a higher throughput and lower latency. Asking for strict ordering forces the system to do extra work to enforce wall-clock or causality-based ordering. We use different data stores with appropriate replication and partitioning to scale with data. This design exercise highlights that a construct, a producer-consumer queue, that's simple to realize in a single-OS based system becomes much more difficult in a distributed setting.

Distributed Messaging Queue/Quiz on the Distributed Messaging Queue's Design/README.md

Quiz on the Distributed Messaging Queue's Design

'''

1. What kind of messages do dead-letter queues contain?

A)

The messages that have been consumed successfully.

B)

The messages that failed to be consumed and have reached the maximum attempts limit.

C)

The messages that have been produced by a process that's dead now.

D)

None of the above

'''

'''

2. The purpose of replicating queues on multiple servers is to enhance the _____ of the system.

A)

security

B)

consistency

C)

availability

D)

None of the above

'''

'''

3. In the cluster of independent hosts model, which component is responsible for replicating messages in the other nodes?

- A)
The front-end server
- B)
The cluster manager
- C)
Any random host within the cluster
- D)
None of the above

4. The purpose of a metadata service is _____.

- A)
to store messages in the queues along with their metadata
- B)
to store, retrieve, and update the metadata of queues in the metadata store and cache
- C)
to provide security features in accessing the queues
- D)
None of the above

5. Which of the following is responsible for partitioning the queue and assigning a primary node to each partition?

- A)
The internal cluster manager
- B)
The external cluster manager
- C)
The primary node
- D)
One of the secondary nodes

Answers on the Distributed Messaging Queue's Design

1. What kind of messages do dead-letter queues contain? (B)

- A)
The messages that have been consumed successfully.
- B)
The messages that failed to be consumed and have reached the maximum attempts limit.

Explanation

A dead letter cannot be delivered or returned because it lacks appropriate directions.

Similarly, a queue named after the dead-letter contains those messages that have failed to be consumed after the maximum attempts limit.

C)

The messages that have been produced by a process that's dead now.

D)

None of the above

'''

'''

2. The purpose of replicating queues on multiple servers is to enhance the _____ of the system. (C)

A)

security

B)

consistency

C)

availability

Explanation

The purpose of replicating the queues and corresponding messages on multiple nodes is to increase the system's availability. When one node is down, another stand-by server can serve the request.

D)

None of the above

'''

'''

3. In the cluster of independent hosts model, which component is responsible for replicating messages in the other nodes? (C)

A)

The front-end server

B)

The cluster manager

C)

Any random host within the cluster

Explanation

In the cluster of independent hosts model, a random host can receive the message responsible for replicating messages in other hosts in the respective queues.

D)

None of the above

'''

'''

4. The purpose of a metadata service is _____. (B)

A)

to store messages in the queues along with their metadata

B)

to store, retrieve, and update the metadata of queues in the metadata store and cache

Explanation

Metadata service acts as a middleware between the frontend and metadata storages (and caches). It is responsible for storing, retrieving, and updating the metadata of queues in the metadata stores and caches.

C)
to provide security features in accessing the queues

D)
None of the above
'''

'''

5. Which of the following is responsible for partitioning the queue and assigning a primary node to each partition? (A)

A)
The internal cluster manager

Explanation

The internal cluster manager partitions a queue into multiple parts and assigns a primary node to each partition. In contrast, the external cluster manager assigns clusters to each queue partition.

B)
The external cluster manager

C)
The primary node

D)
One of the secondary nodes
'''

Distributed Messaging Queue/Requirements of a Distributed Messaging Queue's Design/README

Requirements of a Distributed Messaging Queue's Design

Requirements

In a distributed messaging queue, data resides on several machines. Our aim is to design a distributed messaging queue that has the following functional and non-functional requirements.

Functional requirements

- Listed below are the actions that a client should be able to perform:

- Queue creation: The client should be able to create a queue and set some parameters for example, queue name, queue size, and maximum message size.

- Send message: Producer entities should be able to send messages to a queue that's intended for them.

- Receive message: Consumer entities should be able to receive messages from their respective queues.

- Delete message: The consumer processes should be able to delete a message from the queue after a successful processing of the message.

- Queue deletion: Clients should be able to delete a specific queue.
'''

A queue size is the maximum number of messages a queue can contain. For example, Amazon's standard queue size is unlimited. However, for inflight messages, the size is limited to 120,000.

Generally, most popular services like Amazon Simple Queue Service (SQS) and Microsoft Message Queue (MSMQ) allow a maximum size of 256KB and 4MB, respectively.

How do we send messages larger than the predefined size?

Various approaches are used to send messages larger than the specified allowed size. For example, one approach is to divide the message into small chunks at the sender's side and combine it at the receiver's side. However, this approach incurs extra complexity on both the sender and receiver sides.

Amazon SQS provides extended libraries through which a user can send a message containing a reference to a message payload in Amazon S3 by making calls to several APIs, including `AmazonSQSExtendedClient`, `ExtendedClientConfiguration`, and so on. Similarly, Microsoft MSMQ provides an `MQSendLargeMessage` API to send a larger message than 4MB as an XML document.

Therefore, we will overlook this discussion by assuming that our system supports one of the strategies followed by Amazon or Microsoft.

'''

Non-functional requirements

Our design of a distributed messaging queue should adhere to the following non-functional requirements:

- **Durability:** The data received by the system should be durable and shouldn't be lost. Producers and consumers can fail independently, and a queue with data durability is critical to make the whole system work, because other entities are relying on the queue.

- **Scalability:** The system needs to be scalable and capable of handling the increased load, queues, producers, consumers, and the number of messages. Similarly, when the load reduces, the system should be able to shrink the resources accordingly.

- **Availability:** The system should be highly available for receiving and sending messages. It should continue operating uninterrupted, even after the failure of one or more of its components.

- **Performance:** The system should provide high throughput and low latency.

Single-server messaging queue

Before we embark on our journey to map out the design of a distributed messaging queue, we should recall how queues are used within a single server where the producer and consumer processes are also on the same node. A producer or consumer can access a single-server queue by acquiring the locking mechanism to avoid data inconsistency. The queue is considered a critical section where only one entity, either the producer or consumer, can access the data at a time.

However, several aspects restrain us from using the single-server messaging queue in today's distributed systems paradigm. For example, it becomes unavailable to cooperating processes, producers and consumers, in the event of hardware or network failures. Moreover, performance takes a major hit as contention on the lock increases. Furthermore, it is neither scalable nor durable.

'''

Such a queue is also called a bounded queue, due to its fixed size within the system.

'''

[Multiple producers and consumers interact via a single messaging queue](./queue.jpg)

Building blocks we will use

The design of a distributed messaging queue utilizes the following building blocks:

[The building blocks used to design a distributed messaging queue](./bb.jpg)

- Database(s) will be required to store the metadata of queues and users.
- Caches are important to keep frequently accessed data, whether it be data pertaining to users or queues metadata.

- Load balancers are used to direct incoming requests to servers where the metadata is stored. In our discussion on messaging queues, we focused on their functional and non-functional requirements. Before moving on to the process of designing a distributed messaging queue, it's essential for us to discuss some key considerations and challenges that may affect the design.

Distributed Messaging Queue/System Design The Distributed Messaging Queue/README.md

System Design: The Distributed Messaging Queue

What is a messaging queue?

A messaging queue is an intermediate component between the interacting entities known as producers and consumers. The producer produces messages and places them in the queue, while the consumer retrieves the messages from the queue and processes them. There might be multiple producers and consumers interacting with the queue at the same time.

Here is an illustration of two applications interacting via a single messaging queue:

[An example of two applications interacting via a single messaging queue](./mq.jpg)

Motivation

A messaging queue has several advantages and use cases.

- Improved performance: Improved performance: A messaging queue enables asynchronous communication between the two interacting entities, producers and consumers, and eliminates their relative speed difference. A producer puts messages in the queue without waiting for the consumers. Similarly, a consumer processes the messages when they become available. Moreover, queues are often used to separate out slower operations from the critical path and, therefore, help reduce client-perceived latency. For example, instead of waiting for a specific task that's taking a long time to complete, the producer process sends a message, which is kept in the queue if there are multiple requests, for the required task and continues its operations. The consumer can notify us about the fate of the processing, whether a success or failure, by using another queue.

- Better reliability: The separation of interacting entities via a messaging queue makes the system more fault tolerant. For example, a producer or consumer can fail independently without affecting the others and restart later. Moreover, replicating the messaging queue on multiple servers ensures the system's availability if one or more servers are down.

- Granular scalability: Asynchronous communication makes the system more scalable. For example, many processes can communicate via a messaging queue. In addition, when the number of requests increases, we distribute the workload across several consumers. So, an application is in full control to tweak the number of producer or consumer processes according to its current need.

- Easy decoupling: A messaging queue decouples dependencies among different entities in a system. The interacting entities communicate via messages and are kept unaware of each other's internal working mechanisms.

- Rate limiting: Messaging queues also help absorb any load spikes and prevent services from becoming overloaded, acting as a rudimentary form of rate limiting when there is a need to avoid dropping any incoming request.

- Priority queue: Multiple queues can be used to implement different priorities—for example, one queue for each priority—and give more service time to a higher priority queue.

Messaging queue use cases

A messaging queue has many use cases, both in single-server and distributed environments. For example, it can be used for interprocess communication within one operating system. It also enables communication between processes in a distributed environment. Some of the use cases of a messaging queue are discussed below.

1. Sending many emails: Emails are used for numerous purposes, such as sharing information, account verification, resetting passwords, marketing campaigns, and more. All of these emails written for different purposes don't need immediate processing and, therefore, they don't disturb the system's core functionality. A messaging queue can help coordinate a large number of emails between different senders and receivers in such cases.

'''

The core functionality includes writing, formatting, adding relevant objects in emails, and sending and receiving emails.

'''

2. Data post-processing: Many multimedia applications need to process content for different viewer needs, such as for consumption on a mobile phone and a smart television. Oftentimes, applications upload the content into a store and use a messaging queue for post-processing of content offline. Doing this substantially reduces client-perceived latency and enables the service to schedule the offline work at some appropriate time—probably late at night when the compute capacity is less busy.

3. Recommender systems: Some platforms use recommender systems to provide preferred content or information to a user. The recommender system takes the user's historical data, processes it, and predicts relevant content or information. Since this is a time-consuming task, a messaging queue can be incorporated between the recommender system and requesting processes to increase and quicken performance.

How do we design a distributed messaging queue?

We divide the design of a distributed messaging queue into the following five lessons:

1. Requirements: Here, we focus on the functional and non-functional requirements of designing a distributed messaging queue. We also discuss a single server messaging queue and its drawbacks in this lesson.

2. Design consideration: In this lesson, we discuss some important factors that may affect the design of a distributed messaging queue—for example, the order of placing messages in a queue, their extraction, their visibility in the queue, and the concurrency of incoming messages.

3. Design: In this lesson, we discuss the design of a distributed messaging queue in detail. We also describe the process of replication of queues and the interaction between various building blocks involved in the design.

4. Evaluation: In this lesson, we evaluate the design of a distributed messaging queue based on its functional and non-functional requirements.

5. Quiz: At the end of the chapter, we evaluate understanding of the design of a distributed messaging queue via a quiz.

Let's start by understanding the requirements of designing a distributed messaging queue.

Distributed Monitoring/Introduction to Distributed Monitoring/README.md

Introduction to Distributed Monitoring

Need for monitoring

Let's go over how the failure of a single service can affect the smooth execution of related systems. To avoid cascading failures, monitoring can play a vital role with early warnings or steering us to the root cause of faults.

Let's consider a scenario where a user uploads a video, intro-to-system-design, to YouTube. UI service in server A takes the video information and gives the data to service 2 in server B. Service 2 makes an entry in the database and stores the video in blob storage. Another service 3, in server C manages the replication and synchronization of databases X and Y.

In this scenario, service 3 fails due to some error, and service 2 makes an entry in the database X. The database X crashes, and the request to fetch a video is routed to database Y. The user wants to play the video intro-to-system-design, but it will give an error of "Video not found"

[Monitoring](./monitoring)

The example above is relatively simple. In reality, complex problems are encountered since we have many data centers across the globe, and each has millions of servers. Due to a decreasing human administrators to servers ratio, it's often not feasible to manually find the problems. Having a monitoring system reduces operational costs and encourages an automated way to detect failures.

Downtime cost

There are fault-tolerant system designs that hide most of the failures from the end users, but it's crucial to catch the failures before they snowball into a bigger problem. The unplanned outage in services can be costly. For example, in October 2021, Meta's applications were down for nearly nine hours, resulting in a loss of around \$13 million per hour. Such losses emphasize the potential impact of outages.

The IT infrastructure is spread widely around the globe. The illustration below the next paragraph gives an overview of distributed data centers of major cloud providers across the globe, circa 2021. The data centers are connected through private or public networks. Monitoring the servers in geo-separated data centers is essential.

According to Amazon, on December 7, 2021, "At 7:30 AM PST, an automated activity to scale capacity of one of the AWS services hosted in the main AWS network triggered an unexpected behavior from a large number of clients inside the internal network. This resulted in a large surge of connection activity that overwhelmed the networking devices between the internal network and the main AWS network, resulting in communication delays between these networks. These delays increased latency and errors for services communicating between these networks, resulting in even more connection attempts and retries. This led to persistent congestion and performance issues on the devices connecting the two networks." According to one estimate, the outage cost of Amazon was \$66,240 per minute.

[An overview of globally distributed data centers of AWS, Azure, and Google](./overview.jpg)

Types of monitoring

Let's consider an example to understand the types of errors we want to monitor. At Educative whenever a learner connects to an executable environment, a container is assigned. Consider service 1 in server A, which is responsible for allocating a container whenever a learner connects. Another service, 2 on server B takes this information and informs the service responsible for UI. The UI service running in server C updates the UI for the learner. Let's assume that service 2 fails because of some error, and the learner sees the error of "Cannot connect"

How do the Educative developers find out that a learner is facing this error?

[Types](./types)

Now, what if a learner makes a request and it never reaches the servers of Educative. How will Educative know that a learner is facing an issue?

With the above examples, we can divide our monitoring focus into two broad categories of errors:

- Service-side errors: These are errors that are usually visible to monitoring services as they occur on servers. Such errors are reported as error 5xx in HTTP response codes.
- Client-side errors: These are errors whose root cause is on the client-side. Such errors are reported as error 4xx in HTTP response codes. Some client-side errors are invisible to the

service when client requests fail to reach the service.

Weâ ll explore how to design a monitoring service to handle both scenarios in the upcoming chapter Monitoring Server-side Errors and Monitoring Client-side Errors. We want our monitoring systems to analyze our globally distributed services. It allows a better understanding of the systemâ s components and agility to detect and respond to faults.

Distributed Monitoring/Prerequisites of a Monitoring System/README.md

Prerequisites of a Monitoring System

Monitoring: metrics and alerting

A good monitoring system needs to clearly define what to measure and in what units (metrics). The monitoring system also needs to define threshold values of all metrics and the ability to inform appropriate stakeholders (alerts) when values are out of acceptable ranges. Knowing the state of our infrastructure and systems ensures service stability. The support team can respond to issues more quickly and confidently if they have access to information on the health and performance of the deployments. Monitoring systems that collect measurements, show data, and send warnings when something appears wrong are helpful for the support team.

To further understand metrics, alerts, and their connection with monitoring, weâ ll go over their significance, their potential benefits, and the data we might want to keep track of.

'''

Question

What are the conventional approaches to handle failures in IT infrastructure?

Answer

The two conventional approaches to handling IT infrastructure failures are the reactive and proactive approaches.

In the reactive approach, corrective action is taken after the failure occurs. In this approach, even if DevOps takes quick action to find the cause of the error and promptly handle the failures, it causes downtime. As a result, there will be system downtime in the reactive approach, which is generally undesirable for continuously running applications.

In a proactive approach, proactive actions are taken before failure occurs. Therefore, it prevents downtimes and associated losses. The proactive approach works on predicting system failures to take corrective action to avoid the failure. This approach offers better reliability by preventing downtime.

In modern services, completely avoiding problems is not possible. Something is always failing inside huge data centers and network deployments. The goal is to find the impending problems early on and design systems in such a way that service faults are invisible to the end users.

'''

Metrics

Metrics objectively define what we should measure and what units will be appropriate. Metric values provide an insight into the system at any point in time. For example, a web serverâ s ability to handle a certain amount of traffic per second or its ability to join a pool of web servers are examples of high-level data correlated with a componentâ s specific purpose or activity. Another example can be measuring network performance in terms of throughput (megabits per second) and latency (round-trip time). We need to collect values of metrics with minimal performance penalty. We may use user-perceived latency or the amount of computational resources to measure this penalty.

Values that track how many physical resources our operating system uses can be a good starting point. If we have a monitoring system in place, we donâ t have to do much additional work to data regarding processor load, CPU statistics like cache hits and misses, RAM usage by OS and processes, page faults, disc space, disc read and write latencies, swap space usage, and so on. Metrics provided by many web servers, database servers, and other software help us determine whether everything is running smoothly or not.

'''

Note: Connect to the following terminal to see details about the CPU utilization of processes that are currently active on the virtual machine.

'''

We use the top command to view Linux processes. Running this command opens an interactive view of the running system containing a summary of the system and a list of processes or threads. The default view has the following:

- On the top, we can see how long the machine has been turned on, how many users are logged in and the average load on the machine for the past few minutes.
- On the next line, we can see the state (running, sleeping, or stopped) of tasks running on the machine.
- Next, we have the CPU consumption values.
- Lastly, we have an overview of physical memory how much of it is free, used, buffered, or available.

Now, let's take the following steps to see a change in the CPU usage:

Populate the metrics

The metrics should be logically centralized for global monitoring and alerting purposes. Fetching metrics is crucial to the monitoring system. Metrics can either be pushed or pulled into a monitoring system, depending on the preference of the user.

Here, we confront a fundamental design challenge now: Do we utilize push or pull? Should the server proactively send the metrics' values out, or should it only expose an endpoint and wait reactively for an inquiry?

In pull strategy, each monitored server merely needs to store the metrics in memory and send them to an exposed endpoint. The exposed endpoint allows the monitoring application to fetch the metrics itself. Servers sending too much data or sending data too frequently can't overload the monitoring system. The monitoring system will pull data as per its own schedule.

In other situations, though, pushing may be beneficial, such as when a firewall prevents the monitoring system from accessing the server directly. The monitoring system has the ability to adjust a global configuration about the data to be collected and the interval at which servers and switches should push the data.

Push and pull terminologies might be confusing. Whenever we discuss push or pull strategy, we'll consider it from the monitoring system's perspective. That is, either the system will pull the metrics values from the applications, or the metrics will be pushed to the monitoring system. To avoid confusion, we'll stick to the monitoring system's viewpoint.

'''

Question

Logging is the act of keeping records of events in a software system. How does it help in monitoring?

Answer

In logging, the application servers log the information into the file. The information can be CPU usage, application-related information, and other relevant properties that we deem necessary to backtrace or debug a file when a problem is encountered. We can populate our metrics based on the values logged in the logs.

Logs and metrics both help in monitoring a service. But this isn't always true since processing the log information takes time. In real time, we need to act swiftly for early detection of issues. So, logging is also one of the inputs of metrics. Logging is just a mechanism to

collect information, and the monitoring system can use it for collecting necessary information.

Logging can also help to temporarily keep the data on a server to absorb any momentary data spikes or to decouple data generation and monitoring systems.

'''

'''

Note: At times, we use the word `metrics` when we should have used `metric values`. we can figure out which of them is being referred to through the context they're being used.

'''

Persist the data

Figuring out how to store the metrics from the servers that are being monitored is important. A centralized in-memory metrics repository may be all that's needed. However, for a large data center with millions of things to monitor, there will be an enormous amount of data to store, and a time-series database can help in this regard.

Time-series databases help maintain durability, which is an important factor. Without a historical view of events in a monitoring system, it isn't very useful. Samples having a value of time stamp are stored in chronological sequence. So, a whole metric's timeline can be shown in the form of a time series.

'''

Time series data is continuous, and a time-series database (TSDB) allows storing data associated with timestamps. Using TSDB is an efficient method of storing and retrieving massive amounts of timestamped data.

'''

Application metrics

We may need to add code or APIs to expose metrics we care about for other components, notably our own applications. We embed logging or monitoring code in our applications, called code instrumentation, to collect information of interest.

Looking at metrics as a whole can shed light on how our systems are performing and how healthy they are. Monitoring systems employ these inputs to generate a comprehensive view of our environment, automate responses to changes like commissioning more EC2 instances if the application's traffic increases, and warn humans when necessary. Metrics are system measurements that allow analyzing historical trends, correlations, and changes in the performance, consumption, or error rates.

Alerting

Alerting is the part of a monitoring system that responds to changes in metric values and takes action. There are two components to an alert definition: a metrics-based condition or threshold, and an action to take when the values fall outside of the permitted range.

Distributed Monitoring/System Design Distributed Monitoring/README.md

System Design: Distributed Monitoring

Monitoring

The modern economy depends on the continual operation of IT infrastructure. Such infrastructure contains hardware, distributed services, and network resources. These components are interlinked in such infrastructure, making it challenging to keep everything functioning smoothly and without application downtime.

It's challenging to know what's happening at the hardware or application level when our infrastructure is distributed across multiple locations and includes many servers. Components can run into failures, response latency overshoot, overloaded or unreachable hardware, and containers running out of resources, among others. Multiple services are running in such an infrastructure, and anything can go awry.

When one of the services goes down, it can be the reason for other services to crash, and as a result, the application is unavailable to users. If we don't know what went wrong early, it

could take us a lot of time and effort to debug the system manually. Moreover, for larger services, we need to ensure that our services are working within our agreed service-level agreements. We need to catch important trends and signals of impending failures as early warnings so that any concerns or issues can be addressed.

Monitoring helps in analyzing such complex infrastructure where something is constantly failing. Monitoring distributed systems entails gathering, interpreting, and displaying data about the interactions between processes that are running at the same time. It assists in debugging, testing, performance evaluation, and having a bird's-eye view over multiple services.

How will we design a distributed monitoring system?

We've divided the distributed monitoring system design into the following chapters and lessons.

1. Distributed Monitoring

- Introduction to Distributed Monitoring: Learn why monitoring in a distributed system is crucial, how costly downtime is, and the types of monitoring.
- Prerequisites for a Monitoring System: Explore a few essential concepts about metrics and alerting in a monitoring system.

2. Monitoring Server-side Errors

- Designing a Monitoring System: Define the requirements and high-level design of the monitoring system.
- A Detailed Design of the Monitoring System: Go into the details of designing a monitoring system, and explore the components involved.
- Visualize Data in a Monitoring System: Learn a unique way to visualize an enormous amount of monitoring data.

3. Monitor Client-side Errors

- Focus on Client-side Errors: Get introduced to client-side errors and why it's important to monitor them.
- Design a Client-side Monitoring System: Learn to design a system that monitors the client-side errors.

In the next lesson, we'll look at why monitoring is essential in a distributed system through an example. We'll also look at the downtime cost of failures and monitoring types.

Distributed Search/Design of a Distributed Search/README.md

Design of a Distributed Search

High-level design

Let's shape the overall design of a distributed search system before getting into a detailed discussion. There are two phases of such a system, as shown in the illustration below. The offline phase involves data crawling and indexing in which the user has to do nothing. The online phase consists of searching for results against the search query by the user.

[High-level design of a distributed search system](./design.jpg)

- The crawler collects content from the intended resource. For example, if we build a search for a YouTube application, the crawler will crawl through all of the videos on YouTube and extract textual content for each video. The content could be the title of the video, its description, the channel name, or maybe even the video's annotation to enable an intelligent search based not only on the title and description but also on the content of that video. The crawler formats the extracted content for each video in a JSON document and stores these JSON documents in a distributed storage.
- The indexer fetches the documents from a distributed storage and indexes these documents using MapReduce, which runs on a distributed cluster of commodity machines. The indexer uses a distributed data processing system like MapReduce for parallel and distributed index construction. The constructed index table is stored in the distributed storage.
- The distributed storage is used to store the documents and the index.
- The user enters the search string that contains multiple words in the search bar.
- The searcher parses the search string, searches for the mappings from the index that are stored in the distributed storage, and returns the most matched results to the user. The

searcher intelligently maps the incorrectly spelled words in the search string to the closest vocabulary words. It also looks for the documents that include all the words and ranks them.

'''

MapReduce: As stated by Wikipedia, â MapReduce is a programming model and an associated implementation for processing and generating big data sets with a parallel, distributed algorithm on a cluster of commodity machines.â

'''

'''

The index table consists of terms and their mappings.

'''

API design

Since the user only sends requests in the form of a string, the API design is quite simple.

Search: The search function runs when a user queries the system to find some content.

'''

search(query)

'''

'''

Parameter	Description
-----------	-------------

query	This is the textual query entered by the user in the search bar, based on which the results are found.
-------	--

'''

Detailed discussion

Since the indexer is the core component in a search system, we discussed an indexing technique and the problems associated with centralized indexing in the previous lesson. In this lesson, we consider a distributed solution for indexing and searching.

Distributed indexing and searching

Letâ s see how we can develop a distributed indexing and searching system. We understand that the input to an indexing system is the documents we created during crawling. To develop an index in a distributed fashion, we employ a large number of low-cost machines (nodes) and partition or divide the documents based on the resources they have. All the nodes are connected. A group of nodes is called a cluster.

'''

For performing distributed indexing, the machines in the cluster typically have dual-processor x86 processors running Linux, with 2â 4 GB of memory per machine. It is not necessary that all machines are of the same specification, though they should be somewhat comparable. The MapReduce framework is smart enough to give more work to stronger machines.

'''

We use numerous small nodes for indexing to achieve cost efficiency. This process requires us to partition or split the input data (documents) among these nodes. However, a key question needs to be addressed: How do we perform this partitioning?

The two most common techniques used for data partitioning in distributed indexing are these below:

- Document partitioning: In document partitioning, all the documents collected by the web crawler are partitioned into subsets of documents. Each node then performs indexing on a subset of documents that are assigned to it.
- Term partitioning: The dictionary of all terms is partitioned into subsets, with each subset residing at a single node. For example, a subset of documents is processed and indexed by a node containing the term â search.â

[Types of data partitioning in a distributed search](./types.jpg)

In term partitioning, a search query is sent to the nodes that correspond to the query terms. This provides more concurrency because a stream of search queries with different query terms will be served by different nodes. However, term partitioning turns out to be a difficult task in practice. Multiword queries necessitate sending long mapping lists between groups of nodes for merging, which can be more expensive than the benefits from the increased concurrency.

'''

Mapping lists: A list of documents in which the term appeared with the frequency of the term in each document.

'''

In document partitioning, each query is distributed across all nodes, and the results from these nodes are merged before being shown to the user. This method of partitioning necessitates less inter-node communication. In our design, we use document partitioning.

Following document partitioning, let's look into a distributed design for index construction and querying, which is shown in the illustration below. We use a cluster that consists of a number of low-cost nodes and a cluster manager. The cluster manager uses a MapReduce programming model to parallelize the index's computation on each partition. MapReduce can work on significantly larger datasets that are difficult to be handled by a single large server.

[Distributed indexing and searching in a parallel fashion on multiple nodes in a cluster of commodity machines](./search_and_index.jpg)

The system described above works as follows:

****Indexing****

- We have a document set already collected by the crawler.
- The cluster manager splits the input document set into N number of partitions, where N is equal to three in the illustration above. The size of each partition is decided by the cluster manager given the size of the data, the computation, memory limits, and the number of nodes in the cluster. All the nodes may not be available for various reasons. The cluster manager monitors the health of each node through periodic heartbeats. To assign a document to one of the N partitions, a hashing function can be utilized.
- After making partitions, the cluster manager runs indexing algorithms for all the N partitions simultaneously on the N number of nodes in a cluster. Each indexing process produces a tiny inverted index, which is stored on the node's local storage. In this way, we produce tiny inverted indices rather than one large inverted index.

****Searching****

- In the search phase, when a user query comes in, we run parallel searches on each tiny inverted index stored on the nodes' local storage generating N queries.
- The search result from each inverted tiny index is a mapping list against the queried term (we assume a single word/term user query). The merger aggregates these mapping lists.
- After aggregating the mapping lists, the merger sorts the list of documents from the aggregated mapping list based on the frequency of the term in each document.
- The sorted list of documents is returned to the user as a search result. The documents are shown in sorted (ascending) order to the user.

'''

Note: We've designed a search system where we utilized a distributed system and parallelized the indexing and searching process. This helped us handle large datasets by working on the smaller partitions of documents. It should be noted that both searching and indexing are performed on the same node. We refer to this idea as colocation.

'''

The proposed design works, and we can replicate it across the globe in various data centers to facilitate all users. Thus, we can achieve the following advantages:

- Our design will not be subject to a single point of failure (SPOF).
- Latency for all users will remain small.
- Maintenance and upgrades in individual data centers will be possible.
- Scalability (serving more users per second) of our system will be improved.

Replication

We make replicas of the indexing nodes that produce inverted indices for the assigned partitions. We can answer a query from several sets of nodes with replicas. The overall concept is simple. We continue to use the same architecture as before, but instead of having only one group of nodes, we have R groups of nodes to answer user queries. R is the number of replicas. The number of replicas can expand or shrink based on the number of requests, and each group of nodes has all the partitions required to answer each query.

Each group of nodes is hosted on different availability zones for better performance and availability of the system in case a data center fails.

'''

Note: A load balancer component is necessary to spread the queries across different groups of nodes and retry in case of any error.

'''

'''

Availability zones (AZs) are isolated locations within data center regions where public cloud services are launched and operated. Regions are geographic locations where data centers for public cloud service providers are located.

'''

Replication factor and replica distribution

Generally, a replication factor of three is enough. A replication factor of three means three nodes host the same partition and produce the index. One of the three nodes becomes the primary node, while the other two are replicas. Each of these nodes produces indexes in the same order to converge on the same state.

To illustrate, let's divide the data, a document set, into four partitions. Since the replication factor is three, one partition will be hosted by three nodes. We'll assume that there are two availability zones (AZ1 and AZ2). And in each availability zone, we have two nodes. Each node acts as a primary for only one partition (For example, Node 1 in AZ1 is the primary node for partition P1). The three copies (pink, blue, and purple) for a partition are shared between the two AZ

AZ instances so that two copies are in one zone and the third copy is in another zone. Three colors represent three replicas of each partition. For example, the following is true for partition P4:

- The first replica, represented by the color pink, is placed in Node 2 of AZ2
- The second replica, represented by the color blue, is placed in Node 1 of AZ2
- The third replica, represented by the color purple, is placed in Node 2 of AZ1

Each group in the illustration below consists of one replica from all of the four partitions (P1, P2, P3, P4)

[The replica distribution: Each node contains one primary partition and two replicas](./replica_distribution.jpg)

In the above illustration, the primary replica for P_1 is indicated by the dark purple color, the primary replica for P_2 is represented by the dark blue color, and the primary replica for P_3 and P_4 is represented by the dark pink color.

Now that we have completed replication, let's see how indexing and searching are performed in these replicas.

****Indexing with replicas****

From the diagram above, we assume that each partition is forwarded to each replica for index computation. Let's look at the example where we want to index partition P_1. This means that the same partition will be forwarded to all three replicas in both availability zones. Therefore, each node will compute the index simultaneously and reach the same state.

The advantage of this strategy is that the indexing operation will not suffer if the primary node fails.

Searching with replicas

We have three copies of each partition's index. The load balancer chooses one of the three copies of each partition to perform the query. An increased number of copies improves the scalability and availability of the system. Now, the system can handle three times more queries in the same amount of time.

Summary

In this lesson, we learned how to handle a large number of data, and a large number of queries with these strategies:

- Parallel indexing and searching, where both of these processes are colocated on the same nodes.
- Replicating each partition, which means that we replicate the indexing and searching processes as well.

We successfully designed a system that scales with read (search) and write (indexing) operations colocated on the same node. But, this scaling method brings some drawbacks. We'll look into the drawbacks and their solutions in the next lesson.

Distributed Search/Evaluation of a Distributed Search's Design/README.md

Evaluation of a Distributed Search's Design

Availability

We utilized distributed storage to store these items:

- Documents crawled by the indexer.
- Inverted indexes generated by the indexing nodes.

Data is replicated across multiple regions in distributed storage, making cross-region deployment for indexing and search easier. The group of indexing and search nodes merely needs to be replicated in different availability zones. Therefore, we deploy the cluster of indexing and search nodes in different availability zones. So, if a failure occurs in one place, we can process the requests from another cluster. Multiple groups of indexing and search nodes help to achieve high indexing and search availability. Moreover, in each cluster, if a node dies, another can take its place.

The indexing is performed offline, not on the user's critical path. We don't need to replicate the indexing operations synchronously. It is unnecessary to respond to the user search queries with the latest data that has just been added to the index. So, we don't have to wait for the replication of the new index to respond to the search queries. This makes the search available to the users.

'''

Note: Once we replicate the latest data in all groups of indexing nodes and the search nodes have downloaded it, then the search queries are performed on the latest data.

[Guarantees provided by the distributed search](./evaluation.jpg)

Scalability

Partitioning is an essential component of search systems to scale. When we increase the number of partitions and add more nodes to the indexing and search clusters, we can scale in terms of data indexing and querying.

The strong isolation of indexing and search processes help indexing and search scale independently and dynamically.

Fast search on big data

We utilized a number of nodes, each of which performs search queries in parallel on smaller inverted indices. The result from each search node is then merged and returned to the user.

Reduced cost

We used cheaper machines to compute indexes and perform searches. If one node fails, we don't have to recompute the complete index. Instead, some of the documents need to be indexed again.

Conclusion

A search system is required for almost every application. We have seen that it isn't possible to develop a search system that can run on a single node. We utilized a parallel computation framework and low-cost machines to build a search system that is available, scalable, and highly performant.

Distributed Search/Indexing in a Distributed Search/README.md

Indexing in a Distributed Search

We'll first describe what indexing is, and then we'll make our way toward distributing index over many nodes.

Indexing

Indexing is the organization and manipulation of data that's done to facilitate fast and accurate information retrieval.

Build a searchable index

The simplest way to build a searchable index is to assign a unique ID to each document and store it in a database table, as shown in the following table. The first column in the table is the ID of the text and the second column contains the text from each document.

'''

Simple Document Index

ID	Document Content
----	------------------

1	Elasticsearch is the distributed and analytics engine that is based on REST APIs.
---	---

2	Elasticsearch is a Lucene library-based search engine.
---	--

3	Elasticsearch is a distributed search and analytics engine built on Apache Lucene.
---	--

'''

The size of the table given above would vary, depending on the number of documents we have and the size of those documents. The table above is just an example, and the content from each document only consists of one or two sentences. With an actual, real-world example, the content of every document in the table could be pages long. This would make our table quite large. Running a search query on the document-level index given above isn't a fast process. On each

search request, we have to traverse all the documents and count the occurrence of the search string in each document.

'''

Note: For a fuzzy search, we also have to perform different pattern-matching queries. Many strings in the documents would somehow match the searched string. First, we must find the unique candidate strings by traversing all of the documents. Then, we must single out the most approximate matched string out of these strings. We also have to find the occurrence of the most matched string in each document. This means that each search query takes a long time.

'''

'''

Fuzzy search: This type of search uses approximate string matching rather than exact matching to match the results against the search term.

'''

The response time to a search query depends on a few factors:

- The data organization strategy in the database.
- The size of the data.
- The processing speed and the RAM of the machine that's used to build the index and process the search query.

Running search queries on billions of documents that are document-level indexed will be a slow process, which may take many minutes, or even hours. Let's look at another data organization and processing technique that helps reduce the search time.

Inverted index

An inverted index is a HashMap-like data structure that employs a document-term matrix. Instead of storing the complete document as it is, it splits the documents into individual words. After this, the document-term matrix identifies unique words and discards frequently occurring words like "to," "they," "the," "is," and so on. Frequently occurring words like these are the terms. The document-term matrix maintains a term-level index through this identification of unique words and deletion of unnecessary terms.

For each term, the index computes the following information:

- The list of documents in which the term appeared.
- The frequency with which the term appears in each document.
- The position of the term in each document.

'''

Inverted Index

Term	Mapping ([doc], [freq], [[loc]])
elasticsearch	([1, 2, 3], [1, 1, 1], [[1], [1], [1]])
distributed	([1, 3], [1, 1], [[4], [4]])
restful	([1], [1], [[5]])
search	([1, 2, 3], [1, 1, 1], [[6], [4], [5]])
analytics	([1, 3], [1, 1], [[8], [7]])
engine	([1, 2, 3], [1, 1, 1], [[9], [5], [8]])
heart	([1], [1], [[12]])
elastic	([1], [1], [[15]])

```

stack          ( [1], [1], [[16]] )
lucene         ( [2, 3], [1, 1], [[9], [12]] )
library        ( [2], [1], [[10]] )
Apache         ( [3], [1], [[11]] )
'''

```

In the table above, the `Term` column contains all the unique terms that are extracted from the documents. Each entry in the `Mapping` column consists of three lists:

- A list of documents in which the term appeared.
- A list that counts the frequency with which the term appears in each document.
- A two-dimensional list that pinpoints the position of the term in each document. A term can appear multiple times in a single document, which is why a two-dimensional list is used.

A document-term matrix is a mathematical matrix that represents the frequency of terms in a list of documents.

'''

Note: Instead of lists, the mappings could also be in the form of tuples such as doc, freq, and loc.

'''

Inverted index is one of the most popular index mechanisms used in document retrieval. It enables efficient implementation of boolean, extended boolean, proximity, relevance, and many other types of search algorithms.

****Advantages of using an inverted index****

- An inverted index facilitates full-text searches.
- An inverted index reduces the time of counting the occurrence of a word in each document at the run time because we have mappings against each term.

Disadvantages of using an inverted index

- There is storage overhead for maintaining the inverted index along with the actual documents. However, we reduce the search time.

- Maintenance costs (processing) on adding, updating, or deleting a document. To add a document, we extract terms from the document. Then, for each extracted term, we either add a new row in the inverted index or update an existing one if that term already has an entry in the inverted index. Similarly, for deleting a document, we conduct processing to find the entries in the inverted index for the deleted document's terms and update the inverted index accordingly.

'''

Boolean searching is based on a symbolic logic method established by George Boole, an English mathematician. It allows you to limit, broaden, or define your search by combining words and phrases with the words AND, OR, and NOT (also known as Boolean operators). Source: Wikipedia

Extended Boolean search overcomes the disadvantages of the Boolean search. The main disadvantage of the boolean search is that it doesn't incorporate term weights in boolean queries, producing either too small or too big result set. Extended boolean search makes use of partial matching and term weights. It assigns a score to the similarity between queries and documents. This way, a document may be relevant if it matches part of the searched terms and is returned as a result, whereas it was not in the Boolean search. Source: Wikipedia)

The proximity search finds documents wherein two or more term occurrences that are separately matching are within a certain distance, where distance is the number of intermediate words or characters. Source: Wikipedia)

Relevance search measures the accuracy of the relationship between the search query and the search results. Source: Wikipedia
'''

'''

Full-text search draws attention to searching for text within large amounts of electronically recorded text data and delivers results that include some or all of the words in the query. Traditional search, on the other hand, would provide exact matches.
'''

****Disadvantages of using an inverted index****

- There is storage overhead for maintaining the inverted index along with the actual documents. However, we reduce the search time.
- Maintenance costs (processing) on adding, updating, or deleting a document. To add a document, we extract terms from the document. Then, for each extracted term, we either add a new row in the inverted index or update an existing one if that term already has an entry in the inverted index. Similarly, for deleting a document, we conduct processing to find the entries in the inverted index for the deleted document's terms and update the inverted index accordingly.

Searching from an inverted index

Consider a system that has the following mappings when we search for the word "search engine"

Term	Mapping
search	([1, 2, 3], [1, 1, 1], [[6], [4], [5]])
engine	([1, 2, 3], [1, 1, 1], [[9], [5], [8]])

Both of these words are found in documents 1, 2, and 3. Both words appear once in each document.

The word "search" is located at position 6 in document 1, at position 4 in document 2, and position 5 in document 3.

The word "engine" is located at position 9 in document 1, position 5 in document 2, and position 8 in document 3.

A single term can appear in millions of documents. Thus, the list of documents returned against a search query could be very long.

'''

Question

Would this technique work when too many documents are found against a single term?

Answer

It probably wouldn't work to return all the documents that are found. Instead, we should sort them based on the relevance to the search query. The top results should be returned to the user, instead of returning all the documents.
'''

Factors of index design

Here are some of the factors that we should keep in mind while designing an index:

- Size of the index: How much computer memory, and RAM, is required to keep the index. We keep the index in the RAM to support the low latency of the search.
- Search speed: How quickly we can find a word from an inverted index.

- Maintenance of the index: How efficiently the index can be updated if we add or remove a document.
- Fault tolerance: How critical it is for the service to remain reliable. Coping with index corruption, supporting whether invalid data can be treated in isolation, - dealing with defective hardware, partitioning, and replication are all issues to consider here.
- Resilience: How resilient the system is against someone trying to game the system and guard against search engine optimization (SEO) schemes, since we return only a handful of relevant results against a search.

In light of the design factors listed above, let's look at some problems with building an index on a centralized system.

'''

Invalid data: The data that makes no sense or doesn't exist can affect the performance of our search when added to the index. We have to detect such data before adding it to the index.

'''

Indexing on a centralized system

In a centralized search system, all the search system components run on a single node, which is computationally quite capable. The architecture of a centralized search system is shown in the following illustration:

[The architecture of a centralized search system](./arch.jpg)

- The indexing process takes the documents as input and converts them into an inverted index, which is stored in the form of a binary file.
- The query processing or search process interprets the binary file that contains the inverted index. It also computes the intersection of the inverted lists for a given query to return the search results against the query.

These are the problems that come with the architecture of a centralized search system:

- SPOF (single point of failure)
- Server overload
- Large size of the index

SPOF: A centralized system is a single point of failure. If it's dead, no search operation can be performed.

Server overload: If numerous users perform queries and the queries are complicated, it stresses the server (node).

Large size of the index: The size of the inverted index increases with the number of documents, placing resource demands on a single server. The bigger the computer system, the higher the cost and complexity of managing it.

'''

Note: With a distributed system, low-cost computer systems are utilized, which is cost effective overall.

'''

An inverted index needs to be loaded into the main memory when adding a document or running a search query. A large portion of the inverted index must fit into the RAM of the machine for efficiency.

According to Google analytics in 2022, there are hundreds of billions of web pages, the total size of which is around 100 petabytes. If we make a search system for the worldwide web, the inverted index size will also be in petabytes. This means we have to load petabytes of data into the RAM. It's impractical and inefficient to increase the resources of a single machine for indexing a billion pages instead of shifting to a distributed system and utilizing the

power of parallelization. Running a search query on a single, large inverted index results in slow response time.

'''

Note: Searching a book from a shelf that holds a hundred books is easier than searching a book from a shelf holding a million books. The search time increases with the volume of data we search from.

'''

Attacks on centralized indexing can have a higher impact than attacks on a distributed indexing system. Furthermore, the odds of bottlenecks (which can arise in server bandwidth or RAM) are also lower in a distributed index.

In this lesson, we learned about indexing, and we looked into the problems of indexing on a centralized system. The next lesson presents a distributed solution for indexing.

Distributed Search/Requirements of a Distributed Search System's Design/README.md

Requirements of a Distributed Search System's Design

Requirements

Let's understand the functional and non-functional requirements of a distributed search system.

Functional requirements

The following is a functional requirement of a distributed search system:

- Search: Users should get relevant content based on their search queries.

[The functional requirement of a distributed search system](./fr.jpg)

Non-functional requirements

Here are the non-functional requirements of a distributed search system:

- Availability: The system should be highly available to the users.

- Scalability: The system should have the ability to scale with the increasing amount of data. In other words, it should be able to index a large amount of data.

- Fast search on big data: The user should get the results quickly, no matter how much content they are searching.

- Reduced cost: The overall cost of building a search system should be less.

[The non-functional requirement of a distributed search system](./nfr.jpg)

Resource estimation

Let's estimate the total number of servers, storage, and bandwidth that is required by the distributed search system. We'll calculate these numbers using an example of a YouTube search.

Number of servers estimation

To estimate the number of servers, we need to know how many daily active users per day are using the search feature on YouTube and how many requests per second our single server can handle. We assume the following numbers:

- The number of daily active users who use the search feature is three million.

- The number of requests a single server can handle is 1,000.

The number of servers required is calculated using this formula:

Number of active users/queries handled per server = 3K servers

If three million users are searching concurrently, three million search requests are being generated at one time. A single server handles 1,000 requests at a time. Dividing three million by 1,000 gives us 3,000 servers.

[The number of servers required for the YouTube search service](./number_of_servers.jpg)

Storage estimation

Each video's metadata is stored in a separate JSON document. Each document is uniquely identified by the video ID. This metadata contains the title of the video, its description, the channel name, and a transcript. We assume the following numbers for estimating the storage required to index one video:

- The size of a single JSON document is 200 KB.
- The number of unique terms or keys extracted from a single JSON document is 1,000.
- The amount of storage space required to add one term into the index table is 100 Bytes.

The following formula is used to compute the storage required to index one video:

Total storage/video = Storage /doc +(Terms /doc ^ Storage /term
'''

Total Storage Required to Index One Video on YouTube		
Storage per JSON doc (KB) (Bytes)	No. of terms per doc	Storage per term
200	1000	100
300		

'''

In the table above, we calculate the storage required to index one video. We have already seen that the total storage required per video is 300 KB. Assuming that, on average, the number of videos uploaded per day on YouTube is 6,000, let's calculate the total storage required to index the videos uploaded per day. The following formula is used to compute the storage required to index the videos uploaded to YouTube in one day:

Total storage/day =No. of videos /day ^ Total storage/video

'''

Total Storage Required to Index Videos per Day on YouTube		
No. of videos per day day(GB)	Total storage per video (KB)	Total storage per
6000	300	1.8

'''

The total storage required to index 6,000 videos uploaded per day on YouTube is 1.8 GB. This storage requirement is just an estimation for YouTube. The storage need will increase if we provide a distributed search system as a service to multiple tenants.

[Summarizing the storage requirement of a distributed search system for videos uploaded to YouTube per day](./daily_storage.jpg)

Bandwidth estimation

The data is transferred between the user and the server on each search request. We estimate the bandwidth required for the incoming traffic on the server and the outgoing traffic from the server. Here is the formula to calculate the required bandwidth:

Total bandwidth =Total requests_second ^ Total query_size

****Incoming traffic****

To estimate the incoming traffic bandwidth, we assume the following numbers:

- The number of search requests per day is 150 million.
- The search query size is 100 Bytes.

We can use the formula given above to calculate the bandwidth required for the incoming

traffic.

'''

Bandwidth Required for Incoming Search Queries per Second		
No. of requests per second (Mb/s)	Query size (Bytes)	Bandwidth
1736.11	100	1.39

'''

****Outgoing traffic****

Outgoing traffic is the response that the server returns to the user on the search request. We assume that the number of suggested videos against a search query is 80, and one suggestion is of the size 50 Bytes. Suggestions consist of an ordered list of the video IDs.

To estimate the outgoing traffic bandwidth, we assume the following numbers:

- The number of search requests per day is 150 million.
- The response size is 4,000 Bytes.

We can use the same formula to calculate the bandwidth required for the outgoing traffic.

'''

Bandwidth Required for Outgoing Traffic per Second		
No. of requests per second	Query size (Bytes)	Bandwidth (Mb/s)
1736.11	4000	55.56

'''

[Summarizing the bandwidth requirements of a video search](./bandwidth.jpg)

'''

Note: The bandwidth requirements are relatively modest because we are assuming text results. Many search services can return small thumbnails and other media to enhance the search page. The bandwidth needs per page are intentionally low so that the service can provide near real-time results to the client.

'''

Building blocks we will use

We need a distributed storage in our design. Therefore, we can use the blob store, a previously discussed building block, to store the data to be indexed and the index itself. We'll use a generic term, that is, "distributed storage" instead of the specific term "blob store."

To conclude, we explained what the search system's requirements are. We made resource estimations. And lastly, we mentioned the building block that we'll use in our design of a distributed search system.

[Distributed storage: Blob store](./bb.jpg)

Distributed Search/Scaling Search and Indexing/README.md

Scaling Search and Indexing

Although the proposed design in the previous lesson seems reasonable, still, there are a couple of serious drawbacks. We'll discuss these drawbacks below:

1. Colocated indexing and searching: We've created a system that colocates indexing and searching on the same node. Although it seems like efficient usage of resources, it has its downsides as well. Searching and indexing are both resource-intensive operations. Both operations impact the performance of each other. Also, this colocated design doesn't scale efficiently with varying indexing and search operations over time. Colocating both these

operations on the same machine can lead to an imbalance, and it results in scalability issues.

2. Index recomputation: We assume that each replica will compute the index individually, which leads to inefficient usage of resources. Furthermore, index computation is a resource-intensive task with possibly hundreds of stages of pipelined operations. Thus, recomputing the same index over different replicas requires powerful machines. Instead, the logical approach is to compute the index once and replicate it across availability zones.

Because of these key reasons, we'll look at an alternative approach for distributed indexing and searching.

Solution

Rather than recomputing the index on each replica, we compute the inverted index on the primary node only. Next, we communicate the inverted index (binary blob/file) to the replicas. The key benefit of this approach is that it avoids using the duplicated amount of CPU and memory for indexing on replicas.

'''

Question

What are the disadvantages of the above-proposed solution?

Answer

Since the inverted index will be transferred to the replicas, this will introduce a transmission latency to copy the inverted index file because the size of the index file can be very large.

When the primary node receives new indexing operations, the inverted index file changes. Each replica needs to fetch the latest version of the file after a certain amount of indexing operations reaches a defined threshold.

'''

Separate the indexing and search

With the advent of networking and virtualization technologies, cloud computing has emerged as successful technology. We have access to massive amounts of bandwidth (up to 100 Gbps) and scalable distributed storage in such a technology. These advancements allow for a strong separation between indexing and search without the negative consequence of indexing latency. Because of this isolation, indexing wouldn't affect search scalability and vice versa. Also, instead of recomputing the index on the replica nodes, which wastes resources, we can just replicate the index files.

We'll use these technologies to redesign our distributed indexing and searching system. There are three components involved in this search system design:

1. Indexer: It consists of a group of nodes to compute the index.
2. Distributed storage: This system is used to store partitions and the computed index.
3. Searcher: It consists of a number of nodes to perform searching.

The illustration below depicts the generation and transfer of an inverted index between an indexer and a searcher node:

[The indices produced by the indexing nodes are stored on the distributed storage, and the nodes involved in the search reads indices from the distributed storage to produce a result for the user's query](./query.jpg)

In the above illustration, a single node is shown for each indexing and searching operation. But, in reality, there would be an N number of nodes in the indexing phase, one node per partition (set of documents), that produces inverted indices. The inverted index is stored in the form of binary files on the nodes' local storage. Caching these blob files will result in performance improvement. These binary files are also pushed to a distributed storage. In the case of a hardware failure, a new searcher or indexer machine is added, and a copy of the data is retrieved from the distributed storage.

When the upload is complete, the searcher nodes download the index files. Depending upon user search patterns, the searching nodes will maintain a cache of frequently asked queries and

serve data from RAM. A user search query will be extended to all searcher nodes, which will generate responses according to their respective indices. A merger node in the front-end servers will combine all search results and present them to the user.

The indexing process indexes the new documents as soon as they are available. At the same time the searcher nodes fetch the updated indices to provide improved search results.

Indexing explained

Until now, we have explained the development of a highly scalable and performant design using low-cost nodes. However, we are unaware of the internals of the indexing nodes. In this section, we'll learn how indexing is performed with a MapReduce distributed model and parallel processing framework.

The MapReduce framework is implemented with the help of a cluster manager and a set of worker nodes categorized as Mappers and Reducers. As indicated by its name, MapReduce is composed of two phases:

1. The Map phase
2. The Reduction phase

Furthermore, the input to MapReduce is a number of partitions, or set of documents, whereas its output is an aggregated inverted index.

Let's understand the purpose of the above components:

- Cluster manager: The manager initiates the process by assigning a set of partitions to Mappers. Once the Mappers are done, the cluster manager assigns the output of Mappers to Reducers.
- Mappers: This component extracts and filters terms from the partitions assigned to it by the cluster manager. These machines output inverted indexes in parallel, which serve as input to the Reducers.
- Reducers: The reducer combines mappings for various terms to generate a summarized index.

The cluster manager ensures that all worker nodes are efficiently utilized in the cluster. The MapReduce is built to work under partial failures. If one node fails, it reschedules the work on another node.

Note that the Reducers cannot start as long as the Mappers are working. This means that the cluster manager can use the same node as a Mapper as well as a Reducer.

The slides below depict a simplified setup of how MapReduce can be used to generate an inverted index:

[Indexing](./indexing)

To keep it simple, we have just shown two indicators for each term in the above illustration: the list of documents in which the term appears and the list of the frequency of the term in each document (refer to Indexing for details).

'''

Note: The above MapReduce setup is a simplified version of what happens in practice. A complex pipeline of the MapReduce framework is required to manage the complexities of a real-world search engine. However, the fundamental principles are the same as we presented here.

'''

Summary

In this lesson, we have resolved two key problems of scalability (due to colocated indexing and searching) and resource wastage (due to index recomputation) by using dedicated nodes for indexing and searching. Both operations rely on distributed storage. Furthermore, we presented a simplified description of the MapReduce framework to parallelize the indexing process.

System Design: The Distributed Search

Why do we need a search system?

Nowadays, we see a search bar on almost every website. We use that search bar to pick out relevant content from the enormous amount of content available on that website. A search bar enables us to quickly find what weâre looking for. For example, there are plenty of courses present on the Educative website. If we didnât have a search feature, users would have to scroll through many pages and read the name of each course to find the one theyâre looking for.

Letâs take another example. There are billions of videos uploaded and stored on YouTube. Imagine if YouTube didnât provide us with a search bar. How would we find a specific video among the millions of videos that have been posted on YouTube over the years? It would take months to navigate through all of those videos and find the one we need. Users find it challenging to find what theyâre looking for simply by scrolling around.

[Search bar](./searchBar.jpg)

Search engines are an even bigger example. We have billions of websites on the Internet. Each website has many web pages and there is plenty of content on each of these web pages. With so much content, the Internet would practically be useless without search engines, and users would end up lost in a sea of irrelevant data. Search engines are, essentially, filters for the massive amount of data available. They let users quickly obtain information that is of true interest without having to sift through too many unnecessary web pages.

Behind every search bar, there is a search system.

What is a search system?

A search system is a system that takes some text input, a search query, from the user and returns the relevant content in a few seconds or less. There are three main components of a search system, namely:

- A crawler, which fetches content and creates documents.
- An indexer, which builds a searchable index.
- A searcher, which responds to search queries by running the search query on the index created by the indexer.

'''

For a search engine, a document consists of the text extracted from a web page. In a movie storeâs web page, a document could be a JSON object containing titles, descriptions, and other metadata of the videos upon which we want to perform search queries. The documents could be JSON or any other suitable format. Documents are stored on a distributed storage like S3 or HDFS.

'''

[The three phases of a search system](./3phases.jpg)

Note: We have a separate chapter dedicated to the explanation of the crawler component. In this chapter, weâll focus on indexing.

How will we design a distributed search system?

We divided the design of a distributed search system into five lessons:

1. Requirements: In this lesson, we list the functional and non-functional requirements of a distributed search system. We also estimate our systemâs resources, such as servers, storage, and the bandwidth needed to serve a number of queries.
2. Indexing: This lesson provides us with background knowledge on the process of indexing with the help of an example. After discussing indexing, we also look into a centralized architecture of distributed search systems.
3. Initial design: This lesson consists of the high-level design of our system, its API, and

the details of the indexing and searching process.

4. Final design: In this lesson, we evaluate our previous design and revamp it to make it more scalable.

5. Evaluation: This lesson explains how our designed distributed search system fulfills its requirements.

Let's start by understanding the requirements of designing a distributed search system.

Distributed Task Scheduler/Design Considerations of a Distributed Task Scheduler/README.md

Design Considerations of a Distributed Task Scheduler

Queueing

A distributed queue is a major building block used by a scheduler. The simplest scheduling approach is to push the task into the queue on a first-come, first-served basis. If there are 10,000 nodes (resources) in a cluster (cloud), the task scheduler quickly extracts tasks from the queue and schedules them on the nodes. But, if all the resources are currently busy, then tasks will need to wait in the queue, and small tasks might need to wait longer.

This scheduling mechanism can affect the reliability of the system, availability of the system and priority of tasks. There could be cases where we want urgent execution of a task—for example, a task that notifies a user that their account was accessed from an unrecognized device. So, we can't rely only on the first-come, first-serve to schedule tasks. Instead, we categorize the tasks and set appropriate priorities. We have the following three categories for our tasks:

- Tasks that can't be delayed.
- Tasks that can be delayed.
- Tasks that need to be executed periodically (for example, every 5 minutes, or every hour, or every day).

[Multiple queues based on the task categories](./multiplequeues.jpg)

Our system ensures that tasks in non-urgent queues are not starved. As soon as some task's delay limit is about to be reached, it is moved to the urgent tasks queue so that it gets service. We'll see how the task scheduler implements priorities later in this lesson.

Let's explore some parameters that help the scheduler efficiently utilize resources and provide reliable service to the users.

Execution cap

Some tasks take very long to execute and occupy the resource blocking other tasks. The execution cap is an important parameter to consider while scheduling tasks. If we completely allocate a resource to a single task and wait for that task's completion, some tasks might halt because of a bug in the task script that doesn't let it finish its execution. We let the clients set the execution cap for their tasks. After that specified time, we should stop task execution, release the resource, and allocate it to the next task in the queue. If the task execution stops due to the execution cap limit, our system notifies the respective clients of these instances. The client needs to do appropriate remedial actions for such cases.

If clients don't set the execution cap, the scheduler uses its default upper bound on the maximum allowed time to kill the tasks. Suppose a task actually takes longer—for example, if we are training a machine learning model. In that case, the scheduler might need to pause and resume a task many times to accommodate other tasks. It wouldn't be fair to the short task that has to wait for two days to use a resource for two seconds.

Cloud providers can't let a task execute for an unlimited time for a basic (free) account, because using their resources costs a certain fee to the providers. To handle such cases, clients are informed about maximum usage limits so that they can handle long task execution. For example, clients may design their task in such a way that they checkpoint after some time and load from that state to resume progress in case resources are taken from the client due to usage limit.

'''

Question

What if a long task is 90% executed, but before it completes, the machine that was executing this task fails?

Answer

The task scheduler will re-execute the task on some other machine. Tasks need to be either idempotent, which is discussed later in the lesson, or they should be able to restore their state from a previous checkpoint. Once the state is saved, we can resume that task's execution on any other machine. This makes our system fault tolerant and saves our resources.

'''

Prioritization

There are tasks that need urgent execution. For example, in a social application like Facebook the users can mark themselves safe during an emergency situation, such as an earthquake. The tasks that carry out this activity should be executed in a timely manner, otherwise this feature would be useless to Facebook users. Sending an email notification to the customers that their account was debited a certain amount of money is another example of tasks that require urgent execution.

To prioritize the tasks, the task scheduler maintains a delay tolerance parameter for each task and executes the task close to its delay tolerance. Delay tolerance is the maximum amount of time a task execution could be delayed. The task that has the shortest delay tolerance time is executed first. By using a delay tolerance parameter, we can postpone the tasks with longer delay tolerance values to make room for urgent tasks during peak times.

'''

Question

How do we determine the value of delay tolerance?

Answer

Since there are different categories of tasks in various applications, the application owners or clients can set or automate the values themselves, depending upon the task category. For example, in a social media application like Facebook, we can generate a newsfeed, suggest friends, allow users to mark themselves safe after a disaster, send notifications about a live stream event, and many more. Out of the listed tasks, the priority tasks should be to mark a person safe during an earthquake and send notifications about live stream events. Clients can tighten the delay tolerance values of these tasks down to milliseconds or a few seconds while tasks like suggesting friends can be delayed for days.

The task scheduling system itself can set the delay tolerance value depending on the task category and its severity.

There are different costs for different priorities (there are usually higher costs for high-priority tasks, for example) so that customers can carefully categorize their tasks.

'''

Resource capacity optimization

There could be a time when resources are close to the overload threshold (for example, above 80% utilization). This is called peak time. The same resource may be idle during off-peak times. So, we have to think about better utilization of the resources during off-peak times and how to keep resources available during peak times.

There are tasks that don't need urgent execution. For example, in a social application like Facebook, suggesting friends is not an urgent task. We can make a separate queue for tasks like this and execute them in off-peak times. If we consistently have more work to do than the available resources, we might have a capacity problem, and to solve that, we should commission more resources. A cloud provider needs to have a target resources-to-demand ratio. When demand starts increasing, the ratio will move towards 0. If the ratio starts changing over time, the

provider might decide to commission more or fewer resources.

Task idempotency

If the task executes successfully, but for some reason the machine fails to send an acknowledgement, the scheduler will schedule the task again. The task is executed again, and we end up with the wrong result, which means the task was non-idempotent. An example of non-idempotence is shown in the following illustration:

```
[non-idempotent](./nonidempotency)
```

We don't want the final result to change when executing the task again. This is critical in financial applications while transferring money. We require that tasks are idempotent. An idempotent task produces the same result, no matter how many times we execute it. The execution of an idempotent task is shown in the following illustration:

```
'''
```

Idempotence is a characteristic of certain operations in mathematics and computer science that allows them to be applied several times without influencing the outcome.

```
'''
```

```
[idempotent](./idempotent)
```

Let's make the task of uploading a video to the database an idempotent operation. We don't want the video to be duplicated in the database in case the uploader didn't receive the acknowledgment. Idempotency ensures that the video is not duplicated. This property is added in the implementation by the developers where they identify the video by something (for example, its name) and overwrite the old one. This way, no matter how many times someone uploads it, the final result is the same. Idempotency enables us to simply re-execute a failed task.

```
'''
```

Question

How should we handle task execution that can never be completed because of an infinite loop in the payload of that task?

Answer

We need to mark and kill such tasks. So, for this purpose, we can set time limits. If it takes more than the specified execution cap value, we can kill the task. But it's challenging to differentiate between a buggy task and a long task. We can handle it at the application level where the clients take care of the long tasks by saving the state at different times. Clients can also resume from that state if the task scheduler kills that task, assuming that the task contains an infinite loop.

```
'''
```

Schedule and execute untrusted tasks

Before proceeding, let's ask ourselves a question: What are the untrusted tasks, and how should we manage them?

```
'''
```

Untrusted tasks are the tasks where there is a chance of some malicious instructions in the task script that can affect the execution of the other tasks. The affected tasks could be those of the other tenants. Untrusted tasks can also harm the operating system or the resource on which they're executed.

```
'''
```

Programs have latent bugs and might have malicious intent. When using task schedulers, we should be careful that one task does not impact other tasks negatively. If we provide infrastructure as a service, security is an essential component. This is because it becomes easier for tenants to harm each other's tasks by executing malicious code in the shared environment. Execution of malicious code can also damage our infrastructure. So, we need to keep the following considerations in mind:

- Use appropriate authentication and resource authorization.
- Consider code sandboxing using dockers or virtual machines.
- Use performance isolation between tasks by monitoring tasksâ resource utilization and capping (or terminating) badly behaving tasks.

'''

Question

What happens when the same task fails multiple times?

Answer

We can use a dead-letter queue facility to isolate repeatedly failing tasks.

'''

Distributed Task Scheduler/Design of a Distributed Task Scheduler/README.md

Design of a Distributed Task Scheduler

Components

We can consider scheduling at many levels. We could be asked to design scheduling that is done internally by an organization to run tasks on their own cluster of machines. There, they have to find ample resources and need to decide which task to run first.

On the other hand, we could also be asked to design scheduling that a cloud provider uses to schedule tasks coming from multiple clients. Cloud providers need to decide which task to run first and which clients to handle first to provide appropriate isolation between different tenants.

So, in general, the big components of our system are:

- Clients: They initiate the task execution.
- Resources: The task is executed on these components.
- Scheduler: A scheduler performs processes between clients and resources and decides which task should get resources first.

[Scheduler putting tasks into a queue for resource allocation](./sys.jpg)

As shown in the above illustration, it is necessary to put the incoming tasks into a queue. It is because of the following reasons:

- We might not have sufficient resources available right now.
- There is task dependency, and some tasks need to wait for others.
- We need to decouple the clients from the task execution so that they can hand off work to our system. Our system then queues it for execution.

Letâs design a task scheduling system that should be able to schedule any task. Often, many tasks are relatively short-livedâfrom seconds to minutes. For long-running tasks, we might rely on the ability of periodic checksumming and restoration at the application level to recover from possible failures.

Letâs assume that some single server in our fleet can meet the computational needs of each task. For tasks that need many servers, either the application would need to break them down into smaller tasks for our system or employ long-term resource acquisition from the cluster manager.

Design

When a task comes for scheduling, it should contain the following information with it:

- Resource requirements: The requirements include how many CPU cores it needs, how much RAM is required to execute this task, how much disk space is required, what should the disk access rate be (input/output rate per second, or IOPS), and how many TCP ports the task needs for the

execution, and so on. But, it is difficult for the clients to quantify these requirements. To remedy this situation, we have different tiers of resources like basic, regular, and premium. The client can specify the requirement in terms of these tiers.

- Dependency: Broadly, tasks can be of two types: dependent and independent.
 - Dependent tasks require executing one or more additional tasks for their complete execution. These tasks must run in a sequence. For a dependent task, the client should provide a list of the tasks on which a given task is dependent.
 - Independent tasks donâ t depend on the execution of any other task. Independent tasks can run in parallel. We should know whether a task is dependent or independent. The dependency information helps to execute both dependent tasks in order and independent tasks in parallel for efficient utilization of resources.

The design of the task scheduler is shown in the following illustration:

[The design of task scheduler](./details.jpg)

- Clients: The clients of the cloud providers are individuals or organizations from small to large businesses who want to execute their tasks.
- Rate limiter: The resources available for a client depend on the cost they pay. It is important to limit the number of tasks for the reliability of our service. For instance, X number of tasks per hour are allowed to enter the system. Others will get a message like â Limitedâ instead of accepting the task and responding late. A rate limiter limits the number of tasks the client schedules based on its subscription. If the limit is exceeded, it returns an error message to the client that the rate limit has been exceeded.
- Task submitter: The task submitter admits the task if it successfully passes through the rate limiter. There isnâ t a single task submitter. Instead, we have a cluster of nodes that admit the increasing number of tasks.
- Unique ID generator: It assigns unique IDs to the newly admitted tasks.
- Database: All of the tasks taken by the task submitter are stored in a distributed database. For each task, we have some attributes, and all of the attributes except one are stored in the relational database.
 - Relational database (RDB): A relational database stores task IDs, user IDs, required resources, execution caps, the total number of attempts made by the client, delay tolerance, and so on, as shown in the following table. We can find the details on the RDB in [Databases > Types of Databases](#).
 - Graph database (GDB): This is a non-relational database that uses the graph data structure to store data. We use it to build and store a directed acyclic graph (DAG) of dependent tasks, topologically sorted by the task submitter, so that we can schedule tasks according to that DAG. We can find more details of the graph DB [Databases > Types of Databases](#).

'''

Database Schema

Column Name	Datatype	Description
TaskID	Integer	Uniquely identifies each task.
UserID	Integer	This is the ID of the task owner.
SchedulingType or annually.	VarChar	This can be either once, daily, weekly, monthly,
TotalAttempts task execution fails.	Integer	This is the maximum number of retries in case a
ResourceRequirements	VarChar	Clients have to specify the type of the offered resource categories, such as Basic, Regular, or Premium. The specified resource category is saved in the form of a string in the RDB.
ExecutionCap	Time	This is the maximum time allowed for the task execution. (This time starts when a resource is allocated to the task.)

Status	VarChar	This can be waiting, in progress, done, or failed.
DelayTolerance	Time	This indicates how much delay we can sustain before starting a task.
ScriptPath	VarChar	The path of the script that needs to be executed. The script is a file placed in a file system. The file should be made accessible so that it can be executed, just like how we mount Google Drive in the Google Colaboratory and then execute our code files there.

'''

'''

Note: If we use geo-replicated data stores, we can run multiple instances of our task scheduling system in different data centers to achieve even larger scale and higher resource utilization.

'''

- Batching and prioritization: After we store the tasks in the RDB, the tasks are grouped into batches. Prioritization is based on the attributes of the tasks, such as delay tolerance or the tasks with short execution cap, and so on. The top K priority tasks are pushed into the distributed queue, where K limits the number of elements we can push into the queue. The value of K depends on many factors, such as currently available resources, the client or task priority, and subscription level.

- Distributed queue: It consists of a queue and a queue manager. The queue manager adds, updates, or deletes tasks in the queue. It keeps track of the types of queues we use. It is also responsible for keeping the task in the queue until it executes successfully. In case a task execution fails, that task is made visible in the queue again. The queue manager knows which queue to run during the peak time and which queue to run during the off-peak time.

- Queue manager: The queue manager deletes a task from the queue if it executes successfully. It also makes the task visible if its previous execution failed. It retries for the allowed number of attempts for a task in case of a failed execution.

- Resource manager: The resource manager knows which of the resources are free. It pulls the tasks from the distributed queue and assigns them resources. The resource manager keeps track of the execution of each task and sends back their statuses to the queue manager. If a task goes beyond its promised or required resource use, that task will be terminated, and the status is sent back to the task submitter, which will notify the client about the termination of the task through an error message.

- Monitoring service: It is responsible for checking the health of the resource manager and the resources. If some resource fails, it alerts the administrators to repair the resource or add new resources if required. If resources are not being used, it alerts the administrators to remove them or power them off.

'''

Question

Why do we store tasks in a database? Why should we not push the tasks directly to the queue?

Answer

The queue does not hold data permanently. We pay a cost for the queue service we use. So, we just push those tasks that are ready for execution shortly to the queue. The tasks that are successfully executed need to be removed from the queue. Moreover, there are different scheduling types. A task could be scheduled once, daily, weekly, monthly, or annually. Therefore, we have to save the task somewhere in our storage.

'''

Task submitter

As we have seen above, every component we use in the design of the distributed task scheduler is distributed and therefore scalable and available. But, the task submitter could be a single point of failure. So, to handle this, we use a cluster of nodes. Each node must admit the

tasks, send the tasks to a unique ID generator for ID assignment, and then store the task along with the task ID in the distributed database.

There is a cluster manager to which each node sends a heartbeat that indicates the node is working correctly. Each node updates the cluster manager about the admitted tasks. The cluster manager maintains a list of tasks and the node ID that admitted that task. In case a node fails to execute a task, the cluster manager hands over that task to another node in the cluster. The cluster manager is itself replicated.

Above, we designed a task scheduling system. We'll discuss the design considerations of our task scheduler in the next lesson.

Distributed Task Scheduler/Evaluation of a Distributed Task Scheduler's Design/README.md

Evaluation of a Distributed Task Scheduler's Design

Availability

The first component in our design was a rate limiter that is appropriately replicated and ensures availability. Task submission is done by several nodes. If a node that submits a task fails, the other nodes take its place. The queue in which we push the task is also distributed in nature, ensuring availability. We always have resources available because we continuously monitor if we need to add or remove resources. Each component in the design is distributed and makes the overall system available.

Durability

We store the tasks in a persistent distributed database and push the tasks into the queue near their execution time. Once a task is submitted, it is in the database until its execution.

Scalability

Our task scheduler provides scalability because the task submitter is distributed in our design. We can add more nodes to the cluster to submit an increasing number of tasks. The tasks are then saved into a distributed relational database, which is also scalable. The tasks from RDB are then pushed to a distributed queue, which can scale with an increasing number of tasks. We can add more queues for different types of tasks. We can also add more resources depending on the resource-to-demand ratio.

[Non-functional requirements fulfilled by our task scheduler system](./evaluation.jpg)

Fault tolerance

A task is not removed from the queue the first time it is sent for execution. If the execution fails, we retry for the maximum number of allowed attempts. If the task contains an infinite loop, we kill the task after some specified time and notify the user.

Bounded waiting time

We don't let the users wait for an infinite time. We have a limit on the maximum waiting time. If the limit is reached and we are unable to schedule the task for some reason, we notify the users and ask them to try again.

Conclusion

We discussed the difference between an OS-level task scheduler and a data center-level task scheduler. We explained that the data center-level task scheduling needs a distributed solution because of multiple tenants and dispersed resources. We learned that the queue is a major building block of a task scheduler. We also used distributed queues where we can scale with an increasing number of tasks to utilize an increasing number of resources.

This lesson helped us to evaluate the issues with the FIFO queue. It was observed that the main job of the task scheduler is to set the priorities of the tasks for which we used a delay tolerance parameter. We discussed how the task scheduler determines the delay tolerance value and used different distributed databases to store task details. We ensured that the dependent tasks are executed in order by running the tasks according to DAG stored in the graph database. Depending upon the number of tasks (or demand), we added or removed resources to optimize the

capacity. In the end, we used the monitoring service that alerts the administrators in case we need to add or remove resources.

Distributed Task Scheduler/Requirements of a Distributed Task Scheduler's Design/README.md

Requirements of a Distributed Task Scheduler's Design

Requirements

Let's start by understanding the functional and non-functional requirements for designing a task scheduler.

Functional requirements

The functional requirements of the distributed task scheduler are as follows:

- Submit tasks: The system should allow the users to submit their tasks for execution.
- Allocate resources: The system should be able to allocate the required resources to each task.
- Remove tasks: The system should allow the users to cancel the submitted tasks.
- Monitor task execution: The task execution should be adequately monitored and rescheduled if the task fails to execute.
- Efficient resource utilization: The resources (CPU and memory) must be used efficiently in terms of time, cost, and fairness. Efficiency means that we do not waste resources. For example, if we allocate a heavy resource to a light task that can easily be executed on a cheap resource, it means that we have not efficiently utilized our resources. Fairness is all tenants's ability to get the resources with equally likely probability in a certain cost class.
- Release resources: After successfully executing a task, the system should take back the resources assigned to the task.
- Show task status: The system should show the users the current status of the task.

[Functional requirements of the distributed task scheduler](./fr.jpg)

Non-functional requirements

The non-functional requirements of the distributed task scheduler are as follows:

- Availability: The system should be highly available to schedule and execute tasks.
- Durability: The tasks received by the system should be durable and should not be lost.
- Scalability: The system should be able to schedule and execute an ever-increasing number of tasks per day. Fault-tolerance: The system must be fault-tolerant by providing services uninterrupted despite faults in one or more of its components.
- Bounded waiting time: This is how long a task needs to wait before starting execution. We must not execute tasks much later than expected. Users shouldn't be kept on waiting for an infinite time. If the waiting time for users crosses a certain threshold, they should be notified.

[Non-functional requirements of distributed task scheduler](./nfr.jpg)

So far in this lesson, we have learned about task schedulers in general and distinguished between centralized and distributed task schedulers. Lastly, we listed the requirements of the distributed task scheduler system.

Building blocks we will use

We'll utilize the following building blocks in the design of our task scheduling system:

[Building blocks of a task scheduler](./bb.jpg)

- Rate limiter is required to limit the number of tasks so that our system is reliable.
- A sequencer is needed to uniquely identify tasks.
- Database(s) are used to store task information.
- A distributed queue is required to arrange tasks in the order of execution.
- Monitoring is essential to check the health of the resources and to detect failed tasks to provide reliable service to the users.

Weâ ve identified the requirements of the task scheduler. In the next lesson, weâ ll design task scheduling system according to these requirements.

Distributed Task Scheduler/System Design The Distributed Task Scheduler/README.md

System Design: The Distributed Task Scheduler

What is a task scheduler?

A task is a piece of computational work that requires resources (CPU time, memory, storage, network bandwidth, and so on) for some specified time. For example, uploading a photo or a video on Facebook or Instagram consists of the following background tasks:

1. Encode the photo or video in multiple resolutions.
2. Validate the photo or video to check for content monetization copyrights, and many more.

'''

Content monetization is a way of leveraging content so that a service can profit from it as users consume it.

'''

The successful execution of all the above tasks makes the photo or video visible. However, a photo and video uploader does not need to stop the above tasks to complete.

Another example is when we post a comment on Facebook. We donâ t hold the comment poster until that comment is delivered to all the followers. That delivery is delegated to an asynchronous task scheduler to do offline.

In a system, many tasks contend for limited computational resources. A system that mediates between tasks and resources by intelligently allocating resources to tasks so that task-level and system-level goals are met is called a task scheduler.

When to use a task scheduler

A task scheduler is a critical component of a system for getting work done efficiently. It allows us to complete a large number of tasks using limited resources. It also aids in fully utilizing the systemâ s resources, provides users with an uninterrupted execution experience, and so on. The following are some of the use cases of task scheduling:

Single-OS-based node: It has many processes or tasks that contend for the nodeâ s limited computational resources. So, we could use a local OS task scheduler that efficiently allocates resources to the tasks. It uses multi-feedback queues to pick some tasks and runs them on some processor.

Cloud computing services: Where there are many distributed resources and various tasks from multiple tenants, there is a strong need for a task scheduler to utilize cloud computing resources efficiently and meet tenantsâ demands. A local OS task scheduler isnâ t sufficient for this purpose because the tasks are in the billions, the source of the tasks is not single, and the resources to manage are not in a single machine. We have to go for a distributed solution.

Large distributed systems: In this system, many tasks run in the background against a single request by a user. Consider that there are millions to billions of users of a popular system like Facebook, WhatsApp, or Instagram. These systems require a task scheduler to handle billions of tasks. Facebook schedules its tasks against billions of parallel asynchronous requests by its users using Async.

'''

uninterrupted execution experience: Running periodic tasks without user intervention to initiate the task execution again and again.

asynchronous requests: Asynchronous requests are those that are not on a clientâ s critical path and often have some delay tolerance. Users request these tasks, and the system completes the tasks while the requester does not need to wait for the completion of the work. Notifications about the final state of the task are communicated to the requester some time in the future.

'''

'''

Note: Async is Facebook's own distributed task scheduler that schedules all its tasks. Some tasks are more time-sensitive, like the tasks that should run to notify the users that the livestream of an event has started. It would be pointless if the users received a notification about the livestream after it had finished. Some tasks can be delayed, like tasks that make friend suggestions to users. Async schedules tasks based on appropriate priorities.

'''

Distributed task scheduling

The process of deciding and assigning resources to the tasks in a timely manner is called task scheduling. The visual difference between an OS-level task scheduler and a data center-level task scheduler is shown in the following illustration:

[An OS-level task scheduler vs. a data center-level task scheduler]

The OS task scheduler schedules a node's local tasks or processes on that node's computational resources. At the same time, the data center's task scheduler schedules billions of tasks coming from multiple tenants that use the data center's resources.

Our goal is to design a task scheduler similar to the data center-level task scheduler where the following is considered:

- Tasks will come from many different sources, tenants, and sub-systems.
- Many resources will be dispersed in a data center (or maybe across many data centers).

The above two requirements make the task scheduling problem challenging. We'll design a distributed task scheduler that can handle all these tasks by making it scalable, reliable, and fault-tolerant.

How will we design a task scheduling system?

1. Requirements: We'll identify the functional and non-functional requirements of a task scheduling system in this lesson.
 2. Design: This lesson will discuss the system design of our task scheduling system and explores the components of the system and database schema.
 3. Design considerations: In this lesson, we'll highlight some design factors, such as task prioritization, resource optimization, and so on.
 4. Evaluation: We'll evaluate our design of task scheduler based on our requirements.
- Let's start by understanding the requirements of a task scheduling system.

Domain Name System/How the Domain Name System Works/README.md

How the Domain Name System Works

Through this lesson, we'll answer the following questions:

'''

How is the DNS hierarchy formed using various types of DNS name servers?

How is caching performed at different levels of the Internet to reduce the querying burden over the DNS infrastructure?

How does the distributed nature of the DNS infrastructure help its robustness?

'''

Let's get started.

DNS hierarchy

As stated before, the DNS isn't a single server that accepts requests and responds to user queries. It's a complete infrastructure with name servers at different hierarchies.

'''

name servers can respond to users' DNS queries

'''

There are mainly four types of servers in the DNS hierarchy:

'''

DNS resolver: Resolvers initiate the querying sequence and forward requests to the other DNS name servers. Typically, DNS resolvers lie within the premise of the user's network. However, DNS resolvers can also cater to users' DNS queries through caching techniques, as we will see shortly. These servers can also be called local or default servers.

Root-level name servers: These servers receive requests from local servers. Root name servers maintain name servers based on top-level domain names, such as .com, .edu, .us, and so on. For instance, when a user requests the IP address of educative.io, root-level name servers will return a list of top-level domain (TLD) servers that hold the IP addresses of the .io domain.

Top-level domain (TLD) name servers: These servers hold the IP addresses of authoritative name servers. The querying party will get a list of IP addresses that belong to the authoritative servers of the organization.

Authoritative name servers: These are the organization's DNS name servers that provide the IP addresses of the web or application servers.

'''

[DNS hierarchy for resolution of domain/host names](./arch.jpg)

'''

Question

How are DNS names processed? For example, will educative.io be processed from left to right or right to left?

Answer

Unlike UNIX files, which are processed from left to right, DNS names are processed from right to left. In the case of educative.io, the resolvers will first resolve the .io part, then educative, and so on.

Visually, however, the DNS hierarchy can be viewed as a tree.

'''

Iterative versus recursive query resolution

There are two ways to perform a DNS query:

'''

Iterative: The local server requests the root, TLD, and the authoritative servers for the IP address.

Recursive: The end user requests the local server. The local server further requests the root DNS name servers. The root name servers forward the requests to other name servers.

'''

In the following illustration (on the left), DNS query resolution is iterative from the perspective of the local/ISP server:

[Iterative versus recursive query](./process.jpg)

'''

Note: Typically, an iterative query is preferred to reduce query load on DNS infrastructure.

'''

'''

These days, we'll find many third-party public DNS resolvers offered by Google, Cloudflare, OpenDNS, and many more. The interesting fact is that these public DNS servers may provide quicker responses than the local ISP DNS facilities.

'''

Caching

Caching refers to the temporary storage of frequently requested resource records. A record is a data unit within the DNS database that shows a name-to-value binding. Caching reduces response time to the user and decreases network traffic. When we use caching at different hierarchies,

it can reduce a lot of querying burden on the DNS infrastructure. Caching can be implemented in the browser, operating systems, local name server within the user's network, or the ISP's resolvers.

The slideshow below demonstrates the power of caching in the DNS:

[Caching](./dns_flow)

DNS as a distributed system

Although the DNS hierarchy facilitates the distributed Internet that we know today, it's a distributed system itself. The distributed nature of DNS has the following advantages:

'''

It avoids becoming a single point of failure (SPOF).

It achieves low query latency so users can get responses from nearby servers.

It gets a higher degree of flexibility during maintenance and updates or upgrades. For example, if one DNS server is down or overburdened, another DNS server can respond to user queries.

'''

There are 13 logical root name servers (named letter A through M) with many instances spread throughout the globe. These servers are managed by 12 different organizations.

Let's now go over how DNS is scalable, reliable, and consistent.

Highly scalable

Due to its hierarchical nature, DNS is a highly scalable system. Roughly 1,000 replicated instances of 13 root-level servers are spread throughout the world strategically to handle user queries. The working labor is divided among TLD and root servers to handle a query and, finally, the authoritative servers that are managed by the organizations themselves to make the entire system work. As shown in the DNS hierarchy tree above, different services handle different portions of the tree enabling scalability and manageability of the system.

Reliable

Three main reasons make the DNS a reliable system:

'''

Caching: The caching is done in the browser, the operating system, and the local name server, and the ISP DNS resolvers also maintain a rich cache of frequently visited services. Even if some DNS servers are temporarily down, cached records can be served to make DNS a reliable system.

Server replication: DNS has replicated copies of each logical server spread systematically across the globe to entertain user requests at low latency. The redundant servers improve the reliability of the overall system.

Protocol: Although many clients use DNS over unreliable user datagram protocol (UDP), UDP has its advantages. UDP is much faster and, therefore, improves DNS performance. Furthermore, Internet service's reliability has improved since its inception, so UDP is usually favored over TCP. A DNS resolver can resend the UDP request if it didn't get a reply to a previous one. The request-response needs just one round trip, which provides a shorter delay as compared to TCP, which needs a three-way handshake before data exchange.

'''

'''

Question

What happens if a network is congested? Should DNS continue using UDP?

Answer

Typically, DNS uses UDP. However, DNS can use TCP when its message size exceeds the original packet size of 512 Bytes. This is because large-size packets are more prone to be damaged in congested networks. DNS always uses TCP for zone transfers.

Some clients prefer DNS over TCP to employ transport layer security for privacy reasons.

'''

Consistent

DNS uses various protocols to update and transfer information among replicated servers in a hierarchy. DNS compromises on strong consistency to achieve high performance because data is read frequently from DNS databases as compared to writing. However, DNS provides eventual consistency and updates records on replicated servers lazily. Typically, it can take from a few seconds up to three days to update records on the DNS servers across the Internet. The time it takes to propagate information among different DNS clusters depends on the DNS infrastructure, the size of the update, and which part of the DNS tree is being updated.

Consistency can suffer because of caching too. Since authoritative servers are located within the organization, it may be possible that certain resource records are updated on the authoritative servers in case of server failures at the organization. Therefore, cached records at the default/local and ISP servers may be outdated. To mitigate this issue, each cached record comes with an expiration time called time-to-live (TTL).

'''

TTL refers to the amount of time a record can be cached by the querying party. TTL values are reported in the units of seconds.

'''

'''

Question

To maintain high availability, should the TTL value be large or small?

Answer

To maintain high availability, the TTL value should be small. This is because if any server or cluster fails, the organization can update the resource records right away. Users will experience non-availability only for the time the TTL isn't expired. However, if the TTL is large, the organization will update its resource records, whereas users will keep ping the outdated server that would have crashed long ago. Companies that long for high availability maintain a TTL value as low as 120 seconds. Therefore, even in case of a failure, the maximum downtime is a few minutes.

'''

Test it out

Let's run a couple of commands. Click on the terminal to execute the following commands. Copy the following commands in the terminal to run them. Study the output of the commands:

'''

```
nslookup www.google.com
```

```
dig www.google.com
```

'''

```
[nslookup](./nslookup1.jpg)
```

```
[nslookup](./nslookup2.jpg)
```

The nslookup output

The Non-authoritative answer, as the name suggests, is the answer provided by a server that is not the authoritative server of Google. It isn't in the list of authoritative nameservers that Google maintains. So, where does the answer come from? The answer is provided by second, third, and fourth-hand name servers configured to reply to our DNS query—for example, our university or office DNS resolver, our ISP nameserver, our ISP's ISP nameserver, and so on. In short, it can be considered as a cached version of Google's authoritative nameservers response. If we have multiple domain names, we'll realize that we receive a cached response most of the time.

If we run the same command multiple times, we'll receive the same IP addresses list but in a different order each time. The reason for that is DNS is indirectly performing load balancing. It's an important term that we'll gain familiarity with in the coming lessons.

The dig output

The Query time: 4 msec represents the time it takes to get a response from the DNS server. For various reasons, these numbers may be different in our case.

The 300 value in the ANSWER SECTION represents the number of seconds the cache is maintained in the DNS resolver. This means that Google's ADNS keeps a TTL value of five minutes(300sec/60)

'''

Note: We invite you to test different services for their TTL and query times to strengthen your understanding. You may use the above terminal for this purpose.

'''

'''

Question

If we need DNS to tell us which IP to reach a website or service, how will we know the DNS resolver's IP address? (It seems like a chicken-and-egg problem!)

Answer

End users's operating systems have configuration files (/etc/resolv.conf in Linux) with the DNS resolvers's IP addresses, which in turn obtain all information for them. (Often, DHCP provides the default DNS resolver IP address along with other configurations.) The end-systems request DNS resolves for any DNS queries. DNS resolvers have special software installed to resolve queries through the DNS infrastructure. The root server's IP addresses are within the special software. Typically, the Berkeley Internet Name Domain (BIND) software is used on DNS resolvers. The InterNIC maintains the updated list of 13 root servers.

'''

So, we break the chicken-and-egg problem by seeding each resolver with a priori knowledge of root DNS servers (whose IPs rarely change).

Domain Name System/Introduction to Domain Name System (DNS)/README.md

Introduction to Domain Name System (DNS)

The origins of DNS

Let's consider the example of a mobile phone where a unique number is associated with each user. To make calls to friends, we can initially try to memorize some of the phone numbers. However, as the number of contacts grows, we'll have to use a phone book to keep track of all our contacts. This way, whenever we need to make a call, we'll refer to the phone book and find the number we need.

Similarly, computers are uniquely identified by IP addresses—for example, 104.18.2.119 is an IP address. We use IP addresses to visit a website hosted on a machine. Since humans cannot easily remember IP addresses to visit domain names (an example domain name being educative.io), we need a phone book-like repository that can maintain all mappings of domain names to IP addresses. In this chapter, we'll see how DNS serves as the Internet's phone book.

[The origins of DNS](./dns_brief.jpg)

What is DNS?

The domain name system (DNS) is the Internet's naming service that maps human-friendly domain names to machine-readable IP addresses. The service of DNS is transparent to users. When a user enters a domain name in the browser, the browser has to translate the domain name to IP address by asking the DNS infrastructure. Once the desired IP address is obtained, the user's request is forwarded to the destination web server.

The slides below show the high-level flow of the working of DNS:

[What is DNS](./dns_flow)

The entire operation is performed very quickly. Therefore, the end user experiences minimum delay. We'll also see how browsers save some of the frequently used mappings for later use in the next lesson.

Important details

Let's highlight some of the important details about DNS, some of which we'll cover in the next lesson:

****Name servers**:** It's important to understand that the DNS isn't a single server. It's a complete infrastructure with numerous servers. DNS servers that respond to users's queries are

called name servers.

****Resource records****: The DNS database stores domain name to IP address mappings in the form of resource records (RR). The RR is the smallest unit of information that users request from the name servers. There are different types of RRs. The table below describes common RRs. The three important pieces of information are type, name, and value. The name and value change depending upon the type of the RR.

[Resource records](./common_types_of_rr.jpg)

****Caching****: DNS uses caching at different layers to reduce request latency for the user. Caching plays an important role in reducing the burden on DNS infrastructure because it has to cater to the queries of the entire Internet.

****Hierarchy****: DNS name servers are in a hierarchical form. The hierarchical structure allows DNS to be highly scalable because of its increasing size and query load. In the next lesson, we'll look at how a tree-like structure is used to manage the entire DNS database.

Introduction/README.md

What is system design?

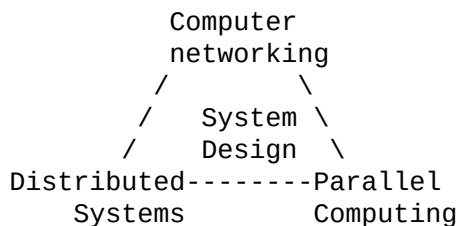
'''

System design is the process of defining components and their integration, APIs, and data models to build large-scale systems that meet a specified set of functional and non-functional requirements.

'''

System design uses the concepts of computer networking, parallel computing, and distributed systems to craft systems that scale well and are performant.

'''



'''

System design aims to build systems that are reliable, effective, and maintainable, among other characteristics.

'''

â€¢ Reliable systems handle faults, failures, and errors.

â€¢ Effective systems meet all user needs and business requirements.

â€¢ Maintainable systems are flexible and easy to scale up or down. The ability to add new features also comes under the umbrella of maintainability.

'''

Modern system design using building blocks.

Key-value Store/Design of a Key-value Store/README.md

Design of a Key-value Store

Requirements

Let's list the requirements of designing a key-value store to overcome the problems of traditional databases.

Functional requirements

The functional requirements are as follows:

- Configurable service: Some applications might have a tendency to trade strong consistency for higher availability. We need to provide a configurable service so that different applications could use a range of consistency models. We need tight control over the trade-offs between availability, consistency, cost-effectiveness, and performance.

- Ability to always write: The applications should always have the ability to write into the key-value storage. If the user wants strong consistency, this requirement might not always be fulfilled due to the implications of the CAP theorem.

- Hardware heterogeneity: The system shouldn't have distinguished nodes. Each node should be functionally able to do any task. Though servers can be heterogeneous, newer hardware might be more capable than older ones.

Non-functional requirements

The non-functional requirements are as follows:

- Scalable: Key-value stores should run on tens of thousands of servers distributed across the globe. Incremental scalability is highly desirable. We should add or remove the servers as needed with minimal to no disruption to the service availability. Moreover, our system should be able to handle an enormous number of users of the key-value store.

- Available: We need to provide continuous service, so availability is very important. This property is configurable. So, if the user wants strong consistency, we'll have less availability and vice versa.

- Fault tolerance: The key-value store should operate uninterrupted despite failures in servers or their components.

Question

Why do we need to run key-value stores on multiple servers?

'''

Answer

A single-node-based hash table can fall short due to one or more of the following reasons:

No matter how big a server we get, this server can't meet data storage and query requirements.

Failure of this one mega-server will result in service downtime for everyone.

So, key-value stores should use many servers to store and retrieve data.

'''

Assumptions

We'll assume the following to keep our design simple:

- The data centers hosting the service are trusted (non-hostile).

- All the required authentication and authorization are already completed.

- User requests and responses are relayed over HTTPS.

API design

Key-value stores, like ordinary hash tables, provide two primary functions, which are get and put.

Let's look at the API design.

The get function

The API call to get a value should look like this:

'''

get(key)

'''

We return the associated value on the basis of the parameter key. When data is replicated, it

locates the object replica associated with a specific key thatâs hidden from the end user. If done by the system if the store is configured with a weaker data consistency model. For example, in eventual consistency, there might be more than one value returned against a key.

'''

Parameter	Description
key	Itâs the key against which we want to get value.

'''

The put function

The API call to put the value into the system should look like this:

'''

put(key, value)
'''

It stores the value associated with the key. The system automatically determines where data should be placed. Additionally, the system often keeps metadata about the stored object. Such metadata can include the version of the object.

Parameter	Description
key	It's the key against which we have to store value.
value	It's the object to be stored against the key.

'''

Question

We often keep hashes of the value (and at times, value + associated key) as metadata for data integrity checks. Should such a hash be taken after any data compression or encryption, or should it be taken before?

Answer

The correct answer might depend on the specific application. Still, we can use hashes either before or after any compression or encryption. But weâll need to do that consistently for put and get operations.

'''

Data type

The key is often a primary key in a key-value store, while the value can be any arbitrary binary data.

'''

Note: Dynamo uses MD5 hashes on the key to generate a 128-bit identifier. These identifiers help the system determine which server node will be responsible for this specific key.

'''

In the next lesson, weâll learn how to design our key-value store. First, weâll focus on scalability, replication, and versioning of our data to our system. Then, weâll ensure the functional requirements and make our system fault tolerant. Weâll fulfill a few of our non-functional requirements first because implementing our functional requirements depends on the method chosen for scalability.

'''

Note: This chapter is based on Dynamo, which is an influential work in the domain of key-value stores.

Dynamo: Amazon's Highly Available Key-value Store

(<https://assets.amazon.science/ac/1d/eb50c4064c538c8ac440ce6a1d91/dynamo-amazons-highly-available-key-value-store.pdf>)

'''

Key-value Store/Enable Fault Tolerance and Failure Detection/README.md

Enable Fault Tolerance and Failure Detection

Handle temporary failures

Typically, distributed systems use a quorum-based approach to handle failures. A quorum is the

minimum number of votes required for a distributed transaction to proceed with an operation. If a server is part of the consensus and is down, then we can't perform the required operation. This affects the availability and durability of our system.

We'll use a sloppy quorum instead of strict quorum membership. Usually, a leader manages the communication among the participants of the consensus. The participants send an acknowledgment after committing a successful write. Upon receiving these acknowledgments, the leader responds to the client. However, the drawback is that the participants are easily affected by the network outage. If the leader is temporarily down and the participants can't reach it, they declare the leader dead. Now, a new leader has to be reelected. Such frequent elections have a negative impact on performance because the system spends more time picking a leader than accomplishing any actual work.

In the sloppy quorum, the first n healthy nodes from the preference list handle all read and write operations. The n healthy nodes may not always be the first n nodes discovered when moving clockwise in the consistent hash ring.

Let's consider the following configuration with $n = 3$. If node A is briefly unavailable or unreachable during a write operation, the request is sent to the next healthy node from the preference list, which is node D in this case. It ensures the desired availability and durability. After processing the request, the node D includes a hint as to which node was the intended receiver (in this case, A). Once node A is up and running again, node D sends the request information to A so it can update its data. Upon completion of the transfer, D removes this item from its local storage without affecting the total number of replicas in the system.

[Handle temporary failures](./failures)

This approach is called a hinted handoff. Using it, we can ensure that reads and writes are fulfilled if a node faces temporary failure.

'''

Note: A highly available storage system must handle data center failure due to power outages, cooling failures, network failures, or natural disasters. For this, we should ensure replication across the data centers. So, if one data center is down, we can recover it from the other.

'''

'''

Question

What are the limitations of using hinted handoff?

Answer

A minimal churn in system membership and transient node failures are ideal for hinted handoff. However, hinted replicas may become unavailable before being restored to the originating replica node in certain circumstances.

'''

Handle permanent failures

In the event of permanent failures of nodes, we should keep our replicas synchronized to make our system more durable. We need to speed up the detection of inconsistencies between replicas and reduce the quantity of transferred data. We'll use Merkle trees for that.

In a Merkle tree, the values of individual keys are hashed and used as the leaves of the tree. There are hashes of their children in the parent nodes higher up the tree. Each branch of the Merkle tree can be verified independently without the need to download the complete tree or the entire dataset. While checking for inconsistencies across copies, Merkle trees reduce the amount of data that must be exchanged. There's no need for synchronization if, for example, hash values of two trees' roots are the same and their leaf nodes are also the same. Until the process reaches the tree leaves, the hosts can identify the keys that are out of sync when the nodes exchange the hash values of children. The Merkle tree is a mechanism to implement anti-entropy, which means to keep all the data consistent. It reduces data transmission for

synchronization and the number of discs accessed during the anti-entropy process.

The following slides explain how Merkle trees work:

[Merkle tree](./markle)

Anti-entropy with Merkle trees

Each node keeps a distinct Merkle tree for the range of keys that it hosts for each virtual node. The nodes can determine if the keys in a given range are correct. The root of the Merkle tree corresponding to the common key ranges is exchanged between two nodes. We'll make the following comparison:

- Compare the hashes of the root node of Merkle trees.
- Do not proceed if they're the same.
- Traverse left and right children using recursion. The nodes identify whether or not they have any differences and perform the necessary synchronization.

The following slides explain more about how Merkle trees work.

'''

Note: We assume the ranges defined are hypothetical for illustration purposes.

'''

[anti-entropy](./ring)

The advantage of using Merkle trees is that each branch of the Merkle tree can be examined independently without requiring nodes to download the tree or the complete dataset. It reduces the quantity of data that must be exchanged for synchronization and the number of disc accesses that are required during the anti-entropy procedure.

The disadvantage is that when a node joins or departs the system, the tree's hashes are recalculated because multiple key ranges are affected.

We want our nodes to detect the failure of other nodes in the ring, so let's see how we can add it to our proposed design.

Promote membership in the ring to detect failures

The nodes can be offline for short periods, but they may also indefinitely go offline. We shouldn't rebalance partition assignments or fix unreachable replicas when a single node goes down because it's rarely a permanent departure. Therefore, the addition and removal of nodes from the ring should be done carefully.

Planned commissioning and decommissioning of nodes results in membership changes. These changes form history. They're recorded persistently on the storage for each node and reconciled among the ring members using a gossip protocol. A gossip-based protocol also maintains an eventually consistent view of membership. When two nodes randomly choose one another as their peer, both nodes can efficiently synchronize their persisted membership histories.

Let's learn how a gossip-based protocol works by considering the following example. Say node A starts up for the first time, and it randomly adds nodes B and E to its token set. The token set has virtual nodes in the consistent hash space and maps nodes to their respective token sets. This information is stored locally on the disk space of the node.

Now, node A handles a request that results in a change, so it communicates this to B and E. Another node, D, has C and E in its token set. It makes a change and tells C and E. The other nodes do the same process. This way, every node eventually knows about every other node's information. It's an efficient way to share information asynchronously, and it doesn't take a lot of bandwidth.

[ring](./promote)

'''

Question

Keeping in mind our consistent hashing approach, can the gossip-based protocol fail?

Answer

Yes, the gossip-based protocol can fail. For example, the virtual node, N1, of node A wants to be added to the ring. The administrator asks N2, which is also a virtual node of A. In such a case, both nodes consider themselves to be part of the ring and won't be aware that they're on the same server. If any change is made, it will keep on updating itself, which is wrong. This is called logical partitioning.

'''

'''

Question 2

How can we prevent logical partitioning?

Answer

We can make a few nodes play the role of seeds to avoid logical partitions. We can define a set of nodes as seeds via a configuration service. This set of nodes is known to all the working nodes since they can eventually reconcile their membership with a seed. So, logical partitions are pretty rare.

'''

Decentralized failure detection protocols use a gossip-based protocol that allows each node to learn about the addition or removal of other nodes. The join and leave methods of the explicit node notify the nodes about the permanent node additions and removals. The individual nodes detect temporary node failures when they fail to communicate with another node. If a node fails to communicate to any of the nodes present in its token set for the authorized time, then it communicates to the administrators that the node is dead.

Conclusion

A key-value store provides flexibility and allows us to scale the applications that have unstructured data. Web applications can use key-value stores to store information about a user's session and preferences. When using a user key, all the data is accessible, and key-value stores are ideal for rapid reads and write operations. Key-value stores can be used to power real-time recommendations and advertising because the stores can swiftly access and present fresh recommendations.

Key-value Store/Ensure Scalability and Replication/README.md

Ensure Scalability and Replication

Add scalability

Let's start with one of the core design requirements: scalability. We store key-value data in storage nodes. With a change in demand, we might need to add or remove storage nodes. It means we need to partition data over the nodes in the system to distribute the load across all nodes.

For example, let's consider that we have four nodes, and we want 25% of the requests to go to each node to balance the load equally. The traditional way to solve this is through the modulo operator. When a request comes in, we assign a request ID, calculate its hash, and find the remainder by taking the modulus with the number of nodes available. The remainder value is the node number, and we send the request to that node to process it.

The following slides explain this process:

[We get the hash of the requestID and take modulus with the number of nodes to find the node that should process the request](./add_scalability1.jpg)

[We perform the required operations on the RequestID to get the node](./add_scalability2.jpg)

[Node 2 will process the request](./add_scalability3.jpg)

We want to add and remove nodes with minimal change in our infrastructure. But in this method, when we add or remove a node, we need to move a lot of keys. This is inefficient. For example, node 2 is removed, and suppose for the same request ID, the new server to process a request will be node 1 because $10 \% 3 = 1$.

10%3=1. Nodes hold information in their local caches, like keys and their values. So, we need to move that request's data to the next node that has to process the request. But this replication can be costly and can cause high latency.

Next, we'll learn how to copy data efficiently.

'''

Question

Why didn't we use load balancers to distribute the requests to all nodes?

Answer

Load balancers distribute client requests according to an algorithm. That algorithm can be as simple as explained above, or it can be something detailed, as described in the next section. The next method we'll discuss can be one of the ways the load balancers balance the requests across the nodes.

'''

Consistent Hashing

Consistent hashing is an effective way to manage the load over the set of nodes. In consistent hashing, we consider that we have a conceptual ring of hashes from 0 to n-1, where n is the number of available hash values. We use each node's ID, calculate its hash, and map it to the ring. We apply the same process to requests. Each request is completed by the next node that it finds by moving in the clockwise direction in the ring.

Whenever a new node is added to the ring, the immediate next node is affected. It has to share its data with the newly added node while other nodes are unaffected. It's easy to scale since we're able to keep changes to our nodes minimal. This is because only a small portion of overall keys need to move. The hashes are randomly distributed, so we expect the load of requests to be random and distributed evenly on average on the ring.

[Consistent Hashing](./consistent_hashing)

The primary benefit of consistent hashing is that as nodes join or leave, it ensures that a minimal number of keys need to move. However, the request load isn't equally divided in practice. Any server that handles a large chunk of data can become a bottleneck in a distributed system. That node will receive a disproportionately large share of data storage and retrieval requests, reducing the overall system performance. As a result, these are referred to as hotspots.

As shown in the figure below, most of the requests are between the N4 and N1 nodes. Now, N1 has to handle most of the requests compared to other nodes, and it has become a hotspot. That means non-uniform load distribution has increased load on a single server.

'''

Note: It's a good exercise to think of possible solutions to the non-uniform load distribution before reading on.

'''

[non-uniform](./non_uniform.jpg)

Use virtual nodes

We'll use virtual nodes to ensure a more evenly distributed load across the nodes. Instead of applying a single hash function, we'll apply multiple hash functions onto the same key.

Let's take an example. Suppose we have three hash functions. For each node, we calculate three hashes and place them into the ring. For the request, we use only one hash function. Wherever the request lands onto the ring, it's processed by the next node found while moving in the clockwise direction. Each server has three positions, so the load of requests is more uniform. Moreover, if a node has more hardware capacity than others, we can add more virtual nodes by using additional hash functions. This way, it'll have more positions in the ring and serve more requests.

[virtual nodes](./virtual_nodes)

Advantages of virtual nodes

Following are some advantages of using virtual nodes:

'''

If a node fails or does routine maintenance, the workload is uniformly distributed over other

nodes. For each newly accessible node, the other nodes receive nearly equal load when it comes back online or is added to the system.

It's up to each node to decide how many virtual nodes it's responsible for, considering the heterogeneity of the physical infrastructure. For example, if a node has roughly double the computational capacity as compared to the others, it can take more load.

'''

We've made the proposed design of key-value storage scalable. The next task is to make our system highly available.

Data replication

We have various methods to replicate the storage. It can be either a primary-secondary relationship or a peer-to-peer relationship.

Primary-secondary approach

In a primary-secondary approach, one of the storage areas is primary, and other storage areas are secondary. The secondary replicates its data from the primary. The primary serves the write requests while the secondary serves read requests. After writing, there's a lag for replication. Moreover, if the primary goes down, we can't write into the storage, and it becomes a single point of failure.

'''

Question

Does the primary-secondary approach fulfill the requirements of the key-value store that we defined in the System Design: The Key-value Store lesson?

Answer

One of our requirements is that we need the ability to always write. This approach is good for the always read option. However, this approach doesn't include the ability to always write because it will overload the primary storage. Moreover, if a primary server fails, we need to upgrade a secondary to a primary. The availability of write will suffer as we won't allow writes during the switch-over time.

'''

Peer-to-peer approach

In the peer-to-peer approach, all involved storage areas are primary, and they replicate the data to stay updated. Both read and write are allowed on all nodes. Usually, it's inefficient and costly to replicate in all n nodes. Instead, three or five is a common choice for the number of storage nodes to be replicated.

[Peer-to-peer relationship](./p2p.jpg)

We'll use a peer-to-peer relationship for replication. We'll replicate the data on multiple hosts to achieve durability and high availability. Each data item will be replicated at n hosts, where n is a parameter configured per instance of the key-value store. For example, if we choose n to be 5, it means we want our data to be replicated to five nodes.

Each node will replicate its data to the other nodes. We'll call a node coordinator that handles read or write operations. It's directly responsible for the keys. A coordinator node assigned the key K . It's also responsible for replicating the keys to $n-1$ successors on the ring (clockwise). These lists of successor virtual nodes are called preference lists. To avoid putting replicas on the same physical nodes, the preference list can skip those virtual nodes whose physical node is already in the list.

Let's consider the illustration given below. We have a replication factor, n , set to 3. For key K , the replication is done on the next three nodes: B, C, and D. Similarly, for key L , the replication is done on nodes C, D, and E.

[Replication in key-value store](./kv.jpg)

Key-value Store/System Design The Key-value Store/README.md

System Design: The Key-value Store

Introduction to key-value stores

Key-value stores are distributed hash tables (DHTs). A key is generated by the hash function

and should be unique. In a key-value store, a key binds to a specific value and doesn't assume anything about the structure of the value. A value can be a blob, image, server name, or anything the user wants to store against a unique key.

Usually, it's preferred to keep the size of value relatively smaller (KB to MB). We can put large data in the blob store and put links to that data in the value field. Key-value stores are useful in many situations, such as storing user sessions in a web application and building NoSQL databases.

It's challenging to scale traditional databases with strong consistency and high availability in a distributed environment. Many real-world services like Amazon, Facebook, Instagram, Netflix, and many more use primary-key access to a data store instead of traditional online transaction processing (OLTP) databases. Examples of key-value store usage include bestseller lists, shopping carts, customer preferences, session management, sales rank, and product catalogs.

[Key-value store](./kv.jpg)

'''

Note: Many applications might not require a rich programming model provided by a traditional relational database management system (RDBMS). Using RDBMS for such applications is often expensive in terms of cost and performance.

'''

How will we design a key-value store?

We've divided the key-value system design into the following four lessons:

- Design a Key-value Store: We'll define the requirements of a key-value store and design the API.
- Ensure Scalability and Replication: We'll learn to achieve scalability using consistent hashing and replicate the partitioned data.
- Versioning Data and Achieving Configurability: We'll learn to resolve conflicts that occur due to changes made by more than one entity, and we'll make our system more configurable for different use cases.
- Enable Fault Tolerance and Failure Detection: We'll learn to make a key-value store fault tolerant and how to detect failures in the system.

Key-value Store/Versioning Data and Achieving Configurability/README.md

Versioning Data and Achieving Configurability

Data versioning

When network partitions and node failures occur during an update, an object's version history might become fragmented. As a result, it requires a reconciliation effort on the part of the system. It's necessary to build a way that explicitly accepts the potential of several copies of the same data so that we can avoid the loss of any updates. It's critical to realize that some failure scenarios can lead to multiple copies of the same data in the system. So, these copies might be the same or divergent. Resolving the conflicts among these divergent histories is essential and critical for consistency purposes.

'''

The version history contains the details of objects that have been modified.

'''

[Data Versioning](./version)

To handle inconsistency, we need to maintain causality between the events. We can do this using the timestamps and update all conflicting values with the value of the latest request. But time isn't reliable in a distributed system, so we can't use it as a deciding factor.

Another approach to maintaining causality effectively is by using vector clocks. A vector clock is a list of (node, counter) pairs. There's a single vector clock for every version of an object. If two objects have different vector clocks, we're able to tell whether they're causally related or not (more on this in a bit). Unless one of the two changes is reconciled,

the two are deemed at odds.

Modify the API design

We talked about how we can decide if two events are causally related or not using a vector clock value. For this, we need information about which node performed the operation before and what its vector clock value was. This is the context of an operation. So, weâ ll modify our API design as follows.

The API call to get a value should look like this:

'''

get(key)

'''

Parameter	Description
-----------	-------------

key	This is the key against which we want to get value.
-----	---

We return an object or a collection of conflicting objects along with a context. The context holds encoded metadata about the object, including details such as the objectâ s version.

The API call to put the value into the system should look like this:

'''

put(key, context, value)

'''

Parameter	Description
-----------	-------------

key	This is the key against which we have to store value.
-----	---

context	This holds the metadata for each object.
---------	--

value	This is the object that needs to be stored against the key.
-------	---

The function finds the node where the value should be placed on the basis of the key and stores the value associated with it. The context is returned by the system after the get operation. If we have a list of objects in context that raises a conflict, weâ ll ask the client to resolve it.

'''

Here the client is any front-end server in our trusted data center. It does not mean the end-user.

'''

To update an object in the key-value store, the client must give the context. We determine version information using a vector clock by supplying the context from a previous read operation. If the key-value store has access to several branches, it provides all objects at the leaf nodes, together with their respective version information in context, when processing a read request. Reconciling disparate versions and merging them into a single new version is considered an update.

'''

Note: This process of resolving conflicts is comparable to how itâ s done in Git. If Git is asked to merge multiple versions into one, merging is performed automatically. Itâ s up to the client (the developer) to resolve conflicts manually if automatic conflict resolution is not possible. Along the same lines, our system can try automatic conflict resolution and, if not possible, ask the application to provide a final resolved value.

'''

Vector clock usage example

Letâ s consider an example. Say we have a write operation request. Node A handles the first version of the write request, E1. The corresponding vector clock has node information and its counterâ that is, [A,1]. Node A handles another write for the same object on which the previous write was performed. So, for E2, we have [A,2]. E1 is no longer required because E2 was updated on the same node. E2 reads the changes made by E1, and then new changes are made. Suppose a

network partition happens. Now, the request is handled by two different nodes, B and C. The context with updated versions, which are E3, and their related clocks, which are ([A,2], [B,1] and ([A,2], [C,1]), are now in the system.

Suppose the network partition is repaired, and the client requests a write again, but now we have conflicts. The context ([A,3], [B,1], [C,1]) of the conflicts are returned to the client. After the client does reconciliation and A coordinates the write, we have E5 with the clock ([A,4]).

[Vector clock usage example](./example)

Compromise with vector clocks limitations

The size of vector clocks may increase if multiple servers write to the same object simultaneously. It's unlikely to happen in practice because writes are typically handled by of the top n nodes in a preference list.

For example, if there are network partitions or multiple server failures, write requests may be processed by nodes not in the top n nodes in the preference list. As a result we can have a long version like this: ([A,10], [B,4], [C,1], [D,2], [E,1], [F,3], [G,5], [H,7], [I,2], [J,2], [K,1], [L,1]). It's a hassle to store and maintain such a long version history.

We can limit the size of the vector clock in these situations. We employ a clock truncation strategy to store a timestamp with each (node, counter) pair to show when the data item was last updated by the node. Vector clock pairs are purged when the number of (node, counter) pairs exceeds a predetermined threshold (let's say 10). Because the descendant linkages can't be precisely calculated, this truncation approach can lead to a lack of efficiency in reconciliation.

The get and put operations

One of our functional requirements is that the system should be configurable. We want to control the trade-offs between availability, consistency, cost-effectiveness, and performance. So, let's achieve configurability by implementing the basic get and put functions of the key-value store.

Every node can handle the get (read) and put (write) operations in our system. A node handling a read or write operation is known as a coordinator. The coordinator is the first among the top n nodes in the preference list.

There can be two ways for a client to select a node:

- We route the request to a generic load balancer.
- We use a partition-aware client library that routes requests directly to the appropriate coordinator nodes.

Both approaches have their benefits. The client isn't linked to the code in the first approach whereas lower latency is achievable in the second. The latency is lower due to the reduced number of hops because the client can directly go to a specific server.

Let's make our service configurable by having an ability where we can control the trade-offs between availability, consistency, cost-effectiveness, and performance. We can use a consistency protocol similar to those used in quorum systems.

Let's take an example. Say n in the top n of the preference list is equal to 3. It means three copies of the data need to be maintained. We assume that nodes are placed in a ring. Say A, B, C, D, and E is the clockwise order of the nodes in that ring. If the write function is performed on node A, then the copies of that data will be placed on B and C. This is because B and C are the next nodes we find while moving in a clockwise direction of the ring.

'''

A quorum is the minimum number of votes that a distributed transaction has to obtain in order to be allowed to perform an operation in a distributed system. A quorum-based technique is implemented to enforce consistent operation in a distributed system. (Wikipedia)

Usage of r and w

Now, consider two variables, r and w . The r means the minimum number of nodes that need to be part of a successful read operation, while w is the minimum number of nodes involved in a successful write operation. So if $r=2$, it means our system will read from two nodes when we have data stored in three nodes. We need to pick values of r and w such that at least one node is common between them. This ensures that readers could get the latest-written value. For that we'll use a quorum-like system by setting $r + w > n$.

The following table gives an overview of how the values of n , r and w , affect the speed of reads and writes:

'''

Value Effects on Reads and Writes

n	r	w	Description
3	2	1	It won't be allowed as it violates our constraint $r + w > n$.
3	2	2	It will be allowed as it fulfills constraints.
3	3	1	It will provide speedy writes and slower reads since readers need to go to all n replicas for a value.
3	1	3	It will provide speedy reads from any node but slow writes since we now need to write to all n nodes synchronously.

'''

Let's say $n = 3$, which means we have three nodes where the data is copied to. Now, for $w = 2$ the operation makes sure to write in two nodes to make this request successful. For the third node, the data is updated asynchronously.

[Usage](./usage)

In this model, the latency of a get operation is decided by the slowest of the r replicas. The reason is that for the larger value of r , we focus more on availability and compromise consistency.

The coordinator produces the vector clock for the new version and writes the new version locally upon receiving a `put()` request for a key. The coordinator sends n highest-ranking nodes with the updated version and a new vector clock. We consider a write successful if at least w nodes respond. Remember that the coordinator writes to itself first, so we get w writes in total.

Requests for a `get()` operation are made to the n highest-ranked reachable nodes in a preference list for a key. They wait for r answers before returning the results to the client. Coordinators return all dataset versions that they regard as unrelated if they get several datasets from the same source (divergent histories that need reconciliation). The conflicting versions are then merged, and the resulting key's value is rewritten to override the previous versions.

By now, we've fulfilled the scalability, availability, conflict-resolution, and configurable service requirements. The last requirement is to have a fault-tolerant system. Let's discuss how we'll achieve it in the next lesson.

Load Balancer/Advanced Details of Load Balancers/README.md

Advanced Details of Load Balancers

This lesson will focus on some of the well-known algorithms used in the local load balancers.

Weâll also understand how load balancers are connected to form a hierarchy, sharing work across different tiers of LBs.

Algorithms of load balancers

Load balancers distribute client requests according to an algorithm. Some well-known algorithms are given below:

'''

Round-robin scheduling: In this algorithm, each request is forwarded to a server in the pool in a repeating sequential manner.

Weighted round-robin: If some servers have a higher capability of serving clientsâ requests, then itâs preferred to use a weighted round-robin algorithm. In a weighted round-robin algorithm, each node is assigned a weight. LBs forward clientsâ requests according to the weight of the node. The higher the weight, the higher the number of assignments.

Least connections: In certain cases, even if all the servers have the same capacity to serve clients, uneven load on certain servers is still a possibility. For example, some clients may have a request that requires longer to serve. Or some clients may have subsequent requests on the same connection. In that case, we can use algorithms like least connections where newer arriving requests are assigned to servers with fewer existing connections. LBs keep a state of the number and mapping of existing connections in such a scenario. Weâll discuss more about state maintenance later in the lesson.

Least response time: In performance-sensitive services, algorithms such as least response time are required. This algorithm ensures that the server with the least response time is requested to serve the clients.

IP hash: Some applications provide a different level of service to users based on their IP addresses. In that case, hashing the IP address is performed to assign usersâ requests to servers.

URL hash: It may be possible that some services within the application are provided by specific servers only. In that case, a client requesting service from a URL is assigned to a certain cluster or set of servers. The URL hashing algorithm is used in those scenarios.

'''

There are other algorithms also, like randomized or weighted least connections algorithms.

Static versus dynamic algorithms

Algorithms can be static or dynamic depending on the machineâs state. Letâs look at each of these categories individually:

'''

Static algorithms donât consider the changing state of the servers. Therefore, task assignment is carried out based on existing knowledge about the serverâs configuration. Naturally, these algorithms arenât complex, and they get implemented in a single router or commodity machine where all the requests arrive.

Dynamic algorithms are algorithms that consider the current or recent state of the servers. Dynamic algorithms maintain state by communicating with the server, which adds a communication overhead. State maintenance makes the design of the algorithm much more complicated.

'''

Dynamic algorithms require different load balancing servers to communicate with each other to exchange information. Therefore, dynamic algorithms can be modular because no single entity will do the decision-making. Although this adds complexity to dynamic algorithms, it results in improved forwarding decisions. Finally, dynamic algorithms monitor the health of the servers and forward requests to active servers only.

'''

Note: In practice, dynamic algorithms provide far better results because they maintain a state of serving hosts and are, therefore, worth the effort and complexity.

'''

Stateful versus stateless LBs

While static and dynamic algorithms are required to consider the health of the hosting servers, a state is maintained to hold session information of different clients with hosting servers.

If the session information isn't kept at a lower layer (distributed cache or database), load balancers are used to keep the session information. Below, we describe two ways of handling session maintenance through LBs:

'''

Stateful

Stateless

'''

Stateful load balancing

As the name indicates, stateful load balancing involves maintaining a state of the sessions established between clients and hosting servers. The stateful LB incorporates state information in its algorithm to perform load balancing.

Essentially, the stateful LBs retain a data structure that maps incoming clients to hosting servers. Stateful LBs increase complexity and limit scalability because session information of all the clients is maintained across all the load balancers. That is, load balancers share their state information with each other to make forwarding decisions.

[Stateful load balancing](./stateful.jpg)

Stateless load balancing

Stateless load balancing maintains no state and is, therefore, faster and lightweight. Stateless LBs use consistent hashing to make forwarding decisions. However, if infrastructure changes (for example, a new application server joining), stateless LBs may not be as resilient as stateful LBs because consistent hashing alone isn't enough to route a request to the correct application server. Therefore, a local state may still be required along with consistent hashing.

[Stateless load balancers using hash buckets to map requests to end servers](./hash.jpg)

Therefore, a state maintained across different load balancers is considered as stateful load balancing. Whereas, a state maintained within a load balancer for internal use is assumed as stateless load balancing.

Types of load balancers

Depending on the requirements, load balancing can be performed at the network/transport and application layer of the open systems interconnection (OSI) layers.

'''

Layer 4 load balancers: Layer 4 refers to the load balancing performed on the basis of transport protocols like TCP and UDP. These types of LBs maintain connection/session with the clients and ensure that the same (TCP/UDP) communication ends up being forwarded to the same back-end server. Even though TLS termination is performed at layer 7 LBs, some layer 4 LBs also support it.

Layer 7 load balancers: Layer 7 load balancers are based on the data of application layer protocols. It's possible to make application-aware forwarding decisions based on HTTP header URLs, cookies, and other application-specific data—for example, user ID. Apart from performing TLS termination, these LBs can take responsibilities like rate limiting users, HTTP routing, and header rewriting.

'''

'''

Note: Layer 7 load balancers are smart in terms of inspection. However layer 4 load balancers are faster in terms of processing. For instance, Google has developed a high-performance distributed load balancer called Maglev for their cloud platform.

Load balancer deployment

We discussed the trade-offs of load balancing performed at different OSI layers. In practice, however, a single layer LB isn't enough for a large data center. In fact, multiple layers of load balancers coordinate to take informed forwarding decisions. A traditional data center may have a three-tier LB as shown below:

[Three-tier load balancer in a typical data center](./3tiers.jpg)

Tier-0 and tier-1 LBs

If DNS can be considered as the tier-0 load balancer, equal cost multipath (ECMP) routers are the tier-1 LBs. From the name of ECMP, it's evident that this layer divides incoming traffic on the basis of IP or some other algorithm like round-robin or weighted round-robin. Tier-1 LBs will balance the load across different paths to higher tiers of load balancers.

ECMP routers play a vital role in the horizontal scalability of the higher-tier LBs.

Tier-2 LBs

The second tier of LBs include layer 4 load balancers. Tier-2 LBs make sure that for any connection, all incoming packets are forwarded to the same tier-3 LBs. To achieve this goal, a technique like consistent hashing can be utilized. But in case of any changes to the infrastructure, consistent hashing may not be enough. Therefore, we have to maintain a local or global state as we'll see in the coming sections of the lesson.

Tier-2 load balancers can be considered the glue between tier-1 and tier-3 LBs. Excluding tier-2 LBs could result in erroneous forwarding decisions in case of failures or dynamic scaling of LBs.

Tier-3 LBs

Layer 7 LBs provide services at tier 3. Since these LBs are in direct contact with the back-end servers, they perform health monitoring of servers at HTTP level. This tier enables scalability by evenly distributing requests among the set of healthy back-end servers and provides high availability by monitoring the health of servers directly. This tier also reduces the burden on end-servers by handling low-level details like TCP-congestion control protocols, the discovery of Path MTU (maximum transmission unit), the difference in application protocol between client and back-end servers, and so on. The idea is to leave the computation and data serving to the application servers and effectively utilize load balancing commodity machines for trivial tasks. In some cases, layer 7 LBs are at the same level as the service hosts.

'''

TCP makes use of a congestion window to estimate the number of in-flight packets in a connection to avoid network congestion. Two well-known TCP congestion protocols are Tahoe and Reno.

'''

'''

Path MTU is the largest packet size that a source can send on the network beyond which packet fragmentation will happen. This number is equal to the smallest MTU on the network path.

'''

'''

To summarize, tier 1 balances the load among the load balancers themselves. Tier 2 enables a smooth transition from tier 1 to tier 3 in case of failures, whereas tier 3 does the actual load balancing between back-end servers. Each tier performs other tasks to reduce the burden on end-servers.

'''

Practical example

Let's look at an example where requests from a client come in and get forwarded to different application servers based on the application data inside the client's network packets.

[Practical example](./example1.jpg)
[Practical example](./example2.jpg)

Let's look at the illustration above in the following steps:

1. R1 indicates request 1 coming through one of the ECMP routers (tier-1 LBs).
2. ECMP routers forward R1 to any of the three available tier-2 LBs using a round-robin algorithm. Tier-2 LBs take a hash of the source IP address(IPc) and forward the packet to the next tier of LBs.
3. Tier-3, upon receiving the packet, offloads TLS and reads the HTTP(S) data. By observing the requested URL, it forwards the request to the server handling requests for slides.

R2 takes the same path but a different end-server because the requested URL contains document instead of slides. Tier-3 LBs are preconfigured to forward requests to application servers based on the application data. For instance, a typical HAProxy server can have the following configuration in tier-3 LBs:

```
mode HTTP //define which mode the LB will work on, TCP for tier-1 LBs
acl slidesApp path_end -i /presentation //define a category of applications if the path ends in /presentation
use_backend slidesServers if slidesApp // use a set of backend servers if the request arrives for slidesApp
backend slidesServers // listing servers serving slidesApp
server slides1 192.168.12.1:80 //using slides1 server to serve slidesApp.
```

HAProxy sample configuration for

layer 7 load balancers

'''

Question 1

After a request reaches a back-end server, should the response be routed back through each tier of the load balancers?

Answer

No, the server can send the response directly to the routers (tier-1 LBs) through tier-3 LBs, which can forward the response from the data center. Such a response is called direct routing (DR) or direct server return (DSR).

'''

Question 2

Why don't the servers directly send the response to the routers (tier-1 LBs) instead of tier-3 LBs?

Answer

Tier-3 LBs maintain some state of connection—for example, SSL encryption/decryption. This is necessary to give clients a seamless experience.

'''

Question 3

According to you, which tier of LBs is more prone to bugs?

Answer

Tier 3 has more complexity, which makes it more prone to bugs.

'''

Question 4

The image above shows a higher number of tier-3 load balancers than that of tier-2. What do you think is the reason for such a representation?

Answer

Tier 3 performs application-specific analysis and substantially more sophisticated computations. Therefore, handling the same number of queries as tier 2 requires a higher number of machines. Furthermore, tier-3 LBs maintain the state of a large number of application servers, which may not be possible using the same number of LBs as tier 2.

Implementation of load balancers

Different kinds of load balancers can be implemented depending on the number of incoming requests, organization, and application-specific requirements:

Hardware load balancers

Load balancers were introduced in the 1990s as hardware devices. Hardware load balancers work as stand-alone devices and are quite expensive. Nonetheless, they have their performance benefits and are able to handle a lot of concurrent users. Configuration of hardware-based solutions is problematic because it requires additional human resources. Therefore, they aren't the go-to solutions even for large enterprises that can afford them. Availability can be an issue with hardware load balancers because additional hardware will be required to failover to in case of failures. Finally, hardware LBs can have higher maintenance/operational costs and compatibility issues making them less flexible. Not to mention that hardware LBs have vendor locks as well.

Software load balancers

Software load balancers are becoming increasingly popular because of their flexibility, programmability, and cost-effectiveness. That's all possible because they're implemented on commodity hardware. Software LBs scale well as requirements grow. Availability won't be an issue with software LBs because small additional costs are required to implement shadow load balancers on commodity hardware. Additionally, software LBs can provide predictive analysis that can help prepare for future traffic patterns.

Cloud load balancers

With the advent of the field of cloud computing, Load Balancers as a Service (LBaaS) has been introduced. This is where cloud owners provide load balancing services. Users pay according to their usage or the service-level agreement (SLA) with the cloud provider. Cloud-based LBs may not necessarily replace a local on-premise load balancing facility, but they can perform global traffic management between different zones. Primary advantages of such load balancers include ease of use, management, metered cost, flexibility in terms of usage, auditing, and monitoring services to improve business decisions. An example of how cloud-based LBs can provide GSLB is given below:

[GSLB is obtained through LBaaS, and the regions contain data centers that are the property of application providers]

'''

Note: Another interesting implementation of load balancers comes in the form of client-side load balancing. Client-side load balancing is suited where there are numerous services, each with many instances (such as load balancing in Twitter). Our focus, however, is on traditional load balancers because most three-tier applications employ these in their design.

'''

[GSLB is obtained through LBaaS, and the regions contain data centers that are the property of application providers](./gslb.jpg)

Conclusion

LBs have come a long way to become a service offered in the cloud, starting since their inception in the form of hardware. They're a key component of any enterprise-level service. Horizontal scalability of hosting servers will always require a good load balancing layer capable of providing load balancing, session maintenance, TLS offloading, service discovery,

and more.

Load Balancer/Global and Local Load Balancing/README.md

Global and Local Load Balancing

Introduction

From the previous lesson, it may seem like load balancing is performed only within the data center. However, load balancing is required at a global and a local scale. Let's understand function of each of the two:

'''

Global server load balancing (GSLB): GSLB involves the distribution of traffic load across multiple geographical regions.

Local load balancing: This refers to load balancing achieved within a data center. This type of load balancing focuses on improving efficiency and better resource utilization of the hosting servers in a data center.

'''

Let's understand each of the two techniques below.

Global server load balancing

GSLB ensures that globally arriving traffic load is intelligently forwarded to a data center. For example, power or network failure in a data center requires that all the traffic be rerouted to another data center. GSLB takes forwarding decisions based on the users' geographical locations, the number of hosting servers in different locations, the health of data centers, and so on.

In the next lesson, we'll also learn how GSLB offers automatic zonal failover. GSLB service can be installed on-premises or obtained through Load Balancing as a Service (LBaaS).

The illustration below shows that the GSLB can forward requests to three different data centers. Each local load balancing layer within a data center will maintain a control plane connection with the GSLB providing information about the health of the LBs and the server farm. GSLB uses this information to drive traffic decisions and forward traffic load based on each region's configuration and monitoring information.

[Usage of global load balancing to send user requests to different regions](./global_local.jpg)

Now, we'll discuss how the domain name system (DNS) can perform GSLB.

'''

LBaaS is a cloud-based load balancing technique that follows a pay-per-use model. LBaaS offers the flexibility of automatic scaling up or down depending upon the traffic load.

'''

Load balancing in DNS

We understand that DNS can respond with multiple IP addresses for a DNS query. In the lesson on DNS, we discussed that it's possible to do load balancing through DNS while looking at the output of nslookup. DNS uses a simple technique of reordering the list of IP addresses in response to each DNS query. Therefore, different users get a reordered IP address list. It results in users visiting a different server to entertain their requests. In this way, DNS distributes the load of requests on different data centers. This is performing GSLB. In particular, DNS uses round-robin to perform load balancing as shown below:

[Load balancing in DNS](./lb)

As shown above, round-robin in DNS forwards clients to data centers in a strict circular order. However, round-robin has the following limitations:

'''

Different ISPs have a different number of users. An ISP serving many customers will provide the same cached IP to its clients, resulting in uneven load distribution on end-servers.

Because the round-robin load-balancing algorithm doesn't consider any end-server crashes, it keeps on distributing the IP address of the crashed servers until the TTL of the cached entries expires. Availability of the service, in that case, can take a hit due to DNS-level load balancing.

Despite its limitations, round-robin is still widely used by many DNS service providers. Furthermore, DNS uses short TTL for cached entries to do effective load balancing among different data centers.

'''

'''

Note: DNS isn't the only form of GSLB. Application delivery controllers (ADCs) and cloud-based load balancing (discussed later) are better ways to do GSLB.

'''

'''

Application delivery controllers (ADCs) are part of the application delivery network (ADN). They can be considered the superset of LBs offering various services, including load balancing. The primary task of ADCs is to perform web acceleration to reduce the load from the server farm. Some well-known services between layers 3 and 7 include caching, SSL offloading, proxy/reverse proxy services, IP traffic optimization, and many more. ADCs also implement GSLB.

'''

The need for local load balancers

DNS plays a vital role in balancing the load, but it suffers from the following limitations:

'''

The small size of the DNS packet (512 Bytes) isn't enough to include all possible IP addresses of the servers.

There's limited control over the client's behavior. Clients may select arbitrarily from the received set of IP addresses. Some of the received IP addresses may belong to busy data centers.

Clients can't determine the closest address to establish a connection with. Although DNS geolocation and anycasting-like solutions can be implemented, they aren't trivial solutions.

In case of failures, recovery can be slow through DNS because of the caching mechanism, especially when TTL values are longer.

'''

To solve some of the above problems, we need another layer of load balancing in the form of local LB. In the next lesson, we'll discuss different details about local load balancers.

What is local load balancing?

Local load balancers reside within a data center. They behave like a reverse proxy and make their best effort to divide incoming requests among the pool of available servers. Incoming clients' requests seamlessly connect to the LB that uses a virtual IP address (VIP).

'''

Reverse proxy is a component at the edge of the server-side network that sits between the server and the outside world. It can provide features like privacy, security, caching, etc.

'''

'''

A VIP is an address that has no physical machine to correspond to. In fact, a number of machines will use the same Virtual IP address. The failure of an interface on a network results in the loss of the packets sent to it. However, using VIP, packets are automatically rerouted to a failover machine. From the outside, a VIP will appear as a single machine. Internally, however, a group of machines will use the same address.

DNS queries are usually responded with VIPs.

'''
'''

Question

Can DNS be considered a global server load balancer (GSLB)?

Answer

Yes, there are actually two ways of doing global traffic management (GTM):

GTM through ADCs: Some ADCs implement GSLB. In that case, ADCs have a real-time view of the hosting servers and forward requests based on the health and capacity of the data center.

GTM through DNS: DNS does GSLB by analyzing the IP location of the client. For each user requesting IP for a domain name (for example, www.educative.io), DNS-based GSLB forwards the IP address of the data center geographically closer to the requesting IP location.

'''

In the next lesson, weâ ll explore some advanced details of local load balancers.

Load Balancer/Introduction to Load Balancers/README.md

Introduction to Load Balancers

What is load balancing?

Millions of requests could arrive per second in a typical data center. To serve these requests, thousands (or a hundred thousand) servers work together to share the load of incoming requests.

'''

Note: Here, itâ s important that we consider how the incoming requests will be divided among the available servers.

'''

A load balancer (LB) is the answer to the question. The job of the load balancer is to fairly divide all clientsâ requests among the pool of available servers. Load balancers perform this job to avoid overloading or crashing servers.

The load balancing layer is the first point of contact within a data center after the firewall. A load balancer may not be required if a service entertains a few hundred or even a few thousand requests per second. However, for increasing client requests, load balancers provide the following capabilities:

- Scalability: By adding servers, the capacity of the application/service can be increased seamlessly. Load balancers make such upscaling or downscaling transparent to the end users.
- Availability: Even if some servers go down or suffer a fault, the system still remains available. One of the jobs of the load balancers is to hide faults and failures of servers.
- Performance: Load balancers can forward requests to servers with a lesser load so the user can get a quicker response time. This not only improves performance but also improves resource utilization.

Hereâ s an abstract depiction of how load balancers work:

[Simplified working of a load balancer](./lbsimplified.jpg)

Placing load balancers

Generally, LBs sit between clients and servers. Requests go through to servers and back to clients via the load balancing layer. However, that isnâ t the only point where load balancers are used.

Letâ s consider the three well-known groups of servers. That is the web, the application, and the database servers. To divide the traffic load among the available servers, load balancers can be used between the server instances of these three services in the following way:

'''

Place LBs between end users of the application and web servers/application gateway.
Place LBs between the web servers and application servers that run the business/application logic.

Place LBs between the application servers and database servers.

'''

[Possible usage of load balancers in a three-tier architecture](./lb_usage.jpg)

In reality, load balancers can be potentially used between any two services with multiple instances within the design of a system.

Services offered by load balancers

LBs not only enable services to be scalable, available, and highly performant, they offer some key services like the following:

- Health checking: LBs use the heartbeat protocol to monitor the health and, therefore, reliability of end-servers. Another advantage of health checking is the improved user experience.

- TLS termination: LBs reduce the burden on end-servers by handling TLS termination with the client.

- Predictive analytics: LBs can predict traffic patterns through analytics performed over traffic passing through them or using statistics of traffic obtained over time.

- Reduced human intervention: Because of LB automation, reduced system administration efforts are required in handling failures.

- Service discovery: An advantage of LBs is that the clients's requests are forwarded to appropriate hosting servers by inquiring about the service registry.

- Security: LBs may also improve security by mitigating attacks like denial-of-service (DoS) at different layers of the OSI model (layers 3, 4, and 7).

As a whole, load balancers provide flexibility, reliability, redundancy, and efficiency to the overall design of the system.

'''

The heartbeat protocol is a way of identifying failures in distributed systems. Using this protocol, every node in a cluster periodically reports its health to a monitoring service.

'''

'''

TLS termination, also called TLS/SSL offloading, is the establishment of secure communication channels between clients and servers through encryption/decryption of data.

'''

'''

Service registry is a repository of the (micro)services and the instances available against each service.

'''

'''

The DoS is an attack where a client floods the server with traffic to exhaust the server's resources (processing and/or memory) such that it is unable to process a legitimate user's request.

'''

'''

The open systems interconnection (OSI) model is a conceptual framework that divides the problem of connecting any two machines into seven different layers.

'''

'''

Flexibility: Add or remove machines transparently on the fly.

Reliability: Buggy hosts can be removed through health monitoring which makes the system reliable.

Redundancy: Multiple paths leading to the same destination or failed server's load is rerouted to the failover machine.

Efficiency: Divide load evenly on all machines to use them effectively from the point of view of the service provider.

'''

'''

Question

What if load balancers fail? Are they not a single point of failure (SPOF)?

Answer

Load balancers are usually deployed in pairs as a means of disaster recovery. If one load balancer fails, and there's nothing to failover to, the overall service will go down. Generally, to maintain high availability, enterprises use clusters of load balancers that use heartbeat communication to check the health of load balancers at all times. On failure of primary LB, the backup can take over. But, if the entire cluster fails, manual rerouting can also be performed in case of emergencies.

'''

In the coming lessons, we'll see how load balancers can be used in complex applications and which type of load balancer is appropriate for which use case.

Monitor Client-side Errors/Design of a Client-side Monitoring System/README.md

Design of a Client-side Monitoring System

A service has no visibility of the errors that don't occur at its infrastructure. Still, such failures are equally frustrating for the customers, and they might have to ask their friends, "Is the service X down for you as well?" or head to sites like Downtdetector to see if anyone else is reporting the same issues. They might report the problem via a Tweet or some other communication channel. However, all such cases have a slow feedback loop. As a service provider, we want to detect such problems as quickly as possible to take remedial measures. Let's design such a system.

Initial design

To ensure that the client's requests reach the server, we'll act as clients and perform reachability and health checks. We'll need various vantage points across the globe. We can run a service, let's call it prober, that periodically sends requests to the service to check availability. This way, we can monitor reachability to our service from many different places.

[Prober checking availability of website](./probe.jpg)

Issues with probers

We can have the following issues with probers:

- Incomplete coverage: We might not have good coverage across all autonomous systems. There are 100,000 unique autonomous systems on the Internet as of March 2021. It's not cost-effective even possible to put those many probes across the globe. Country or ISP-specific regulations and the need for periodic maintenance are additional hurdles to implementing such a scheme.

- Lack of user imitation: Such probes might not represent a typical user behavior to explain how a typical user will use the service.

'''

Note: The initial design is based on active probing.

'''

Improve the design

Instead of using a prober on vantage points, we can embed the probes into the actual application instead. We'll have the following two components:

- Agent: This is a prober embedded in the client application that sends the appropriate service reports about any failures.

- Collector: This is a report collector independent of the primary service. It's made independent to avoid the situations where client agents want to report an error to the failed service. We summarize errors reports from collectors and look for spikes in the errors graph to see client-side issues.

The following illustration shows how an agent reaches an independent collector when primary service isn't reachable:

[Evaluating the agent's report to see a spike in errors](./evaluate.jpg)

These collectors are a hierarchy of big data processing systems. We can place them near the client network, and over time, we can accumulate these statistics from all such localized sites. We'll use online stream processing systems to make such a system near real-time. If we're mainly looking for summary statistics, our system can tolerate the loss of some error reports. Some reports will be relative to the overall user population. We might say 1% of service users are "some." If we don't want to lose any reports, we'll need to design a with more care, which will be more expensive.

Now, we'll solve the following concerns:

- Can a user activate and deactivate client-side reports?
- How do client-side agents reach collectors under faulty conditions?
- How will we protect user privacy?

Activate and deactivate reports

We'll use a custom HTML header to send appropriate information to the collectors. Though a client accesses the service via a browser, a specific browser should know about this feature to appropriately fill in the header information in the HTTP requests. For organizations that make browsers and provide services (for example, Chromium-based browsers), such features can be incorporated and standardized over time.

Another solution can be to use a client-side application that the service controls, and then we can easily include such headers over HTTP.

The client can fill in the request header if the client has already consented to that. The service can then reply with appropriate values for the policy and collection endpoints.

[Activate and deactivate reports](./activate_deactivate.jpg)

Reach collectors under faulty conditions

The collectors need to be in a different failure domain from the web service endpoint that we're trying to monitor. The client side can try various collectors in different failure domains until one works. We can see a similar pattern in the following examples. At times, we refer to such a phenomenon as being outside the blast radius of a fault.

If we want to see the reachability of an IP, we host the service on a different IP. If we monitor the availability of a domain, we host the collector on a different domain. And if we want to detect that an autonomous system route isn't hijacked, we host the service in a

different autonomous system. Though, for last-mile errors, there isn't much we could do as a service provider. We might accumulate such events at the client side and report them on the next connectivity. A service can influence the remaining component failures.

'''

Reaching Collectors Under Faulty Conditions

1.2.3.4 unreachable	Different server IP
Can't resolve example.com	Different domain
AS 1234 hijacked	Different ASN
CDN available	Different/no CDN
Last-mile problems	No readily available fall-back for the service

Protect user privacy

The human user who uses the client-side software should be in full control to precisely know what data is collected and sent with each request. The user should also be able to reactivate the feature any time they wish. If we use our client-side application (and not a browser application), we have a lot of flexibility in what diagnostic could be included in the report. For a browser-based client, we can avoid the following information:

- We can avoid including traceroute hops to see a client to the service path. Users can be susceptible to their geographic location. It might be akin to collecting location information.
- We can avoid including which DNS resolver is being used. Again, details of DNS can leak some information about the location.
- We can avoid including round-trip-time (RTT) and packet loss information.

'''

Note: As a guiding rule, we should try to collect as little information as possible, and it must only be used for the specific purpose a user gave consent for.

'''

Ideally, for a web-based client, we should only collect the information that's logged in the weblog when any request has been successful.

We shouldn't use any active probing except to test the service's standard functionality and report such probes' results. So, traceroute and RTT or packet loss information is excluded.

Any intermediary (like ISPs or middleboxes) can't change, add, or remove the error reporting mechanism due to encryption. Similarly, designated collectors are the only place where such data can go.

Conclusion

In a distributed system, it's difficult to detect and respond to errors on the client side. It's necessary to monitor such events to provide a good user experience.

We can handle errors using an independent agent that sends service reports about any failures to a collector. Such collectors should be independent of the primary service in terms of infrastructure and deployment.

Monitor Client-side Errors/Focus on Client-side Errors in a Monitoring System/README.md

Focus on Client-side Errors in a Monitoring System

Client-side errors

In a distributed system, clients often access the service via an HTTP request. We can monitor our web and application servers' logs if a request fails to process. If multiple requests fail, we can observe a spike in internal errors (error 500).

Those errors whose root cause is on the client side are hard to respond to because the service has little to no insight into the client's system. We might try to look for a dip in the load compared to averages, but such a graph is usually hard. It can have false positives and false negatives due to factors such as unexpectedly variable load or if a small portion of the client population is affected.

There are many factors that can cause failures that can result in clients being unable to reach the server. These include the following:

- Failure in DNS name resolution.
- Any failure in routing along the path from the client to the service provider.
- Any failures with third-party infrastructure, such as middleboxes and content delivery networks (CDNs).

[Server-side errors versus client-side errors](./server_client.jpg)

Failures due to a routing bug

Let's look at a real-world example of an error that impacted a large number of service customers, but the service wasn't readily aware of it.

One of Google's peer ISPs accidentally announced Internet routes that it wasn't supposed to. As a result, the traffic of many of Google's customers started routing through unintended ISPs and wasn't reaching Google. Clients were frustrated because they weren't able to reach Google, while Google might have been unaware of such problems right away because these issues didn't happen on its infrastructure.

[BGP leak](./bgp_leak.jpg)

The above leak isn't unique. Similar issues keep arising. Another such leakage happened on April 16, 2021, when an AS mistakenly announced over 30,000 BGP prefixes. This resulted in a 100 times spike in the inbound traffic to their network. However, an increase in influx was observed, and the problem was solved.

The impacted services' monitoring systems might not catch the above events readily. Monitoring such situations is crucial so that the application remains available for all of its customers. Therefore, in the next lessons, we'll go through methods that help us to monitor the situations mentioned above.

Monitor Server-side Errors/Design of a Blob Store/README.md

Design of a Monitoring System

Requirements

Let's sum up what we want our monitoring system to do for us:

- Monitor critical local processes on a server for crashes.
- Monitor any anomalies in the use of CPU/memory/disk/network bandwidth by a process on a server.
- Monitor overall server health, such as CPU, memory, disk, network bandwidth, average load, and so on.
- Monitor hardware component faults on a server, such as memory failures, failing or slowing disk, and so on.
- Monitor the server's ability to reach out-of-server critical services, such as network file systems and so on.
- Monitor all network switches, load balancers, and any other specialized hardware inside a

data center.

- Monitor power consumption at the server, rack, and data center levels.
- Monitor any power events on the servers, racks, and data center.
- Monitor routing information and DNS for external clients.
- Monitor network links and pathsâ latency inside and across the data centers.
- Monitor network status at the peering points.
- Monitor overall service health that might span multiple data centersâ for example, a CDN and its performance.

We want automated monitoring that identifies an anomaly in the system and informs the alert manager or shows the progress on a dashboard. Cloud service providers provide a health status of their services:

- AWS: <https://health.aws.amazon.com/health/status>
- Azure: <https://status.azure.com/en-us/status>
- Google: <https://status.cloud.google.com/>

Building block we will use

The design of distributed monitoring will consist of the following building block:

Blob storage: Weâ ll use blob storage to store our information about metrics.

High-level design

The high-level components of our monitoring service are the following:

- Storage: A time-series database stores metrics data, such as the current CPU use or the number of exceptions in an application.
- Data collector service: This fetches the relevant data from each service and saves it in the storage.
- Querying service: This is an API that can query on the time-series database and return the relevant information.

[High-level design of a monitoring system](./system.jpg)

Letâ s dive deep into the components mentioned above in the next lesson.

Monitor Server-side Errors/Detailed Design of a Monitoring System/README.md

Detailed Design of a Monitoring System

Weâ ll discuss the core components of our monitoring system, identify the shortcomings of our design, and improve the design to fulfill our requirements.

Storage

Weâ ll use time-series databases to save the data locally on the server where our monitoring service is running. Then, weâ ll integrate it with a separate storage node. Weâ ll use blob storage to store our metrics.

We need to store metrics and know which action to perform if a metric has reached a particular value. For example, if CPU usage exceeds 90%, we generate an alert to the end user so the alert receiver can do take the necessary steps, such as allocate more resources to scale. For this purpose, we need another storage area that will contain the rules and actions. Letâ s call it rules database. Upon any violation of the rules, we can take appropriate action.

Here, we have identified two more components in our designâ that is, a rules and action database.

and a storage node (a blob store).

[Adding blob storage and a rules and action database](./system.jpg)

Data collector

We need a monitoring system to update us about our several data centers. We can stay updated if the information about our processes reaches us, which is possible through logging. We'll choose a pull strategy. Then, we'll extract our relevant metrics from the logs of the application. As discussed in our logging design, we used a distributed messaging queue. The message in the queue has the service name, ID, and a short description of the log. This will help us identify the metric and its information for a specific service. Exposing the relevant metrics to the data collector is necessary for monitoring any service so that our data collector can get the metrics from the service and store them into the time-series database.

A real-world example of a monitoring system based on the pull-based approach is DigitalOcean. It monitors millions of machines that are dispersed globally.

'''

DigitalOcean, Inc. is an American cloud infrastructure provider that allows developers to deploy and scale applications that run simultaneously on multiple computers.

'''

'''

Question

What are some drawbacks of using a push-based approach?

Answer

The push-based monitoring tool collects metric data from the applications and servers and sends it to a central collection platform. Each microservice sends its metrics to the monitoring system, resulting in a heavy traffic load on the infrastructure. This is how monitoring might become a bottleneck for business operations. The monitoring can be near real-time. However, if appropriate care isn't taken, it can overwhelm the infrastructure with continual push requests from all services, resulting in network floods. We must also install daemons on each of these targets to send metrics to the monitoring server, which requires additional work.

'''

Service discoverer

The data collector is responsible for fetching metrics from the services it monitors. This way the monitoring system doesn't need to keep a track of services. Instead, it can find them using a discoverer service. We'll save the relative information of the services we have to monitor. We'll use a service discovery solution and integrate with several platforms and tools, including EC2, Kubernetes, and Consul. This will allow us to discover which services we have to monitor. Similar dynamic discovery can be used for newly commissioned hardware.

Let's add our newly identified component to our existing design.

[Adding the service discoverer](./discoverer.jpg)

Querying service

We want a service to access the database and fetch the relevant query results. We need this because we want to view the errors like values of a particular node's memory usage, or send an alert if a metric exceeds the set limit. Let's add the two components we need along with querying.

Alert Manager

The alert manager is responsible for sending alerts upon violations of set rules. It can send alerts as an email, a Slack message, and so on.

Dashboard

We can set dashboards by using the collected metrics to display the required information. For example, the number of requests in the current week.

Let's add the components discussed above, which completes our design of the monitoring system.

[Detailed design of monitoring system](./dashboard.jpg)

Our all-in-one monitoring service works for actively tracking systems and services. It collects and stores data, and it supports searches, graphs, and alerts.

Pros

- The design of our monitoring service ensures the smooth working of the operations and keeps an eye on signs of impending problems.
- Our design avoids overloading the network traffic by fetching the data itself.
- The monitoring service provides higher availability.

Cons

- The system seems scalable, but managing more servers to monitor can be a problem. For example, we have a dedicated server responsible for running the monitoring service. It can be a single point of failure (SPOF). To cater to SPOF, we can have a failover server for our monitoring system. Then, we also need to maintain consistency between actual and failover servers. However, such a design will also hit a scalability ceiling as the number of servers further increase.
- Monitoring collects an enormous amount of data 24/7, and keeping it forever might not be feasible. We need a policy and mechanisms to delete unwanted data periodically to efficiently utilize the resources.

Let's think of a way to overcome the problems with our monitoring service.

Improving our design

We want to improve our design so that our system can scale better and decide what data to keep and what to delete. Let's see how the push-based approach works. In a push-based approach, the application pushes its data to the monitoring system.

[Push-based monitoring system](./push.jpg)

We used a pull-based strategy to avoid network congestion. This also allows the applications to be free of the aspect that they have to send the relevant monitoring data to the system. Instead, the monitoring system fetches or pulls the data itself. To cater to scaling needs, we need to apply a push-based approach too. We'll use a hybrid approach by combining our pull-based strategy with the push-based strategy.

We'll keep using a pull-based strategy for several servers within a data center. We'll also assign several monitoring servers for hundreds or thousands of servers within a data center. Let's say one server monitoring 5,000 servers. We'll call them secondary monitoring servers.

Now, we'll apply the push-based strategy. The secondary monitoring systems will push their data to a primary data center server. The primary data center server will push its data to a global monitoring service responsible for checking all the data centers spread globally.

We'll use blob storage to store our excessive data, apply elastic search, and view our relevant stats using a visualizer. As our servers or data centers increase, we'll add more monitoring systems. The design for this is given below.

'''

Note: Using a hierarchy of systems for scaling is a common design pattern in system design. By increasing nodes on a level or introducing additional levels in the hierarchy, we get the ability to scale according to our current needs.

'''

[Monitoring systems pull the data from various servers and then push the data to the data

center monitoring system](./monitor1.jpg)

[The data center monitoring systems push the data to the global monitoring system](./monitor2.jpg)

'''

Question 1

What happens if a local or global monitoring system is down?

Answer

We can store the data locally and wait for the system to be up and running again. But there's a limit for the local data storage. So, either we delete previous data or we don't store new data. To make a decision, relevant policies need to be created.

'''

'''

Question 2

How can a monitoring system reliably work if it uses the same infrastructure in a data center that it was supposed to monitor? Consider this given that a failure of a network in a data center can knock out the monitoring components.

Answer

The actual deployment of a monitoring system needs special care. We might have an internal, monitoring-specific network to isolate it from the common network. We should use a separate instance of blob stores and other services.

'''

It also helps to have external components to the monitoring, where external might mean an independent service provider's infrastructure. However, designing such a system is complex and is more expensive.

Humans need to consume enormous amounts of data, and even after different kinds of data summarizations, the data can still be huge. Next, we'll tackle how to present enormous data to human administrators.

Monitor Server-side Errors/Visualize Data in a Monitoring System/README.md

Visualize Data in a Monitoring System

Large data centers have millions of servers, and visualizing the health data for all of them is challenging. An important aspect of monitoring a fleet of servers is to know which ones are alive and which ones are offline. A modern data center can house many thousands of servers in a building. We can use a heat map to display information about thousands of servers compactly in a data center.

A heat map is a data visualization technique that shows the magnitude of a phenomenon in two dimensions by using colors.

Using heat maps to troubleshoot

We'll identify if a server is down by using heat maps. Each rack of servers is named and is sorted by data center, then cluster, then row, so problems common at any of these levels are readily apparent.

A heat map depicting the operational state of a large number of components is an effective method. The health of each component is indicated by the color of each cell in a big matrix. Nodes with green cells operate within permitted parameters, while nodes with red cells are nonresponsive on multiple tries.

Below, we have a heat map displaying the server's state.

[Viewing servers in a data center using heat maps](./cluster.jpg)

We can use heat maps for the globally distributed systems and continuously share the health information of a server. We can use one bit (one for live, zero for dead). For 1,000,000 servers, we have 125 KB of data. We can quickly find out which server is down by the red color and focus on the problematic parts.

We can create similar heat maps to get a bird's-eye view of any resource, like filesystems, networking switches, links, and so on.

Summary

- Monitoring systems are critical in distributed systems because they help in analyzing the system and alerting the stakeholders if a problem occurs.

- We can make a monitoring system scalable using a hybrid of the push and pull methods.

- Heat maps are a powerful tool for visualization and help us learn about the health of thousands of servers in a compact space.

Non-functional System Characteristics/Availability/README.md

Availability

What is availability?

Measuring availability

Mathematically, availability, A, is a ratio. The higher the A value, the better. We can also write this up as a formula:

$$A \text{ (in percent)} = (\text{total time} - \text{amount of time service was down}) / \text{total time} * 100$$

[the nines](./nine_availability.jpg)

We measure availability as a number of nines. The following table shows how much downtime is permitted when we're using a given number of nines.

Availability and service providers

Each service provider may start measuring availability at different points in time. Some cloud providers start measuring it when they first offer the service, while some measure it for specific clients when they start using the service. Some providers might not reduce their reported availability numbers if their service was not down for all the clients. The planned downtimes are excluded. Downtime due to cyberattacks might not be incorporated into the calculation of availability. Therefore, we should carefully understand how a specific provider calculates their availability numbers.

Non-functional System Characteristics/Fault Tolerance/README.md

Fault Tolerance

What is fault tolerance?

Real-world, large-scale applications run hundreds of servers and databases to accommodate billions of users' requests and store significant data. These applications need a mechanism that helps with data safety and eschews the recalculation of computationally intensive tasks by avoiding a single point of failure.

Fault tolerance refers to a system's ability to execute persistently even if one or more of components fail. Here, components can be software or hardware. Conceiving a system that is hundred percent fault-tolerant is practically very difficult.

Let's discuss some important features for which fault-tolerance becomes a necessity.

Availability focuses on receiving every client's request by being accessible 24/7.

Reliability is concerned with responding by taking specified action on every client's request.

Fault tolerance techniques

Failure occurs at the hardware or software level, which eventually affects the data. Fault tolerance can be achieved by many approaches, considering the system structure. Let's discuss the techniques that are significant and suitable for most designs.

Replication

One of the most widely-used techniques is replication-based fault tolerance. With this technique, we can replicate both the services and data. We can swap out failed nodes with healthy ones and a failed data store with its replica. A large service can transparently make the switch without impacting the end customers.

We create multiple copies of our data in separate storage. All copies need to update regularly for consistency when any update occurs in the data. Updating data in replicas is a challenging job. When a system needs strong consistency, we can synchronously update data in replicas. However, this reduces the availability of the system. We can also asynchronously update data in replicas when we can tolerate eventual consistency, resulting in stale reads until all replicas converge. Thus, there is a trade-off between both consistency approaches. We compromise either on availability or on consistency under failures—a reality that is outlined in the CAP theorem.

Checkpointing

Checkpointing is a technique that saves the system's state in stable storage when the system state is consistent. Checkpointing is performed in many stages at different time intervals. The primary purpose is to save the computational state at a given point. When a failure occurs in the system, we can get the last computed data from the previous checkpoint and start working from there.

Checkpointing also comes with the same problem that we have in replication. When the system has to perform checkpointing, it makes sure that the system is in a consistent state, meaning that all processes are stopped. This type of checkpointing is known as synchronous checkpointing. On the other hand, checkpointing in an inconsistent state leads to data inconsistency problems. Let's look at the illustration below to understand the difference between a consistent and an inconsistent state:

Consistent state: The illustration above shows no communication among the processes when the system performs checkpointing. All the processes are sending or receiving messages before and after checkpointing. This state of the system is called a consistent state.

Inconsistent state: The illustration also displays that processes communicate through messages when the system performs checkpointing. This indicates an inconsistent state, because when we get a previously saved checkpoint, Process i will have a message (m₁) and Process j will have no record of message sending.

Non-functional System Characteristics/Maintainability/README.md

Maintainability

What is maintainability?

Besides building a system, one of the main tasks afterward is keeping the system up and running by finding and fixing bugs, adding new functionalities, keeping the system's platform updated, and ensuring smooth system operations. One of the salient features to define such requirements of an exemplary system design is maintainability. We can further divide the concept of maintainability into three underlying aspects:

Operability: This is the ease with which we can ensure the system's smooth operational running under normal circumstances and achieve normal conditions under a fault.

Lucidity: This refers to the simplicity of the code. The simpler the code base, the easier it is to understand and maintain it, and vice versa.

Modifiability: This is the capability of the system to integrate modified, new, and unforeseen features without any hassle.

Measuring maintainability

Maintainability, M, is the probability that the service will restore its functions within a specified time of fault occurrence. M measures how conveniently and swiftly the service regain its normal operating conditions.

For example, suppose a component has a defined maintainability value of 95% for half an hour. In that case, the probability of restoring the component to its fully active form in half an hour is 0.95.

'''

Note: Maintainability gives us insight into the system's capability to undergo repairs and modifications while it's operational.

'''

We use (mean time to repair) MTTR as the metric to measure M.

$$MTTR = \text{Total Maintenance Time} / \text{Total Number of Repairs}$$

In other words, MTTR is the average amount of time required to repair and restore a failed component. Our goal is to have as low a value of MTTR as possible.

Maintainability and reliability

Maintainability can be defined more clearly in close relation to reliability. The only difference between them is the variable of interest. Maintainability refers to time-to-repair, whereas reliability refers to both time-to-repair and the time-to-failure. Combining maintainability and reliability analysis can help us achieve availability, downtime, and uptime insights.

Non-functional System Characteristics/Reliability/README.md

Reliability

What is reliability?

Reliability, R, is the probability that the service will perform its functions for a specified time. R measures how the service performs under varying operating conditions. We often use mean time between failures (MTBF) and mean time to repair (MTTR) as metrics to measure R.

$$MTBF = \text{total elapsed time} - \text{sum of down time} / \text{total number of failures}$$

$$MTTR = \text{total maintenance time} / \text{total number of repairs}$$

(We strive for a higher MTBF value and a lower MTTR value.)

Reliability and availability

Reliability and availability are two important metrics to measure compliance of service to agreed-upon service level objectives (SLO).

The measurement of availability is driven by time loss, whereas the frequency and impact of failures drive the measure of reliability. Availability and reliability are essential because they enable the stakeholders to assess the health of the service.

Reliability (R) and availability (A) are two distinct concepts, but they are related.

Mathematically, A is a function of R. This means that the value of R can change independently, and the value of A depends on R. Therefore, it's possible to have situations where we have:

low A, low R
low A, high R
high A, low R
high A, high R (desirable)

Non-functional System Characteristics/Scalability/README.md

Scalability

What is scalability?

Scalability is the ability of a system to handle an increasing amount of workload without compromising performance. A search engine, for example, must accommodate increasing numbers of users, as well as the amount of data it indexes.

The workload can be of different types, including the following:

• Request workload: This is the number of requests served by the system.
• Data/storage workload: This is the amount of data stored by the system.

Dimensions

Here are the different dimensions of scalability:

• Size scalability: A system is scalable in size if we can simply add additional users and resources to it.

• Administrative scalability: This is the capacity for a growing number of organizations or users to share a single distributed system with ease.

• Geographical scalability: This relates to how easily the program can cater to other regions while maintaining acceptable performance constraints. In other words, the system can readily service a broad geographical region, as well as a smaller one.

Different approaches of scalability

Vertical scalability • scaling up

Vertical scaling, also known as scaling up, refers to scaling by providing additional capabilities (for example, additional CPUs or RAM) to an existing device. Vertical scaling allows us to expand our present hardware or software capacity, but we can only grow it to the limitations of our server. The dollar cost of vertical scaling is usually high because we might need exotic components to scale up.

Horizontal scalability • scaling out

Horizontal scaling, also known as scaling out, refers to increasing the number of machines in the network. We use commodity nodes for this purpose because of their attractive dollar-cost benefits. The catch here is that we need to build a system such that many nodes could collectively work as if we had a single, huge server.

Pub-sub/Design of a Pub-sub System/README.md

Design of a Pub-sub System

First design

In the previous lesson, we discussed that a producer writes into topics, and consumers subscribe to a topic to read messages from that topic. Since new messages are added at the end of the queue, we can use distributed messaging queues for topics.

The components we'll need have been listed below:

- Topic queue: Each topic will be a distributed messaging queue so we can store the messages sent to us from the producer. A producer will write their messages to that queue.

- Database: Weâ ll use a relational database that will store the subscription details. For example, we need to store which consumer has subscribed to which topic so we can provide the consumers with their desired messages. Weâ ll use a relational database since our consumer-related data is structured and we want to ensure our data integrity.
- Message director: This service will read the message from the topic queue, fetch the consumers from the database, and send the message to the consumer queue.
- Consumer queue: The message from the topic queue will be copied to the consumerâ s queue so the consumer can read the message. For each consumer, weâ ll define a separate distributed queue.
- Subscriber: When the consumer requests a subscription to a topic, this service will add an entry into the database.

The consumer will subscribe to a topic, and the system will add the subscriberâ s details to database. The producer will write into the topics, and the message director will read the message from the queue, fetch the details to whom it should add the message, and send it to them. The consumers will consume the message from their queue.

'''

Note: Weâ ll use fail-over services for the message director and subscriber to guard against failures.

'''

[Using the distributed messaging queue](./using.jpg)

Using the distributed messaging queues makes our design simple. However, the huge number of queues needed is a significant concern. If we have millions of subscribers for thousands of topics, defining and maintaining millions of queues is expensive. Moreover, weâ ll copy the message for a topic in all subscriber queues, which is unnecessary duplication and takes up space.

'''

Question 1

Is there a way to avoid maintaining a separate queue for each reader?

Answer

In messaging queues, the message disappears after the reader consumes it. So, what if we add a counter for each message? The counter value decrements as a subscriber consumes the message. It does not delete the message until the counter becomes zero. Now, we donâ t need to keep a separate queue for each reader.

'''

'''

Question 2

What is the problem with the previous approach?

Answer

The unread messages can become a bottleneck if we use the conventional queue API. For example, if 9 out of 10 readers have consumed the message present at the start of the queue, then that message wonâ t be deleted until the tenth consumer has also consumed the message, and the first nine consumers wonâ t be able to move forward.

Weâ ll need to change the storage interface so that consumers can independently consume data. Our system will need to keep sufficient metadata and track what information each consumer has consumed and delete a message when the information has been consumed by all consumers. It resembles with reference count mechanism in Linuxâ s hard link of files.

'''

Second design

Let's consider another approach to designing a pub-sub system.

High-level design

At a high level, the pub-sub system will have the following components:

- Broker: This server will handle the messages. It will store the messages sent from the producer and allow the consumers to read them.
- Cluster manager: We'll have numerous broker servers to cater to our scalability needs. We need a cluster manager to supervise the broker's health. It will notify us if a broker fails.
- Storage: We'll use a relational database to store consumer details, such as subscription information and retention period.
- Consumer manager: This is responsible for managing the consumers. For example, it will verify if the consumer is authorized to read a message from a certain topic or not.

Besides these components, we also have the following design considerations:

- Acknowledgment: An acknowledgment is used to notify the producer that the received message has been stored successfully. The system will wait for an acknowledgment from the consumer if it has successfully consumed the message.
- Retention time: The consumers can specify the retention period time of their messages. The default will be seven days, but it is configurable. Some applications like banking applications require the data to be stored for a few weeks as a business requirement, while an analytical application might not need the data after consumption.

[High-level design of a pub-sub system](./high_level_design.jpg)

Let's understand the role of each component in detail.

Broker

The broker server is the core component of our pub-sub system. It will handle write and read requests. A broker will have multiple topics where each topic can have multiple partitions associated with it. We use partitions to store messages in the local storage for persistence. Consequently, this improves availability. Partitions contain messages encapsulated in segments. Segments help identify the start and end of a message using an offset address. Using segments, consumers consume the message of their choice from a partition by reading from a specific offset address. The illustration below depicts the concept that has been described above.

[A depiction of how messages are stored within segments inside a partition](./how_messages_are_stored.jpg)

As we know, a topic is a persistent sequence of messages stored in the local storage of the broker. Once the data has been added to the topic, it cannot be modified. Reading and writing message from or to a topic is an I/O task for computers, and scaling such tasks is challenging. This is the reason we split the topics into multiple partitions. The data belonging to a single topic can be present in numerous partitions. For example, let's assume we have Topic A and we allocate three partitions for it. The producers will send their messages to the relevant topic. The messages received will be sent to various partitions on basis of the round-robin algorithm. We'll use a variation of round-robin: weighted round-robin. The following slides show how the messages are stored in various partitions belonging to a single topic.

[Broker](./how_broker_works)

...

Question 1

Strict ordering ensures that the messages are stored in the order in which they are produced.

How can we ensure strict ordering for our messages?

Answer

We'll assign each partition a unique ID, `partition_ID`. The user can provide the `partition_ID` while writing into the system. In this way, the messages will be sent to the specified partition, and the ordering will be strict. Our API call to write into the pub-sub system looks like this:

```
write(topic_ID, partition_ID, message)
```

If the user does not provide the `partition_ID`, we'll use the weighted round-robin algorithm to decide which message has to be sent to which partition.

It might seem strange to give the ability to choose a partition to the client of pub-sub. However, such a facility can be the basis from where clients can get data for some specific time period—for example, getting data from yesterday. For simplicity, we won't include time-based reading in our design.

```
'''
```

```
'''
```

Question 2

What problems can arise if all partitions are on the same broker?

Answer

If the broker fails or dies, all the messages in the partitions will be lost. To avoid this, we need to make sure that the partitions are spread on different brokers.

```
'''
```

```
'''
```

Question 3

Why can't we use blob stores like S3 to keep messages, instead of the broker's local storage?

Answer

Blob stores like S3 are not optimized for writing and reading short-sized data. If our data is geo-replicated, the above problem is exacerbated.

Therefore, we used the server's local persistent store with append-based writing. Traditional hard disks are specially tuned to provide good write performance with writing to contiguous tracks or sectors. Reading throughput and latency is also good for contiguous regions of the disk because it allows extensive data caching.

```
'''
```

```
'''
```

Question 4

If we use a round-robin algorithm to send messages to a partition, how does the system know where to look when it is time to read?

Answer

Our system will need to keep appropriate metadata persistently. This metadata will keep mappings between the logical index of segment or messages to the server identity or partition identifier.

We'll discuss consumer manager later in the lesson, which will keep the required information.

```
'''
```

```
[Multiple topics](./multiple_topics)
```

We discussed that a message will be stored in a segment. We'll identify each segment using an offset. Since these are immutable records, the readers are independent and they can read messages anywhere from this file using the necessary API functions. The following slides show

the segment level detail.

[A new entry will be added at the end of the file](./1.jpg)

[The consumers can read anywhere from the file. Producers add to the end of file](./2.jpg)

The broker solved the problems that our first design had. We avoided a large number of queues by partitioning the topic. We introduced parallelism using partitions that avoided bottlenecks while consuming the message.

Cluster manager

Weâ ll have multiple brokers in our cluster. The cluster manager will perform the following tasks:

- Broker and topics registry: This stores the list of topics for each broker.
- Manage replication: The cluster manager manages replication by using the leader-follower approach. One of the brokers is the leader. If it fails, the manager decides who the next leader is. In case the follower fails, it adds a new broker and makes sure to turn it into an updated follower. It updates the metadata accordingly. Weâ ll keep three replicas of each partition on different brokers.

[Replication at the partitioning level](./replication.jpg)

Consumer manager

The consumer manager will manage the consumers. It has the following responsibilities:

- Verify the consumer: The manager will fetch the data from the database and verify if the consumer is allowed to read a certain message. For example, if the consumer has subscribed to Topic A (but not to Topic B), then it should not be allowed to read from Topic B. The consumer manager verifies the consumerâ s request.
- Retention time management: The manager will also verify if the consumer is allowed to read the specific message or not. If, according to its retention time, the message should be inaccessible to the consumer, then it will not allow the consumer to read the message.
- Message receiving options management: There are two methods for consumers to get data. The first is that our system pushes the data to its consumers. This method may result in overloading the consumers with continuous messages. Another approach is for consumers to request the system to read data from a specific topic. The drawback is that a few consumers might want to know about a message as soon as it is published, but we do not support this function.

Therefore, weâ ll support both techniques. Each consumer will inform the broker that it wants the data to be pushed automatically or it needs the data to read itself. We can avoid overloading the consumer and also provide liberty to the consumer. Weâ ll save this information in the relational database along with other consumer details.

- Allow multiple reads: The consumer manager stores the offset information of each consumer. Weâ ll use a key-value to store offset information against each consumer. It allows fast fetching and increases the availability of the consumers. If Consumer 1 has read from offset 0 and has sent the acknowledgment, weâ ll store it. So, when the consumer wants to read again, can provide the next offset to the reader for reading the message.

Finalized design

The finalized design of our pub-sub system is shown below.

[Finalized design](./final.jpg)

Conclusion

We saw two designs of pub-sub, using queues and using our custom storage optimized for writing

and reading small-sized data.

There are numerous use cases of a pub-sub. Due to decoupling between producers and consumers, the system can scale dynamically, and the failures are well-contained. Additionally, due to proper accounting of data consumption, the pub-sub is a system of choice for a large-scale system that produces enormous data. We can determine precisely which data is needed and not needed.

Pub-sub/Introduction to Pub-sub/README.md

Introduction to Pub-sub

Pub-sub messaging offers asynchronous communication. Let's explore the use cases where it is beneficial to have a pub-sub system.

Use cases of pub-sub

A few use cases of pub-sub are listed below:

- Improved performance: The pub-sub system enables push-based distribution, alleviating the need for message recipients to check for new information and changes regularly. It encourages faster response times and lowers the delivery latency.

- Handling ingestion: The pub-sub helps in handling log ingestion. The user-interaction data can help us figure out useful analyses about the behavior of users. We can ingest a large amount of data to the pub-sub system, so much so that it can deliver the data to any analytical system to understand the behavior patterns of users. Moreover, we can also log the details of the event that's happening while completing a request from the user. Large services like Met use a pub-sub system called Scribe to know exactly who needs what data, and remove or archive processed or unwanted data. Doing this is necessary to manage an enormous amount of data.

- Real-time monitoring: Raw or processed messages of an application or system can be provided to multiple applications to monitor a system in real time.

- Replicating data: The pub-sub system can be used to distribute changes. For example, in a leader-follower protocol, the leader sends the changes to its followers via a pub-sub system. It allows followers to update their data asynchronously. The distributed caches can also refresh themselves by receiving the modifications asynchronously. Along the same lines, applications like WhatsApp that allow multiple views of the same conversation—for example, on a mobile phone and a computer—can elegantly work using a pub-sub, where multiple views can act either as a publisher or a subscriber.

'''

Log ingestion refers to importing logs into a database for immediate use, such as monitoring. Huge applications like Facebook produce massive amounts of data and logs. Real-time fetching and processing of logs allow us to run analysis and take quick actions in case of failures.

'''

[Motivation and use cases of the pub-sub system](./real-time.jpg)

'''

Question 1

What are the similarities and differences between a pub-sub system and queues?

Answer

The pub-sub system and queues are similar because they deliver information that's produced by the producer to the consumer. The difference is that only one consumer consumes a message in the queue, while there can be multiple consumers of the same message in a pub-sub system.

'''

'''

Question 2

How are producers and consumers decoupled from one another in a pub-sub system?

Answer

Producers don't know who'll end up reading their information. They just send it to the system and it is read by the consumer. Producers are not affected by slow consumers, the count of consumers, or the failure of consumers. We can scale them independently.

'''

Requirements

We aim to design a pub-sub system that has the following requirements.

Functional requirements

Let's specify the functional requirements of a pub-sub system:

- Create a topic: The producer should be able to create a topic.
- Write messages: Producers should be able to write messages to the topic.
- Subscription: Consumers should be able to subscribe to the topic to receive messages.
- Read messages: The consumer should be able to read messages from the topic.
- Specify retention time: The consumers should be able to specify the retention time after which the message should be deleted from the system.

'''

A topic is a collection of related events or messages. The producers can create a topic and write the message into a topic. Consumers can subscribe to a topic and read from that topic.

'''

- Delete messages: A message should be deleted from the topic or system after a certain retention period as defined by the user of the system.

Non-functional requirements

We consider the following non-functional requirements when designing a pub-sub system:

- Scalable: The system should scale with an increasing number of topics and increasing writing (by producers) and reading (by consumers) load.
- Available: The system should be highly available, so that producers can add their data and consumers can read data from it anytime.
- Durability: The system should be durable. Messages accepted from producers must not be lost and should be delivered to the intended subscribers.
- Fault tolerance: Our system should be able to operate in the event of failures.
- Concurrent: The system should handle concurrency issues where reading and writing are performed simultaneously.

API Design

We'll exclude some parameters from the functions below, such as the producer or consumer's identifier. Let's assume that this information is available from the underlying connection context. The API design for this problem is as follows:

****Create a topic****

The API call to create a topic should look like this:

'''

create(topic_ID, topic_name)

'''

This function returns an acknowledgment if it successfully creates a topic, or an error if it fails to do so.

'''

Parameter	Description
topic_ID	It uniquely identifies the topic.

topic_name	It contains the name of the topic.
------------	------------------------------------

'''

****Read a message****

The API call to read data from the system should look like this:

'''

read(topic_ID)

'''

The topic is found using topic_ID, and the call will return an object containing the message to the caller.

'''

Parameter	Description
-----------	-------------

topic_ID	It is the ID of the topic against which the message will be read.
----------	---

'''

****Subscribe to a topic****

The API call to subscribe to a topic from the system should look like this:

'''

subscribe(topic_ID)

'''

The function adds the consumer as a subscriber to the topic that has the topic_ID.

'''

Parameter	Description
-----------	-------------

topic_ID	The ID of the topic to which the consumer will be subscribed.
----------	---

'''

****Unsubscribe from a topic****

The API call to unsubscribe from a topic from the system should look like this:

'''

unsubscribe(topic_ID)

'''

The function removes the consumer as a subscriber from the topic that has the topic_ID.

'''

Parameter	Description
-----------	-------------

topic_ID	The ID of the topic against which the consumers will be unsubscribed.
----------	---

'''

****Delete a topic****

The API call to delete a topic from the system should look like this:

'''

delete_topic(topic_ID)

'''

The function deletes the topic on the basis of the topic_ID.

'''

Parameter	Description
-----------	-------------

topic_ID	The ID of the topic which is to be deleted.
----------	---

```
'''
```

```
## Building blocks we will use
```

The design of pub-sub utilizes many building blocks that have been discussed in the initial chapters. Weâ ll consider the following lessons on building blocks.

[The building blocks weâ ll use](./bb.jpg)

- Database: Weâ ll use databases to store information like subscription details.
 - Distributed messaging queue: Weâ ll use use a messaging queue to store messages sent by the producer.
 - Key-value: Weâ ll use a key-value store to hold information about consumers.
- In the next lesson, weâ ll focus on designing a pub-sub system.

```
## Pub-sub/System Design The Pub-sub Abstraction/README.md
```

```
# System Design: The Pub-sub Abstraction
```

```
## What is a pub-sub system?
```

Publish-subscribe messaging, often known as pub-sub messaging, is an asynchronous service-to-service communication method thatâ s popular in serverless and microservices architectures. Messages can be sent asynchronously to different subsystems of a system using the pub-sub system. All the services subscribed to the pub-sub model receive the message thatâ s pushed in the system.

For example, when Cristiano Ronaldo, a famous athlete, posts on Instagram or shares a tweet, all of his followers are updated. Here, Cristiano Ronaldo is the publisher, his post or tweet is the message, and all of his followers are subscribers.

[The pub-sub system](./Pub-sub-sys.jpg)

```
## Motivation
```

The hardware infrastructure of distributed systems consists of millions of machines. Using a pub-sub system to communicate asynchronously increases scalability. Producers and consumers are disconnected and operate independently, thereby allowing us to scale and develop them separately. The decoupling between components, producers and consumers, allows greater scalability because adding or removing any component doesnâ t affect the other components.

```
## How do we design a pub-sub system?
```

We have divided the pub-sub system design into the following lessons:

1. Introduction: In this lesson, we learn about the use cases of the pub-sub system, define its requirements, and design the API for it.
2. Design: In this lesson, we discuss two designs of the pub-sub system, one with messaging queues and the other with a broker.

```
## README.md
```

```
# Grokking-Modern-System-Design-Interview
```

Content here is extracted from the same-named course by Educative.io for previews only. To achieve better study efficiency and study complete course contents, please buy membership of Educative.io.

[the course](./system_design_course.jpg)

```
## Rate Limiter/Design of a Rate Limiter/README.md
```

Design of a Rate Limiter

High-level design

A rate limiter can be deployed as a separate service that will interact with a web server, as shown in the figure below. When a request is received, the rate limiter suggests whether the request should be forwarded to the server or not. The rate limiter consists of rules that should be followed by each incoming request. These rules define the throttling limit for each operation. Let's go through a rate limiter rule from Lyft, which has open-sourced its rate limiting component.

```
'''yaml
domain: messaging
descriptors:
  -key: message_type
    value: marketing
    rate_limit:
      unit: day
      request_per_unit: 5
Rate-limiting rules from Lyft
'''
```

[High-level design](./design)

In the above rate-limiting rule, the unit is set to day and the request_per_unit is set to 5. These parameters define that the system can allow five marketing messages per day.

Detailed design

The high-level design given above does not answer the following questions:

Where are the rules stored?

How do we handle requests that are rate limited?

In this section, we'll first expand the high-level architecture into several other essential components. We'll also explain each component in detail, as shown in the following figure.

[The rate limiter accepts or rejects requests based on throttle rules](./detailed_design.jpg)

Let's discuss each component that is present in the detailed design of a rate limiter.

Rule database: This is the database, consisting of rules defined by the service owner. Each rule specifies the number of requests allowed for a particular client per unit of time.

Rules retriever: This is a background process that periodically checks for any modifications to the rules in the database. The rule cache is updated if there are any modifications made to the existing rules.

Throttle rules cache: The cache consists of rules retrieved from the rule database. The cache serves a rate-limiter request faster than persistent storage. As a result, it increases the performance of the system. So, when the rate limiter receives a request against an ID (key), it checks the ID against the rules in the cache.

Decision-maker: This component is responsible for making decisions against the rules in the cache. This component works based on one of the rate-limiting algorithms that are discussed in the next lesson.

Client identifier builder: This component generates a unique ID for a request received from a client. This could be a remote IP address, login ID, or a combination of several other attributes, due to which a sequencer can't be used here. This ID is considered as a key to store the user data in the key-value database. So, this key is passed to the decision-maker for further service decisions.

In case the predefined limit is crossed, APIs return an HTTP response code 429 Too Many Requests, and one of the following strategies is applied to the request:

Drop the request and return a specific response to the client, such as `429 too many requests` or `503 service unavailable`.

If some requests are rate limited due to a system overload, we can keep those requests in a queue to be processed later.

Request processing

When a request is received, the client identifier builder identifies the request and forwards it to the decision-maker. The decision-maker determines the services required by request, then checks the cache against the number of requests allowed, as well as the rules provided by the service owner. If the request does not exceed the count limit, it is forwarded to the request processor, which is responsible for serving the request.

The decision-maker takes decisions based on the throttling algorithms. The throttling can be hard, soft, or elastic. Based on soft or elastic throttling, requests are allowed more than the defined limit. These requests are either served or kept in the queue and served later, upon the availability of resources. Similarly, if hard throttling is used, requests are rejected, and a response error is sent back to the client.

'''

Question

In the event of a failure, a rate limiter is unable to perform the task of throttling. In these scenarios, should the request be accepted or rejected?

Answer

In such a scenario, the proposed system should adhere to the non-functional requirements, including availability and fault tolerance. So, in case of failure, the rate limiter will be unable to perform. However, the default decision would be not to throttle any request. The reason for this is that we would have many rate limiters at a different service level. Even if there is no other rate limiter, the load balancer performs this task at a certain level, as explained earlier.

'''

Race condition

There is a possibility of a race condition in a situation of high concurrency request patterns. It happens when the `get-then-set` approach is followed, wherein the current counter is retrieved, incremented, and then pushed back to the database. While following this approach, some additional requests can come through that could leave the incremented counter invalid. This allows a client to send a very high rate of requests, bypassing the rate-limiting controls. To avoid this problem, the locking mechanism can be used, where one process can update the counter at a time while others wait for the lock to be released. Since this approach can cause a potential bottleneck, it significantly degrades performance and does not scale well.

Another method that could be used is the `set-then-get` approach, wherein a value is incremented in a very performant fashion, avoiding the locking approach. This approach works in the absence of minimum contention. However, one might use other approaches where the allowed quota is divided into multiple places and divide the load on them, or use sharded counters to scale an approach.

'''

Note: We can use sharded counters for rate-limiting under the highly concurrent nature of traffic. By increasing the number of shards, we reduce write contention. Since we have to collect counters from all shards, our reading may slow down.

'''

A rate limiter should not be on the client's critical path

Let's assume a real-time scenario where millions of requests hit the front-end servers. Each request will retrieve, update, and push back the count to the respective cache. After all these operations, the request is sent forward to be served. This approach could cause latency if

there is a high number of requests. To avoid numerous computations in the client's critical path, we should divide the work into offline and online parts.

Initially, when a client's request is received, the system will just check the respective count. If it is less than the maximum limit, the system will allow the client's request. In second phase, the system updates the respective count and cache offline. For a few requests, this won't have any effect on the performance, but for millions of requests, this approach increases performance significantly.

Let's understand the online and offline updates approach with an example. In the following set of illustrations, when a request is received, its ID is forwarded to the rate limiter that will check the condition `if(request(ID).Count <= Max. Limit)` by retrieving data from the cache. For simplicity, assume that one of the requests ID is 101, that is, `request(ID) = 101`. The following table shows the number of requests made by each client and the maximum number of requests per unit time that a client can make.

'''

Request ID	Maximum Limit	Count
100	5	4
101	5	3

'''

If the condition is true, the rate limiter will first respond back to the front-end server with an Allowed signal. The corresponding count and other relevant information are updated offline in the next steps. The rate limiter writes back the updated data in the cache. Following this approach reduces latency and avoids the contention that incoming requests could have caused.

[Rate limiter](./critical_path)

'''

Note: We've seen a form of rate limiting in TCP network protocol, where the recipient can throttle the sender by advertising the size of the window (the outstanding data a recipient is willing to receive). The sender sends the minimum value of either the congestion window or the advertised window. Many network traffic shapers use similar mechanisms to provide preferential treatment to different network flows.

'''

Conclusion

In this lesson, we discussed the design of a rate limiter in distributed systems. Let's analyze the non-functional requirements we promised in the previous lesson.

- Availability: If a rate limiter fails, multiple rate limiters will be available to handle the incoming requests. So, a single point of failure is eliminated.
- Low latency: Our system retrieves and updates the data of each incoming request from the cache instead of the database. First, the incoming requests are forwarded if they do not exceed the rate limit, and then the cache and database are updated.
- Scalability: The number of rate limiters can be increased or decreased based on the number of incoming requests within the defined limit.

Now, our system provides high availability, low latency, and scalability in light of the above discussion.

Rate Limiter/Quiz on the Rate Limiter's Design/README.md

Rate Limiter Algorithms

Algorithms for rate limiting

The task of a rate limiter is directed by highly efficient algorithms, each of which has distinct advantages and disadvantages. However, there is always a choice to choose an algorithm or combination of algorithms depending on what we need at a given time. While different algorithms are used in addition to those below, we'll take a look at the following popular algorithms.

- Token bucket
- Leaking bucket
- Fixed window counter
- Sliding window log
- Sliding window counter

Token bucket algorithm

This algorithm uses the analogy of a bucket with a predefined capacity of tokens. The bucket is periodically filled with tokens at a constant rate. A token can be considered as a packet of some specific size. Hence, the algorithm checks for the token in the bucket each time we receive a request. There should be at least one token to process the request further.

The flow of the token bucket algorithm is as follows:

Assume that we have a predefined rate limit of R and the total capacity of the bucket is C

1. The algorithm adds a new token to the bucket after every $1/R$ seconds.
2. The algorithm discards the new incoming tokens when the number of tokens in the bucket is equal to the total capacity C of the bucket.
3. If there are N incoming requests and the bucket has at least N tokens, the tokens are consumed, and requests are forwarded for further processing.
4. If there are N incoming requests and the bucket has a lower number of tokens, then the number of requests accepted equals the number of available tokens in the bucket.

The following illustration represents the working of the token bucket algorithm.

[How the token bucket algorithm works](./token_bucket.jpg)

The following illustration demonstrates how token consumption and rate-limiting logic work. In this example, the capacity of the bucket is three, and it is refilled at a rate of three tokens per minute.

[Token bucket](./token_bucket)

Essential parameters

We require the following essential parameters to implement the token bucket algorithm:

- Bucket capacity(C): The maximum number of tokens that can reside in the bucket.
- Rate limit (R): The number of requests we want to limit per unit time.
- Refill rate ($1/R$): The number of tokens put into the bucket per unit time.
- Requests count (N): This parameter tracks the number of incoming requests and compares them with the bucket's capacity.

Advantages

- This algorithm can cause a burst of traffic as long as there are enough tokens in the bucket.
- It is space efficient. The memory needed for the algorithm is nominal due to limited states.

Disadvantages

- From the implementation perspective, a lock might require taking a token from the bucket that can increase the request's processing delay if contention on the lock increases.
- Choosing an optimal value for the essential parameters is a difficult task.

The leaking bucket algorithm

The leaking bucket algorithm is a variant of the token bucket algorithm with slight modifications. Instead of using tokens, the leaking bucket algorithm uses a bucket to contain incoming requests and processes them at a constant outgoing rate. This algorithm uses the analogy of a water bucket leaking at a constant rate. Similarly, in this algorithm, the requests arrive at a variable rate. The algorithm process these requests at a constant rate in a first-in-first-out (FIFO) order.

Let's look at how the leaking bucket algorithm works in the illustration below:

Essential parameters

The leaking bucket algorithm requires the following parameters.

- Bucket capacity (C): This determines the maximum capacity of the bucket. The algorithm will discard the incoming requests when the bucket reached its maximum limit of C.
- Inflow rate (R_{in}): This parameter shows the inflow rate of requests. This is a varying quantity that depends on the application and nature of requests. We use this parameter to find the initial capacity of the bucket.
- Outflow rate (R_{out}): This determines the number of requests processed per unit time.

Advantages

- Due to a constant outflow rate (R_{out}), it avoids the burst of requests, unlike the token bucket algorithm.
- This algorithm is also space efficient since it requires just three states: inflow rate (R_{in}), outflow rate (R_{out}), and bucket capacity (C).
- Since requests are processed at a fixed rate, it is suitable for applications with a stable outflow rate.

Disadvantages

- A burst of requests can fill the bucket, and if not processed in the specified time, recent requests can take a hit.
- Determining an optimal bucket size and outflow rate is a challenge.

Fixed window counter algorithm

This algorithm divides the time into fixed intervals called windows and assigns a counter to each window. When a specific window receives a request, the counter is incremented by one. Once the counter reaches its limit, new requests are discarded in that window.

As shown in the below figure, a dotted line represents the limit in each window. If the counter is lower than the limit, forward the request; otherwise, discard the request.

[Fixed window counter algorithm: Discard the request exceeding the limit](./fixed_window.jpg)

There is a significant problem with this algorithm. A burst of traffic greater than the allowed requests can occur at the edges of the window. In the below figure, the system allows a maximum of ten requests per minute. However, the number of requests in the one-minute window from 01:30 to 02:30 is 20, which is greater than the allowed number of requests.

[Edge case problem in the fixed window counter algorithm. The number of requests in one minute from 01:30 to 02:30 exceeds the predefined limit of 10 requests per minute](./edge_cases.jpg)

Essential parameters

The fixed window counter algorithm requires the following parameters:

Window size (W): It represents the size of the time window. It can be a minute, an hour, or any other suitable time slice.

Rate limit (R): It shows the number of requests allowed per time window.

Requests count (N): This parameter shows the number of incoming requests per window. The incoming requests are allowed if N is less than or equal to R.

Advantages

- It is also space efficient due to constraints on the rate of requests.
- As compared to token bucket-style algorithms (that discard the new requests if there aren't

enough tokens), this algorithm services the new requests.

Disadvantages

- A consistent burst of traffic (twice the number of allowed requests per window) at the window edges could cause a potential decrease in performance.

Sliding window log algorithm

The sliding window log algorithm keeps track of each incoming request. When a request arrives, its arrival time is stored in a hash map, usually known as the log. The logs are sorted based on the time stamps of incoming requests. The requests are allowed depending on the size of the log and arrival time.

The main advantage of this algorithm is that it doesn't suffer from the edge conditions, as compared to the fixed window counter algorithm.

Let's understand how the sliding window log algorithm works in the illustration below. Assume that we have a maximum rate limit of two requests in a minute.

[Sliding window log algorithm](./sliding_window)

Essential parameters

The following parameters are required to implement the sliding window log algorithm:

- Log size (L): This parameter is similar to the rate limit (R) as it determines the number of requests allowed in a specific time frame.
- Arrival time (T): This parameter tracks incoming requests' time stamps and determines their count.
- Time range ($T_{\{r\}}$): This parameter determines the time frame. The time stamps of the old requests are deleted if they do not fall in this range. The start time of the window is defined based on the first incoming request and expires after one minute. Similarly, when another request after the expiry time arrives the window ranges are updated accordingly.

Advantages

- The algorithm doesn't suffer from the boundary conditions of fixed windows.

Disadvantages

- It consumes extra memory for storing additional information, the time stamps of incoming requests. It keeps the time stamps to provide a dynamic window, even if the request is rejected.

Sliding window counter algorithm

Unlike the previously fixed window algorithm, the sliding window counter algorithm doesn't limit the requests based on fixed time units. This algorithm takes into account both the fixed window counter and sliding window log algorithms to make the flow of requests more smooth.

Let's look at the flow of the algorithm in the below figure.

[A sliding window counter algorithm, where the green shaded area shows the rolling window of 1 minute](./sliding_window_counter.jpg)

In the above figure, we've 88 requests in the previous window while 12 in the current window. We've set the rate limit to 100 requests per minute. Further, the rolling window overlaps 15 seconds with the current window. Now assume that a new request arrives at 02:15. We'll decide which request to accept or reject using the mathematical formulation:

'''

$$\text{Rate} = R_p \times (\text{time frame} - \text{overlap time}) / \text{time frame} + R_c$$

'''

Here, R_p is the number of requests in the previous window, which is 88. R_c is the number of requests in the current window, which is 12. The time frame is 60 seconds in our case, and

overlap time is 15 seconds.

Rate=88 x (60-15)/60 + 12

Rate=78 <100

As 78 is less than 100, so the incoming request is allowed.

Essential parameters

This algorithm is relatively more complex than the other algorithms described above. It requires the following parameters:

- Rate limit (R) It determines the number of maximum requests allowed per window.
- Size of the window (W): This parameter represents the size of a time window that can be a minute, an hour, or any time slice.
- The number of requests in the previous window ($R_{\{p\}}$): It determines the total number of requests that have been received in the previous time window.
- The number of requests in the current window ($R_{\{c\}}$): It represents the number of requests received in the current window.
- Overlap time ($O_{\{t\}}$): This parameter shows the overlapping time of the rolling window with the current window.

Advantages

- The algorithm is also space efficient due to limited states: the number of requests in the current window, the number of requests in the previous window, the overlapping percentage, and so on.
- It smooths out the bursts of requests and processes them with an approximate average rate based on the previous window.

Disadvantages

- This algorithm assumes that the number of requests in the previous window is evenly distributed, which may not always be possible.

A comparison of rate-limiting algorithms

The two main factors that are common among all the rate-limiting algorithms are:

- Memory: This feature refers to the number of states an algorithm requires to maintain for a normal operation. For example, if one algorithm requires fewer variables (states) than the other, it is more space efficient.
- Burst: This refers to an increase of traffic in a unit time exceeding the defined limit.

The following table shows the space efficiency and burst of traffic for all algorithms that have been described in this lesson.

'''

A Comparison of Rate-limiting Algorithms

Algorithm	Space efficient
-----------	-----------------

Allows burst?	
---------------	--

Token bucket	Yes
--------------	-----

Yes, it allows a burst of traffic within defined limit.

Leaking bucket	Yes
----------------	-----

No

Fixed window counter	Yes
----------------------	-----

Yes, it allows bursts at the edge of the time window and can exceed the defined limit.

Sliding window log	No, maintaining the log requires extra storage.
--------------------	---

No

Sliding window counter	Yes, but it requires relatively more space than other space efficient
------------------------	---

```
algorithms.      Smooths out the burst
'''
```

Conclusion

In this lesson, we explored various popular rate-limiting algorithms. We also shed light on the advantages and disadvantages of these algorithms. Each of these algorithms can be deployed based on the user choice and the type of use case.

Rate Limiter/Requirements of a Rate Limiter's Design/README.md

Requirements of a Rate Limiter's Design

Requirements

Our focus in this lesson is to design a rate limiter with the following functional and non-functional requirements.

Functional requirements

- To limit the number of requests a client can send to an API within a time window.
- To make the limit of requests per window configurable.
- To make sure that the client gets a message (error or notification) whenever the defined threshold is crossed within a single server or combination of servers.

Non-functional requirements

- Availability: Essentially, the rate limiter protects our system. Therefore, it should be highly available.
- Low latency: Because all API requests pass through the rate limiter, it should work with a minimum latency without affecting the user experience.
- Scalability: Our design should be highly scalable. It should be able to rate limit an increasing number of clients's requests over time.

Types of throttling

A rate limiter can perform three types of throttling.

1. Hard throttling: This type of throttling puts a hard limit on the number of API requests. So, whenever a request exceeds the limit, it is discarded.
2. Soft throttling: Under soft throttling, the number of requests can exceed the predefined limit by a certain percentage. For example, if our system has a predefined limit of 500 messages per minute with a 5% exceed in the limit, we can let the client send 525 requests per minute.
3. Elastic or dynamic throttling: In this throttling, the number of requests can cross the predefined limit if the system has excess resources available. However, there is no specific percentage defined for the upper limit. For example, if our system allows 500 requests per minute, it can let the user send more than 500 requests when free resources are available.

```
'''
```

Linux operating systems provide a kernel feature known as cgroups (control groups) that limits accounts for, and isolates the resources's CPU time, system memory, disk storage, I/O, and network bandwidth's of a collection of processes. By using cgroups, the system administrator can monitor, deny access to specific resources, and reconfigure the cgroups dynamically on a running system. The cgroups feature provides the following benefits through which the system administrator gains a fine-grain control of the system resources:

- Resource limiting: Using this feature, a restriction can be imposed on groups not to exceed configured memory limit and file system cache.
- Prioritization: Through this feature, some groups can be prioritized to use a larger share of CPU cycles or disk I/O throughput.
- Accounting: This feature is used to measure a group's resource usage, which could also be used for billing purposes.

- Control: The system administrator can control groups of processes, their checkpoints, and restart via this feature.

Providing such rich features, cgroup can be used at a system (single server) level to limit resource usage not only for a single user but also for groups of users or processes.

We can extend the above concepts by designing a service that takes input from the rate-limiting service and enforces the limits on local nodes of a cluster.

Where to place the rate limiter

There are three different ways to place the rate limiter.

1. On the client side: It is easy to place the rate limiter on the client side. However, this strategy is not safe because it can easily be tampered with by malicious activity. Moreover, the configuration on the client side is also difficult to apply in this approach.

2. On the server side: As shown in the following figure, the rate limiter is placed on the server-side. In this approach, a server receives a request that is passed through the rate limiter that resides on the server.

[Rate limiter placed at the server side](./rate_limiter_server.jpg)

3. As middleware: In this strategy, the rate limiter acts as middleware, throttling requests to API servers as shown in the following figure.

[Rate limiter as middleware](./rate_limiter_middleware.jpg)

Placing a rate limiter is dependent on a number of factors and is a subjective decision, based on the organization's technology stack, engineering resources, priorities, plan, goals, and on.

Note: Many modern services use APIs to provide their functionality to the clients. API endpoints can be a good vantage point to rate limit the incoming client traffic because all traffic passes through them.

Two models for implementing a rate limiter

One rate limiter might not be enough to handle enormous traffic to support millions of users. Therefore, a better option is to use multiple rate limiters as a cluster of independent nodes. Since there will be numerous rate limiters with their corresponding counters (or their rate limit), there are two ways to use databases to store, retrieve, and update the counters along with the user information.

1. A rate limiter with a centralized database: In this approach, rate limiters interact with a centralized database, preferably Redis or Cassandra. The advantage of this model is that the counters are stored in centralized databases. Therefore, a client can't exceed the predefined limit. However, there are a few drawbacks to this approach. It causes an increase in latency if an enormous number of requests hit the centralized database. Another extensive problem is the potential for race conditions in highly concurrent requests (or associated lock contention).

2. A rate limiter with a distributed database: Using an independent cluster of nodes is another approach where the rate-limiting state is in a distributed database. In this approach, each node has to track the rate limit. The problem with this approach is that a client could exceed a rate limit at least momentarily, while the state is being collected from everyone's when sending requests to different nodes (rate-limiters). To enforce the limit, we must set up sticky sessions in the load balancer to send each consumer to exactly one node. However, this approach lacks fault tolerance and poses scaling problems when the nodes get overloaded.

Aside from the above two concepts, another problem is whether to use a global counter shared by all the incoming requests or individual counters per user. For example, the token bucket algorithm can be implemented in two ways. In the first method, all requests can share the total number of tokens in a single bucket, while in the second method, individual buckets are assigned to users. The choice of using shared or separate counters (or buckets) depends on the use case and the rate-limiting rules.

'''

Question 1

Can a rate limiter be used as a load balancer?

Answer

Load balancers prevent too many requests from being forwarded to an application server. They either reject the request based on a limit or send the request to a queue for later processing. However, the load balancer is unbiased towards incoming requests by treating them equally. For example, let's assume that our web service exposes several operations. Some of these operations are fast, and some are slow. A request for slow operations takes more time and processing power than fast operations. The load balancer doesn't know the cost of such operations. Therefore, we aim to limit the number of requests for a particular operation, we should do it on the application server rather than load balancer level.

'''

'''

Question 2

Assume a scenario in which a client intends to send requests for a particular service using two virtual machines (VMs), where one is using a VPN to a different region. Suppose that the throttling identifier works based on user credentials. Therefore, the user ID would be the same for both sessions. Moreover, let's assume that requests from different VMs can hit different data centers. How would the throttling work to prevent the user from exceeding the rate limit in this scenario?

Answer

To rate limit the incoming requests, we have two different choices to place the rate limiter.

Rate limiter per data center: One way to throttle the incoming requests from the user is to use rate limiting per data centers. Each data center will have its own rate-limiter, which limits the incoming requests. In this approach, the rate (count or rate limit) is relatively lower. Therefore, a limited number of requests are allowed per unit time. Moreover, this approach provides lower latency since the requests are normally directed to the nearest data centers located geographically. Often, latency within a data center is less than one millisecond and multiple redundant paths are available in case of some link failure.

A shared rate limiter across data centers: Another approach is to use a shared rate limiter across multiple data centers. This way, requests received from both VMs will be throttled by the single rate limiter. The number of requests allowed in this case is higher. However, this approach is relatively slower, as prior to directing a request to any nearest data center, it will pass through the shared rate limiter. Latency is often high and variable across geographically distributed data centers and not a lot of redundant paths are available.

'''

Building blocks we will use

The design of the rate limiter utilizes the following building blocks that we discussed in the initial chapters.

[Building blocks in the design of a rate limiter](./bb.jpg)

- Databases are used to store rules defined by a service provider and metadata of users using the service.
- Caches are used to cache the rules and users' data for frequent access.

- Queues are essential for holding the incoming requests that are allowed by the rate limiter. In the next lesson, we'll focus on a high-level and detailed design of a rate limiter based on the requirements discussed in this lesson.

Rate Limiter/System Design The Rate Limiter/README.md

System Design: The Rate Limiter

What is a rate limiter?

A rate limiter, as the name suggests, puts a limit on the number of requests a service fulfills. It throttles requests that cross the predefined limit. For example, a client using a particular service's API that is configured to allow 500 requests per minute would block further incoming requests for the client if the number of requests the client makes exceeds that limit.

Why do we need a rate limiter?

A rate limiter is generally used as a defensive layer for services to avoid their excessive usage, whether intended or unintended. It also protects services against abusive behaviors that target the application layer, such as denial-of-service (DOS) attacks and brute-force password attempts.

Below, we have a list of scenarios where rate limiters can be used to make the service more reliable.

- Preventing resource starvation: Some denial of service incidents are caused by errors in software or configurations in the system, which causes resource starvation. Such attacks are referred to as friendly-fire denial of service. One of the common use cases of rate limiters is to avoid resource starvation caused by such denial of service attacks, whether intentional or unintentional.

- Managing policies and quotas: There is also a need for rate limiters to provide a fair and reasonable use of resources' capacity when they are shared among many users. The policy refers to applying limits on the time duration or quantity allocated (quota).

- Controlling data flow: Rate limiters could also be used in systems where there is a need to process a large amount of data. Rate limiters control the flow of data to distribute the work evenly among different machines, avoiding the burden on a single machine.

- Avoiding excess costs: Rate limiting can also be used to control the cost of operations. For example, organizations can use rate limiting to prevent experiments from running out of control and avoid large bills. Some cloud service providers also use this concept by providing freemium services to certain limits, which can be increased on request by charging from users.

[Throttling the number of requests to API servers via a rate limiter](./sys.jpg)

How will we design a rate limiter?

In the following lessons, we will learn about the following:

1. Requirements: This is where we discuss the functional and non-functional requirements of the rate limiter. We also describe the types of throttling and locations where a rate limiter can be placed to perform its functions efficiently.
2. High-level design: In this section, we look at the high-level design to provide an overview of a rate limiter.
3. Detailed design: In this section, we discuss the detailed design of a rate limiter and explain various building blocks involved in the detailed design.
4. Rate limiter algorithms: In this lesson, we explain different algorithms that play a vital role in the operations of a rate limiter.
5. Quiz: To assess your understanding of rate limiters, we've provided a quiz at the end of this chapter.

In the next lesson, let's start by understanding the requirements and design of a rate limiter.

Design of a Unique ID Generator

In the previous lesson, we saw that we need unique identifiers for many use cases, such as identifying objects (for example, Tweets, uploaded videos, and so on) and tracing the execution flow in a complex web of services. Now, we'll formalize the requirements for a unique identifier and discuss three progressively improving designs to meet our requirements.

Requirements for unique identifiers

The requirements for our system are as follows:

- Uniqueness: We need to assign unique identifiers to different events for identification purposes.
- Scalability: The ID generation system should generate at least a billion unique IDs per day.
- Availability: Since multiple events happen even at the level of nanoseconds, our system should generate IDs for all the events that occur.
- 64-bit numeric ID: We restrict the length to 64 bits because this bit size is enough for many years in the future. Let's calculate the number of years after which our ID range will wrap around.

Total numbers available = $2^{64} = 1.8446744 \times 10^{19}$

Estimated number of events per day = 1 billion = 10^9

Number of events per year = 365 billion = 365×10^9

Number of years to deplete identifier range = $2^{64} / (365 \times 10^9) = 50,539,024.8595$ years

64 bits should be enough for a unique ID length considering these calculations.

Let's dive into the possible solutions for the problem mentioned above.

First solution: UUID

A straw man solution for our design uses UUIDs (universally unique IDs). This is a 128-bit number and it looks like 123e4567e89b12d3a456426614174000

123e4567e89b12d3a456426614174000 in hexadecimal. It gives us about 10^{38} numbers. UUIDs have different versions. We opt for version 4, which generates a pseudorandom number.

'''

Straw man solution: Often a simple solution whose primary purpose is to facilitate discussion towards better solutions.

'''

Each server can generate its own ID and assign the ID to its respective event. No coordination is needed for UUID since it's independent of the server. Scaling up and down is easy with UUID and this system is also highly available. Furthermore, it has a low probability of collisions. The design for this approach is given below:

[Generating a unique ID using the UUID approach](./uuid.jpg)

Cons

Using 128-bit numbers as primary keys makes the primary-key indexing slower, which results in slow inserts. A workaround might be to interpret an ID as a hex string instead of a number. However, non-numeric identifiers might not be suitable for many use cases. The ID isn't of 64-bit size. Moreover, there's a chance of duplication. Although this chance is minimal, we can't claim UUID to be deterministically unique. Additionally, UUIDs given to clients over time might not be monotonically increasing. The following table summarizes the requirements we have fulfilled using UUID:

'''

Requirements Filled with UUID

Unique

Scalable

Available

64-bit numeric ID

Using UUID â, â, â, â,

'''

Second solution: using a database

Letâs try mimicking the auto-increment feature of a database. Consider a central database that provides a current ID and then increments the value by one. We can use the current ID as a unique identifier for our events.

[Using a central database to generate unique IDs](./db.jpg)

'''

Point to Ponder

Question

What can be a potential problem of using a central database?

Answer

This design has a considerable problem: a single point of failure. Reliance on one database can severely affect a system. The entire system will stop working if the central database goes down.

'''

To cater to the problem of a single point of failure, we modify the conventional auto-increment feature that increments by one. Instead of incrementing by one, letâs rely on a value m , where m equals the number of database servers we have. Each server generates an ID, and the following ID adds m to the previous value. This method is scalable and prevents the duplication of IDs. The following image provides a visualization of how a unique ID is generated using a database:

[Generating IDs using the value of m](./1.jpg)

[Generating IDs using the value of m](./2.jpg)

Pros

This approach is scalable. We can add more servers, and the value of m will be updated accordingly.

Cons

Though this method is somewhat scalable, itâs difficult to scale for multiple data centers. The task of adding and removing a server can result in duplicate IDs. For example, suppose $m=3$, and server A generates the unique IDs 1, 4, and 7. Server B generates the IDs 2, 5, and 8, while server C generates the IDs 3, 6, and 9. Server B faces downtime due to some failure. Now, the value m is updated to 2. Server A generates 9 as its following unique ID, but this ID has already been generated by server C. Therefore, the IDs arenât unique anymore.

The table below highlights the limitations of our solution. A unique ID generation system shouldnât be a single point of failure (SPOF). It should be scalable and available.

'''

	Requirements Filled by UUID versus Using a Database			
	Unique	Scalable	Available	64-bit numeric ID
Using UUID	â,	â,	â,	â,
Using a database	â,	â,	â,	â,

'''

Third solution: using a range handler

Letâs try to overcome the problems identified in the previous methods. We can use ranges in a central server. Suppose we have multiple ranges for one to two billion, such as 1 to 1,000,000; 1,000,001 to 2,000,000; and so on. In such a case, a central microservice can provide a range to a server upon request.

Any server can claim a range when it needs it for the first time or if it runs out of the range. Suppose a server has a range, and now it keeps the start of the range in a local variable. Whenever a request for an ID is made, it provides the local variable value to the requestor and increments the value by one.

Let's say server 1 claims the number range 300,001 to 400,000. After this range claim, the user ID 300,001 is assigned to the first request. The server then returns 300,002 to the next user, incrementing its current position within the range. This continues until user ID 400,000 is released by the server. The application server then queries the central server for the next available range and repeats this process.

This resolves the problem of the duplication of user IDs. Each application server can respond to requests concurrently. We can add a load balancer over a set of servers to mitigate the load of requests.

We use a microservice called range handler that keeps a record of all the taken and available ranges. The status of each range can determine if a range is available or not. The state that is, which server has what range assigned to it can be saved on a replicated storage.

This microservice can become a single point of failure, but a failover server acts as the savior in that case. The failover server hands out ranges when the main server is down. We can recover the state of available and unavailable ranges from the latest checkpoint of the replicated store.

[Design of the range handler microservice](./range_handler.jpg)

Pros

This system is scalable, available, and yields user IDs that have no duplicates. Moreover, we can maintain this range in 64 bits, which is numeric.

Cons

We lose a significant range when a server dies and can only provide a new range once it's live again. We can overcome this shortcoming by allocating shorter ranges to the servers, although ranges should be large enough to serve identifiers for a while.

The following table sums up what this approach fulfills for us:

	Requirements Filled by These Three Options			64-bit numeric ID	
	Unique	Scalable	Available		
Using UUID	â	â	â	â	â
Using a database	â	â	â	â	â
Using a range handler	â	â	â	â	â

We developed a solution that provides us with a unique ID, which we can assign to various events and even use as a primary key. But what if we add a requirement that the ID is time sortable too?

Sequencer/System Design Sequencer/README.md

System Design: Sequencer

Motivation

There can be millions of events happening per second in a large distributed system. Commenting on a post on Facebook, sharing a Tweet, and posting a picture on Instagram are just a few examples of such events. We need a mechanism to distinguish these events from each other. One such mechanism is the assignment of globally unique IDs to each of these events.

Assigning a primary key to an entry in a database is another use case of a unique ID. Usually, the auto-increment feature in databases fulfills this requirement. However, that feature won't work for a distributed database, where different nodes independently generate the identifiers.

For this case, we need a unique ID generator that acts as a primary key in a distributed settingâ for example, a horizontally-sharded table.

A unique ID helps us identify the flow of an event in the logs and is useful for debugging. A real-world example of unique ID usage is Facebookâ s end-to-end performance tracing and analysis system, Canopy. Canopy uses TraceID to identify an event uniquely across the execution path that may potentially perform hundreds of microservices to fulfill one user request.

'''

Canopy: An End-to-End Performance Tracing and Analysis System

'''

[Assigning a unique TraceID to each event](./seq.jpg)

How do we design a sequencer?

Weâ ve divided the sequencerâ s comprehensive design into the following two lessons:

1. Design of a Unique ID Generator: After enlisting the requirements of the design, we discuss three ways to generate unique IDs: using UUID, using a database, and using a range handler.
2. Unique IDs with Causality: In this lesson, we incorporate an additional factor of time in the generation of IDs and explain the process by taking causality into consideration.

Unique IDs are important for identifying events and objects within a distributed system. However, designing a unique ID generator within a distributed system is challenging. In the next lesson, letâ s look at the requirements for a distributed unique ID generation system.

Sequencer/Unique IDs with Causality/README.md

Unique IDs with Causality

Causality

In the previous lesson, we generated unique IDs to differentiate between various events. Apart from having unique identifiers for events, weâ re also interested in finding the sequence of these events. Letâ s consider an example where Peter and John are two Twitter users. John posts a comment (event A), and Peter replies to Johnâ s comment (event B). Event B is dependent on event A and canâ t happen before it. The events are not concurrent here.

We can also have concurrent eventsâ that is, two events that occur independently of each other. For example, if Peter and John comment on two different Tweets, thereâ s no happened-before relationship or causality between them. Itâ s essential to identify the dependence of one event over the other but not in the case of concurrent events.

'''

Note: The scenario described above can also be handled by assigning a unique ID and encoding the dependence of events using a social graph. We might also use a separate time data structure and a simple unique ID. However, we want a unique ID to do double dutyâ provide unique identification and also help with the causality of events.

'''

[visualization](./nc)

Some applications need the events to have unique identifiers and carry any relevant causality information. An example of this is giving an identifier to the concurrent writes of a key into a key-value store to implement the last-write-wins strategy.

We can either use logical or physical clocks to infer causality. Some systems have additional requirements where we want event identifiersâ causality to map wall-clock time. An example of this is a financial application that complies with the European MiFID regulations. MiFID requires clocks to be within 100 microseconds of UTC to detect anomalies during high-volume/high-speed market trades.

'''

Note: There are many subtleties associated with logical or physical clocks. We can refer to the text below titled â Time in a Distributed Systemâ to refresh our concepts of time.

'''

We use time to determine the sequence of events in our life. For example, if Sam took a bath at 6 a.m. and ate breakfast at 7:00 a.m., we can determine that Sam took a bath before breakfast by the time stamps of each event. Time stamps, therefore, can be used to maintain causality.

Optional Revision: Time in a Distributed System

Physical clocks

There are often two types of physical clocks available in a computer: the time-of-day clock and monotonic counters.

The time-of-day clock

- This usually has lower resolution in comparison to monotonic counters.
- Network Time Protocol (NTP) can move the clock forward or backward, so it's not always monotonic.
- It may or may not incorporate leap seconds.

Monotonic counters

- Monotonic counters usually have higher resolution than time-of-day clocks.
- Monotonic counters should be used for the duration between two events rather than for the time.
- These aren't meaningful across different nodes. For instance, even on the same server with multiple processors, there can be a different counter per processor. The application needs to be careful when using counters from different processors.
- The NTP might adjust it without violating monotonicity.
- The NTP can only speed up or slow down the counter rate of change by up to 0.05%.

Reasons for clock drift

Physical clocks drift over time due to many reasons:

- Temperature differences
- The equipment's age
- Manufacturing defects
- Virtualized clocks

For instance, a server with a clock drift of 200 parts per million implies a six millisecond drift if synced every 30 seconds, or a 17 seconds drift if re-synced every 24 hours.

A study showed that on the public Internet, an NTP couldn't get clock accuracy better than 3 ms, and it can spike to one second when there's congestion on the network. Otherwise, NTPs use multiple time servers and discards outliers.

The trade-off: complexity and cost versus clock accuracy

It's possible to keep clock drift consistently small using GPS and atomic clocks, careful deployment, and monitoring. However, such a system comes with an added dollar cost, as well as increased system complexity.

Logical clocks

- Lamport clocks provide us with happened-before relationships. If event A happens before event B, then the clock value of A will be less than the clock value of B. There's a subtle point that given any two clock values of two events from any two servers, we can't compare them to infer the happened-before relationship because those two events can be concurrent (meaning not causally related).
- We can use vector clocks to infer the happened-before relationships using the clock values. For this, we'll need a counter per participating entity in the vector.
- We should note that happened-before might not mean that two events were causally related. It might be the case that one event happened before the other. Usually, we need application-level context on top of the happened-before mechanism to infer real causality.

Use UNIX time stamps

UNIX time stamps are granular to the millisecond and can be used to distinguish different events. We have an ID-generating server that can generate one ID in a single millisecond. Any

request to generate a unique ID is routed to that server, which returns a time stamp and then returns a unique ID. The ability to generate an ID in milliseconds allows us to generate a thousand identifiers per second. This means we can get $24 \text{ (hour)} * 60 \text{ (min/hour)} * 60 \text{ (sec/min)} * 1000 \text{ (ID/sec)} = 86400000 \text{ IDs}$

$24(\text{hour}) \hat{=} 60(\text{min/hour}) \hat{=} 60(\text{sec/min}) \hat{=} 1000(\text{ID/sec}) = 86400000 \text{ IDs}$ in a day. That's less than per day.

'''

Note: Connect to the following terminal to view the UNIX time stamp in milliseconds.

'''

Our system works well with generating IDs, but it poses a crucial problem. The ID-generating server is a single point of failure (SPOF), and we need to handle it. To cater to SPOF, we can add more servers. Each server generates a unique ID for every millisecond. To make the overall identifier unique across the system, we attach the server ID with the UNIX time stamp. Then, we add a load balancer to distribute the traffic more efficiently. The design of a unique ID generator using a UNIX time stamps is given below:

[Using the time stamp as an ID](./timestamps.jpg)

Pros

This approach is simple, scalable, and easy to implement. It also enables multiple servers to handle concurrent requests.

Cons

For two concurrent events, the same time stamp is returned and the same ID can be assigned to them. This way, the IDs are no longer unique.

'''

ID	Requirements Fulfilled by Each Approach				64-bit numeric
	Causality maintained	Unique	Scalable	Available	
Using UUID â ĩ,		â ĩ,	â ĩ,	â ĩ,	
Using a database â ĩ,		â ĩ,	â ĩ,	â ĩ,	
Using a range handler â ĩ,		â ĩ,	â ĩ,	â ĩ,	
Using UNIX time stamps weak		â ĩ,	weak	â ĩ,	â

'''

Twitter Snowflake

Let's try to use time efficiently. We can use some bits out of our targetted 64 bits for storing time and the remaining for other information. An overview of division is below:

[Overview of the division of bits in Twitter Snowflake](./snowflakes.jpg)

The explanation of the bits division is as follows:

â ¢ Sign bit: A single bit is assigned as a sign bit, and its value will always be zero. It makes the overall number positive. Doing so helps to ensure that any programming environment using these identifiers interprets them as positive integers.

â ¢ Time stamp: 41 bits are assigned for milliseconds. The Twitter Snowflake default epoch will be used. Its value is 1288834974657, which is equivalent to November 4, 2010, 01:42:54 UTC. We can initiate our own epoch when our system will be deployed, say January 1, 2022, at 12 midnight can be the start of our epoch from zero. The maximum time to deplete this range is shown below:

Time to range depletion = $2^{41} \text{ identifiers} / 365(\text{days/year}) \times 24 \text{ (hours/day)} \times 60 \text{ (minutes/hour)} \times 60 \text{ (seconds/minutes)} \times 1000 \text{ (identifiers/sec)} = \text{approximately 69 years}$

The above calculations give us 69 years before we need a new algorithm to generate IDs. As we saw earlier, if we can generate 1,000 identifiers per second, we aren't able to get our target of a billion identifiers per day. Though now, in the Snowflake proposal, we have ample identifiers available when we utilize worker ID and machine local sequence numbers.

â€¢ Worker number: The worker number is 10 bits. It gives us $2^{10} = 1,024$ worker IDs. The server creating the unique ID for its events will attach its ID.

â€¢ Sequence number: The sequence number is 12 bits. For every ID generated on the server, the sequence number is incremented by one. It gives us $2^{12} = 4,096$ unique sequence numbers. We'll reset it to zero when it reaches 4,096. This number adds a layer to avoid duplication.

The following slides show the conversion of the time stamp to UTC.
[snowflakes](./conversion)

Pros

Twitter Snowflake uses the time stamp as the first component. Therefore, they're time sortable. The ID generator is highly available as well.

Cons

IDs generated in a dead period are a problem. The dead period is when no request for generating an ID is made to the server. These IDs will be wasted since they take up identifier space. The unique range possible will deplete earlier than expected and create gaps in our global set of user IDs.

'''

Question

Can you find another shortcoming in the design shown above?

Answer

The physical clocks are unreliable. For such clocks, the error can be 17 seconds per day. If we measure time using these on a server, the time drifts away.

Considering a single server, we won't be affected by the drifting away of time since all transactions land on a single server. But in a distributed environment, the clocks won't remain synced.

Due to the unreliability of measuring accurate time, no matter how often we synchronize these clocks with each other or other clocks with accurate measurement methods, there will always be a skew between the various clocks involved in a distributed system.

'''

'''

Physical clock: A device used to measure and indicate time, e.g., pendulums, hourglass, quartz clock, etc.

'''

Another weak point of this system is its reliance on time. NTP can affect the working of this system. If the clock on one of the servers drifts two seconds in the future, other servers are two seconds behind. The NTP clock recognizes it and recalibrates its clock. Now, all servers will be aligned. However, in that drifting process, IDs could have been generated for a time that hasn't occurred yet, and now we'll have a pair of possible nonconcurrent events with the same time stamp. Lastly, the causality of our events won't be maintained.

'''

Note: The Network Time Protocol (NTP) is a networking protocol for clock synchronization between computer systems over packet-switched, variable-latency data networks. NTP intends to synchronize all participating computers within a few milliseconds of Coordinated Universal Time (UTC). It mitigates the effects of variable network latency.

'''

Having accurate time still remains an issue. We can read a machine's time-of-day clock with

microsecond or even nanosecond resolution. Even with this fine-grained measurement, the risks of NTP remain. Since we can't rely on physical clocks, let's put logical clocks to use.

Risks of NTP

NTP servers can be susceptible to man-in-the-middle attacks unless packets are cryptographically signed for authentication. The computational overhead involved can make this impractical on busy servers, particularly during denial of service attacks. NTP message spoofing from a man-in-the-middle attack can be used to move clocks on client computers and allow several attacks based on bypassing cryptographic key expiration. Some of the services affected by fake NTP messages identified are TLS, DNSSEC, various caching schemes (such as DNS cache), BGP, Bitcoin, and many persistent login schemes.

The following table gives an overview of the requirements that have been fulfilled using different design approaches.

ID	Causality maintained	Requirements Fulfilled by Each Approach			
		Unique	Scalable	Available	64-bit numeric
Using UUID	â€”	â€”	â€”	â€”	
Using a database	â€”	â€”	â€”	â€”	
Using a range handler	â€”	â€”	â€”	â€”	
Using UNIX time stamps	â€”	â€”	weak	â€”	â€”
Using Twitter Snowflake	â€”	â€”	â€”	â€”	

Using logical clocks

We can utilize logical clocks (Lamport and vector clocks) that need monotonically increasing identifiers for events.

Lamport clocks

In Lamport clocks, each node has its counter. All of the system's nodes are equipped with a numeric counter that begins at zero when first activated. Before executing an event, the numeric counter is incremented by one. The message sent from this event to another node has the counter value. When the other node receives the message, it first updates its logical clock by taking the maximum of its clock value. Then, it takes the one sent in a message and then executes the message.

Lamport clocks provide a unique partial ordering of events using the happened-before relationship. We can also get a total ordering of events by tagging unique node/process identifiers, though such ordering isn't unique and will change with a different assignment of node identifiers. However, we should note that Lamport clocks don't allow us to infer causality at the global level. This means we can't simply compare two clock values on any server to infer a happened-before relationship. Vector clocks overcome this shortcoming.

Vector clocks

Vector clocks maintain causal history—that is, all information about the happened-before relationships of events. So, we must choose an efficient data structure to capture the causal history of each event.

Consider the design shown below. Weâll generate our ID by concatenating relevant information just like the Twitter snowflake, with the following division:

- Sign bit: A single bit is assigned as a sign bit, and its value will always be zero.
- Vector clock: This is 53 bits and the counters of each node.
- Worker number: This is 10 bits. It gives us $2^{10} = 1,024$ worker IDs.

The following slides explain the unique ID generation using vector clocks, where the nodes A, B, and C reside in a data center.

'''

Note: In the following slides, we havenât converted the data to bits for the sake of understanding. The pattern weâll use for the unique ID is the following:

[vector-clock][worker-id]
'''

[vector clocks](./vector)

Our approach with vector clocks works. However, in order to completely capture causality, a vector clock must be at least n nodes in size. As a result, when the total number of participating nodes is enormous, vector clocks require a significant amount of storage. Some systems nowadays, such as web applications, treat every browser as a client of the system. Such information increases the ID length significantly, making it difficult to handle, store, use, and scale.

'''

Requirements Fulfilled by Each Approach				
ID	Causality maintained	Unique	Scalable	Available
Using UUID	â	â	â	â
Using a database	â	â	â	â
Using a range handler	â	â	â	â
Using UNIX time stamps	weak	â	weak	â
Using Twitter Snowflake	weak	â	â	â
Using vector clocks	â	â	weak	â

'''

'''

Question

Would a global clock help solve our problem?

Answer

Since we donât have a global clock, even if each node can assign unique time stamps to the events happening, these time stamps will come from clocks running at different rates. This would make it harder to compare them, and they wonât be unique.

However, if we have a global clock that gives us time upon request and is always accurate, the

we can maintain the causality of events, as well as a unique ID. Such a clock would be significantly valuable, but time is tricky to handle in distributed systems.

TrueTime API

Google's TrueTime API in Spanner is an interesting option. Instead of a particular time stamp it reports an interval of time. When asking for the current time, we get back two values: the earliest and latest ones. These are the earliest possible and latest possible time stamps.

Based on its uncertainty calculations, the clock knows that the actual current time is somewhere within that interval. The width of the interval depends, among other things, on how long it has been since the local quartz clock was last synchronized with a more accurate clock source.

Google deploys a GPS receiver or atomic clock in each data center, and clocks are synchronized within about 7 ms. This allows Spanner to keep the clock uncertainty to a minimum. The uncertainty of the interval is represented as epsilon.

The following slides explain how TrueTime's time master servers work with GPS and atomic clocks in multiple data centers.

[TrueTime](./Google)

The following slides explain how time is calculated when the client asks to give TrueTime.

[TrueTime](./TrueTime)

Spanner guarantees that two confidence intervals don't overlap (that is, $A_{\text{earliest}} < A_{\text{latest}} < B_{\text{earliest}} < B_{\text{latest}}$), then B definitely happened after A.

We generate our unique ID using TrueTime intervals. Let's say the earliest interval is T_{E} , the latest is T_{L} , and the uncertainty is μ . We use T_{E} in milliseconds as a time stamp in our unique ID.

- Time stamp: The time stamp is 41 bits. We use T_{E} as a time stamp.

- Uncertainty: The uncertainty is four bits. Since the maximum uncertainty is claimed to be 64 ms, we'll use four bits for storing it.

- Worker number: This is 10 bits. It gives us $2^{10} = 1,024$ worker IDs.

- Sequence number: This is eight bits. For every ID generated on the server, the sequence number is incremented by one. It gives us $2^8 = 256$ combinations. We'll reset it to zero when it reaches 256.

[Node B generating a unique ID for its event using TrueTime]

Pros

TrueTime satisfies all the requirements. We're able to generate a globally unique 64-bit identifier. The causality of events is maintained. The approach is scalable and highly available.

Cons

If two intervals overlap, then we're unsure in what order A and B occurred. It's possible they're concurrent events, but a 100% guarantee can't be given. Additionally, Spanner is expensive because it ensures high database consistency. The dollar cost of a Spanner-like system is also high due to its elaborate infrastructure needs and monitoring.

The updated table provides the comparison between the different system designs for generating unique ID.

'''

		Requirements Fulfilled by Each Approach			
ID	Causality maintained	Unique Scalable	Available	64-bit numeric	
Using UUID	â ĩ,	â ĩ,	â ĩ,		
â ĩ,					
Using a database	â ĩ,	â ĩ,	â ĩ,		
â ĩ,					
Using a range handler	â ĩ,	â ĩ,	â ĩ,		
â ĩ,					
Using UNIX time stamps	â ĩ,	weak	â ĩ,		â ĩ,
weak					
Using Twitter Snowflake	â ĩ,	â ĩ,	â ĩ,		
weak					
Using vector clocks	â ĩ,	weak	â ĩ,		can e
â ĩ,					
Using TrueTime	â ĩ,	â ĩ,	â ĩ,		
â ĩ,					

'''

Summary

We want to avoid duplicate identifiers. Consider what will happen if duplicate payment or purchase orders are generated.

UUIDs provide probabilistic guarantees about the keysâ non-collision. Deterministically getting non-collision guarantees might need consensus among different distributed entities or stores and read from the replicated store.

As key length becomes large, it often causes slower tuple updates in a database. Therefore, identifiers should be big enough but not too big.

Often, itâ s desirable that no one is able to guess the next ID. Otherwise, undesirable data leaks can happen, and the organizationâ s competitors may learn how many orders were processed in a day by simply looking at order IDs. Adding a few random numbers to the bits of the identifier make it hard to guess, although this comes at a performance cost.

We can use simple counters for generating unique IDs if we donâ t want to relate ID to time. Fetching time stamps is slower than simple counters.

We can just use simple counters for generating unique IDs if we donâ t want to relate ID to time. Fetching time stamps is slower than simple counters, though this requires that we store generated IDs persistently. The counter needs to be stored in the database. Storage comes with its own issues. These include multiple concurrent writes becoming overwhelming for the database and the database being the single point of failure.

For some distributed databases, such as Spanner, it can hurt to generate monotonically increasing or decreasing IDs. Google reports the following: â In fact, using monotonically increasing (or decreasing) values as row keys does not follow best practices in Spanner because it creates hotspots in the database, leading to a reduction in performance.â

Note: Globally ordering events is an expensive procedure. A feature that was fast and simple in a centralized database (auto-increment based ID) becomes slow and complicated in its distributed counterpart due to some fundamental constraints (such as consensus, which is difficult among remote entities).

For example, Spanner, a geographically distributed database, reports that if a read-update transaction on a single cell (one column in a single row) has a latency of 10 milliseconds (ms), then the maximum theoretical frequency of issuing of sequence values is 100 per second. This maximum applies to the entire database, regardless of the number of client application instances, or the number of nodes in the database. This is because a single node always manages a single row. If we could compromise on the requirements for global orderings and gapless identifiers, we would be able to get many identifiers in a shorter time, that is, a better performance.

Sharded Counters/Detailed Design of Sharded Counters/README.md

Detailed Design of Sharded Counters

Detailed design

We'll now discuss the three primary functionalities of the sharded counter: creation, write, and read in detail. We'll answer many important questions by using Twitter as an example. These questions include:

- How many shards should be created against each new tweet?
- How will the shard value be incremented for a specific tweet?
- What will happen in the system when read requests come from the end users?

Sharded counter creation

As we discussed earlier, when a user posts a tweet on Twitter, the `\createCounter` API is called. The system creates multiple counters for each newly created post by the user. The following is the list of main counters created against each new tweet:

- Tweet like counter
- Tweet reply counter
- Tweet retweet counter
- Tweet view counter in case a tweet contains video

Now, the question is how does the system decide the number of shards in each counter? The number of shards is critical for good performance. If the shard count is small for a specific write workload, we face high write contention, which results in slow writes. On the other hand, if the shard count is too high for a particular write profile, we encounter a higher overhead on the read operation. The reason for slower reads is because of the collection of values from different shards that might reside on different nodes inside geographically distributed data centers. The reading cost of a counter value rises linearly with the number of shards because the values of all shards of a respective counter are added. The writes scale linearly as we add new shards due to increasing requests. Therefore, there is a trade-off between making writes quick versus read performance. We'll discuss how we can improve read performance later.

The decision about the number of shards depends on many factors that collectively try to predict the write load on a specific counter in the short term. For tweets, these factors include follower count. The tweet of a user with millions of followers gets more shards than a user with few followers on Twitter because there is a possibility that their tweets will get many, often millions, of likes. Sometimes, a celebrity tweet includes one or more hashtags. The system also creates the sharded counter for this hashtag because it has a high chance of being marked as a trend.

Many human-centered activities often have a long-tailed activity pattern, where many people are concentrated on a relatively small set of activities. Perhaps shortened attention spans might be playing a role here. It means that after some time, the flurry of likes will die down, and we might not need as many shards now for a counter as we once needed. Similarly, our initial prediction for future writes might turn out to be wrong, and we might need more shards to handle write requests. We require that our system can dynamically expand or shrink the number of shards based on the current need.

We need to monitor the write load for all the shards to appropriately route requests to specific shards, possibly using load balancers. Such a feedback mechanism can also help us

decide when to close down some of the shards for a counter and when to add additional shards. This process does not only provide good performance for the end user but also utilizes our resources at near-optimal levels.

'''

Question

What happens when a user with just a few followers has a post go viral on Twitter?

Answer

The system needs to detect such cases where a counter unexpectedly starts getting very high write traffic. We'll dynamically increase the number of shards of the affected counter to mitigate the situation.

'''

Burst of writes requests

As we mentioned earlier, millions of users interact with our example celebrity's tweet, which eventually sends a burst of write requests to the system. The system assigns each write request to the available shards of the specified counter of the particular tweet. How does the system select these shards operating on different computational units (nodes) to assign the write requests? We can use three approaches to do this:

Round-robin selection

One way to solve the above problem is to use a round-robin selection of shards. For example, let's assume the number of shards is 100. The system starts with shard_1, then shard_2, and continues until it reaches shard_100. Usually, round-robin work allocation either overloads or underutilizes resources because scheduling is done without considering the current load conditions. However, if each request is similar (and roughly needs the same time to serve), a round-robin approach can be used and is attractive for its simplicity.

In the following illustrations, we assume that user requests are first handed out to an appropriate server by the load balancer. Then, each such server uses its own round-robin scheduling to use a shard:

[Round-robin technique](./rr)

Random selection

Another simple approach can be to uniformly and randomly select a shard for writing. The challenge with both round-robin selection and random selection is with variable load changes on the nodes (where shards are hosted). It is hard to appropriately distribute the load on available shards. Load variability on nodes is common because a physical node is often being used for multiple purposes.

Metrics-based selection

The third approach is shard selection based on specific metrics. For example, a dedicated node (load balancer) manages the selection of the shards by reading the shards' status. The below slides go over how sharded counters are created:

[Metrics](./metrics)

Manage read requests

When the user sends the read request, the system will aggregate the value of all shards of the specified counter to return the total count of the feature (such as like or reply). Accumulating values from all the shards on each read request will result in low read throughput and high read latency.

The decision of when the system will sum all shards values is also very critical. If there is high write traffic along with reads, it might be virtually impossible to get a real current value because by the time we report a read value to the client, it will have already changed. So, periodically reading all the shards of a counter and caching it should serve most of the use cases. By reducing the accumulation period, we can increase the accuracy of read values.

'''

Question

Can you think of a use case where sharded counters with the above-mentioned consistency model might not be suitable?

Answer

We might not use shared counters with a relaxed consistency model where we need strong consistency. An example can be read-then-write scenarios where we first need to get the accurate value of something before deciding to modify it (actually, such a scenario will need transaction support).

'''

Using sharded counters for the Top K problem

This section will discuss how we can use sharded counters to solve a real-world problem known as the Top K problem. We'll continue to use the real-time application Twitter as an example, where calculating trends is one of the Top K problems for Twitter. Here, K represents the number of top trends. Many users use various hashtags in their tweets. It is a huge challenge to manage millions of hashtags' counts to show them in individual users' trends timelines on their locality.

The sharded counter is the key to the above problem. As discussed earlier, on Twitter, the system creates the counters for each hashtag and decides the shard count according to the user's followers who used the hashtag in the tweet. When users on Twitter use the same hashtag again in their tweet, the count maintains the same counter created initially on the first use of that hashtag.

Twitter shows trends primarily based on the popularity of the specific hashtag in a user's region. Twitter will maintain a separate counter for each discussed metric with a global hashtag counter. Let's discuss each of the metrics below:

- Region-wise hashtag count indicates the number of tweets with the same hashtag used within a specific geographical region. For example, thousands of tweets with the same tags from New York City suggest that users in the New York area may see this hashtag in their trends timeline.
- A time window indicates the amount of time during which tweets with specific tags are posted

[The system calculates the 24-hour count of the specified hashtag](./topk.jpg)

Below is more detail on the above illustration:

The global hashtag counter represents the total of all location-based counters.

Location-based counters represent their current count when the system reaches the set threshold in a specified time and the hashtag becomes a trend for some users. For example, Twitter sets 10,000 as a threshold. When location-based hashtag counts reach 10,000, Twitter shows these hashtags in the trends timeline of the users of the respective country where the hashtag is being used. The specified hashtag may be displayed worldwide if counts increase in all countries.

Next, we'll discuss Top K tweets in a user's homepage timeline. The Top K tweets include accounts the user is following, tweets the user has liked, and retweets of accounts the user follows. Tweets get priority in Top K problems based on follower count and time. Twitter also shows promoted tweets and some tweets of accounts the user doesn't follow in the user's home timeline, depending on the tweet's popularity and location.

'''

Note: Twitter also uses other metrics to optimize Top K selection, but we've discussed the leading metrics here.

'''

Placement of sharded counters

An important concern is where we should place shared counters. Should they reside on the same nodes as application servers, in separate nodes in the data center, in nodes of CDN at the edge

of a network near the end users? The exact answer to this question depends on our specific use case. For Twitter, we can compute counts by placing sharded counters near the user, which can also help to handle heavy hitter and Top K problems efficiently.

'''

Question

Should we lock all shards of a counter before accumulating their values?

Answer

No. Reads can happen concurrently with writes without the need for an across-shards lock. This lock will decimate the write performance, the original reason we used sharded counters. Under relaxed consistency model, where the value of a counter might not reflect the exact current value, there is no need for simultaneous read locks across all shards.

However, depending on the specific use case, such a mechanism might be used when reads frequency is very low.

'''

Reads can store counter values in appropriate data stores and rely on the respective data stores for read scalability. The Cassandra store can be used to maintain views, likes, comments, and many more counts of the users in the specified region. These counts represent the last computed sum of all shards of a particular counter.

When users generate a timeline, read requests are forwarded to the nearest servers, and then the persisted values in the store can be used to respond. This storage also helps to show the region-wise Top K trends. The list of local Top K trends is sent to the application server, and then the application server sorts all the lists to make a list of global Top K trends. Eventually, the application server sends all countersâ details to the cache.

We also need storage for the sharded counters, which store all information about them with their metadata. The Redis or Memcache servers can play a vital role here. For example, each tweetâs unique ID can become the key, and the value of this key can be a counter ID, or a list of countersâ IDs (like counter, reply counter, and so on). Furthermore, each counter ID has its own key-value store where the counter (for example, a likes counter) ID is a key and the value is a list of assigned shards.

The job of identifying the relevant counter and mapping all write requests to the appropriate counter in sharded counters can be done in parallel. We map the all-write request to the appropriate counter, and then each counter chooses a shard randomly based on some metrics to do increments and decrements. In contrast, we reduce periodically to aggregate the value of all shards of the particular counter. Then, these counter values can be stored in the Cassandra store. The slides below help illustrate these points:

[Placement](./placement)

Evaluation of the sharded counters

This section will evaluate the sharded counters and explain how sharded counters will increase performance by providing high availability and scalability.

Availability

A single counter for any feature (such as like, view, or reply) has a high risk of a single point of failure. Sharded counters eliminate a single point of failure by running many shards for a particular counter. The system remains available even if some shards go down or suffer a fault. This way, sharded counters provide high availability.

Scalability

Sharded counters allow high horizontal scaling as needed. Shards running on additional nodes can be easily added to the system to scale up our operation capacity. Eventually, these additional shards also increase the systemâs performance.

Reliability

Another primary purpose of the sharded counters is to reduce the massive write request by mapping each write request to a particular shard. Each write request is handled when it comes, and there is no request waiting in the queue. Due to this, the hit ratio increases, and the system's reliability also increases. Furthermore, the system periodically saves the computed counts in stable storage—Cassandra, in this case.

Conclusion

Nowadays, sharded counters are a key player in improving the overall performance of giant services. They provide high scalability, availability, and reliability. Sharded counters solve significant issues, including the heavy hitters and Top K problems, that are very common in large-scale applications.

Sharded Counters/High-level Design of Sharded Counters/README.md

High-level Design of Sharded Counters

High-level solution sketch

Managing millions of tweet likes requires many counters operating on many nodes. To manage these counters, we need an efficient system that can provide high performance and scalability as the number of users grows.

What will happen when a single tweet on Twitter gets a million likes, and the application server receives a write request against each like to increment the relevant counter? These millions of requests are eventually serialized in a queue for data consistency. Such serialization is one way to deal with concurrent activity, though at the expense of added delay. Real-time applications want to keep the quality of experience high by providing as minimum as possible latency for the end user.

Let's see the illustration below to understand this problem:

[Celebrity vs. common people tweet](./twitter.jpg)

A single counter for each tweet posted by a celebrity is not enough to handle millions of users. The solution to this problem is a sharded counter, also known as a distributed counter, where each counter has a specified number of shards as needed. These shards run on different computational units in parallel. We can improve performance and reduce contention by balancing the millions of write requests across shards.

First, a write request is forwarded to the specified tweet counter when the user likes that tweet. Then, the system chooses an available shard of the specified tweet counter to increment the like count. Let's look at the illustration below to understand sharded counters having specified shards:

[Counters and their shards working on different computational units](./counters.jpg)

In the above illustration, the total number of shards per counter is $(N+1)$. We'll use an appropriate value for N according to our needs. Let's discuss an example to understand how sharded counters handle millions of write and read requests for a single post.

Let's assume that a famous YouTube channel with millions of subscribers uploads a new video. The server receives a burst of write requests for video views from worldwide users. First, a new counter initiates for a newly uploaded video. The server forwards the request to the corresponding counter, and our system chooses the shard randomly and updates the shard value, which is initially zero. In contrast, when the server receives read requests, it adds the values of all the shards of a counter to get the current total.

We can use a sharded counter for every scenario where we need scalable counting (such as Facebook posts and YouTube videos).

API design for sharded counters

This section discusses the APIs that will be called for sharded counters. Our API design will help us understand the interactions between sharded counters and their callers. To make our discussion concrete, weâll discuss each API function in the context of Twitter. Letâs develop APIs for each of the following functionalities:

- Create counter
- Write counter
- Read counter

Although the above list of API functions is not exhaustive, they represent some of the most important ones.

Create counter

The `\createCounter` API initializes a distributed counter for use. The `\createCounter` API is given below:

```
'''
createCounter(counter_id, number_of_shards)
'''
```

```
'''
```

Parameter	Description
-----------	-------------

<code>counter_id</code>	It represents the unique ID of the counter. The caller of this API can use a sequencer to get a unique identifier. See the lesson on sequencer building blocks for more details.
-------------------------	--

<code>number_of_shards</code>	It specifies the number of shards for the counter.
-------------------------------	--

We can use an appropriate data store to keep our metadata, which includes counter identifiers, their number of shards, and the mapping of shards to physical machines.

Letâs consider Twitter as an example to understand how an application uses the above API. The `\createCounter` API is used when a user posts something on social media. For instance, if a user posts a tweet on Twitter, the application server calls the `\createCounter` API. The `content_type` parameter is the post type that the system uses to decide the number of counters that need to be created. For example, the system needs a view counter only if the tweet contains a video clip.

To find an appropriate value for `number_of_shards`, we can use the following heuristics:

- The `followers_count` parameter denotes the followersâ count of the user who posts a tweet.
- The `post_type` parameter specifies whether the post is public or protected. Protected tweets are for the followers only, and in this case, we have a better predictor of the number of shards.

Write counter

The `\writeCounter` API is used when we want to increment (or decrement) a counter. In reality, specific shard of the counter is incremented or decremented, and our service makes that decision based on multiple factors, which weâll discuss later. The `\writeCounter` API is given below:

```
'''
writeCounter(counter_id, action_type)
'''
```

```
'''
```

Parameter	Description
-----------	-------------

<code>counter_id</code>	It is the unique identifier (provided at the time of counter creation).
-------------------------	---

<code>action_type</code>	It specifies the intended action (increment or decrement value of the
--------------------------	---

counter). We extract the required information about the counter from our data store.
'''

In our Twitter example, the \writeCounter API is used when users act (by liking, replying, and so on) on someone else's post or their own post.

Read counter

The \readCounter API is used when we want to know the current value of the counter. Our system fetches appropriate information from the datastore to collect value from all shards. The \readCounter API is given below:

'''
readCounter(counter_id)
'''

'''

Parameter	Description
-----------	-------------

counter_id	It is the unique identifier (provided at the time of counter creation). For Twitter, the counter_id will be decided based on the following metrics: The tweet_id specifies the tweet's unique ID for which the request is generated. We can use tweet_id to get the counter_id for all the counters of the features (likes, retweets, and so on).
------------	---

'''

The \readCounter API is called when users want to see the number of likes or view counts on a specific tweet. Usually, this API is triggered by another API when users want to see their home or user timeline.

The following section will discuss what happens in the back-end system when all the above APIs are called.

Sharded Counters/Quiz on the Sharded Counters' Design/README.md

Quiz on the Sharded Counters' Design

'''

1 What kind of problem would occur if our system created a counter with a few shards for a post from a user with one million followers?

A)

High read contention

B)

High write contention

C)

Users face delays on a read request

D)

Users get a quick response on the liked post

'''

'''

2 (Select all that apply.) What problem might arise if shards are selected in an order (sequentially) rather than randomly?

A)

The write request queue increases.

B)

Maximum shard utilization decreases.

C)

One-by-one selection of shards will take more time.

D)

The user will not get quick responses.

'''

'''

3 (Select all that apply.) Which situation needs a larger number of sharded counters to handle the traffic on YouTube?

A)

A video posted by a channel with millions of subscribers

B)

A video with a few likes over a long period of time

C)

A video with millions of views but a few dislikes

D)

Long videos posted by a channel with a few subscribers

'''

Answers

1. (B)

2. (A) (B) (D)

3. (A) (C)

Sharded Counters/System Design The Sharded Counters/README.md

System Design: The Sharded Counters

Problem statement

Real-time applications like Facebook, Twitter, and YouTube have high user traffic. Users interact with the applications and perform multiple operations (view, like, comment, and so on) depending on the application's structure. For instance, an image is posted on a Facebook page that has millions of followers, and the post's likes rapidly increase after each millisecond. Here, it might be easy to count the likes for this single image, but what will we do when thousands of such images or videos are uploaded simultaneously by many celebrities, each with millions of followers. This problem is known as the heavy hitters problem.

The above scenario shows how a simple counting operation becomes challenging to manage with precision and performance. The following figure shows YouTube's videos that were viewed by millions of users in a 24-hour span in August 2021:

[YouTube videos' views in 24 hours](./views.jpg)

On average, six thousand tweets are sent on Twitter within one second, which equals 360,000 tweets per minute and about 500 million tweets per day. A challenging task is to handle billions of likes on these 500 million tweets per day. The following table shows the most liked tweets in one day as of 2022:

[Most liked tweets in a single day](./likes.jpg)

How will we handle millions of write requests coming against the likes on thousands of tweets per minute? The challenge is that writing takes more time than reading, and concurrent activity makes this problem harder. As the number of concurrent writes increases for some counter (which might be a variable residing in a node's memory), the lock contention increases non-linearly. After some point, we might spend most of the time acquiring the lock so that we could safely update the counter.

How will we design sharded counters?

We have divided the design of sharded counters into three lessons:

1. High-level Design: We'll discuss the high-level design of sharded counters in this lesson. In addition, we'll also briefly explain the API design.
 2. Detailed Design: This lesson will dive deeply into the design of sharded counters. Moreover, we'll also evaluate our proposed design.
 3. Quiz: We'll review major concepts of sharded counters design with a quiz.
- Let's begin with the high-level solution sketch of sharded counters.

Spectacular Failures/AWS Kinesis Outage Affecting Many Organizations/README.md

AWS Kinesis Outage Affecting Many Organizations

Amazon Kinesis allows us to aggregate, process, and analyze real-time streaming data to get timely insights and react quickly to the information it provides. It continuously captures gigabytes of data from hundreds of thousands of sources per second. The Kinesis service's frontend handles authentication, throttling, and distributes workloads to its back-end "workhorse" cluster via database sharding. On November 25, 2020, the Amazon Kinesis service disrupted in the US-East-1 (Northern Virginia) region, affecting thousands of other third-party services. The failure was significant enough to take out a large portion of Internet services.

Sequence of events

- According to Amazon, the event was triggered by adding a small capacity to the AWS front-end fleet of servers, scheduled from 2:44 a.m. PST to 3:47 a.m. PST.

- The addition of the new capacity caused all the servers in the fleet to exceed the maximum number of threads that are allowed by an operating system configuration.

- Due to exceeding the limit on threads, cache construction was failing to complete, and front-end servers were ending with useless shard maps that left them unable to route requests to back-end clusters.

- Other primary Amazon services also stopped working, including Amazon Cognito and CloudWatch.

- Kinesis Data Streams is used by Amazon Cognito to gather and analyze API usage patterns. Because of the long-running issue with Kinesis Data Streams, a hidden error in the buffering code which is required for Cognito services caused the Cognito web servers to start blocking the backlogged Kinesis Data Stream buffers. As a result, Cognito customers witnessed an increase in API failures and latencies for Cognito user pools and identity pools, making it impossible for external users to authenticate or receive temporary AWS credentials.

- CloudWatch uses Kinesis Data Streams to process metrics and log data. The PutMetricData and PutLogEvents APIs in CloudWatch encountered higher error rates and latencies, and alerts were set to INSUFFICIENT DATA. The great majority of metrics were unable to be processed due to higher error rates and latencies. When CloudWatch was experiencing these greater issues, internal and external clients couldn't persist all metric data to the CloudWatch service.

[Sequence of events](./seq.jpg)

- Due to the problems with CloudWatch metrics, two services were also impacted. First, AutoScaling policies that rely on CloudWatch measurements suffered delays. Second, Lambda experienced the effect. Currently, posting metric data to CloudWatch is required as a part of Lambda function invocation. If CloudWatch is unavailable, Lambda metric agents are meant to buffer metric data locally for a period of time. The metric data buffering became so large that it generated memory congestion on the underlying service hosts utilized for Lambda function invocations, leading to higher error rates.

- Increased API failures and event processing delays plagued CloudWatch Events and EventBridge. EventBridge is used by Elastic Container Service (ECS) and Elastic Kubernetes Service (EKS) to

drive internal processes for managing client clusters and jobs. This has an impact on new cluster provisioning, existing cluster scaling, and task deprovisioning.

- Apart from the service issues, Amazon also experienced delays in communicating service status to their customers. Amazon uses two dashboards for communication with their customers, Service Health Dashboard and Personal Health Dashboard. The Service Health Dashboard informs all customers of events, such as the current one. It was down because of its dependency on Cognito, which was impacted by this event.

- Apart from the disruption in the Amazon services, this also had a ripple effect on thousands of third-party online services, applications, and websites. This includes Adobe Spark, Acorns, Coinbase, Washington Post, and hundreds of such services.

Analysis

- Complexity of enhancing scalability: Weâve been stressing the need for horizontal scalability in all of our design problems. This outage event shows that in practice, adding more capacity to a serving cluster so that we donât deny any client requests can be challenging and can have unintended side effects.

- The need for a trained team: Training the production team to deal with such unforeseen situations is a challenging task, but itâs worth it and can result in expedited recovery. In addition, randomly restarting the front-end servers and suspecting that the memory pressure causes the problem alludes to the challenges of reaching the root causes under the stress of time.

- Reading from authoritative servers during bootstrap: During the bootstrap process, itâs a good idea to take data from the authoritative metadata store rather than from the front-end servers to reduce the impact of such failures.

- Identification of faults in initial stages: Thereâs a need for automated processes to identify the causes of failure within the initial stages of its occurrence.

- Proper testing mechanisms can reduce the severity of the fault: The new capacity thatâs causing the servers in the fleet to exceed the maximum number of threads is a potential bug that should have been fixed earlier than before restarting the servers. This reflects the inability to properly test the system. There should be a kind of simulator to test all cases before deploying additional capacity into the fleet.

- Finding potential bugs before planned events: The issue with Kinesis Data Streams triggered a latent bug in the buffering code that caused the Cognito web servers to begin to block on the backlogged Kinesis Data Stream buffers. The potential bugs should be identified and fixed before critical planned events, such as adding capacity or software and hardware maintenance. Adopting automated processes for resource allocation: The Cognito team sought to reduce the effect of the Kinesis faults in the early phases of the event by providing extra capacity and therefore boosting their ability to buffer calls to Kinesis. Instead of manually assigning capacity, there should be an automated system to increase or reallocate capacity in such events.

Lessons learned

- Testing: Proper testing and identifying the potential bugs are essential. In this case, the number of threads exceeding a maximum limit defined by the operating system seems to be a possible bug.

- Ready operations team: A bug bringing an overall system to a halt is the single point of failure, which is possible in a complex system. The production team should be trained and ready for such events.

- Reducing number of servers: To get significant headroom in the thread count used as the total

threads, thereâs a need to move to more powerful CPU and memory servers. This will reduce the total number of servers and the threads required by each server to communicate across the fleet. Having fewer servers means that each server maintains fewer threads. Amazon is adding fine-grained alarming for thread consumption in the service.

- Front-end fleet changes: Several changes are required to radically improve the cold start time for the front-end fleet. Moreover, the front-end server cache needs to be moved to a dedicated fleet.

- Avoiding recurrent failures: To avoid recurrent failures in the future, extensive AWS services, like CloudWatch and others, must be moved to a separate, partitioned front-end fleet.

'''

Question

What possible measures should Amazon have adopted to safeguard against the kind of failures they faced in Kinesis Data Streams?

Answer

Possible solutions

- Dividing the region into independent failure domains would have reduced the blast radius of the event and made it possible for the production team to quickly recover from the problem.

- They should have a system like Facebookâs Resource Allowance System for capacity reservation at the time of planned and unplanned events.

- Building an application across multiple clouds or AWS regions would have made it easier for the affected customers to recover quickly.

- Thereâs a need to uncouple services to an extreme extent to eliminate cross-dependency issues.

- Failures in a complex system are inevitable. However, some important services like the status dashboard should be hosted on different servers, either inside the service or in some third-partyâs infrastructure.

'''

Spectacular Failures/AWS Wide Spread Outage/README.md

AWS Wide Spread Outage

Introduction

Several Amazon services and other services that depend on AWS were disrupted by an outage incident that spanned more than eight hours on Tuesday, December 7, 2021, at approximately 7:30 a.m. PST. The incident impacted everything from home consumer products to numerous commercial services.

This hours-long outage made headlines in the popular media, such as this one from the Financial Times: âFrom angry Adele fans to broken robot vacuums: AWS outage ripples through the US.â The outage affected millions of users worldwide, including individuals who were using the AWS online stores and other businesses that relied heavily on AWS for providing their services.

The disruption caused by AWS emphasized the need for a decentralized Internet where services donât rely on a small number of giant companies. According to Gartner, 80% of the cloud market is handled by just five companies. Amazon, with a 41% share of the cloud computing market, is the largest.

Outages like the one above remind us of famous Lamportâs quip: âA distributed system is one in which the failure of a computer you didnât even know existed can render your own computer unusable.â

Sequence of events

- An automated action to expand the capacity of one of the AWS services near the main AWS network elicited unusual behavior from a significant number of customers within the internal network.

- As a result, there was a significant increase in connection activity, which swamped the networking equipment that connected the internal network to the main AWS network.

- Communication between these networks got delayed. These delays enhanced latency and failures for services interacting between these networks, leading to a rise in retries and ping requests.

- As a result, the devices connecting the two networks experienced constant overload and performance difficulties.

- This overload instantly affected the availability of real-time monitoring data for AWS internal operations teams, hampering their ability to identify and remedy the cause of the congestion.

- Operators relied on logs to figure out what was going on and initially observed heightened internal DNS failures.

The following slides show the series of events that led to the outage.

[Sequence of Events](./arch)

Analysis

- Hampered AWS services: The networking difficulties affected a variety of AWS services, impacting customers that utilized these service capabilities. Since the primary AWS network remained unaffected, certain client applications that donâ t depend on these capabilities suffered relatively minor consequences as a result of this occurrence. AWS users, such as Amazon RDS, EMR, and Workspaces, were unable to generate new resources due to the inability of the system to launch new EC2 instances.

- Impaired control plane: Apart from the AWS services, the AWS control planes that are used for establishing and managing AWS resources were also impacted. These control planes take advantage of internal network-hosted services. For example, EC2 instances werenâ t affected by this event but EC2 APIs suffered from increased latency and error rates.

- Slow restoration: Since DNS is the basis for all communication across the web, operators focused on moving the internal DNS traffic away from congested areas of the network in order to improve availability. However, since monitoring services were unavailable, operators had to identify and disable major sources of traffic manually. This further improved the availability of services.

- Elastic Load Balancers (ELB): Current Elastic Load Balancers were unaffected by the incident. However, the rising API error rates and latencies for the ELB APIs resulted in longer provisioning times for new load balancers.

Lessons learned

- Independent communication system: While the intention of having an internal network thatâ s separate from the main network is the right idea, they werenâ t truly independent. A sequence of events highlighted their dependency. Finding such dependencies is crucial to truly benefit from independent networks for internal service use and external client use.

Contingency plan: Although AWS takes measures to prepare its infrastructure for sudden surges in customer requests or power usage, the organization still found itself in a difficult situation due to the unusual severity of the failure. Investing in greater risk-based contingency planning benefits organizations during times of crisis.

- Ready operations team: A bug bringing an overall system to a halt is a single point of failure, which is possible in a complex system. The production team should be trained and ready for such events.

- Multiple cloud computing providers: Organizations can replicate their operations among many cloud computing providers so that no single failure knocks them out of action. However, this is easier said than done. An alternative approach is to employ different regions of the same provider for various purposes.

- Testing: Carrying out proper testing and identifying the potential bugs are both essential. In this case, overwhelming the network devices resulted in communication delays between these networks.

'''

Question

What can we do to safeguard against the series of faults experienced by Amazon?

Answer

We suggest the following solutions:

End-to-end transparency at each layer gives the information required to run the sites and services properly.

Building an application across multiple clouds or AWS regions would have made it easier for the affected customers to recover quickly.

Thereâs a need to uncouple services to an extreme extent to eliminate cross-dependency issues.

'''

Spectacular Failures/Facebook, WhatsApp, Instagram, Oculus Outage/README.md

Facebook, WhatsApp, Instagram, Oculus Outage

In October 2021, Facebook experienced a global outage for about six hours, affecting its other affiliates, including Messenger, WhatsApp, Mapillary, Instagram, and Oculus. Popular media reported the impact of this failure prominently. The New York Times reported the following headline: âGone in Minutes, Out for Hours: Outage Shakes Facebook.â

According to one estimate, this outage cost Facebook about \$100 million in revenue losses and many billions due to the declining stock of the company.

Letâs look at the sequence of events that caused this global problem.

The sequence of events

The following sequence of events led to the outage of Facebook and its accompanied services:

A routine maintenance system was needed to find out the spare capacity on Facebookâs backbone network.

Due to a configuration error, the maintenance system disconnected all the data centers from each other on the backbone network. Earlier, an automated configuration review tool was used to look for any issues in the configuration, but tools like these arenât perfect. In this specific case, the review tool missed the problems present in a configuration.

The authoritative domain name systems (DNSs) of Facebook had a health check rule that if it couldnât reach Facebookâs internal data centers, it would stop replying to client DNS queries by withdrawing the routes.

When the network routes where Facebookâs authoritative DNS was hosted were withdrawn, all cached mapping of human-readable names to IPs soon timed out at all public DNS resolvers. When a client resolves `www.facebook.com`, the DNS resolver first goes to one of the root DNS servers which provides a list of authoritative DNS servers for `.com`. The resolver connects to one of them, and then they provide IPs for the authoritative DNS servers for Facebook. However, after route withdrawal, it was impossible to reach them.

Then, no one was able to reach Facebook and its subsidiaries.

The slides below depict the events in pictorial form.

Analysis

Some of the key takeaways from the series of events shown above are the following:

- From common activity to catastrophe: The withdrawal or addition of the network routes is a relatively common activity. The confluence of bugs (first a faulty configuration, and then a bug in an audit tool not able to detect such a problem) triggered a chain of events, which resulted in cascading failures. A cascading failure is when one failure can trigger another failure, ultimately bringing the whole system down.
- Reasons for slow restoration: It might seem odd that it took six hours to restore the service. Wasn't it easy to reannounce the withdrawn routes? At the scale that Facebook is operating on, rarely is anything done manually, and there are automated systems to perform changes. The internal tools probably relied on the DNS infrastructure, and when all data centers are offline from the backbone, it would have been virtually impossible to use those tools. Manual intervention would have been necessary. Manually bootstrapping a system of this scale isn't easy. The usual physical and digital security mechanisms that were in place made a slow process to intervene manually.
- Low probability events can occur: In retrospect, it might seem odd that authoritative DNS systems disconnect themselves if internal data centers aren't accessible. This is another example where a very rare event, such as none of the data centers being accessible, happened, triggering another event.
- Pitfalls of automation: Facebook has been an early advocate of automating network configuration changes, effectively saying that software can do a better job of running the network than humans, who are more prone to errors. However, software can have bugs, such as this one.

Lessons learned

- Ready operations team: There can be a hidden single point of failure in complex systems. The best defense against such faults is to have the operations team ready for such an occurrence through regular training. Thinking clearly under high-stress situations becomes necessary to deal with such events.
- Simple system design: As systems get bigger, they become more complex, and they have emergent behaviors. To understand the overall behavior of the system, it might not be sufficient to understand only the behavior of its components. Cascading failures can arise. This is one reason to keep the system design as simple as possible for the current needs and evolve the design slowly. Unfortunately, there's no perfect solution to deal with this problem. We should accept the possibility of such failures, perform continuous monitoring, have the ability to solve issues when they arise, and learn from the failures to improve the system.
- Contingency plan: Some third-party services rely on Facebook for single sign-on. When the outage occurred, third-party services were up and running, but their clients were unable to use them because Facebook's login facility was also unavailable. This is another example of assuming that some service will always remain available and of a hidden single point of failure.
- Hosting DNS to independent third-party providers: There are a few services that are so robustly designed and perfected over time that their clients start assuming that the service is and will always be 100% available. The DNS is one such service, and it has been very carefully crafted. Designers often assume that it will never fail. Hosting DNS to independent third-party providers might be a way to guard against such problems. DNS allows multiple authoritative servers, and an organization might have many at different places. Although, we should note that DNS at Facebook's scale isn't simple, is tightly connected to their backbone infrastructure

and changes frequently. Delegating such a piece to an independent third party is expensive, and it might reveal internal service details. So, there can be a trade-off between business and robustness needs.

- Trade-offs: There can be some surprising trade-offs. An example here is the need for data security and the need for rapid manual repair. Because so many physical and digital safeguards were in place, manual intervention was slow. This is a catch-22 situation—lowering security needs can cause immense trouble, and slow repair for such events can also make it hard for the companies. The hope is that the need for such repairs is a very rare event.

- Surge in load: The failure of large players disrupts the entire Internet. Third-party public resolvers, such as Google and Cloudflare, saw a surge in the load due to unsuccessful DNS retries.

- Resuming the service: Restarting a large service isn't as simple as flipping a switch. When the load suddenly becomes almost zero, turning them up suddenly may lead to a many megawatt uptick in power usage. This might even cause issues for the electric grid. Complex systems usually have a steady state, and if they go out of that steady state, care must be taken to bring them back.

'''

Question

What can we do to safeguard against the kinds of faults experienced by Facebook?

Answer

Some possible solutions can look like the following:

Network verification has recently gained momentum and has shown promise in catching bugs early on. Such tools use an abstract model of the infrastructure.

There can be more than one layer of auditing. Second layers might use a simulator to make sure that after the configuration changes, critical network infrastructures remain available from multiple global vantage points.

Every effort should be taken to reduce the scope of a configuration change to avoid cascading effects.

Critical infrastructure might be programmed in such a way that if something bad happens, it could return to the last known good state. This is easier said than done owing to the sheer number of components.

- Network verification is the usage of network tools to investigate, troubleshoot, maintain, operate, and report to ensure that hardware, software, and network configuration will operate error-free.

'''

Spectacular Failures/Introduction to Distributed System Failures/README.md

Introduction to Distributed System Failures

Introduction

Once in a while, we encounter the failure of a service that's a household name, and individuals and businesses react to them. As system designers, we might wonder how carefully designed services that have been perfected over years by experienced teams can also fail.

This chapter discusses some of the major failures of well-known services and the measures that can be taken to mitigate such failures.

The following two factors contribute to failures:

- Diverse users: Most services have a vibrant user community, and as their needs evolve, so do the software products. If a software doesn't update in the way it provides new features and services, it will become stale over time. However, it might not have the features customers

want.

- Complex systems: Systems are complex, and they usually have emergent properties where the sum of system components is more complex than the individual pieces.

Types of failure in distributed systems

Most modern services are designed in such a way that failures are contained, and others might be localized for some users. Let's explore the types of failures we can observe in the distributed systems.

- System failure: A software or hardware failure is the most common cause of system failure. The contents in the primary memory are lost when a system fails. However, the data in secondary storage or replicas remains unaffected. The system reboots during such a breakdown.

- Method failure: Such failures suspend the working of distributed systems. It may also make the system execute the processes incorrectly or enter a deadlock state.

- Communication medium failure: Such failures occur when one component or service of a system can't reach the other internal or external entities.

- Secondary storage failure: In such failures, the secondary storage or replicas are down. The data in these nodes becomes inaccessible, so primary nodes need to generate another replica to ensure reliability and fault tolerance.

Vantage points

Something is always failing for large services. It's preferred to have a graceful degradation so that only a small portion of users are impacted for a short time. Therefore, we need globally dispersed vantage points to independently see the service status.

'''

Note: Many independent services, such as Downtdetector, exist for crowd-sourced problem reporting. It's interesting to note that if we visit such a service and see the status of our favorite applications, we'll see that there's always someone in the world facing some service problem.

'''

[Types of failure in a distributed system](./types.jpg)

Importance of independent service providers

One of the design goals of the original Internet was to provide resilience so that if one part fails, the rest can still operate.

With the emergence of a handful of service providers over the last decade, critics have raised concerns about such centralization and the potential impacts of failures. See [It's time to decentralize the internet, again: What was distributed is now centralized by Google, Facebook, etc.](#) by Bruce Davie. Most companies provide some kind of dashboard to enable users to see the service status. However, some failures might even knock out these dashboards. Affected companies then communicate to their customers via services like Twitter to announce the updates. Independent third-party services are valuable for failure detection and status dissemination.

'''

Note: The broader concept here is to use independent failure domains. A failure domain is a concept where anything failing inside a domain or network shouldn't affect other components or services in other domains. At times, we say that two domains are independent if they're outside the blast radius of each other.

'''

[Importance of independent service](./independence.jpg)

In the following lessons, we'll discuss the failures of well-known services by giant companies, the causes of failures, and what mitigation techniques can be used to avoid these failures.

Although failures are a great way to learn what went wrong and what the original designers could do to avoid such failures, weâ d like to prevent them from occurring at all.

System Design Web Crawler/Design Improvements of a Web Crawler/README.md

Design Improvements of a Web Crawler

Introduction

This lesson gives us a detailed rundown of the improvements that are required to enhance the functionality, performance, and security of our web crawler design. We have divided this lesson into two sections:

1. Functionality and performance enhancement design improvementsâ extensibility and multi-worker architecture.
2. Security-enhancement design improvementsâ crawler traps.

Letâ s dive into these sections.

Design improvements

Our current design is simplistic and has some inherent shortcomings and challenges. Letâ s highlight them one by one and make some adjustments to our design along the way.

- Shortcoming: Currently, our design supports the HTTP protocol and only extracts textual content. This leads to the question of how we can extend our crawler to facilitate multiple communication protocols and extract various types of files.

Adjustment: Since we have two separate components for serving communication handling and extracting, HTML Fetcher and Extractor, letâ s discuss their modifications one by one.

1. HTML Fetcher: We have only discussed the HTTP module in this component so far because of the widely-used HTTP URLs scheme. We can easily extend our design to incorporate other communication protocols like File Transfer Protocol (FTP). The workflow will then have an intermediary step where the crawler invokes the concerned communication module based on the URLâ s scheme. The subsequent steps will remain the same.

2. Extractor: Currently, we only extract the textual content from the downloaded document placed in the Document Input Stream (DIS). This document contains other file types as well, for example, images and videos. If we wish to extract other content from the stored document, we need to add new modules with functionalities to process those media types. Since we use a blob store for content storage, storing the newly-extracted content comprising text, images, and videos wonâ t be a problem.

[The extensibility of the HTML fetcher and extractor](./extensibility.jpg)

- Shortcoming: The current design doesnâ t explain how the multi-worker concept integrates in this system.

- Adjustment: Every worker needs a URL to work on from the priority queue. Different websites require different times for workers to finish crawling, so each worker will dequeue a new URL upon its availability.

There are several ways to achieve this multi-worker architecture for our system. Some of them are as follows:

1. Domain level log: The crawler assigns one whole domain to a worker. All the URLs branching out of the initial URL are the responsibility of the same crawler. We ensure this by caching the hash of the hostname against the worker ID for guaranteed future assignment to the same worker. This also helps us avoid any redundant crawling of that domainâ s web pages by any worker.

This approach is best-suited for achieving reverse URL indexing, which involves traversing the web address directories in a reversed order. It ensures the storage efficiency of the URL storing process for later use and prevents extensive repetitive string matching processes for

duplication testing.

2. Range division: The range of URLs given to each crawler; the crawler distributes a range of URLs from the priority queue among workers to avoid clashes. Like the previous approach, the crawler must hash the range associated with each worker.

3. Per URL crawling: Queue all the URLs; a worker takes a URL and enqueues the subsequently found URLs into the priority queue. Those new URLs are readily available to crawl through for other workers. To avoid enqueueing multiple similar links that direct to the same web page, we calculate the checksum of the canonicalized URL.

Crawler traps

A crawler trap is a URL or a set of URLs that cause indefinite crawler resource exhaustion. This section dedicates itself to the classification, identification, and prevention of crawler traps.

Classification

There can be many classification schemes for crawler traps, but let's classify them based on the URL scheme.

Mostly, web crawler traps are the result of poor website structuring, such as:

- URLs with query parameters: These query parameters can hold an immense amount of values, all while generating a large number of useless web pages for a single domain:

HTTP://www.abc.com?query.

- URLs with internal links: These links redirect in the same domain and can create an infinite cycle of redirection between the web pages of a single domain, making the crawler crawl the same content over and over again.

- URLs with infinite calendar pages: These have never-ending combinations of web pages based on the varying date values, and can create a large number of pointless web pages for a single domain.

- URLs for the dynamic content generation: These are query-based and can generate a massive number of web pages based on the dynamic content resulting from these queries. Such URLs might become a never-ending crawl on a single domain.

- URLs with repeated/cyclic directories: They form an indefinite loop of redirections. For example, HTTP://www.abc.com/first/second/first/second/....

[Classification of web crawler traps](./classification.jpg)

Mostly, the crawler traps are unintended because of the poor structuring of the website. Interestingly, crawler traps can also be placed intentionally to dynamically generate an endless series of web pages with the pure purpose of exhausting a crawler's bandwidth. These traps severely impact the throughput of a crawler and hamper its performance.

'''

Throughput: useful content crawled in a time period

'''

These crawler traps might be detrimental to the crawler, but they also seriously affect the website's SEO ranking.

Identification

It's essential to categorize the crawling traps to correctly identify them and think of design adjustments accordingly.

There are mainly two layers to the process of identifying a crawler trap:

1. Analyzing the URL scheme: The URLs with a poor URL structure, for example, those with cyclic directories: HTTP://www.abc.com/first/second/first/second/... will create crawler traps. Hence filtering such URLs beforehand will rescue our crawler resources.

2. Analyzing the total number of web pages against a domain: An impossibly large number of web

pages against a domain in the URL frontier is a strong indicator of a crawler trap. So, limiting the crawl at such a domain will be of utmost importance.

[The identification process of web crawler traps](./id.jpg)

Solution

Our design lacks details on the responsible crawling mechanism to avoid crawler traps for the identifications described above.

Crawling is a resource and time-consuming task, and it's of extreme importance to efficiently avoid crawler traps in order to achieve timely and helpful crawling. Once the crawler starts communicating with the server for downloading the web page content, there are multiple factors that it needs to take into consideration, mainly at the HTML-fetcher level. Let's go over the factors individually.

1. The crawler must implement an application-layer logic to counter the crawler traps. This logic might be based on the observed number of web pages exceeding a specified threshold.

The crawler must be intelligent enough to limit its crawling at a specific domain after a finite amount of time or web page visits. A crawler should smartly exit a web page and store that URL as a no-go area for future traversals to ensure performance effectiveness.

2. When initiating communication with the web server, the crawler needs to fetch a file named robots.txt. This file contains the dos and don'ts for a crawler listed by the web masters. Adhering to this document is an integral part of the crawling process. It also allows the crawler to access certain domain-specified web pages prioritized for crawling, without limiting its access to specific web pages.

Another essential component of this document is the revisit frequency instructions for the crawler. A popular website might demand a frequent revisit, contrary to a website that rarely updates its content. This standard of websites communicating with the web crawlers is called the Robots Exclusion Protocol. This protocol prevents the crawlers from unnecessarily spending crawling resources on uncrawlable web pages.

[The robots exclusion protocol](./robots.txt.jpg)

'''

The robots.txt file doesn't protect crawlers from malicious or intended crawler traps. The other explained mechanisms must handle those traps.

'''

3. Since each domain has a limited incoming and outgoing bandwidth allocation, the crawler needs to be polite enough to limit its crawling at a specific domain. Instead of having a static crawl speed for every domain, a better approach is to adjust the crawl speed based on a domain's Time to First Byte (TTFB) value. The higher the TTFB value, the slower the server. so, crawling that domain too fast might lead to more time-out requests and incomplete crawling.

These modifications in the design will ensure a crawler capable of avoiding crawler traps and hence optimizing resources' usage.

System Design Web Crawler/Design of a Web Crawler/README.md

Design of a Web Crawler

Design

This lesson describes the building blocks and the additional components involved in the design and workflow of the web crawling process with respect to its requirements.

Components

Here are the details of the building blocks and the components needed for our design:

- Scheduler: This is one of the key building blocks that schedules URLs for crawling. It's composed of two units: a priority queue and a relational database.

1. A priority queue (URL frontier): The queue hosts URLs that are made ready for crawling based on the two properties associated with each entry: priority and updates frequency.

As a requirement, we need to assign variable priorities to URLs, depending on their content. This attribute defines the precedence of a URL while in the URL frontier.

For recrawling purposes, we need to define the recrawl frequency for each URL. This attribute ensures a defined number of placements in the URL frontier for each URL.

2. Relational database: It stores all the URLs along with the two associated parameters mentioned above. The database gets populated by new requests from the following two input streams:

- The user's added URLs, which include seed and runtime added URLs.
- The crawler's extracted URLs.

Question 1

Can we estimate the size of the priority queue? What are the pros and cons of a centralized and distributed priority queue.

Answer

Assumption: Let's assume that at any point, we have roughly one million URLs in the URL frontier to be crawled upon.

Let's calculate the size of the priority queue needed to store all of these URLs. Size of priority queue = 1 million URLs * 2048 Bytes = 2.048 GB

2.048 GB is a reasonable amount of space for a queue, indicating that we might not need to implement a distributed mechanism for the URL frontier.

[PriorityQueue](./pq1.jpg)

However, a centralized queue has limited read/write bandwidth and is a single point of failure. Therefore, having a sub-queue for each worker will be the best approach.

[PriorityQueue](./pq2.jpg)

If we use our distributed queue, all workers can get data out of the same queue if they want. But having independent queues can further optimize the crawling process, especially in the case of high priority and more frequent crawls like those news websites that need more than regular workers to do the frequent crawling. This approach will inherently facilitate the case of increasing queue size.

Having a single queue is beneficial for the deduplication of redundant links and better for the overall crawler resources. We'll handle the recrawling priority and frequency another way, as explained in the next sections, for which we need the distribution mechanism.

Question 2

How will we distribute the URL frontier among different workers and what purpose will this serve?

Answer

As was defined earlier, the URL frontier is a priority queue used in the scheduler and it holds the URLs that need to be crawled through. When we talk about its distribution, we mean taking the hash value of the hostname's URLs and mapping them to specific workers. This way, each worker will have its own sub-queue.

This fulfills the following two requirements of our system:

The workers don't individually connect to more than one host web servers at a time.

The workers don't overburden the host web servers with concurrent requests because we use FIFO sub-queues.

- DNS resolver: The web crawler needs a DNS resolver to map hostnames to IP addresses for HTML content fetching. Since DNS lookup is a time-consuming process, a better approach is to create a customized DNS resolver and cache frequently-used IP addresses within their time-to-live because they're bound to change after their time-to-live.

- HTML fetcher: The HTML fetcher initiates communication with the server that's hosting the URL(s). It downloads the file content based on the underlying communication protocol. We focus mainly on the HTTP protocol for textual content, but the HTML fetcher is easily extendable to other communication protocols, as is mentioned in the section on the non-functional requirements of the web crawler.

'''

Question

How does the crawler handle URLs with variable priorities?

Answer

The crawler has to be vigilant enough at each stage to differentiate between various priority levels of URLs.

Let's see how the crawler design handles such cases stage by stage:

1. Since we implement our URL frontier as a priority queue of a scheduler, it automatically handles the placement based on parameter values. We have chosen the fault tolerance and periodicity parameters as priority indicators for our URLs.

The assignment of these parameters depends on the nature of the web pages' content. If it is a news web page, crawling through it multiple times in a day is appropriate and required for our index to be up-to-date. Any ordinary web page with occasional updates happening might have a standard visit frequency of, let's say, two weeks.

2. Similarly, at the HTML-fetcher level where the crawler is communicating with the host server based on the robots.txt guidelines, it can communicate the concerned parameter's value for the fetched URLs back to the scheduler in the storing phase.

Instead of having indicators associated with the URLs, another solution can be to have separate queues for different priorities. We can then dequeue from these queues based on the priorities assigned to them. This approach just requires the URL placement in the respective queue and doesn't need scripts to schedule based on the extra parameters associated.

It all depends on the scale of our crawling application.

'''

- Service host: This component acts as the brain of the crawler and is composed of worker instances. For simplicity, we will refer to this whole component or a single worker as a crawler. There are three main tasks that this service host/crawler performs:

1. It handles the multi-worker architecture of the crawling operation. Based on the availability, each worker communicates with the URL frontier to dequeue the next available URL for crawling.
2. Each worker is responsible for acquiring the DNS resolutions of the incoming URLs from the DNS resolver.
3. Each worker acts as a gateway between the scheduler and the HTML fetcher by sending the necessary DNS resolution information to the HTML fetcher for communication initiation.

[Service host interaction with other components](./service_hosts.jpg)

- Extractor: Once the HTML fetcher gets the web page, the next step is to extract two things from the webpage: URLs and the content. The extractor sends the extracted URLs directly and the content with the document input stream (DIS) to the duplicate eliminator. DIS is a cache that is used to store the extracted document, so that other components can access and process it. Over here, we can use Redis as our cache choice because of its advanced data structure functionality.

Once it is verified that the duplicates are absent in the data stores, the extractor sends the URLs to the task scheduler that contains the URL frontier and stores the content in blob storage for indexing purposes.

- Duplicate eliminator: Since the web is all interconnected, the probability of two different URLs referring to the same web page or different URLs referring to various web pages having the same content is evident. The crawler needs a component to perform a dedup test to eliminate the risk of exploiting resources by storing and processing the same content twice. The duplicate eliminator calculates the checksum value of each extracted URL and compares it against the URL checksum data store. If found, it discards the extracted URL. Otherwise, it adds a new entry to the database with the calculated checksum value.

[Dedup testing in action](./dedup.jpg)

The duplicate eliminator repeats the same process with the extracted content and adds the new webpage's checksum value in the document checksum data store for future matchings.

Our proposed design for the duplicate eliminator can be made robust against these two issues:

1. By using URL redirection, the new URL can pass through the URL dedup test. But, the second stage of the document dedup wouldn't allow content duplication in the blob storage.
2. By changing just one Byte in a document, the checksum of the modified document is going to come out different than the original one.

- Blob store: Since a web crawler is the backbone of a search engine, storing and indexing the fetched content and relevant metadata is immensely important. The design needs to have a distributed storage, such as a blob store, because we need to store large volumes of unstructured data.

The following illustration shows the pictorial representation of the overall web crawler design:

[The overall web crawler design](./design.jpg)

Workflow

1. Assignment to a worker: The crawler (service host) initiates the process by loading a URL from the URL frontier's priority queue and assigns it to the available worker.

2. DNS resolution: The worker sends the incoming URL for DNS resolution. Before resolving the URL, the DNS resolver checks the cache and returns the requested IP address if it's found. Otherwise, it determines the IP address, sends it back to the worker instance of the crawler, and stores the result in the cache.

'''

The implementation of the URL frontier dequeuing the URLs from a FIFO priority queue and enqueueing all the extracted URLs back into the queue rather than crawling them one after another ensures that we are crawling the web in a breadth-first search (BFS) rather than a depth-first-search (DFS) manner.

'''

'''

Question

Can we use DFS instead of BFS?

Answer

We can use DFS when we want to utilize a website's persistent connection to traverse all the web pages on that specific domain. This saves time as it helps us avoid reconnecting with the same website repeatedly in case the session expires.

'''

3. Communication initiation by the HTML fetcher: The worker forwards the URL and the associated IP address to the HTML fetcher, which initiates the communication between the crawler and the host server.

4. Content extraction: Once the worker establishes the communication, it extracts the URLs and the HTML document from the web page and places the document in a cache for other components to process it.

5. Dedup testing: The worker sends the extracted URLs and the document for dedup testing to the duplicate eliminator. The duplicate eliminator calculates and compares the checksum of both the URL and the document with the checksum values that have already been stored.

The duplicate eliminator discards the incoming request in case of a match. If there's no match, it places the newly-calculated checksum values in the respective data stores and gives the go-ahead to the extractor to store the content.

6. Content storing: The extractor sends the newly-discovered URLs to the scheduler, which stores them in the database and sets the values for the priority and recrawl frequency variables.

The extractor also writes the required portions of the newly discovered document currently in the DIS into the database.

7. Recrawling: Once a cycle is complete, the crawler goes back to the first point and repeats the same process until the URL frontier queue is empty.

'''

Note: Because of multiple instances of each service and microservices architecture, our design can make use of client-side load balancing (see Client-side Load Balancer for Twitter).

'''

The following slideshow gives a detailed overview of the web crawler workflow:

[Workflow](./workflow)

The URLs stored in the scheduler's database have priority and periodicity assigned to them. Enqueuing new URLs into the URL frontier depends on these two factors.

'''

Question

How frequently does the crawler need to re-crawl?

Answer

Since crawling is not a one-time activity, the crawler needs to plan its default revisits with a suitable frequency. Let's define this frequency of revisits as two weeks. The crawler revisits all the standard priority URLs every two weeks. Once the crawler visits a URL, it re-appends the same URL into the URL frontier with a default next visit time that's equal to two weeks.

The default revisit-time is application-dependent rather than system-dependent, so we change this variable to cater to our needs. It's a compelling case of assigning the priority comparator to URLs. Our system can predict the content change on a website using predictive analysis of the previous content changes. This way we can confidently assign the priority and revisit-time to each URL. For example, our analysis might show frequent changes on news websites and our crawler might suggest revisiting them on a high priority after every five minutes.

'''

In the next lesson, we'll explore some shortcomings in our design and their potential workarounds.

System Design Web Crawler/Evaluation of Web Crawler's Design/README.md

Evaluation of Web Crawler's Design

Reviewing design requirements

Let's evaluate how our design meets the non-functional requirements of the proposed system.

Scalability

Our design states that scaling our system horizontally is vital. Therefore, the proposed design incorporates the following design choices to meet the scalability requirements:

- The system is scalable to handle the ever-increasing number of URLs. It includes all the required resources, including schedulers, web crawler workers, HTML fetchers, extractors, and blob stores, which are added/removed on demand.
- In the case of a distributed URL frontier, the system utilizes consistent hashing to distribute the hostnames among various crawling workers, where each worker is running on a server. With this, adding or removing a crawler server isn't a problem.

Extensibility and modularity

So far, our design is only focusing on a particular type of communication protocol: HTTP. But according to our non-functional requirements, our system's design should facilitate the inclusion of other network communication protocols like FTP.

To achieve this extensibility, we only need to add additional modules for the newly required communication protocols in the HTML fetcher. The respective modules will then be responsible for making and maintaining the required communications with the host servers.

Along the same lines, we expect our design to extend its functionality for other MIME types as well. The modular approach for different MIME schemes facilitates this requirement. The worker will call the associated MIME's processing module to extract the content from the document stored in the DIS.


```
'''
MIME: A multipurpose Internet mail extension, or MIME type, is an Internet standard that
describes the contents of Internet files based on their natures and formats.
'''
```

```
[The requirements met by our web crawler](./r.jpg)
```

Consistency

Our system consists of several crawling workers. Data consistency among crawled content is crucial. So, to avoid data inconsistency and crawl duplication, our system computes the checksums of URLs and documents and compares them with the existing checksums of the URLs and documents in the URL and document checksum data stores, respectively.

Apart from deduplication, to ensure the data consistency by fault-tolerance conditions, all the servers can checkpoint their states to a backup service, such as Amazon S3 or an offline disk, regularly.

Performance

Our web crawler's performance depends on the following factors:

- URLs crawled per second: We can improve this factor by adding new workers to the system.
- Utilizing blob storage for content storing: This ensures higher throughput for the massive amount of unstructured data. It also indicates a fast retrieval of the stored content, because a single blob can support up to 500 requests per second.
- Efficient implementation of the robots.txt file guideline: We can implement this performance factor by having an application-layer logic of setting the highest precedence of robots.txt guidelines while crawling.
- Self-throttling: We can have various application-level checks to ensure that our web crawler doesn't hamper the performance of the website host servers by exhausting their resources.

```
'''
```

Fulfilling Non-functional requirements

Requirement	Techniques
Scalability increase/decrease in load to achieve fault tolerance	- Addition/removal of different servers based on the - Consistent hashing to manage server's addition and removal - Regular backup of the servers in Amazon S3 backup service
Extensibility and Modularity HTML fetcher downloaded document in DIS	- Addition of a newer communication protocol module in the - Addition of new MIME schemes while processing the
Consistency Documents in the respective datastores	- Calculation and comparison of checksums of URLs and
Performance	- Increasing the number of workers performing the crawl - Blob stores for storing the content - High priority to robots.txt file guidelines while crawling - Self-throttle at a domain while crawling
Scheduling various priority URLs	- Pre-defined default recrawl frequency, or - Separate queues and their associated frequencies for

```
'''
```

Scheduling

As was established previously, we may need to recrawl URLs on various frequencies. These frequencies are determined by the application of the URL. We can determine the frequency of recrawl in two different ways:

1. We can assign a default or a specific recrawling frequency to each URL. This assignment depends on the application of the URL defining the priority. A default frequency is assigned to the standard-priority URLs and a higher recrawl frequency is given to the higher-priority URLs.

Based on each URL's associated recrawl frequency, we can decide to enqueue URLs in the priority queue from the scheduler's database. The priority defines the place of a URL in the queue.

2. The second method is to have separate queues for various priority URLs, use URLs from high-priority queues first, and subsequently move to the lower-priority URLs.

Conclusion

The web crawler system entails a multi-worker design that uses a microservices architecture. Besides achieving the basic crawling functionality, our design provides insights into the potential shortcomings and challenges associated with our design and further rectifies them with appropriate design modifications. The noteworthy features of our design are as follows:

1. Identification and design modification for crawler traps
2. Extensibility of HTML fetching and content extraction modules

System Design Web Crawler/Requirements of a Web Crawler's Design/README.md

Requirements of a Web Crawler's Design

Requirements

Let's highlight the functional and non-functional requirements of a web crawler.

Functional requirements

These are the functionalities a user must be able to perform:

- Crawling: The system should scour the WWW, spanning from a queue of seed URLs provided initially by the system administrator.

'''

Question 1

Where do we get these seed URLs from?

Answer

There are two possible ways to create or gather seed URLs:

We can manually create them

We can scan the IP addresses for the presence of web servers

These seed URLs must be of good quality.

'''

'''

Question 2

Why are good quality seed URLs important and what happens if the seed quality is not good?

Answer

When we model WWW as a graph where URLs link one node to another, our goal is to discover as much of it as possible during the crawl.

Using a low-quality seed might limit the discovery to just a fraction of the WWW graph.

'''

'''

Question 3

How do we select seed URLs for crawling?

Answer

There are multiple approaches to selecting seed URLs. Some of them are:

Location-based: We can have different seed URLs depending on the location of the crawler.

Category-based: Depending on the type of content we need to crawl, we can have various sets of seed URLs.

Popularity-based: This is the most popular approach. It combines both the aforementioned approaches. It groups the seed URLs based on hot topics in a specific area.

'''

- Storing: The system should be able to extract and store the content of a URL in a blob store. This makes that URL and its content processable by the search engines for indexing and ranking purposes.

- Scheduling: Since crawling is a process that's repeated, the system should have regular scheduling to update its blob store's records.

Non-functional requirements

- Scalability: The system should inherently be distributed and multithreaded, because it has to fetch hundreds of millions of web documents.

- Extensibility: Currently, our design supports HTTP(S) communication protocol and text files storage facilities. For augmented functionality, it should also be extensible for different network communication protocols, able to add multiple modules to process, and store various file formats.

- Consistency: Since our system involves multiple crawling workers, having data consistency among all of them is necessary.

- Performance: The system should be smart enough to limit its crawling to a domain, either by time spent or by the count of the visited URLs of that domain. This process is called self-throttling. The URLs crawled per second and the throughput of the content crawled should be optimal.

'''

Websites usually host a robot.txt file, which communicates domain-specified limitations to the crawler. The crawler should adhere to these limitations by all means.

'''

- Improved user interface & customized scheduling: Besides the default recrawling, which is a functional requirement, the system should also support the functionality to perform non-routine customized crawling on the system administrator's demands.

Resource estimation

We need to estimate various resource requirements for our design.

****Assumptions****

These are the assumptions we'll use when estimating our resource requirements:

- There are a total of 5 billion web pages.
- The text content per webpage is 2070 KB.
- The metadata for one web page is 500 Bytes.

'''

A study suggests that the average size of a webpage content is 2070KB (2.07MB) based on 892 processed websites.

'''

'''

The metadata consists of a webpage title and description of the web page showing its purpose.
'''

Storage estimation

The collective storage required to store the textual content of 5 billion web pages is:
Total storage per crawl=5 Billion^{*} (2070 KB+500B)=10.35PB

[The total storage required by the web crawler system](./storage.jpg)

Traversal time

Since the traversal time is just as important as the storage requirements, let's calculate the approximate time for one-time crawling. Assuming that the average HTTP traversal per webpage is 60 ms, the time to traverse all 5 billion pages will be:

'''

Webpage traversal is a function of the page size. Smaller web pages take less than 60 ms, and large pages take longer; however, the number 60 ms is the average traversal time.

'''

Total traversal time=5 Billion^{*} 60 ms=0.3 Billion seconds=9.5 years

It'll take approximately 9.5 years to traverse the whole Internet while using one instance of crawling, but we want to achieve our goal in one day. We can accomplish this by designing our system to support multi-worker architecture and divide the tasks among multiple workers running on different servers.

Number of servers estimation for multi-worker architecture

Let's calculate the number of servers required to finish crawling in one day. Assume that there is only one worker per server.

No. of days required by 1 server to complete the task=9.5 years^{*} 365 days^{*} 24 hours^{*} 60 minutes^{*} 60 seconds^{*} 3468 days

One server takes 3,468 days to complete the task.

How many servers would we need to complete this same task in one day?

We would need 3,468 servers to complete the same task in just one day.

[The number of servers required for the web crawler system](./servers.jpg)

'''

If there are a number of threads per server, we'll divide 3,468 by n. For example, if one server is capable of executing ten threads at a time, then the number of servers is reduced to 3468/10 $\hat{=}$ 347 servers

'''

Bandwidth estimation

Since we want to process 10.35PB of data per day the total bandwidth required would be:

10.35PB/86400sec^{*} 120GB/sec^{*} 960Gb/sec

960Gb/sec is the total required bandwidth. Now, assume that the task is distributed equally among 3468 servers to accomplish the task in one day. Thus, the per server bandwidth would be:

(960Gb/sec) / 3468 servers $\hat{=}$ 277Mb/sec per server

[The total bandwidth required for the web crawler system](./bandwidth.jpg)

Let's play around with the initial assumptions and see how the estimates change in the

following calculator:

'''

Estimates Calculator for the Web Crawler

Number of Webpages	5 Billion
Text Content per Webpage	2070 KB
Metadata per Webpage	500 Bytes
Total Storage	10.35PB
Total Traversal Time on One Server	9.5 Years
Servers Required to Perform Traversal in One Day	3468 Servers
Bandwidth Estimate	958.33 Gb/sec

'''

Building blocks we will use

Here is the list of the main building blocks weâ ll use in our design:

[Building blocks in high-level design](./bb.jpg)

- Scheduler is used to schedule crawling events on the URLs that are stored in its database.
- DNS is needed to get the IP address resolution of the web pages.
- Cache is utilized in storing fetched documents for quick access by all the processing modules.
- Blob storeâ s main application is to store the crawled content.
- The HTML fetcher establishes a network communication connection between the crawler and the web hosts.
- The service host manages the crawling operation among the workers.
- The extractor extracts the embedded URLs and the document from the web page.
- The duplicate eliminator performs dedup testing on the incoming URLs and the documents.

In the next lesson, weâ ll focus on the high-level and detailed design of a web crawler.

[The components in a high-level design](./components.jpg)

Besides these basic building blocks, our design includes some additional components as well:

System Design Web Crawler/System Design Web Crawler/README.md

System Design: Web Crawler

Introduction

A web crawler is an Internet bot that systematically scours the world wide web (WWW) for content, starting its operation from a pool of seed URLs. This process of acquiring content from the WWW is called crawling. It further saves the crawled content in the data stores. The process of efficiently saving data for subsequent use is called storing.

'''

To go or move swiftly about, over, or through in search of something.

'''

'''

Stored URLs that serve as a starting point for a crawler.

'''

This is the first step thatâ s performed by search engines; the stored data is used for indexing and ranking purposes. This specific design problem is limited to web crawlers and doesnâ t include explanations of the later stages of indexing and ranking in the search engines.

[An overview of the web crawler system](./overview.jpg)

Additional benefits

The additional utilities of a web crawler are as follows:

- Web pages testing: We use web crawlers to check the validity of the links and structures of web pages.
- Web page monitoring: We use web crawlers to monitor the content or structure updates on web pages.
- Site mirroring: Web crawlers are an effective way to mirror popular websites.
- Copyright infringement check: Web crawlers fetch content and check for copyright infringement issues.

In this chapter, we'll design a web crawler and evaluate how it fulfills the functional and non-functional requirements.

The output of the crawling process is the data that's the input for the subsequent processing phases: data cleaning, indexing, page relevance using algorithms like page ranks, and analytics. To learn about some of these subsequent stages, refer to our chapter on distributed search.

'''

Mirroring is like making a dynamic carbon copy of a website. Mirroring refers to network services available by any protocol, such as HTTP or FTP. The URLs of these sites differ from the original sites, but the content is similar or almost identical.

'''

How will we design a Web crawler?

This chapter consists of four lessons that encompass the overall design of the web crawler system:

1. Requirements: This lesson enlists the functional and non-functional requirements of the system and estimates calculations for various system parameters.
 2. Design: This lesson analyzes a bottom-up approach for a web-crawling service. We get a detailed overview of all the individual components leading to a combined operational mechanism to meet the requirements.
 3. Improvements: This lesson provides all the design improvements required to counter shortcomings, especially the crawler traps. These crawler traps include links with query parameters, internal links redirection, links holding infinite calendar pages, links for dynamic content generation, and links containing cyclic directories.
 4. Evaluation: This lesson provides an in-depth evaluation of our design choices to check if they meet all the standards and requirements we expect from our design.
- Let's begin with defining the requirements of a web crawler.